

IHARQ BenchGuard

Master Architecture Specification T29

Layout-Corrected Final Successor R1

Semantic authority unchanged; Layer 9 E0–E5 table presentation corrected

Semantic architecture identity	T29
Layout-successor identity	T29-LAYOUT-CORRECTED-R1
Preserved predecessor PDF	01_Master_Architecture_Specification_FINAL_T29.pdf
Preserved predecessor SHA-256	44ed84166b3f4aae4b9524541d035fc9025b0bf2b63a12e09d05047a7ec0214e
Change class	Formatting-only successor; the Layer 9 E0–E5 maturity ladder is placed in a landscape longtable and one long artifact path is made line-breakable.
Semantic change	None. No layer, module, submodule, E-mode, record, artifact, interface, phase, ablation, topology, ownership, method, metric, claim, or limitation is changed.
Compatibility	Fully compatible with Registry R43, Plan R40, Protocol R41, Playbook R40, and the integrated Layers 1–9 R3 authorities.
Status	FINAL LAYOUT-CORRECTED / SOURCE-PRESERVING SEMANTICS / NOT FROZEN

IHARQ BenchGuard

T29 Layer 6 R12 Closed-Loop Bidirectional Authority Synchronization

Compatibility set: L6-NB-R12-CLOSED-LOOP-UPSTREAM-SYNC-R3-20260727

Current architecture revision	T29 Layer 6 R12 closed-loop bidirectional authority synchronization
Preserved predecessor	01_Architecture_T28_LAYER6_R11_COMPLETE_BIDIRECTIONAL_SYNC.tex
Predecessor SHA-256	8d70a2f637c90d1ba897d4ffda8a13d5568a13888385af3588d2a09fde1ce9da
Synchronized Layer 6 successor	08_Layer6_Nuts_and_Bolts_R12_CLOSED_LOOP_UPSTREAM_ACKNOWLEDGED.md
Layer 6 identity	L6-NB-R12-CLOSED-LOOP-UPSTREAM-ACKNOWLEDGED
Change class	Additive closed-loop authority receipt, revision graph and correction-cycle closure; no topology, method, phase, ablation or ownership change
Two-way rule	Architecture constrains Layer 6 topology and ownership; Layer 6 R12 acknowledges the exact T29/R40/R39/R39/R39/R10/R4 successor set. Accepted owner changes create immutable successors, invalidation roots, regeneration and consumer acknowledgment. Only Architecture may change topology.

IHARQ BenchGuard

T20 Layer 3 Full Bidirectional Synchronization Cover

Compatibility set: L3-NB-R16-BIDIRECTIONAL-SYNC-R2-20260721

Current architecture revision	T20 Layer 3 full bidirectional synchronization
Synchronization master	00_IHARQ_Layer3_Post_Finalization_Bidirectional_Synchronization_Master_R2.md
Preserved predecessor	01_IHARQ_Master_Architecture_T19_LAYER3_NUTS_AND_BOLTS_BIDIRECTIONAL_SYNC_R1.tex
Predecessor SHA-256	d4e8bb10684b8ea48d86fbaf0d8a46f92e03f385e999d8e9a111b275f2dd3312
Change class	Additive synchronization and audit hardening; no topology, method, phase, ablation or ownership change
Effective-authority rule	T20 cover/addendum controls synchronization within Architecture authority. T19/T18 content remains controlling for unaffected architecture. Nested predecessor current/final labels are historical inside T20.
Two-way rule	Architecture constrains Layer 3 topology/ownership; Layer 3 returns realization and impact evidence. Only Architecture can change topology or ownership.

IHARQ BenchGuard

Stretch C Maximum Architecture Document

Functional, Modular, Policy-Learning, Embodiment, and Dashboard Architecture

Document type	Architecture specification and implementation-roadmap document
System identity	Layered reliability architecture for low-calibration neural intent readiness
Scope level	Stretch C maximum version
Stretch C additions	MyoSuite/OpenSim demo, learning-to-defer, contextual bandit, RL policy, d
Primary experimental anchor	Public EEG motor-imagery datasets plus simulated closed-loop readiness
Embodiment scope	Simulated assistive-action proof of concept only, not clinical validation
Architecture style	Functional graph with governance, reliability, policy, simulation, embodimen

Prepared as a from-scratch architecture document for the most extended
implementation path.

Contents

1	Purpose and Scope of This Stretch C Architecture Document	22
2	Stretch C Definition and Non-Negotiable Scope Boundaries	22
2.1	Definition of Stretch C	22
2.2	Scientific claim boundaries	23
2.3	Why Stretch C is architecturally valuable	24
3	Architectural Style: Functional Graph, Not a Simple Stack	24
3.1	Main architectural planes	24
3.2	System-level flow	25
3.3	Why feedback exists	25
4	Revised Layer Numbering for the Stretch C Architecture	26
4.1	Mapping from the original layer design to the revised Stretch C design	27
5	System-Wide Interface Contracts	28
5.1	Canonical records	28
5.2	Record design principles	47
6	Layer 0: Claim-Safety and Scope Governance	48
6.1	Problem solved by Layer 0	48
6.2	Inputs	48
6.3	Submodules	49
6.4	Outputs	51
6.5	Outgoing links	51
7	Layer 1: Public-Data and Protocol Anchor	52
7.1	Problem solved by Layer 1	52
7.2	Layer 1 mission, thesis role, and contribution boundary	53
7.3	Scope, non-scope, and boundary rule	53
7.4	Design principles	55
7.5	Input contract	57
7.5.1	Primary and metadata inputs	57
7.5.2	Configuration and protocol inputs	58
7.5.3	Constraint inputs from Layer 0	58
7.6	Output contract and downstream consumers	59

7.7	Internal submodule architecture	60
7.8	Submodule dossiers	61
7.8.1	Dataset registry manager	61
7.8.2	Dataset loader	62
7.8.3	Metadata normalizer	63
7.8.4	Task and label ontology mapper	63
7.8.5	Preprocessing registry	64
7.8.6	Artifact and channel-quality annotator	64
7.8.7	Split and protocol manager	65
7.8.8	Trial/window generator	65
7.8.9	Data validation and leakage auditor	66
7.8.10	Dataset-card and protocol-card generator	67
7.8.11	Provenance and manifest builder	67
7.9	Canonical Layer 1 records and schemas	68
7.10	Public-dataset registry and dataset-status framework	69
7.11	Split, low-calibration, and leakage-control protocol	69
7.12	Dependency logic	70
7.13	Feedforward and loopback behavior	71
7.14	Layer 1 evaluation design and exit conditions	72
7.15	Reproducibility, repository structure, and implementation artifacts	73
7.16	Failure modes and defensive design	74
7.17	Layer 1 readiness levels	75
7.18	Outputs and outgoing links	76
8	Layer 2: Decoder and Baseline Measurement Spine	76
8.1	Problem solved by Layer 2	76
8.2	Layer 2 thesis role and contribution boundary	77
8.3	Scope, non-scope, and boundary rule	77
8.4	Design principles	79
8.5	Input contract	79
8.6	Output contract	81
8.7	Canonical <code>PredictionRecord</code> schema	82
8.8	Canonical <code>ModelRegistryRecord</code> schema and registry contract	83
8.9	Model-family tiers	84
8.10	Internal module structure	85

8.10.1	Submodule 1: Baseline trainer	86
8.10.2	Submodule 2: Prediction logger	87
8.10.3	Submodule 3: Low-calibration curve builder	88
8.10.4	Submodule 4: Subject difficulty profiler	89
8.10.5	Submodule 5: Model-family registry	90
8.10.6	Submodule 6: Ensemble comparison builder	91
8.10.7	Submodule 7: Compact SSL adapter	92
8.10.8	Submodule 8: Downstream readiness validator	93
8.11	Dependency logic	94
8.12	Feedforward and loopback behavior	95
8.13	Evaluation design	97
8.14	Reproducibility artifacts and repository placement	98
8.15	Implementation phases and readiness levels	99
8.16	Outputs	100
8.17	Outgoing links	100
8.18	Definition of done	100
9	Layer 3: Calibration, Uncertainty, and Selective Prediction	100
9.1	Problem solved by Layer 3	100
9.2	Architectural role and boundaries	101
9.3	Input contract and upstream dependencies	102
9.4	Split discipline and leakage guard	104
9.5	Output contract and downstream consumers	105
9.6	Submodules	106
9.6.1	Submodule 1: Probability calibrator	107
9.6.2	Submodule 2: Reliability audit	108
9.6.3	Submodule 3: Uncertainty extractor	110
9.6.4	Submodule 4: Selective-prediction baseline	111
9.6.5	Submodule 5: Threshold registry	112
9.7	Dependency logic	113
9.8	Feedforward and loopback behavior	114
9.9	Evaluation design	115
9.10	Reproducibility artifacts and repository placement	116
9.11	Dashboard, cards, and thesis outputs	117
9.12	Outputs	118

9.13	Outgoing links	118
9.14	Definition of done	118
10	Layer 4: IHARQ Evidence Verification	119
10.1	Problem solved by Layer 4	119
10.2	Engineering responsibility and boundary discipline	120
10.3	Position in the architecture	121
10.4	Input contract	122
10.5	Input validation and degraded modes	124
10.6	Output contract	125
10.7	Action vocabulary	128
10.8	Reason-code and explanation taxonomy	129
10.9	Submodules	130
10.10	Decision logic	140
10.11	Evidence request and combination behavior	141
10.12	Fallback and unsafe-state handling	142
10.13	RegimeRisk feedback integration	143
10.14	Feedforward and loopback behavior	144
10.15	Evaluation and ablation design	145
10.16	Metrics	147
10.17	Stress tests and failure modes	149
10.18	Reproducibility and implementation artifacts	150
10.19	Dashboard, cards, and thesis outputs	151
10.20	Outputs	151
10.21	Outgoing links	152
10.22	Definition of done	152
11	Layer 5: RegimeRisk Temporal Trust Monitoring	153
11.1	Problem solved by Layer 5	153
11.2	Layer 5 thesis role and contribution boundary	153
11.3	Engineering responsibility, scope, and boundaries	154
11.4	Layer 5 identity and design principles	155
11.5	Conceptual model of temporal trust	156
11.6	Input contract	157
11.6.1	Primary and strongly recommended inputs	157

11.6.2	Temporal feature inputs	158
11.6.3	Optional Stretch C and feedback inputs	158
11.6.4	Input classification and validation	159
11.7	Output contract	160
11.8	Canonical RegimeStateRecord	161
11.9	Canonical TrustStateRecord	163
11.10	Dependency logic	163
11.11	Submodule structure and responsibilities	164
11.11.1	Submodule 5.1: Temporal feature builder	165
11.11.2	Submodule 5.2: Volatility-like monitor	166
11.11.3	Submodule 5.3: Drawdown-like monitor	167
11.11.4	Submodule 5.4: Regime classifier	168
11.11.5	Submodule 5.5: Failure-risk predictor	170
11.11.6	Submodule 5.6: Trust allocator	171
11.11.7	Submodule 5.7: Stop-loss module	172
11.11.8	Submodule 5.8: Feedback adapter	173
11.12	Feedforward and loopback contracts	174
11.12.1	Feedback to Layer 4: IHARQ Evidence Verification	174
11.12.2	Feedback to Layer 6: Adaptive Readiness Policy	174
11.12.3	Feedback to Layer 7: Simulated Closed-Loop Readiness	175
11.12.4	Feedback to Layer 9: MyoSuite/OpenSim Embodiment Demo	175
11.12.5	Feedback to Layer 10: Dashboard, Cards, and Reproducibility	175
11.13	Regime vocabulary, transitions, and threshold governance	176
11.14	Evaluation design	176
11.14.1	Evaluation questions	177
11.14.2	Metrics	177
11.14.3	Baselines and ablations	177
11.14.4	Matched-comparison discipline	178
11.14.5	StressForge-Lite interaction	178
11.14.6	Failure interpretation	178
11.15	Reproducibility and artifact structure	179
11.16	Implementation phases and readiness levels	180
11.17	Claim-safety, limitations, and defensive design	181
11.18	Layer 5 dossier template	183

11.19	Outputs	183
11.20	Feedback to other layers	184
11.21	Definition of done	184
12	Layer 6: Adaptive Readiness Policy Layer	184
12.1	Problem solved by Layer 6	184
12.2	Merged policy-learning ownership and source-section status	185
12.3	Engineering responsibility, scope, and boundaries	186
12.4	Layer 6 identity and design principles	187
12.5	Architectural position and policy plane ownership	188
12.6	Input contract	189
12.6.1	Canonical <code>ReadinessContextRecord</code> schema	191
12.7	Decision-time legality, leakage-control, and record partition	193
12.8	Output contract	194
12.8.1	Canonical <code>PolicyActionRecord</code> schema	196
12.8.2	Canonical <code>DeferralRecord</code> schema	197
12.8.3	Canonical <code>PolicyUpdateTrace</code> fields	197
12.9	Dependency logic	198
12.10	Policy action space	199
12.11	Policy context	201
12.12	Label derivation and target construction	202
12.13	Cost, reward, and utility framework	203
12.14	Submodule structure and responsibilities	204
12.15	Submodule 6.0: Rule-based IHARQ-lite reference policy	205
12.16	Submodule 6.1: Evidence-quality estimator	206
12.17	Submodule 6.2: Supervised Adaptive-IHARQ	207
12.18	Submodule 6.3: Learning-to-defer	208
12.19	Submodule 6.4: Contextual bandit policy	209
12.20	Submodule 6.5: Reinforcement-learning readiness policy	210
12.21	Feedforward and loopback contracts	211
12.22	Simulation, StressForge, and embodiment integration	212
12.23	Evaluation and ablation design	213
12.24	Metrics	214
12.25	Reproducibility and implementation artifacts	215
12.26	Dashboard, cards, and thesis outputs	216

12.27	Implementation phases and readiness levels	217
12.28	Failure modes and defensive design	218
12.29	Layer 6 outputs	220
12.30	Definition of done	220
13	Layer 7: Simulated Closed-Loop Readiness Environment	221
13.1	Problem solved by Layer 7	221
13.2	Engineering responsibility, scope, and boundaries	222
13.3	Layer identity and functional position	223
13.4	Input contracts	224
13.5	Dependency logic	227
13.6	Environment state	229
13.7	Episode and session design	230
13.8	Action space and action validity	230
13.9	Transition logic	232
13.10	Reward, cost, and consequence model	233
13.11	Evidence-request and temporal-decision logic	234
13.12	RegimeRisk coupling	234
13.13	StressForge coupling	235
13.14	Policy-evaluation protocol	236
13.15	Frozen-policy evaluation versus simulation-authorized updates	236
13.16	Embodiment gateway	237
13.17	Submodule architecture	238
13.18	Output contracts and record schemas	241
13.19	Metrics and evaluation design	244
13.20	Ablation framework	245
13.21	Validation, sanity checks, and reproducibility	246
13.22	Dashboard and thesis artifacts	247
13.23	Failure modes and defensive design	248
13.24	Implementation roadmap and readiness levels	249
13.25	Layer 7 thesis evidence produced	250
14	Layer 8: StressForge-Lite Stress Generator	250
14.1	Problem solved by Layer 8	250
14.2	Engineering responsibility, scope, and boundaries	251

14.3 Layer 8 identity and module status table	252
14.4 Input contracts	253
14.5 Output contracts	254
14.6 StressProfileRecord schema	256
14.7 Stress ontology and stress profiles	257
14.8 Intensity ladder and schedule framework	258
14.9 Stress injection architecture	259
14.10 Module dossiers	261
14.10.1 Submodule 8.1: Stress registry	261
14.10.2 Submodule 8.2: Scenario builder	261
14.10.3 Submodule 8.3: Schedule engine	262
14.10.4 Submodule 8.4: Injection adapter	262
14.10.5 Submodule 8.5: Consistency guard	263
14.10.6 Submodule 8.6: Run orchestrator	264
14.10.7 Submodule 8.7: Metric collector	264
14.10.8 Submodule 8.8: Stress-card generator and failure-taxonomy writer	265
14.11 Per-stressor specialization	265
14.11.1 Delay stress	265
14.11.2 Dropout stress	266
14.11.3 Noisy-intent stress	267
14.11.4 Weakness stress	267
14.11.5 Drift stress	267
14.11.6 Fatigue-like degradation stress	268
14.11.7 Combined stress	268
14.12 Combined-stress design protocol	269
14.13 Dependency logic	269
14.14 Cross-layer contracts	270
14.15 Feedforward and loopback behavior	271
14.16 Layer 8 metrics and evaluation architecture	273
14.17 Ablation and criticism-defense matrix	275
14.18 Dashboard, cards, and reporting requirements	276
14.19 Reproducibility requirements	277
14.20 Readiness levels and implementation roadmap	278
14.21 Failure modes and defensive design	279

14.22	Layer 8 thesis evidence	280
15	Layer 9: MyoSuite/OpenSim Embodiment Demo	281
15.1	Layer 9 thesis role and problem statement	281
15.2	Layer 9 research identity	282
15.3	Layer 9 engineering identity, boundaries, and scope	283
15.4	Detailed embodiment scope, non-scope, and allowed claims	284
15.5	Stakeholders and concerns	285
15.6	Layer 9 position in the full architecture graph	285
15.7	Embodiment modes and maturity ladder	287
15.8	Internal module decomposition	290
15.9	Module cards and responsibilities	291
15.9.1	L9.1 Embodiment session controller	292
15.9.2	L9.2 Record intake and validator	292
15.9.3	L9.3 Readiness context assembler	293
15.9.4	L9.4 Proxy command mapper	294
15.9.5	L9.5 Readiness safety gate	295
15.9.6	L9.6 Simulator adapter interface	296
15.9.7	L9.7 Static replay adapter	296
15.9.8	L9.8 MyoSuite adapter	297
15.9.9	L9.9 OpenSim replay adapter	298
15.9.10	L9.10 Stress embodiment injector	299
15.9.11	L9.11 Outcome and reward bridge	299
15.9.12	L9.12 Transition and safety logger	300
15.9.13	L9.13 Dashboard/card/export packager	301
15.10	Formal input contract	302
15.11	Input validation rules	305
15.12	Formal output contract	306
15.13	EmbodimentStateRecord schema	308
15.14	EmbodimentOutcomeRecord schema	309
15.15	Simulator adapter interface	310
15.16	Proxy command mapping contract	311
15.17	Readiness safety gate and safety state machine	313
15.18	Feedforward and feedback-loop behavior	314
15.19	MyoSuite branch design	315

15.20	OpenSim branch design	316
15.21	StressForge-Lite integration	316
15.22	Policy-learning and closed-loop integration	317
15.23	Reward and outcome logic	317
15.24	Layer 9 metrics	319
15.25	Embodiment data flow and operational flow	319
15.26	Evaluation and A13 ablation linkage	320
15.27	Dashboard and reporting linkage	321
15.28	Reproducibility and artifact management	322
15.29	Failure modes and defensive design	323
15.30	Implementation roadmap and definition of done	324
15.31	Traceability and claim-safety checklist	325
15.32	Layer 9 readiness levels and definition of done	326
15.33	Layer 9 thesis evidence	327
16	Layer 10: Dashboard, Cards, and Reproducibility Layer	327
16.1	Layer 10 identity, thesis role, and bottleneck	327
16.2	Stakeholders and thesis inspection questions	329
16.3	Engineering responsibility, scope, and implementation boundary	330
16.3.1	In-scope responsibilities	330
16.3.2	Out-of-scope responsibilities	332
16.3.3	Core, supporting, optional, stretch, and future components	332
16.4	Input contracts	333
16.4.1	Canonical records consumed by Layer 10	337
16.4.2	Required metadata and validation rules	344
16.5	Output contracts	346
16.5.1	DashboardMetricRecord schema	348
16.5.2	DashboardViewRecord schema	349
16.5.3	Artifact identity contract	350
16.6	Dependency logic	351
16.7	Internal module architecture	352
16.7.1	Module 10.1: Record Loader and Canonical Artifact Reader	354
16.7.2	Module 10.2: Schema Validator and Completeness Auditor	355
16.7.3	Module 10.3: Record Indexer and Lineage Resolver	356
16.7.4	Module 10.4: Metric Summarizer and DashboardMetricRecord Builder	357

16.7.5	Module 10.5: View Model Builder, Renderer, and Scientific Observability	
	Pages	358
16.7.6	Module 10.6: Card Generator and Card Completeness Checker	359
16.7.7	Module 10.7: Export Manager for Thesis Figures and Tables	360
16.7.8	Module 10.8: Reproducibility Manifest Builder	361
16.7.9	Module 10.9: Warning Engine and Claim-Safety Gate	362
16.7.10	Module 10.10: Audit Logger and Artifact Registry	363
16.8	Dashboard page specifications	363
16.8.1	Project overview and claim-safety page	364
16.8.2	Dataset and split page	364
16.8.3	Decoder baseline page	364
16.8.4	Calibration, uncertainty, and selective-prediction page	364
16.8.5	IHARQ decision-flow page	365
16.8.6	RegimeRisk timeline page	365
16.8.7	Adaptive policy page	366
16.8.8	Learning-to-defer page	366
16.8.9	Contextual bandit and RL page	366
16.8.10	StressForge page	367
16.8.11	Layer 9 MyoSuite/OpenSim/static-replay embodiment page	367
16.8.12	Ablation and comparison page	368
16.8.13	Failure taxonomy page	368
16.8.14	Cards and export page	368
16.9	Cards architecture	369
16.9.1	CardRecord schema and universal card contract	370
16.10	Traceability framework	370
16.10.1	Record-to-view traceability	371
16.10.2	View-to-card and artifact-to-thesis traceability	373
16.11	Feedforward and loopback behavior	373
16.12	Evaluation design for Layer 10	375
16.12.1	Baselines	375
16.12.2	Ablations	375
16.12.3	Metrics and success criteria	376
16.13	System-level ablation ladder exposed by Layer 10	377
16.14	Reproducibility architecture	378
16.14.1	Required configuration files	378

16.14.2 Expected output files	379
16.14.3 ReproducibilityManifest schema and fields	380
16.14.4 Repository organization	380
16.15 Testing and quality assurance	382
16.16 Layers 4–7 evaluation-lineage dashboard addendum	383
16.17 Layers 8–10 evaluation-lineage dashboard addendum	384
16.18 Implementation roadmap and maturity model	386
16.19 Failure modes, defensive design, and validity threats	387
16.20 Expected thesis evidence from Layer 10	388
17 Cross-Layer Functional Module Architecture	389
17.1 Purpose of the Cross-Layer Module View	390
17.2 Stakeholders and Concerns for the Cross-Layer View	390
17.3 Cross-Layer Architectural Principles	391
17.4 Module Taxonomy and Layer Mapping	392
17.5 Standard Module Specification Template	396
17.6 Detailed Cross-Layer Module Specifications	397
17.6.1 Module A: Data and Protocol Engine	397
17.6.2 Module B: Decoder and Baseline Measurement Engine	399
17.6.3 Module C: Calibration and Uncertainty Engine	401
17.6.4 Module D: Evidence Verification Engine	402
17.6.5 Module E: Temporal Regime and Trust Engine	405
17.6.6 Module F: Adaptive Policy Engine	407
17.6.7 Module G: Simulation and Stress Engine	408
17.6.8 Module H: Embodiment Adapter Engine	410
17.6.9 Module I: Dashboard and Reporting Engine	412
17.7 Cross-Layer Interface Contract Matrix	414
17.8 Feedback Loops and Permitted Cross-Layer Interactions	425
17.9 Runtime Scenarios	427
17.10 Cross-Layer Quality Attribute Scenarios	429
17.11 Cross-Layer Failure Modes and Defensive Design	430
17.12 Cross-Layer Evaluation and Ablation Plan	431
17.13 Implementation Artifact Map	433
17.14 Layers 4–7 Evaluation-Lineage Propagation Addendum	435
17.15 Layers 8–10 Evaluation-Lineage Propagation Addendum	436

17.16	Cross-Layer Consistency Checklist	438
18	Detailed Input-Output Proximity Map	439
18.1	Purpose of the Proximity Map	440
18.2	Proximity-Map Principles	440
18.3	Global Feed-Forward Record Chain	441
18.4	Control and Feedback Paths	443
18.5	Expanded Module-Level Proximity Table	448
18.6	Record-Level Dictionary and Ownership Map	459
18.7	Granularity Alignment Map	489
18.8	Boundary Validation and Quality Gates	494
18.9	Failure, Ambiguity, and Contradiction Map	496
18.10	Proximity Map for Ablation Studies	498
18.11	Proximity Map for Dashboard and Thesis Artifacts	500
18.12	Implementation Ordering Implied by the Map	502
18.13	Claim-Safety and Scope-Control Map	503
18.14	Expected Implementation Artifacts	504
18.15	Layers 0–3 evaluation-slice proximity addendum	507
18.16	Layers 4–7 evaluation-lineage proximity addendum	508
18.17	Layer 9 embodiment proximity addendum	508
18.18	Layer 7 policy-update proximity addendum	509
18.19	Layers 8–10 evaluation-lineage proximity addendum	509
18.20	Final Consistency Checklist	510
19	Retired detailed dossier: Policy-Learning Architecture merged into Layer 6	511
20	Retired detailed dossier: MyoSuite/OpenSim Embodiment Architecture merged into Layer 9	512
21	Retired detailed dossier: Dashboard Architecture in Detail merged into Layer 10	512
22	Ablation and Evaluation Architecture	512
22.1	Why ablation is central	512
22.2	Layer responsibility, boundaries, and implementation role	513
22.3	Whole-system evaluation questions	514
22.4	Evaluation modes	514

22.5	Core ablation ladder	517
22.6	Internal evaluation modules	521
22.7	Matched-comparison discipline	524
22.8	Input contract	525
22.9	Output contract	527
22.10	Metrics architecture	529
22.11	Statistical and robustness analysis	531
22.12	Claim-evidence matrix	532
22.13	Failure modes and negative-result interpretation	534
22.14	Reproducibility and implementation linkage	535
22.15	Dashboard and reporting linkage	536
22.16	Claim-safety and boundary discipline	537
22.17	Definition of thesis-ready evaluation	538
23	Implementation Pipeline for Stretch C	539
23.1	Thesis role and contribution of the implementation pipeline	540
23.2	Pipeline design principles	540
23.3	Layer boundary, scope, and expected implementation artifacts	541
23.4	Capability tiers and thesis-valid stopping points	542
23.5	Phase overview matrix	543
23.6	Consistent phase template	546
23.7	Detailed phase specifications	546
23.7.1	Phase 0: Repository, configuration, and record schema	546
23.7.2	Phase 1: Public data and split protocol	548
23.7.3	Phase 2: Baseline decoders	549
23.7.4	Phase 3: Calibration and uncertainty	550
23.7.5	Phase 4: IHARQ-lite evidence verification	552
23.7.6	Phase 5: RegimeRisk temporal trust	553
23.7.7	Phase 6: Evidence-quality estimator and supervised Adaptive-IHARQ . .	554
23.7.8	Phase 7: Learning-to-defer	555
23.7.9	Phase 8: Simulated closed-loop readiness	556
23.7.10	Phase 9: StressForge-Lite	557
23.7.11	Phase 10: Contextual bandit	558
23.7.12	Phase 11: Reinforcement-learning policy	560
23.7.13	Phase 12: MyoSuite embodiment demo	561

23.7.14 Phase 13: OpenSim replay or optional comparison	562
23.7.15 Phase 14: Dashboard and cards	563
23.7.16 Phase 15: Final thesis integration	564
23.8 Phase dependency and record-lineage matrix	565
23.9 Dependency classes and mockability rules	567
23.10 Feedforward and loopback behavior	567
23.11 Validation, leakage, and claim-safety gates	569
23.12 Implementation-readiness levels	571
23.13 Risk, fallback, and criticism-control matrix	571
23.14 Layers 4–7 evaluation-lineage pipeline addendum	572
23.15 Layers 8–10 evaluation-lineage pipeline addendum	574
23.16 Final pipeline definition of done	575
24 Recommended Repository Architecture	575
24.1 Purpose and architectural role	576
24.2 Input and output contract of the repository layer	577
24.3 Repository design principles	578
24.4 Canonical top-level repository tree	579
24.5 Configuration architecture	586
24.6 Source-code module architecture and allowed dependencies	588
24.7 Canonical record and artifact storage architecture	591
24.8 Schema, manifest, and registry architecture	600
24.9 Experiment orchestration and command structure	602
24.10 Dashboard, cards, and thesis-export contract	603
24.11 Testing, validation, and defensive-design architecture	604
24.12 Reproducibility package architecture	606
24.13 Data, large-file, and model-artifact policy	607
24.14 Optional and Stretch C extension boundaries	607
24.15 Repository-to-layer and repository-to-module mapping	608
24.16 Module-level repository contracts	613
24.17 Feedforward and loopback behavior in repository terms	616
24.18 Evaluation design for the repository layer	618
24.19 Repository-level failure modes and safeguards	618
24.20 Academic and claim-safety boundaries	619
24.21 Layers 4–7 evaluation-lineage repository addendum	620

24.22	Layer 7 policy-update repository addendum	621
24.23	Layers 8–10 evaluation-lineage repository addendum	621
24.24	Consistency checklist	622
25	Failure Modes and Defensive Design	623
25.1	Purpose and architectural role	623
25.2	Layer boundaries and internal responsibility	624
25.3	Relationship to the full architecture	625
25.4	Stakeholders and inspection questions	628
25.5	Input contract	629
25.6	Output contract	632
25.7	Failure-mode taxonomy	633
25.7.1	A. Governance and claim-safety failures	633
25.7.2	B. Public-data and protocol failures	634
25.7.3	C. Decoder, calibration, and selective-prediction failures	634
25.7.4	D. IHARQ evidence-verification failures	635
25.7.5	E. RegimeRisk temporal-trust failures	636
25.7.6	F. Adaptive policy and learning-to-defer failures	636
25.7.7	G. Contextual bandit and reinforcement-learning failures	637
25.7.8	H. StressForge-Lite failures	637
25.7.9	I. Simulated closed-loop environment failures	638
25.7.10	J. MyoSuite/OpenSim embodiment failures	639
25.7.11	K. Dashboard, cards, and reproducibility failures	639
25.7.12	L. Complexity, scope, and implementation-feasibility failures	640
25.8	Central defensive-design matrix	641
25.9	Quality gates and failure responses	643
25.10	Claim-impact matrix	645
25.11	Negative-result and degradation handling	646
25.12	Dashboard and card warning map	648
25.13	Repository and test linkage	649
25.14	Implementation-phase risk map	650
25.15	Optional-component fallback discipline	652
25.16	Repair-action table	653
25.17	Layers 4–7 evaluation-lineage failure-mode addendum	654
25.18	Layers 8–10 evaluation-lineage failure-mode addendum	655

25.19	Final consistency checklist for the Failure Modes and Defensive Design section	656
26	What the Final Stretch C System Should Demonstrate	658
26.1	Purpose and architectural role	658
26.2	Engineering responsibility, boundary, and internal structure	658
26.3	Final Demonstration section input and output contract	660
26.4	Dependency logic	663
26.5	Feedforward and loopback behavior	664
26.6	Demonstration tiers and thesis-valid stopping points	665
26.7	Final demonstration-evidence matrix	667
26.8	Layers 4–7 evaluation-lineage final-demonstration addendum	674
26.9	Layers 8–10 evaluation-lineage final-demonstration addendum	674
26.10	Required claim-evidence discipline	675
26.11	Negative-result and fallback interpretation	676
26.12	Connection to the implementation pipeline	677
26.13	Dashboard, cards, repository, and reproducibility linkage	679
26.14	Final system acceptance criteria	680
26.15	Final consistency checklist for the Final Demonstration section	681
27	Final Architecture Summary	682
27.1	Final system identity	682
27.2	Whole-architecture flow as a functional graph	683
27.3	Layer integration summary	685
27.4	Evidence and record-contract summary	686
27.5	Capability tiers and stoppable thesis value	688
27.6	Evaluation and ablation synthesis	689
27.7	Implementation and repository closure	690
27.8	Quality attributes and failure-mode defenses	691
27.9	Claim-safety and non-claim boundaries	692
27.10	Layers 4–7 evaluation-lineage closure	692
27.11	Layers 8–10 evaluation-lineage closure	693
27.12	Final thesis contribution statement	694
28	Selected Technical References and Tool Anchors	694
28.1	Architectural responsibility and boundaries of this section	695
28.2	Layer input and output contracts	695

28.3 Internal module structure of the Selected Technical References and Tool Anchors section	698
28.4 Reference and tool-anchor selection rules	701
28.5 Anchor taxonomy	701
28.6 Layer-to-anchor matrix	704
28.7 Preserved and strengthened existing anchors	709
28.8 Tool-anchor card schema	711
28.9 Method-anchor card schema	712
28.10 Record and artifact traceability map	713
28.11 Dependency logic	715
28.12 Feedforward and loopback behavior	716
28.13 Evaluation and ablation linkage	718
28.14 Versioning, reproducibility, and implementation requirements	719
28.15 Dashboard and card linkage	720
28.16 Failure-mode linkage	721
28.17 Claim-safety and boundary discipline	722
28.18 Final thesis citation cleanup requirements	723
28.19 Selected anchor bibliography for architecture-stage use	723
29 Layer 1 M8 synchronization addendum (T12)	726
29.1 Architecture realization lock	726
29.2 Canonical output and ownership clarification	726
29.3 Source-truth and representation-view rule	727
29.4 Layer 1 completion and boundary status	727
29.5 Synchronization references	728
30 Layer 1 bidirectional synchronization and change-propagation lock (T13)	729
30.1 Architecture-to-Layer-1 binding	729
30.2 Layer-1-to-architecture feedback	729
30.3 Dependency invalidation	729
30.4 Current closure	729
30.5 Controlling references	729
31 Layer 1 bidirectional integrity and compatibility lock (T14)	730
31.1 Current compatible revisions	730
31.2 Topology and ownership protection	730

31.3	Reverse change propagation	730
31.4	Current closure	730
32	Layer 1 green-light revalidation and source-assertion firewall (T15)	731
32.1	Newly explicit source contradiction	731
32.2	Green-light scope	731
32.3	Preservation and synchronization	731
33	Layer 1 stage-appropriate green-light and lifecycle clarification (T16)	732
33.1	Lifecycle sequence	732
33.2	Stage-appropriate status	732
33.3	Authority and non-overclaim	732
33.4	Mutable technical baseline correction	732
34	T17 Layer 2 Nuts-and-Bolts Post-Finalization Synchronization	732
34.1	Exact Layer 2 module inventory remains unchanged	733
34.2	Clarified producer, consumer, and lifecycle boundaries	733
34.3	Phase and ablation boundary confirmation	734
34.4	Architecture handoff status	734
35	T18 Layer 2 Bidirectional Synchronization and Compatibility Lock	735
35.1	Compatible companion revisions	735
35.2	Bidirectional but asymmetric authority	735
35.3	Reverse change propagation	735
35.4	No-drift rule	735
35.5	Current closure	735
36	T19 Layer 3 Detailed-Design Realization and Bidirectional Synchronization Lock	736
36.1	Architectural conclusion	736
36.2	Ten-module realization lock	736
36.3	Architecture-owned handoff and noninterference clarifications	737
36.4	Artifact-class boundary	737
36.5	Reverse invalidation contract	738
36.6	Architecture readiness statement	738
37	T20 Layer 3 Full Bidirectional Synchronization and Reciprocal Authority Lock	739

37.1	Effective companion set	739
37.2	Architecture to Layer 3 binding obligations	740
37.3	Layer 3 to Architecture reverse-impact obligations	740
37.4	Ten-module reciprocal architecture matrix	741
37.5	Contradiction, successor and invalidation rule	743
37.6	T20 acceptance statement	743
38	T21 Layer 4 Nuts-and-Bolts Post-Finalization Synchronization	743
38.1	Purpose, authority and preservation	743
38.2	Layer 4 fixed architecture closure	744
38.3	Principal product-family closure	744
38.4	Action, reason and ablation invariants	745
38.5	A5, A6 and A7 boundary	745
38.6	T21 acceptance statement	746
39	T22 Layer 4 Post-Finalization Bidirectional Synchronization	746
39.1	Purpose and precedence	746
39.2	Architecture to Layer 4 contract [AUTHORITY_TO_LAYER4]	746
39.3	Layer 4 to Architecture return contract [LAYER4_TO_AUTHORITY]	746
39.4	Bidirectional module and product closure	746
39.5	Reverse-impact and correction protocol	746
39.6	Cross-layer noninterference	747
39.7	T22 acceptance statement	747
40	Layer 4 Complete-Chain Bidirectional Synchronization	748
41	T24 Layer 5 R7 Post-Finalization Bidirectional Synchronization	749
41.1	Identity, scope, and precedence	749
41.2	Exact Layer 5 topology and responsibility closure	749
41.3	Canonical product-family interpretation	750
41.4	Complete governed boundary set	750
41.5	Bidirectional authority contract	751
41.6	Maturity qualification	751
42	T25 Layer 5 R8 Full Bidirectional Synchronization and Successor Rebinding	751
42.1	Identity and scope	751
42.2	Normative successor set	751

42.3	Authority-to-realization and realization-to-authority handshake	751
42.4	Exact retained Layer 5 closure	752
42.5	Bidirectional synchronization packet	752
42.6	Invalidation roots	752
42.7	Maturity boundary	753
43	T26 Layer 5 R9 Full Bidirectional Semantic Closure	753
43.1	Identity, preservation and scope	753
43.2	Domain authority and conflict precedence	753
43.3	Two-way document handshake	754
43.4	Machine-verifiable semantic assertions	754
43.5	Versioned change-packet rule	755
43.6	Maturity boundary	755
A	T27 Layer 6 R11 Post-Finalization Architecture Closure	756
A.1	Architecture disposition	756
A.2	Exact canonical Layer 6 taxonomy	756
A.3	Workstream and decision allocation	756
A.4	Principal architecture products	756
A.5	Cross-layer ownership invariants	757
A.6	Maturity boundary	757
A	T28 Layer 6 R11 Complete Bidirectional Authority Synchronization	758
A.1	Purpose and noninterference	758
A.2	Exact Layer 6 surfaces and ownership	758
A.3	Workstream and decision checksum	758
A.4	Bidirectional interface closure	758
A.5	Product topology closure	759
A.6	Maturity boundary	759
A	T29 Layer 6 R12 Closed-Loop Bidirectional Authority Synchronization	760
A.1	Purpose	760
A.2	Closed-loop successor set	760
A.3	Round-trip authority rule	760
A.4	Forward and reverse correction cycle	760
A.5	Completeness and maturity	760

1 Purpose and Scope of This Stretch C Architecture Document

This document defines the maximum, most extended architecture of IHARQ BenchGuard under the Stretch C scope. It is written as an architecture document, not as a general proposal, not as a literature review, and not as an implementation notebook. Its purpose is to make the system implementable by specifying the functional modules, submodules, input records, output records, feedback paths, simulation interfaces, policy-learning interfaces, dashboard outputs, and implementation order.

The architecture is built from scratch around the final intended system rather than around a minimum viable prototype. The minimum and target versions remain useful as fallback execution levels, but the architecture in this document is deliberately designed so that the following components are first-class citizens:

- public EEG benchmark and low-calibration evaluation spine;
- calibrated uncertainty and evidence-quality estimation;
- IHARQ-style evidence verification with accept, wait, request-evidence, combine, fallback, uncertain, and unsafe actions;
- RegimeRisk temporal trust monitoring for drift, fatigue-like degradation, instability, changing noise, and changing decoder reliability;
- learned readiness policies, including supervised Adaptive-IHARQ, learning-to-defer, contextual-bandit adaptation, and reinforcement-learning policy learning in simulation;
- StressForge-Lite stress generation for delay, dropout, weakness, noisy intent, fatigue-like degradation, drift, and combined stress;
- MyoSuite/OpenSim embodied simulation demo, used only as a simulated proof of concept for assistive-action readiness and not as clinical validation;
- dashboard and reporting infrastructure for interactive inspection, reliability cards, regime cards, controller cards, stress cards, failure taxonomy, and reproducibility artifacts.

The central architectural question is:

How can a low-calibration neural-intent decoder be surrounded by reliability, uncertainty, temporal trust, adaptive decision, simulated embodiment, and reporting modules so that the system decides not only what the user probably intended, but whether the evidence is safe, stable, and timely enough to support simulated assistive action?

This document therefore focuses on **system function**. It answers what each part does, why each part exists, what problem it solves, what it consumes, what it produces, how it is independent from other parts, and how it feeds the rest of the system.

2 Stretch C Definition and Non-Negotiable Scope Boundaries

2.1 Definition of Stretch C

Stretch C is the maximum version of the architecture. It assumes that the project is not limited to offline classification metrics. Instead, the project becomes a complete reliability architec-

ture with offline evidence evaluation, adaptive readiness policy learning, simulated closed-loop behavior, stress testing, optional musculoskeletal embodiment, and an observability dashboard.

The Stretch C system is composed of five capability tiers:

Tier	Name	Meaning in the architecture
C0	Public-data reliability spine	Public EEG datasets, fixed splits, baseline decoders, calibrated uncertainty, per-trial prediction logs, low-calibration curves, and reproducible evaluation.
C1	IHARQ and RegimeRisk reliability control	Rule-based evidence verification, temporal trust monitoring, fallback decisions, unsafe-state detection, and reliability cards.
C2	Adaptive policy layer	Learned evidence-quality estimator, cost-based readiness pseudo-labels, supervised Adaptive-IHARQ, learning-to-defer, and selective-prediction baselines.
C3	Bandit/RL simulation layer	Contextual-bandit threshold/action adaptation and reinforcement-learning readiness policy inside a declared simulated closed-loop environment.
C4	Embodiment and dashboard layer	MyoSuite/OpenSim simulated assistive-action demonstration plus a dashboard that exposes decoder, reliability, regime, policy, stress, and embodiment behavior.

In this document, Stretch C means **C0 through C4**. The document is therefore intentionally more comprehensive than a minimum paper architecture.

2.2 Scientific claim boundaries

Claim-safety rule: The project may claim that it evaluates a reliability architecture for low-calibration neural-intent readiness using public EEG datasets, simulated closed-loop decision consequences, and optional simulated musculoskeletal embodiment. It must not claim that it validates a real clinical BCI-FES system, a real implanted BCI controller, a real rehabilitation intervention, or real patient-safe stimulation.

The MyoSuite/OpenSim part is a **simulated embodiment demo**. It shows how readiness decisions can gate or modulate simulated assistive actions through declared proxy mappings, readiness safety gates, and adapter-based simulator interfaces. It does not prove clinical safety. It does not prove that EEG motor-imagery classes directly control real muscles. It does not replace user studies, clinical testing, biomechanics validation, or medical-device verification.

The contextual-bandit and reinforcement-learning parts are also bounded. They are simulated policy-learning modules. They may be used to compare decision policies under declared reward functions and stress profiles. They must not be described as clinically deployed learning controllers.

2.3 Why Stretch C is architecturally valuable

The maximum version is worth designing because it prevents the thesis from being seen as only another EEG classifier benchmark. The architecture becomes a reliability and decision-control framework. The classification decoder is still important, but it is only one component inside a larger system that asks:

- Is the decoder confident?
- Is that confidence calibrated?
- Is the evidence stable across windows, channels, and models?
- Has the operating regime changed?
- Should the system accept now, wait, request more evidence, defer, fallback, or stop?
- What is the cost of a wrong action compared with a delayed or fallback action?
- How does the decision behave in a simulated closed loop when delay, fatigue-like degradation, dropout, weakness, and drift occur?
- How would the readiness decision influence a simulated assistive-action environment?
- Can the entire behavior be inspected through cards and dashboard views?

3 Architectural Style: Functional Graph, Not a Simple Stack

The original project was expressed as layers. The layer view is still useful for explanation and traceability. However, the Stretch C implementation should not be coded as a rigid one-pass stack. It should be implemented as a **functional graph** with records passed between modules and with controlled feedback from monitoring, policy, simulation, and dashboard modules.

3.1 Main architectural planes

The most stable architecture is divided into seven planes:

Plane	Name	Responsibility
P0	Governance plane	Claim safety, scope control, public-data validity, thesis-language discipline, and artifact traceability.
P1	Measurement plane	Dataset preparation, subject/session splitting, low-label protocols, decoder training, prediction logging, baseline metrics, and calibration.
P2	Evidence plane	Evidence-quality estimation, IHARQ evidence verification, multi-window stability, channel-subset stability, model-family agreement, and fallback logic.

P3	Regime plane	Temporal trust, regime-state estimation, drift/fatigue-like risk, model trust scores, stop-loss warnings, and stricter evidence recommendations.
P4	Policy plane	Adaptive-IHARQ, learning-to-defer, selective prediction, cost-based pseudo-labels, contextual bandit, and reinforcement-learning readiness policy.
P5	Simulation and embodiment plane	Closed-loop readiness sessions, StressForge-Lite stress profiles, MyoSuite/OpenSim adapter, simulated assistive actions, and reward/state logging.
P6	Reporting and dashboard plane	Cards, dashboard, plots, failure taxonomy, ablations, reproducibility package, and thesis-facing summaries.

3.2 System-level flow

The system-level flow can be summarized as follows:

Public EEG datasets

- > fixed protocol and low-calibration splits
- > baseline decoders and model families
- > raw prediction records
- > calibration and uncertainty records
- > evidence-quality and IHARQ records
- > RegimeRisk temporal trust records
- > Adaptive-IHARQ / defer / bandit / RL policy records
- > simulated closed-loop readiness sessions
- > StressForge-Lite stress conditions
- > optional MyoSuite/OpenSim embodiment demo
- > dashboard, cards, figures, ablations, and reproducibility artifacts

This flow is not strictly linear because RegimeRisk can feed back into IHARQ and Adaptive-IHARQ, the policy layer can change thresholds or action selection in the simulation layer, StressForge-Lite can create test conditions for the simulation and embodiment layers, and the dashboard receives records from all modules.

3.3 Why feedback exists

Feedback is included only where it solves a specific problem:

- RegimeRisk feedback to IHARQ solves the problem that a confidence threshold that worked early in a session may become unsafe after drift or fatigue-like degradation.
- RegimeRisk feedback to Adaptive-IHARQ gives the learned policy context about whether the current operating state is stable, noisy, drift-risk, fatigue-risk, or unsafe.
- Simulation feedback to bandit/RL policies provides reward or regret signals so policies can compare accepting, waiting, deferring, and fallback decisions.

- StressForge-Lite feedback creates declared difficult conditions so the system can be evaluated under known stress profiles rather than only under clean offline trials.
- Dashboard feedback is human-inspection feedback, not control feedback. It exposes failure patterns and supports debugging, but it should not silently change experimental results.

4 Revised Layer Numbering for the Stretch C Architecture

The original Layer 0–Layer 9 design is preserved for traceability, but the Stretch C architecture benefits from a revised numbering that makes embodiment and dashboard explicit. The revised architecture uses eleven layers. Layers are not all sequential runtime layers; some are governance or reporting layers.

Revised layer	Name	Role in Stretch C
Layer 0	Claim-Safety and Scope Governance	Controls scientific claims, public-data limits, simulation limits, clinical non-claims, and artifact traceability.
Layer 1	Public-Data and Protocol Anchor	Builds the dataset/testbed foundation: public EEG, subject/session logic, fixed splits, low-calibration protocols, and dataset cards.
Layer 2	Decoder and Baseline Measurement Spine	Trains classical, Riemannian, deep, compact SSL when available, and ensemble baselines; produces per-window and per-trial prediction logs.
Layer 3	Calibration, Uncertainty, and Selective Prediction	Calibrates decoder probabilities, computes uncertainty features, and establishes threshold/selective baselines.
Layer 4	IHARQ Evidence Verification	Performs decoder-agnostic, post-calibration, reason-coded evidence verification with accept, wait, request-evidence, combine, fallback, uncertain, and unsafe decisions.
Layer 5	RegimeRisk Temporal Trust Monitoring	Detects temporal instability, drift-risk, fatigue-like degradation, dropout risk, and changing model trust; feeds context into Layers 4 and 6.
Layer 6	Adaptive Readiness Policy Layer	Implements evidence-quality estimation, supervised Adaptive-IHARQ, learning-to-defer, contextual bandit, and RL readiness policy.
Layer 7	Simulated Closed-Loop Readiness Environment	Runs repeated readiness sessions with state, actions, delayed consequences, reward, declared stress/regime variables, safety events, and policy logs.
Layer 8	StressForge-Lite Stress Generator	Creates bounded stress profiles for delay, dropout, noisy intent, weakness, drift, fatigue-like degradation, and combined stress.

Layer 9	MyoSuite/OpenSim Embodiment Demo	Maps readiness-gated neural intent into simulated musculoskeletal assistance through a controlled adapter interface.
Layer 10	Dashboard, Cards, and Reproducibility Layer	Provides cross-layer scientific observability, standardized cards, thesis-ready exports, artifact lineage, claim-safety warnings, ablation visibility, and a reproducibility package.

4.1 Mapping from the original layer design to the revised Stretch C design

Original design element	Revised Stretch C location	Reasoning
Original Layer 0: Claim-Safety and Validity	Revised Layer 0	Preserved, but expanded to cover MyoSuite/OpenSim and policy-learning safety boundaries.
Original Layer 1: Public-Data Anchor and Full-Spectrum Transfer	Revised Layer 1	Preserved as the experimental anchor; transfer claims are governed by Layer 0.
Original Layer 2: Supporting Evaluation Scaffold	Revised Layer 2	Renamed to emphasize that this is the decoder and measurement spine, not only an auxiliary scaffold.
Original Layer 3: Calibration and Uncertainty	Revised Layer 3	Preserved, with selective prediction and uncertainty baselines made explicit because they are necessary controls for IHARQ and deferral claims.
Original Layer 4: Wireless-Inspired IHARQ	Revised Layer 4	Preserved as the main evidence-control layer. It remains decoder-agnostic and post-decoder.
Original Layer 5: Finance-Inspired RegimeRisk	Revised Layer 5	Preserved as the temporal trust layer, with stronger feedback to Layers 4, 6, 7, and 9.
Original Layer 6: Adaptive-IHARQ Learned Readiness Verifier	Revised Layer 6	Expanded into the full adaptive readiness policy layer: supervised policy, learning-to-defer, contextual bandit, and RL.
Original Layer 7: Simulated Closed-Loop Readiness	Revised Layer 7	Preserved, now serving as the environment for contextual bandit and RL policy evaluation.
Original Layer 8: StressForge-Lite	Revised Layer 8	Preserved, now explicitly connected to Layer 7 and Layer 9.
Original Layer 9: Cards, Reporting, and Reproducibility	Revised Layer 10	Moved to Layer 10 so that the MyoSuite/OpenSim embodiment demo can become its own Layer 9. Reporting remains cross-cutting.

Original optional MyoSuite/OpenSim demo	Revised Layer 9	Promoted from optional future note to explicit Stretch C layer with bounded claim language.
Original optional dashboard	Revised Layer 10	Promoted from communication artifact to observability and reproducibility interface.

5 System-Wide Interface Contracts

The architecture should be implemented around explicit records rather than hidden intermediate variables. Each module should read and write structured records. This makes the system debuggable, reproducible, and dashboard-ready.

5.1 Canonical records

Record	Core contents	Produced by / consumed by
DatasetRecord	Dataset/source identity, subject/session/run structure, task set/labels, channels, sampling rates, preprocessing ID, split ID, protocol version, license/source notes, inclusion/exclusion notes where applicable, and claim-scope tags.	Produced by Layer 1; consumed by all experimental layers.
WindowRecord	Dataset/subject/session/run/trial/window identity, parent-trial lineage, window-role tag, start/end time, segment pointer, overlap group, artifact flags, channel mask, normalized/original label, split membership, <code>label_budget_id</code> , preprocessing ID, preprocessing version, and config hash.	Produced by Layer 1; consumed by Layers 2–10, with Layer 9 using provenance/flag context and Layer 10 using read-only reporting lineage.
SplitRecord	Split ID, protocol type, split level, train/validation/test/calibration/adaptation membership, <code>label_budget_id</code> , calibration budget, seed, stratification rule, split version, and leakage-check status.	Produced by Layer 1; consumed by Layers 2–6 and Layers 8–10, with Layer 9 preserving split/protocol provenance for embodiment mapping.
PreprocessingRecord	Filtering, resampling, channel selection, referencing, epoching, normalization method and fitting scope, artifact policy, implementation script, preprocessing version, and config hash.	Produced by Layer 1; consumed by Layers 2, 3, and 8–10, with Layer 9 using preprocessing provenance for embodiment/demo lineage.

LabelMapRecord	Dataset-specific original labels, normalized labels, task family, protocol inclusion/exclusion status, rest/no-action policy, exclusion reason, label-map version, and proxy-intent limitation.	Produced by Layer 1; consumed by Layers 0, 2, 7, 9, and 10 for claim-safety checks, decoder/task mapping, proxy-intent command boundaries, and reporting lineage.
ArtifactFlagRecord	Missing-channel, bad-channel, suspected-artifact, short-trial, missing-label, sampling-rate-mismatch, and window-overlap-risk flags with affected dataset/subject/session/run/trial/window IDs, severity, action, source, and notes.	Produced by Layer 1; consumed by Layers 2, 4, 5, and 8–10, including Layer 9 embodiment/demo safety context and Layer 10 limitation reporting.
DatasetCard	Dataset identity, source/citation notes, task description, subject/session coverage, recording/channel details, label summary, inclusion or exclusion rationale, public-data limitations, protocol compatibility, allowed and forbidden claims, known risks, and warning status.	Produced by Layer 1; consumed by Layer 10 and thesis chapters as a read-only public-data scope and limitation artifact governed by Layer 0.
ProtocolCard	Protocol ID/version, protocol type, split logic, low-calibration budget, preprocessing ID, windowing ID, leakage controls, downstream consumers, protocol limitations, and diagnostic-readiness warnings.	Produced by Layer 1; consumed by Layer 10 and thesis chapters as a read-only protocol-scope artifact; no dashboard or downstream layer may silently rewrite it.
ValidationReport	Schema validity, metadata completeness, distribution summaries, split disjointness, leakage checks, duplicate/window-lineage checks, train-only preprocessing fitting status, label-budget integrity, comparison-key completeness, reproducibility status, and pass/warning/fail state.	Produced by Layer 1; consumed by all layers, evaluation scripts, Layer 10, and reproducibility packaging as the public-data/protocol validation gate.

Layer1Manifest	Protocol version, dataset IDs, paths to records/cards/reports, validation-report path, ablation-readiness path, config files, config hash, seed, code version, output paths, generation timestamp, validation status, ablation-readiness status, matched-comparison key completeness, and known limitations.	Produced by Layer 1; consumed by every downstream layer, Layer 10 artifact-lineage views, and the reproducibility package.
layer1_ablation_readiness_data_protocol	Protocol ablation-readiness status, matched-comparison key completeness, missing-key warnings, diagnostic-only downgrade reasons, protocol-version provenance, and links to Layer 1 validation reports.	Produced by Layer 1; consumed by evaluation scripts, Layer 10, the Implementation Pipeline section, the Failure Modes and Defensive Design section, and the Final Demonstration section as an evaluation/reporting artifact rather than a new runtime record; Layer 10 must expose it as read-only readiness lineage.
PredictionRecord	Schema version, dataset/subject/session/run IDs, trial/window IDs, split ID, canonical <code>split_role</code> , <code>label_budget_id</code> , model and decoder IDs, <code>model_family</code> , <code>predicted_class</code> , <code>probability_vector</code> or declared logits/decision-score fallback, <code>score_type</code> , <code>class_order</code> , <code>time_start/time_end</code> where available, <code>timestamp_or_order</code> where temporal replay is required, evaluation-only <code>label_true</code> , seed, config hash, artifact flags, optional latency, and local helper fields only when explicitly mapped to canonical exported names.	Produced by Layer 2; consumed by Layers 3–8, by Layer 9 only as bounded provenance/readiness-input context rather than embodiment-safety validation, and by Layer 10 for dashboard, reporting, cards, and reproducibility views.

ModelRegistryRecord	Schema version, model ID, decoder ID, model family/name, implementation path, artifact path, prediction-record path, training split and split_role , label_budget_id , hyperparameters, selected checkpoint, label_map_version , score_type , class_order , seed, config/environment hash, code version, training status, failed/skipped/unsupported-run reason, optional ensemble_membership , and registry schema path model_registry_schema.yaml .	Produced by Layer 2; consumed by Layer 3 calibration/uncertainty and score-schema validation, Layers 4–6 evidence and policy provenance, Layer 7 simulated replay when model identity is required, Layer 8 stress replay, Layer 10 dashboard/model cards, and reproducibility manifests.
BaselineMetricRecord	Decoder baseline metrics such as accuracy, balanced accuracy, F1 or AUC where applicable, confusion-matrix lineage, confidence intervals or seed summaries, dataset/model/subject/session aggregation level, label_budget_id , split_role , metric version/dictionary entry, comparison key, and diagnostic-only flags where matching fails.	Produced by Layer 2 evaluation; consumed by Layer 10 dashboard and by evaluation, ablation, and thesis-reporting sections as A0 decoder-baseline evidence.
LowCalibrationCurveRecord	Low-label or low-calibration budget ID, dataset/model/subject aggregation level, performance values across budgets, seed summaries, split_role , comparison key, confidence intervals where available, curve version, and budget-failure notes.	Produced by Layer 2 low-label evaluation; consumed by Layer 10 and thesis/evaluation artifacts, and optionally by Layer 3 budget-aware diagnostic analysis without changing split or threshold-selection rules.
SubjectProfileRecord	Subject/session identity, split-consistent baseline performance, difficulty metrics, low-label summaries, optional subject/session instability notes, aggregation version, and provenance to source PredictionRecord and BaselineMetricRecord rows.	Produced by Layer 2 decoder evaluation; consumed by Layer 5 RegimeRisk diagnostics, Layer 6 policy-context analysis where allowed, Layers 7–8 replay/stress analyses where subject profiles are explicitly allowed, and Layer 10 subject-profile/dashboard views.

EnsembleControlRecord	Ordinary majority-vote, probability-averaging, confidence-threshold, longer-window, multi-window, disagreement, aggregation-rule, model-family membership, matched-comparison, and delay-burden information used as A4-style ordinary-combination controls.	Produced by Layer 2 ensemble comparison builder; consumed by Layers 3 and 4 for calibration/disagreement and IHARQ comparisons, by Layer 10 for ensemble-control tables/cards, and by ablation/evaluation scripts to prevent confusing IHARQ with simple model combination.
FailureCaseIndex	Decoder-level failure cases, high-risk examples, raw high-confidence wrong candidates, difficult-subject/session references, failed/skipped run links, source prediction IDs, and limitation notes.	Produced by Layer 2 evaluation and logging; consumed by Layers 4 and 5 for offline failure analysis, by Layer 10 for failure/dashboard views, and by thesis defensive-design sections.
Layer2ReadinessReport	Downstream-readiness status for PredictionRecord , ModelRegistryRecord , budget, subject-profile, score-schema, class-order, ensemble-control, and consumer-compatibility coverage, with missing-field warnings and diagnostic-only downgrade reasons.	Produced by Layer 2 downstream readiness validator; consumed by Layers 3–10, Layer 10 dashboard/readiness panels, evaluation scripts, reproducibility manifests, and final thesis reporting as a compatibility artifact rather than a new runtime decision record.
ModelCard	Decoder/model-family identity, registry linkage, protocol and split lineage, score type, class order, baseline metrics, low-calibration behavior, failed-run status, limitations, and claim boundary.	Produced or exported from Layer 2 records and rendered by Layer 10; consumed by thesis baseline reporting, dashboard model views, and reproducibility packages.
BenchmarkCard	Baseline table lineage, low-calibration curves, subject-profile summaries, ensemble-control summaries, failure/warning notes, matched-comparison keys, diagnostic-only status, and source-record references.	Produced from Layer 2 evaluation artifacts and rendered by Layer 10; consumed by thesis ablation/reporting sections as decoder-benchmark evidence, not readiness-safety evidence.

CalibrationRecord	Raw and calibrated probabilities, raw and calibrated confidence, calibration method/scope, calibration split ID, reliability-bin membership, correctness for evaluation-only scoring, Brier/NLL components, ECE-style summaries, source PredictionRecord and model identifiers, and config hash.	Produced by Layer 3; consumed by Layers 4 and 6, optionally by Layer 5 summaries, and by Layer 10 dashboard/reliability-card/reproducibility views.
UncertaintyRecord	Linked prediction/calibration IDs, confidence, entropy, normalized entropy, margin, variation ratio, top-1/top-2 classes, top-1/top-2 margin label pair, model disagreement, conformal set size if used, selective-accept flag, uncertainty profile, and config hash.	Produced by Layer 3; consumed by Layers 4 and 6, optionally by Layer 5 summaries, and by Layer 10 uncertainty/dashboard views.
SelectivePredictionRecord	Threshold ID, uncertainty feature, selection split, evaluation split, coverage, rejection rate, retained accuracy, retained error rate, risk, high-confidence wrong rate, class/subject/session coverage, calibration budget, and operating-point lineage.	Produced by Layer 3; consumed by Layers 4, 6, 10, and the ablation chapter as A2/A3 selective-prediction baseline evidence.
ThresholdRegistryRecord	Threshold type, feature, selected value or rule, selected-on split, selection rule, target constraint, applicable models, allowed consumers, version, leakage status, rationale, and fairness note.	Produced by Layer 3 threshold registry; consumed by Layers 4, 6, 10, and the experiment controller, which must fail fast or mark runs exploratory when a downstream module requests an unregistered threshold.
ReliabilityAuditReport	Reliability diagrams, ECE-style summaries, Brier score, NLL, confidence histograms, high-confidence wrong tables, class/subject/session/model-family audits, risk-coverage summaries, source-record links, and warning flags.	Produced by Layer 3 reliability audit; consumed by Layer 10, reliability cards, thesis evaluation chapters, and failure/negative-result reporting.

SplitIntegrityReport	Split membership checks, calibration/threshold/test disjointness checks, leakage warnings, eligibility tables, invalid-run flags, nested-CV fold legality where used, and final-claim eligibility state.	Produced by Layer 3 calibration split and leakage guard; consumed by the experiment controller, Layer 10 reproducibility/readiness views, and thesis limitation reporting.
LeakageWarningRecord	Source split IDs, offending membership overlap or leakage condition, affected dataset/subject/session/window/model/budget IDs, severity, required action, and final-evaluation block/exploratory-only status.	Produced by Layer 3 leakage guard or Layer 2 validation when invalid split model/budget; consumed by the experiment controller, Layer 1/protocol repair workflow, Layer 10 warning panels, and claim-safety audits.
CalibrationEligibilityTable	By label/dataset/budget calibration eligibility, missing-class or sparse-budget status, method eligibility, fallback method, invalid-run reason, and split/legal-use status.	Produced by Layer 3 leakage and calibration-eligibility guard; consumed by calibrators, threshold selection, Layer 10, and evaluation scripts to prevent hidden test-driven calibration choices.
HighConfidenceWrongTable	Confident wrong predictions, source prediction/calibration/uncertainty IDs, class-pair errors, subject/session/model-family/budget grouping, confidence threshold, correctness source, and diagnostic-only status.	Produced by Layer 3 reliability audit/high-confidence error miner; consumed by Layers 4 and 6 for offline failure analysis only, by Layer 10 high-confidence-error views, and by thesis negative-result sections without becoming a decision-time feature.
CalibrationDashboard	Not ready. Calibration and uncertainty metrics, bin tables, risk-coverage curves, filters, warning flags, card references, source-record IDs, and export lineage for Layer 10.	Produced by Layer 3 dashboard/card exporter; consumed by Layer 10 dashboard backend, card generator, reproducibility manifests, and thesis figures/tables.
ReliabilityCard	Calibration, uncertainty, selective-prediction, threshold, high-confidence-error, and group-wise reliability summaries with source records, warning banners, diagnostic-only status, and claim boundaries.	Produced from Layer 3 records and rendered by Layer 10; consumed by thesis reliability evaluation and dashboard appendices.

Layer3MatchedOperatingPointTable	Matched coverage and matched-cost threshold/selective-prediction operating points, A1–A3 comparison keys, threshold-sufficiency notes, calibration-failure notes, high-confidence-wrong summaries, and diagnostic-only flags.	Produced by Layer 3; consumed by evaluation scripts, Layer 10, the Implementation Pipeline section, the Failure Modes and Defensive Design section, and the Final Demonstration section as an evaluation/reporting artifact rather than a new runtime record.
EvidenceQualityRecord	Evidence-quality score, reliable/unreliable proxy target, confidence, entropy, margin, stability features, signal-quality features, disagreement features, predicted failure probability if used, missing-evidence flags, decision-time/evaluation-only partition status, and evidence-source identifiers.	Produced by Layer 4 IHARQ evidence verification and by Layer 6 learned evidence-quality estimation when enabled; consumed by Layers 4–7, Layer 10, and evaluation scripts.
IHARQDecisionRecord	Canonical Layer 4 stored action token, optional human-readable action/display label, reason code, reason text, accepted prediction if any, evidence request count, combined evidence summary, fallback status, unsafe status, uncertainty status, evidence-quality features, RegimeRisk context if available, action cost, rule path, global/local ablation IDs when emitted during evaluation, matched-comparison key, diagnostic-only flag, seed, and configuration hash. The canonical stored tokens are <code>accept</code>, <code>wait</code>, <code>request_evidence</code>, <code>combine_evidence</code>, <code>fallback</code>, <code>uncertain</code>, and <code>unsafe</code>; downstream aliases must be represented through explicit mapping fields rather than by rewriting the stored Layer 4 token.	Produced by Layer 4; consumed by Layers 6–10 and evaluation scripts.

CombinedEvidenceRecord	Source window/source IDs, combination method, before/after confidence, before/after entropy, before/after margin, combined class, combined probability or vote summary, eligibility reason, final combined decision pointer, delay/evidence-cost fields, configuration hash, and provenance.	Produced by Layer 4 evidence combiner; consumed by Layers 4, 6, 7, and 10, evaluation scripts, and reproducibility manifests.
FallbackDecisionRecord	Fallback route, fallback reason, fallback target, fallback availability, fallback cost, selected fallback state, fallback success or failure during evaluation, fallback compatibility mode, source decision/evidence IDs, and claim boundary.	Produced by Layer 4 fallback router; consumed by Layers 6, 7, 9, and 10, evaluation scripts, failure taxonomy, and thesis defensive-design sections.
UnsafeEventRecord	Unsafe trigger, unsafe reason, blocked action, relevant evidence, RegimeRisk context, stress context, recommended conservative outcome, simulation/proxy boundary label, source decision/evidence IDs, severity, and warning text.	Produced by Layer 4 unsafe-state handling when evidence verification materializes an unsafe event; consumed by Layers 7, 9, and 10, failure taxonomy, thesis safety discussion, and claim-safety cards.
IHARQAblationRecord	Layer 4 ablated-variant ID, global/local ablation IDs, decisions and metrics under confidence-only, entropy-only, margin-only, no-stability, no-fallback, no-unsafe, no-combination, fallback-only, and regime-unaware variants, matched-comparison key, metric-dictionary ID, diagnostic-only flag, negative-result note, and claim-evidence pointer.	Produced by Layer 4 ablation and matched-comparison exporter; consumed by evaluation scripts, Layer 10, the ablation/evaluation architecture, and thesis ablation chapters.
IHARQTraceRecord	Full rule path, thresholds used, feature values, reason-code hierarchy, input file paths/hashes, configuration hash, metric-dictionary ID, dashboard trace fields, figure/table lineage, and reproducibility pointers.	Produced by Layer 4 decision logger; consumed by Layer 10, reproducibility packages, debugging workflows, and thesis figures/tables.

EvaluationOutcomeRecord	Layer-local metric outputs by dataset, subject, session, model, threshold, policy, regime, stress profile, embodiment mode, ablation step, global/local ablation ID, matched-comparison key, metric-dictionary ID, diagnostic-only flag, dashboard/export lineage, negative-result note, and claim-evidence pointer where applicable.	Produced by layer-local evaluation exporters, including Layer 4 IHARQ evaluation, Layer 5 RegimeRisk evaluation when evaluated, Layer 6 policy-readiness evaluation, Layer 8 A12 stress evaluation, and Layer 9 A13 embodiment evaluation; consumed by evaluation scripts, Layer 10, thesis tables, claim-evidence matrix generation, and final demonstration artifacts.
RegimeStateRecord	Regime label, regime probability, RegimeRisk score, risk trend, volatility-like score, drawdown-like score, explicit drift_risk_score , fatigue_like_risk_score , dropout_risk_score , failure-risk score, trust-score summary, trusted set, optional trust_state_record_ids , stop-loss status, downstream recommendation such as normal, stricter evidence, trusted-set restriction, fallback, no-action, unsafe, cooldown, or recovery monitoring, reason codes, optional stress/embodiment/transition/policy IDs, and provenance.	Produced by Layer 5; consumed by Layers 4, 6, 7, 9, and 10. Layer 6 and Layer 10 may follow linked TrustStateRecord rows when independent trust timelines are enabled.
TrustStateRecord	Source RegimeStateRecord ID, dataset/subject/session/trial/window/instance identity, trust target type and ID, trust score, trust trend, trust band, trusted-set membership, update source, decay/recovery state, reason codes, source evidence IDs, simulation/evaluation tags where applicable, and provenance.	Produced by Layer 5 when trust_allocation has an independent lifecycle, target identity, policy use, or dashboard view; consumed by Layer 6 and Layer 10, and optionally referenced by Layers 4, 7, and 9 through linked RegimeStateRecord fields.

<code>regime_timelines.csv</code>	Time-indexed regime labels, regime-risk scores, risk trends, recovery markers, regime transitions, stress/session references where applicable, source-record IDs, configuration hashes, and claim-boundary tags.	Produced by Layer 5; consumed by Layer 10 regime timeline views, RegimeCard , thesis figures, matched-comparison reports, and reproducibility packages.
<code>trust_scores.csv</code>	Decoder/model-family/evidence-source/channel/window/fallback trust scores, trusted-set membership, trust trend, trust band, update source, decay/recovery state, reason codes, source RegimeStateRecord or TrustStateRecord IDs, and provenance.	Produced by Layer 5 trust allocator; consumed by Layers 4 and 6 where allowed, Layer 10 trust-state views, RegimeCard , and thesis temporal-trust summaries.
<code>stop_loss_warnings.csv</code>	Stop-loss warning status, trigger, severity, affected record IDs, conservative recommendation, fallback/no-action/unsafe/cooldown/recovery instruction, source evidence, timestamp or simulated step, and simulation/proxy boundary label.	Produced by Layer 5 stop-loss module; consumed by Layers 4, 6, 7, 9, and 10, failure taxonomy, RegimeCard , and claim-safety reporting.
<code>ReadinessContextRecord</code>	Decision-time policy context with identity/split fields, decoder outputs, calibrated probabilities, uncertainty, evidence quality, IHARQ state, RegimeRisk /trust/stop-loss fields, action history, stress variables, optional embodiment state, and evaluation-only target fields kept separate from policy inputs.	Assembled and validated by Layer 6 from Layers 2–5 records and, only in declared simulation/evaluation modes, from Layer 7–9 context such as transition, stress, and embodiment records; consumed by Layer 6 policies, by Layers 7–9 where simulation-compatible policy context is required, and audited read-only by Layer 10.
<code>DeferralRecord</code>	Defer decision, deferral target, deferral reason, fallback availability, direct-versus-defer expected cost, matched-coverage group, and simulation-only flag where a supervisor proxy or embodiment route is used.	Produced by Layer 6; consumed by Layers 7, 9, and 10.

PolicyActionRecord	Policy identity, family, selected readiness action, optional source Layer 4 IHARQ action token, action-origin or alias-map reference, action probability, selected threshold, legality mask, expected cost/reward, exploration flag, reason code, context hash, config hash, simulation-only flag, and claim boundary. Layer 6 may add policy-only actions such as defer and no_action , but must preserve Layer 4 tokens when consuming IHARQDecisionRecord .	Produced by Layer 6; consumed by Layers 7–10.
PolicyUpdateTrace	Simulation-authorized policy-update trace with policy ID, policy family, context ID, state summary, action, reward, next-state ID, update step, exploration mode, training-or-evaluation mode, frozen-policy flag, simulator ID, seed, configuration hash, matched-comparison key, disabled-update status where applicable, and simulation-only boundary.	Produced by Layers 6 and 7 only for declared contextual-bandit/RL simulation-update runs or disabled-update audits; consumed by Layer 6 policy analysis, Layer 10 policy-update trace cards, ablation scripts, and reproducibility manifests.
ClosedLoopSessionRecord	Session-level environment metadata including dataset, subject/session identifiers where available, policy ID, stress profile ID, seed, replay mode, episode length, environment version, reward/action/evidence configuration hashes, update/frozen mode, and claim boundary.	Produced by Layer 7; consumed by Layer 10, reproducibility appendices, dashboard session timelines, and thesis simulation summaries.
EpisodeSummaryRecord	Rollout/session aggregate with global/local ablation IDs, metric-dictionary ID, matched-comparison key, diagnostic-only flag where applicable, reward/cost summary, safety-event summary, stress/embodiment mode, policy family, and claim-evidence pointer.	Produced by Layer 7; consumed by Layer 10, evaluation scripts, matched-comparison reports, and thesis result tables.

PolicyRolloutRecord	Full behavior of one policy under one session and stress profile, including policy identity, frozen/update mode, action sequence pointers, transition pointers, reward-trace pointers, evidence-request/delay behavior, safety events, optional embodiment linkage, and claim boundary.	Produced by Layer 7; consumed by Layer 6 policy analysis, ablation scripts, Layer 10 dashboards, and reproducibility manifests.
RewardTraceRecord	Reward and cost decomposition over time with reward version, reward terms, source transition IDs, state/action/next-state linkage, delay/evidence/fallback/defer/no-action/unsafe/embodiment costs where applicable, regret fields where available, seed, configuration hash, update/frozen mode, and simulation-only warning.	Produced by Layer 7; consumed by Layer 6 bandit/RL modules, Layer 10 reward dashboards, policy-update audits, ablation scripts, and reproducibility packages. It is distinct from Layer 9-specific Layer9RewardTrace .
EnvironmentDiagnosticRecord	Warning on warnings, failed transitions, invalid actions, reward anomalies, leakage checks, unmatched-comparison warnings, missing safety-event or reward-trace links, mismatched demo-manifest identifiers, and disabled-update audit status.	Produced by Layer 7; consumed by debugging tools, validation reports, reproducibility packages, Layer 10 warning panels, and claim-safety review.
TransitionRecord	State before action, selected readiness action after Layer 7 validation, optional source Layer 4 IHARQ action token, mapped simulator command where applicable, next state, declared reward, done flag, stress condition, embodiment response, safety event, latency, endpoint error, command success, and optional policy-update trace linkage. It separates readiness tokens from simulator commands, reward terms, adapter statuses, and update-trace references.	Produced by Layer 7, with Layer 9 writing embodiment-linked transition records or pointers where the embodiment adapter is active; consumed by Layer 6 contextual-bandit/RL training and analysis, Layer 9 embodiment adapters where active, Layer 10, and ablation/evaluation scripts.

StressProfileRecord	Stress profile ID, stress run ID, stress family/name/version, intensity label and numeric value, calibrated parameter bounds, target layer/record/field, injection point, pre/post stressed-value pointers, seed, schedule linkage through stress_schedule_id , clean-baseline pairing ID where applicable, output links, delay/dropout/weakness/noisy-intent/drift/fatigue-like/combined-stress parameters, invalid/impossible/unsafe simulation flags, claim boundary, and A12/dashboard/export lineage.	Produced by Layer 8; consumed by Layers 7, 9, and 10, with Layer 7 and Layer 9 consuming it alongside StressScheduleRecord ; Layer 10 consumes it read-only for StressForge pages, stress cards, matched-stress validation, claim-evidence views, and reproducibility manifests.
StressScheduleRecord	Stress schedule ID, parent stress profile ID, stress run ID where available, schedule type, per-window or per-step stress value, active component stressors, onset/duration, constant/ramp/burst/periodic/stochastic/A12/A13 generated scripted state, timestamp or simulated step index, seed, target layer/record linkage, clean-baseline pairing ID where applicable, affected-variable pointer, and dashboard/export lineage. Combined stress is represented by active component stressors under a declared stress family rather than by a separate incompatible schedule vocabulary.	Produced by Layer 8; consumed by Layers 7, 9, and 10 for stress replay, stress-conditioned embodiment through Layer 9 stress embedding, A12/A13 matched comparisons, dashboard schedule inspection, and reproducibility manifests.

EmbodimentStateRecord	Run/dataset/subject/session/trial/ identity, simulator branch and version, mapping ID, stress profile ID, stress schedule ID where active, source IHARQ action as the preserved Layer 4 token, policy action and deferral status where enabled, regime/risk context, Layer 9 safety-gate state, mapped command, command block reason or adapter status, simulated state reference, target reference, endpoint error, command success, over/under-assist flags, safety event, reward terms, ablation and matched-comparison IDs, metric-dictionary/negative- result/claim-evidence pointers, seed, config hash, and record version. It must not store unsupported mapping, simulator error, hold/no-op, or unsafe-stop outcomes as if they were Layer 4 action tokens.	Produced by Layer 9; consumed by Layers 6 and 7 only under declared simulation-only policy-analysis modes and by Layer 10 for embodiment pages, cards, safety-event views, export indexes, and final demonstration evidence.
EmbodimentOutcomeRecord	Endpoint-error, command success/failure, over/under-assist, smoothness proxy, stress sensitivity, blocked-command, fallback/no-action, wrong-accept-to-action flag for evaluation-only analysis, delay cost, safety-event pointer, transition pointer, command-timeline pointer, endpoint-curve pointer, optional screenshot or reward-trace linkage, accepted-missingness reason, comparison key, A13 outcome-summary provenance, diagnostic-only flag, reward-feedback flag where enabled, and non-clinical claim boundary.	Produced by Layer 9; consumed by Layers 6 and 7 only under declared simulation-only policy-analysis modes, and by Layer 10 for embodiment pages, cards, export indexes, failure taxonomy, A13 tables, claim-evidence views, and final demonstration evidence.

SafetyEventRecord	Safety-gate event ID, source policy/IHARQ/regime/stress/embodiment IDs, attempted command, blocked or permitted command status, unsafe/fallback reason, response taken, safety-gate version, timestamp or simulated step, severity, and warning text.	Produced by Layer 7 when the closed-loop environment materializes unsafe, blocked, fallback, no-action, or simulated safety events, and by Layer 9 where embodiment safety events are materialized; consumed by Layers 0, 7, 9, and 10, failure taxonomy, safety-event dashboards, and claim-safety cards.
Layer9RewardTrace	Layer 9-specific simulation-only reward-decomposition trace with reward version, reward terms, source transition and embodiment IDs, action, state/next-state linkage, seed, config hash, reward-feedback enabled status, reward-hacking warning status, and clinical-reward prohibition tag.	Produced by Layer 9 when reward-enabled embodiment is configured; consumed by Layer 10 reward-trace filters, embodiment cards, A13 evaluation, and reproducibility manifests. Layer 7 policy-simulation reward histories remain represented by RewardTraceRecord , not by this Layer 9 artifact.
Layer9EmbodimentDemonstration	Embodiment run ID, embodiment mode, adapter branch, simulator/static-replay version, command-mapping ID, config hashes, seed, source-record paths, command timeline, endpoint-error curve, safety logs, reward-trace pointer where enabled, screenshots/video references where generated, dashboard-compatible export bundle path, warning set, accepted-missingness markers, and file hashes.	Produced by Layer 9; consumed by Layer 10 embodiment exports, reproducibility package, A13 table, failure taxonomy, card/export page, and final demonstration checklist.
StressCard	Stress purpose, stress family, intensity, schedule, injection point, affected modules, expected and observed failure signatures, stress-conditioned metrics, sensitivity curves, comparison tables, claim boundary, reproducibility paths, and dashboard export lineage.	Produced by Layer 8 directly or rendered by Layer 10 from Layer 8 records; consumed by StressForge dashboard pages, thesis robustness sections, reproducibility manifests, and claim-safety review.

EmbodimentCard	Layer 9 embodiment evidence summary with claim boundary, dataset/provenance, embodiment mode, simulator branch, mapping, stress profile and schedule where active, metrics, figures, failures, safety events, negative-result notes, reproducibility command, limitations, claim-evidence ID, and file hashes.	Produced by Layer 9 directly or rendered by Layer 10 from Layer 9 records; consumed by embodiment dashboard pages, thesis appendix, examiner review, reproducibility package, and claim-safety review.
ControllerCard	Policy identity, input record set, action space, legality rules, target or reward route, training protocol, baselines, global/local ablation IDs, matched-comparison key, metrics, negative-result notes, limitations, failure cases, lineage, status tier, and claim boundary.	Produced by Layer 6 and rendered/exported by Layer 10; consumed by the thesis appendix, dashboard controller-card page, claim-safety review, and reproducibility package.
ClosedLoopEnvironmentCard	Layer 7 environment assumptions, state visibility, action space, reward configuration, stress profile/schedule, seed, schema version, update/frozen mode, supported and unsupported claims, negative-result notes where relevant, and limitations.	Produced by Layer 7 or Layer 10 from Layer 7 records; consumed by dashboard closed-loop pages, thesis appendix, and reproducibility package.
PolicyRolloutCard	Policy rollout identity, source ClosedLoopSessionRecord, PolicyRolloutRecord, TransitionRecord, RewardTraceRecord, optional PolicyUpdateTrace, safety-event links, stress/embodiment mode, matched-comparison key, metric-dictionary ID, and claim boundary.	Produced by Layer 7 or Layer 10 from Layer 7 records; consumed by dashboard policy-comparison views, ablation reports, thesis figures, and reproducibility package.

DashboardMetricRecord	Dashboard-ready metric ID, view name, metric name, metric level, metric value, units, time axis, dataset/subject/session/model/policy/stress/embodiment filters, optional policy-update, embodiment, and reward-trace filters, source records, source layers, ablation step, matched-comparison ID, metric-dictionary reference, diagnostic-only status, negative-result note reference, claim-evidence ID, configuration ID, run ID, warning flags, card references, export references, and source-lineage completeness status.	Produced by Layer 10 from validated records and evaluation artifacts; consumed by dashboard pages, DashboardViewRecords , card generator, export manager, artifact manifest, thesis tables, final demonstration evidence, and reproducibility manifests.
DashboardViewRecord	Page/view ID, page role, panel IDs, filter state, source DashboardMetricRecord IDs, source-record IDs, evaluation-lineage references, warning banners, completeness state, diagnostic-only state, export references, thesis usage location, generation config, code version, and generation timestamp.	Produced by Layer 10; consumed by dashboard rendering, static dashboard exports, cards/export page, reproducibility manifests, claim-safety review, and final thesis artifact inventory.
CardRecord	Card ID, card type, card version, source records, source layers, configuration ID, run ID, main metrics, figures or tables, interpretation text, supported claim or limitation, warning flags, artifact paths, reproduction command, thesis usage location, completeness status, and generation timestamp.	Produced by Layer 10 card generator or source-layer card exporters under the Layer 10 card contract; consumed by dashboard cards/export page, static card exports, reproducibility manifests, claim-safety review, thesis chapters, appendices, repository release, and final thesis artifact inventory.

ReproducibilityManifest	Project/version identity, run IDs, configuration IDs, code and environment versions, seeds, source-record paths, card-record paths, figure/table paths, dashboard-view paths, artifact checksums where feasible, warning-rule version, claim-ledger version, known limitations, and reproduction steps.	Produced by Layer 10 manifest builder from saved records, export logs, artifact manifests, and repository metadata; consumed by future reproducers, examiner review, repository release, thesis reproducibility appendix, final demonstration evidence, and package-index checks.
ExperimentSummaryRecord	Aggregate experiment identity, evaluated layer or slice, ablation step, matched-comparison group, metric-dictionary references, source-record IDs, diagnostic-only status, negative-result references, and claim-evidence linkage.	Produced by evaluation scripts; consumed by Layer 10 ablation pages, aggregate metric views, matched-cost/coverage/stress/embodiment summaries, contribution summaries, and thesis tables.
FailureTaxonomy	Failure category, source layer, source record, detection rule, representative examples, frequency or severity, affected claims, mitigation or limitation text, and diagnostic-only status.	Produced by Layer 10 from validated failure audits, safety events, high-confidence wrong cases, stress failures, and embodiment failures; consumed by thesis discussion, limitations, claim-safety review, and dashboard failure-taxonomy pages.
ReproducibilityPackage	Export package index containing required configs, manifests, seeds, code version, environment, source records, figures, tables, cards, checksums, run registry entries, and rerun commands.	Produced by Layer 10 export and reproducibility modules; consumed by repository release, artifact review, future reruns, examiner inspection, and thesis appendix.
MetricDictionaryEntry	Metric name, definition, directionality, units, required fields, aggregation unit, split role, local and global ablation step, matched-comparison requirement, diagnostic-only downgrade rule, and claim boundary, including stress and embodiment metric warnings where applicable.	Produced by evaluation/configuration scripts; consumed by Layers 0, 8, 9, and 10, the Ablation and Evaluation Architecture, the Failure Modes and Defensive Design section, and the Final Demonstration section to prevent ambiguous metric claims.

MatchedComparisonReport	Matched source-record IDs, comparison keys, ablation step, metric dictionary references, coverage/cost/stress/embodiment matching status, command-mapping and simulator-configuration matching where A13 is evaluated, missing-key warnings, and diagnostic-only downgrade status.	Produced by evaluation scripts and Layer 10 exports; consumed by Layer 0, Layer 10, the Ablation and Evaluation Architecture, the Failure Modes and Defensive Design section, and the Final Demonstration section for thesis-ready comparison claims.
ClaimEvidenceMatrix	Claim-to-artifact mapping, evidence-mode label, ablation-step linkage, metric dictionary references, dashboard/card/export lineage, warning inheritance, negative-result note references, and allowed/prohibited wording.	Produced by Layer 0 and final reporting scripts; consumed by Layer 10, the Implementation Pipeline section, the Failure Modes and Defensive Design section, the Final Demonstration section, final thesis writing, and reproducibility audits.
NegativeResultNote	Bounded interpretation of calibration failure, threshold sufficiency, selective-prediction sufficiency, missing matchability, weaker-than-baseline results, stress-sensitive failures, adapter-failed embodiment runs, reward-dominated findings, or diagnostic-only A12/A13 outcomes, with affected claim and fallback status.	Produced by evaluation scripts, Layer 3, Layer 8, Layer 9, Layer 10, the Failure Modes and Defensive Design section, or the Final Demonstration section; consumed by final reporting, dashboard limitation views, claim-evidence review, and thesis limitation sections.

5.2 Record design principles

Every record should include at least the following fields where applicable: dataset ID, subject ID, session ID if available, trial ID, window ID when applicable, split ID, random seed, configuration hash, model ID, stress profile ID, stress schedule ID, and timestamp or simulated time index. Evaluation and reporting artifacts should additionally include ablation-step ID, matched-comparison ID where relevant, metric-dictionary ID, operating-point ID where relevant, diagnostic-only status, negative-result note ID where relevant, claim-evidence ID, source-record list, dashboard/card/export lineage, and warning flags. This prevents later confusion when the dashboard, cards, ablations, and paper figures need to connect results across modules.

The architecture should save records to simple, readable formats. Recommended formats are CSV for small summary tables, Parquet for large per-window logs, JSON or YAML for configurations, and PNG/PDF for figures. The file format is less important than the rule that every module output must be inspectable and reproducible.

6 Layer 0: Claim-Safety and Scope Governance

6.1 Problem solved by Layer 0

Layer 0 prevents the system from overclaiming. The project is ambitious because it connects public EEG decoding, readiness decisions, temporal risk, policy learning, stress simulation, musculoskeletal embodiment, and dashboarding. Without claim governance, the final thesis could accidentally imply clinical validation, patient safety, or real BCI-FES control. Layer 0 exists to stop that.

Layer 0 also governs the claim-safety slice of the cross-cutting ablation and evaluation architecture. A result is not claim-safe merely because it appears in an ablation table, metric export, card, dashboard page, repository artifact, final demonstration matrix, or thesis figure. Each evaluation artifact must declare its evidence mode, ablation step where applicable, source-record lineage, allowed claim class, prohibited claim class, diagnostic-or-confirmatory status, negative-result linkage where applicable, and limitation warning before it can support thesis wording. This prevents the ablation system from turning public EEG, proxy readiness labels, stress simulations, policy rollouts, embodiment demos, dashboard exports, or final Stretch C capability summaries into unsupported clinical or deployment claims.

6.2 Inputs

- research question, thesis scope, implementation stage, experiment logs, and figures;
- evaluation-mode labels, ablation-step IDs, metric dictionaries, matched-comparison reports, dashboard/card/export lineage, negative-result notes, and claim-evidence matrix entries;
- dataset limitations, including public EEG labels and absence of true clinical readiness labels;
- Layers 0–3 evaluation-slice controls, including A0 raw-decoder evidence, A1–A3 calibration/uncertainty/selective-prediction evidence, A4 ordinary-combination or ordinary-control evidence, matched operating-point status, and ablation-readiness warnings;
- final demonstration and implementation-readiness status inputs, including core, target, stretch, optional, diagnostic, future-work, non-thesis-ready, invalid-due-to-leakage, invalid-due-to-claim-violation, and C0–C4 capability-tier labels;
- simulation limitations, including synthetic fatigue/drift/stress and proxy embodiment;
- policy-learning limitations, including supervised Adaptive-IHARQ, learning-to-defer, contextual-bandit, and reinforcement-learning reward-design dependence and simulation-only training;
- Layer 9 MyoSuite/OpenSim/static-replay embodiment-demo limitations, including no rehabilitation-efficacy validation, no FES or exoskeleton safety validation, no patient-specific biomechanical validation, no real-world actuation safety claim, and no medical-device readiness claim.

6.3 Submodules

Submodule	Function	Output
Claim boundary checker	Checks whether a statement is supported by public-data protocol evidence, offline EEG decoding, calibration/uncertainty, selective prediction, IHARQ evidence verification, temporal-trust monitoring, policy evaluation, simulated closed loop, StressForge, MyoSuite, OpenSim, dashboard aggregation, repository artifact lineage, or final demonstration evidence. It rejects claim wording that is stronger than the weakest required evidence source.	Approved, weakened, rejected, diagnostic-only, optional-only, non-thesis-ready, invalid-due-to-leakage, or invalid-due-to-claim-violation claim wording.
Evidence provenance tracker	Records whether a result came from offline EEG, calibration/uncertainty analysis, selective prediction, IHARQ evidence verification, temporal-trust monitoring, policy evaluation, simulated closed loop, StressForge, MyoSuite, OpenSim, dashboard aggregation, repository packaging, or final demonstration synthesis. It also records ablation-step ID, metric source, source-record pointer, matched-comparison key, MetricDictionaryEntry reference, NegativeResultNote pointer where applicable, figure/table/card/export lineage, seed, and configuration hash where applicable.	Provenance tags for all major figures, tables, cards, dashboard exports, ablation summaries, metric dictionary exports, claim-evidence matrix entries, and final demonstration matrices.

Evaluation claim-evidence gate	Links each thesis claim to its evaluation mode, required records, ablation comparison, metric dictionary entry, matched-comparison report where relevant, figure/table/card/export lineage, <code>ClaimEvidenceMatrix</code> row, <code>NegativeResultNote</code> if applicable, and limitation statement. It blocks unsupported clinical, deployment, real-time-control, real patient-safety, real rehabilitation-efficacy, real-FES/exoskeleton-safety, or medical-device-readiness wording even when a numerical metric improves.	Claim-evidence matrix entries, invalid-claim flags, diagnostic-only labels, warning-inheritance rules, negative-result linkage, and thesis-safe claim templates.
Scope, tier, and status labeler	Marks results and demonstrations with the locally allowed status vocabulary: core, target, stretch, optional, diagnostic, future work, core-ready, target-ready, stretch-ready, diagnostic-only, optional-only, non-thesis-ready, invalid due to leakage, invalid due to claim violation, and C0–C4 capability tier where the final demonstration contract uses tiered readiness.	Stage, readiness-status, and capability-tier labels in tables, dashboard views, cards, repository manifests, and final demonstration summaries.
Clinical non-claim guard	Blocks language that implies clinical deployment, patient safety, therapeutic efficacy, real FES/exoskeleton safety, patient-specific biomechanics, or medical-device readiness.	Thesis-safe wording templates and non-clinical limitation text.

Policy-learning safety guard	Marks supervised Adaptive-IHARQ, learning-to-defer, contextual-bandit, and RL outputs as public-data/proxy-label or simulation-only policy-learning results.	Policy safety notes, controller-card warnings, reward/cost limitation text, and dashboard/export warnings for policy-learning views.
Embodiment safety guard	Marks Layer 9 MyoSuite/OpenSim/static-replay outputs as simulated proof of concept only, not rehabilitation therapy, real-body control, patient-specific biomechanics, FES/exoskeleton safety, or medical-device readiness.	Embodiment-card warnings, non-clinical limitation text, blocked-command claim notes, and dashboard/export warnings for Layer 9 and Layer 10.

6.4 Outputs

Layer 0 produces a claim ledger, limitation statements, allowed and prohibited claim classes, scope labels, readiness-status labels, C0–C4 capability-tier labels where applicable, provenance tags, **ClaimEvidenceMatrix** entries, invalid-claim flags, diagnostic-only and optional-only labels, non-thesis-ready and invalid-result labels, warning-inheritance rules, **NegativeResultNote** references where applicable, and required wording for the thesis, dashboard, cards, ablation tables, matched-comparison tables, metric exports, repository manifests, final demonstration matrices, and paper figures. For Layers 1–3 evidence, the ledger distinguishes public-data protocol evidence, raw decoder baseline evidence, ordinary-control evidence, calibrated-reliability evidence, uncertainty evidence, selective-prediction baseline evidence, matched operating-point evidence, and ablation-readiness evidence; none of these may be described as clinical readiness or real action safety. For Layer 6 and Layer 7 policy evidence, the ledger must distinguish supervised/proxy-label evidence, learning-to-defer evidence, contextual-bandit evidence, reinforcement-learning evidence, and simulated closed-loop evidence without implying real deployment safety. For Layer 8 stress evidence, the ledger must distinguish bounded perturbation testing from real-world robustness validation. For Layer 9 and Layer 10 embodiment evidence, the ledger must explicitly mark MyoSuite/OpenSim/static-replay outputs as simulation-only evidence and must block language that implies rehabilitation efficacy, real-device control, patient safety, or clinical deployment.

6.5 Outgoing links

Layer 0 links to every layer because every output must be scoped correctly. It especially controls Layer 6 policy-learning claims, Layer 8 stress-robustness claims, Layer 9 embodiment claims, and Layer 10 dashboard/export claims. It also constrains the Ablation and Evaluation Architecture section by requiring every A0–A13 result, metric dictionary entry, matched-comparison table, dashboard view, card, repository manifest, final demonstration matrix, and thesis figure to inherit the correct evidence mode, status label, source-record lineage, and limitation warning. For the Layers 0–3 evaluation slice, Layer 0 ensures that public-data protocol quality, raw

decoder baselines, ordinary-control comparisons, calibration, uncertainty, selective-prediction, matched operating points, and ablation-readiness results remain framed as reliability evidence, not as clinical BCI-FES validation.

Layer 0 also feeds the Implementation Pipeline, Recommended Repository Architecture, Failure Modes and Defensive Design, What the Final Stretch C System Should Demonstrate, and Final Architecture Summary sections. Those sections may consume Layer 0 claim-safety configuration, `ClaimEvidenceMatrix` rows, warning-inheritance rules, negative-result links, and status labels, but they must not strengthen a claim beyond the evidence class declared by Layer 0.

7 Layer 1: Public-Data and Protocol Anchor

7.1 Problem solved by Layer 1

Layer 1 makes the project feasible, reproducible, and academically defensible. Since real clinical BCI-FES data and patient-specific stimulation systems are not available, the architecture must rely on public EEG datasets and open protocols. The purpose is not to claim that public EEG is identical to clinical rehabilitation. The purpose is to create a stable public-data anchor for studying the reliability bottleneck under controlled, inspectable, and reproducible conditions.

The bottleneck solved by Layer 1 is the **data-protocol bottleneck**: every downstream reliability, uncertainty, evidence-verification, RegimeRisk, adaptive-policy, stress, replay, embodiment, dashboard, and thesis-reporting claim depends on whether the experimental data foundation is stable, leakage-free, versioned, and traceable. If the data foundation is weak, later improvements may be criticized as artifacts of arbitrary preprocessing, hidden splits, window leakage, undocumented label mappings, subject/session confusion, or unfair low-calibration protocols. If Layer 1 is strong, every downstream result can be traced back to a public dataset, original meta-data, normalized metadata, label map, preprocessing version, split ID, low-calibration budget, window lineage, configuration hash, validation report, and public-data claim boundary.

Layer 1 is therefore also the public-data and protocol slice of the evaluation architecture. It does not run the system-level ablation ladder, but it determines whether the ladder is fair: A0 raw decoder results, A1 calibration results, A2/A3 threshold and selective-prediction results, and all later comparisons are valid only if they share the declared dataset, subject/session/trial/window lineage, split identity, preprocessing identity, label-map identity, low-calibration budget, seed policy, and protocol version. If those matching keys are absent or inconsistent, later ablation tables may be useful diagnostics but cannot support thesis-ready claims.

Layer 1 therefore answers the thesis-level foundation question:

Given the thesis constraint that only public EEG data and offline/pseudo-online simulation are available, how can heterogeneous public motor-imagery EEG datasets be transformed into immutable, validated, low-calibration-ready, subject/session-aware records that all downstream reliability and readiness layers can consume without hidden assumptions?

This layer protects the thesis against several predictable criticisms: public-data overclaiming, clinical-safety overreach, arbitrary dataset inclusion, unclear task labels, hidden preprocessing, train/test leakage, window-overlap leakage, inconsistent low-calibration budgets, unfair decoder comparison, unreproducible splits, and dashboard-level recomputation that silently changes the protocol. If Layer 1 were removed, the project would lose its empirical contract, its public-data

claim boundary, its low-calibration protocol foundation, its dataset/protocol cards, and the traceability required for thesis-grade reproducibility.

7.2 Layer 1 mission, thesis role, and contribution boundary

Layer 1 establishes the reproducible public-data and protocol foundation of IHARQ BenchGuard by transforming heterogeneous public EEG datasets into standardized, validated, versioned, low-calibration-ready, subject/session-aware records that can be consumed by downstream reliability, uncertainty, evidence-verification, regime-monitoring, policy-learning, simulation, stress-testing, embodiment, dashboard, and reporting modules.

Identity	Meaning inside the architecture
Engineering identity	Loads public EEG datasets through reproducible adapters; normalizes subject/session/run/trial/channel metadata; records preprocessing, label mapping, artifact flags, split logic, and windowing; validates leakage controls; and publishes versioned records and manifests.
Methodological identity	Defines the public-data protocol before any decoder, uncertainty module, IHARQ module, policy learner, stress evaluator, or dashboard view is executed. The layer freezes data inclusion, split, preprocessing, label, and window contracts so later modules cannot silently tune the data foundation.
Thesis-evidence identity	Provides the empirical substrate that allows the thesis to study low-calibration reliability, uncertainty, evidence sufficiency, temporal trust, readiness decisions, stress robustness, simulated consequences, and reproducibility under public-data constraints.
Claim-safety identity	Makes public EEG honest: it supports reproducible study of reliability bottlenecks and proxy intent labels, but it does not establish clinical rehabilitation efficacy, patient-safe FES control, or deployment readiness.

Claim-safety rule: Layer 1 may claim that the thesis has a reproducible, public-data, protocol-governed experimental anchor. It must not claim that public EEG validates patient rehabilitation, real stimulation safety, clinical deployment, or real closed-loop BCI-FES efficacy. All public EEG labels used later for simulated command mapping remain proxy intent labels unless a future clinical protocol is separately approved.

7.3 Scope, non-scope, and boundary rule

Layer 1 owns the data and protocol contract. It does not own decoder performance, probability calibration, evidence verification, temporal trust scoring, adaptive readiness decisions, simulated embodiment, or clinical interpretation.

In scope	Layer 1 responsibility
----------	------------------------

Public EEG dataset selection	Maintain a dataset registry with included, optional, stretch, future, and excluded datasets; record inclusion reasons, exclusion reasons, source notes, task type, subject/session structure, channels, sampling rate, label set, and claim boundary.
Dataset loading	Load selected datasets through reproducible adapters and create initial dataset-level, subject-level, session-level, run-level, and trial-level records without silently preprocessing the signal.
Metadata normalization	Map heterogeneous dataset fields into a common schema while preserving original dataset-specific metadata.
Task and label ontology	Map original labels to normalized labels such as motor-imagery left hand, right hand, feet, rest, or excluded labels; record whether labels are binary, multiclass, rest-inclusive, rest-exclusive, or proxy-intent only.
Preprocessing registry	Record filtering, resampling, channel selection, referencing, epoching, normalization, artifact handling, and windowing choices as versioned configurations and logs.
Artifact and channel-quality annotation	Mark missing channels, bad channels, suspected artifacts, short trials, sampling-rate mismatches, missing labels, and window-overlap risks without hiding them from downstream modules.
Split and low-calibration protocols	Generate immutable within-subject, cross-session, cross-subject, train/validation/test, calibration, held-out test, and low-label-budget split manifests.
Trial and window generation	Convert trials into traceable downstream units while preserving parent-trial, run, session, subject, dataset, split, label, artifact, preprocessing, and configuration lineage.
Validation and leakage auditing	Check schema validity, split disjointness, subject/session separation, parent-trial grouping, duplicate windows, normalization fitting scope, label-budget integrity, metadata completeness, and repeated-generation reproducibility.
Dataset and protocol cards	Generate thesis-facing and dashboard-facing dataset cards and protocol cards that state origin, usage, limitations, allowed claims, forbidden claims, supported splits, and known risks.
Manifest and provenance	Build a Layer 1 manifest that indexes records, cards, logs, configs, code version, seeds, hashes, timestamps, validation status, and known limitations.
Evaluation matching keys and ablation readiness	Preserve the dataset, subject, session, run, trial, window, split, preprocessing, label-map, budget, seed, and protocol-version identifiers that later matched-comparison validators use for A0–A13 ablations. Layer 1 records whether these keys are complete and whether any comparison must be downgraded to diagnostic status because the protocol foundation is incomplete.

Out of scope	Owned elsewhere
Decoder training and model selection	Layer 2. Layer 1 produces records and split contracts, not trained models or performance-ranked baselines.
Probability calibration and uncertainty interpretation	Layer 3. Layer 1 may define validation partitions but does not compute ECE, Brier, entropy, margin, or selective prediction behavior.
IHARQ evidence verification and accept/wait/request/combine logic	Layer 4. Layer 1 provides window lineage and protocol identity; it does not decide evidence sufficiency.
RegimeRisk and temporal trust scoring	Layer 5. Layer 1 preserves subject/session/window sequences but does not score regimes.
Adaptive policy learning, contextual bandit, and RL readiness policies	Layer 6. Layer 1 does not optimize deferral or readiness actions.
Closed-loop replay, reward, and consequence modeling	Layer 7. Layer 1 provides replayable windows but does not define reward functions or simulated consequence models.
Stress generation and stress-conditioned evaluation	Layer 8. Layer 1 may expose stress targets such as channels or windows, but it does not synthesize stress profiles.
MyoSuite/OpenSim command mapping	Layer 9. Layer 1 provides proxy intent labels and label-map warnings; embodiment logic belongs downstream.
Dashboard interaction and recomputation	Layer 10. Layer 1 generates cards and manifests; dashboards must read saved records rather than silently rebuilding protocols.
Metric computation, statistical summaries, and ablation ranking	The Ablation and Evaluation Architecture section and Layer 10. Layer 1 supplies stable records and matching keys; it does not decide whether a decoder, threshold, calibration method, or readiness policy is superior.
Clinical safety validation	Layer 0 governance and future clinical work. Public EEG is not a substitute for patient-specific clinical validation.

Boundary rule. Layer 1 may define, validate, version, and publish data-protocol records. It must not use downstream model performance to change dataset inclusion, preprocessing choices, split definitions, label mapping, or window-generation rules unless that change is explicitly declared as a new Layer 1 protocol version and all dependent layers are rerun.

7.4 Design principles

Principle	Meaning	Architectural implication
-----------	---------	---------------------------

Record-first design	Layer 1 is not merely a collection of scripts; it is a record-producing contract layer.	The primary outputs are <code>DatasetRecord</code> , <code>WindowRecord</code> , <code>SplitRecord</code> , <code>PreprocessingRecord</code> , <code>LabelMapRecord</code> , <code>ArtifactFlagRecord</code> , <code>DatasetCard</code> , <code>ProtocolCard</code> , <code>ValidationReport</code> , <code>Layer1Manifest</code> , and <code>layer1_ablation_readiness_l1_v1.json</code> .
Protocol before model	Decoder training must not begin before data protocols are frozen.	Dataset registry, label map, preprocessing config, split manifest, window schema, and validation report are generated before Layer 2 starts.
Public-data honesty	Public EEG is a feasible empirical anchor, not clinical validation.	Dataset and protocol cards include public-data limitation statements, proxy-intent notes, and forbidden-claim warnings.
Heterogeneity preservation	Standardization must not erase dataset differences.	Original labels, original subject/session/run IDs, original channel names, sampling rates, and source notes are stored alongside normalized fields.
Split immutability	Splits are artifacts, not hidden random decisions inside model code.	Any split change creates a new split ID and protocol version.
Leakage prevention by construction	Leakage should be prevented before training, not discovered after results are produced.	Subject/session/trial/window disjointness, train-only fitting, budget integrity, and duplicate checks are part of Layer 1 validation.
Configuration hashing	Every output must be tied to exact configuration.	Dataset selection, label map, preprocessing, split rules, windowing, seed, and protocol version are included in a config hash.
Downstream neutrality	Layer 1 must not favor CSP, Riemannian, deep, compact SSL, IHARQ, policy learning, or dashboards.	All downstream consumers read shared records through stable interfaces.
Evaluation-matchability	Later ablations are fair only when variants can be matched on the same public-data and protocol keys.	Dataset, subject, session, run, trial, window, split, preprocessing, label-map, low-calibration budget, seed, and protocol-version identifiers are treated as mandatory comparison keys rather than incidental metadata.

Dashboard readiness	Data-governance artifacts should be inspectable from the start.	Dataset cards, protocol cards, class distributions, split summaries, metadata completeness, validation reports, and ablation-readiness warnings are exported for Layer 10.
---------------------	---	--

7.5 Input contract

Layer 1 receives raw or semi-raw public-data information, configuration files, and Layer 0 claim-safety constraints. Inputs are separated by role so that downstream modules can distinguish primary data from metadata, protocol choices, evaluation-only fields, and stretch artifacts.

7.5.1 Primary and metadata inputs

Input	Source	Required	Layer 1 handling
Public EEG dataset files or APIs	Public dataset repositories, MOABB-compatible loaders, or documented local mirrors	Yes	Loaded through dataset adapters; source/version/license notes are registered; no silent preprocessing is allowed inside the loader.
Subject metadata	Dataset metadata	Yes when available	Normalized into subject IDs while preserving original subject identifiers and missingness notes.
Session and run metadata	Dataset metadata	Strongly recommended	Used for cross-session protocols, temporal grouping, and leakage checks; missingness is reported rather than hidden.
Trial metadata and timing	Event tables or trial annotations	Yes for windowing	Mapped to trial IDs, cue/onset/end timing, task labels, and source pointers.
Channel and montage metadata	Dataset headers or documentation	Yes	Original channel names, selected channels, excluded channels, channel masks, and montage notes are recorded.
Sampling rate metadata	Dataset headers	Yes	Original and target rates are stored; mismatches produce flags or validation warnings.
Original task labels	Dataset events/annotations	Yes	Mapped through <code>LabelMapRecord</code> ; original labels are preserved.
Artifact notes if available	Dataset flags or quality annotations	Optional	Converted into <code>ArtifactFlagRecord</code> entries without forcing a universal rejection policy.

7.5.2 Configuration and protocol inputs

Input	Required	Meaning
<code>datasets.yaml</code>	Yes	Dataset inclusion status, source pointer, task family, subject/session policy, license/citation note, and claim boundary.
<code>labels.yaml</code>	Yes	Original-to-normalized label mappings, binary/multiclass/rest policy, excluded labels, and proxy-intent notes.
<code>preprocessing.yaml</code>	Yes	Filtering, resampling, referencing, channel selection, epoching, normalization, artifact policy, and train-only fitting rules.
<code>splits.yaml</code>	Yes	Within-subject, cross-session, cross-subject, train/validation/test, calibration, held-out test, and low-label-budget protocols.
<code>windows.yaml</code>	Yes	Window duration, stride, overlap policy, evidence/stability/regime/replay window tags, and parent-trial lineage rules.
<code>validation.yaml</code>	Yes	Schema checks, split checks, leakage checks, missingness thresholds, duplicate checks, repeated-generation checks, and completeness checks for the comparison keys required by matched ablation evaluation.
Seed configuration	Yes	Deterministic split and sampling behavior for reproducibility.
Protocol version	Yes	Version label for the entire Layer 1 protocol state.

7.5.3 Constraint inputs from Layer 0

Layer 0 constraint	Layer 1 response
Public EEG is not clinical validation.	Dataset cards and protocol cards explicitly state that public-data experiments support reliability-method evaluation, not clinical rehabilitation efficacy.
Offline and pseudo-online evidence must be labeled honestly.	Records include scope tags such as public-data, proxy-intent, offline, pseudo-online, simulation-only, and not-clinical-validation.
Simulation and embodiment claims are bounded.	Labels used later for Layer 7 or Layer 9 are marked as proxy intent labels, not real stimulation commands.
Thesis claims require provenance.	Every record carries dataset, split, preprocessing, label-map, seed, protocol-version, and config-hash provenance where applicable.
Evaluation claims require matched source records.	Layer 1 exposes the identifiers needed for matched ablation comparisons and marks missing or inconsistent keys so later evaluation can downgrade the result to diagnostic-only rather than silently treating it as confirmatory.

Warnings must be visible. Dataset/protocol cards and Layer 10 exports receive limitation text, ablation-readiness warnings, and invalid-claim warning fields.

7.6 Output contract and downstream consumers

Layer 1 outputs are contracts for the rest of the architecture. Once any output is consumed by Layer 2 or later, it must not be silently changed. A change to dataset selection, preprocessing, split logic, label mapping, or windowing creates a new protocol version and a new manifest.

Output	Primary consumer	Purpose
DatasetRecord	Layers 2–10	Defines standardized dataset, subject, session, run, task, channel, sampling-rate, split, preprocessing, license/source, and claim-scope metadata.
WindowRecord	Layers 2–10	Defines the downstream signal segment with trial/window identity, parent-trial lineage, timing, segment pointer, label, split membership, label-budget ID, artifact flags, channel mask, preprocessing ID, and config hash.
SplitRecord	Layers 2–6, 8–10	Defines train/validation/test/calibration/adaptation membership, split level, label-budget/low-calibration budget, seed, stratification, and leakage-check status.
PreprocessingRecord	Layers 2, 3, 8–10	Defines filtering, resampling, channel selection, referencing, epoching, normalization fitting scope, artifact policy, implementation script, preprocessing version, and preprocessing config hash.
LabelMapRecord	Layers 0, 2, 7, 9, 10	Defines original label, normalized label, task family, inclusion status, exclusion reason, rest/no-action policy, label-map version, and proxy-intent limitation.
ArtifactFlagRecord	Layers 2, 4, 5, 8–10	Defines missing channels, suspected artifacts, bad channels, short trials, sampling-rate mismatches, label problems, and window-overlap risk.
DatasetCard	Layer 10 and thesis chapters	Explains dataset origin, tasks, subjects, sessions, recording details, labels, inclusion reason, limitations, allowed claims, forbidden claims, and known risks.
ProtocolCard	Layer 10 and thesis chapters	Explains split logic, low-calibration budgets, preprocessing ID, windowing ID, leakage controls, downstream consumers, and protocol limitations.

ValidationReport	All layers	Confirms schema validity, metadata completeness, distribution summaries, split disjointness, leakage checks, and reproducibility status.
Layer1Manifest	All layers	Master index linking records, cards, configs, hashes, seeds, code version, output paths, timestamp, validation status, limitations, and ablation-readiness status for the public-data/protocol foundation.
layer1_ablation_readiness_validation_report	Final thesis, Layer 10, and thesis evaluation chapters	Public-data/protocol ablation-readiness artifact containing matched-comparison key completeness, missing-key warnings, diagnostic-only downgrade reasons, validation-report links, and protocol-version provenance.

7.7 Internal submodule architecture

The original six Layer 1 responsibilities—dataset loading, metadata normalization, preprocessing registry, low-calibration splitting, dataset-card generation, and trial/window generation—are preserved and expanded into eleven implementation-ready submodules. The first seven are core for the thesis; the remaining four are supporting but should be implemented for thesis-grade reproducibility.

Submodule	Status	Engineering responsibility	Primary outputs
Dataset registry manager	Core	Declares which datasets are included, excluded, optional, stretch, or future and records claim boundaries.	Dataset registry table, inclusion/exclusion report.
Dataset loader	Core	Loads public EEG datasets through reproducible adapters and creates initial source records.	Raw access log, initial DatasetRecord .
Metadata normalizer	Core	Standardizes subject/session/run/trial/channel/sampling rate metadata without deleting original fields.	Unified metadata table, metadata sampling report.
Task and label ontology mapper	Core	Maps original labels to normalized motor-imagery/rest/excluded labels and proxy-intent notes.	LabelMapRecord , label ontology table.
Preprocessing registry	Core	Records every signal transformation and fitting scope as a versioned protocol.	PreprocessingRecord , preprocessing config, preprocessing log.

Artifact and channel-quality annotator	Supporting	Flags missing channels, bad channels, suspected artifacts, short trials, mismatches, and overlap risks.	ArtifactFlagRecord , channel-mask table.
Split and protocol manager	Core	Creates immutable train/validation/test, within-subject, cross-session, cross-subject, and low-calibration splits.	SplitRecord , split manifests, budget tables.
Trial/window generator	Core	Converts trials into traceable windows for decoder, temporal reliability, stress, and replay modules.	WindowRecord , window lineage map.
Data validation and leakage auditor	Core	Checks schemas, split disjointness, duplicate records, metadata completeness, train-only fitting, leakage, and completeness of the comparison keys needed for matched evaluation.	ValidationReport , leakage audit report, ablation-readiness warnings, and layer1_ablation_readiness_l1_v1.json
Dataset-card and protocol-card generator	Supporting	Generates thesis-facing and dashboard-facing documentation cards from records and manifests, including public-data limitations and evaluation-readiness notes.	DatasetCard , ProtocolCard .
Provenance and manifest builder	Supporting	Links all Layer 1 artifacts to configs, hashes, seeds, code versions, timestamps, validation status, comparison-key completeness, ablation-readiness status, and known limitations.	Layer1Manifest , layer1_ablation_readiness_l1_v1.json , reproducibility index.

7.8 Submodule dossiers

7.8.1 Dataset registry manager

Purpose and bottleneck addressed. The dataset registry manager prevents vague statements such as “we use public EEG datasets” by forcing the architecture to declare exactly which datasets are used, why they are included, and what they are allowed to support. It addresses the dataset-scope bottleneck: without a registry, downstream results could mix datasets, tasks, subjects, sessions, and claim boundaries without a defensible inclusion rationale.

Engineering responsibility. The registry stores stable dataset IDs, dataset names, sources, task types, inclusion status, subjects, sessions, runs, channels, sampling rates, labels, license/source notes, and claim boundaries. Core status is reserved for datasets required by the thesis. Secondary, optional, stretch, future, and excluded statuses are documented separately.

Inputs and outputs. Inputs are `datasets.yaml`, public dataset documentation, source/version notes, and Layer 0 claim constraints. Outputs are the dataset registry table, dataset inclusion/exclusion report, and registry fields inside `DatasetRecord` and `Layer1Manifest`.

Processing logic. The manager validates that each dataset has a stable ID, source pointer, task family, metadata availability statement, label set, subject/session summary, and claim boundary. Excluded datasets are not silently dropped; they are recorded with exclusion reasons such as unclear labels, insufficient metadata, incompatible task type, unstable access, small sample count, or license uncertainty.

Dependencies and consumers. It depends hard on Layer 0 claim-safety constraints and dataset-source availability. It has soft dependencies on MOABB-style loaders or dataset-specific documentation. Consumers are the dataset loader, metadata normalizer, card generator, Layer 2 experiment controller, Layer 10 dashboard, and thesis methodology chapter.

Evaluation, failure modes, and evidence. Evaluation checks whether every candidate dataset has a status, rationale, source note, and claim boundary. Failure modes include missing source links, ambiguous licenses, unsupported task labels, and dataset changes after protocol freeze. The thesis evidence produced is the dataset registry table and inclusion/exclusion appendix.

7.8.2 Dataset loader

Purpose and bottleneck addressed. The dataset loader makes public datasets loadable through reproducible adapters while avoiding hidden preprocessing. It addresses the loadability bottleneck: if datasets cannot be loaded deterministically, later reproducibility claims fail.

Engineering responsibility. Each dataset should be accessed through a `DatasetAdapter` with functions such as `load_metadata()`, `load_subject()`, `load_session()`, `load_trials()`, and `export_dataset_records()`. The loader may read files and annotations, but transformations such as filtering, normalization, or epoch trimming belong to the preprocessing registry.

Inputs and outputs. Inputs are registered dataset source pointers, subject/session selections, raw headers, event annotations, and loader configuration. Outputs are raw access logs, initial `DatasetRecord` entries, source-file pointers, trial pointers, and load-status reports.

Processing logic. The loader resolves source paths, checks availability, enumerates subjects/sessions/runs/trials, reads channel and sampling-rate metadata, extracts event labels, and emits initial records with original identifiers preserved.

Dependencies and consumers. Hard dependencies are the dataset registry and accessible public data. Optional dependencies include MOABB, MNE, Braindecode, or dataset-specific readers. Mock data can be used for schema development. Consumers are metadata normalization, label mapping, preprocessing registry, validation auditor, and Layer 2 smoke tests.

Evaluation, failure modes, and evidence. Evaluation uses load-pass rate, subject/session/trial count checks, source-pointer validity, and repeated-load consistency. Failure modes include dataset access changes, corrupted files, unsupported formats, missing event tables, or inconsistent channel metadata. Evidence appears in the loader report, manifest, and

dataset cards.

7.8.3 Metadata normalizer

Purpose and bottleneck addressed. The metadata normalizer converts heterogeneous public-dataset metadata into one common architecture language. It addresses the comparability bottleneck: without common fields, Layers 2–10 cannot fairly group by subject, session, run, trial, split, or label budget.

Engineering responsibility. It standardizes `dataset_id`, `subject_id`, `session_id`, `run_id`, `trial_id`, `original_label`, `normalized_label`, `channel_names`, `sampling_rate`, `trial_start`, `trial_end`, `source_pointer`, `preprocessing_id`, and `split_id`. Original fields remain stored alongside normalized fields.

Inputs and outputs. Inputs are loader records, dataset documentation, channel metadata, trial timing, and label annotations. Outputs are normalized metadata tables, metadata mapping reports, completeness reports, and enriched `DatasetRecord` entries.

Processing logic. The normalizer maps dataset-specific identifiers into stable canonical IDs, checks missingness, records imputation or absence, standardizes channel-name representation, and attaches source lineage.

Dependencies and consumers. Hard dependencies are loader output and registry definitions. Consumers are split manager, window generator, validation auditor, Layer 2, Layer 5 temporal trust analysis, Layer 7 replay, and Layer 10 dashboard filters.

Evaluation, failure modes, and evidence. Evaluation uses metadata completeness percentage, identifier uniqueness, hierarchy validation, and mapping reproducibility. Failure modes include subject/session confusion, missing trial IDs, ambiguous run structure, and channel-name inconsistencies. Thesis evidence appears as metadata completeness tables and subject/session coverage summaries.

7.8.4 Task and label ontology mapper

Purpose and bottleneck addressed. The label mapper prevents semantic drift between datasets. It addresses the task-label bottleneck: if one dataset’s “left hand”, another dataset’s event code, and another dataset’s rest condition are mixed without explicit mapping, downstream comparisons become invalid.

Engineering responsibility. It creates `LabelMapRecord` entries that map original labels to normalized labels such as `MI_LEFT_HAND`, `MI_RIGHT_HAND`, `MI_FEET`, `MI_TONGUE`, `REST`, `NO_ACTION`, or `EXCLUDED`. It also records binary versus multiclass protocols, rest-inclusive versus rest-exclusive protocols, and proxy-intent limitations for downstream simulation or embodiment.

Inputs and outputs. Inputs are original task labels, dataset documentation, `labels.yaml`, and Layer 0 claim constraints. Outputs are `LabelMapRecord`, a task ontology table, exclusion notes, and label sections in dataset/protocol cards.

Processing logic. The mapper preserves original labels, assigns normalized classes, marks labels excluded from a given protocol, records exclusion reasons, and attaches proxy-intent notes when labels may later be mapped to simulated actions.

Dependencies and consumers. It depends on dataset registry and metadata normalization. Consumers are Layer 2 decoders, Layer 3 calibration evaluation, Layer 7 replay, Layer 9 simulated command mapping, Layer 10 cards, and Layer 0 claim-safety warnings.

Evaluation, failure modes, and evidence. Evaluation checks one-to-one or justified many-to-one mapping, class counts, protocol inclusion consistency, and label coverage. Failure modes include ambiguous labels, unsupported tasks, inconsistent rest semantics, and accidental inclusion of excluded classes. Evidence appears in label ontology tables and protocol-card limitations.

7.8.5 Preprocessing registry

Purpose and bottleneck addressed. The preprocessing registry makes signal transformations explicit. It addresses the hidden-preprocessing bottleneck: results cannot be defended if filtering, resampling, epoching, channel selection, normalization, or artifact handling are buried in model code.

Engineering responsibility. The registry records filter type and band, notch behavior if used, resampling method, channel selection and exclusion, referencing scheme, epoch window, normalization method and fitting scope, artifact policy, output format, windowing connection, implementation script, preprocessing version, and config hash.

Inputs and outputs. Inputs are normalized metadata, raw segment pointers, channel metadata, `preprocessing.yaml`, and train-only fitting rules. Outputs are `PreprocessingRecord`, preprocessing logs, transformed segment pointers where applicable, and preprocessing summaries for cards and Layer 10.

Processing logic. The registry does not merely apply transformations; it records every decision. If any transformation is fitted, the fit scope must be declared: training-only, subject-training-only, calibration-only, or explicitly no fitting. Test data must never fit preprocessing parameters.

Dependencies and consumers. Hard dependencies are metadata records, split rules for fitted transformations, and configuration files. Soft dependencies include signal-processing libraries. Consumers are Layer 2 decoders, Layer 3 calibration, Layer 8 stress tests, Layer 10 reproducibility, and thesis methodology.

Evaluation, failure modes, and evidence. Evaluation checks config completeness, deterministic output, train-only fitting, and preprocessing-version consistency. Failure modes include hidden global normalization, undocumented channel removal, inconsistent epoch definitions, or preprocessing drift across datasets. Evidence appears as preprocessing tables, config hashes, and reproducibility logs.

7.8.6 Artifact and channel-quality annotator

Purpose and bottleneck addressed. This annotator exposes data-quality problems without necessarily removing them. It addresses the quality-opacity bottleneck: downstream failures may be caused by missing channels, artifacts, short trials, or sampling mismatches that should be visible in records.

Engineering responsibility. It creates flags such as `MISSING_CHANNEL`, `BAD_CHANNEL`, `ARTIFACT_SUSPECTED`, `SHORT_TRIAL`, `LABEL_MISSING`, `SAMPLING_RATE_MISMATCH`, and `WINDOW_OVERLAP_RISK`. It also records channel masks and quality notes.

Inputs and outputs. Inputs are channel metadata, raw or preprocessed segment summaries, dataset-provided quality notes, and validation thresholds. Outputs are `ArtifactFlagRecord`, channel-mask tables, quality summaries, and warning fields in `WindowRecord` and cards.

Processing logic. The annotator applies configured checks, attaches flags at dataset, subject,

session, run, trial, or window level, and records whether flags are advisory, exclusionary, or downstream-choice signals.

Dependencies and consumers. It depends on metadata, preprocessing registry, and validation thresholds. Consumers are Layer 2 model filters, Layer 4 evidence-quality features, Layer 5 temporal trust, Layer 8 stress selection, Layer 10 dashboard warnings, and thesis failure analysis.

Evaluation, failure modes, and evidence. Evaluation checks flag reproducibility, channel-mask consistency, and correlation with validation warnings. Failure modes include over-rejecting data, under-flagging artifacts, or using inconsistent thresholds across datasets. Evidence appears as artifact-rate tables and subject/session quality summaries.

7.8.7 Split and protocol manager

Purpose and bottleneck addressed. The split and protocol manager is one of the most important Layer 1 submodules. It addresses the fairness and leakage bottleneck: decoders, uncertainty modules, IHARQ, RegimeRisk, and policies are only comparable if they operate on immutable, leakage-free, low-calibration-aware splits.

Engineering responsibility. It defines within-subject, cross-session, cross-subject, low-calibration, validation, calibration, adaptation, and held-out test protocols. It produces `SplitRecord` files, split manifests, budget tables, split IDs, and split-version metadata.

Inputs and outputs. Inputs are normalized metadata, label maps, protocol version, seeds, `splits.yaml`, and Layer 0 scope constraints. Outputs are `SplitRecord`, split manifest tables, low-calibration budget tables, leakage-check inputs, and protocol-card sections.

Processing logic. The manager assigns membership at the safest available level: subject for cross-subject, session for cross-session, trial for within-subject, and never independent overlapping windows across train/test. Label budgets such as zero calibration, one trial per class, two trials per class, five trials per class, ten trials per class, and full-label upper bound can be declared when dataset size permits.

Dependencies and consumers. Hard dependencies are metadata normalization, label mapping, seed policy, and validation rules. Consumers are Layer 2 training/evaluation, Layer 3 calibration, Layer 4 IHARQ evaluation, Layer 5 temporal analysis, Layer 6 policy training, Layer 8 stress tests, Layer 10 dashboards, and the ablation chapter.

Evaluation, failure modes, and evidence. Evaluation checks split disjointness, subject/session separation, class distribution, low-budget integrity, seed determinism, and repeated split regeneration. Failure modes include train/test overlap, validation/test leakage, calibration-budget inflation, and using test data for split tuning. Evidence appears as split manifests, budget tables, and leakage audit reports.

7.8.8 Trial/window generator

Purpose and bottleneck addressed. The trial/window generator converts trials into downstream consumable units while preserving lineage. It addresses the temporal-unit bottleneck: Layer 2 needs training units, Layer 4 needs evidence windows, Layer 5 needs sequences, Layer 7 needs replay windows, and Layer 8 needs stress targets, but all must trace back to parent trials.

Engineering responsibility. It creates `WindowRecord` entries with window ID, trial ID, parent-trial ID, dataset ID, subject ID, session ID, run ID, start/end time, duration, stride,

window-role tag, overlap group ID, segment pointer, original label, normalized label, split membership, label-budget ID, artifact flags, channel mask, preprocessing ID, preprocessing version, and config hash.

Inputs and outputs. Inputs are normalized trials, preprocessing records, split records, label maps, artifact flags, and `windows.yaml`. Outputs are `WindowRecord`, window lineage maps, segment pointers, and window validation summaries.

Processing logic. Windows may be full-trial windows, fixed sliding windows, evidence windows, stability windows, RegimeRisk windows, or simulation replay windows. Layer 1 produces these windows and tags their intended use, but it does not decide IHARQ actions, RegimeRisk scores, or simulated rewards.

Dependencies and consumers. Hard dependencies are split records, metadata, labels, preprocessing, and artifact flags. Consumers are Layers 2–8, especially decoder training, evidence verification, temporal trust monitoring, replay, stress tests, and dashboard time-series views.

Evaluation, failure modes, and evidence. Evaluation checks window duration, sample length, channel count, label inheritance, parent-trial identity, split membership, overlap group, segment pointer existence, and reproducibility. Failure modes include duplicate windows, windows crossing split boundaries, inconsistent labels, and missing segment pointers. Evidence appears as window lineage validation and replay-readiness reports.

7.8.9 Data validation and leakage auditor

Purpose and bottleneck addressed. The auditor is the formal exit gate of Layer 1. It addresses the validity bottleneck: downstream training should not begin if records are malformed, incomplete, inconsistent, or leaky.

Engineering responsibility. It checks schema validity, required fields, identifier uniqueness, subject/session/run/trial hierarchy, class distributions, split disjointness, overlap leakage, normalization fitting scope, label-budget integrity, duplicate records, source pointer existence, metadata completeness, and repeated-generation reproducibility.

Inputs and outputs. Inputs are all Layer 1 records, configs, split manifests, preprocessing logs, and validation thresholds. Outputs are `ValidationReport`, leakage audit reports, metadata completeness reports, pass/warning/fail statuses, readiness-level labels, and the `layer1_ablation_readiness_l1_v1.json` reporting artifact.

Processing logic. The auditor runs before Layer 2. Hard failures block downstream execution. Warnings may permit execution only if they are documented in the manifest, cards, and Layer 1 ablation-readiness artifact. The auditor also supports mock-record validation during development.

Dependencies and consumers. It depends on every upstream Layer 1 submodule. Consumers are all downstream layers, Layer 10, and thesis appendices. Layer 2 should perform a smoke test against auditor-approved records but should not patch Layer 1 silently.

Evaluation, failure modes, and evidence. Evaluation is the validation report itself. Failure modes include false negatives in leakage detection, overly permissive missingness thresholds, or warnings being ignored downstream. Evidence appears as the Layer 1 validation table and leakage audit appendix.

7.8.10 Dataset-card and protocol-card generator

Purpose and bottleneck addressed. Cards make Layer 1 thesis-ready and dashboard-ready. They address the communication and claim-boundary bottleneck: without cards, the system may be reproducible in code but unclear to readers, examiners, or dashboard users.

Engineering responsibility. The generator creates dataset cards containing dataset identity, task description, subjects/sessions, recording details, labels, inclusion reason, filtering/exclusion decisions, public-data limitations, protocol compatibility, known risks, and citation/source notes. It also creates protocol cards containing protocol ID, protocol type, split logic, calibration budget, preprocessing ID, windowing ID, leakage controls, downstream consumers, and limitations.

Inputs and outputs. Inputs are dataset registry entries, label maps, metadata summaries, preprocessing records, split records, validation reports, and Layer 0 claim constraints. Outputs are `DatasetCard` and `ProtocolCard` files for Layer 10 and thesis chapters.

Processing logic. Cards are generated from records, not manually rewritten from memory. Every card includes allowed claims and forbidden claims. If metadata is missing or a protocol is proxy-only, the card states it directly.

Dependencies and consumers. It depends on the manifest builder, registry, label mapper, split manager, preprocessing registry, and validation auditor. Consumers are Layer 10, architecture chapter, methodology chapter, results discussion, and reproducibility appendix.

Evaluation, failure modes, and evidence. Evaluation checks card completeness, agreement with records, limitation coverage, and dashboard readability. Failure modes include stale cards, missing limitation text, or cards that imply stronger claims than Layer 0 permits. Evidence appears as dataset/protocol cards and claim-safety warnings.

7.8.11 Provenance and manifest builder

Purpose and bottleneck addressed. The manifest builder creates a single reproducibility index for Layer 1. It addresses the traceability bottleneck: without a manifest, downstream layers cannot reliably know which records, configs, seeds, scripts, and validation statuses produced a result.

Engineering responsibility. The builder writes `Layer1Manifest` with manifest ID, protocol version, dataset IDs, paths to dataset/window/split/preprocessing/label/artifact records, card paths, validation-report path, ablation-readiness artifact path, config file list, config hash, generation timestamp, code version, seed, output folders, validation status, ablation-readiness status, comparison-key completeness, and known limitations.

Inputs and outputs. Inputs are all Layer 1 artifacts, config files, code-version metadata, validation outputs, and matched-comparison key completeness summaries. Outputs are the manifest JSON/YAML file, the `layer1_ablation_readiness_l1_v1.json` reporting artifact, reproducibility index, and manifest summary tables for Layer 10 and the thesis appendix.

Processing logic. The builder hashes configuration files, checks artifact existence, links every artifact path, summarizes validation status, records ablation-readiness status and diagnostic-only downgrade reasons, and marks the protocol state as pass, warning, or fail.

Dependencies and consumers. It depends on all Layer 1 submodules and optional version-control metadata. Consumers are all downstream layers, Layer 10 artifact lineage views, reproducibility packaging, and future reruns.

Evaluation, failure modes, and evidence. Evaluation checks whether every expected arti-

fact is indexed, every config hash is stable, and every downstream consumer can locate required files. Failure modes include stale paths, missing code version, unindexed cards, or manifest status inconsistent with validation reports. Evidence appears in the reproducibility appendix and dashboard artifact lineage panel.

7.9 Canonical Layer 1 records and schemas

Layer 1 records should be explicit enough that downstream modules can operate without guessing. The full schema may be implemented as JSON, Parquet, CSV plus sidecar JSON, or typed Python data classes, but the architecture requires at least the following fields. For evaluation, the existing identifiers in these records form the mandatory matched-comparison key. The architecture therefore does not require Layer 1 to create a separate runtime ablation record; it requires Layer 1 to preserve the fields that make later ablation records trustworthy and to emit the lightweight `layer1_ablation_readiness_l1_v1.json` reporting artifact when matched-comparison keys are incomplete, warning-level, or diagnostic-only.

Record	Required or recommended fields
DatasetRecord	dataset_id, dataset_name, source_name, source_version, license_note, subject_id, original_subject_id, session_id, run_id, task_set, channel_names_original, channel_names_selected, sampling_rate_original, sampling_rate_target, preprocessing_id, split_id, protocol_version, claim_scope_tag.
WindowRecord	window_id, trial_id, parent_trial_id, dataset_id, subject_id, session_id, run_id, window_start_time, window_end_time, window_duration, window_stride, window_role_tag, overlap_group_id, segment_pointer, normalized_label, original_label, split_membership, label_budget_id, artifact_flags, channel_mask_id, preprocessing_id, preprocessing_version, config_hash.
SplitRecord	split_id, protocol_name, split_level, train_membership, validation_membership, test_membership, calibration_membership, adaptation_membership, label_budget_id, calibration_budget, seed, stratification_rule, leakage_checks_passed, split_version.
PreprocessingRecord	preprocessing_id, preprocessing_version, filter_config, resampling_config, channel_config, reference_config, epoch_config, normalization_config, normalization_fit_scope, artifact_policy, implementation_script, config_hash.
LabelMapRecord	label_map_id, label_map_version, dataset_id, original_label, normalized_label, task_family, protocol_inclusion, exclusion_reason, rest_policy, proxy_intent_note.
ArtifactFlagRecord	flag_id, flag_type, dataset_id, subject_id, session_id, run_id, trial_id, window_id, severity, action, source, notes.

Layer1Manifest	manifest_id, protocol_version, dataset_ids, dataset_records_path, window_records_path, split_records_path, preprocessing_records_path, label_map_records_path, artifact_flag_records_path, cards_path, validation_report_path, ablation_readiness_path, config_files, config_hash, seed, generation_timestamp, code_version, output_paths, validation_status, ablation_readiness_status, matched_comparison_key_completeness, known_limitations.
-----------------------	--

7.10 Public-dataset registry and dataset-status framework

The dataset registry should explicitly separate core evidence from optional or future ambitions.

Status	Meaning
Core	Required for the main thesis experiments and must pass Layer 1 validation before serious Layer 2 training begins.
Secondary	Useful for additional generality or cross-dataset robustness, but not required for the core thesis claim.
Optional	Used only if time, compute, and data quality permit.
Stretch	Used for advanced demonstrations, robustness exploration, or future-facing extension.
Excluded	Considered but rejected with a documented reason.
Future	Mentioned as future work only and not included in thesis evidence.

Inclusion criteria include public availability, motor-imagery or relevant EEG task, subject-level metadata, session/run/trial structure, sufficient samples for low-calibration evaluation, compatibility with preprocessing, clear label mapping, reproducible access, reasonable documentation, and appropriate usage terms. Exclusion criteria include unclear labels, insufficient metadata, unrelated task type, unstable access, untraceable manual preprocessing, too few samples, unusable subject/session structure, unclear license status, or excessive risk of overclaiming.

7.11 Split, low-calibration, and leakage-control protocol

Layer 1 must define low-calibration budgets before model training and before evaluation variants are compared. Example budget IDs may include B0 for zero-calibration or transfer-only baselines if feasible, B1 for one labeled trial per class, B2 for two labeled trials per class, B5 for five labeled trials per class, B10 for ten labeled trials per class, and BFull for the full-label upper-bound reference. The actual values must be adapted to each dataset and recorded in **SplitRecord** and **ProtocolCard**.

Leakage type	Layer 1 defense
Subject leakage	Enforce subject-disjoint train/validation/test membership when the protocol is cross-subject.
Session leakage	Enforce session-disjoint membership when the protocol is cross-session.

Trial/window leakage	Split at subject, session, or trial level before windowing, or ensure all windows from the same parent trial remain in one split.
Normalization leakage	Fit normalization only on permitted training/calibration data; test data must not fit preprocessing parameters.
Label-budget leakage	Prevent additional labeled examples from entering low-calibration training, calibration, or threshold selection.
Hyperparameter leakage	Do not use held-out test results to revise split, preprocessing, or label mapping.
Duplicate leakage	Detect duplicate or near-duplicate trial/window IDs across splits.
Dashboard leakage	Layer 10 reads saved Layer 1 records and cards; it must not recompute or silently alter protocols.

7.12 Dependency logic

Dependency class	Layer 1 dependency logic
Hard dependencies	Layer 0 claim-safety constraints, selected public dataset access, dataset registry, label mapping, preprocessing config, split config, window config, seed policy, and validation rules. Without these, Layer 1 cannot produce thesis-grade records.
Soft dependencies	MOABB, MNE, Braindecode, dataset-specific loaders, local caching, optional quality annotations, and visualization utilities. These improve implementation but should not be hidden assumptions.
Optional dependencies	Secondary datasets, stretch datasets, compact SSL preprocessing compatibility checks, advanced artifact scoring, and dashboard preview exports.
Conceptual dependencies	Public-data honesty, protocol immutability, leakage prevention, subject/session awareness, low-calibration relevance, and downstream neutrality.
Mockable dependencies	Synthetic mini-datasets, mock <code>DatasetRecord</code> , mock <code>WindowRecord</code> , mock split manifests, and dummy cards can be used during schema and dashboard development.
Forbidden dependencies	Decoder performance, calibrated confidence, IHARQ success, RegimeRisk scores, policy rewards, stress outcomes, embodiment success, or dashboard preference must not determine Layer 1 protocol choices unless a new protocol version is declared.

Layer 2 may run only after the dataset registry, label map, preprocessing records, split records, window records, validation report, and Layer 1 manifest exist with pass or documented-warning status. Thesis-ready ablation evaluation may use Layer 1 outputs only when the validation report confirms that the required matched-comparison keys are complete or that any gaps are explicitly marked as diagnostic limitations.

7.13 Feedforward and loopback behavior

The normal feedforward path is:

Layer 0 claim-safety constraints → Layer 1 dataset/protocol records → Layer 2 decoder measurement → Layers 3–6 reliability and readiness logic → Layers 7–9 simulation, stress, and embodiment → Layer 10 dashboard, cards, and reproducibility exports.

Layer 1 outputs feed forward as follows: **DatasetRecord**, **WindowRecord**, **SplitRecord**, **PreprocessingRecord**, and **LabelMapRecord** feed Layer 2; split and validation partitions feed Layer 3; window lineage feeds Layer 4; subject/session/window sequences feed Layer 5; protocol identity and context fields feed Layer 6; replay windows feed Layer 7; artifact and channel fields feed Layer 8; dataset/window/split/preprocessing provenance, artifact flags, proxy-intent label maps, and command-boundary warnings feed Layer 9; and all cards, reports, manifests, validation outputs, and ablation-readiness artifacts feed Layer 10. The same identifiers also feed the Ablation and Evaluation Architecture section as matching keys for A0–A13 comparisons, especially A0–A4 in the public-data, decoder, calibration, and selective-prediction slice.

Layer 1 permits only scientifically justified loopbacks:

Source	Target	Signal and trigger	Action	Safety constraint and evaluation
Validation auditor	Registry, metadata, split, preprocessing, or window modules	Schema failure, leakage failure, missing required fields, duplicate windows, invalid budget, or train-only fitting violation	Stop downstream execution and repair the responsible Layer 1 config or adapter	New validation report required; if repaired after protocol freeze, create a new protocol version and manifest.
Layer 0 claim-safety governance	Dataset/protocol cards and label maps	Scope warning, invalid clinical wording, proxy-intent risk, or public-data limitation missing	Update card text, scope tags, and forbidden-claim warnings	No downstream performance result may override Layer 0 claim boundaries.
Layer 2 smoke test	Layer 1 schema/manifest builder	Decoder cannot read required fields, segment pointers, split IDs, labels, or preprocessing IDs	Fix schema/export compatibility or add documented fields	Must not change splits or preprocessing based on model accuracy; rerun validation and update manifest if artifacts change.

Layer 10 dashboard audit	Card generator or manifest builder	Dashboard detects missing card fields, stale paths, incomplete filters, or unindexed artifacts	Regenerate cards or manifest from saved records	Dashboard must not recompute hidden protocols; regenerated artifacts must match source records.
Thesis review/audit	Dataset registry and cards	Examiner identifies unclear inclusion rationale or overclaiming risk	Clarify inclusion/exclusion rationale and limitation text	Scientific claim strength may be reduced, but experimental protocol is not silently retuned.

7.14 Layer 1 evaluation design and exit conditions

Layer 1 is not evaluated by classification accuracy. It is evaluated by protocol quality, reproducibility, validity, leakage control, record completeness, and downstream readiness.

Evaluation question	Evidence artifact
Can the selected public datasets be loaded reproducibly?	Dataset loader report, source access log, repeated-load comparison, and manifest.
Is metadata normalized consistently?	Metadata mapping report, completeness report, hierarchy checks, and normalized tables.
Are label mappings explicit and defensible?	LabelMapRecord , label ontology table, class-distribution table, and dataset cards.
Are train/validation/test/calibration splits fixed and leakage-free?	SplitRecord , split manifest, split disjointness check, and leakage audit.
Do low-calibration budgets match the thesis question?	Budget table, protocol cards, and label-budget integrity report.
Are windows traceable to parent trials?	Window lineage validation, overlap-group report, and segment-pointer checks.
Are preprocessing choices explicit and leakage-controlled?	PreprocessingRecord , train-only fitting report, preprocessing config hash, and preprocessing log.
Can Layer 2 consume records without hidden assumptions?	Downstream smoke test using at least one lightweight decoder or mock decoder.
Can Layer 10 inspect Layer 1 outputs?	Dataset-card, protocol-card, manifest, and validation-report dashboard preview.
Can later ablation comparisons match records fairly?	Comparison-key completeness report covering dataset, subject, session, run, trial, window, split, preprocessing, label-map, low-calibration budget, seed, and protocol-version identifiers.
Can evaluation-only labels remain separated from decision-time fields?	Split-role audit, label-access policy, and validation report showing that labels and correctness helpers are used for scoring or calibration only where permitted.

Can Layer 1 be reproduced exactly?	Reproduction command, fixed seed, config hash, output file list, and repeated-generation pass/fail result.
------------------------------------	--

Required Layer 1 metrics include number of included datasets, number of subjects/sessions/runs, trials/windows per class, metadata completeness percentage, split disjointness status, window lineage pass rate, artifact-flag rate, low-calibration budget validity, matched-comparison key completeness, evaluation-only label separation status, reproducibility pass/fail, card completeness score, and downstream compatibility status.

Success criteria are: all core datasets load; required metadata fields pass schema checks or have documented limitations; label maps are explicit; split leakage checks pass; window lineage pass rate is effectively complete for usable windows; preprocessing fitting scope is declared and leakage-safe; matched-comparison keys required by the evaluation architecture are present or explicitly warned; evaluation-only labels are separated from decision-time fields; Layer 1 manifest indexes all required artifacts; Layer 2 smoke test passes; and cards contain public-data limitation statements. Failure interpretation must be conservative: a failed leakage check invalidates downstream experiments under that protocol version; a missing metadata field narrows claims; an ambiguous label may require exclusion; a failed reproduction check blocks thesis-grade results until resolved.

7.15 Reproducibility, repository structure, and implementation artifacts

Layer 1 should be reproducible independently of the rest of the architecture. A representative reproduction command is:

```
python run_layer1_protocol.py -config configs/layer1/main.yaml -seed
42 -protocol-version L1-v1
```

Expected configuration and artifact structure:

```
IHARQ-BenchGuard/
  configs/
    datasets/
      datasets.yaml
      labels.yaml
    preprocessing/
      preprocessing_l1_v1.yaml
    splits/
      splits_l1_v1.yaml
    windows/
      windows_l1_v1.yaml
    validation/
      validation_l1_v1.yaml
  src/
    data_protocol/
      dataset_registry.py
      dataset_adapters/
      metadata_normalizer.py
      label_mapper.py
      preprocessing_registry.py
```

```

    artifact_annotator.py
    split_manager.py
    window_generator.py
    validation_auditor.py
    card_generator.py
    manifest_builder.py
records/
  dataset_records/
  window_records/
  split_records/
  preprocessing_records/
  label_map_records/
  artifact_flag_records/
cards/
  dataset_cards/
  protocol_cards/
reports/
  layer1_validation/
  leakage_audits/
  metadata_completeness/
manifests/
  layer1_manifest_l1_v1.json
  layer1_ablation_readiness_l1_v1.json

```

The implementation order should be: define the dataset registry; define label ontology; define metadata schema; define preprocessing registry; define split protocols; define window rules; implement validation checks; generate cards; build the Layer 1 manifest; and run a downstream Layer 2 smoke test. Serious Layer 2 training should not begin until validation checks pass or warnings are explicitly documented.

7.16 Failure modes and defensive design

Failure mode	Consequence	Defensive design
Dataset access changes	Reproduction fails or subject counts change.	Cache source notes, versions, access logs, and manifest paths; record dataset-source limitations.
Metadata mismatch	Downstream grouping by subject/session/run becomes invalid.	Metadata normalization tests and hierarchy validation.
Label mapping ambiguity	Class comparisons become scientifically unclear.	Explicit <code>LabelMapRecord</code> , exclusion rules, and proxy-intent notes.
Train/test leakage	Baseline performance and reliability estimates are inflated.	Split auditor, disjointness checks, and split-before-window rule.
Window overlap leakage	Repeated windows inflate performance or temporal stability.	Parent-trial grouping and <code>overlap_group_id</code> .

Hidden preprocessing	Results cannot be defended or reproduced.	Preprocessing registry, train-only fitting rule, and config hash.
Subject/session confusion	Cross-subject or cross-session claims become invalid.	Subject/session/run hierarchy validation and split-level declaration.
Low-calibration budget inconsistency	Comparisons become unfair.	Budget IDs, budget manifests, and label-budget leakage checks.
Missing matched-comparison key	Ablation tables can accidentally compare different datasets, subjects, sessions, windows, splits, preprocessing versions, label maps, budgets, or seeds.	Validation report marks the affected comparison as diagnostic-only until the missing key or documented exclusion is repaired.
Downstream code assumes extra fields	Integration fails or hidden repairs appear.	Schema validation, mock records, and Layer 2 smoke test.
Dataset cards miss limitations	Thesis overclaiming risk increases.	Layer 0 limitation text integrated into cards and Layer 10 warnings.
Dashboard recomputes protocols silently	Results become untraceable.	Dashboard reads saved records, manifests, and cards only.

7.17 Layer 1 readiness levels

Level	Name	Meaning
L1-RL0	Concept	Layer 1 is only described as a public-data anchor.
L1-RL1	Dataset scope defined	Candidate datasets and inclusion status are listed.
L1-RL2	Metadata schema defined	DatasetRecord and normalized metadata fields are specified.
L1-RL3	Protocol schema defined	Split, low-calibration, preprocessing, and window schemas are specified.
L1-RL4	Cards defined	Dataset-card and protocol-card templates are included.
L1-RL5	Leakage controls defined	Subject, session, trial/window, normalization, and budget leakage rules are stated.
L1-RL6	Implementation-ready	Config files, output paths, reproduction command, and validation command are defined.
L1-RL7	Downstream-ready	Layer 2 and Layer 10 can consume Layer 1 records without hidden assumptions.
L1-RL8	Thesis-ready	Claims, limitations, figures, cards, ablation-readiness warnings, and reproducibility appendix are supported.
L1-RL9	Audit-ready	A reviewer can independently trace every downstream result and matched comparison to Layer 1 artifacts and claim boundaries.

The target maturity for the thesis version is L1-RL8 at minimum and L1-RL9 if the final repository package includes stable manifests, cards, validation reports, and rerunnable configuration.

7.18 Outputs and outgoing links

Layer 1 outputs standardized datasets, immutable split files, metadata tables, label-map records, preprocessing records, artifact/channel-quality records, dataset cards, protocol cards, leakage audit reports, validation reports, window records, ablation-readiness warnings, the `layer1_ablation_readiness_l1_v1.json` artifact, matched-comparison key completeness status, and a Layer 1 manifest.

Layer 1 feeds Layer 2 with `DatasetRecord`, `WindowRecord`, `SplitRecord`, `PreprocessingRecord`, `LabelMapRecord`, artifact flags, train/validation/test/calibration membership, and label-budget/low-calibration budgets. It feeds Layer 3 with calibration/validation/test identities and label information. It feeds Layer 4 with window lineage and evidence-window groupings. It feeds Layer 5 with subject/session/window sequences and quality flags. It feeds Layer 6 with protocol identity, label-budget context, and readiness-context metadata. It feeds Layer 7 with replayable window sequences. It feeds Layer 8 with artifact/channel/stress-target metadata. It feeds Layer 9 with dataset/window/split/preprocessing provenance, artifact flags, proxy-intent label maps, and command-boundary warnings. It feeds Layer 10 with dataset cards, protocol cards, manifests, validation reports, metadata completeness summaries, split summaries, ablation-readiness artifacts, and claim-safety warnings. It feeds Layer 0 with public-data limitation statements and any invalid-claim risks discovered during dataset/protocol definition.

8 Layer 2: Decoder and Baseline Measurement Spine

8.1 Problem solved by Layer 2

Layer 2 provides the controlled prediction source and baseline measurement spine of IHARQ BenchGuard. It is intentionally not presented as the main novelty of the thesis. The main thesis novelty begins after raw decoding: calibration, uncertainty analysis, IHARQ evidence verification, RegimeRisk temporal trust, adaptive readiness decisions, simulated consequences, stress evaluation, optional embodiment, dashboarding, cards, and reproducibility packaging. However, none of those post-decoder claims are credible unless Layer 2 produces fair, reproducible, inspectable, and sufficiently rich decoder evidence.

The bottleneck solved by Layer 2 is the measurement-bottleneck problem: downstream reliability modules need raw predictions, probability vectors, logits when available, model-family disagreement, low-calibration performance behavior, subject/session heterogeneity, and traceable model metadata. Without a disciplined Layer 2, the architecture would collapse into either an ungrounded reliability wrapper or an ordinary classifier benchmark. With Layer 2, every downstream reliability, readiness, and dashboard claim can be traced back to concrete `PredictionRecord` artifacts and baseline measurement tables.

Layer 2 therefore answers the thesis-level measurement question:

Given standardized public EEG data and low-calibration protocols from Layer 1, what baseline decoding evidence can be produced fairly, reproducibly, and richly enough to support downstream reliability and readiness evaluation?

This layer protects the thesis against several predictable criticisms: weak baselines, unfair comparison, hidden prediction outputs, decoder-specific conclusions, vague low-calibration claims, ignored subject variability, and the possibility that IHARQ is merely ordinary ensemble voting.

Its purpose is not to prove clinical readiness. Its purpose is to create the empirical substrate that later layers calibrate, verify, stress, defer, adapt, visualize, and discuss.

Layer 2 owns the decoder-baseline slice of the ablation architecture. In the system-level A0–A13 ladder, Layer 2 is responsible for making A0 raw-decoder evidence, model-family baselines, low-calibration curves, and ordinary ensemble or multi-window controls fair, reproducible, and inspectable. It does not decide the final ablation ranking; instead, it emits the **PredictionRecord**, registry, baseline tables, and ensemble-control records that allow the evaluation architecture to compare raw decoding against calibration, thresholding, selective prediction, IHARQ, and later readiness modules under matched conditions.

8.2 Layer 2 thesis role and contribution boundary

Layer 2 has three simultaneous identities:

Identity	Meaning inside the architecture
Engineering identity	Trains and runs decoder model families under fixed public-data, split, seed, preprocessing, and low-label-budget protocols.
Measurement identity	Produces PredictionRecord files, model registry entries, baseline metric tables, low-calibration curves, subject difficulty profiles, ensemble-control records, and failed-run logs.
Thesis-evidence identity	Creates the empirical substrate from which Layers 3–10 evaluate calibration, uncertainty, evidence quality, regime risk, readiness policy, simulated closed-loop consequences, stress robustness, embodiment-readiness consequences, and reproducibility.

Claim-safety rule: Layer 2 may claim that the thesis uses a reproducible, decoder-agnostic baseline measurement spine. It must not claim that raw decoder confidence is calibrated, that high decoder accuracy implies readiness, that decoder prediction alone is safe enough for action, or that public EEG results establish clinical deployment validity.

If Layer 2 were removed, the architecture would lose its prediction source, baseline fairness anchor, low-calibration measurement engine, subject-variability evidence source, ensemble-control baseline, and model traceability foundation. Layers 3–6 and Layer 10 would either need to rerun hidden decoder scripts or make reliability claims without auditable prediction artifacts.

8.3 Scope, non-scope, and boundary rule

Layer 2 owns decoder-side measurement. It does not own readiness decisions.

In scope	Layer 2 responsibility
Decoder training	Train sanity, classical, Riemannian, compact deep, optional compact SSL/foundation-style, and ensemble-control decoders where feasible.
Fixed protocols	Consume Layer 1 DatasetRecord , WindowRecord , split manifests, preprocessing metadata, and label-budget protocols without redefining dataset governance.

Low-calibration measurement	Run declared calibration/training label budgets and produce budgeted prediction logs, curves, and tables.
Prediction logging	Save per-window and per-trial raw predictions, probability vectors, logits when available, decision scores, identifiers, labels where evaluation permits, and split/budget/model metadata.
Subject/session profiling	Produce subject difficulty, instability, low-label fragility, and model-family sensitivity profiles.
Model registry	Standardize model IDs, decoder IDs, hyperparameters, seeds, training status, artifact paths, prediction paths, and configuration hashes.
Ensemble controls	Produce ordinary majority-vote, probability-averaging, confidence-threshold, and multi-window voting controls so IHARQ is not confused with simple model combination.
Decoder-level reporting	Produce baseline tables, low-calibration figures, subject-profile figures, ensemble-control tables, model cards, failed-run logs, and reproducibility artifacts for Layer 10.

Out of scope	Owned elsewhere
Probability calibration after decoding	Layer 3. Layer 2 may store raw confidence fields but must not interpret them as calibrated trust.
Uncertainty interpretation and selective prediction	Layer 3. Layer 2 provides raw logits, probabilities, decision scores, and correctness helpers.
IHARQ accept/wait/request/combine/fall-back/unsafe decisions	Layer 4. Layer 2 only supplies prediction and liability evidence.
RegimeRisk temporal trust scoring	Layer 5. Layer 2 supplies time-indexed prediction logs, performance histories, and subject profiles.
Adaptive policy learning, contextual bandit, and RL policy training	Layer 6. Layer 2 does not optimize readiness policies.
Simulated closed-loop consequence modeling	Layer 7. Layer 2 prediction logs may seed replay sessions but do not define rewards or consequences.
Stress generation	Layer 8. Layer 2 outputs are tested under stress, but stress schedules are not created here.
MyoSuite/OpenSim command mapping	Layer 9. Layer 2 does not claim embodiment validation.
Dashboard design	Layer 10. Layer 2 provides dashboard-ready records and figures but does not own dashboard interaction design.
Clinical or patient-safety claims	Layer 0 governance. Layer 2 results remain public-dataset and simulation evidence only.

Boundary rule. Layer 2 predicts and measures. It does not decide readiness. It produces raw and baseline decoding evidence; Layers 3–6 decide how trustworthy, stable, sufficient, or actionable that evidence is.

8.4 Design principles

Principle	Meaning	Architectural implication
Decoder agnosticism	Downstream reliability should not depend on one model family.	All decoders write the same PredictionRecord schema.
Contract-first outputs	Later layers must know exactly what Layer 2 produces.	Schemas for prediction records and model registry entries are defined before full training.
Fair comparison	Model comparisons must use equivalent data, splits, budgets, and reporting units.	Split IDs, label-budget IDs, preprocessing versions, seeds, and missing-run policies are recorded.
Low-calibration centrality	The thesis studies limited calibration burden.	Performance is reported as a function of calibration/training label budget.
Subject awareness	BCI subjects and sessions are not interchangeable.	Subject/session metrics, difficulty profiles, instability flags, and failure cases are first-class outputs.
Ensemble control	IHARQ must be distinguished from ordinary voting or averaging.	Ensemble baselines are mandatory controls, not decorative add-ons.
Ablation readiness	Raw decoder, longer-window, voting, averaging, confidence-threshold-control, and ensemble variants must be usable as matched baselines rather than informal comparisons.	PredictionRecord , ModelRegistryRecord , BaselineMetricRecord , ensemble-control records, LowCalibrationCurveRecord , and SubjectProfileRecord outputs expose the identifiers needed for A0 and A4-style comparisons and for later A1–A5 comparisons.
Failure transparency	Failed or unsupported model runs can bias results if hidden.	Failed runs, skipped runs, missing probabilities, missing logits, convergence failures, and incompatibilities are logged.
Claim restraint	Decoder performance is not the final contribution.	Layer 2 is documented as measurement infrastructure, not as proof of readiness.

8.5 Input contract

Layer 2 starts only after Layer 1 has produced stable public-data records and split/protocol artifacts. The layer must not silently repair missing data-governance artifacts; it should either validate them or fail with an explicit reason.

Input	Producer	Class	Purpose
-------	----------	-------	---------

DatasetRecord	Layer 1	Primary	Defines dataset ID, subject/session/run information, channels, sampling rate, task labels, preprocessing metadata, split membership, and license/source notes.
WindowRecord	Layer 1	Primary	Defines training and inference units: trial/window ID, segment pointer, time boundaries, artifact flags, channel mask, task label, and split membership.
SplitRecord and split manifest	Layer 1	Protocol	Define train/validation/test/calibration/adaptation membership, split role, split version, label-budget linkage, and leakage-control policy.
Label-budget configuration	Layer 1 or experiment config	Protocol	Defines low-calibration budgets such as absolute trials, percentage budgets, session fractions, or transfer/adaptation settings.
Preprocessing metadata	Layer 1	Metadata	Ensures model inputs are traceable to the exact filtering, resampling, windowing, normalization, channel handling, and artifact policy.
Subject/session metadata	Layer 1	Metadata	Enables subject difficulty profiling, temporal analysis, session-wise metrics, and RegimeRisk history features.
Channel metadata	Layer 1	Metadata	Enables compatibility checks, channel-aware models, and later channel-stability interpretation.
Pretrained encoder checkpoint	External package or prior model	Optional/stretch	Allows compact SSL/foundation-style baselines if available, legal, computationally feasible, and explicitly documented.
Held-out labels	Layer 1 split policy	Evaluation-only	Used only for evaluation and never for training, tuning, budget selection, or calibration leakage.

Before training or inference, Layer 2 validates that every record contains dataset ID, subject ID, split ID, split role, trial ID, window ID when applicable, label, preprocessing version, and split membership; that windows from the same trial are not leaked across incompatible splits; that label mappings are consistent; that artifact flags are handled or excluded according to protocol; that channel count and sampling rate are compatible with each selected model family; that label budgets do not use held-out test labels; that seed policy is fixed; and that a configuration hash is generated.

8.6 Output contract

Layer 2 outputs must be readable by downstream layers without rerunning the decoders. Each output should include dataset ID, subject ID, session ID when available, split ID, label-budget ID, model ID, decoder ID, seed, configuration hash, schema version, output path, and timestamp or simulated time index when applicable.

Output	Downstream consumer	Purpose
Trained model artifacts	Layer 2 reruns, reproducibility package	Allow predictions to be regenerated and model-family behavior to be audited.
PredictionRecord	Layers 3–10 and the Ablation and Evaluation Architecture section	Provides raw decoder outputs for calibration, uncertainty, IHARQ, RegimeRisk, policy contexts, simulated closed-loop replay, stress tests, embodiment-provenance inputs, dashboard views, and A0/A1/A2/A3/A4 matched comparisons.
BaselineMetricRecord	Layer 10, thesis results tables	Establish decoder performance anchors and prevent weak-baseline criticism.
LowCalibrationCurveRecord	Layer 10, thesis figures, tables and curves	Show performance as calibration/training labels increase.
SubjectProfileRecord	Layers 5–7 and 10, thesis outputs	Identify easy, low-label fragile, model-dependent, temporally unstable, difficult, and ambiguous subjects/sessions.
ModelRegistryRecord	All downstream layers entries	Trace which model, seed, split, budget, code path, score type, class order, and artifact generated each prediction.
EnsembleControlRecord	Layers 3, 4, 10, and the Ablation and Evaluation Architecture section	Provide voting, averaging, confidence-threshold controls, longer-window or multi-window controls, disagreement features, delay-burden lineage, and A4-style ordinary-combination baselines.
Training and inference logs	Layer 10, reproducibility package	Record runtime, loss, validation metrics, early stopping, missing outputs, reload checks, and failures.
FailureCaseIndex	Layers 4, 5, and 10	Supports failure taxonomy, high-risk examples, difficult-subject analysis, failed/skipped-run visibility, and defensive discussion.

Layer2ReadinessReport	Layers 3–10	States whether PredictionRecord, ModelRegistryRecord, budget, subject-profile, score-schema, class-order, and ensemble-control coverage are sufficient for each downstream consumer, with Layer 9 limited to provenance/readiness-input compatibility rather than embodiment-safety validation.
-----------------------	-------------	---

8.7 Canonical PredictionRecord schema

The PredictionRecord is the most important Layer 2 artifact. It is the bridge between decoder training and every downstream reliability layer. It may contain raw confidence-like information, but it must not claim calibrated uncertainty.

Field	Type	Required	Purpose
schema_version	string	Yes	Allows future schema changes without breaking downstream modules.
dataset_id	string	Yes	Dataset traceability.
subject_id	string	Yes	Subject-wise analysis and filtering.
session_id	string	If available	Session-wise analysis and temporal tracking.
run_id	string	If available	Dataset run traceability.
split_id	string	Yes	Split traceability and leakage checks.
split_role	string	Yes	Train, validation, test, calibration, threshold-validation, adaptation, or simulation-replay designation.
trial_id	string	Yes	Trial-level alignment.
window_id	string	Yes for windowed data	Window-level alignment.
time_start, time_end	number	Recommended	Window or trial time boundaries for temporal ordering and replay support.
timestamp_or_order	string/number	Recommended for temporal or replay use	Prediction timestamp, simulated time index, or deterministic order key used by RegimeRisk, stress replay, Layer 7 sessions, and Layer 9 provenance checks.
label_true	string/int	Yes when available	Evaluation target; hidden when protocol forbids label access.
model_id	string	Yes	Stable model registry key.
decoder_id	string	Yes	Specific trained instance.
model_family	string	Yes	Sanity, classical, Riemannian, deep, compact SSL, ensemble, or other declared family.
label_budget_id	string	Yes	Low-calibration condition.

<code>seed</code>	<code>int</code>	Yes	Reproducibility.
<code>config_hash</code>	<code>string</code>	Yes	Configuration traceability.
<code>predicted_class</code>	<code>string/int</code>	Yes	Raw predicted label.
<code>probability_vector</code>	<code>array/list</code>	Strongly recommended	Required for calibration, reliability diagrams, Brier/NLL/ECE, entropy, and margin analysis.
<code>logits</code>	<code>array/list</code>	If available	Supports calibration methods and deeper diagnostic analysis.
<code>decision_score</code>	<code>number</code>	If applicable	Alternative score for LDA/SVM or models without probability vectors.
<code>prediction_rank</code>	<code>list</code>	Optional	Class ranking for diagnostic analysis.
<code>inference_latency_ms</code>	<code>number</code>	Optional	Runtime and dashboard analysis.
<code>artifact_flag</code>	<code>bool/string</code>	Recommended	Alignment with Layer 1 quality flags.
<code>notes</code>	<code>string</code>	Optional	Known limitations, unsupported fields, or failure details.

Layer 2 may additionally compute raw derived fields such as `top1_probability`, `top2_probability`, `raw_margin`, `is_correct`, `error_type_raw`, and `model_agreement_group`. These are decoder-level helpers only. Canonical cross-layer field names are `label_true`, `predicted_class`, `probability_vector`, and `split_role`; any implementation shorthand such as `y_true`, `y_pred`, or `prob_vector` must be mapped to the canonical names before export. For evaluation, `is_correct` and `error_type_raw` are evaluation-only fields; they may be used to score A0 raw-decoder baselines and to support high-confidence-wrong audits, but they must not become decision-time policy features. Calibration, trust, selective prediction, and readiness interpretation remain Layer 3 and later responsibilities.

8.8 Canonical ModelRegistryRecord schema and registry contract

Every model output used in the thesis must be traceable to a `ModelRegistryRecord`. No benchmark table, curve, dashboard view, or downstream decision record should cite a decoder result that lacks a registry entry.

Registry field	Description
<code>schema_version</code>	Registry schema version used for validation and downstream compatibility checks.
<code>model_id</code> and <code>decoder_id</code>	Stable identifiers for the model family and the specific trained instance.
<code>model_family</code> and <code>model_name</code>	Human-readable and categorical model identity.
<code>implementation_path</code>	Code path, package reference, or script entry point.
<code>dataset_id</code> , <code>split_id</code> , <code>split_role</code> , <code>label_budget_id</code>	Protocol traceability.
<code>seed</code>	Random seed used for training, sampling, and initialization.
<code>hyperparameters</code>	Complete hyperparameter dictionary, including fixed and selected values.

<code>preprocessing_version</code>	Links to Layer 1 preprocessing and label-map metadata.
<code>label_map_version</code>	
<code>training_start_time</code>	Runtime traceability.
<code>training_end_time</code>	
<code>training_status</code>	Success, failed, skipped, or unsupported.
<code>failure_reason</code>	Required if failed, skipped, or unsupported.
<code>validation_metric</code>	Main validation metric and selection value.
<code>score_type</code> and <code>class_order</code>	Declares whether outputs are probabilities, logits, decision scores, or label-only fallback and records the class ordering used by probability vectors/logits.
<code>ensemble_membership</code>	Optional list or group ID for models used in ensemble-control records.
<code>selected_checkpoint</code>	Checkpoint path or selection rule.
<code>artifact_path</code>	Path to trained model or fitted pipeline.
<code>prediction_record_path</code>	Path to prediction records produced by the model.
<code>config_hash</code> and optional <code>environment_hash</code>	Reproducibility and environment traceability.

Expected registry outputs include `records/model_registry/model_registry.csv`, `records/model_registry/model_registry.parquet`, `records/model_registry/model_registry.yaml`, `records/model_registry/model_registry_schema.yaml`, `logs/failed_runs/failed_model_runs.csv`, and model-family cards for Layer 10.

8.9 Model-family tiers

The architecture should support multiple model families because IHARQ, RegimeRisk, and adaptive policy evaluation must not depend on one arbitrary decoder. The implementation can be phased, but the documentation should identify which model families are core, recommended, optional, stretch, or future work.

Tier	Model family	Status	Thesis role
Tier 0	Majority class, random, simple logistic or linear baseline	Core	Sanity lower bounds; detects broken pipelines and prevents inflated claims.
Tier 1	CSP plus LDA/SVM-style pipelines	Core where compatible	Classical motor-imagery baseline and interpretable anchor.
Tier 2	Riemannian geometry classifiers	Strongly recommended	Strong classical EEG baseline and reviewer-facing comparison point.
Tier 3	Compact deep EEG models such as EEGNet or temporal convolutional models	Recommended if compute allows	Modern neural baseline without turning the thesis into foundation-model training.

Tier 4	Compact SSL or foundation-style encoders	Stretch/feasible	Strong representation baseline when public checkpoints, licenses, compute, and input compatibility allow.
Tier 5	Ensembles, voting, averaging, confidence-threshold, longer-window aggregation, and multi-window voting	Core control	Distinguishes IHARQ evidence verification from ordinary model combination, simple rejection, or delay-heavy aggregation; provides A4-style controls for the evaluation architecture.

For each family, Layer 2 documents why it is included, input format, preprocessing assumptions, hyperparameters, training protocol, validation strategy, low-label handling, output fields, limitations, failure cases, and rerun command. Comparisons must use the same splits, label budgets, preprocessing source, test labels, reporting units, seed policy, and missing-run policy unless a deviation is explicitly justified.

8.10 Internal module structure

Submodule	Status	Primary function	Primary output
Baseline trainer	Core	Trains decoder families under fixed splits and low-label budgets.	Trained model artifacts, validation metrics, training logs, ModelRegistryRecord entries.
Prediction logger	Core	Saves per-window and per-trial decoder outputs in the canonical schema.	PredictionRecord , inference logs, failure index.
Low-calibration curve builder	Core	Measures decoder behavior as calibration/training labels increase.	Budgeted prediction records, low-calibration tables and curves.
Subject difficulty profiler	Core	Identifies easy, low-label fragile, model-dependent, temporally unstable, difficult, and ambiguous subjects/sessions.	Subject difficulty profiles, subject plots, difficult-subject index.
Model-family registry	Core	Standardizes IDs, hyperparameters, seeds, statuses, score types, class order, artifacts, and prediction paths.	ModelRegistryRecord tables, registry YAML/JSON, model cards, failed-run table.

Ensemble comparison builder	Core control	Creates ordinary ensemble, voting, averaging, longer-window, multi-window, and confidence-threshold-control baselines plus disagreement features.	Ensemble prediction records, disagreement table, delay-burden notes, and ensemble baseline metrics.
Compact SSL adapter	Optional/stretch	Wraps compact SSL or foundation-style encoders when feasible.	SSL-derived features, prediction records, stretch comparison tables.
Downstream readiness validator	Supporting	Checks whether Layers 3–10 can consume the Layer 2 outputs or lineage they legitimately require, with Layer 9 limited to provenance/readiness-input compatibility.	Layer2ReadinessReport and schema-compliance report.

8.10.1 Submodule 1: Baseline trainer

Purpose. The Baseline Trainer is the computational engine of Layer 2. It trains sanity, classical, Riemannian, compact deep, optional compact SSL/foundation-style, and ensemble-component decoders under fixed public-data splits and low-label budgets.

Bottleneck addressed. Without a controlled baseline trainer, all later layers would be evaluated on arbitrary, inconsistent, or unreproducible decoder outputs.

Aspect	Specification
Scientific role	Provides the baseline performance anchors that make post-decoder reliability claims interpretable. It shows what raw decoders can and cannot do before calibration, IHARQ, RegimeRisk, and policy adaptation.
Engineering responsibility	Train declared model families; enforce Layer 1 splits; enforce label budgets; save artifacts; log training status; create registry entries; expose validation metrics and failed-run reasons.
Inputs	WindowRecord , DatasetRecord , split manifest, label-budget config, model config, preprocessing metadata, and seed policy.
Outputs	Trained model artifacts, training logs, validation metrics, selected checkpoints, failed-run records, and ModelRegistryRecord entries.
Dependencies	Hard: Layer 1 records and split manifest. Soft: model libraries such as scikit-learn, PyTorch, MNE/MOABB-style tooling, or Riemannian packages. Optional: pretrained encoder checkpoints. Mockable: tiny toy datasets, dummy WindowRecords, and synthetic model outputs for schema tests.
Consumers	Prediction Logger, Model-Family Registry, Low-Calibration Curve Builder, Layer 10 reproducibility package.

Feedforward links	Sends trained artifacts and registry entries to Prediction Logger; sends validation metrics to baseline tables and model selection logic.
Feedback/loopback	If split validation fails, signal Layer 1 to regenerate split manifests. If model compatibility fails, signal the registry to mark the run as unsupported rather than silently dropping it.
Evaluation	Training coverage across datasets, subjects, budgets, model families, seeds, and splits; split integrity; repeated-seed reproducibility; reloadability of artifacts; transparent failure reporting.
Failure modes	Leakage, convergence failure, incompatible channel count, insufficient low-label samples, unsupported dataset format, missing probabilities, unstable validation selection, excessive compute.
Reproducibility	Each run needs a config file, seed, config hash, model ID, environment note, artifact path, and rerun command.
Thesis evidence	Baseline metric tables, model-family comparison, low-label training feasibility, failure-rate table, and reproducibility appendix entries.
Limitations and risks	Stronger models may be computationally infeasible; failed runs may bias aggregate results if not reported; validation selection must not leak test information.

8.10.2 Submodule 2: Prediction logger

Purpose. The Prediction Logger saves all decoder outputs in the canonical `PredictionRecord` schema. It is the main bridge from decoder training to calibration, IHARQ, RegimeRisk, policy learning, stress evaluation, embodiment replay, and dashboarding.

Bottleneck addressed. Without prediction logging, downstream modules would depend on hidden in-memory outputs and the thesis would become difficult to reproduce.

Aspect	Specification
Scientific role	Makes raw decoder evidence auditable at window and trial level, enabling later reliability claims to be traced to exact predictions.
Engineering responsibility	Run inference on validation/test/replay records; save predicted labels, raw probability vectors, logits when available, decision scores, identifiers, labels where allowed, latencies, and failure notes.
Inputs	Trained model artifact, <code>ModelRegistryRecord</code> , <code>WindowRecord</code> , <code>SplitRecord</code> /split manifest, evaluation split designation, and label-budget ID.
Outputs	<code>PredictionRecord</code> files, inference logs, missing-output warnings, schema-validation report, and failure case index.
Dependencies	Hard: trained artifact, registry entry, Layer 1 window records, split IDs. Soft: inference runtime and aggregation utilities. Mockable: dummy model artifact and synthetic <code>WindowRecords</code> for schema tests.
Consumers	Layers 3–6, Layer 8, Layer 9 replay logic, Layer 10 dashboard and cards.

Feedforward links	Sends raw probabilities/logits to Layer 3; model-family predictions and disagreement inputs to Layer 4; time-indexed histories to Layer 5; benchmark-ready logs to Layer 10.
Feedback/loopback	If probability vectors are unavailable, mark fallback score type and notify Layer 3 compatibility checks. If record fields are missing, stop downstream export until schema validation passes.
Evaluation	PredictionRecord completeness, probability availability rate, logit availability rate, schema pass rate, inference reload consistency, and record count agreement with split manifests.
Failure modes	Overwritten predictions, mismatched labels, lost window identifiers, unversioned files, inconsistent class ordering, missing probabilities, invalid aggregation, and silent failed inference.
Reproducibility	Prediction files are versioned and never overwritten without a new run ID; class-order mapping and raw-versus-calibrated status are stored.
Thesis evidence	Provides the auditable prediction substrate for all calibration, uncertainty, IHARQ, RegimeRisk, policy, stress, and dashboard chapters.
Limitations and risks	Large per-window logs can be storage-heavy; probability outputs from different model families may not be equally meaningful until Layer 3 calibration.

8.10.3 Submodule 3: Low-calibration curve builder

Purpose. The Low-Calibration Curve Builder measures decoder behavior as available calibration/training labels increase.

Bottleneck addressed. Without explicit budget curves, the thesis cannot make precise claims about low-calibration behavior or calibration burden.

Aspect	Specification
Scientific role	Converts the vague idea of “low calibration” into measurable curves, budgeted tables, and subject-wise burden analysis.
Engineering responsibility	Define budgets; trigger budgeted training or evaluation; aggregate accuracy, balanced accuracy, F1, AUC where applicable, confusion matrices, and subject/session metrics; generate curves and budget reports.
Inputs	Label-budget config, split manifest, Layer 1 records, trained or trainable decoder configs, PredictionRecords, and model registry entries.
Outputs	Budgeted PredictionRecords, low-calibration metric tables, performance-versus-budget plots, budget sensitivity report, and budget coverage report.
Dependencies	Hard: valid budgets and split manifests. Soft: plotting libraries and aggregation utilities. Optional: cross-subject or session-adaptation protocol. Mockable: toy budget schedules and synthetic predictions.
Consumers	Layer 10, thesis results chapter, Layer 5 subject-risk context, Layer 6 policy features through downstream records.

Feedforward links	Sends budgeted predictions to Layer 3 and budgeted performance summaries to Layer 10; sends low-label fragility indicators to Subject Difficulty Profiler.
Feedback/loopback	If a budget has too few samples for a subject/class, mark the budget as invalid or sparse and notify Layer 1/protocol configuration rather than silently resampling.
Evaluation	Budget coverage, no test-label leakage, curve reproducibility under fixed seeds, subject-level curve availability, and traceability from plotted points back to PredictionRecords.
Failure modes	Budget leakage, non-comparable budgets across datasets, sparse classes, unstable tiny-sample training, overclaiming from noisy low-budget results, and hidden missing budgets.
Reproducibility	Budget definitions are stored in <code>configs/layer2_experiments/label_budgets.yaml</code> ; every curve point stores split ID, budget ID, seed, and model ID.
Thesis evidence	Low-calibration burden curves, budgeted ablation tables, and discussion of where decoders fail under limited calibration.
Limitations and risks	Low-label estimates can have high variance; results should be reported with subject/session spread rather than only aggregate means.

Budgets may be absolute trial counts, percentages, session fractions, within-subject calibration subsets, or cross-subject adaptation budgets. The selected budget family must be declared before training and kept fixed across comparable models.

8.10.4 Submodule 4: Subject difficulty profiler

Purpose. The Subject Difficulty Profiler identifies easy, unstable, low-label fragile, model-dependent, difficult, and ambiguous subjects or sessions.

Bottleneck addressed. Average decoder performance can hide subject-specific failure regimes. This submodule makes heterogeneity visible.

Aspect	Specification
Scientific role	Supports a defensible discussion of subject variability, calibration burden, and failure regimes in public EEG motor-imagery data.
Engineering responsibility	Compute subject/session metrics, variance across sessions and budgets, model-family sensitivity, high-confidence error patterns at the raw level, and difficult-subject indexes.
Inputs	PredictionRecord files, LowCalibrationCurveRecord metrics, DatasetRecord, subject/session metadata, SplitRecord/split manifests, and ModelRegistryRecord entries.
Outputs	SubjectProfileRecord table, subject/session plots, difficult-subject index, subject profile cards, and instability flags.
Dependencies	Hard: subject IDs and enough prediction coverage. Soft: session metadata, time indices, model-family diversity. Optional: channel-level or artifact flags for richer profiling. Mockable: synthetic subject groups.
Consumers	Layer 5 RegimeRisk, Layer 10 dashboard and cards, failure taxonomy, thesis discussion.

Feedforward links	Sends subject difficulty and instability summaries to Layer 5; sends filterable subject profiles to Layer 10; informs Layer 4 evidence-quality analysis for difficult cases.
Feedback/loopback	If a subject has insufficient coverage, mark the profile as ambiguous and request more complete prediction records or exclude only with documented reason.
Evaluation	Subject-profile coverage, stability of categories across seeds, agreement between per-budget and aggregate profiles, and visibility of difficult cases in dashboard/reporting outputs.
Failure modes	Misclassifying subjects due to sparse data, hiding high variance with averages, conflating artifact-heavy data with subject difficulty, and using test outcomes for inappropriate training feedback.
Reproducibility	Category thresholds, aggregation rules, seeds, and source PredictionRecord paths are stored with profile outputs.
Thesis evidence	Subject difficulty tables, instability plots, difficult-case examples, and limitations about heterogeneity and generalization.
Limitations and risks	Difficulty categories are empirical descriptors, not clinical labels. They should be used for analysis and simulation, not diagnosis or patient stratification claims.

Recommended categories are: easy subject, low-label fragile subject, model-dependent subject, temporally unstable subject, difficult subject, and ambiguous subject. Category assignment should prefer transparent thresholds or rank-based rules and should report sensitivity to threshold choices.

8.10.5 Submodule 5: Model-family registry

Purpose. The Model-Family Registry standardizes model identity, hyperparameters, seeds, statuses, artifacts, and prediction paths.

Bottleneck addressed. Without a registry, the thesis cannot reliably trace which model produced which prediction, metric, figure, dashboard view, or downstream decision.

Aspect	Specification
Scientific role	Protects fairness and reproducibility by making model comparisons auditable.
Engineering responsibility	Assign stable IDs; track configs, hyperparameters, seeds, preprocessing versions, training status, validation metrics, artifact paths, prediction paths, environment notes, and failure reasons.
Inputs	Model config, training logs, split manifest, label-budget config, preprocessing metadata, artifact paths, and prediction paths.
Outputs	ModelRegistryRecord files such as <code>model_registry.csv/parquet</code> , <code>model_registry.yaml</code> , failed-run table, registry validation report, model cards.
Dependencies	Hard: model IDs and config hashes. Soft: environment hashing and model-card templates. Mockable: registry-only dummy entries for pipeline tests.
Consumers	All downstream layers, especially Layer 10 dashboard filters and reproducibility package.

Feedforward links	Supplies model metadata to Prediction Logger, Layer 3 model-wise calibration, Layer 4 disagreement grouping, Layer 5 model trust history, and Layer 10 filters/cards.
Feedback/loopback	If a downstream record references an unknown model ID, registry validation fails and the pipeline returns to Layer 2 record generation.
Evaluation	Registry completeness, one-to-one mapping between prediction files and registry entries, failed-run transparency, duplicate-ID detection, and config-hash consistency.
Failure modes	Orphan prediction files, duplicate model IDs, missing hyperparameters, unreported skipped runs, stale artifact paths, and irreproducible selected checkpoints.
Reproducibility	Registry files are versioned and checked by automated tests such as <code>test_model_registry.py</code> .
Thesis evidence	Reproducibility appendix, model cards, fair-comparison table, and traceability statements.
Limitations and risks	Registry discipline adds implementation overhead, but without it the downstream architecture cannot be audited.

8.10.6 Submodule 6: Ensemble comparison builder

Purpose. The Ensemble Comparison Builder creates ordinary ensemble baselines and disagreement features so that IHARQ is not mistaken for simple voting, averaging, confidence thresholding, or multi-window accumulation.

Bottleneck addressed. A reviewer may otherwise argue that IHARQ’s benefit is only the benefit of combining multiple decoders.

Aspect	Specification
Scientific role	Provides matched controls for claims that IHARQ evidence verification adds value beyond ordinary ensemble combination or simple rejection.
Engineering responsibility	Build majority-vote ensembles, probability-averaging ensembles, optional confidence-weighted ensembles, confidence-threshold baselines, longer-window/multi-window voting controls, and model-family disagreement features.
Inputs	Compatible <code>PredictionRecord</code> files, <code>ModelRegistryRecord</code> entries, class-order mappings, model-family group definitions, and low-calibration budget IDs.
Outputs	Ensemble <code>PredictionRecords</code> , <code>EnsembleControlRecord</code> outputs, disagreement table, ensemble baseline metrics, delay-burden notes, ensemble card, and matched comparison tables.
Dependencies	Hard: compatible class labels and aligned trial/window IDs across models. Soft: probability vectors for soft averaging. Optional: calibrated outputs from Layer 3 for later calibrated ensemble variants, clearly separated from raw Layer 2 ensemble controls. Mockable: synthetic predictions with known disagreement.
Consumers	Layer 3, Layer 4, Layer 10, thesis ablation chapter.

Feedforward links	Sends ensemble PredictionRecords to Layer 3; disagreement features and ordinary-combination baselines to Layer 4; ablation/control tables to Layer 10.
Feedback/loopback	If model alignment is incomplete, record missing-model conditions and restrict ensemble construction to declared compatible groups.
Evaluation	Single-best comparison, majority-vote performance, probability-averaging performance, confidence-threshold baseline, multi-window voting baseline, disagreement coverage, and matched comparison against IHARQ.
Failure modes	Misaligned windows, incompatible class ordering, missing probabilities, unfair inclusion of stronger models in ensembles, overinterpreting ensemble gains as evidence verification gains.
Reproducibility	Ensemble definitions, member model IDs, aggregation rule, class-order mapping, and source prediction paths are stored.
Thesis evidence	IHARQ ablation controls, ensemble comparison tables, disagreement feature distributions, and defense against “IHARQ is just voting” criticism.
Limitations and risks	Ensemble controls are not a replacement for IHARQ; they are comparison baselines. Their results should be reported honestly even if strong.

Required controls are single best decoder, majority vote, probability averaging, confidence-threshold rejection, longer-window or multi-window voting, and an ensemble disagreement score.

8.10.7 Submodule 7: Compact SSL adapter

Purpose. The Compact SSL Adapter wraps feasible compact self-supervised or foundation-style EEG encoders so they can participate in the same Layer 2 measurement contract as classical, Riemannian, and compact deep baselines.

Bottleneck addressed. Without an adapter boundary, stretch representation baselines could bypass Layer 1 protocols, write incompatible prediction files, or be overclaimed as a new foundation-model contribution.

Aspect	Specification
Scientific role	Provides an optional stronger representation baseline while preserving the thesis boundary that Layer 2 is a measurement spine, not a foundation-model training contribution.
Engineering responsibility	Validate checkpoint license, input compatibility, channel and sampling assumptions, feature extraction path, frozen/fine-tuned status, score type, and registry metadata before any predictions are exported.
Inputs	Layer 1 records, compact SSL or foundation-style checkpoint, adapter config, split manifest, label-budget config, and seed policy.
Outputs	SSL-derived features where retained, PredictionRecord files, ModelRegistryRecord entries, feasibility notes, skipped-run reasons, and stretch comparison tables.

Dependencies	Hard: valid Layer 1 records and declared adapter configuration. Optional/stretch: legal checkpoint, compatible input shape, enough compute, and reproducible feature extractor. Mockable: frozen dummy encoder with known output dimension.
Consumers	Baseline Trainer, Prediction Logger, Model-Family Registry, Layer 3 calibration, Layer 10 model cards and stretch comparison views.
Feedforward links	Sends adapter-produced predictions through the same PredictionRecord path as every other decoder family; sends feasibility and limitation notes to cards and reports.
Feedback/loopback	If checkpoint, license, channel layout, or sampling-rate assumptions are incompatible, mark the run as skipped or unsupported in ModelRegistryRecord rather than weakening Layer 1 protocol or silently changing preprocessing.
Evaluation	Compatibility pass rate, feature-shape validation, prediction-schema pass rate, reproducible rerun check, and fair comparison under identical splits and budgets.
Failure modes	Unsupported channel montage, hidden external training assumptions, license ambiguity, excessive compute, incompatible class labels, and overclaiming representation transfer as clinical or deployment evidence.
Reproducibility	Adapter configs, checkpoint identifiers, license notes, feature-extraction hashes, and rerun commands are stored with the model registry and model cards.
Thesis evidence	Optional stretch baseline tables and limitation notes that show whether stronger representation baselines change downstream reliability conclusions.
Limitations and risks	This adapter is optional/stretch. If infeasible, the architecture remains valid provided the skipped status and reason are reported.

8.10.8 Submodule 8: Downstream readiness validator

Purpose. The Downstream Readiness Validator checks that Layer 2 outputs can be consumed by their declared downstream layers without rerunning decoders or inventing hidden fields.

Bottleneck addressed. A prediction spine is only useful if later layers can read the records, resolve model identity, preserve split discipline, and distinguish decision-time inputs from evaluation-only labels.

Aspect	Specification
Scientific role	Protects downstream calibration, IHARQ, RegimeRisk, policy, stress, simulation, embodiment-provenance, and dashboard claims from being based on ad hoc decoder reruns or undocumented transformations.
Engineering responsibility	Validate schema version, required identity fields, split_role , label-budget IDs, model registry linkage, score availability, class order, evaluation-only fields, source paths, and consumer-specific minimum fields for Layers 3–10.

Inputs	PredictionRecord, ModelRegistryRecord, BaselineMetricRecord, LowCalibrationCurveRecord, SubjectProfileRecord, ensemble-control records, schema files, split manifests, and downstream consumer checklists.
Outputs	Layer2ReadinessReport, schema-compliance report, missing-field list, consumer-compatibility matrix, diagnostic-only downgrade list, and blocking-error list.
Dependencies	Hard: saved Layer 2 outputs and schema definitions. Soft: downstream validator stubs, dashboard-loader tests, and metric-dictionary checks. Mockable: synthetic records and intentionally malformed records.
Consumers	Layers 3–10, Layer 10 dashboard loader, evaluation scripts, repository tests, and final reproducibility package.
Feedforward links	Sends a pass/fail readiness matrix to downstream layers and Layer 10; exposes which consumers are ready, diagnostic-only, or blocked.
Feedback/loopback	If required fields, registry links, score types, or class-order mappings are missing, return to the producing Layer 2 submodule and repair or mark the output as unsupported before downstream evaluation.
Evaluation	Schema-compliance rate, consumer-readiness pass rate, missing-field count, registry-link pass rate, score-availability coverage, and correctness/evaluation-field separation checks.
Failure modes	Hidden alias fields, orphan predictions, missing <code>split_role</code> , missing class order, missing score type, test-label leakage into decision-time fields, and consumer-specific assumptions not represented in records.
Reproducibility	Readiness reports are versioned and linked to schema files, source record paths, config hashes, and downstream loader tests.
Thesis evidence	Demonstrates that downstream reliability and readiness claims are built from saved, auditable Layer 2 artifacts rather than opaque decoder state.
Limitations and risks	Passing readiness checks does not mean downstream layers are scientifically successful; it only certifies that Layer 2 provides the required auditable inputs.

8.11 Dependency logic

Layer 2 has the following dependency classes:

Dependency type	Examples	Rule
Hard dependencies	DatasetRecord, WindowRecord, SplitRecord/split manifest, label-budget config, preprocessing metadata, label map, seed policy	Must exist before Layer 2 can run. Missing hard dependencies stop the pipeline.

Soft dependencies	Specific decoder libraries, plotting utilities, GPU availability, optional environment hashing	Absence may reduce coverage but should not invalidate the architecture if documented.
Optional/stretch dependencies	Compact SSL/foundation checkpoints, external pretrained encoders, advanced deep models	Used only when feasible; otherwise marked as stretch/skipped with reasons.
Conceptual dependencies	Layer 0 claim-safety rules and Layer 1 public-data protocol	Bound what Layer 2 is allowed to claim and what data it may consume.
Data dependencies	Labels, channels, sampling rate, subject/session metadata, artifact flags	Validated before training and preserved in outputs.
Metric dependencies	Accuracy, balanced accuracy, F1, AUC where applicable, confusion matrix, subject-wise metrics	Computed with declared aggregation and split policy.
Software dependencies	Python environment, model libraries, schema validators, storage format support	Versioned or summarized in reproducibility notes.
Mockable dependencies	Tiny synthetic records, dummy model artifacts, synthetic prediction vectors	Used for unit tests, schema tests, and downstream integration before full training finishes.

Layer 2 must reach at least the record-contract stage before serious Layer 3 evaluation begins. It should reach downstream-ready status before thesis-level claims are made from Layers 3–10 using decoder-derived evidence.

8.12 Feedforward and loopback behavior

Layer 2 is primarily feedforward, but it has controlled validation loopbacks. It should not create arbitrary feedback loops that change scientific protocol after seeing test results.

Source	Target	Signal	Trigger	Action and safety constraint
Layer 2 validation	Layer 1 protocol	Split-leakage warning	Same trial/window appears across incompatible splits, missing split IDs, or label-budget leakage	Regenerate or repair split manifests before training; do not tune using test outcomes.

Baseline Trainer	Model Registry	Failed/skipped run status	Convergence failure, unsupported input shape, missing dependency, insufficient labels	Register failure explicitly; do not silently remove the model from denominator or coverage tables.
Prediction Logger	Layer 3 compatibility check	Missing probability/logit notice	A model outputs only class labels or decision scores	Mark <code>score_type</code> ; Layer 3 may use fallback calibration methods or exclude with documented reason.
Low-Calibration Builder	Layer 1/protocol config	Sparse-budget warning	Budget has too few samples or missing classes	Mark budget invalid/sparse or revise predeclared budget policy before final evaluation; no test-driven budget selection.
Subject Profiler	Layer 5	Difficulty and instability summary	Subject/session shows high variance, low-label fragility, or repeated error bursts	Use as prior risk context; do not relabel the subject clinically.
Ensemble Builder	Layers 3–4 and Layer 10	Disagreement features and ensemble controls	Multiple aligned model-family predictions exist	Use for calibration/disagreement features, IHARQ comparison, and dashboard ablations; do not conflate ensemble agreement with calibrated evidence quality.
Downstream readiness validator and Layer 10 dashboard checks	Layer 2 records	Traceability or consumer-compatibility error	Figure/table points or downstream loaders cannot be traced to registry and PredictionRecord paths, required fields, score type, or class order	Rebuild or exclude the artifact with explicit failure note; do not let downstream layers infer hidden decoder state.

The safety constraint for every loopback is that Layer 2 may correct invalid records, incomplete metadata, or incompatible artifacts, but it must not use held-out test outcomes to redesign splits, budgets, or hyperparameters after evaluation.

8.13 Evaluation design

Layer 2 is evaluated both as a decoder-performance layer and as an architecture-support layer.

Decoder metric	Purpose
Accuracy	Basic decoding performance.
Balanced accuracy	Robustness to class imbalance.
F1 score	Imbalanced-class and class-wise performance.
AUC where applicable	Binary or one-vs-rest separability analysis when valid.
Confusion matrix	Class-level error structure.
Subject-wise performance	Heterogeneity and subject reliability analysis.
Session-wise performance	Temporal/session variability.
Low-label performance	Calibration burden analysis.
High-confidence wrong rate at raw level	Pre-calibration warning signal; later interpreted by Layer 3.

Architecture-support metric	Purpose
PredictionRecord completeness	Ensures downstream modules can consume outputs.
Probability availability rate	Measures how often calibration and uncertainty analysis can use probability vectors.
Logit availability rate	Measures support for calibration methods requiring logits.
Model registry completeness	Measures traceability from result to model configuration.
Failed-run rate	Makes implementation limits visible.
Subject-profile coverage	Ensures subject difficulty analysis is possible.
Budget coverage	Ensures low-calibration claims are supported.
Ensemble-control coverage	Ensures IHARQ comparisons are fair.
Downstream compatibility pass rate	Measures whether Layers 3–10 can read their declared Layer 2 outputs or lineage without retraining or hidden field inference.

Required comparisons include majority-class or random sanity baselines, simple linear/logistic baselines where appropriate, CSP plus LDA/SVM-style pipelines, Riemannian classifiers, compact deep EEG models where feasible, compact SSL/foundation-style encoders where feasible, single best decoder, majority-vote ensemble, probability-averaging ensemble, confidence-threshold baseline, and longer-window or multi-window voting baseline.

Layer 2 aligns these comparisons with the system-level ablation ladder rather than creating a separate local ladder. A0 corresponds to raw decoder predictions and baseline metric tables. A4-style ordinary-combination controls correspond to longer windows, voting, averaging, ensembles, and multi-window aggregation. Layer 2 also supplies the raw confidence fields used by Layer 3 for A1–A3 calibration, threshold, and selective-prediction comparisons, but it does not interpret those fields as calibrated reliability.

Ablations should separately test model-family removal, probability/logit availability, label-budget choice, subject-profile aggregation, ensemble-control inclusion, longer-window delay burden, and window-to-trial aggregation rule. These ablations must be matched by dataset, subject/session, split, preprocessing, label map, low-calibration budget, seed, model family where applicable, and aggregation unit; unmatched comparisons are diagnostic only. Stress-conditioned use of Layer 2 outputs belongs mainly to Layer 8, but Layer 2 must preserve enough identifiers and time indices to make stress replay possible.

Success criteria for Layer 2 are not defined only by high accuracy. A thesis-ready Layer 2 succeeds when it produces fair, complete, traceable, reusable, and downstream-compatible prediction evidence. Failure is interpreted defensively: poor decoder performance, high subject variability, missing probabilities, or failed runs are not hidden; they become evidence about limitations and reliability risk.

8.14 Reproducibility artifacts and repository placement

Layer 2 should produce a reproducibility trail from input records to final figures.

Artifact	Expected path or role
Decoder configs	<code>configs/decoders/</code> with subfolders for sanity, CSP/LDA, CSP/SVM, Riemannian, EEGNet/TCN, compact SSL, and ensembles.
Layer 2 experiment configs	<code>configs/layer2_experiments/label_budgets.yaml</code> , <code>split_policy.yaml</code> , <code>seed_policy.yaml</code> .
Prediction schema	<code>records/prediction_records/prediction_schema.yaml</code> .
Prediction records	<code>records/prediction_records/</code> in CSV or Parquet, preferably Parquet for large per-window logs.
Model registry	<code>records/model_registry/</code> plus <code>model_registry_schema.yaml</code> and registry validation report.
Low-calibration outputs	<code>records/low_calibration_curves/</code> and <code>figures/layer2_low_calibration/</code> .
Subject profiles	<code>records/subject_profiles/</code> and <code>figures/layer2_subject_profiles/</code> .
EnsembleControlRecord outputs	<code>records/ensemble_records/</code> .
Model artifacts	<code>models/layer2_decoders/</code> .
Training and inference logs	<code>logs/training_logs/</code> , <code>logs/inference_logs/</code> , and <code>logs/failed_runs/</code> .
Cards	<code>cards/model_cards/</code> and <code>cards/layer2_benchmark_cards/</code> .
Tests	<code>tests/test_prediction_record_schema.py</code> , <code>test_split_integrity.py</code> , and <code>test_model_registry.py</code> .

Every thesis figure or table derived from Layer 2 should identify source PredictionRecord paths, registry version, split ID, budget ID, model IDs, aggregation rule, metric definition, and generated figure/table path. Results should appear in the thesis as baseline tables, low-calibration curves, subject difficulty plots, ensemble-control ablations, model cards, and reproducibility appendices.

8.15 Implementation phases and readiness levels

Layer 2 should be implemented after Layer 1 produces stable records and before Layers 3–5 depend on predictions.

Order	Task	Exit artifact
1	Define <code>PredictionRecord</code> schema.	<code>prediction_schema.yaml</code> .
2	Define model registry schema.	<code>model_registry_schema.yaml</code> .
3	Implement sanity baselines.	Sanity <code>PredictionRecords</code> .
4	Implement classical CSP/LDA/SVM baseline.	Classical baseline records and metrics.
5	Implement Riemannian baseline.	Riemannian records and metrics.
6	Implement compact deep model if feasible.	Deep model records and metrics.
7	Implement compact SSL/foundation-style baseline if feasible.	Stretch SSL records and documented feasibility limits.
8	Implement prediction logger.	Standardized <code>PredictionRecord</code> outputs.
9	Implement low-calibration curve builder.	Budget curves and tables.
10	Implement subject difficulty profiler.	Subject profiles and difficult-subject index.
11	Implement ensemble comparison builder.	Ensemble controls and disagreement table.
12	Validate outputs for downstream layers.	Layers 3–10 readiness report with consumer-specific pass/block status.

Level	Name	Meaning
L2-RL0	Concept only	Layer 2 is only described as a decoder layer.
L2-RL1	Boundary defined	Scope, non-scope, incoming links, outgoing links, and claim boundaries are defined.
L2-RL2	Record contracts defined	<code>PredictionRecord</code> and model registry schemas exist.
L2-RL3	Sanity baselines implemented	Majority/simple baselines run and produce records.
L2-RL4	Core baselines implemented	Classical and Riemannian baselines run under fixed protocols.
L2-RL5	Low-calibration curves produced	Label-budget experiments produce thesis-ready curves.
L2-RL6	Subject profiles produced	Easy/difficult/unstable subject profiles exist.
L2-RL7	Ensemble controls produced	Voting, averaging, threshold, multi-window, and disagreement baselines exist.
L2-RL8	Downstream-ready	Layers 3–10 can consume their declared Layer 2 outputs or lineage without rerunning decoders.
L2-RL9	Thesis-ready	Figures, tables, cards, limitations, and defense statements are written.

Layer 2 should not feed Layer 3 in serious evaluation until at least L2-RL2. It should not support thesis-level claims until at least L2-RL8.

8.16 Outputs

Layer 2 outputs trained model artifacts, `PredictionRecord` files, `ModelRegistryRecord` entries, `BaselineMetricRecord` accuracy/balanced-accuracy/F1/AUC tables where applicable, confusion matrices, `LowCalibrationCurveRecord` outputs, `SubjectProfileRecord` outputs, `EnsembleControlRecord` outputs, model-family disagreement tables, training logs, inference logs, `FailureCaseIndex` entries, `ModelCard` outputs, `BenchmarkCard` outputs, and `Layer2ReadinessReport`.

8.17 Outgoing links

Layer 2 feeds Layer 3 with predicted classes, raw probability vectors, logits when available, decision scores, split IDs, split roles, model IDs, label-budget IDs, score type, class order, raw confidence helpers, and evaluation-only correctness helpers. It feeds Layer 4 with `PredictionRecord` files, model-family disagreement, window-level prediction history, ensemble outputs, and raw error indicators. It feeds Layer 5 with time-indexed prediction logs, recent performance histories, `SubjectProfileRecord` outputs, model-family performance variation, and error-burst indicators. It feeds Layer 6 primarily through calibrated, uncertainty, IHARQ, and `RegimeRisk` records derived from Layer 2 outputs while preserving decoder identifiers and raw decoder fields for `ReadinessContextRecord` traceability. It feeds Layer 7 and Layer 8 replay-style evaluations with prediction histories that can be replayed, degraded, or stress-conditioned. It feeds Layer 9 as a provenance/input candidate alongside downstream readiness and policy records, never as a direct embodiment-safety claim. It feeds Layer 10 with benchmark tables, low-calibration figures, subject profiles, model registry views, prediction logs, ensemble comparison tables, model cards, and reproducibility artifacts.

8.18 Definition of done

Layer 2 is complete when per-window and per-trial `PredictionRecord` files exist for all selected datasets, subjects, sessions where available, splits, label budgets, seeds, and required model families; when every prediction file can be traced to a `ModelRegistryRecord`; when low-calibration curves, subject profiles, ensemble controls, longer-window or multi-window controls, delay-burden notes, and failed-run logs are generated; when A0 and A4-style baseline evidence can be matched against Layer 3 calibration/selective-prediction records without hidden protocol changes; when downstream schema and lineage checks for Layers 3–10 pass according to their declared consumption paths; and when all Layer 2 claims are written as measurement and reproducibility claims rather than clinical or deployment claims.

9 Layer 3: Calibration, Uncertainty, and Selective Prediction

9.1 Problem solved by Layer 3

A decoder probability is not automatically trustworthy. A model can be confident and wrong, especially under low calibration, subject variability, session shift, nonstationarity, channel instability, noisy windows, class ambiguity, and model overconfidence. Layer 3 addresses the bottleneck between **raw decoder scores** and **readiness-usable evidence**: before IHARQ, Adaptive-IHARQ, learning-to-defer, contextual bandit, reinforcement-learning, or dashboard logic can use a prediction, the prediction must be transformed into calibrated, uncertainty-aware, traceable, and split-safe evidence.

Layer 3 is therefore the architecture’s **trust-conversion layer**. It converts Layer 2 `PredictionRecord` files into `CalibrationRecord`, `UncertaintyRecord`, `SelectivePredictionRecord`, `ThresholdRegistryRecord`, and reliability-audit artifacts. It determines whether probabilities are calibrated, whether uncertainty features are meaningful, how much coverage can be retained under rejection, and which thresholds are fair baselines for IHARQ and Adaptive-IHARQ comparisons.

The thesis role of this layer is to protect the central reliability claim from the criticism that the system merely trusts raw confidence. Without Layer 3, the architecture would have predictions but not decision-grade reliability evidence; Layer 4 would risk becoming confidence thresholding on uncalibrated scores; Layer 6 would learn policies from unstable or unfair threshold choices; and Layer 10 would show plots that are not traceable to formal records. Layer 3 supports the thesis claim that readiness decisions are built on auditable uncertainty evidence rather than on opaque decoder outputs.

Layer 3 also owns the calibration, uncertainty, threshold, and selective-prediction slice of the ablation architecture. It supplies the system-level A1–A3 comparisons that determine whether calibrated confidence, simple confidence thresholds, entropy/margin rejection, and risk-coverage baselines already solve the wrong-accept problem before IHARQ, RegimeRisk, or adaptive policies are credited with added value. This makes Layer 3 both a functional trust-conversion layer and a fairness gate for later claims.

The safe claim boundary is strict: Layer 3 does *not* claim that calibration guarantees safety, clinical readiness, or correct action. A calibrated model can still be wrong. Layer 3 only makes decoder uncertainty measurable, reproducible, and usable by downstream evidence-verification and policy layers. Calibration failure is reported rather than hidden, because failure pockets are themselves evidence for why IHARQ, RegimeRisk, and adaptive readiness mechanisms are needed.

9.2 Architectural role and boundaries

Layer 3 sits between Layer 2 and the readiness layers:

Layer 2 raw prediction outputs → Layer 3 calibration and uncertainty transformation →
Layer 4 evidence verification → Layer 6 adaptive readiness policies → Layer 10 dashboard and
thesis artifacts.

The layer is in the measurement plane with direct support for the evidence, policy, and dashboard planes. Its technical responsibility is to calibrate scores when feasible, compute uncertainty features, audit reliability, generate selective-prediction baselines, register thresholds, and export structured records. It is not responsible for training decoders, deciding final accept/wait/fallback/unsafe actions, learning adaptive policies, simulating embodiment, or making deployment claims.

Boundary	In scope for Layer 3	Out of scope or downstream
Decoder scores	Calibrate raw probabilities/logits and document score limitations.	Training model weights or changing the decoder objective; this remains Layer 2.

Uncertainty	Compute confidence, entropy, margin, variation ratio, disagreement, conformal proxies, and label-free uncertainty profiles.	Deciding final readiness actions; this remains Layer 4 or Layer 6.
Selective prediction	Build rejection/risk-coverage baselines and operating points for A2/A3 comparisons.	Treating selective prediction as the full IHARQ system.
Thresholds	Select, version, and register thresholds using validation-only or nested-CV rules and preserve matched-coverage or matched-cost comparison points.	Hidden test-set tuning or undocumented downstream thresholds.
Reliability audit	Report aggregate and group-wise calibration quality, including failures, high-confidence wrong cases, and low-calibration instability.	Hiding poor calibration or claiming calibration is equivalent to safety.
Dashboard export	Produce plot-ready records, tables, and cards.	Building a production dashboard; Layer 10 handles presentation.

Core submodules are the probability calibrator, uncertainty extractor, reliability audit, selective-prediction baseline, threshold registry, and split/leakage guard. Supporting submodules include group-wise audit, dashboard export, calibration-budget analysis, and high-confidence error mining. Optional or stretch components include conformal prediction-set proxies, model-disagreement features when ensembles exist, subject/session-aware diagnostic calibration, and temporal summaries for Layer 5 RegimeRisk. Future work includes online recalibration and deployment-time monitoring; these should not be claimed as completed unless explicitly implemented.

9.3 Input contract and upstream dependencies

Layer 3 consumes structured outputs from Layer 2 and metadata from Layer 1. It must not operate on anonymous arrays unless those arrays can be traced back to dataset, subject, session, split, model, seed, and configuration records. For ablation-ready evaluation, the same traceability must also support matched coverage, matched cost, matched label budget, matched threshold-selection split, and matched aggregation unit comparisons.

Input type	Required content	Producer	Layer 3 usage
------------	------------------	----------	---------------

Primary input	PredictionRecord: dataset ID, subject ID, session ID where available, trial/window IDs, split ID, canonical split_role , decoder ID, model ID/model family, predicted_class , probability_vector or declared score fallback, logits or decision scores when available, score_type , class_order , timestamp_or_order where temporal analysis is planned, label_budget_id , evaluation-only label_true for calibration/audit splits, seed, config hash, and correctness helper fields only when allowed and mapped to canonical exported names.	Layer 2	Calibration, uncertainty extraction, selective prediction, traceability.
Auxiliary input	DatasetRecord and WindowRecord: task labels, channels, sampling rate, preprocessing metadata, artifact flags, label map, window timing, trial/window linkage.	Layer 1	Group-wise calibration, reliability cards, window/trial aggregation.
Metadata input	ModelRegistryRecord: schema version, model family, hyperparameters, training seed, checkpoint path, score_type , class_order , label_map_version , training split and split_role , failed-run status, optional ensemble_membership , and registry schema path.	Layer 2	Per-model calibration, model-family comparison, reproducibility, score-schema validation.
Split input	SplitManifest: train/calibration/threshold-validation/test membership, split seed, subject/session split logic, nested-CV fold IDs if used.	Layer 1 or experiment controller	Leakage prevention, calibration fitting, threshold fairness.
Evaluation-only input	Labels and correctness fields for calibration, validation, audit, and final testing.	Layer 1/2	Calibration fitting and evaluation only; never used as inference-time features.

Proxy or stretch input	Ensemble member predictions, repeated-window histories, stress labels, temporal order, regime annotations, conformal calibration sets.	Layers 2, 5, 8, or experiment controller	Disagreement, temporal summaries, stress-conditioned reliability, conformal proxies when feasible.
------------------------	--	--	--

The minimum required `PredictionRecord` fields are: `dataset_id`, `subject_id`, `session_id` where available, `trial_id`, `window_id` for windowed decoding, `split_id`, `split_role`, `decoder_id`, `model_id`, `model_family`, `predicted_class`, `probability_vector` unless a declared logits-only or decision-score fallback exists, `score_type`, `class_order`, `logits` when available, `timestamp_or_order` when temporal analysis is planned, `label_budget_id`, `seed`, and `config_hash`. `label_true` is required for calibration, threshold-validation, audit, and evaluation splits but must remain evaluation-only and must not be exposed as a decision-time feature. Local implementation aliases such as `y_true`, `y_pred`, and `prob_vector` may remain inside scripts only if they are explicitly mapped to `label_true`, `predicted_class`, and `probability_vector` before cross-layer export.

Before calibration or uncertainty extraction begins, Layer 3 performs input validation: probability values must be finite and within $[0, 1]$ after conversion; probability vectors must sum to one within tolerance; probability-vector length must match the `class_order`; `label_true` where available, `predicted_class`, and `probability_vector` must use the same label-map version and class order; calibration, threshold-validation, and final test roles must be disjoint unless a declared cross-validation protocol is used; missing probabilities or declared score fallbacks, labels required for evaluation, model IDs, `split_role`, `score_type`, `class_order`, or config hashes must be flagged; hidden aliases such as `y_true`, `y_pred`, or `prob_vector` must be rejected unless mapped to canonical exported names; and records from invalid or incomplete model runs must be excluded or marked with a documented reason.

9.4 Split discipline and leakage guard

Layer 3 has a non-negotiable test-set lockout rule. Calibration parameters, method selection, threshold selection, operating-point rules, and policy features must not be tuned using final test labels. The allowed split roles are:

Split role	Allowed use in Layer 3
Training split	Decoder training and optional raw diagnostic context. It is not the primary source for final calibration-quality claims.
Calibration split	Fits calibration parameters such as temperature scaling, Platt scaling, isotonic mapping, vector scaling, or a declared no-calibration baseline.
Threshold-validation split	Selects selective-prediction thresholds, target coverage points, risk constraints, matched-coverage or matched-cost operating points, and threshold registry entries.
Test split	Final locked evaluation only. Test labels can compute final metrics but must not choose calibration parameters or thresholds.

Stress/shift split	Evaluates declared perturbations, temporal shifts, or regime conditions without changing the locked selection rule.
Nested-CV folds	Used when data are small; fold roles must still separate fitting, selection, and evaluation within each repeat.

The leakage guard is a core supporting submodule. It produces a `SplitIntegrityReport`, `LeakageWarningRecord`, and `CalibrationEligibilityTable`. If calibration and test membership overlap without a declared cross-validation protocol, the run is invalid for final thesis claims. If thresholds are selected on final test labels, the threshold record must be marked as an oracle diagnostic only and excluded from deployable or fair-baseline comparisons.

9.5 Output contract and downstream consumers

Layer 3 outputs records, not hidden variables. Every output must have a downstream consumer, a source configuration, and a regeneration path.

Output record or artifact	Core content	Consumer
<code>CalibrationRecord</code>	Raw and calibrated probabilities, raw and calibrated confidence, calibration method, calibration scope, calibration split, reliability-bin membership, correctness, Brier/NLL components, config hash.	Layers 4, 6, 10; optional Layer 5 summaries.
<code>UncertaintyRecord</code>	Confidence, entropy, normalized entropy, margin, variation ratio, top-1/top-2 classes, label-free uncertainty profile, disagreement, conformal set size if used, selective accept flag.	Layers 4, 6, 10; optional Layer 5 summaries.
<code>SelectivePredictionRecord</code>	Threshold ID, uncertainty feature, selection split, evaluation split, coverage, rejection rate, retained accuracy, retained error rate, risk, high-confidence wrong rate, class/subject/session coverage, calibration budget.	Layers 4, 6, 10, ablation chapter.
<code>ThresholdRegistryRecord</code>	Threshold value or rule, feature, selected-on split, selection rule, target constraint, applicable models, allowed consumers, version, leakage status, rationale, fairness note.	Layers 4, 6, 10; experiment controller.
<code>ReliabilityAuditReport</code>	Reliability diagrams, ECE-style summaries, Brier score, NLL, confidence histograms, high-confidence wrong tables, class/subject/session/model-family audits.	Layer 10 and thesis evaluation chapters.

CalibrationDashboard	Historical metrics, bin tables, curves, filters, warning flags, and card references.	Layer 10 dashboard backend.
SplitIntegrityReport	Split membership checks, leakage warnings, eligibility tables, invalid-run flags.	Experiment controller, Layer 10 reproducibility view.

The canonical `CalibrationRecord` should include `record_id`, `dataset_id`, `subject_id`, `session_id`, `trial_id`, `window_id`, `split_id`, `decoder_id`, `model_family`, `raw_prob_vector`, `calibrated_prob_vector`, `raw_confidence`, `calibrated_confidence`, `calibration_method`, `calibration_scope`, `calibration_split_id`, `ece_bin_id`, `correctness`, `brier_component`, `nll_component`, and `config_hash`.

The canonical `UncertaintyRecord` should include `record_id`, `linked_prediction_record_id`, `linked_calibration_record_id`, `confidence`, `entropy`, `normalized_entropy`, `margin`, `variation_ratio`, `top1_class`, `top2_class`, `top1_top2_margin_label_pair`, `model_disagreement`, `conformal_set_size`, `selective_accept_flag`, `uncertainty_profile`, and `config_hash`.

The canonical `ThresholdRegistryRecord` should include `threshold_id`, `threshold_type`, `feature`, `selected_value`, `selected_on`, `selection_rule`, `target_constraint`, `applicable_models`, `allowed_consumers`, `version`, `leakage_status`, `rationale`, and `fairness_note`. No threshold may appear in Layer 4, Layer 6, or Layer 10 without a registry entry.

9.6 Submodules

Submodule	Function	Output
Probability calibrator	Applies no-calibration baselines, temperature scaling, isotonic calibration, Platt scaling, vector scaling, or another declared feasible method using split-safe calibration data.	<code>CalibrationRecord</code> , calibration parameter files, calibration method report.
Reliability audit	Computes ECE-style calibration error, Brier score, NLL, reliability diagrams, confidence histograms, high-confidence wrong rates, and group-wise calibration tables.	<code>ReliabilityAuditReport</code> , calibration figures and tables.
Uncertainty extractor	Computes confidence, entropy, normalized entropy, margin, variation ratio, top-2 class pair, disagreement, conformal proxies when used, and label-free uncertainty profiles.	<code>UncertaintyRecord</code> .

Selective prediction baseline	Measures reject/coverage trade-offs using raw confidence, calibrated confidence, entropy, margin, probability-gap, conformal set-size, and clearly marked oracle baselines when used only for diagnostics.	<code>SelectivePredictionRecord</code> , risk-coverage and accuracy-coverage curves.
Threshold registry	Stores threshold choices so IHARQ, Adaptive-IHARQ, policy-learning, and dashboard comparisons are fair, versioned, and leakage-controlled.	<code>ThresholdRegistryRecord</code> , threshold configuration files.
Calibration split and leakage guard	Validates split roles, calibration eligibility, threshold selection legality, and test-set lockout.	<code>SplitIntegrityReport</code> , <code>LeakageWarningRecord</code> .
High-confidence error miner	Finds confident wrong predictions and clusters them by subject, session, class pair, model family, budget, stress condition, and regime where available.	<code>HighConfidenceWrongTable</code> , <code>ErrorClusterReport</code> , downstream failure seed list.
Dashboard and card exporter	Converts formal Layer 3 records into plot-ready metric files and reliability-card inputs.	<code>CalibrationDashboardMetricRecord</code> , reliability-card snippets.

9.6.1 Submodule 1: Probability calibrator

Purpose. The probability calibrator converts raw decoder probabilities, logits, or declared score proxies into probabilities whose confidence values better reflect empirical correctness. Its goal is not to improve classification accuracy directly; its goal is to make confidence values more meaningful for downstream evidence verification.

Bottleneck addressed. Raw EEG decoder probabilities can be overconfident or underconfident under low calibration, model mismatch, nonstationarity, subject variability, and noisy windows. Downstream readiness logic must not confuse high softmax probability with trust.

Scientific role. The calibrator tests whether decoder families differ not only in accuracy but also in probability reliability. It supports thesis questions about whether SSL, deep, Riemannian, classical, and ensemble decoders produce usable uncertainty under low-label conditions.

Engineering responsibility. Fit calibration mappings using only eligible calibration data, apply those mappings to validation/test records without altering the decoder prediction history, save parameters, and export traceable `CalibrationRecord` files.

Item	Specification
Inputs	<code>PredictionRecord</code> , <code>SplitManifest</code> , label map, probability vector or logits, calibration method configuration, model registry metadata.
Outputs	Calibrated probability vector, <code>CalibrationRecord</code> , calibration parameter file, calibration method report, before/after metric table.

Processing logic	Validate probabilities and labels; choose declared calibration scope; fit method on calibration split; apply mapping to non-fitting splits; preserve raw scores; compute raw and calibrated confidence; save method parameters and config hash.
Dependencies	Hard: valid probabilities/logits or declared score conversion, label map, calibration split. Soft: logits for temperature scaling, enough labels per class, model-family metadata. Optional: nested-CV folds, subject/session diagnostics. Mockable: synthetic predictions with known calibration error.
Consumers	Uncertainty extractor, reliability audit, selective-prediction baseline, Layers 4, 6, and 10.
Feedforward	Sends calibrated confidence and calibrated probability vectors to uncertainty extraction, IHARQ evidence verification, policy context, dashboard views, and reliability cards.
Feedback/loopback	If a method overfits or fails under sparse labels, emit a method-warning record to the experiment controller and fall back to the no-calibration or simpler declared method; do not tune on the test set.
Evaluation	Compare no calibration, temperature scaling, Platt scaling, isotonic calibration, vector scaling, and pooled/per-model scopes where feasible using ECE-style error, Brier score, NLL, reliability diagrams, and high-confidence wrong rate.
Failure modes	Test leakage, sparse calibration overfitting, class-missing calibration splits, logits unavailable, probability vectors not normalized, subject-level failures hidden by global calibration.
Reproducibility	Save calibration config, method parameters, split IDs, random seeds, class order, software versions, and mapping from each output record to source prediction file.
Thesis evidence	Raw-vs-calibrated reliability diagrams, calibration-budget curves, per-model calibration table, and limitations for low-label settings.
Limitations and risks	Calibration improves probability interpretation but does not guarantee correctness or safety; per-subject and per-session calibration are diagnostic unless enough split-safe data exist.

Recommended calibration modes are: no-calibration baseline; temperature scaling when logits or compatible scores exist; Platt scaling for binary or one-vs-rest settings; isotonic calibration when enough data exist and overfitting is monitored; per-model calibration when data allow; pooled/global calibration as a low-label baseline; and subject/session-aware calibration only as diagnostic or clearly justified analysis.

9.6.2 Submodule 2: Reliability audit

Purpose. The reliability audit measures whether probabilities and uncertainty scores are trustworthy enough to support downstream readiness decisions and exposes where they fail.

Bottleneck addressed. Aggregate accuracy can hide high-confidence errors, class-specific overconfidence, subject-specific miscalibration, session shift, and calibration collapse at low label budgets.

Scientific role. The audit produces the evidence that Layer 3 is not cosmetic. It answers whether raw probabilities are calibrated, whether calibration improves reliability, whether some models are accurate but poorly calibrated, and whether failures concentrate by subject, session, task, model family, stress condition, or calibration budget.

Engineering responsibility. Compute repeatable calibration metrics and tables from `CalibrationRecord` and `UncertaintyRecord` files; generate figures and plot-ready records; preserve failed or poor calibration cases rather than deleting them.

Item	Specification
Inputs	<code>CalibrationRecord</code> , <code>UncertaintyRecord</code> , labels for evaluation splits, dataset/subject/session metadata, model registry, calibration-budget IDs.
Outputs	<code>ReliabilityAuditReport</code> , reliability diagrams, ECE-style summaries, Brier/NLL tables, confidence histograms, high-confidence wrong table, class/subject/session/model-family audits.
Processing logic	Bin predictions by confidence; compare bin confidence with empirical accuracy; compute proper scoring rules; stratify by dataset, subject, session, class, model family, budget, and stress; export warning flags when reliability is poor.
Dependencies	Hard: calibrated or raw probability vectors, correctness labels for evaluation, group metadata. Soft: enough samples per bin/group, repeated seeds. Optional: temporal order and stress labels. Mockable: toy probability vectors with known calibration.
Consumers	Layer 10 dashboard, thesis evaluation chapter, Layer 4 failure analysis, Layer 6 ablation analysis, optional Layer 5 trend summaries.
Feedforward	Sends reliability metrics to dashboard cards, high-confidence wrong examples to IHARQ failure analysis, and group-wise warnings to evaluation reports.
Feedback/loopback	If a group-wise audit shows severe calibration failure, mark the affected model/budget/group as high risk for downstream ablation; do not silently remove it unless the exclusion rule was predeclared.
Evaluation	Before/after calibration comparisons, group-wise tables, confidence histograms with correctness overlays, calibration-budget curves, and high-confidence wrong rate at fixed confidence levels.
Failure modes	Bin instability under small samples, misleading aggregate ECE, poor handling of imbalanced classes, dashboard-only plots without source records.
Reproducibility	Store bin definitions, group filters, metric formulas, bootstrap/repeat settings, seeds, and source record pointers for every figure.
Thesis evidence	Demonstrates whether calibration and uncertainty are necessary, where they succeed, and where downstream readiness logic must compensate.
Limitations and risks	Calibration metrics summarize probability reliability; they do not alone prove clinical safety or deployment readiness.

9.6.3 Submodule 3: Uncertainty extractor

Purpose. The uncertainty extractor converts every prediction and calibration record into label-free uncertainty features that downstream modules can use at inference or replay time, while keeping label-dependent diagnostics clearly separated.

Bottleneck addressed. Downstream readiness layers need more than top-1 probability. Ambiguity can appear as high entropy, small top-1/top-2 margin, unstable windows, model disagreement, or large prediction sets even when top-1 confidence is not low.

Scientific role. The extractor enables research questions about which uncertainty signals predict errors, how uncertainty changes under low calibration budgets, and whether richer uncertainty features improve Layer 4/6 decisions compared with raw confidence alone.

Engineering responsibility. Compute reproducible formulas, define feature directionality, distinguish raw and calibrated sources, handle missing logits or missing ensemble data, and export standardized `UncertaintyRecord` files.

Item	Specification
Inputs	<code>PredictionRecord</code> , <code>CalibrationRecord</code> , model registry, window/trial IDs, ensemble predictions when available, conformal calibration data when used.
Outputs	<code>UncertaintyRecord</code> , feature summary tables, uncertainty distributions, label-free feature matrix for Layers 4 and 6.
Processing logic	Select raw or calibrated probability source; compute confidence, entropy, normalized entropy, margin, variation ratio, top-2 label pair, disagreement, conformal set size/proxy; aggregate window-level features to trial-level summaries when needed; mark label-dependent fields as evaluation-only.
Dependencies	Hard: probability vector or calibrated probability vector, class count, label map. Soft: calibrated probabilities, logits, repeated windows, ensemble outputs. Optional: conformal calibration set. Mockable: probability vectors with known entropy/margin behavior.
Consumers	Layer 4 IHARQ, Layer 6 policy context, Layer 10 uncertainty dashboards, optional Layer 5 rolling uncertainty trends.
Feedforward	Sends calibrated confidence, entropy, margin, variation ratio, disagreement, and profiles into <code>ReadinessContextRecord</code> construction.
Feedback/loopback	If a feature is missing or non-comparable for a model family, export a missing-feature warning and prevent downstream policies from treating that missing value as real evidence.
Evaluation	Error-detection analysis, risk-coverage curves per feature, feature correlation with correctness, downstream ablations using raw confidence vs calibrated confidence vs full uncertainty set.
Failure modes	Mixing label-derived features into inference-time policy context, inconsistent class ordering, unstandardized feature scales, missing ensemble data, unstable window aggregation.
Reproducibility	Define formulas, raw/calibrated source, feature direction, aggregation level, missing-value behavior, and config hash.

Thesis evidence	Shows whether uncertainty features carry information beyond accuracy and whether downstream readiness layers benefit from them.
Limitations and risks	Features such as high-confidence wrong flags require labels and are evaluation-only; conformal proxies are optional and must not be overclaimed if implemented approximately.

Required label-free features include calibrated confidence, raw confidence, entropy, normalized entropy, margin, variation ratio, top-1 class, top-2 class, top-1/top-2 pair, and uncertainty profile. Optional features include model disagreement, conformal set size or proxy, rolling confidence trend, rolling entropy trend, and window-consistency summaries. Evaluation-only fields include correctness, high-confidence wrong flag, empirical bin accuracy, and proxy reliable/unreliable labels.

9.6.4 Submodule 4: Selective-prediction baseline

Purpose. The selective-prediction baseline is the fairness anchor for IHARQ and Adaptive-IHARQ. It measures how much reliability can be gained by simply rejecting low-confidence or high-uncertainty predictions before introducing richer readiness actions.

Bottleneck addressed. A reviewer could argue that IHARQ is merely confidence thresholding. Layer 3 prevents this by constructing strong confidence, calibrated-confidence, entropy, margin, probability-gap, and conformal-set-size baselines before downstream readiness claims are made.

Scientific role. Selective prediction quantifies the risk-coverage trade-off and defines how much coverage must be sacrificed to reduce wrong accepted decisions. This gives a rigorous baseline for later IHARQ, deferral, bandit, and RL comparisons.

Engineering responsibility. Generate threshold grids, risk-coverage curves, retained-accuracy curves, rejection-error curves, operating points, and `SelectivePredictionRecord` files using validation-selected thresholds and locked evaluation splits.

Item	Specification
Inputs	<code>CalibrationRecord</code> , <code>UncertaintyRecord</code> , threshold-validation split, evaluation split, calibration-budget metadata, cost definitions if available.
Outputs	<code>SelectivePredictionRecord</code> , risk-coverage curve, accuracy-coverage curve, rejection-error curve, subject-wise coverage curve, operating-point report.
Processing logic	Choose uncertainty score; sweep thresholds or apply registered operating-point rule; compute coverage, rejection rate, retained accuracy, retained error, risk, high-confidence wrong rate, and class/subject/session breakdowns; lock thresholds before test evaluation.
Dependencies	Hard: uncertainty scores, correctness labels for validation/evaluation, split roles. Soft: enough validation samples, cost target, calibration budget schedule. Optional: oracle diagnostic threshold clearly marked non-deployable. Mockable: synthetic uncertainty/correctness pairs.
Consumers	Layer 4 baseline comparison, Layer 6 policy baselines, Layer 10 risk-coverage dashboard, ablation chapter.

Feedforward	Sends operating points and baseline curves to IHARQ and policy comparison; sends accepted/rejected flags for controlled ablations.
Feedback/loopback	If a threshold produces unacceptable coverage collapse or subject-wise rejection imbalance, record the failure and require downstream comparisons to report matched coverage/cost rather than cherry-pick thresholds.
Evaluation	Risk at fixed coverage, coverage at target risk, retained accuracy, rejection rate, high-confidence wrong rate, subject/class/session coverage, and calibration-budget sensitivity.
Failure modes	Test-tuned thresholds, weak raw-confidence-only baseline, over-rejection that makes accuracy look high but coverage useless, unfair subject/session rejection patterns.
Reproducibility	Save threshold grid, selection rule, validation split, evaluation split, feature source, calibration method, budget ID, and config hash.
Thesis evidence	Establishes whether IHARQ and Adaptive-IHARQ outperform simple rejection under matched coverage, risk, or cost.
Limitations and risks	Selective prediction is a baseline, not the full IHARQ system; it predicts or rejects but does not request evidence, combine windows, fallback, or declare unsafe.

9.6.5 Submodule 5: Threshold registry

Purpose. The threshold registry prevents hidden tuning and inconsistent threshold usage. Every threshold used by Layer 3, Layer 4, Layer 6, or Layer 10 must have a versioned registry entry.

Bottleneck addressed. Threshold choices can change results dramatically. Without a registry, the thesis can be criticized for arbitrary thresholds, unfair IHARQ comparisons, or accidental test leakage.

Scientific role. The registry makes selective-prediction, IHARQ, Adaptive-IHARQ, and policy baselines auditable. It clarifies whether thresholds are global, per-model, per-family, per-budget, subject-diagnostic, stress-specific, or oracle-only.

Engineering responsibility. Store threshold values or rules, selection split, target constraint, allowed consumers, version, leakage status, rationale, and fairness note; block downstream modules from using unregistered thresholds in final evaluation.

Item	Specification
Inputs	Selective-prediction results, validation metrics, target coverage/risk/cost rules, downstream consumer declarations.
Outputs	<code>ThresholdRegistryRecord</code> , <code>threshold_registry.yaml</code> , threshold summary table, dashboard threshold view.
Processing logic	Assign threshold ID; store feature and value/rule; record selected-on split; define target constraint; define applicable models and consumers; record leakage status; version and hash the registry.

Dependencies	Hard: split-safe selection results, feature definitions, downstream permission list. Soft: cost definitions, model-family grouping. Optional: stress-specific and subject-diagnostic thresholds. Mockable: hand-written threshold registry for integration tests.
Consumers	Layer 4 IHARQ ablations, Layer 6 policy baselines, Layer 10 dashboard, experiment runner.
Feedforward	Supplies legal thresholds and operating modes to downstream layers; prevents unregistered test-driven threshold use.
Feedback/loopback	If Layer 4 or Layer 6 requests an unknown threshold, the experiment should fail fast or run only in exploratory mode with a warning.
Evaluation	Registry coverage audit: every threshold appearing in results must map to a registry entry with no-test-label status or oracle-only status.
Failure modes	Unregistered thresholds, threshold reuse across incompatible models, threshold chosen on test data, missing version, ambiguous fairness note.
Reproducibility	Save registry file, config hash, selected-on split, rule code version, and downstream-use permissions.
Thesis evidence	Demonstrates fair comparison between IHARQ, selective prediction, Adaptive-IHARQ, and policy-learning variants.
Limitations and risks	Registered thresholds are only as valid as the selection protocol; sparse validation splits may produce unstable operating points.

9.7 Dependency logic

Layer 3 has hard dependencies on valid `PredictionRecord` files, split manifests, label maps, class-order consistency, probability/logit/score fields, model IDs, subject/session metadata where available, and reproducible configuration hashes. It has soft dependencies on sufficient calibration data, logits for temperature scaling, repeated seeds, ensemble member predictions, window-level histories, stress labels, and enough samples per reliability bin. Optional dependencies include conformal calibration sets, subject/session diagnostic calibration, rolling temporal summaries for RegimeRisk, cost definitions for selective prediction, and dashboard-specific export schemas.

Conceptually, Layer 3 depends on the separation between *accuracy* and *reliability*. A decoder can be accurate but poorly calibrated, or less accurate but better at indicating uncertainty. Data dependencies include public EEG dataset metadata, split roles, calibration labels, and evaluation labels. Model dependencies include decoder family, score type, checkpoint identity, and seed. Metric dependencies include ECE-style calibration summaries, Brier score, NLL, retained accuracy, coverage, rejection rate, risk at coverage, and high-confidence wrong rate. Software dependencies include calibration-method implementations, schema validation, reliability-plot generation, and record export.

During development, Layer 3 can be mocked with synthetic `PredictionRecord` files whose probability vectors have known calibration errors, known confidence/correctness relations, and known threshold behavior. Calibration-method code, threshold registry integration, dashboard export, and downstream Layer 4/6 consumers can all be tested before full EEG experiments complete. However, thesis-level claims require real Layer 2 prediction outputs under declared

splits and budgets.

9.8 Feedforward and loopback behavior

The normal feedforward path is: Layer 2 emits raw `PredictionRecord` files; Layer 3 validates scores and split roles; the probability calibrator creates `CalibrationRecord`; the uncertainty extractor creates `UncertaintyRecord`; the reliability audit produces reports; the selective-prediction baseline creates `SelectivePredictionRecord`; the threshold registry produces `ThresholdRegistryRecord`; Layer 4 consumes calibrated confidence and uncertainty for evidence verification; Layer 6 consumes label-free uncertainty fields as policy context; Layer 10 renders calibration, uncertainty, risk-coverage, threshold, and high-confidence-error views.

Layer 3 can also emit carefully constrained loopback signals:

Source	Target	Signal	Trigger	Action and safety constraint
Split leakage guard	Layer 1/experiment controller	<code>LeakageWarningRecord</code>	Calibration, validation, and test roles overlap invalidly.	Stop final evaluation or rerun with corrected split. Do not repair using test labels.
Input validator	Layer 2	Missing probability/logit notice	<code>PredictionRecords</code> contain labels only, malformed probabilities, missing IDs, or class-order mismatch.	Mark score type, repair record generation if possible, or exclude model with documented reason.
Probability calibrator	Experiment controller	Sparse-calibration warning	Calibration split has missing classes or too few examples for chosen method.	Fall back to simpler declared method or mark method invalid. Do not tune method on final test.
Reliability audit	Layer 4/6 evaluation	High-confidence failure seed list	Confident wrong cases exceed declared audit threshold or cluster by subject/session/class.	Use cases for failure analysis and ablations; do not use test-derived flags as inference-time features.
Selective baseline	Layer 6 comparison	Coverage-collapse warning	Threshold achieves high retained accuracy by rejecting too many cases or rejecting one group disproportionately.	Require matched-coverage/matched-cost reporting and subject-wise coverage disclosure.

Threshold registry	Layer 4/6 runner	Unknown-threshold failure	A downstream module requests a threshold not in the registry.	Fail fast for final experiments or mark exploratory run as non-final.
Dashboard exporter	Layer 10	Plot-data warning	A dashboard curve lacks source records or config hash.	Exclude from reproducibility package until source traceability exists.

These loopbacks exist for scientific and engineering reasons: leakage prevention, schema integrity, calibration validity, failure analysis, fairness of rejection, threshold auditability, and reproducibility. They are not arbitrary feedback loops and must not change locked final test evaluations after seeing test outcomes.

9.9 Evaluation design

Layer 3 should be evaluated as a full reliability-preparation layer, not merely as a metrics script. The evaluation asks whether raw probabilities are calibrated, whether calibration improves reliability, whether uncertainty features predict errors, whether selective-prediction baselines are strong, whether low-label calibration is stable, whether failures are group-specific, and whether downstream Layers 4 and 6 benefit from Layer 3 features.

Evaluation question	Evidence
Are raw decoder probabilities calibrated?	Raw reliability diagrams, ECE-style summaries, Brier score, NLL, confidence histograms, high-confidence wrong rates.
Does calibration improve reliability?	Before/after calibration comparison by model, dataset, subject, session, class, budget, and seed.
Which uncertainty features predict errors?	Error-detection analysis, risk-coverage curves per feature, retained accuracy at matched coverage, feature/correctness association.
How strong are selective-prediction baselines?	Risk-coverage, accuracy-coverage, rejection-error, coverage-at-target-risk, and high-confidence-wrong-at-coverage curves.
Does calibration remain meaningful under low-label budgets?	Calibration-budget curves, threshold variability, sparse-budget warnings, low-calibration limitations.
Do aggregate metrics hide failures?	Class-wise, subject-wise, session-wise, dataset-wise, model-family-wise, stress-conditioned tables.
Do Layer 3 outputs help downstream decisions?	Ablations: Layer 4 with raw confidence only; Layer 4 with calibrated confidence; Layer 4 with calibrated confidence plus entropy and margin; Layer 6 with raw confidence context; Layer 6 with full Layer 3 context.
Do thresholding or selective prediction already solve the wrong-accept problem?	Matched A1/A2/A3 comparisons against A0 decoder predictions, reporting matched coverage, matched wrong-accept cost, retained accuracy, rejection burden, high-confidence wrong rate, and threshold-selection lineage.

Core metrics are accuracy for context, retained accuracy, coverage, rejection rate, accepted accuracy, wrong-accept rate under selective acceptance, ECE-style calibration error, Brier score, NLL, high-confidence wrong rate, risk at coverage, coverage at target risk, class-wise calibration, subject-wise calibration, session-wise calibration, and threshold instability across seeds/budgets. If AUROC-style error detection or AURC-style summaries are used, they must be defined as uncertainty-analysis metrics rather than safety guarantees. Each metric must appear in the metric dictionary with directionality, required fields, aggregation unit, split role, and claim boundary.

Low-calibration evaluation should include a zero-calibration/raw baseline, very-low calibration budgets such as one, two, five, or ten labeled trials per class where feasible, low-percentage calibration subsets, medium calibration subsets, and a full-calibration reference under the declared protocol. For each budget, Layer 3 should report calibration stability, ECE/Brier/NLL, confidence histograms, high-confidence wrong rate, selective-prediction curves, threshold variability, and downstream effects on IHARQ or policy decisions.

Layer 3 aligns its local evaluation with the system-level A0–A13 ladder rather than defining an incompatible local ladder. In the Layers 0–3 slice, A0 is supplied by Layer 2 raw decoder predictions; A1 tests whether calibration improves reliability visibility beyond raw scores; A2 tests simple confidence-threshold or matched-coverage rejection; A3 tests entropy, margin, uncertainty, and selective-prediction baselines; and A4 is supplied primarily by Layer 2 ordinary voting, averaging, longer-window, and ensemble controls when those controls are compared against Layer 3 reliability records. Layer 3 must therefore export enough information for later A5 and A9 comparisons to show whether IHARQ and adaptive policies add value beyond raw confidence, calibrated confidence, simple thresholding, and selective prediction.

The required Layer 3 ablation outputs are raw-versus-calibrated reliability tables, A1 calibration deltas, A2 threshold registry comparisons, A3 risk-coverage and accuracy-coverage curves, high-confidence-wrong audits, matched-coverage or matched-cost operating-point tables, and negative-result notes when calibration or selective prediction is sufficient under a declared protocol. This directly protects against the criticism that IHARQ is only thresholding raw confidence.

9.10 Reproducibility artifacts and repository placement

Layer 3 should be implemented as a reproducible module rather than a notebook-only analysis. Recommended repository placement is:

```
src/layer3_calibration_uncertainty/
|-- calibrators/
|   |-- temperature_scaling.py
|   |-- platt_scaling.py
|   |-- isotonic_calibration.py
|   '-- method_registry.py
|-- audits/
|   |-- reliability_diagrams.py
|   |-- calibration_metrics.py
|   |-- high_confidence_wrong.py
|   '-- groupwise_audit.py
|-- uncertainty/
|   |-- confidence.py
|   |-- entropy.py
```



```

|   |-- margin.py
|   |-- disagreement.py
|   '-- conformal_proxy.py
|-- selective_prediction/
|   |-- risk_coverage.py
|   |-- threshold_search.py
|   '-- operating_points.py
|-- registry/
|   |-- threshold_registry.py
|   |-- schema_validation.py
|   '-- split_integrity.py
'-- export/
    |-- calibration_records.py
    |-- uncertainty_records.py
    |-- dashboard_metrics.py
    '-- reliability_cards.py

```

Recommended configuration files are `configs/layer3/calibration_methods.yaml`, `uncertainty_features.yaml`, `selective_prediction.yaml`, `threshold_registry.yaml`, `split_integrity_rules.yaml`, `groupwise_audit.yaml`, and `dashboard_export.yaml`. Required output files include `calibration_records.parquet`, `uncertainty_records.parquet`, `selective_prediction_records.parquet`, `threshold_registry.yaml`, `reliability_audit_report.json`, reliability diagrams, risk-coverage curve data, high-confidence wrong tables, matched-coverage and matched-cost operating-point tables, group-wise audit tables, dashboard metric records, and a Layer 3 run manifest.

A reproducible command template is:

```

python run_layer3.py \
  --prediction-records results/layer2/predictions.parquet \
  --split-manifest data/manifests/split_manifest.csv \
  --config configs/layer3/calibration_uncertainty.yaml \
  --seed 42 \
  --out results/layer3/

```

Every Layer 3 figure or table in the thesis should be traceable to an output file, configuration hash, split ID, model ID, seed, and source `PredictionRecord`. Calibration plots should be regenerated from records, not screenshots. Dashboard metrics should be derived from formal Layer 3 records. Thresholds should be stored in the registry before downstream modules use them.

9.11 Dashboard, cards, and thesis outputs

Layer 3 should provide Layer 10 with the following dashboard views: calibration overview, reliability diagrams, confidence histograms, uncertainty distributions, high-confidence wrong cases, risk-coverage curves, matched-coverage and matched-cost threshold comparisons, threshold registry view, group-wise reliability view, and calibration-budget view. The corresponding cards are calibration cards, uncertainty cards, selective-prediction cards, threshold cards, high-confidence error cards, and group-wise reliability cards.

Expected thesis artifacts include raw-vs-calibrated reliability diagrams, calibration-budget curves, confidence histograms with correctness overlays, high-confidence wrong example tables, risk-coverage curves, subject-wise calibration heatmaps or tables, model-family calibration comparisons, and threshold registry summary tables. These artifacts belong in the reliability evaluation chapter, ablation chapter, dashboard appendix, and reproducibility appendix.

9.12 Outputs

Layer 3 outputs calibrated prediction records, uncertainty records, calibration plots, reliability metrics, selective-prediction baselines, threshold registries, split-integrity reports, high-confidence wrong tables, matched-coverage and matched-cost operating-point tables, group-wise audit tables, risk-coverage curves, dashboard metric records, reliability cards, negative-result notes for calibration or threshold sufficiency, and thesis-ready figures/tables. The most important formal outputs are **CalibrationRecord**, **UncertaintyRecord**, **SelectivePredictionRecord**, and **ThresholdRegistryRecord**.

9.13 Outgoing links

Layer 3 feeds Layer 4 with calibrated confidence, entropy, margin, variation ratio, disagreement where available, selective-prediction baseline flags, high-confidence wrong examples for offline failure analysis, matched-coverage or matched-cost baseline points, and registered thresholds for IHARQ-lite ablations. It feeds Layer 6 with label-free uncertainty features, risk-coverage baselines, operating points, and threshold candidates for adaptive policy comparison. It optionally feeds Layer 5 with rolling confidence trends, rolling entropy trends, calibration drift proxies, and high-confidence failure bursts in offline evaluation. It feeds Layer 10 with calibration dashboard views, uncertainty distributions, risk-coverage curves, threshold registry tables, high-confidence wrong tables, reliability cards, and reproducibility manifests.

Layer 3 does not decide final accept, wait, request-evidence, combine, fallback, uncertain, or unsafe actions. It provides the calibrated and traceable evidence records that make those decisions scientifically auditable in later layers.

9.14 Definition of done

Layer 3 is complete when every eligible Layer 2 prediction file passes split and schema validation; at least the no-calibration baseline and one feasible calibration method are evaluated; **CalibrationRecord** and **UncertaintyRecord** files exist for every selected decoder, dataset, subject/session where available, split, seed, and label budget; reliability audits report aggregate and group-wise calibration; selective-prediction baselines generate risk-coverage and accuracy-coverage curves; matched-coverage or matched-cost threshold comparisons are exported; every threshold used downstream has a **ThresholdRegistryRecord**; negative results such as threshold sufficiency or calibration failure are visible; leakage checks pass; dashboard metric records and reliability-card inputs are generated; and downstream mock consumers in Layers 4, 6, and 10 can read the exported records without manual intervention.

For thesis-level claims, Layer 3 should reach the equivalent of L3-R9: contracts, calibration methods, uncertainty features, selective baselines, leakage controls, evaluation plans, downstream interfaces, figures, tables, cards, limitations, and criticism defenses must all be present.

10 Layer 4: IHARQ Evidence Verification

10.1 Problem solved by Layer 4

A normal decoder forces a class prediction from a short EEG segment. However, the evidence may be weak, unstable, corrupted, contradictory, badly calibrated, regime-sensitive, or not safe enough to act on. Layer 4 is the explicit evidence-verification layer. It asks whether the current neural-intent evidence is strong, stable, calibrated, and safe enough to support simulated readiness action now.

Layer 4 exists because the architecture should not move directly from **PredictionRecord** or calibrated probability to action. Layer 2 predicts intent; Layer 3 calibrates and measures uncertainty; Layer 5 monitors temporal regime risk; Layer 6 may learn adaptive readiness policies; Layer 7 tests simulated closed-loop consequences; and Layer 10 reports the results. Layer 4 sits between these functions and converts measured evidence into a transparent readiness decision.

The bottleneck addressed by Layer 4 is therefore not only low accuracy. The deeper bottleneck is **forced acceptance under incomplete evidence**. In low-calibration EEG, a decoder may be confident but wrong, locally stable but globally unreliable, spatially fragile across channel subsets, or temporarily risky because of drift, fatigue-like degradation, dropout, or noise. Accuracy alone does not tell the architecture when to trust, delay, request more evidence, combine windows, fallback, mark uncertainty, or block action.

IHARQ is inspired by the reliability logic of hybrid automatic repeat request systems, but it is not a literal communication protocol. In this architecture, IHARQ means **Intent Hybrid Automatic Repeat reQuest**: a BCI-specific, decoder-agnostic, post-calibration, reason-coded evidence-verification mechanism. It may accept, wait, request additional evidence, combine evidence, fallback, mark the state uncertain, or declare unsafe. It is not merely confidence thresholding, not a communication-layer transplant, not clinical safety validation, and not the learned controller of Layer 6.

The thesis role of Layer 4 is to transform the project from an ordinary classifier benchmark into a reliability-aware readiness architecture. A standard EEG benchmark asks which decoder predicts the label. Layer 4 asks a more defensible thesis question:

When should a low-calibration neural-intent architecture trust, delay, request more evidence, combine evidence, fallback, or block a predicted intent under uncertainty, stability, agreement, signal-quality, and temporal-risk constraints?

The central claim supported by this layer is not that IHARQ always improves raw accuracy. A stronger and safer claim is that a reason-coded IHARQ evidence-verification layer can reduce wrong accepted decisions, expose unsafe or contradictory evidence states, and improve accepted-decision reliability compared with accept-all decoding, confidence thresholding, entropy rejection, margin rejection, calibrated selective prediction, simple multi-window voting, probability averaging, ordinary ensembles, and fallback-only baselines at matched coverage, evidence cost, or action cost.

If Layer 4 were removed, the architecture would lose its explicit trust gate. Layer 6 would learn from unverified evidence, Layer 7 would simulate consequences of raw or threshold-only predictions, Layer 9 could receive embodiment-gating signals without a transparent readiness check, and Layer 10 would report decisions without reason-coded evidence traces. The thesis would be vulnerable to the criticism that it merely wraps a classifier with plots rather than

building an auditable reliability-control architecture.

10.2 Engineering responsibility and boundary discipline

Layer 4 is a post-decoder and post-calibration decision-control module. It does not train the main decoder, create public-data splits, perform raw EEG preprocessing, calibrate probabilities, learn the final adaptive policy, simulate the closed-loop environment, control MyoSuite/OpenSim, or render the dashboard. It consumes upstream records and produces evidence-aware readiness decisions.

Boundary	In scope for Layer 4	Out of scope or delegated layer
Evidence assembly	Build evidence features from prediction, calibration, uncertainty, stability, agreement, signal quality, and optional <code>RegimeStateRecord</code> context.	Raw EEG preprocessing, dataset cleaning, and split construction remain Layer 1; decoder training remains Layer 2.
Decision logic	Choose interpretable actions: accept, wait, request evidence, combine evidence, fallback, uncertain, or unsafe.	Learned policy optimization belongs to Layer 6; closed-loop consequence simulation belongs to Layer 7.
Calibration use	Use calibrated probabilities and uncertainty features from Layer 3.	Probability calibration, reliability diagrams, and threshold selection baselines are Layer 3 responsibilities.
Temporal risk use	Consume <code>RegimeRisk</code> and stop-loss signals when Layer 5 is active.	Full temporal regime modeling and model-trust trend estimation remain Layer 5 responsibilities.
Fallback and unsafe logic	Route to declared fallback options, no-action modes, uncertainty, or unsafe blocks under defined triggers.	Clinical safety validation, medical-device verification, and real patient-safety claims remain out of scope under Layer 0.
Logging and traceability	Save decision records, evidence-quality records, combined-evidence records, fallback logs, unsafe-event logs, ablation records, and dashboard traces.	Dashboard rendering and interactive user interface design remain Layer 10 responsibilities.

The boundary rule is: **Layer 4 verifies evidence and chooses an interpretable readiness action. It may consume risk context and produce policy-ready features, but it must not become the decoder, the calibration layer, the temporal regime model, the learned policy, the simulator, or the dashboard.**

Core submodules are the evidence feature builder, IHARQ rule engine, decision logger, input validator, reason-code registry, and minimum fallback/uncertain/unsafe handlers. Supporting submodules are the window stability checker, channel-subset checker, model-family agreement

checker, evidence combiner, fallback router, unsafe-state monitor, ablation exporter, and dashboard trace exporter. Optional or stretch submodules include RegimeRisk-aware threshold tightening, conformal-set combination if Layer 3 supplies conformal proxies, weighted model-family trust using Layer 5 trust scores, supervisor-proxy fallback in Layer 7 simulation, and embodiment-compatible hold/stop signals for Layer 9. Future-work items include online evidence acquisition from a live BCI stream and clinically validated safety controllers; those must not be claimed unless actually implemented and validated.

10.3 Position in the architecture

Layer 4 is the bridge between measurement and readiness control:

Layer 2 raw prediction outputs → Layer 3 calibration and uncertainty transformation → Layer 4 IHARQ evidence verification → Layer 6 adaptive readiness policies → Layer 7 simulated closed-loop readiness environment → Layer 10 dashboard and thesis artifacts.

Layer 5 provides a conditional side input:

Layer 5 RegimeRisk temporal trust monitoring → Layer 4 threshold tightening, request logic, fallback recommendation, or unsafe blocking.

Layer 4 should therefore be documented and implemented as both a **control gate** and an **evidence transformer**. It transforms calibrated prediction evidence into action candidates, reason codes, costs, and traces that later layers can use without rerunning decoders.

Architectural relation	Layer 4 role
From Layer 2	Receives standardized prediction information, model IDs, model-family labels, trial/window IDs, raw probabilities or logits when available, aligned multi-model predictions, and ensemble-control outputs.
From Layer 3	Receives calibrated probabilities, confidence, entropy, margin, variation ratio, disagreement, selective-prediction baselines, threshold registry entries, calibration metadata, and high-confidence wrong-case tables for offline failure analysis.
From Layer 5	Receives RegimeStateRecord fields such as regime label, RegimeRisk score, risk trend, volatility-like feature, drawdown-like feature, drift risk, fatigue-like risk, dropout risk, model trust scores, and stop-loss warning.
To Layer 6	Sends readiness-action candidates, evidence-quality features, reason-coded pseudo-decisions, action costs, fallback status, uncertainty labels, and safe/unsafe flags for supervised Adaptive-IHARQ, learning-to-defer, contextual bandit, and RL comparisons.
To Layer 7	Sends action decisions used inside simulated sessions, including wait, request, combine, fallback, no-action, and unsafe-block behavior with declared costs and delays.

To Layer 9	Sends embodiment-compatible gating status indirectly through Layer 7 or policy records, such as hold, no-action, conservative assistive command, or unsafe stop. It must not claim direct real-body safety.
To Layer 10	Sends dashboard-ready logs, IHARQ timelines, evidence-flow records, reason-code distributions, request/combine traces, fallback summaries, unsafe-event tables, ablation records, and reliability cards.

10.4 Input contract

Layer 4 must not rely on informal hidden variables. Every input is a record with identity fields, provenance fields, split information, and validation behavior. Inputs are classified as primary evidence inputs, stability evidence inputs, risk-context inputs, safety inputs, evaluation-only inputs, dashboard inputs, proxy inputs, or stretch inputs.

Input record	Required	Source	Main purpose
PredictionRecord	Yes	Layer 2	Provides predicted class, logits or decision scores, raw probabilities when available, model ID, model family, trial ID, window ID, label-budget ID, split ID, seed, and configuration hash.
CalibrationRecord	Yes for calibrated mode	Layer 3	Provides calibrated probabilities, calibration method, calibration split, ECE, Brier score, NLL, reliability-bin membership, and calibration metadata.
UncertaintyRecord	Yes	Layer 3	Provides confidence, entropy, margin, variation ratio, disagreement, conformal set size if used, selective coverage flags, and uncertainty feature provenance.
WindowHistoryRecord	Yes for stability or combine mode	Layer 1/2/3 aggregation	Provides recent windows, temporal index, adjacent predictions, probability history, uncertainty history, and prior IHARQ actions for stability and evidence accumulation.
ArtifactFlagRecord	Strongly recommended	Layer 1 or preprocessing/audit module	Provides artifact flags, channel masks, missing-channel flags, noise indicators, dropout indicators, and signal-validity metadata.

ChannelSubsetStabilityRecord	Strongly recommended when channel subsets exist	Layer 4 internal or precomputed from Layer 2 records	Provides spatial robustness evidence across declared channel subsets, channel-dropout tests, or left/right/central subsets.
ModelAgreementRecord	Strongly recommended when multiple model families exist	Layer 2 and Layer 4	Provides agreement score, model-family vote distribution, trusted-family set, validation trust weights, and disagreement reason codes.
RegimeStateRecord	Optional but important in Stretch C	Layer 5	Provides RegimeRisk score, regime label, risk trend, volatility-like and drawdown-like features, drift/fatigue/dropout risk, trust scores, and stop-loss warnings.
ThresholdRegistryRecord	Yes for fair comparisons	Layer 3	Provides registered threshold values, threshold feature, target constraint, selection split, applicable model IDs, allowed consumers, and leakage status.
EvaluationOutcomeRecord	Evaluation-only	Layer 1/2/7	Provides true label, correctness, simulated outcome, reward, late-assist status, unsafe event status, or future-window evidence. It must not influence offline decisions unless the experiment explicitly defines an oracle or retrospective analysis.

Minimum metadata fields for every Layer 4 input are dataset ID, subject ID, session ID when available, run ID when available, trial ID, window ID, split ID, label-budget ID, model ID, model family, calibration method or calibration ID, random seed, configuration hash, and timestamp or simulated time index. For public EEG datasets where some metadata do not exist, the field should be present with a declared missing value rather than silently omitted.

Input class	Examples and meaning
Primary evidence input	PredictionRecord , CalibrationRecord , and UncertaintyRecord . These are required to make any calibrated IHARQ decision.
Stability evidence input	WindowHistoryRecord , ChannelSubsetStabilityRecord , and ModelAgreementRecord . These determine whether evidence is consistent across time, space, and model families.

Risk-context input	RegimeStateRecord , stop-loss warning, rolling model trust scores, or drift/fatigue/dropout risk. These tighten or modify decision logic but should be labeled unavailable in Layer 4-only mode.
Safety input	Severe artifact flags, missing channels, corrupted probability vector flags, unsafe regime state, dropout flags, and integrity errors. These can block acceptance or route fallback.
Evaluation-only input	True labels, future-window correctness, simulated outcomes, rewards, and embodiment errors. These are used after decision-making to evaluate IHARQ, not to make ordinary offline decisions.
Proxy input	Label-free proxies such as entropy trends, confidence drops, or agreement instability. These must be labeled as proxies, not clinical safety evidence.
Stretch input	Supervisor-proxy actions, MyoSuite/OpenSim hold/stop status, or stress-conditioned variables in Layer 7/9 experiments. These are valid only in declared simulation modes.
Dashboard/export specification input	Versioned dashboard view schema, card/export requirements, metric-dictionary references, and claim-evidence field requirements used to ensure Layer 4 records are dashboard-ready without allowing Layer 10 to recompute IHARQ decisions.

Layer 4-specific helper records such as **WindowHistoryRecord**, **ChannelSubsetStabilityRecord**, and **ModelAgreementRecord** are local or precomputed evidence views derived from canonical **WindowRecord**, **PredictionRecord**, **CalibrationRecord**, **UncertaintyRecord**, **ModelRegistryRecord**, and **ArtifactFlagRecord** inputs. They should not be treated as new upstream canonical records unless a later propagation pass explicitly promotes them into the system-wide record registry. This keeps Layer 4 compatible with the existing Layer 1–3 contract while still allowing stable local schemas for evidence verification.

10.5 Input validation and degraded modes

Layer 4 must validate all inputs before decision-making. Validation failures should produce explicit records rather than silent fallback behavior.

Validation check	Failure response
Missing prediction	Return Uncertain or Unsafe depending on severity; log MISSING_REQUIRED_RECORD .
Missing calibrated probability	Use an uncalibrated fallback path only if explicitly allowed in configuration; otherwise Request evidence or Uncertain .
Invalid probability vector	Block acceptance; log schema violation and invalid probability reason code.
Confidence outside $[0, 1]$	Mark input invalid and block acceptance.

Mismatched dataset/subject/session/trial/window IDs	Block decision path and record integrity error .
Missing window history	Disable stability and combine logic; do not assume stability. The decision must include a degraded-evidence reason code.
Missing channel mask or channel-subset outputs	Disable channel-subset stability checks; log degraded spatial evidence.
Missing model-family alternatives	Disable model-family agreement; compare against single-model IHARQ and log that agreement evidence was unavailable.
Missing signal-quality flags	Treat signal quality as unknown; do not treat it as clean. The configuration may require fallback, request evidence, or uncertainty.
Missing RegimeRisk	Use regime-unaware IHARQ mode; mark <code>regime_aware=false</code> .
Evaluation-only label accidentally present in decision input	Reject run or mark leakage violation unless an explicitly declared oracle analysis is being performed.

Mockable dependencies during development include synthetic `PredictionRecord` files with known probabilities, artificial uncertainty records, simulated window histories, synthetic channel-subset disagreement, fake model-family agreement, and dummy `RegimeRisk` scores. Non-mockable dependencies for thesis-level evaluation are valid prediction IDs, split-safe calibration records, stored uncertainty records, declared thresholds, and a decision configuration hash.

10.6 Output contract

Layer 4 outputs are thesis artifacts. They must be schema-based, saved, reusable, dashboard-readable, evaluation-readable, and traceable to their inputs. Every output should identify its downstream consumer and its evaluation status.

Output record	Purpose	Consumers
<code>IHARQDecisionRecord</code>	Stores final action as a canonical Layer 4 action token with a human-readable display label, accepted prediction if any, fallback status, unsafe status, uncertainty status, evidence request count, combined evidence summary, action cost, reason code, rule path, global/local ablation IDs when emitted during evaluation, and claim boundary.	Layers 6, 7, 9, 10, evaluation scripts.

EvidenceQualityRecord	Stores evidence-quality score, reliable/unreliable proxy target, confidence, entropy, margin, stability features, signal-quality features, disagreement features, predicted failure probability if used, missing-evidence flags, and decision-time/evaluation-only partition status.	Layers 4, 6, 7, 10, evaluation scripts.
CombinedEvidenceRecord	Stores which windows or sources were combined, the combination method, before/after confidence, before/after entropy, combined class, eligibility reason, final combined decision, and delay/evidence-cost fields.	Layers 4, 6, 7, 10.
FallbackDecisionRecord	Stores fallback route, fallback reason, fallback target, fallback availability, fallback cost, fallback success or failure during evaluation, and fallback compatibility mode.	Layers 6, 7, 9, 10.
UnsafeEventRecord	Stores unsafe trigger, unsafe reason, blocked action, relevant evidence, RegimeRisk context, stress context, recommended conservative outcome, and simulation/proxy boundary label.	Layers 7, 9, 10, thesis safety discussion.
IHARQAblationRecord	Stores decisions and metrics under ablated versions of Layer 4, such as confidence-only, entropy-only, margin-only, no stability, no fallback, no unsafe, no combination, or regime-unaware IHARQ.	Evaluation layer, Layer 10, thesis ablation chapter.
IHARQTraceRecord	Stores full rule path, thresholds used, feature values, reason-code hierarchy, input file paths, configuration hash, metric dictionary ID, dashboard trace fields, and figure/table lineage.	Layer 10, reproducibility package, debugging.
EvaluationOutcomeRecord	Stores Layer 4 metric outputs by dataset, subject, session, model, threshold, ablation step, and matched-comparison key.	Evaluation scripts, Layer 10, thesis tables.

The minimum **IHARQDecisionRecord** schema includes:

```
dataset_id
subject_id
session_id
run_id
```

```
trial_id
window_id
split_id
label_budget_id
model_id
model_family
calibration_id
decision_time_index
predicted_class
accepted_class_if_any
action
reason_code
reason_text
rule_path
confidence
entropy
margin
calibrated_probability_top1
window_stability_score
channel_stability_score
model_agreement_score
signal_quality_flag
regime_risk_score_if_available
fallback_available
fallback_selected
unsafe_flag
evidence_request_count
combined_window_count
action_cost
evaluation_mode
global_ablation_id
local_ablation_id
system_variant_id
matched_comparison_key
metric_dictionary_id
dashboard_export_id
claim_boundary
negative_result_note_id_if_any
diagnostic_only_flag
configuration_hash
random_seed
```

Output design rules are mandatory. Every decision must be reproducible from saved input records and configuration. Every action must have a reason code. Every unsafe decision must identify the unsafe trigger. Every fallback decision must identify the fallback route. Every combined decision must identify the windows or sources used. Every record must be traceable to dataset, subject, session, trial, and window. Every output should be dashboard-readable without rerunning the decoder. Every evaluation output should also be linked to an ablation-step ID, matched-comparison key, metric dictionary entry, claim-evidence matrix row, and negative-result note when applicable.

10.7 Action vocabulary

Layer 4’s action vocabulary is one of its main architectural contributions. The vocabulary must remain explicit, finite, reason-coded, and comparable against simpler baselines.

The canonical stored action tokens for `IHARQDecisionRecord` are `accept`, `wait`, `request_evidence`, `combine_evidence`, `fallback`, `uncertain`, and `unsafe`. Human-facing prose and dashboard labels may display these as **Accept**, **Wait**, **Request evidence**, **Combine evidence**, **Fallback**, **Uncertain**, and **Unsafe**. Downstream policy, simulation, embodiment, and dashboard modules may expose aliases such as `request-evidence` or `combine only` through an explicit action-map or display-label field; stored Layer 4 records should use the canonical tokens.

Action	Meaning	When used
Accept	Use the current predicted intent as ready for simulated action.	Evidence is strong, calibrated, stable, sufficiently low-risk, and no higher-priority unsafe or fallback rule blocks acceptance.
Wait	Delay action without immediately specifying a targeted extra evidence source.	Evidence is incomplete but not dangerous; short delay is less costly than immediate acceptance.
Request evidence	Acquire or evaluate another window, evidence source, model-family prediction, channel-subset check, signal-quality check, or <code>RegimeRisk</code> update.	Evidence is ambiguous but may become reliable within the declared evidence budget.
Combine evidence	Fuse current and previous windows or sources under declared eligibility rules.	Multiple weak windows or sources are individually insufficient but mutually consistent and safe to combine.
Fallback	Route to a safer or more conservative alternative.	Main evidence path is unreliable, but a declared alternative is safer than accepting or staying undecided.
Uncertain	Mark the state as undecidable under current evidence.	No justified accept or fallback decision exists, but no explicit unsafe trigger is active.
Unsafe	Block action and record a safety event.	Evidence is contradictory, signal quality is severe, <code>RegimeRisk</code> is high, stop-loss is active, stress state exceeds declared bounds, or fallback is unavailable while action risk is high.

Accept is allowed only when calibrated confidence is sufficiently high, uncertainty is sufficiently low, top-class margin is sufficiently strong, adjacent-window stability is acceptable or not required by the protocol, channel-subset stability is acceptable or explicitly unavailable, model-family agreement is acceptable or not used, signal-quality flags are not severe, `RegimeRisk` is acceptable when active, and no unsafe trigger is active. It must log accepted class, confidence,

stability scores, agreement scores, thresholds used, reason code, and whether RegimeRisk was active.

Wait is used when evidence is incomplete but not dangerous. It must log waiting reason, current uncertainty, allowed waiting budget, prior wait count, and whether the wait count is approaching the maximum.

Request evidence is used when current evidence is ambiguous, confidence and stability disagree, confidence is moderate but insufficient, model-family disagreement may be resolved by more evidence, or the evidence-request budget has not yet been exhausted. It must log requested evidence type, request count, maximum request budget, expected reason for improvement, and outcome after the requested evidence arrives.

Combine evidence is used when several weak windows or sources become more reliable together and the combination does not hide contradiction. It must log window or source IDs, combination method, before/after confidence, before/after entropy, combined class, combination reason, and final action after combination.

Fallback is used when the main evidence path is not trusted and a safer alternative exists. Possible fallback routes include conservative decoder, majority/no-action baseline, model family with higher current trust, extended evidence request, safe no-action controller, simulated supervisor proxy in Layer 7, or hold/stop state in Layer 9 embodiment simulation. It must log fallback route, reason, cost, availability, compatibility mode, and evaluation outcome when available.

Uncertain is used when evidence is insufficient, fallback is unavailable or unjustified, unsafe threshold is not crossed, and no action can be academically justified. It must log uncertainty reason, missing or conflicting evidence, and whether additional evidence could resolve the state.

Unsafe is reserved for severe conditions and must not be used as a vague synonym for low confidence. It must log unsafe trigger, blocked action, risk score, evidence contradiction, recommended conservative outcome, and whether the unsafe condition occurred during stress or embodiment simulation.

10.8 Reason-code and explanation taxonomy

Every Layer 4 decision must have a reason code and a human-readable explanation. Reason codes support thesis-level qualitative analysis, dashboard inspection, and reproducibility.

Category	Example reason codes
Acceptance	ACCEPT_HIGH_CONF_STABLE, ACCEPT_COMBINED_STABLE_EVIDENCE, ACCEPT_LOW_RISK_AGREEMENT.
Waiting	WAIT_INCOMPLETE_EVIDENCE, WAIT_NEAR_THRESHOLD, WAIT_TEMPORAL_CONTEXT_NEEDED.
Request evidence	REQ_LOW_MARGIN, REQ_MODEL_DISAGREEMENT, REQ_WINDOW_INSTABILITY, REQ_CHANNEL_CHECK, REQ_REGIME_UPDATE.
Combine evidence	COMBINE_CONSISTENT_WEAK_WINDOWS, COMBINE_STABLE_LOW_CONFIDENCE, COMBINE_MODEL_CONSENSUS.

Fallback	FALLBACK_LOW_TRUST, FALLBACK_HIGH_REGIME_RISK, FALLBACK_SIGNAL_QUALITY, FALLBACK_MODEL_FAMILY_DISAGREEMENT.
Uncertain	UNCERTAIN_NO_VALID_FALLBACK, UNCERTAIN_CONFLICTING_EVIDENCE, UNCERTAIN_MISSING_EVIDENCE.
Unsafe	UNSAFE_HIGH_REGIME_RISK, UNSAFE_SIGNAL_CORRUPTION, UNSAFE_CONTRADICTIONARY_EVIDENCE, UNSAFE_STOP_LOSS, UNSAFE_STRESS_BOUNDARY.
System integrity	INVALID_SCHEMA, MISSING_REQUIRED_RECORD, ID_MISMATCH, LEAKAGE_VIOLATION, INVALID_PROBABILITY_VECTOR.

For every decision, Layer 4 should be able to answer: why was this action chosen; which evidence supported it; which evidence blocked acceptance; which threshold was crossed; whether RegimeRisk was active; whether fallback was available; whether a simpler baseline would have accepted or rejected; and what the final outcome was during evaluation.

The thesis value of reason codes is that they make failure analysis possible. The thesis can report not only how often IHARQ succeeds, but why it fails, such as whether false accepted actions concentrate in high-confidence model disagreement, channel instability, or drift-risk regimes.

10.9 Submodules

Layer 4 consists of independently documentable, testable, and ablatabl submodules. The compact submodule map is:

Submodule	Function	Output
Evidence feature builder	Assembles confidence, entropy, margin, agreement, stability, signal-quality, request-count, and optional RegimeRisk features.	Evidence feature vector and draft EvidenceQualityRecord.
Window stability checker	Compares predictions, calibrated probabilities, uncertainty, and accepted decisions across adjacent windows.	Window stability score, stability timeline, and reason codes.
Channel-subset checker	Tests whether different channel subsets support the same intent.	Channel stability score, corrupted-channel flag, and channel disagreement reason code.
Model-family agreement checker	Compares classical, Riemannian, deep, compact SSL, ensemble, and fallback predictions when available.	Agreement score, trusted-family set, and disagreement reason code.

IHARQ rule engine	Applies deterministic, interpretable accept/wait/request/combine/fallback/uncertain/unsafe rules.	IHARQDecisionRecord.
Evidence combiner	Combines probabilities, logits, votes, or conformal proxies across windows or sources where allowed.	CombinedEvidenceRecord and optional updated decision.
Fallback router	Selects fallback mode based on evidence risk, fallback availability, model trust, and action cost.	FallbackDecisionRecord.
Decision logger	Saves action, reason, thresholds, features, rule path, outcomes, and dashboard traces.	IHARQTraceRecord , IHARQ logs, and ablation records.
Input validator and degraded-mode handler	Validates required records, identity alignment, probability-vector integrity, split/label-budget consistency, leakage separation, and declared degraded modes before the rule engine acts.	Validation status, degraded-evidence flags, integrity reason codes, and blocked or unavailable decision paths.
Reason-code registry	Owns the finite reason-code vocabulary and maps reason codes to actions, warning classes, dashboard labels, and claim-evidence entries.	Versioned reason-code configuration, explanation text, reason-code metadata, and display labels.
Unsafe-state monitor	Detects severe artifacts, contradictory evidence, high RegimeRisk/stop-loss states, stress-boundary violations, and embodiment-proxy unsafe triggers without claiming clinical safety validation.	Unsafe override flag, unsafe trigger fields, UnsafeEventRecord , and simulation/proxy boundary labels.
Ablation and matched-comparison exporter	Builds Layer 4-local A5/A6/A7 evaluation rows, local submodule-ablation rows, matched-comparison keys, metric-dictionary links, and diagnostic-only flags.	IHARQAblationRecord , Layer 4 EvaluationOutcomeRecord , matched-comparison reports, and metric-dictionary entries.
Dashboard and claim-evidence exporter	Converts validated Layer 4 traces into read-only dashboard/card/export and claim-evidence lineage without recomputing decisions.	Dashboard-ready records, claim-evidence rows, negative-result-note links, export manifests, and trace-completeness warnings.

The detailed dossiers below focus on the decision-path submodules because they own the evidence-verification behavior. The validator, reason-code registry, unsafe-state monitor, ablation exporter, matched-comparison exporter, dashboard exporter, claim-evidence exporter,

metric dictionary, and negative-result-note support are nevertheless required implementation-support submodules because the input-validation, reason-code, evaluation-lineage, dashboard, repository, failure-mode, and definition-of-done contracts already depend on them.

Evidence feature builder

Purpose. The evidence feature builder assembles all relevant evidence into a single feature vector used by the IHARQ decision process. It prevents the rule engine from directly reading scattered prediction, uncertainty, signal-quality, regime, and model-agreement files.

Bottleneck addressed. Upstream records are heterogeneous. Confidence, entropy, margin, stability, agreement, and risk have different sources and missing-data behavior. Without a feature builder, Layer 4 would be difficult to reproduce and easy to turn into ad-hoc threshold code.

Scientific role. It creates the measurable evidence state used to test whether reliability decisions require more than calibrated confidence alone.

Engineering responsibility. It joins aligned `PredictionRecord`, `CalibrationRecord`, `UncertaintyRecord`, `WindowHistoryRecord`, `ArtifactFlagRecord`, `ModelAgreementRecord`, optional `RegimeStateRecord`, and `ThresholdRegistryRecord`. It validates IDs, normalizes feature names, applies missing-data flags, and produces a schema-stable feature vector.

Inputs. Prediction records, calibrated probabilities, uncertainty features, window history, signal-quality flags, channel-subset summaries, model-family agreement, request-count state, previous IHARQ action, and optional `RegimeRisk` fields.

Outputs. Evidence feature vector, draft `EvidenceQualityRecord`, missing-evidence flags, and feature provenance fields.

Internal processing logic. The builder extracts calibrated top-1 probability, top-1/top-2 margin, entropy, confidence, variation ratio, model disagreement, recent-window consistency, channel-subset consistency, signal-quality flags, evidence request count, previous IHARQ action, `RegimeRisk` score, and stop-loss warning. Each feature has a definition, value range, source record, missing-data policy, and thesis interpretation.

Dependencies. Hard dependencies are aligned IDs and valid prediction, calibration, and uncertainty records. Soft dependencies are window history, model-family agreement, and signal-quality flags. Optional dependencies are `RegimeRisk` and conformal proxies. Mockable dependencies include synthetic histories and dummy risk scores.

Consumers. IHARQ rule engine, evidence combiner, fallback router, decision logger, Layer 6 policy features, Layer 7 simulation state, and Layer 10 evidence-flow dashboard.

Feedforward connections. Sends the evidence vector to the rule engine, combiner, fallback router, ablation exporter, and dashboard trace exporter.

Feedback or loopback. If required fields are missing or invalid, it sends validation failure signals to the decision logger and may trigger **Uncertain**, **Request evidence**, or **Unsafe** depending on configuration.

Evaluation method. Measure schema-compliance rate, missing-evidence frequency, feature-completeness rate, feature-outcome association, and whether evidence-quality features separate correct accepted decisions from wrong accepted decisions.

Failure modes. Feature leakage from evaluation labels, ID misalignment, inconsistent model-family naming, hidden missing-data assumptions, and overinterpretation of proxy features.

Reproducibility requirements. Store feature schema version, input file paths, input hashes, configuration hash, seed, model ID, calibration ID, split ID, and missing-data policy.

Thesis evidence produced. Evidence-quality tables, feature distributions, reason-code feature summaries, and evidence-flow plots that show why IHARQ is not raw confidence thresholding.

Limitations and risks. Evidence features are only as trustworthy as upstream records. Signal-quality and RegimeRisk fields may be proxy measures rather than directly validated safety indicators.

Window stability checker

Purpose. The window stability checker decides whether adjacent windows support the same intent or indicate instability.

Bottleneck addressed. A single short EEG window may be ambiguous. A prediction can look confident in one window and flip in the next. Layer 4 needs temporal consistency evidence before accepting action.

Scientific role. It tests whether local temporal stability is associated with accepted-decision reliability and whether request/combine logic improves low-calibration decision quality.

Engineering responsibility. It computes stability features from recent predictions, calibrated probabilities, uncertainties, timestamps, prior IHARQ actions, and combination eligibility constraints.

Inputs. Recent `PredictionRecord` entries, calibrated probabilities, entropy, margin, predicted classes, decision timestamps, window IDs, and trial/session boundaries.

Outputs. Window stability score, flip rate, probability trajectory smoothness, entropy trend, margin trend, accepted-window consistency, combination eligibility flag, and stability reason code.

Internal processing logic. Candidate measures include same-class consistency across recent windows, top-class flip rate, probability trajectory smoothness, entropy trend, margin trend, time distance between windows, agreement among accepted windows, and whether windows are close enough to be interpreted as the same intent state.

Dependencies. Hard dependencies are ordered window IDs and matching trial/session IDs. Soft dependencies are calibrated probability history and prior IHARQ actions. Optional dependencies are longer-window predictions and future-window retrospective evaluation.

Consumers. Rule engine, evidence combiner, RegimeRisk features in Layer 5, Layer 6 policy learning, Layer 7 delay simulation, and Layer 10 timeline plots.

Feedforward connections. Sends stability score and reason code to accept, request, combine, uncertain, and unsafe decisions.

Feedback or loopback. If instability is detected and evidence budget remains, the checker may trigger a request for another window. If instability persists beyond budget, it can trigger fallback, uncertainty, or unsafe depending on severity.

Evaluation method. Compare IHARQ with and without the stability checker; measure wrong-accept rate, coverage, request rate, delay cost, and stability-correctness relation.

Failure modes. Stable wrong predictions, unstable correct predictions, excessive waiting, and combining windows from different intent states.

Reproducibility requirements. Store history length, temporal neighborhood rule, stability metric definitions, threshold IDs, window list, and stability score per decision.

Thesis evidence produced. Stability-correctness plots, flip-rate failure tables, request/combo success rates, and examples of high-confidence unstable predictions.

Limitations and risks. Stability is not correctness. A stable decoder can still be wrong. The thesis must report accepted accuracy and wrong-accept rate rather than treating stability as ground truth.

Channel-subset checker

Purpose. The channel-subset checker tests whether the prediction depends too heavily on unstable, corrupted, or narrow spatial evidence.

Bottleneck addressed. EEG predictions can be fragile to channel dropout, artifacts, or subject-specific channel patterns. A readiness architecture should know when spatial evidence is inconsistent.

Scientific role. It supports the claim that Layer 4 checks spatial robustness rather than trusting a decoder output blindly.

Engineering responsibility. It compares predictions or confidence under declared channel subsets, channel masks, dropout tests, or spatial groups such as left, right, central, motor-area, or dataset-specific subsets when feasible.

Inputs. Channel masks, channel-subset predictions, signal-quality flags, electrode/channel metadata, missing-channel indicators, and optional channel-dropout predictions.

Outputs. Channel stability score, subset agreement, confidence degradation under subset removal, corrupted-channel flag, and channel disagreement reason code.

Internal processing logic. Candidate measures include subset-level class agreement, confidence drop under channel removal, channel-specific artifact flags, sensitivity to missing channels, and discrepancy between motor-area and non-motor-area evidence when channel metadata supports that distinction.

Dependencies. Hard dependencies are channel identity and channel-mask compatibility. Soft dependencies are available channel-subset predictions. Optional dependencies are channel-dropout stress tests and signal-quality audit outputs.

Consumers. Rule engine, fallback router, unsafe-state monitor, Layer 5 dropout-risk estimation, Layer 7 stress simulation, and Layer 10 spatial-robustness dashboard views.

Feedforward connections. Sends channel stability and corrupted-channel flags to accept, request, fallback, and unsafe logic.

Feedback or loopback. Severe channel corruption may trigger fallback to no-action, request a signal-quality check, or produce unsafe block. Mild spatial disagreement may request evidence or tighten acceptance.

Evaluation method. Ablate the channel-subset checker; evaluate performance under channel-dropout stress; measure unsafe-accept rate, wrong-accept rate, request rate, and accepted accuracy under spatial perturbations.

Failure modes. Channel-subset tests may be unavailable for some models; overly strict spatial rules may reduce coverage; channel metadata can differ across datasets.

Reproducibility requirements. Store channel grouping definitions, channel masks, dropout settings, subset prediction file paths, and thresholds.

Thesis evidence produced. Channel-stability tables, dropout robustness curves, corrupted-channel failure examples, and ablations showing whether spatial checks add value.

Limitations and risks. Spatial robustness is dataset- and montage-dependent. It should be presented as evidence robustness under public EEG records, not as a claim about real electrode safety.

Model-family agreement checker

Purpose. The model-family agreement checker determines whether different decoder families support the same intent.

Bottleneck addressed. A single decoder can be confidently wrong. Agreement across classical, Riemannian, deep, compact SSL, ensemble, or fallback families can provide additional evidence, while disagreement can expose fragile decisions.

Scientific role. It separates IHARQ from ordinary ensemble voting. Agreement is used as an evidence feature, but IHARQ can still wait, request evidence, combine, fallback, mark uncertainty, or declare unsafe.

Engineering responsibility. It aligns model-family predictions by dataset, subject, session, trial, window, split, and seed; computes agreement features; and records disagreement reason codes.

Inputs. Predictions from classical, Riemannian, deep, compact SSL, ensemble, or fallback models; validation trust scores; calibrated probabilities when available; model-family registry entries; and low-calibration budget metadata.

Outputs. Agreement score, majority or weighted vote summary, entropy of model-family votes, calibrated probability agreement, trusted-family set, strong-vs-weak baseline disagreement, and disagreement reason code.

Internal processing logic. Candidate measures include majority agreement, weighted agreement by validation trust, entropy of model-family votes, pairwise family disagreement, calibrated top-class agreement, and mismatch between the target model and conservative/fallback models.

Dependencies. Hard dependencies are aligned trial/window IDs and consistent class labels across models. Soft dependencies are calibrated probabilities across families. Optional dependencies are Layer 5 model trust scores.

Consumers. Rule engine, evidence combiner, fallback router, Layer 6 policy learner, Layer 7 simulation, and Layer 10 model-agreement dashboard.

Feedforward connections. Sends agreement features to accept, request, combine, fallback, and uncertain logic.

Feedback or loopback. Strong disagreement may trigger request evidence, fallback to a trusted family, or unsafe state if paired with high RegimeRisk or severe instability.

Evaluation method. Compare IHARQ with and without model-family agreement; compare against ordinary ensemble voting; measure wrong-accept rate, accepted accuracy, coverage, and model-disagreement failure concentration.

Failure modes. Apparent agreement can reflect shared bias; families may not all be available

for every dataset; weighted trust can leak if selected on evaluation data.

Reproducibility requirements. Store aligned model list, model registry version, trust-weight source, calibration status, agreement metric, and split-safe selection rule.

Thesis evidence produced. Model-family disagreement tables, ensemble-vs-IHARQ comparisons, disagreement reason-code analysis, and ablations proving IHARQ is more than ensemble voting.

Limitations and risks. Agreement is not truth. It is an evidence feature and must be evaluated against labels and simulated outcomes.

IHARQ rule engine

Purpose. The IHARQ rule engine converts the evidence feature vector into an interpretable readiness action.

Bottleneck addressed. Evidence features alone do not define action. The architecture needs a deterministic, auditable decision ladder that reviewers can inspect, reproduce, and ablate.

Scientific role. It operationalizes the thesis claim that readiness should be controlled by explicit multi-evidence verification rather than raw decoder acceptance.

Engineering responsibility. It applies ordered rules for input validity, unsafe priority, signal quality, RegimeRisk, confidence, uncertainty, stability, channel robustness, model agreement, evidence combination, request budget, fallback, uncertainty, and acceptance.

Inputs. Evidence feature vector, thresholds, request budget, combination configuration, fallback availability, risk context, threshold registry entries, and action-cost configuration.

Outputs. IHARQDecisionRecord, reason code, reason text, rule path, action cost, accepted class if any, fallback and unsafe flags, and evidence request count.

Internal processing logic. The recommended decision priority is: validate inputs; check hard unsafe conditions; check severe signal-quality failure; apply RegimeRisk tightening if active; evaluate confidence and uncertainty; evaluate window stability; evaluate channel-subset stability; evaluate model-family agreement; test combination eligibility; decide request evidence; decide fallback; decide uncertain; accept only if no higher-priority concern blocks acceptance; log the rule path.

Dependencies. Hard dependencies are evidence feature schema, threshold registry, action vocabulary, and deterministic rule order. Soft dependencies are stability, channel, agreement, and signal-quality features. Optional dependencies are RegimeRisk-aware rules and stretch simulation states.

Consumers. Evidence combiner, fallback router, decision logger, Layer 6 policy learning, Layer 7 simulation, Layer 9 embodiment gating, and Layer 10 dashboard.

Feedforward connections. Sends final action, reason, accepted class, and action cost to downstream layers.

Feedback or loopback. When action is **Request evidence**, the rule engine emits a request signal to the window-history/evidence-acquisition component. When action is **Fallback** or **Unsafe**, it emits a conservative signal to Layer 7/9 simulation modes. These loops are bounded by declared budgets and must never use evaluation labels for ordinary decisions.

Evaluation method. Compare full IHARQ against confidence-only IHARQ, selective prediction, entropy rejection, margin rejection, multi-window voting, probability averaging, ordinary

ensembles, fallback-only rules, and regime-unaware IHARQ.

Failure modes. Arbitrary thresholds, over-requesting, excessive fallback, hidden rule-order conflicts, over-abstention, and unintended equivalence to confidence thresholding.

Reproducibility requirements. Store rule-engine configuration, threshold file, reason-code registry, rule-order version, action-cost table, and decision traces.

Thesis evidence produced. Matched-coverage and matched-cost comparisons, ablation tables, reason-code distributions, action timelines, and wrong-accept reduction analysis.

Limitations and risks. Rule-based logic is interpretable but may not be optimal. Layer 6 may learn policies later, but Layer 4 must remain the transparent reference policy.

Evidence combiner

Purpose. The evidence combiner fuses multiple windows or sources when individual evidence is weak but consistent.

Bottleneck addressed. One window may not meet acceptance criteria, but several temporally close and mutually consistent weak windows may produce stronger evidence. Conversely, careless combination can hide contradiction.

Scientific role. It makes the “request” and “combine” parts of IHARQ concrete and testable, allowing comparison against longer-window decoding, majority voting, and probability averaging.

Engineering responsibility. It checks combination eligibility, applies the configured combination method, records the sources used, and updates the decision only when combination is allowed.

Inputs. Window history, calibrated probabilities, logits when available, predicted classes, uncertainty values, evidence weights, RegimeRisk trust weights if active, channel/model agreement, and combination configuration.

Outputs. `CombinedEvidenceRecord`, combined class, combined probability or vote summary, before/after confidence, before/after entropy, before/after margin, combination reason, and optional updated `IHARQDecisionRecord`.

Internal processing logic. Possible methods include probability averaging, logit averaging, majority vote, weighted vote by confidence, weighted vote by model trust, weighted vote by RegimeRisk trust, temporal smoothing, and conformal-set intersection or union if Layer 3 supplies conformal proxies.

Dependencies. Hard dependencies are valid aligned windows and declared combination method. Soft dependencies are calibrated probabilities and uncertainty history. Optional dependencies are RegimeRisk weights or conformal proxies.

Consumers. Rule engine, decision logger, Layer 6 policy learner, Layer 7 simulation, and Layer 10 request/combine view.

Feedforward connections. Sends combined evidence record to decision logger and downstream layers; may send updated action if combined evidence crosses acceptance or fallback thresholds.

Feedback or loopback. If combination fails because evidence is contradictory or too old, the signal returns to the rule engine to request evidence, fallback, uncertain, or unsafe action.

Evaluation method. Compare against no-combine IHARQ, longer-window decoding, multi-window majority voting, and probability averaging. Measure wrong-accept rate, accepted accuracy, delay, evidence count, and contradiction rate.

Failure modes. Combining windows from different intent states, averaging away uncertainty, hiding contradictory evidence, and increasing delay too much.

Reproducibility requirements. Store window IDs, method, weights, eligibility rule, thresholds, before/after values, and configuration hash.

Thesis evidence produced. Combination success/failure tables, evidence-budget curves, before/after reliability plots, and examples where request-plus-combine changes the decision.

Limitations and risks. Combination improves evidence only if consistency assumptions hold. If public EEG windowing does not support meaningful request simulation, combination must be labeled as offline proxy analysis.

Fallback router

Purpose. The fallback router selects a safer path when the main evidence is insufficient or risky.

Bottleneck addressed. Rejecting a prediction is not always enough. The system may need a conservative decoder, no-action state, hold state, or supervisor-proxy route depending on context.

Scientific role. It demonstrates that Layer 4 represents readiness-control alternatives, not only accept/reject thresholding.

Engineering responsibility. It maps evidence conditions to declared fallback routes, checks route availability by context, computes fallback cost, and records fallback decisions.

Inputs. Evidence features, fallback availability table, RegimeRisk score, model trust scores, signal-quality state, action-cost model, simulation mode, and embodiment compatibility flags.

Outputs. `FallbackDecisionRecord`, selected fallback route, fallback reason, fallback target, fallback cost, fallback availability, and fallback outcome during evaluation.

Internal processing logic. Recommended hierarchy is: more evidence if available and low cost; conservative decoder if trusted; model family with higher current trust; no-action safe mode; simulated supervisor/expert proxy only in simulation; unsafe block if no reliable fallback exists.

Dependencies. Hard dependencies are declared fallback vocabulary and route-availability configuration. Soft dependencies are model trust scores and simulation context. Optional dependencies are supervisor-proxy and MyoSuite/OpenSim hold/stop commands.

Consumers. Rule engine, decision logger, Layer 6 defer/policy modules, Layer 7 simulation, Layer 9 embodiment adapter, and Layer 10 fallback dashboard.

Feedforward connections. Sends fallback status and route to downstream policies and simulation.

Feedback or loopback. If fallback fails or is unavailable, the router returns an unresolved-risk signal to the rule engine, which may choose **Uncertain** or **Unsafe**. In simulation, repeated fallback failure can contribute to Layer 5 stop-loss logic.

Evaluation method. Compare full IHARQ with no-fallback IHARQ and fallback-only base-

lines. Measure wrong-accept rate, fallback rate, fallback correctness, action cost, delay, and simulation reward.

Failure modes. Over-fallback, fallback to a weak model, hidden coverage reduction, and inconsistent fallback availability across offline and simulation modes.

Reproducibility requirements. Store fallback route table, availability matrix, cost model, selected route, unavailable routes, and outcome when evaluated.

Thesis evidence produced. Fallback frequency/cause/outcome tables, cost-coverage curves, and qualitative cases where fallback prevented wrong acceptance.

Limitations and risks. Fallback does not imply clinical safety. In offline public EEG, fallback is a simulated or evaluative decision route.

Decision logger

Purpose. The decision logger makes every IHARQ action reproducible, explainable, ablation-ready, and dashboard-ready.

Bottleneck addressed. Without trace logging, IHARQ would be an opaque rule script. The thesis needs records that can regenerate figures, tables, cards, and failure analyses.

Scientific role. It enables matched comparisons, reason-code analysis, and criticism defense.

Engineering responsibility. It stores input IDs, feature values, thresholds, rule path, action, reason code, fallback and unsafe flags, evidence request counts, combined-evidence details, evaluation labels when allowed, and simulated outcomes when available.

Inputs. All submodule outputs, final decision, thresholds, config hashes, input record paths, and evaluation outcomes when in evaluation mode.

Outputs. IHARQTraceRecord, IHARQAblationRecord, decision logs, dashboard tables, and reproducibility manifest.

Internal processing logic. The logger writes row-level records for each decision and aggregate summaries for action distribution, reason-code distribution, coverage, wrong-accept rate, request rate, fallback rate, unsafe rate, and trace completeness.

Dependencies. Hard dependencies are schemas, storage paths, run manifest, and configuration hash. Soft dependencies are dashboard export specification. Optional dependencies are simulation and embodiment outcome records.

Consumers. Layer 10, thesis results chapter, Layer 6 training data builder, Layer 7 simulation analysis, and reproducibility package.

Feedforward connections. Sends saved records to downstream dashboards, policies, and evaluation scripts.

Feedback or loopback. Trace completeness failures should block thesis-level figure generation and send a rebuild/exclusion signal to the experiment controller.

Evaluation method. Measure trace-completeness rate, schema-validation rate, figure regeneration success, and consistency between logs and aggregate metrics.

Failure modes. Missing reason codes, unlogged thresholds, inconsistent configuration hashes, dashboard-only metrics that cannot be regenerated, and mixing evaluation labels into decision-time records.

Reproducibility requirements. Store parquet/CSV records, JSON run manifest, YAML configs, schema version, seed, environment file, input hashes, and output hashes.

Thesis evidence produced. Decision tables, dashboard views, reliability cards, ablation tables, and reproducibility appendices.

Limitations and risks. Complete logs can be storage-heavy. The implementation should separate raw per-window logs from aggregated dashboard records.

10.10 Decision logic

Layer 4 should use a transparent decision ladder. The decision order must be explicit so that two implementations with the same inputs and configuration produce the same action.

1. Validate input records and detect leakage or ID mismatch.
2. Check hard unsafe conditions and stop-loss warnings.
3. Check severe signal-quality failure and channel corruption.
4. Apply RegimeRisk threshold tightening if Layer 5 is active.
5. Evaluate calibrated confidence, entropy, margin, and uncertainty features.
6. Evaluate adjacent-window stability.
7. Evaluate channel-subset stability.
8. Evaluate model-family agreement.
9. Decide whether evidence can be combined.
10. Decide whether more evidence should be requested within budget.
11. Decide whether fallback is safer and available.
12. Decide whether the state is uncertain.
13. Accept only if no higher-priority rule blocks acceptance.
14. Log the final decision, reason code, thresholds, and evidence trace.

The system should first determine whether the input is valid. If the input is invalid or contradictory, it should not proceed to acceptance. It should then determine whether any unsafe condition is active. Unsafe checks override acceptance. If the signal is poor, channels are corrupted, RegimeRisk is high, or evidence sources contradict each other, the system should block action, fallback, or request evidence depending on severity and available alternatives.

Only after safety and validity checks pass should the system evaluate whether the current prediction is strong enough to accept. Acceptance should require not only high calibrated confidence, but also acceptable uncertainty, stability, agreement, and regime state when applicable.

Condition	Preferred action
-----------	------------------

Input missing or corrupt	Uncertain or Unsafe depending on severity; always log integrity reason code.
Severe signal-quality failure	Unsafe or Fallback .
High confidence, low entropy, high margin, stable windows, acceptable channel stability, acceptable model agreement, low risk	Accept .
Moderate confidence and stable evidence with budget available	Request evidence or Combine evidence .
Low confidence with fallback available	Fallback .
Low confidence with no fallback and no explicit danger	Uncertain .
Contradictory evidence across windows, channels, or models	Request evidence , Fallback , or Unsafe depending on severity and risk.
High RegimeRisk	Tighten thresholds; route to Request evidence , Fallback , or Unsafe if severe.
Repeated wrong accepts in simulation	Trigger stop-loss, Fallback , or Unsafe .

10.11 Evidence request and combination behavior

The word “request” in IHARQ must be operationalized. In this thesis, an evidence request can mean another temporal window, a longer window, a repeated prediction under the same model, a prediction from another model family, a channel-subset verification, a signal-quality check, a RegimeRisk update, or a fallback decoder output. Public EEG may not provide real online reacquisition; therefore, offline request behavior should be labeled as replay-style or simulated evidence acquisition.

Layer 4 defines an evidence budget:

Budget type	Meaning
Maximum windows	Maximum number of extra windows that may be requested or combined.
Maximum wait time	Maximum simulated delay before the action becomes too late or costly.
Maximum uncertainty	Upper bound above which evidence is too weak to combine.
Maximum request count	Number of repeated requests allowed before fallback, uncertain, or unsafe.
Maximum cost	Total allowed cost for delay, request, fallback, or combination.
Maximum regime risk	Risk threshold above which combination is prohibited or fallback/unsafe is required.

Evidence should be combined only when windows belong to the same trial or valid temporal neighborhood; predicted classes are consistent enough; uncertainty is not extreme; signal quality

is acceptable; RegimeRisk does not prohibit combination; the combination method is declared in configuration; and combination does not hide contradiction. Every combination must produce the list of combined windows, method used, pre-combination confidence, post-combination confidence, pre-combination entropy, post-combination entropy, combined class, combination reason, and final action.

10.12 Fallback and unsafe-state handling

Fallback and unsafe states are central to making Layer 4 more than thresholding. A recommended fallback hierarchy is:

1. Request more evidence if available and low cost.
2. Use a conservative decoder if trusted.
3. Use a model family with higher current trust.
4. Use no-action safe mode.
5. Use simulated supervisor or expert proxy only in Layer 7 simulation.
6. Block action as unsafe if no reliable fallback exists.

Context	Allowed fallback routes
Offline EEG evaluation	Conservative decoder, reject/abstain, request window in replay, uncertainty label, or no-action proxy.
Layer 7 simulation	Wait, request evidence, fallback controller, safe no-action, supervisor proxy, and cost-aware delay.
StressForge condition	Same as simulation, but stress-aware costs and stress triggers are logged.
MyoSuite/OpenSim demo	Hold, no-action, conservative assistive command, or unsafe stop in simulated embodiment only.
Dashboard/reporting	Explanation only. The dashboard should not modify decisions.

Unsafe should be reserved for severe conditions, not ordinary uncertainty. Possible unsafe triggers are severe signal artifact, contradictory evidence across windows and models, high RegimeRisk plus low stability, repeated wrong accepts in simulated environment, stress state exceeding a declared safety boundary, embodiment proxy reporting unstable movement or high endpoint error, and fallback unavailable while acceptance risk is high.

The distinction is important:

State	Meaning
Uncertain	The system cannot justify a prediction under current evidence, but no explicit danger condition has been detected.

Unsafe	The system has detected a declared condition under which action should be blocked.
---------------	--

Layer 4 must not claim that unsafe-state detection proves clinical safety. It identifies unsafe evidence states under the architecture's declared public-data, replay, stress, and simulation criteria.

10.13 RegimeRisk feedback integration

Layer 5 feeds Layer 4 by tightening or modifying evidence decisions. This feedback must be explicit, bounded, and ablatabe.

Layer 4 consumes from **RegimeStateRecord**: regime label, RegimeRisk score, risk trend, volatility-like feature, drawdown-like feature, drift risk, fatigue-like risk, dropout risk, model trust scores, and stop-loss warning.

RegimeRisk condition	Layer 4 effect
Stable regime	Use normal thresholds and ordinary action costs.
Mild risk	Slightly stricter acceptance threshold or lower request threshold.
Drift-risk	Require stronger window stability or request additional evidence.
Fatigue-risk	Reduce accept aggressiveness and increase wait/fallback likelihood.
Dropout-risk	Increase channel-subset checks and fallback likelihood.
High volatility	Increase evidence request or combination requirements.
Stop-loss warning	Block acceptance or route fallback according to stop-loss severity.
Unsafe regime	Produce Unsafe or no-action block under declared criteria.

Every RegimeRisk-to-IHARQ link must document source, target, signal, effect, trigger, action, safety constraint, and evaluation logic.

Feedback field	Specification
Source	RegimeStateRecord from Layer 5.
Target	Evidence feature builder, IHARQ rule engine, evidence combiner, fallback router, and unsafe-state monitor.
Signal	RegimeRisk score, regime label, risk trend, model trust scores, and stop-loss warning.
Trigger	Risk score crosses configured threshold, risk trend worsens, stop-loss is active, or model trust changes.

Action	Tighten thresholds, reduce combination eligibility, request evidence, increase fallback likelihood, or block unsafe action.
Safety constraint	RegimeRisk must never use future labels at decision time and must not be interpreted as clinical safety validation.
Evaluation	Compare regime-aware IHARQ against regime-unaware IHARQ under matched coverage, matched cost, and stress conditions.
Risk	Over-abstention, excessive fallback, threshold overfitting, and false risk alarms.

10.14 Feedforward and loopback behavior

Layer 4 has controlled feedforward outputs and limited loopbacks. It should not draw arbitrary feedback loops. Each loop must have a scientific and engineering reason.

Source	Target	Signal	Trigger	Action and evaluation
Rule engine	Window history or evidence acquisition module	Evidence request	Ambiguous evidence with remaining request budget	Acquire/replay next window or additional evidence; evaluate request success, delay cost, and wrong-accept reduction.
Evidence combiner	Rule engine	Combination success/failure	Multiple weak windows are available	If eligible, update evidence; if contradictory, request, fallback, uncertain, or unsafe; evaluate against no-combine baseline.
Channel checker	Rule engine and fallback router	Spatial instability flag	Channel-subset disagreement or severe channel artifact	Tighten accept, request channel check, fallback, or unsafe; evaluate under dropout stress.
Model-family agreement checker	Rule engine and fallback router	Model disagreement/trusted family set	Cross-family disagreement or trusted model available	Request evidence, fallback to trusted family, or uncertainty; compare against ordinary ensemble voting.

Layer 5 RegimeRisk	Rule engine	Threshold tightening or stop-loss warning	Elevated risk, drift, fatigue-like risk, dropout risk, or stop-loss	Tighten thresholds, block accept, fallback, or unsafe; evaluate regime-aware versus regime-unaware IHARQ.
Decision logger	Experiment con- troller/Layer 10	Trace- completeness warning	Missing reason code, threshold, input hash, or schema field	Rebuild record or exclude figure with failure note; evaluate trace completeness.
Layer 7 simulation	Layer 5 and Layer 4 logs	Repeated wrong accepts or unsafe outcomes	Closed-loop stress run reveals failure bursts	Layer 5 may emit stop-loss context in later simulated episodes; ordinary offline Layer 4 decisions must not use future outcomes.

Feedforward outputs go to Layer 6 as policy features and pseudo-decisions, Layer 7 as simulated actions and costs, Layer 9 as simulation-only gating status, and Layer 10 as traceable dashboard artifacts.

10.15 Evaluation and ablation design

Layer 4 must be evaluated as a contribution, not only described as an architecture idea. After the Ablation and Evaluation Architecture merge, Layer 4 also owns the local evidence-readiness slice of the global ablation ladder. Its primary global ablation identity is *A5* for rule-based IHARQ-lite; it participates in *A6* when the evidence-quality estimator is evaluated against rule-only IHARQ; and it participates in *A7* when RegimeRisk-aware IHARQ is compared against regime-unaware IHARQ. These global IDs must be preserved in exported records, dashboard filters, thesis tables, and claim-evidence matrix entries even when Layer 4 also keeps internal submodule-ablation names.

The evaluation should ask whether IHARQ reduces wrong accepted decisions at matched coverage, improves accepted-decision reliability compared with thresholding, uses evidence requests or combinations effectively, avoids unsafe accepted decisions under stress, benefits from RegimeRisk context, and remains explainable. A Layer 4 result is thesis-ready only when the matched-comparison key, ablation-step ID, metric dictionary entry, source-record pointers, dashboard/card lineage, and claim-boundary label are saved. If these are missing, the result may be used for debugging but must be marked diagnostic-only.

Baseline	Reason for comparison
Accept-all decoder	Measures the cost of forced classification without evidence verification and maps to the global <i>A0</i> reference supplied by Layer 2.

Confidence thresholding	Tests whether IHARQ is more than top-probability rejection and maps to the global <i>A2</i> threshold baseline.
Entropy rejection	Tests against a standard uncertainty rejection baseline and maps to the global <i>A3</i> uncertainty/selective-prediction family.
Margin rejection	Tests against top-class separation and maps to the global <i>A3</i> uncertainty/selective-prediction family.
Calibrated probability thresholding	Tests whether post-calibration thresholding alone is enough and remains part of the global <i>A2</i> registered-threshold comparison.
Selective prediction	Uses Layer 3 risk-coverage baselines as formal global <i>A3</i> controls.
Longer-window decoding	Tests whether request/combine behavior is just longer-window smoothing and must report explicit delay burden.
Multi-window majority voting	Tests against simple temporal aggregation from the global <i>A4</i> ordinary-combination control.
Probability averaging	Tests against ordinary evidence fusion from the global <i>A4</i> control.
Ordinary ensembles	Tests whether model-family agreement is merely ensemble voting rather than reason-coded verification.
Fallback-only baseline	Tests whether full IHARQ adds value beyond conservative routing.
Regime-unaware IHARQ	Tests the contribution of Layer 5 feedback under matched temporal sequences and stress schedules.
Oracle-like upper bound	Optional and clearly labeled; estimates headroom but cannot be used as a fair deployable baseline or claim-supporting result.

Required ablations are:

Ablation	Purpose
Remove window stability checker	Tests value of temporal consistency.
Remove channel-subset checker	Tests value of spatial robustness.
Remove model-family agreement checker	Tests value of cross-model evidence.
Remove evidence combiner	Tests value of multi-window accumulation.
Remove fallback router	Tests value of safe routing alternatives.
Remove unsafe-state monitor	Tests event-blocking and unsafe-accept reduction.

Remove RegimeRisk feedback	Tests value of temporal trust context; this is the local Layer 4 view of the global <i>A7</i> comparison.
Use confidence-only IHARQ	Tests whether IHARQ is more than thresholding.
Use entropy-only or margin-only IHARQ	Tests whether the rule engine merely repackages a single uncertainty score.
Remove reason-code logging from analysis	Tests whether explanation artifacts are necessary for thesis interpretation, while not changing decision outcomes.
Disable claim-evidence export	Diagnostic-only stress test for reporting completeness; it must not be used as a thesis-ready scientific result.

Evaluation questions include: Does Layer 4 reduce wrong accepted decisions at matched coverage? Does it improve accepted-decision reliability compared with confidence thresholding, entropy rejection, margin rejection, calibrated selective prediction, longer windows, voting, averaging, and ordinary ensembles? Does requesting or combining evidence improve decisions under low calibration after accounting for delay and evidence cost? Does fallback reduce high-risk acceptance without hiding bad decoder performance? Does RegimeRisk-aware IHARQ outperform regime-unaware IHARQ under temporal instability? Which submodule contributes most? When does Layer 4 fail, become over-conservative, or add traceability without improving performance?

Matched-comparison discipline is mandatory. Layer 4 comparisons must match dataset, subject, session, trial/window identity where feasible, split ID, preprocessing ID, label-budget ID, decoder family, seed, calibration ID, threshold registry entry, IHARQ configuration, evidence-budget configuration, fallback-route configuration, RegimeRisk configuration when active, stress profile when active, and dashboard export configuration. Unmatched comparisons may appear only with a visible diagnostic label and must not support the core IHARQ claim.

Every Layer 4 ablation row should include: global ablation ID, local ablation ID, system-variant configuration, baseline ID, matched-comparison key, primary metric set, secondary metric set, dashboard view ID, card/report pointer, limitation statement, and negative-result note when the module does not improve the declared trade-off. Negative results are not failures to hide; they are evidence that IHARQ is redundant, over-conservative, threshold-equivalent, or useful only under narrow evidence conditions.

10.16 Metrics

Layer 4 needs metrics beyond raw accuracy. Each metric must appear in the metric dictionary with a definition, denominator, direction, required record fields, aggregation level, supported claim, and limitation note. Subject-level aggregation is required wherever subject IDs exist; session-level aggregation should be reported where available. Pooled-only reporting weakens the claim unless clearly justified.

Core readiness metric	Meaning
Accepted accuracy	Accuracy among accepted decisions.
Coverage	Fraction of eligible cases accepted.

Effective coverage	Fraction of eligible cases accepted after accounting for fallback, unsafe, wait, request, or no-action routing.
Wrong-accept rate	Fraction of all or accepted cases incorrectly accepted; the denominator must be stated.
Unsafe-accept rate	Fraction of high-risk, unsafe-labeled, stress-flagged, or simulation-unsafe cases incorrectly accepted.
Evidence request rate	How often additional evidence is requested.
Average evidence count	Average number of windows or sources used before decision.
Fallback rate	Frequency of fallback decisions.
Uncertain rate	Frequency of undecidable cases.
Unsafe rate	Frequency of blocked unsafe states.
Decision delay	Cost of waiting or requesting evidence.
Action cost	Declared cost of accept, wait, request, combine, fallback, uncertain, or unsafe.
Matched-cost delta	Difference against baseline under the same declared cost model.
Matched-coverage delta	Difference against baseline at comparable coverage or with coverage-normalized reporting.

Reliability or explanation metric	Meaning
Accepted ECE	Calibration error among accepted cases.
High-confidence wrong rate	Frequency of wrong high-confidence cases.
Risk-coverage curve	Reliability as coverage varies.
Confidence-risk alignment	Whether low risk corresponds to correctness.
Stability-correctness relation	Whether stable evidence is more correct than unstable evidence.
Reason-code distribution	Which reasons dominate decisions.
Reason-code error concentration	Which reasons are associated with failures.
Submodule trigger frequency	How often each checker changes a decision.
Evidence-combination success	Whether combined evidence improves confidence, entropy, correctness, or action cost relative to the pre-combination decision.
Fallback routing success	Whether fallback reduces wrong accepts or unsafe events without excessive burden.
Dashboard trace completeness	Fraction of decisions with complete explainability, lineage, and claim-boundary fields.
Claim-evidence completeness	Fraction of Layer 4 claims backed by required records, metrics, figures, cards, limitations, and invalidation conditions.

When Layer 4 feeds Layer 7, additional simulation metrics include task success under IHARQ, late-assist rate, over-assist rate, unsafe event count, reward contribution, stop-loss compliance, and stress-conditioned action cost. These metrics are simulation-derived and must inherit the Layer 0 claim-safety and Ablation and Evaluation Architecture warnings that they do not validate real clinical closed-loop safety.

10.17 Stress tests and failure modes

Layer 4 must be tested under difficult conditions because its value should appear when evidence is degraded.

Stress condition	Expected Layer 4 behavior
Delay	Balance waiting and requesting evidence against lateness cost.
Dropout	Increase uncertainty, channel checks, fallback, or unsafe blocks.
Noisy intent	Increase request, combine, fallback, or uncertain actions.
Drift	Rely more on RegimeRisk feedback and tighten acceptance.
Fatigue-like degradation	Reduce accept aggressiveness and increase fallback or wait.
Combined stress	Expose limits of IHARQ decision logic and reveal failure clusters.

Failure mode	Defense or required analysis
IHARQ becomes simple thresholding	Include stability, channel robustness, agreement, combination, fallback, unsafe states, and matched baselines.
Over-requests evidence	Include request budget, delay cost, late-assist rate, and ablation.
Over-fallbacks	Track fallback cost, fallback correctness, matched coverage, and action cost.
Combines contradictory windows	Enforce eligibility rules and contradiction logging.
Hides bad decoder performance	Report coverage, wrong-accept rate, rejected/uncertain rate, and raw decoder baseline.
Depends on arbitrary thresholds	Use threshold registry, sensitivity analysis, selection split, and leakage checks.
Becomes too complex	Keep submodules ablatable and independently logged.
Overclaims safety	Use Layer 0 claim-safety wording and public-data/simulation boundaries.
RegimeRisk causes excessive abstention	Compare regime-aware versus regime-unaware IHARQ at matched cost and coverage.

10.18 Reproducibility and implementation artifacts

Layer 4 must be reproducible from saved records. Recommended package structure is:

```
src/layer4_iharq/
  __init__.py
  schemas.py
  input_validator.py
  evidence_feature_builder.py
  window_stability.py
  channel_subset_checker.py
  model_family_agreement.py
  rule_engine.py
  evidence_combiner.py
  fallback_router.py
  unsafe_state_monitor.py
  reason_codes.py
  decision_logger.py
  ablations.py
  matched_comparison_validator.py
  metric_dictionary.py
  negative_result_notes.py
  claim_evidence_export.py
  dashboard_export.py
  run_layer4_iharq.py
```

Recommended configuration files are configs/layer4_iharq/rule_engine.yaml, configs/layer4_iharq/thresholds.yaml, configs/layer4_iharq/evidence_budget.yaml, configs/layer4_iharq/combination_methods.yaml, configs/layer4_iharq/fallback_routes.yaml, configs/layer4_iharq/reason_codes.yaml, configs/layer4_iharq/action_costs.yaml, configs/layer4_iharq/regime_feedback.yaml, configs/layer4_iharq/ablation_plan.yaml, configs/layer4_iharq/matched_comparison.yaml, and configs/layer4_iharq/metric_dictionary.yaml.

Recommended output folders are:

```
results/layer4_iharq/decision_records/
results/layer4_iharq/evidence_quality/
results/layer4_iharq/combined_evidence/
results/layer4_iharq/fallback_logs/
results/layer4_iharq/unsafe_events/
results/layer4_iharq/trace_records/
results/layer4_iharq/ablation_tables/
results/layer4_iharq/evaluation_outcomes/
results/layer4_iharq/matched_comparisons/
results/layer4_iharq/negative_result_notes/
manifests/layer4_iharq/claim_evidence/
figures/layer4_iharq/
cards/layer4_iharq/
```

A reproducible command template is:

```
python run_layer4_iharq.py \
  --prediction-records results/layer2/predictions.parquet \
  --calibration-records results/layer3/calibration_records.parquet \
  --uncertainty-records results/layer3/uncertainty_records.parquet \
  --threshold-registry results/layer3/threshold_registry.yaml \
  --regime-state-records results/layer5/regime_state_records.parquet \
  --config configs/layer4_iharq/rule_engine.yaml \
  --seed 42 \
  --out results/layer4_iharq/
```

Every Layer 4 figure or table in the thesis should be traceable to an output file, configuration hash, split ID, model ID, calibration ID, threshold registry entry, seed, source input records, ablation-step ID, matched-comparison key, metric dictionary entry, dashboard export ID, and claim-evidence matrix row. A result should not be reported if its decision records cannot be regenerated or if its comparison is unmatched without a diagnostic-only label.

10.19 Dashboard, cards, and thesis outputs

Layer 4 should provide Layer 10 with the following dashboard views: IHARQ timeline, reason-code distribution, evidence-flow view, request/combine view, fallback view, unsafe-event view, baseline comparison view, ablation view, matched-comparison view, stress-conditioned IHARQ view, trace-completeness view, and claim-evidence coverage view.

Each IHARQ reliability card should include dataset and split, model family, calibration method, IHARQ configuration, global/local ablation IDs, matched-comparison key, action distribution, accepted accuracy, coverage, wrong-accept rate, evidence request rate, fallback rate, unsafe rate, top reason codes, most common failure modes, ablation summary, negative-result note if applicable, trace-completeness status, dashboard/export lineage, and claim-boundary statement.

Expected thesis artifacts include accepted-accuracy/coverage curves, wrong-accept reduction tables, matched-cost and matched-coverage comparisons, request-budget curves, evidence-combination before/after plots, fallback cause/outcome tables, unsafe-event summaries, reason-code distributions, submodule ablation tables, regime-aware versus regime-unaware comparisons, negative-result notes for threshold-equivalent or over-conservative variants, and qualitative case studies of high-confidence wrong predictions blocked by IHARQ.

Dashboard behavior is read-only. Layer 10 may filter, aggregate, visualize, and export Layer 4 outputs, but it must not recompute IHARQ decisions, retune thresholds, repair missing evidence records, or change ablation labels. Missing lineage must produce a visible warning and block thesis-ready export for the affected Layer 4 claim.

10.20 Outputs

Layer 4 outputs `IHARQDecisionRecord`, `EvidenceQualityRecord`, `CombinedEvidenceRecord`, `FallbackDecisionRecord`, `UnsafeEventRecord`, `IHARQAblationRecord`, `IHARQTraceRecord`, Layer 4 `EvaluationOutcomeRecord` files, matched-comparison reports, metric-dictionary entries, reason-code distributions, action-cost tables, request/combine logs, fallback logs, unsafe-action logs, negative-result notes, dashboard-ready records, claim-evidence matrix entries, and thesis-ready ablation tables. The most important formal output is `IHARQDecisionRecord`, because it is the contract through which later layers consume readiness decisions.

10.21 Outgoing links

Layer 4 feeds Layer 6 with readiness-action candidates, evidence-quality features, reason-coded pseudo-decisions, coverage/cost trade-offs, fallback labels, unsafe flags, uncertainty labels, and action costs. It feeds Layer 7 with simulated readiness decisions, request/wait delays, evidence budgets, fallback/no-action routes, unsafe blocks, and action-cost fields. It feeds Layer 9 indirectly through simulation-compatible gating records such as hold, no-action, conservative assistive command, or unsafe stop. It feeds Layer 10 with IHARQ timelines, evidence-flow plots, reason-code distributions, fallback and unsafe-event views, baseline comparisons, ablation tables, reproducibility manifests, and reliability cards.

Layer 4 receives optional loopback context from Layer 5 RegimeRisk and from Layer 7 simulated outcomes only through declared evaluation or subsequent simulated episodes. It must not use future labels or future outcomes to make ordinary offline decisions.

10.22 Definition of done

Layer 4 is complete when the action vocabulary is formalized; all input and output contracts are schema-defined; `IHARQDecisionRecord`, `EvidenceQualityRecord`, `CombinedEvidenceRecord`, `FallbackDecisionRecord`, `UnsafeEventRecord`, `IHARQAblationRecord`, `IHARQTraceRecord`, and Layer 4 `EvaluationOutcomeRecord` exports are generated; every decision has a reason code; input validation and degraded modes are implemented; window stability, channel-subset stability, model-family agreement, evidence combination, fallback routing, unsafe-state handling, and decision logging are independently ablatable; confidence-only, entropy, margin, selective-prediction, longer-window, voting, averaging, ensemble, fallback-only, and regime-unaware baselines are evaluated at matched coverage, cost, or declared diagnostic status; RegimeRisk-aware mode is available when Layer 5 records exist; dashboard-ready views and reliability cards are exported; claim-evidence matrix entries and negative-result notes exist for each thesis-relevant ablation; and every thesis figure/table can be regenerated from saved records and configurations.

A minimum viable Layer 4 includes calibrated confidence and uncertainty inputs, action vocabulary, accept/wait/request/fallback/uncertain/unsafe decisions, reason codes, decision records, and comparison to confidence thresholding. A strong Layer 4 adds window stability, channel-subset stability, model-family agreement, evidence combination, fallback router, unsafe-state logic, RegimeRisk-aware thresholds, matched ablation ladder, metric dictionary, negative-result reporting, and dashboard cards. The Stretch C version adds policy-ready evidence-quality features, reward/action-cost fields for Layer 7, simulation-ready request/wait/fallback behavior, embodiment-gating compatibility, IHARQ reliability-card exports and baseline-controller-card handoff fields for Layer 6/10, stress-conditioned IHARQ analysis, and complete Ablation and Evaluation Architecture-compatible evaluation lineage.

For thesis-level claims, Layer 4 should reach the equivalent of L4-MRL-9: bottleneck framing, action vocabulary, input-output contracts, rule logic, reason codes, implemented evidence modules, matched baselines, ablations, simulation readiness, dashboard cards, limitations, negative-result notes, metric dictionaries, claim-evidence entries, and claim-safe writing must all be present.

11 Layer 5: RegimeRisk Temporal Trust Monitoring

11.1 Problem solved by Layer 5

A BCI decoder may work under one condition and fail under another. The condition may change because of fatigue-like degradation, attention change, electrode drift, noise, dropout, subject instability, calibration mismatch, stress injection, or simulated outcome feedback. Static thresholds do not handle this well because they treat the current confidence or uncertainty value as if its meaning were stable across the whole session. RegimeRisk exists to estimate whether the current operating state is stable or risky, and whether current decoder evidence should still be trusted under the recent temporal regime.

The bottleneck solved by Layer 5 is therefore not ordinary classification error. Earlier layers already measure prediction, calibration, uncertainty, evidence quality, and IHARQ readiness for a local decision. Layer 5 addresses the temporal trust bottleneck: a low-calibration neural-intent system can move between trustworthy and untrustworthy operating regimes over time, while confidence-only, entropy-only, selective-prediction, and IHARQ-only rules may fail to detect sustained degradation, unstable agreement, dropout risk, fatigue-like decline, drift-risk, repeated wrong accepts, or changing model-family trust.

Layer 5 should be interpreted as the temporal regime and trust engine of the architecture. Its thesis-critical question is:

When a decoder appears confident, should the architecture still trust that confidence under the current temporal operating regime?

Without Layer 5, IHARQ BenchGuard would remain mostly a static reliability pipeline: Layer 2 would benchmark decoders, Layer 3 would calibrate and quantify uncertainty, and Layer 4 would verify immediate evidence. The system would still lack a principled way to say that the same decoder, threshold, evidence source, or readiness action has become less trustworthy after recent instability. With Layer 5, the architecture becomes temporally aware. It can explain when risk is rising, when trust is decaying, when fallback should be preferred, when IHARQ should become stricter, and when simulated embodiment should be held or blocked.

11.2 Layer 5 thesis role and contribution boundary

Layer 5 supports the central thesis argument that low-calibration BCI reliability is not only a pointwise classification problem. It is also an operating-regime problem. A decoder may be acceptable during one segment of a session and unreliable later. A confidence threshold may work during stable periods and fail during drift-risk, fatigue-like, noisy, or dropout-risk periods. A model family may be trustworthy for one subject/session state and less trustworthy under another. Therefore, the architecture needs a temporal trust layer that converts recent behavior into regime-aware caution.

The scientific contribution of Layer 5 is to operationalize temporal trust monitoring for a public-data and simulated-readiness thesis setting. It does not claim clinical fatigue diagnosis, real-time medical safety validation, or true neurological state estimation. Instead, it produces traceable, reproducible, and evaluable records that test whether temporal regime context improves reliability interpretation, failure explanation, IHARQ behavior, adaptive-policy context, simulation outcomes, and dashboard evidence.

Layer 5 directly supports four thesis-level claims:

Claim type	Layer 5 evidence contribution
Temporal unreliability claim	Shows that reliability, uncertainty, evidence quality, agreement, or wrong-accept risk can change across windows, trials, blocks, sessions, subjects, stress profiles, or simulated transitions.
Static-threshold limitation claim	Shows cases where confidence-only, entropy-only, margin-only, selective-prediction, or IHARQ-only rules are insufficient because the temporal context changes the meaning of local evidence.
Regime-aware decision claim	Tests whether regime labels, risk scores, trust scores, and stop-loss states improve or explain readiness decisions when compared with regime-unaware baselines.
Interpretability claim	Produces regime timelines, risk-trend plots, trust-allocation summaries, stop-loss logs, reason codes, and regime cards that make temporal failures inspectable.

Layer 5 protects the thesis against the criticism that IHARQ BenchGuard is only a classifier benchmark or a collection of thresholds. It also protects Layer 6 from the criticism that the adaptive policy receives no temporal context, and it protects Layer 10 from the criticism that the dashboard is cosmetic rather than record-driven. However, Layer 5 must remain claim-safe: its language should use terms such as *fatigue-like degradation*, *drift-risk*, *dropout-risk*, *temporal trust reduction*, and *unsafe simulated readiness state*, not clinical diagnosis or validated patient-safety claims.

11.3 Engineering responsibility, scope, and boundaries

Layer 5 has one primary engineering responsibility: read recent decoder, uncertainty, calibration, evidence, decision, stress, transition, and simulated outcome records, and write a structured **RegimeStateRecord** that represents the current temporal operating regime, risk level, trust allocation, stop-loss status, recommendation, and reason codes. When trust allocation has an independent lifecycle, per-target timeline, or dashboard view, Layer 5 must also write a companion **TrustStateRecord** linked back to the source **RegimeStateRecord**.

Layer 5 is in scope for:

- computing rolling temporal features from prediction, confidence, uncertainty, calibration, stability, evidence, stress, action, and outcome histories;
- estimating regime labels such as stable, noisy, drift-risk, fatigue-risk, dropout-risk, uncertain, and unsafe;
- computing regime-risk scores, risk trends, volatility-like features, drawdown-like features, failure-risk scores, and trust scores;
- allocating trust across decoders, model families, windows, channels, evidence sources, and fallback routes;
- recommending stricter evidence, trusted-set restriction, fallback, no-action, unsafe mode, cooldown, or recovery monitoring;

- feeding regime context into Layer 4 IHARQ, Layer 6 policies, Layer 7 closed-loop simulation, Layer 9 embodiment gating, and Layer 10 dashboard/card generation;
- generating reproducible regime timelines, trust-state tables, risk plots, stop-loss tables, ablation tables, and regime cards.

Layer 5 is out of scope for:

- training the baseline decoder itself;
- replacing Layer 3 calibration or uncertainty estimation;
- replacing Layer 4 IHARQ evidence verification;
- making all final readiness decisions by itself;
- claiming true clinical fatigue detection from public EEG alone;
- claiming validated medical safety or real rehabilitation safety;
- controlling real hardware;
- using future labels or future simulated outcomes to make online decisions;
- silently modifying experimental records through dashboard interaction.

The boundary rule is that Layer 5 supplies temporal context and conservative recommendations. It may constrain or adapt later decisions through declared feedback contracts, but final action selection remains the responsibility of Layer 4, Layer 6, Layer 7, or Layer 9 depending on the experiment mode.

11.4 Layer 5 identity and design principles

Field	Layer 5 specialization
Formal layer name	RegimeRisk Temporal Trust Monitoring.
Functional module name	Module E: Temporal regime and trust engine.
Short name	RegimeRisk.
Primary architectural plane	Regime plane.
Secondary planes	Evidence plane, policy plane, simulation plane, embodiment plane, dashboard plane.
Main role	Estimate temporal operating-state trust and risk from recent system behavior.
Core output record	RegimeStateRecord .
Companion trust record	TrustStateRecord when trust allocation is exported as a separate time-indexed artifact.
Main consumers	Layers 4, 6, 7, 9, and 10.
Implementation phase	After IHARQ-lite evidence verification and before adaptive policy learning.
Main risk if weak	The architecture becomes static-threshold reliability with no temporal trust awareness.

Layer 5 follows eight design principles:

1. Temporal trust is different from pointwise confidence. A pointwise confidence value describes one prediction; temporal trust describes whether recent system behavior makes the current evidence believable.
2. Regime labels must be operationally defined. Stable, noisy, drift-risk, fatigue-risk, dropout-risk, uncertain, and unsafe should have explicit feature patterns, reason codes, and downstream effects.
3. RegimeRisk should be decoder-agnostic. It should work with any model family that emits compatible `PredictionRecord`, `CalibrationRecord`, `UncertaintyRecord`, and decision/evidence records.
4. The layer should produce records, not hidden state. Every regime estimate should be written to `RegimeStateRecord`; every separately consumed trust timeline should be written to `TrustStateRecord`; hidden variables must not drive results without being logged.
5. Feedback must be controlled and explainable. Threshold changes, fallback recommendations, trusted-set restrictions, and unsafe recommendations require source record IDs and reason codes.
6. Stop-loss logic must be conservative. It should prevent repeated wrong acceptance or unsafe simulated behavior, not optimize post-hoc test performance.
7. Stress and embodiment signals must be marked as simulated or proxy-derived. They should never be mixed with public EEG evidence as if they were clinical observations.
8. Evaluation must include negative controls. A regime-risk module should be compared with confidence-only, uncertainty-only, IHARQ-only, random-risk, and shuffled-regime baselines.

11.5 Conceptual model of temporal trust

Layer 5 should not collapse all temporal information into one opaque risk number. It should preserve a structured view with five conceptual objects:

Concept	Meaning
Operating regime	The current temporal condition of the system, such as stable, noisy, drift-risk, fatigue-risk, dropout-risk, uncertain, or unsafe.
Temporal trust	The degree to which current evidence sources remain believable given recent confidence, uncertainty, stability, agreement, action, stress, and outcome histories.
RegimeRisk score	A scalar or vector summary of temporal instability, degradation, and wrong-action risk.
Trust allocation	Distribution of trust across decoder families, evidence sources, windows, channel subsets, and fallback routes, summarized inside <code>RegimeStateRecord</code> and exported as <code>TrustStateRecord</code> when the trust timeline is consumed independently.
Stop-loss warning	A conservative recommendation triggered by sustained degradation, repeated wrong accepts, agreement collapse, severe stress, or unsafe simulated outcomes.

The canonical output structure is:

RegimeStateRecord = regime label + regime probability + risk score + risk trend + volatility-like features + drawdown-like features + drift/fatigue/dropout risk fields + failure-risk score + trust-score summary + trusted set + optional **TrustStateRecord** links + stop-loss status + downstream recommendation + reason codes + provenance.

11.6 Input contract

Layer 5 consumes inputs from earlier reliability layers and optional Stretch C modules. The input contract must distinguish primary inputs, auxiliary inputs, feedback inputs, proxy inputs, simulation-only inputs, and evaluation-only inputs.

11.6.1 Primary and strongly recommended inputs

Input	Source	Status	Purpose
WindowRecord	Layer 1	Required	Provides time/window identity, trial identity, labels where allowed, channel masks, and artifact flags.
PredictionRecord	Layer 2	Required	Provides predicted class, probability vector or score proxy, model ID, dataset/subject/session/trial/window IDs, seed, and configuration hash.
CalibrationRecord	Layer 3	Required when available	Provides calibrated probabilities, ECE/Brier/NLL summaries, calibration method, calibration split, and calibration-error trends.
UncertaintyRecord	Layer 3	Required	Provides confidence, entropy, margin, variation ratio, model disagreement, conformal-set size if used, and selective flags.
SelectivePredictionRecord	Layer 3	Auxiliary	Provides coverage-risk summaries, retained accuracy, rejection rate, high-confidence wrong rate, and threshold behavior for diagnostic comparison.
EvidenceQualityRecord	Layer 4 or Layer 6	Strongly recommended	Provides evidence-quality score, signal-quality features, stability features, and predicted failure probability.

IHARQDecisionRecord Layer 4	Strongly recommended	Provides the canonical Layer 4 action token (<code>accept</code> , <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , <code>fallback</code> , <code>uncertain</code> , or <code>unsafe</code>), reason code, evidence count, fallback status, unsafe status, and action cost.
-----------------------------	----------------------	--

11.6.2 Temporal feature inputs

Input family	Examples	Use in Layer 5
Recent prediction correctness	Rolling accuracy, wrong-accept rate, high-confidence wrong rate, error bursts where labels or outcomes are legally available.	Estimates temporal reliability and target labels for offline failure-risk evaluation.
Confidence trend	Moving average, slope, variance, sudden drop, distance from recent stable range.	Detects confidence instability and confidence drawdown.
Uncertainty trend	Entropy drift, margin decay, disagreement bursts, variation-ratio instability.	Detects rising uncertainty and unstable evidence separation.
Calibration-error trend	Rolling ECE proxy, Brier trend, NLL trend, calibration-bin instability.	Detects calibration mismatch and drift-risk proxies.
Stability trend	Window agreement, channel-subset agreement, signal-quality flags, channel masks.	Detects evidence instability and dropout-risk.
Model-family trend	Ensemble dispersion, model disagreement, trust-rank changes, model-family switching.	Supports trust allocation and model-family shift detection.
IHARQ action history	Counts over canonical Layer 4 tokens (<code>accept</code> , <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , <code>fallback</code> , <code>uncertain</code> , and <code>unsafe</code>), evidence-request budget, reason-code runs.	Detects decision volatility and repeated conservative or unsafe states.

11.6.3 Optional Stretch C and feedback inputs

Input	Source	Status	Use
-------	--------	--------	-----

StressProfileRecord	Layer 8	Optional / Stretch C	Conditions regime tracking under delay, dropout, drift, noisy intent, weakness, fatigue-like degradation, and combined stress.
TransitionRecord	Layer 7	Optional / Stretch C	Allows regime updating across simulated closed-loop steps and enables feedback from simulated outcomes.
EmbodimentStateRecord and EmbodimentOutcomeRecord	Layer 9	Optional / diagnostic Stretch C	Allows saved endpoint-error, unsafe-event, unstable-command, delay, or over/under-assist proxies to be inspected in simulation-only regime analysis; it must not create an undeclared Layer 9-to-Layer 5 runtime update loop.
Policy/action history	Layer 6 or 7	Optional	Allows Layer 5 to distinguish decoder instability from policy-induced instability in simulated sessions.
Subject difficulty profile	Layer 2	Auxiliary	Provides subject/session prior risk context, but must not override observed temporal evidence.

11.6.4 Input classification and validation

Every Layer 5 input should be tagged as one of the following: primary input, auxiliary input, feedback input, evaluation-only input, proxy input, or simulation-only input. Ground-truth labels, future-window evidence, final outcomes, and future simulated events are evaluation-only unless the experiment explicitly models an online setting in which the information would already be known from the past.

Layer 5 should not run in ordinary mode until the following checks pass:

- dataset, subject, session, trial, window, split, label-budget, model, calibration, and threshold IDs are aligned;
- timestamps or simulated time indices are ordered;
- confidence, entropy, margin, probabilities, evidence scores, and risk fields are finite;
- rolling windows do not cross forbidden split boundaries or leak future information;
- test-label information is not used for online regime estimation and is only used for scoring after decisions are made;
- stress and embodiment signals are tagged as simulation-derived;
- missing inputs trigger declared degraded-mode behavior rather than silent failure.

11.7 Output contract

Layer 5 outputs should be first-class research artifacts, not temporary helper variables.

Output	Type	Consumer	Purpose
RegimeStateRecord	Structured record	Layers 4, 6, 7, 9, 10	Main regime and temporal trust output.
TrustStateRecord	Structured companion record	Layers 6, 10; optional Layers 4, 7, and 9 through linked RegimeStateRecord references	Time-indexed trust allocation output when trust scores have their own lifecycle, target identity, trend, or dashboard view.
Regime timeline	Table/figure	Layer 10, thesis	Shows regime changes over windows, trials, sessions, and stress conditions.
Trust-score table	Table/record derived from TrustStateRecord	Layers 4, 6, 10	Shows decoder, model-family, evidence-source, channel/window, and fallback trust.
Risk-trend plot	Figure	Thesis, dashboard	Shows rising, stable, falling, or recovering risk across a session.
Stop-loss warning log	Table/record	Layers 4, 6, 7, 9, 10	Explains when stricter evidence, fallback, no-action, unsafe, or cooldown was recommended.
Stricter-evidence recommendation	Decision recommendation	Layer 4	Tightens IHARQ behavior under risky regimes.
Fallback/no-action/unsafe recommendation	Decision recommendation	Layers 4, 6, 7, 9	Routes downstream behavior toward conservative simulated actions.
Regime card	Markdown/PDF	Layer 10, thesis appendix	Summarizes regime behavior, limitations, traceability, and claim boundaries.
Layer 5 evaluation summary	Structured record/table	Evaluation scripts, Layer 10, thesis	Stores global ablation ID, matched-comparison key, metric dictionary ID, regime-aware versus regime-unaware deltas, negative-result note, and claim-evidence pointer.

Every output must include dataset ID, subject ID, session ID if available, trial/window ID, time index, model ID, split ID, configuration hash, temporal window sizes, regime estimation method, random seed if learned, source record IDs, generation timestamp or simulated time

index, global/local ablation ID when evaluated, matched-comparison key, metric dictionary ID, claim boundary, and diagnostic-only flag when matching fails.

11.8 Canonical RegimeStateRecord

The `RegimeStateRecord` is the main formal contract of Layer 5.

Field	Type	Meaning
<code>record_id</code>	string	Unique identifier for the regime record.
<code>dataset_id</code> , <code>subject_id</code> , <code>session_id</code>	string	Dataset, subject, and session identity.
<code>trial_id</code> , <code>window_id</code> , <code>time_index</code>	string/int/float	Temporal location of the record.
<code>model_id</code>	string	Decoder or model-family identifier.
<code>regime_label</code>	categorical	Stable, noisy, drift-risk, fatigue-risk, dropout-risk, uncertain, or unsafe.
<code>regime_probability</code>	float/vector	Confidence or calibrated probability assigned to the regime label.
<code>regime_risk_score</code>	float	Overall temporal trust risk score.
<code>risk_trend</code>	categorical/float	Rising, falling, stable, recovering, or slope value.
<code>volatility_score</code>	float	Fluctuation score across recent confidence, uncertainty, agreement, action, or outcome signals.
<code>drawdown_score</code>	float	Sustained degradation from recent best or recent stable reliability state.
<code>drift_risk_score</code>	float/optional	Proxy score or flag for calibration, distribution, agreement, or model-family shift away from the recent stable regime.
<code>fatigue_like_risk_score</code>	float/optional	Proxy score or flag for gradual degradation patterns; must be described as fatigue-like and not as a clinical fatigue diagnosis.
<code>dropout_risk_score</code>	float/optional	Proxy score or flag for missing, unstable, or unreliable channels, windows, or evidence sources.
<code>failure_risk_score</code>	float	Estimated probability or proxy risk of wrong accepted decision or unsafe simulated action.
<code>trust_scores</code>	object	Trust values for model families, decoders, windows, channels, evidence sources, and fallback routes.
<code>trust_state_record_ids</code>	list/optional	Links to companion <code>TrustStateRecord</code> rows when trust allocation is exported separately.
<code>trusted_set</code>	list	Sources currently considered usable for evidence combination or routing.
<code>stop_loss_status</code>	categorical	None, warning, active, fallback recommended, no-action recommended, or unsafe recommended.

<code>recommendation</code>	categorical	Normal, stricter evidence, trusted-set restriction, fallback, no-action, unsafe, cooldown, or recovery monitoring.
<code>reason_codes</code>	list	Human-readable and machine-readable explanation labels.
<code>provenance</code>	object	Source records, configuration hashes, split tags, simulation tags, and temporal window settings.

Optional fields include `stress_profile_id`, `embodiment_state_id`, `transition_id`, `policy_action_id`, `calibration_drift_score`, `channel_dropout_score`, `model_family_shift_score`, `drift_risk_flag`, `fatigue_risk_flag`, `dropout_risk_flag`, `recovery_status`, and `evaluation_label`. These fields must be explicitly marked as optional, proxy-derived, simulation-derived, or evaluation-only where applicable.

A schema skeleton should be stored under `contracts/regime_state_record_schema.yaml`; companion trust-allocation schemas should be stored under `contracts/trust_state_record_schema.yaml` and, where a narrower score component schema is useful, `contracts/trust_score_schema.yaml`. An implementation example may also be exported as JSON for unit tests and documentation.

```
{
  "record_id": "regime_000123",
  "dataset_id": "dataset_A",
  "subject_id": "S04",
  "session_id": "session_1",
  "trial_id": "trial_078",
  "window_id": "win_003",
  "time_index": 348,
  "model_id": "eegnet_seed42",
  "regime_label": "drift-risk",
  "regime_probability": 0.74,
  "regime_risk_score": 0.68,
  "risk_trend": "rising",
  "volatility_score": 0.61,
  "drawdown_score": 0.43,
  "drift_risk_score": 0.66,
  "fatigue_like_risk_score": 0.21,
  "dropout_risk_score": 0.18,
  "failure_risk_score": 0.37,
  "trust_scores": {"eegnet": 0.52, "riemannian": 0.71, "csp_lda": 0.49},
  "trust_state_record_ids": ["trust_000123_eegnet", "trust_000123_riemannian"],
  "trusted_set": ["riemannian"],
  "stop_loss_status": "warning",
  "recommendation": "stricter_evidence",
  "reason_codes": ["confidence_volatility_high", "model_disagreement_rising"],
  "provenance": {"config_hash": "abc123", "rolling_window": 20}
}
```

11.9 Canonical TrustStateRecord

The `TrustStateRecord` is the companion contract for trust allocation when trust values need to be consumed, plotted, or evaluated independently of the one-row trust summary inside `RegimeStateRecord`. It must not replace `RegimeStateRecord`. Instead, each `TrustStateRecord` row should link to the source regime record and identify the exact trust target whose score changed.

Field	Type	Meaning
<code>record_id</code>	string	Unique identifier for the trust-state row.
<code>regime_state_record_id</code>	string	Source <code>RegimeStateRecord</code> that produced or justified this trust estimate.
<code>dataset_id</code> , <code>subject_id</code> , <code>session_id</code>	string	Dataset, subject, and session identity.
<code>trial_id</code> , <code>window_id</code> , <code>time_index</code>	string/int/float	Temporal location of the trust update.
<code>trust_target_type</code>	categorical	Decoder, model family, evidence source, channel subset, window group, or fallback route.
<code>trust_target_id</code>	string	Identifier of the trusted or degraded target.
<code>trust_score</code>	float	Current trust value for the target under the active regime.
<code>trust_trend</code>	categorical/float	Rising, falling, stable, recovering, or slope value.
<code>trust_band</code>	categorical	Low, moderate, high, or unusable trust band, using configured thresholds.
<code>trusted_set_membership</code>	boolean/categorical	Whether this target is inside the current trusted set and why.
<code>update_source</code>	categorical	Label-based, label-free proxy, stress-derived, simulation-derived, or evaluation-only source.
<code>decay_recovery_state</code>	object/string	Decay, cooldown, or recovery status affecting the target trust score.
<code>reason_codes</code>	list	Machine-readable and human-readable trust-update explanations.
<code>provenance</code>	object	Source records, configuration hashes, schema version, split tags, simulation tags, and temporal window settings.

A `TrustStateRecord` is required when a trust timeline is consumed by Layer 6 policy context, Layer 10 dashboard views, or a thesis figure/table as a separate artifact. If trust is used only as a compact field in the current regime state, the `trust_scores` and `trusted_set` fields inside `RegimeStateRecord` are sufficient.

11.10 Dependency logic

Layer 5 has hard, soft, optional, conceptual, data, model, metric, and software dependencies.

Dependency type	Dependency	Reason and mockability
Hard data dependency	WindowRecord , PredictionRecord , split manifests, model IDs, time indices.	Required for any regime timeline. Can be mocked with synthetic prediction logs for early tests.
Hard metric dependency	Confidence or score proxy, uncertainty features, model IDs, and ordered windows.	Required for minimum temporal features. If probabilities are missing, score-proxy mode must be declared.
Strong soft dependency	CalibrationRecord , UncertaintyRecord , EvidenceQualityRecord , IHARQDecisionRecord .	Needed for thesis-ready RegimeRisk . Can be mocked by generated uncertainty/evidence tables during development.
Optional Stretch C dependency	StressProfileRecord , TransitionRecord , and diagnostic EmbodimentStateRecord /EmbodimentStateRecord only mode and views where Layer 9 has already saved them.	Needed for stress-conditioned, closed-loop, and diagnostic embodiment-aware analysis. Can be mocked in thesis-ready mode and must not create an undeclared runtime feedback loop from Layer 9 to Layer 5.
Conceptual dependency	Layer 3 uncertainty and Layer 4 IHARQ are separate from temporal trust.	Prevents Layer 5 from reimplementing calibration or local evidence verification.
Software dependency	Pandas/Polars tables, NumPy/SciPy, scikit-learn or similar shallow models, YAML configs, plotting libraries.	Required for record processing, simple regime classifiers, thresholds, and thesis figures; no complex sequence model is required for the core version.

The minimum upstream artifacts that must exist before Layer 5 can run are a split manifest, ordered **WindowRecord** table, ordered **PredictionRecord** table, model registry entries, uncertainty-compatible score fields, and a Layer 5 configuration file. A strong implementation additionally requires calibrated uncertainty, evidence-quality, IHARQ decision, subject profile, trust-state export, and stress/simulation records where applicable.

11.11 Submodule structure and responsibilities

Layer 5 preserves all original submodules and strengthens them into an implementation-aware internal structure. The core path is temporal feature builder → volatility/drawdown monitors → regime classifier → failure-risk predictor → trust allocator → stop-loss module → feedback adapters. Rule-based variants are core and thesis-ready; learned variants are supporting or stretch depending on available data.

Submodule	Status	Engineering responsibility	Primary output
-----------	--------	----------------------------	----------------

Temporal feature builder	Core	Build rolling features over recent windows, trials, actions, and outcomes with leakage control.	Temporal feature table.
Volatility-like monitor	Core	Measure fluctuation in confidence, uncertainty, agreement, correctness proxies, and decisions.	Volatility score and components.
Drawdown-like monitor	Core	Track sustained degradation from recent best or recent stable reliability state.	Drawdown score, duration, recovery indicator.
Regime classifier	Core	Label operating state as stable, noisy, drift-risk, fatigue-risk, dropout-risk, uncertain, or unsafe.	Regime label, probability, reason codes.
Failure-risk predictor	Supporting/contingent when labels exist	Estimate risk of wrong accepted decision or unsafe simulated action.	Failure-risk score and risk bin.
Trust allocator	Core for Stretch C	Estimate trust for decoders, model families, evidence sources, windows, channels, and fallback routes.	TrustStateRecord , trust scores, and trusted set.
Stop-loss module	Core for reliability control	Recommend stricter evidence, trusted-set restriction, fallback, no-action, cooldown, or unsafe mode.	Stop-loss warning and recommendation.
Feedback adapter	Core integration	Convert RegimeStateRecord into Layer 4, 6, 7, 9, and 10-compatible signals.	Auditable feedback records.
Sequence-model regime estimator	Optional/future work	Learn temporal regimes with richer sequence models if data suffices.	Alternative RegimeStateRecord variants for ablation only.

11.11.1 Submodule 5.1: Temporal feature builder

Purpose. The temporal feature builder converts raw recent records into features that describe how the system has behaved over time.

Bottleneck addressed. Local confidence and uncertainty do not expose whether the current prediction occurs after recent instability, repeated wrong decisions, rising uncertainty, evidence collapse, stress injection, or simulated outcome degradation.

Scientific role. Provides the measurable basis for the claim that reliability is temporally structured rather than purely pointwise.

Engineering responsibility. Build rolling short-, medium-, long-, and session-level features

from ordered records. Recommended default windows are short windows of 3–5 windows/trials, medium windows of 10–20 windows/trials, and long windows of 30–100 windows/trials or declared session segments. Exact values must be stored in configuration files, not hard-coded.

Inputs. WindowRecord, PredictionRecord, CalibrationRecord, UncertaintyRecord, EvidenceQualityRecord, IHARQDecisionRecord, subject profiles, and optional StressProfileRecord, TransitionRecord, and diagnostic saved EmbodimentStateRecord/EmbodimentOutcomeRecord views where available.

Outputs. Temporal feature table with confidence history, uncertainty history, correctness history where legal, calibration history, stability history, model-family history, IHARQ action history, stress history, and diagnostic embodiment-history features where saved Layer 9 records are explicitly attached.

Internal processing logic. Sort records by dataset, subject, session, trial, window, split, model, and simulated time. Validate IDs. Compute rolling means, slopes, variances, deltas from recent stable ranges, event counts, burst indicators, missing-data flags, action counts, disagreement rates, and recovery indicators. Mark every feature as label-free, label-based, proxy-based, simulation-derived, or evaluation-only.

Dependencies and consumers. Depends on Layers 1–4 and optionally Layers 7–9. Consumed by all other Layer 5 submodules and exported to Layer 10 for diagnostic inspection.

Feedforward and loopback. Feeds volatility, drawdown, regime classification, failure-risk prediction, trust allocation, and stop-loss. It does not send direct feedback to earlier layers; feedback is emitted only after downstream risk interpretation.

Evaluation method. Unit-test feature windows, boundary conditions, leakage prevention, missing-input behavior, split isolation, and deterministic recomputation. Compare feature distributions under clean and stressed conditions.

Failure modes. Misaligned windows, split leakage, use of future labels, over-smoothed trends, under-smoothed noise, feature explosion, and silent missingness. Defensive design requires provenance fields and degraded-mode logs.

Reproducibility requirements. Save configs/regime_risk/rolling_windows.yaml, temporal feature tables, feature dictionaries, split-lockout checks, and configuration hashes.

Thesis evidence produced. Temporal feature summary tables, feature-drift plots, missingness reports, and pre-regime evidence for temporal unreliability.

Limitations and risks. Rolling features are proxies; they do not prove physiological state changes. Short public sessions may limit long-horizon analysis.

11.11.2 Submodule 5.2: Volatility-like monitor

Purpose. The volatility-like monitor asks whether the signals that support readiness decisions are becoming unstable.

Bottleneck addressed. A single confidence value can look acceptable even while the recent confidence, entropy, margin, agreement, action selection, or correctness proxy is rapidly fluctuating.

Scientific role. Supports the claim that instability itself is a reliability signal and should be distinguished from average uncertainty.

Engineering responsibility. Compute component-level and aggregate volatility scores from

rolling temporal features.

Inputs. Temporal feature table, confidence history, entropy history, margin history, agreement history, correctness-proxy history, model disagreement history, and IHARQ action history.

Outputs. `volatility_score`, `volatility_components`, `volatility_trend`, and volatility reason codes such as `confidence_jump`, `entropy_spike`, `agreement_collapse`, or `decision_switching_high`.

Internal processing logic. Compute robust rolling variance, absolute differences, normalized deviation from recent stable range, burst counts, and change-point proxies. Aggregate with declared weights or a shallow calibrated model. Keep component scores available for ablation.

Dependencies and consumers. Depends on temporal features; consumed by the regime classifier, failure-risk predictor, trust allocator, stop-loss module, Layer 4 feedback adapter, Layer 6 context adapter, and Layer 10 dashboard.

Feedforward and loopback. High volatility may trigger stricter Layer 4 evidence requirements or Layer 6 caution, but volatility alone should not automatically declare unsafe.

Evaluation method. Check whether high-volatility periods align with uncertainty spikes, error bursts, model disagreement bursts, stress injections, or simulated unsafe events. Include a no-volatility-feature ablation.

Failure modes. Natural task variation may be misclassified as risk; volatility may overreact to small samples. Defensive design requires minimum-window conditions and confidence intervals or reliability flags.

Reproducibility requirements. Save volatility formulas, weights, thresholds, component columns, reason-code mapping, and risk-band configuration.

Thesis evidence produced. Volatility timelines, component contribution plots, volatility/error alignment tables, and volatility ablation results.

Limitations and risks. Volatility is a caution signal, not a proof of failure or unsafe action.

11.11.3 Submodule 5.3: Drawdown-like monitor

Purpose. The drawdown-like monitor asks whether the system has moved downward from a previously reliable state and whether that degradation is sustained enough to require intervention.

Bottleneck addressed. A decoder can degrade gradually without one dramatic spike. Static thresholds and pointwise IHARQ may miss sustained decline.

Scientific role. Supports the temporal degradation claim and separates transient fluctuation from persistent reliability decline.

Engineering responsibility. Track sustained degradation in confidence quality, correctness proxies, evidence quality, agreement, calibration behavior, and simulated embodiment outcomes.

Inputs. Temporal feature table, recent best or recent stable baseline, confidence trend, accuracy/proxy trend, evidence-quality trend, agreement trend, calibration trend, and optional embodiment outcome trend.

Outputs. `drawdown_score`, `drawdown_duration`, `recovery_indicator`, and drawdown reason codes such as `sustained_confidence_decline`, `evidence_quality_drawdown`, `repeated_wrong_accepts`, or `endpoint_error_drawdown`.

Internal processing logic. Define a recent stable reference using training/validation protocol or recent past-only statistics. Measure decline magnitude and duration. Estimate recovery only after sustained improvement over a declared cooldown window. Keep label-based and label-free drawdown variants separate.

Dependencies and consumers. Depends on temporal features and optional outcome records. Consumed by regime classifier, failure-risk predictor, trust allocator, stop-loss module, and dashboard.

Feedforward and loopback. Sustained drawdown can trigger stop-loss warnings, fallback recommendations, no-action routing, or stricter IHARQ thresholds. It should be stronger than volatility in conservative control logic.

Evaluation method. Evaluate alignment with error bursts, wrong accepts, stress-induced drift, fatigue-like degradation profiles, and simulated outcome decline. Include a no-drawdown ablation.

Failure modes. A weak initial state may make drawdown ambiguous; recovery criteria may be too strict or too permissive; session length may be too short. Defensive design requires documented reference-state rules.

Reproducibility requirements. Save reference-state method, window lengths, thresholds, cooldown rules, and drawdown/recovery logs.

Thesis evidence produced. Drawdown timelines, degradation/recovery case studies, stop-loss trigger explanations, and no-drawdown ablation results.

Limitations and risks. Drawdown is a temporal reliability proxy and should not be presented as confirmed neurological fatigue.

11.11.4 Submodule 5.4: Regime classifier

Purpose. The regime classifier converts temporal features, volatility, drawdown, trust signals, and optional stress/simulation features into an operating-state label.

Bottleneck addressed. Downstream layers need a compact and explainable representation of the current operating regime rather than a large unstructured feature table.

Scientific role. Provides the categorical representation needed to test whether regime-aware reliability control adds value beyond pointwise confidence and IHARQ-only baselines.

Engineering responsibility. Produce `regime_label`, `regime_probability`, reason codes, and recommended downstream behavior.

Regime label	Operational meaning	Typical downstream effect
stable	Evidence is temporally consistent, uncertainty is acceptable, agreement is acceptable, and no stop-loss warning is active.	Normal IHARQ and policy behavior.
noisy	Evidence is fluctuating but not clearly degraded.	Request more evidence, combine windows, or use slightly stricter thresholds.

drift-risk	Calibration, distribution, model-family trust, or agreement appears to be shifting away from previous stable behavior.	Tighten acceptance and prefer trusted sources.
fatigue-risk	Performance proxy or evidence quality degrades gradually across recent temporal windows.	Reduce repeated accepts and increase caution, fallback, or recovery monitoring.
dropout-risk	Channels, windows, or evidence sources become missing, unstable, or unreliable.	Avoid affected sources and use fallback/no-action if severe.
uncertain	Regime cannot be classified confidently because evidence is insufficient or conflicting.	Avoid aggressive acceptance and prefer request evidence or fallback.
unsafe	Current regime is too risky for simulated action or critical stop-loss is active.	Block simulated action, declare unsafe, or enter safe hold.

Inputs. Temporal features, volatility score, drawdown score, calibration drift proxy, channel/dropout flags, model disagreement, trust scores, stop-loss state, and optional stress/simulation signals.

Outputs. Regime label, regime probability or confidence, risk band, recommended downstream action, and reason codes.

Internal processing logic. The first thesis-ready version should prioritize an interpretable rule-based classifier or calibrated shallow model. Level 1 is a rule-based classifier; Level 2 is a calibrated logistic regression, random forest, gradient boosting model, or small MLP; Level 3 sequence modeling is optional; Level 4 simulation-aware regime estimation is a Stretch C branch. Every learned version must preserve reason codes and ablation compatibility.

Dependencies and consumers. Depends on temporal features, volatility, drawdown, and optional trust/stop-loss inputs. Consumed by Layers 4, 6, 7, 9, and 10.

Feedforward and loopback. Feeds Layer 4 threshold adaptation, Layer 6 policy context, Layer 7 environment state, Layer 9 gating, and Layer 10 timeline views. It does not update earlier raw predictions or calibration outputs.

Evaluation method. Evaluate risk-error alignment, stress-regime alignment, regime stability, regime transition plausibility, and ablations against confidence-only, uncertainty-only, IHARQ-only, random-risk, and shuffled-regime baselines.

Failure modes. Regime labels can become decorative if not validated; learned classifiers can overfit; rule thresholds can be arbitrary. Defensive design requires fixed configuration, validation-only threshold tuning, and negative controls.

Reproducibility requirements. Save regime vocabulary, classifier type, thresholds, training split, validation split, seeds, model artifact if learned, reason-code mapping, and schema version.

Thesis evidence produced. Regime timelines, transition matrices, regime/action tables, regime/error alignment tables, and regime-aware versus regime-unaware comparisons.

Limitations and risks. Regime labels are operational categories for reliability control, not clinical states.

11.11.5 Submodule 5.5: Failure-risk predictor

Purpose. The failure-risk predictor estimates the probability or score of a wrong accepted decision or unsafe simulated action under the current temporal regime.

Bottleneck addressed. Some failures are most important not because the decoder is uncertain, but because the system accepts a wrong or unsafe action while appearing confident.

Scientific role. Supports the thesis claim that temporal regime context helps identify high-risk accepted decisions beyond confidence and uncertainty alone.

Engineering responsibility. Produce `failure_risk_score`, `risk_bin`, calibration-of-risk diagnostics, and reason codes.

Target type	Description	Use
Wrong accepted decision	A prediction was accepted and later known to be wrong.	Offline evaluation and supervised risk learning.
High-confidence wrong case	The model was confident but incorrect.	Detects overconfident failure regimes.
IHARQ failure proxy	IHARQ accepted when evidence later appears unreliable.	Evaluates IHARQ-regime mismatch.
Simulation unsafe event	The simulated environment logged an unsafe event.	Stretch C simulated safety analysis.
Embodiment failure proxy	Endpoint error, over-assist, instability, delay failure, or unsafe proxy.	Embodiment-gated analysis.

Inputs. Regime label, volatility, drawdown, uncertainty history, evidence-quality history, IHARQ action history, trust scores, and legal past outcomes. Evaluation labels may be used after the fact but not for online decision-making.

Outputs. Failure-risk score, risk bin, risk reason codes, and risk calibration diagnostics.

Internal processing logic. Use rule-based scoring or a calibrated shallow classifier first. If supervised targets are available, train only on allowed training/validation splits and score held-out sessions without threshold retuning. Keep label-free proxy risk and label-based risk separate.

Dependencies and consumers. Depends on regime features and outcome labels/proxies where allowed. Consumed by stop-loss, Layer 4, Layer 6, Layer 7, Layer 9, and Layer 10.

Feedforward and loopback. High failure-risk score can recommend stricter evidence or fallback. It cannot use future test labels to change current decisions.

Evaluation method. Evaluate AUROC/AUPRC where appropriate, high-risk precision/recall, risk calibration, risk-coverage curves, wrong-accept reduction, unsafe-event reduction, and comparison to confidence-only and entropy-only risk scores.

Failure modes. Label scarcity, class imbalance, test leakage, overfitting to rare failures, and misleading risk calibration. Defensive design requires split integrity checks, baselines, and calibration plots.

Reproducibility requirements. Save target-construction rules, allowed label sources, model seeds, training/validation/test splits, risk-bin thresholds, and risk-calibration outputs.

Thesis evidence produced. Risk-error alignment plots, high-risk period examples, wrong-accept reduction tables, and risk-coverage curves.

Limitations and risks. Failure-risk scores are estimates or proxies, not guarantees.

11.11.6 Submodule 5.6: Trust allocator

Purpose. The trust allocator asks which sources of evidence the architecture should trust right now.

Bottleneck addressed. A decoder family, channel subset, evidence window, or fallback route can become more or less reliable depending on the temporal regime.

Scientific role. Supports the claim that model/evidence trust is context-dependent and should be tracked instead of treated as fixed after training.

Engineering responsibility. Estimate trust values for decoders, model families, evidence sources, windows, channel subsets, and fallback routes; produce a trusted set and trust timeline.

Trust target	Example interpretation
Decoder trust	EEGNet is currently less trusted than a Riemannian decoder in the current regime.
Model-family trust	Classical models retain stability under one subject/session regime while deep models become unstable.
Evidence-source trust	Channel subset A is more stable than channel subset B.
Window trust	Recent windows are unstable, so longer accumulation or evidence combination is required.
Fallback trust	Conservative fallback or no-action is currently safer than the primary decoder route.

Inputs. Model-family history, decoder performance/proxy history, agreement trend, channel/window stability, calibration and uncertainty trends, failure-risk score, and optional stress/simulation outcomes.

Outputs. `TrustStateRecord` rows, trust-score table, trusted set, fallback trust score, trust timeline, trust reason codes, and `trust_scores` plus optional `trust_state_record_ids` fields inside `RegimeStateRecord`.

Internal processing logic. Compute normalized trust from recent correctness where legal, agreement, evidence stability, uncertainty behavior, calibration trend, failure-risk, and simulation outcomes. Apply decay and recovery rules. Maintain provenance showing whether trust came from labels, proxies, or simulation.

Dependencies and consumers. Depends on temporal features and optional outcomes. Consumed by Layer 4 evidence combination, Layer 6 policy context, Layer 7 state, Layer 9 gating, and Layer 10 trust-allocation view.

Feedforward and loopback. The trusted set can restrict which evidence sources Layer 4 combines. Trust scores can help Layer 6 choose a decoder family or fallback route. Trust changes must be logged as recommendations, not hidden overrides.

Evaluation method. Evaluate whether trust allocation improves matched-coverage wrong-accept rate, reduces unstable source use, and aligns with subject/session/model-family failure taxonomy. Include no-trust-allocator ablation.

Failure modes. Trust may become circular if based on downstream actions; labels may be unavailable; trust may lag recovery. Defensive design requires separate label-free and label-based trust modes, recovery rules, and ablation.

Reproducibility requirements. Save trust update rules, decay/recovery parameters, target IDs, source evidence IDs, `TrustStateRecord` rows, and trust-score outputs.

Thesis evidence produced. Trust-allocation plots, model-family trust shift tables, and trusted-set ablation results.

Limitations and risks. Trust scores summarize operational evidence, not inherent model quality or clinical reliability.

11.11.7 Submodule 5.7: Stop-loss module

Purpose. The stop-loss module asks whether the system has failed enough or degraded enough that it should stop behaving normally and enter a conservative mode.

Bottleneck addressed. Repeated wrong accepts or sustained degradation can be more harmful in a simulated readiness system than isolated mistakes. A reliability architecture needs conservative intervention rules.

Scientific role. Tests whether temporal risk control can reduce repeated wrong acceptance, unsafe simulated actions, or unstable readiness behavior compared with regime-unaware baselines.

Engineering responsibility. Generate stop-loss warnings, recommended actions, cooldown periods, recovery conditions, and unsafe-mode flags.

Trigger	Meaning
Repeated wrong accepts	Too many recent accepted actions were wrong, using only legal past outcomes in online/simulation mode.
High-confidence wrong burst	Several high-confidence wrong predictions occur close together in offline diagnostic or allowed past-outcome mode.
Sustained drawdown	Reliability remains below an acceptable temporal reference state.
Agreement collapse	Models, windows, or channels stop agreeing.
Uncertainty spike	Entropy, disagreement, or margin risk rises sharply.
Stress-critical state	StressForge indicates severe delay, dropout, drift, noisy intent, or fatigue-like degradation.
Embodiment unsafe proxy	Simulated endpoint error, unsafe event, unstable command, or over-assist proxy becomes critical.

Stop-loss actions include stricter evidence, lower evidence-request cap, fallback recommendation, no-action route, trusted-set restriction, unsafe mode, cooldown period, and recovery monitoring.

Inputs. Drawdown score, volatility score, failure-risk score, trust allocator output, IHARQ

action history, wrong-accept history where legal, stress profile, and optional embodiment outcomes.

Outputs. `stop_loss_status`, trigger type, severity, recommended action, cooldown period, recovery condition, reason codes, and affected downstream layer IDs.

Internal processing logic. Use validation-selected thresholds and fixed cooldown/recovery rules. Escalate from none to warning to active to unsafe according to declared risk bands. Release stop-loss only after sustained recovery under a declared rule.

Dependencies and consumers. Depends on volatility, drawdown, failure-risk, trust allocation, and optional outcomes. Consumed by Layer 4, Layer 6, Layer 7, Layer 9, and Layer 10.

Feedforward and loopback. Stop-loss can force Layer 4 into stricter-evidence or fallback mode, constrain Layer 6 action space, update Layer 7 state, block Layer 9 simulated action, and appear in Layer 10 dashboard. Every loopback requires a reason code and original-versus-adjusted decision record.

Evaluation method. Evaluate wrong-accept reduction, unsafe-event reduction, fallback efficiency, evidence cost, over-abstention, cooldown/recovery latency, no-stop-loss ablation, and shuffled-stop-loss negative control.

Failure modes. Over-abstention, post-hoc threshold tuning, endless cooldown, hidden action override, and false unsafe states. Defensive design requires action-cost reporting, fixed thresholds, trace logs, and ablation.

Reproducibility requirements. Save thresholds, triggers, severity mapping, cooldown/recovery rules, warning logs, and configuration hashes.

Thesis evidence produced. Stop-loss event tables, fallback/unsafe action summaries, recovery timelines, and no-stop-loss ablation results.

Limitations and risks. Stop-loss is a simulated temporal risk-control mechanism, not a clinically validated safety controller.

11.11.8 Submodule 5.8: Feedback adapter

Purpose. The feedback adapter translates `RegimeStateRecord` fields into auditable signals for downstream and loopback consumers.

Bottleneck addressed. A temporal trust layer is not useful if its outputs remain isolated from IHARQ, policy, simulation, embodiment, and dashboard layers.

Scientific role. Provides the mechanism through which regime awareness can be evaluated as an architectural contribution.

Engineering responsibility. Emit Layer 4 threshold adapters, Layer 6 context fields and constraints, Layer 7 state updates, Layer 9 gating recommendations, and Layer 10 dashboard exports.

Inputs. `RegimeStateRecord`, `IHARQDecisionRecord`, `ReadinessContextRecord` template, transition state, embodiment state, and dashboard schema.

Outputs. Regime-adjustment logs, policy-context fields, simulation-state fields, embodiment-gate fields, trust-state export links, and dashboard export tables.

Evaluation method. Compare regime-aware and regime-unaware runs under identical splits,

budgets, stress profiles, action costs, and seeds. Check that every downstream change is traceable to a source `RegimeStateRecord`.

Failure modes. Hidden feedback, circular dependency, future-outcome leakage, inconsistent action constraints, and dashboard-only outputs that do not affect evaluation. Defensive design requires source, target, signal, trigger, action, safety constraint, and evaluation logic for every loop.

11.12 Feedforward and loopback contracts

Information flows into Layer 5 from Layers 1–4 as ordered records and from optional Stretch C modules as declared simulation or feedback records. Processed artifacts leave Layer 5 as `RegimeStateRecord`, `TrustStateRecord` where needed, trust scores, stop-loss warnings, and feedback-adaptor records. Loopbacks are allowed only when they solve a scientific and engineering problem and are fully logged.

11.12.1 Feedback to Layer 4: IHARQ Evidence Verification

Layer 5 makes IHARQ regime-aware. The source is `RegimeStateRecord`; the target is Layer 4 rule execution; the signal is regime label, risk score, volatility score, drawdown score, failure-risk score, trusted set, or stop-loss status; the trigger is a configured risk band or stop-loss condition; the action is stricter evidence, threshold adjustment, evidence-request adjustment, trusted-set restriction, fallback recommendation, no-action, or unsafe mode; the safety constraint is that no future label or future outcome may influence the current IHARQ decision; the evaluation logic is a paired regime-aware versus regime-unaware comparison.

Layer 5 signal	Layer 4 use
<code>regime_label</code>	Adjust behavior over canonical Layer 4 tokens <code>accept</code> , <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , <code>fallback</code> , <code>uncertain</code> , and <code>unsafe</code> .
<code>regime_risk_score</code>	Tighten or loosen acceptance thresholds within declared bounds.
<code>volatility_score</code>	Request more evidence or require stability before acceptance.
<code>drawdown_score</code>	Trigger conservative mode under sustained degradation.
<code>failure_risk_score</code>	Prevent high-risk accepts or route to fallback.
<code>trusted_set</code>	Restrict evidence combination to currently trusted sources.
<code>stop_loss_status</code>	Route to fallback, no-action, or unsafe mode.

Whenever Layer 5 changes Layer 4 behavior, the system must log the original IHARQ decision, the regime-adjusted decision, `RegimeStateRecord` ID, threshold or rule change, trigger, reason code, and outcome if available.

11.12.2 Feedback to Layer 6: Adaptive Readiness Policy

Layer 5 becomes part of the `ReadinessContextRecord`. The source is `RegimeStateRecord`; the target is Layer 6 policy context and action constraints; the signal is regime label, risk score,

trend, volatility, drawdown, trust scores, stop-loss status, and reason codes; the trigger is every policy decision point; the action is to add context fields or constrain the action space under critical stop-loss; the safety constraint is that Layer 6 cannot ignore unsafe stop-loss states except in a declared ablation; the evaluation logic is a policy comparison with and without regime context.

Recommended Layer 6 variants are policy without regime context, policy with regime label only, policy with risk score only, policy with full `RegimeStateRecord`, and policy with shuffled `RegimeStateRecord` rows as a negative control. If `stop_loss_status` is unsafe, Layer 6 may not select `accept` unless a named ablation disables the constraint. If stop-loss is active, Layer 6 should choose from `request_evidence`, `fallback`, `no_action`, or `unsafe` unless the experiment explicitly tests unconstrained behavior.

11.12.3 Feedback to Layer 7: Simulated Closed-Loop Readiness

Layer 5 updates the simulated closed-loop state. The source is the latest `RegimeStateRecord`; the target is Layer 7 environment state; the signal is current regime, current risk, trust scores, stop-loss status, and risk trend; the trigger is each simulated step or declared regime-update interval; the action is to update state variables and record transition context; the safety constraint is that simulated outcomes must be used only according to the online/simulation information protocol; the evaluation logic is fewer wrong accepts, fewer unsafe events, improved fallback timing, and better recovery under stress.

A typical Layer 7 step is: expose readiness context, select action with Layer 4 or 6, apply action, compute simulated outcome, update Layer 5 using recent transitions, and log `TransitionRecord` with the new regime state.

11.12.4 Feedback to Layer 9: MyoSuite/OpenSim Embodiment Demo

Layer 5 gates simulated assistive commands. The source is `RegimeStateRecord`; the target is the Layer 9 embodiment adapter; the signal is regime label, trust score, stop-loss status, and unsafe recommendation; the trigger is any attempt to apply a readiness-gated command; the action is allow, allow only with strong evidence, conservative command, `no_action`, `safe_hold`, or unsafe block; the safety constraint is that this remains simulation-only and proxy-mapped; the evaluation logic is endpoint error, unsafe proxy, command stability, over-assist, under-assist, delay, and action-block summaries.

Stable regimes may allow readiness-gated commands if IHARQ or policy accepts. Noisy regimes require stronger evidence. Drift-risk and fatigue-risk regimes prefer conservative commands or `no_action`. Dropout-risk and uncertain regimes prefer hold, fallback, or `no_action`. Unsafe regimes block command execution through `safe_hold`.

11.12.5 Feedback to Layer 10: Dashboard, Cards, and Reproducibility

Layer 5 feeds Layer 10 with regime timelines, risk trends, trust scores, stop-loss warnings, feedback-adjustment logs, ablation summaries, and regime cards. The source is saved Layer 5 records; the target is dashboard/card generation; the signal is all dashboard-ready `RegimeRisk` artifacts; the trigger is experiment completion or dashboard refresh; the action is visualization only; the safety constraint is that Layer 10 must not modify experimental records; the evaluation logic is traceability and reproducibility of every regime plot and card.

Required dashboard views include regime timeline, trust allocation, stop-loss, risk explanation, stress-regime, embodiment-risk, and ablation comparison views.

11.13 Regime vocabulary, transitions, and threshold governance

Layer 5 must define how regimes transition, not only how they are labeled.

From	To	Example trigger
stable	noisy	Sudden entropy spike, confidence jump, or agreement fluctuation.
noisy	stable	Sustained recovery in confidence, agreement, and evidence quality.
stable	drift-risk	Rising calibration error and model-family disagreement.
drift-risk	unsafe	Repeated wrong accepts, agreement collapse, or critical stop-loss trigger.
fatigue-risk	stable	Sustained recovery after cooldown.
dropout-risk	uncertain	Evidence source remains unavailable but unsafe threshold is not reached.
uncertain	unsafe	Stop-loss severity becomes critical or unsafe simulated proxy is triggered.

Every regime transition should log previous regime, new regime, trigger, duration in previous regime, affected thresholds/actions, downstream recommendation, stress/simulation tag, and reason codes.

Risk bands should be governed through configuration:

Risk band	Meaning	Recommended behavior
low	Temporal trust acceptable.	Normal operation.
moderate	Caution required.	Slightly stricter thresholds or evidence requirements.
high	Action risk elevated.	Request evidence, fallback, or restrict trusted set.
critical	Unsafe simulated action risk or active critical stop-loss.	Block action, <code>safe_hold</code> , <code>no_action</code> , or unsafe state.

For every threshold, document name, purpose, default value, tuning split, affected layers, sensitivity analysis, fixed-or-learned status, ablation status, and leakage status. Risk thresholds must not be tuned on final test outcomes.

11.14 Evaluation design

Layer 5 must be evaluated as a scientific module, not only as an implementation convenience. After the Ablation and Evaluation Architecture merge, Layer 5 owns the temporal-trust slice of the global ablation ladder. Its primary global ablation identity is `A7`, where IHARQ or policy behavior with RegimeRisk is compared against matched regime-unaware variants. Layer 5 also supplies stress-conditioned and simulation-derived evidence for `A12` when StressForge

activates temporal instability, and it can supply optional embodiment-risk context for *A13* only as simulation-bound proxy evidence.

11.14.1 Evaluation questions

Question	Meaning
Does Layer 5 detect unstable periods?	Compare regime labels and risk scores with error bursts, uncertainty spikes, stress profiles, dropout events, simulated unsafe events, or other legally available past/outcome signals.
Does Layer 5 improve readiness decisions?	Compare IHARQ/policy with and without RegimeRisk context under matched splits, budgets, temporal sequences, stress profiles, reward/cost settings, and action costs.
Does Layer 5 explain failures?	Check whether high-risk periods align with failure taxonomy, high-confidence wrong cases, stop-loss events, repeated fallback, or unsafe-event clusters.
Does Layer 5 generalize across subjects/sessions?	Test whether regime-risk behavior holds beyond one subject, one split, one session, one stress seed, or one model family.
Does Layer 5 avoid over-abstention?	Measure coverage, evidence cost, fallback rate, request burden, no-action rate, stop-loss activation, recovery latency, and valid-action blocking.
Does Layer 5 improve interpretability?	Verify that regime cards, timelines, risk-error tables, and claim-evidence entries are traceable to records and explain action changes.

11.14.2 Metrics

Layer 5 metrics include risk-error alignment, high-risk precision, high-risk recall, risk calibration, risk-coverage curves, wrong-accept reduction, unsafe-event reduction, high-risk accepted-error rate, regime transition consistency, early-warning lead time where valid, fallback efficiency, evidence cost, request burden, fallback rate, no-action rate, stop-loss activation rate, stop-loss compliance, recovery latency, threshold-adjustment frequency, shuffled-risk control performance, random-risk control performance, card trace-completeness, and claim-evidence completeness.

Every metric must define its denominator and provenance. If a metric uses true labels, future outcomes, simulated stress outcomes, or embodiment outcomes, that metric is evaluation-only and cannot be part of the decision-time `RegimeStateRecord`. Subject-level aggregation is required where possible, and regime-aware gains should be reported separately for clean, unstable, stressed, and recovery periods.

11.14.3 Baselines and ablations

Core baselines are decoder only, decoder plus calibration/uncertainty, decoder plus IHARQ-lite, confidence-only rejection, entropy-only rejection, margin rejection, selective prediction, multi-window voting, probability averaging, ordinary ensembles, fallback-only, and no-regime policy. RegimeRisk-specific comparisons include decoder + IHARQ + RegimeRisk, adaptive policy without RegimeRisk, adaptive policy with regime label only, adaptive policy with risk score

only, adaptive policy with full `RegimeStateRecord`, full Stretch C with `RegimeRisk`, full Stretch C with shuffled `RegimeRisk`, and full Stretch C with random risk score.

Feature ablations should remove volatility features, drawdown features, trust allocator, stop-loss module, stress inputs, embodiment outcomes, and feedback constraints one at a time. Mandatory negative controls are shuffled `RegimeRisk`, random risk score, and no-regime policy under identical temporal sequences. Every ablation should report accuracy only as one metric among coverage, wrong accepts, evidence cost, fallback rate, unsafe events, request burden, stop-loss burden, recovery latency, and interpretability artifacts.

11.14.4 Matched-comparison discipline

Layer 5 comparisons must match dataset, subject, session, trial/window sequence, split, low-calibration budget, decoder family, seed, calibration method, IHARQ configuration, temporal window size, risk-threshold configuration, stop-loss configuration, stress schedule, reward/cost configuration where relevant, and embodiment mode where relevant. A `RegimeRisk`-aware variant may support a thesis claim only if its regime-unaware counterpart is run on the same ordered sequence. If matching fails, the comparison must be labeled diagnostic and the claim-evidence matrix must mark the affected claim as weakened.

11.14.5 StressForge-Lite interaction

Layer 5 should be evaluated under clean baseline, single stress profile, stress intensity sweep, combined stress, held-out stress, and recovery-after-stress scenarios. Expected behavior is that delay can increase action outcome risk, dropout increases dropout-risk, noisy intent increases volatility/disagreement, weakness affects simulation-only embodiment outcome risk, drift increases drift-risk, fatigue-like degradation increases drawdown/fatigue-risk, and combined stress increases stop-loss or unsafe recommendations.

StressForge-Lite conditions are synthetic or controlled stress profiles. They evaluate robustness under declared stress; they do not prove real physiological fatigue detection.

11.14.6 Failure interpretation

If Layer 5 does not improve wrong-accept reduction, it may still be useful if it provides interpretable failure timelines without excessive evidence cost. If it improves wrong-accept reduction but over-abstains, the thesis must report the coverage and fallback burden honestly. If shuffled or random risk performs similarly, the regime features are not adding real signal and must be revised or downgraded. If Layer 5 helps only under synthetic stress, that claim must be labeled as stress-conditioned and simulation-derived. If stop-loss blocks many valid actions, the result should be preserved as a conservative-failure or over-abstention negative-result note rather than hidden.

Each Layer 5 evaluation run must save an ablation-step ID, system-variant config, matched-comparison key, metric dictionary ID, source-record pointers, statistical summary where feasible, dashboard/card pointer, limitation statement, and negative-result note when applicable.

11.15 Reproducibility and artifact structure

Layer 5 must be implemented through configuration, schemas, saved records, and deterministic regeneration scripts.

```
src/regime_risk/
|-- __init__.py
|-- temporal_features.py
|-- volatility_monitor.py
|-- drawdown_monitor.py
|-- regime_classifier.py
|-- failure_risk_predictor.py
|-- trust_allocator.py
|-- stop_loss.py
|-- feedback_adapter.py
|-- schemas.py
'-- tests/

configs/regime_risk/
|-- default.yaml
|-- rolling_windows.yaml
|-- thresholds.yaml
|-- ablations.yaml
|-- matched_comparison.yaml
|-- metric_dictionary.yaml
'-- stress_conditioned.yaml

contracts/
|-- regime_state_record_schema.yaml
|-- trust_state_record_schema.yaml
|-- trust_score_schema.yaml
'-- stop_loss_warning_schema.yaml

results/regime_risk/
|-- regime_state_records.parquet
|-- trust_state_records.parquet
|-- trust_scores.csv
|-- stop_loss_warnings.csv
|-- regime_timelines.csv
|-- risk_error_alignment.csv
|-- matched_comparison_report.csv
|-- negative_result_notes.csv
|-- claim_evidence_entries.json
'-- ablation_results.csv

figures/regime_risk/
|-- regime_timeline_subject_SXX.pdf
|-- risk_trend_plot.pdf
|-- trust_allocation_plot.pdf
|-- stop_loss_events.pdf
'-- risk_coverage_curve.pdf
```

```
cards/regime_cards/
|-- regime_card_dataset_subject_session.md
'-- regime_card_summary.pdf
```

Required configuration files include `configs/regime_risk/default.yaml`, `configs/regime_risk/rolling_windows.yaml`, `configs/regime_risk/thresholds.yaml`, `configs/regime_risk/ablations.yaml`, `configs/regime_risk/matched_comparison.yaml`, `configs/regime_risk/metric_dictionary.yaml`, and `configs/regime_risk/stress_conditioned.yaml`. Required schemas include `contracts/regime_state_record_schema.yaml`, `contracts/trust_state_record_schema.yaml`, `contracts/trust_score_schema.yaml`, and `contracts/stop_loss_warning_schema.yaml`. Expected output files include `regime_state_records.parquet`, `trust_state_records.parquet`, `trust_scores.csv`, `stop_loss_warnings.csv`, `regime_timelines.csv`, `risk_error_alignment.csv`, and `ablation_results.csv`.

Traceability requirements are strict: every plotted risk point, regime label, trust score, `TrustStateRecord` row, stop-loss event, threshold adjustment, ablation row, negative-result note, claim-evidence entry, or dashboard card must trace back to source record IDs, split IDs, configuration hashes, seed values, schema versions, global/local ablation IDs, and matched-comparison keys. Any learned submodule must save random seeds, package versions, training split, validation split, test split, and model artifact hash.

11.16 Implementation phases and readiness levels

Layer 5 should be implemented in maturity phases:

Phase	Name	Expected artifacts
5.0	Contract and schema definition	<code>RegimeStateRecord</code> schema, <code>TrustStateRecord</code> schema, trust-score schema, stop-loss schema, validation script, mock records.
5.1	Temporal feature builder	Rolling features, leakage checks, temporal feature tables.
5.2	Volatility and drawdown monitors	Volatility scores, drawdown scores, reason codes, trend plots.
5.3	Rule-based regime classifier	Regime labels, probabilities/confidence estimates, regime timelines.
5.4	Trust allocator	<code>TrustStateRecord</code> exports, model-family trust scores, trusted set, trust timeline.
5.5	Stop-loss module	Warning log, stricter-evidence/fallback/no-action/unsafe recommendations, recovery conditions.

5.6	Feedback adapters	Layer 4 threshold adapter, Layer 6 context adapter, Layer 7 state adapter, Layer 9 embodiment gate, Layer 10 dashboard exporter.
5.7	Evaluation and ablation	Risk-error alignment, with/without Layer 5 comparisons, feature ablations, shuffled/random risk negative controls.
5.8	Cards and thesis integration	Regime cards, thesis-ready figures, limitation text, final Layer 5 dossier.

Layer 5 readiness levels are:

Level	Name	Meaning
L5-RL0	Concept	RegimeRisk is only described as an idea.
L5-RL1	Bottleneck defined	Temporal trust and static-threshold limitation are clearly stated.
L5-RL2	Input contract defined	Required records and optional simulation inputs are specified.
L5-RL3	Output contract defined	RegimeStateRecord, TrustStateRecord where needed, trust scores, and stop-loss warnings are specified.
L5-RL4	Temporal features implemented	Rolling features are produced and validated.
L5-RL5	Regime labels implemented	Rule-based or learned regime classifier outputs stable labels.
L5-RL6	Feedback integrated	Layer 5 affects IHARQ, policy context, simulation, or embodiment.
L5-RL7	Evaluated	Risk-error alignment and ablations are completed.
L5-RL8	Reproducible	Configs, schemas, seeds, and record outputs are versioned.
L5-RL9	Thesis-ready	Figures, cards, claims, limitations, and defense text are written.

Layer 5 should not be treated as a strong thesis contribution until at least L5-RL7. It should not be used for strong thesis claims until L5-RL9.

11.17 Claim-safety, limitations, and defensive design

Layer 5 uses finance-inspired terms such as volatility, drawdown, trust, and stop-loss. These analogies are useful because they describe temporal risk and sustained degradation, but they must be adapted carefully. The thesis should say that Layer 5 estimates temporal operating-state risk, monitors drift-risk and fatigue-like degradation proxies, makes IHARQ more conservative under unstable regimes, supplies context to adaptive readiness policies, and provides interpretable regime cards. It should not say that Layer 5 detects real fatigue, guarantees safety, functions as a validated clinical risk controller, prevents patient harm, or directly transfers financial risk theory into BCI reliability without adaptation.

Common failure modes and defensive designs are:

Failure mode	Why it matters	Defensive design
Arbitrary risk score	Examiners may reject vague risk claims.	Define features, thresholds, weights, and reason codes.
Decorative regime labels	Labels may not support claims.	Evaluate against error bursts, stress profiles, unsafe proxies, and ablations.
Over-abstention	The system may become too conservative.	Report coverage, request burden, fallback rate, no-action rate, and evidence cost.
Unfair stop-loss tuning	Test outcomes could be exploited.	Tune on training/validation only and freeze before test evaluation.
Fatigue/drift overclaiming	Public data may not support clinical state claims.	Use fatigue-like, drift-risk, and proxy language.
Hidden feedback loop	Downstream changes become irreproducible.	Log original decisions, adjusted decisions, thresholds, triggers, and reason codes.
Policy ignores RegimeRisk	Layer 5 becomes unused.	Define context fields and action constraints; include ablation where disabled.
Cosmetic dashboard	Regime view may not support thesis evidence.	Generate cards, timelines, ablation tables, matched-comparison reports, negative-result notes, and traceable failure explanations.
Simulation mixed with real evidence	Claims become unclear.	Tag provenance for all simulation-derived inputs.
Unmatched regime comparison	A RegimeRisk gain may reflect a different temporal sequence, stress schedule, or reward setting.	Require matched-comparison keys and mark unmatched results diagnostic-only.
Random risk performs similarly	RegimeRisk may not add real temporal information.	Preserve shuffled/random-risk negative controls and downgrade the claim when they match full RegimeRisk.

A defensible thesis paragraph for Layer 5 is:

Layer 5 does not claim to diagnose physiological fatigue or clinically validate BCI safety. Instead, it operationalizes temporal trust monitoring within the public-data and simulated-readiness scope of the thesis. The layer uses rolling prediction, uncertainty, calibration, stability, decision, stress, and simulated outcome signals to estimate operating-regime risk and to provide conservative recommendations to downstream readiness layers. Its contribution is architectural and evaluative: it tests whether temporal regime context improves reliability interpretation, failure explanation, and simulated readiness control compared with static confidence, uncertainty, or IHARQ-only baselines.

11.18 Layer 5 dossier template

Each implementation of Layer 5 should generate a short dossier with the following fields:

- identity: layer name, functional module name, implementation version, status, repository path, config path, output path, and thesis section;
- bottleneck: temporal reliability bottleneck addressed and what would fail without it;
- inputs: required records, optional records, simulation-derived inputs, evaluation-only inputs, and validation checks;
- outputs: `RegimeStateRecord` fields, `TrustStateRecord` fields where needed, trust-score outputs, stop-loss outputs, timeline outputs, dashboard/card outputs;
- temporal features: window sizes, feature groups, leakage prevention, missing-data behavior;
- regime model: vocabulary, classification method, reason codes, thresholds, risk calibration;
- trust allocation: trust targets, update rules, trusted-set rule, fallback-trust rule;
- stop-loss logic: triggers, actions, cooldown, recovery condition, unsafe-mode rule;
- feedback contracts: Layer 4 effect, Layer 6 effect, Layer 7 effect, Layer 9 effect, Layer 10 effect;
- evaluation: metrics, baselines, ablations, stress tests, negative controls, and results;
- limitations: public-data limitations, proxy fatigue/drift limitations, simulation-only limitations, and clinical non-claim wording;
- definition of done: schemas complete, features implemented, regime labels validated, feedback logged, ablations completed, cards generated, thesis text written.

11.19 Outputs

Layer 5 outputs `RegimeStateRecord`, `TrustStateRecord` where trust allocation is separately consumed, trust scores, trusted sets, risk trends, regime timelines, volatility scores, drawdown scores, drift/fatigue/dropout risk fields, failure-risk scores, fallback recommendations, stricter-evidence recommendations, `no_action` recommendations, unsafe-state recommendations, stop-loss warnings, regime transition logs, threshold-adjustment logs, risk-error alignment tables, matched-comparison reports, ablation summaries, negative-result notes, metric-dictionary entries, claim-evidence matrix entries, dashboard-ready tables, and `RegimeCard` outputs.

The minimum viable Layer 5 output is a valid `RegimeStateRecord` with regime label, risk score, risk trend, volatility score, drawdown score, drift/fatigue/dropout risk fields where configured, trust score summary, stop-loss status, recommendation, reason codes, and provenance. A strong Layer 5 adds risk calibration, `TrustStateRecord` exports for trust allocation by model family and evidence source, explicit stop-loss/recovery rules, feedback logs to Layers 4 and 6, simulation-state updates for Layer 7, embodiment-gating compatibility for Layer 9, and dashboard/card exports for Layer 10.

11.20 Feedback to other layers

RegimeRisk feeds Layer 4 by tightening or loosening IHARQ thresholds within declared bounds, changing request-evidence limits, restricting trusted evidence sources, and recommending fallback, no-action, or unsafe mode. It feeds Layer 6 by becoming part of the `ReadinessContextRecord` and by constraining action selection under active stop-loss or unsafe states. It feeds Layer 7 by updating the simulated closed-loop state with current regime, risk, trust, stop-loss, and recovery status. It feeds Layer 9 by gating simulated assistive commands under proxy mappings. It feeds Layer 10 through regime timeline dashboard views, `TrustStateRecord`-backed trust allocation views, stop-loss views, risk-explanation views, stress-regime views, embodiment-risk views, ablation comparisons, and `RegimeCard` outputs.

11.21 Definition of done

Layer 5 is complete when the temporal trust bottleneck is clearly stated; the layer is distinguished from Layer 3 uncertainty and Layer 4 IHARQ; inputs are classified as primary, auxiliary, feedback, proxy, simulation-only, or evaluation-only; `RegimeStateRecord` schema is defined; `TrustStateRecord` is defined when trust allocation is exported separately; the temporal feature builder, volatility-like monitor, drawdown-like monitor, regime classifier, failure-risk predictor, trust allocator, stop-loss module, and feedback adapter are specified and implemented at the declared maturity level; feedback to Layer 4 is logged and explainable; feedback to Layer 6 is part of `ReadinessContextRecord`; feedback to Layer 7 updates simulated closed-loop state; feedback to Layer 9 gates simulated assistive commands; Layer 10 receives regime timeline, trust-state, and regime-card artifacts; risk thresholds are configured and not tuned on final test outcomes; ablations show whether Layer 5 adds value beyond confidence, uncertainty, IHARQ-only baselines, and regime-unaware policies under matched temporal sequences; negative controls such as shuffled risk or random risk are included; matched-comparison reports, metric dictionaries, negative-result notes, and claim-evidence entries exist; claim boundaries avoid clinical fatigue, medical safety, and real rehabilitation validation claims; and all figures, cards, and dashboard views are traceable to saved records.

For thesis-level claims, Layer 5 should reach L5-RL9: implemented features, validated regime labels, integrated feedback, completed ablations, reproducible configs/schemas/seeds/records, dashboard artifacts, limitation text, and claim-safe writing must all be present.

12 Layer 6: Adaptive Readiness Policy Layer

12.1 Problem solved by Layer 6

Rules are interpretable but can be brittle. A fixed threshold may not transfer across subjects, datasets, decoders, regimes, calibration budgets, or stress conditions. Layer 6 solves the policy bottleneck that appears after the system has measured evidence quality but before it decides what readiness action should be taken. Layers 2–5 can produce predictions, calibrated probabilities, uncertainty features, IHARQ decisions, evidence-quality fields, and RegimeRisk state. They do not by themselves answer the policy question:

Given a calibrated neural-intent prediction, its uncertainty, its evidence stability, its IHARQ state, its RegimeRisk state, its recent action history, and its simulated stress or embodiment context, what readiness action should the system take now?

Layer 6 is therefore the adaptive decision-policy core of IHARQ BenchGuard in the Stretch C architecture. It converts the upstream reliability substrate into auditable readiness actions. When it consumes Layer 4 records, it treats `accept`, `wait`, `request_evidence`, `combine_evidence`, `fallback`, `uncertain`, and `unsafe` as canonical stored IHARQ action tokens; when it learns broader policies, it may add explicitly mapped policy-only actions such as `defer`, `no_action`, threshold-mode selection, supervisor routing, or simulation-only action aliases. It is not merely another classifier after the decoder, not a replacement for Layer 4 IHARQ, and not a clinical controller. It is a staged policy-learning layer that compares interpretable rules, learned evidence-quality estimation, supervised multi-action readiness learning, deferral-aware routing, contextual bandit adaptation, and simulation-only reinforcement learning under explicit cost, reward, safety, and claim boundaries.

The thesis role of Layer 6 is to test whether adaptive readiness policies improve the trade-off between correct timely action, wrong-accept avoidance, evidence-request burden, waiting burden, fallback burden, deferral burden, and unsafe-action prevention compared with static thresholds, selective prediction, rule-based IHARQ-lite, and simple deferral baselines. This layer supports the thesis claim that IHARQ BenchGuard is not only a decoder benchmark and not only a rule-based rejection wrapper; it is an architecture for deciding when neural-intent evidence is ready enough, stable enough, and safe enough for simulated assistive-action evaluation.

Layer 6 also protects the thesis against several predictable criticisms. It protects against the criticism that IHARQ is only hand-designed thresholding by introducing learned policy alternatives and matched ablations. It protects against the criticism that adaptive control claims are vague by requiring explicit action spaces, target-construction routes, cost models, reward models, legality rules, and controller cards. It protects against the criticism that learned readiness labels are artificial by openly declaring them as computational proxy labels derived from public EEG task labels, decoder correctness, future-window evidence, fallback comparisons, and simulation rewards. If Layer 6 were removed, the architecture would still measure reliability, but it would not test whether that reliability evidence can be transformed into adaptive readiness decisions under changing context.

Claim-safety rule: Layer 6 may claim simulated, offline, and proxy readiness-policy evidence. It must not claim real patient-safe control, clinical readiness labels, validated rehabilitation efficacy, or real muscle control from public EEG datasets.

12.2 Merged policy-learning ownership and source-section status

This Layer 6 section now absorbs the former standalone *Layer 19: Stretch C Policy-Learning Architecture* dossier. The absorbed material is not treated as an additional runtime layer. Its durable architectural content is owned here because it specifies the same responsibility already assigned to Layer 6: transforming split-safe readiness context into auditable policy actions, deferral routes, evidence-quality estimates, and simulation-only learning traces.

The merge clarifies three ownership rules. First, Layer 6 is the sole owner of the policy-learning plane. Second, Layer 6 must keep the policy-learning components staged from interpretable baselines to simulation-only adaptation, rather than presenting contextual bandit or RL as mandatory core results. Third, the retired dossier's implementation-facing details are preserved inside the Layer 6 contracts, submodules, leakage rules, evaluation design, dashboard outputs, reproducibility artifacts, failure controls, and definition of done.

12.3 Engineering responsibility, scope, and boundaries

The engineering responsibility of Layer 6 is to define, train, evaluate, and log readiness policies through record-based interfaces. It receives structured records from Layers 2–5 and, in simulation modes, from Layers 7–9. It outputs structured policy records, deferral records, learned policy artifacts, reward/regret traces, ablation tables, and controller cards for Layers 7–10.

In scope. Layer 6 includes evidence-quality estimation; supervised Adaptive-IHARQ from declared cost-based pseudo-labels; learning-to-defer; contextual bandit policies in declared simulated environments; reinforcement-learning readiness policies only after Layer 7 simulation is stable; policy logs; reward/regret curves; action distributions; policy ablations; stress-conditioned comparisons; optional embodiment-aware reward terms; and controller-card export.

Out of scope. Layer 6 does not create real clinical readiness labels from public EEG data, validate real patient-safe BCI control, replace clinical trials or medical-device verification, infer real muscle activation from motor-imagery labels, deploy online learning on patients, silently tune policies on test labels, or present simulated reward optimization as clinical efficacy. Dashboard interactions must not alter experimental results unless the configuration, seed, and run identifier are tracked.

Boundary with Layer 4. Layer 4 is evidence verification. It produces interpretable IHARQ-style decisions, reason codes, evidence requests, fallback states, uncertain states, and unsafe states. Layer 6 may learn from Layer 4 outputs and compare against Layer 4, but it must not erase Layer 4’s interpretability role. In records, Layer 6 must preserve the Layer 4 canonical action token exactly as stored in `IHARQDecisionRecord`; display labels such as “Request evidence” or policy aliases such as `defer`, `no_action`, supervisor route, or simulation arm belong to Layer 6 mapping fields rather than replacing the source IHARQ token.

Boundary with Layer 5. Layer 5 estimates temporal trust, `RegimeRisk`, trust allocation, volatility-like behavior, drawdown-like behavior, drift/fatigue/dropout risk fields, and stop-loss warnings. Layer 6 consumes `RegimeStateRecord` fields as context and constraints, and consumes linked `TrustStateRecord` rows only when the policy context explicitly enables independent per-target trust timelines. Layer 6 should not recompute trust timelines, redefine regime labels independently, or relabel Layer 5 fatigue-like proxies as clinical state except in a named feature-ablation experiment.

Boundary with Layer 7. Layer 7 is the environment and transition simulator. Layer 6 is the policy. Layer 6 selects an action; Layer 7 applies the action in replay or simulation and returns transition outcomes, rewards, delays, safety events, and next states. This separation is required for contextual bandit and RL experiments.

Boundary with Layer 8. Layer 8 generates declared stress profiles. Layer 6 may be evaluated under stress variables, but it must mark those variables as simulated stress conditions, not real physiological measurements.

Boundary with Layer 9. Layer 9 is the embodiment adapter. Layer 6 may gate or modulate whether a simulated assistive command is attempted, held, routed to fallback, or blocked, but it must not hard-code MyoSuite or OpenSim internals and must not claim real-body safety.

Core submodules are the evidence-quality estimator, supervised Adaptive-IHARQ, learning-to-defer, policy action-space manager, cost/reward model manager, policy logger, and controller-card exporter. Supporting submodules are context construction, action-legality masking, pseudo-label generation, feature ablation, reason-code explanation, stress-conditioned evaluation, and matched-cost comparison. Optional or stretch submodules are contextual bandit adaptation, RL readiness policy, and embodiment-aware policy extensions. Future-work items

include real online BCI policy adaptation, human-in-the-loop supervision, and clinically validated safety controllers; these must remain outside thesis claims unless actually implemented and validated.

12.4 Layer 6 identity and design principles

Identity field	Layer 6 value
Layer name	Adaptive Readiness Policy Layer
Architectural plane	Policy plane
Primary role	Learn, compare, and select readiness actions under uncertainty, regime risk, declared action costs, simulated stress, and optional simulated embodiment consequences.
Main inputs	ReadinessContextRecord, PredictionRecord, CalibrationRecord, UncertaintyRecord, ThresholdRegistryRecord, EvidenceQualityRecord, IHARQDecisionRecord, RegimeStateRecord, conditional TrustStateRecord rows when independent trust timelines are used, cost/reward configuration, pseudo-label files, StressProfileRecord, TransitionRecord, RewardTraceRecord, optional SafetyEventRecord, and optional EmbodimentStateRecord.
Main outputs	PolicyActionRecord, DeferralRecord, updated EvidenceQualityRecord, learned policy artifacts, policy logs, action distributions, reward/regret curves, ablation tables, and controller cards.
Core thesis status	Core for learned evidence-quality estimation, supervised Adaptive-IHARQ, and learning-to-defer comparisons.
Stretch thesis status	Contextual bandit, RL readiness policy, and embodiment-aware policy results are simulation-only stretch components.
Main risks	Artificial readiness labels, reward arbitrariness, leakage from future labels, simulator overfitting, policy collapse, unnecessary complexity, and clinical overclaim.

Layer 6 follows eight design principles:

1. **Staged policy learning.** Implement policy learning in increasing complexity: evidence-quality estimator $\rightarrow$ supervised Adaptive-IHARQ $\rightarrow$ learning-to-defer $\rightarrow$ contextual bandit $\rightarrow$ reinforcement learning. Each stage must remain useful even if later stages are not completed.
2. **Declared proxy labels.** Any accept, wait, request, defer, fallback, no-action, or unsafe label derived from public EEG is a computational readiness label, not a real clinical readiness label.
3. **Cost and reward transparency.** A policy is not fully specified until its wrong-accept penalty, delay cost, evidence-request cost, fallback cost, unsafe penalty, stress multiplier, and optional embodiment terms are declared.

4. **Baseline discipline.** Every learned policy must be compared against simple confidence thresholds, entropy/margin rejection, selective prediction, longer-window decoding, multi-window voting, probability averaging, ordinary ensembles, IHARQ-lite, and deferral where applicable.
5. **No leakage.** Target construction may use true labels and future-window evidence for training-label generation or oracle analysis, but evaluation-time policy context must not contain hidden correctness or future labels unless the policy has explicitly chosen to wait or request evidence.
6. **Interpretability retained.** Learned policies should output reason codes, action probabilities, expected costs, threshold choices, feature-reliance summaries, action legality masks, or controller-card explanations.
7. **Simulation-only adaptation.** Contextual bandit and RL components are simulated online adaptation mechanisms unless real online data and validation exist.
8. **Record-first implementation.** All Layer 6 inputs, outputs, costs, rewards, and policy decisions must be saved as structured records. Hidden notebook state is not acceptable.

12.5 Architectural position and policy plane ownership

Layer 6 occupies the policy plane between the evidence substrate and the simulated environment. It does not alter public-data validity, train the base decoder, recalibrate probabilities, redefine IHARQ evidence checks, or manufacture real clinical labels. It consumes those upstream products and decides which readiness action is justified under the declared context, cost, legality, and claim boundary.

Neighbor	Relationship to Layer 6	Boundary rule
Layer 0	Supplies claim-safety rules, forbidden claims, leakage constraints, and limitation language.	Layer 6 must mark proxy, offline, simulated, and embodiment-only claims explicitly.
Layer 1	Supplies public-data protocol validity, subject/session/split structure, dataset cards, and calibration-budget definitions.	Layer 6 may not reinterpret public EEG task labels as real readiness or clinical labels.
Layer 2	Supplies predictions, logits, probabilities, model identifiers, baseline families, fallback candidates, and decoder records.	Layer 6 may compare policies over decoder outputs but must not silently retrain or retune decoders.
Layer 3	Supplies calibration, uncertainty, ThresholdRegistryRecord entries, selective-prediction outputs, and threshold baselines.	Layer 6 may consume calibrated uncertainty and registered threshold baselines but may not tune calibration thresholds on test labels.

Layer 4	Supplies IHARQ decisions, evidence-state fields, reason codes, request counts, fallback flags, and unsafe flags.	Layer 6 can learn over these signals but must preserve IHARQ-lite as an interpretable baseline rather than overwrite it.
Layer 5	Supplies RegimeRisk labels, temporal trust scores, drift/fatigue/dropout risk fields, volatility/drawdown-like features, stop-loss warnings, and linked TrustStateRecord rows where independent trust timelines are materialized.	Layer 6 may tighten action legality or preference under high risk or per-target trust change, but must not claim real fatigue, clinical state detection, or independently recompute Layer 5 trust state.
Layer 7	Supplies TransitionRecord , RewardTraceRecord , next states, latencies, SafetyEventRecord links where materialized, EnvironmentDiagnosticRecord warnings, and optional PolicyUpdateTrace entries for simulated policy evaluation.	Bandit/RL updates are allowed only under declared simulation protocols, and frozen-policy evaluation must remain update-disabled or explicitly audited.
Layer 8	Supplies stress profiles and stress schedules.	Stress variables are simulated experimental conditions, not observed patient physiology.
Layer 9	Supplies optional embodiment state, endpoint error, command status, and embodiment safety proxies.	Embodiment-aware terms remain simulated and must not be framed as real muscle control.
Layer 10	Consumes policy logs, controller cards, ablation tables, reward/regret curves, limitation statements, and dashboard records.	Dashboard inspection must not mutate experimental records without a new run ID and configuration hash.

The canonical transformation owned by Layer 6 is:

ReadinessContextRecord+**policy_config**+**legality_rules**+**cost/reward_model** → **PolicyActionRecord**

The final term is used only when a contextual-bandit or RL component performs a simulation-authorized policy update.

12.6 Input contract

The central input to Layer 6 is the **ReadinessContextRecord**. It is a standardized policy context assembled from upstream records and, in simulation modes, from downstream environment records. It defines what the policy is allowed to know at decision time.

Input class	Record or artifact	Purpose in Layer 6	Status
-------------	--------------------	--------------------	--------

Primary input	PredictionRecord	Provides predicted class, probability vector, logits or decision scores, model ID, subject/session/trial/window IDs, split ID, seed, and configuration hash.	Hard
Primary input	CalibrationRecord	Provides calibrated probability, calibration method, reliability-bin membership, ECE/Brier/NLL context, and calibration provenance.	Hard for calibrated policies
Primary input	UncertaintyRecord	Provides confidence, entropy, margin, variation ratio, disagreement, conformal-set proxies, and selective flags.	Hard
Policy input	ThresholdRegistryRecord	Provides registered threshold feature, selected value or rule, selection split, target constraint, applicable model IDs, allowed consumers, leakage status, rationale, and fairness note for threshold-aware baselines and operating-point selection.	Hard for threshold or selective-prediction policies
Primary input	IHARQDecisionRecord	Provides the Layer 4 canonical stored action token, optional display label, reason code, request count, fallback flag, unsafe flag, accepted prediction, rule path, and action cost. Layer 6 must validate the token against <code>accept</code> , <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , <code>fallback</code> , <code>uncertain</code> , and <code>unsafe</code> before mapping it into a broader policy action.	Hard for Adaptive-IHARQ comparisons
Primary input	RegimeStateRecord	Provides regime label, RegimeRisk score, trust-score summary, risk trend, volatility/drawdown features, explicit drift/fatigue/dropout risk fields, trusted set, optional <code>trust_state_record_ids</code> , and stop-loss warnings.	Hard when Layer 5 active
Conditional policy input	TrustStateRecord	Provides per-target trust timeline rows for decoder families, evidence sources, windows, channel subsets, or fallback routes when the policy consumes trust independently rather than only through the compact <code>RegimeStateRecord</code> summary.	Hard only for policies declaring independent trust-timeline features; otherwise not required

Supporting input	EvidenceQualityRecord	Provides evidence-quality score, predicted failure probability, decoder-correctness target, and evidence source identifiers.	Hard after 6.1
Metadata input	Split manifest and DatasetRecord fields	Prevents leakage and supports subject-wise, session-wise, dataset-wise, and calibration-budget evaluation.	Hard
Policy input	Cost and reward configuration	Defines action costs, reward weights, unsafe penalty, delay penalty, request cost, fallback cost, stress multipliers, and terminal conditions.	Hard
Proxy-label input	Pseudo-label files	Provides cost-based oracle labels and deferral targets for supervised training.	Hard for 6.2/6.3
Simulation input	TransitionRecord , RewardTraceRecord , and optional SafetyEventRecord from Layer 7	Provides state, action, reward, reward decomposition, next state, outcome, safety event, and latency for bandit/RL evaluation.	Soft/stretch
Stress input	StressProfileRecord	Provides stress type, intensity, delay, dropout, drift, noisy intent, weakness, fatigue-like degradation, and combined-stress label.	Soft/stretch
Embodiment input	EmbodimentStateRecord , EmbodimentOutcomeRecord , optional SafetyEventRecord links, optional Layer9RewardTrace links, and Layer 9 outcome summaries	Provides simulator type, embodiment mode, command status, endpoint error, safety proxy, safety-event identifier, reward-trace identifier where enabled, and embodiment response for simulation-only policy analysis.	Optional/stretch

12.6.1 Canonical ReadinessContextRecord schema

Field group	Representative fields	Required status
Identity fields	dataset_id, subject_id, session_id, run_id, trial_id, window_id, split_id, label_budget_id, seed, config_hash.	Always
Decoder fields	model_id, model_family, predicted_class, probability_vector, logits where available, decision_score where available.	Always when decoder exists
Calibration fields	calibrated_probability, calibration_method, reliability_bin, calibration_error_estimate, threshold_registry_id.	Required for calibrated policies
Uncertainty fields	confidence, entropy, margin, variation_ratio, disagreement, conformal_set_size where used.	Required
Evidence fields	stability_score, channel_stability, model_agreement, evidence_quality_score, predicted_failure_probability, combined_evidence_summary.	Required for IHARQ-aware policies

IHARQ fields	iharq_action as the canonical Layer 4 token, optional iharq_action_label, iharq_reason_code, fallback_flag, unsafe_flag, accepted_prediction, and evidence_request_count.	Required for Adaptive-IHARQ
Regime fields	regime_label, regime_probability, regime_risk_score, trust_scores, risk_trend, volatility_score, drawdown_score, drift_risk_score, fatigue_like_risk_score, dropout_risk_score, trust_state_record_ids where emitted, and stop_loss_status.	Required when Layer 5 active
Trust timeline fields	trust_state_record_ids, trust_target_type, trust_target_id, trust_score, trust_trend, trust_band, trusted_set_membership, update_source, and decay_recovery_state.	Required only when TrustStateRecord rows are consumed as independent policy features
History fields	previous_action, previous_decision_latency, prior_request_count, recent_accepted_rate, recent_fallback_rate, recent_unsafe_rate.	Required for sequential policies
Stress fields	stress_type, stress_intensity, delay_level, dropout_level, drift_level, noisy_intent_level, weakness_level, combined_stress_flag.	Required for stress experiments
Embodiment fields	simulator_type, embodiment_mode, command_status, endpoint_error, safety_proxy, safety_event_id, reward_trace_id where enabled, demo_manifest_id, and embodiment_state_summary.	Optional/stretch; decision-time use is limited to current or previous simulated state permitted by the declared observation schema
Target fields	true_label, decoder_correct, future_evidence_summary, oracle_costs.	Evaluation-only; never policy input at test time

Layer 6 should not run until input validation passes. Every policy context must have dataset, subject, session/trial/window, split, seed, and configuration identifiers. Policy features must be finite or explicitly marked missing. Target labels must be absent from evaluation-time policy context except for offline scoring. Cost and reward configuration must be loaded from versioned files. Train/validation/test boundaries must be preserved. Future-window information must not be used as an input feature unless the policy action explicitly waits or requests evidence. Simulated stress variables must be tagged as simulated, not real physiological labels. Embodiment state is optional and must not be required for non-embodied policy evaluation. Layer 9 safety-event, reward-trace, endpoint-error, and demo-manifest identifiers may be used only in

declared simulation or post-hoc evaluation routes; future embodiment outcomes must not be exposed to a frozen decision-time policy.

12.7 Decision-time legality, leakage-control, and record partition

Layer 6 must separate **decision-time context records**, **target-construction files**, and **evaluation-only files**. A policy may be trained from correctness-derived pseudo-labels or simulated rewards, but the validation/test `ReadinessContextRecord` must contain only information available at the decision time. Future-window information is allowed only after the policy explicitly selects **wait** or **request_evidence** and the Layer 7 replay rule reveals the next window. True labels, decoder correctness, future rewards, future embodiment outcomes, and oracle costs must never appear as ordinary decision-time features.

Feature or signal family	Decision-time allowed	Training/evaluation use	Rule
Raw prediction and calibrated uncertainty	Yes	Yes	Must come from split-safe prediction, calibration, uncertainty artifacts, and registered threshold references where a threshold or operating point is used, with fixed class order and configuration hash.
IHARQ reason codes and request counts	Yes	Yes	Only current or past IHARQ decisions may be visible; future IHARQ outcomes are evaluation-only unless revealed by a wait/request action.
RegimeRisk temporal summaries	Yes	Yes	Must be computed without future trial labels, future rewards, or hindsight regime labels; any linked <code>TrustStateRecord</code> must share the same causal cutoff and source <code>RegimeStateRecord</code> lineage.
Fallback availability and request budget	Yes	Yes	Must be known from the current run configuration and action history, not from future outcome quality.

Stress profile identifier and stress intensity	Yes in declared stress experiments	Yes	Must be tagged as simulated stress. It cannot be interpreted as measured real patient state.
Embodiment state before action	Optional in embodiment mode	Yes	Only pre-action simulator state and declared adapter state may be used. Endpoint outcomes after the action are reward/evaluation fields.
True label and decoder correctness	No	Yes	May create reliability targets, pseudo-labels, oracle-cost tables, and evaluation metrics only.
Future-window evidence	No, unless requested/waited	Yes	May be used for target construction or oracle analysis; may enter context only after a declared replay transition reveals it.
Reward, next state, endpoint error, or safety proxy outcome	No before action	Yes	Used to score or update bandit/RL policies in simulation mode; hidden from fixed-policy test decisions.
Dashboard filters and manual inspection notes	No	Reporting only	Must not change policy outputs for an already completed run unless a new run ID/configuration hash is created.

Action legality is enforced before policy optimization and before reward maximization. The legality mask must encode at least unsafe override status, stop-loss status, fallback availability, request-budget availability, replay-window availability, embodiment-mode availability, and simulation-only restrictions. A learned policy may rank illegal actions internally for diagnostic analysis, but the emitted `PolicyActionRecord` must either choose a legal action or declare that no legal action other than **no_action** or **unsafe** exists.

12.8 Output contract

Layer 6 outputs must be saved in forms that downstream layers can consume and thesis readers can inspect. Outputs are not only final predictions; they are records of policy reasoning, action legality, expected cost, routing, exploration, reward, and limitations.

Output artifact	Consumer	Purpose
-----------------	----------	---------

PolicyActionRecord	Layers 7, 8, 9, and 10	Logs selected policy action, optional source Layer 4 IHARQ action token, action-origin or alias-map reference, policy family, action probability, registered selected threshold where applicable, expected cost, reward estimate, exploration flag, reason code, context hash, and claim boundary.
DeferralRecord	Layers 7, 9, and 10	Logs whether the policy predicts, defers, waits, requests evidence, routes to fallback, enters no-action safe mode, or asks a simulated supervisor proxy.
EvidenceQualityRecord	Layers 4, 6, 7, and 10	Logs evidence-quality score, predicted failure probability, reliability target, registered threshold candidates where applicable, and explanation summaries.
Learned policy artifact	Experiment runner and future reruns	Saves model weights, fitted sklearn objects, serialized policies, calibration objects, action-space definitions, and feature preprocessors.
PolicyUpdateTrace	Layers 7 and 10	Logs simulation-authorized bandit/RL update events: context, action, reward, next state, update step, exploration mode, frozen/evaluation flag, and simulation-only boundary.
Reward/regret curve	Thesis, Layer 10, and ablation chapter	Shows online adaptation, sequential policy behavior, or simulation reward behavior under declared rewards.
Policy ablation table	Thesis and dashboard	Shows contribution of context groups, policy stages, cost settings, algorithm families, global ablation IDs, matched-comparison keys, and negative-result notes.
Controller card	Layer 10 and thesis appendix	Documents policy identity, inputs, action space, target/reward route, training protocol, metrics, failure cases, negative-result notes, lineage, limitations, and claim boundary.
Policy evaluation summary	Evaluation scripts, Layer 10, and thesis tables	Stores ablation-step ID, matched-comparison key, metric dictionary ID, policy-stage matrix pointer, cost/reward sensitivity pointer, dashboard export ID, and claim-evidence pointer.

12.8.1 Canonical PolicyActionRecord schema

Field	Meaning
policy_id	Unique policy name and version.
policy_family	Evidence estimator, supervised Adaptive-IHARQ, deferral policy, contextual bandit, RL, or embodiment-aware extension.
action	Selected Layer 6 policy action. It may be one of the preserved Layer 4-compatible tokens <code>accept</code> , <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , <code>fallback</code> , <code>uncertain</code> , or <code>unsafe</code> , or a Layer 6 policy-only token such as <code>defer</code> or <code>no_action</code> when declared by the action-space version.
source_iharq_action	Optional preserved Layer 4 source action token when the policy action is derived from <code>IHARQDecisionRecord</code> ; must not contain display labels or policy aliases.
action_origin	Origin of the selected action, such as <code>layer4_iharq</code> , <code>layer6_policy</code> , <code>deferral_route</code> , <code>bandit_arm</code> , or <code>rl_action</code> .
action_alias_map_id	Versioned map that links display labels, hyphenated prose forms, simulator command aliases, or policy-only tokens to the stored action fields.
context_id	Stable identifier of the source <code>ReadinessContextRecord</code> .
source_evidence_ids	Prediction, calibration, uncertainty, threshold registry, <code>IHARQ</code> , <code>RegimeRisk</code> , optional <code>TrustStateRecord</code> , stress, and optional embodiment record identifiers used to build the context.
action_space_version	Version of the action vocabulary and legality-mask definition used by the policy.
cost_model_id and reward_model_id	Versioned identifiers for the cost/reward assumptions used for expected cost, reward, pseudo-labels, or simulation updates.
global_ablation_id and local_ablation_id	Evaluation identifiers linking the policy output to the Ablation and Evaluation Architecture A0–A13 ladder and the Layer 6 local ladder.
matched_comparison_key	Identifier proving that the policy was compared under matched data, split, seed, cost/reward, stress, and embodiment conditions where required.
metric_dictionary_id	Identifier for the declared metric definitions used to score the policy.
selected_threshold	Threshold selected if the policy chooses or uses an operating point; must be traceable through the source <code>ThresholdRegistryRecord</code> or marked not applicable.
action_probability	Probability assigned to the selected action when the policy is probabilistic.
expected_cost	Predicted cost under the declared cost model.
expected_reward	Expected reward where reward modeling is used.

<code>exploration_flag</code>	Whether the selected action was exploratory in bandit/RL mode.
<code>reason_code</code>	Main reason for the action, compatible with Layer 4 and Layer 10 reason-code registries.
<code>legality_mask</code>	Actions legal at the decision point and actions blocked by safety, budget, fallback, or simulation constraints.
<code>context_hash</code>	Hash of the source <code>ReadinessContextRecord</code> .
<code>config_hash</code>	Hash of policy, cost, reward, action-space, and threshold configuration.
<code>training_stage</code>	Core, supervised, deferral, bandit, RL, or stretch.
<code>simulation_only</code>	True for bandit/RL and embodiment-aware experiments unless a real online dataset exists.
<code>claim_boundary</code>	Example values: offline EEG, simulated closed loop, simulated embodiment, dashboard-only, oracle analysis.

12.8.2 Canonical DeferralRecord schema

Field	Meaning
<code>defer_decision</code>	Whether the policy routes away from direct acceptance.
<code>deferral_target</code>	Fallback decoder, safer model family, request evidence, no-action safe controller, safe hold, or simulated supervisor proxy.
<code>deferral_reason</code>	Low confidence, high entropy, unstable evidence, high RegimeRisk, stop-loss warning, stress condition, unsafe IHARQ flag, or fallback advantage.
<code>fallback_available</code>	Whether the requested fallback route exists for the current context.
<code>fallback_prediction</code>	Prediction or conservative action from the fallback source where applicable.
<code>fallback_expected_cost</code>	Expected cost of the deferral route.
<code>direct_expected_cost</code>	Expected cost of direct acceptance.
<code>matched_coverage_group</code>	Evaluation group used to compare deferral with thresholds and Adaptive-IHARQ fairly.
<code>simulation_only</code>	True if the deferral target is a simulated supervisor proxy or embodiment route.

12.8.3 Canonical PolicyUpdateTrace fields

`PolicyUpdateTrace` is required only for contextual-bandit and RL experiments that update a policy inside declared simulation. It must include `policy_id`, `policy_family`, `context_id`, `state_summary`, `action`, `reward`, `next_state_id`, `update_step`, `exploration_mode`, `training_or_evaluation_mode`, `frozen_policy_flag`, `simulator_id`, `seed`, `config_hash`, and `simulation_only`. Held-out fixed-policy evaluation must either omit updates or mark them as disabled.

12.9 Dependency logic

Layer 6 has hard, soft, optional, conceptual, and software dependencies. These dependencies must be explicit because policy-learning results are sensitive to missing features, reward definitions, and split leakage.

Dependency type	Dependency	Reason
Hard data dependency	PredictionRecord , split manifest, subject/session/trial/window identifiers, and model IDs.	Layer 6 cannot construct policy context or evaluate actions without decoder evidence and leakage-safe identity fields.
Hard metric dependency	CalibrationRecord and UncertaintyRecord .	Adaptive readiness must be reliability-aware rather than raw-score-only.
Hard evidence dependency	IHARQDecisionRecord and Layer 4 reason codes.	Required for IHARQ-aware policies, rule-based baselines, pseudo-decisions, fallback status, and unsafe flags.
Hard policy dependency	Versioned action-space, legality-rule, cost-model, and reward-model configurations.	Without these, actions and rewards are not reproducible.
Hard staging dependency	P0/6.0 IHARQ-lite, P1/6.1 evidence-quality estimation, P2/6.2 supervised Adaptive-IHARQ, P3/6.3 learning-to-defer, P4/6.4 contextual bandit, and P5/6.5 RL readiness policy.	Prevents the thesis from jumping directly to black-box adaptation and makes every added policy family comparable to simpler baselines.
Soft temporal dependency	RegimeStateRecord and conditional TrustStateRecord rows.	RegimeStateRecord is required for RegimeRisk-aware policies; TrustStateRecord is required only when independent trust-timeline features are declared. Either can be mocked or omitted only in a named ablation when Layer 5 or trust-timeline export is unavailable.
Soft simulation dependency	TransitionRecord , RewardTraceRecord , and optional SafetyEventRecord from Layer 7.	Required for contextual bandit and RL, but not for offline evidence-quality or supervised policy training.
Optional stress dependency	StressProfileRecord .	Required for StressForge evaluation but not for clean offline policy evaluation.
Optional embodiment dependency	EmbodimentStateRecord , EmbodimentOutcomeRecord , and Layer9RewardTrace where enabled.	Required only for declared simulation-only embodiment-aware policy extensions.

Conceptual dependency	Public EEG labels are task labels, not readiness labels.	Forces transparent target-construction and modest claims.
Software dependency	Stable feature pipeline, model serialization, configuration hashing, seed control, and record validation scripts.	Required for reproducibility and reruns.
Mockable dependency	RegimeRisk fields, linked trust-state fields, fallback sources, stress variables, and simulated supervisor proxy.	Can be mocked during development if clearly marked as development or ablation mode.

The minimum upstream outputs required before Layer 6 can run are: fixed split IDs, `PredictionRecord`, calibrated/uncertainty records or declared raw-score baseline features, IHARQ decision records for IHARQ-aware comparisons, and cost/action-space configuration. Bandit and RL cannot be defended until Layer 7 `TransitionRecord` and `RewardTraceRecord` files exist, with `SafetyEventRecord` and `EnvironmentDiagnosticRecord` links present whenever the run claims safety-event or diagnostic evidence. Embodiment-aware policy cannot be defended until Layer 9 outputs exist.

12.10 Policy action space

The policy action space should preserve the Layer 4 IHARQ action vocabulary and extend it only when the policy experiment requires explicit routing. The Layer 4-compatible stored tokens are `accept`, `wait`, `request_evidence`, `combine_evidence`, `fallback`, `uncertain`, and `unsafe`. Layer 6 policy-only tokens such as `defer` and `no_action` are valid only when an action-space version and alias map declare them as policy routes rather than Layer 4 source actions. The required action vocabulary is:

Action	Meaning	Typical use
<code>accept</code>	Use the current predicted intent.	Evidence is strong, stable, calibrated, and regime risk is acceptable.
<code>wait</code>	Delay without selecting a new evidence source.	Evidence is incomplete but not unsafe, and latency budget allows waiting.
<code>request_evidence</code>	Acquire, reveal, or evaluate another evidence window.	Current evidence may become reliable with more information.
<code>combine_evidence</code>	Fuse current and previous evidence while preserving the Layer 4 canonical combine-evidence token.	Multiple weak windows may become stronger together.

defer	Layer 6 policy-only route to another decision source.	Fallback, safer model, supervisor proxy, no-action route, or request path has lower expected cost; must be linked to a deferral target rather than treated as a Layer 4 IHARQ token.
fallback	Use a conservative baseline or safe controller.	Main policy is not trusted but a safer alternative exists.
no_action	Layer 6 policy-only safe hold or no-action route.	Action is not justified, rest/no-action is appropriate, or no safe route exists; downstream embodiment may map this to hold/no-op command status.
unsafe	Block action and log a safety event.	Evidence is contradictory, RegimeRisk is high, stop-loss is active, or stress condition makes acting invalid.

The action space should be staged rather than introduced all at once:

Stage	Action space	Purpose
S0	<code>accept</code> versus <code>reject</code> .	Sanity baseline only.
S1	<code>accept</code> , <code>wait</code> , <code>request_evidence</code> .	Minimal IHARQ-style policy.
S2	<code>accept</code> , <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , <code>fallback</code> .	Practical Adaptive-IHARQ with Layer 4-compatible stored tokens.
S3	<code>accept</code> , <code>request_evidence</code> , <code>defer</code> , <code>fallback</code> , <code>no_action</code> , <code>unsafe</code> .	Deferral-aware readiness; <code>defer</code> and <code>no_action</code> are policy-only routes with explicit mapping back to source readiness context.
S4	S3 plus threshold/action mode selection.	Contextual bandit adaptation.
S5	Full action space with sequential state.	RL stretch policy.

The policy-learning escalation ladder is separate from the raw action-space ladder. It defines which policy family is allowed to choose from the action space and which claim boundary applies.

Stage	Layer 6 owner	Role	Claim status
P0 / 6.0	Rule-based IHARQ-lite reference policy	Interpretable readiness baseline that applies declared thresholds, stability checks, RegimeRisk warnings, fallback checks, request budgets, and unsafe overrides.	Core baseline

P1 / 6.1	Evidence-quality estimator	Predicts reliability or failure probability from uncertainty, agreement, stability, signal-quality flags where available, and RegimeRisk features.	Core/support
P2 / 6.2	Supervised Adaptive-IHARQ	Learns multi-action readiness decisions from declared cost-based pseudo-labels.	Core
P3 / 6.3	Learning-to-defer	Learns when to route to fallback, safer model, request path, no-action safe mode, or simulated supervisor proxy.	Core base-line/support
P4 / 6.4	Contextual bandit policy	Adapts action mode, threshold, decoder trust, or deferral route under immediate simulated rewards.	Stretch/simulation-only
P5 / 6.5	Reinforcement-learning readiness policy	Learns sequential readiness behavior under delayed consequences in Layer 7 simulation.	Stretch/simulation-only

Action legality rules must be enforced before policy optimization. If Layer 4 or Layer 5 declares unsafe or active stop-loss status, **accept** is illegal except in a named ablation. If Layer 4 emits **uncertain**, Layer 6 may map that state to **request_evidence**, **fallback**, **defer**, or **no_action** only through the declared legality mask and alias map. If fallback is unavailable, **defer** to fallback is illegal. If the evidence-request budget is exhausted, **request_evidence** is illegal. If no future window exists in offline replay, evidence requests beyond available data are illegal. If embodiment is not active, embodiment-routing actions are illegal. During test-time evaluation, oracle actions based on true labels are always illegal.

12.11 Policy context

The policy context is more than a feature vector. It is the formal description of what the policy is allowed to know at decision time. Context levels should be tracked so ablations can prove which information matters.

Context level	Contents	Used by
Ctx-0	Decoder-only context: predicted class, probability vector, raw confidence.	Threshold baselines and accept-all variants.
Ctx-1	Calibration context: calibrated probability, entropy, margin, reliability bin, selective flags.	Selective prediction and evidence-quality estimator.
Ctx-2	Evidence context: stability, agreement, evidence request count, IHARQ reason code, fallback flag, unsafe flag.	Adaptive-IHARQ.

Ctx-3	Regime context: RegimeRisk score, regime label, trust scores, drift/fatigue/dropout risk, optional <code>trust_state_record_ids</code> , per-target <code>TrustStateRecord</code> timeline features when declared, and stop-loss warning.	Regime-aware and trust-timeline-aware policy.
Ctx-4	History context: previous actions, recent requests, recent confidence trend, recent accepted/fallback/unsafe rates.	Bandit and RL.
Ctx-5	Stress context: stress type, intensity, delay, dropout, noisy intent, drift, weakness, combined stress.	StressForge policy evaluation.
Ctx-6	Embodiment context: declared pre-action simulator state, current command status, previous endpoint-error or safety-proxy summaries when causally available, and prior simulated movement response.	Embodiment-aware stretch policies; future endpoint outcomes remain reward/evaluation fields.

Layer 6 should train and evaluate context ablations: decoder only; decoder plus calibration; decoder plus calibration plus evidence stability; decoder plus calibration plus evidence plus RegimeRisk; full context with history; full context with stress; and full context with embodiment state. These ablations make the layer defensible because they show whether RegimeRisk, IHARQ state, history, stress, and embodiment signals add value beyond simple confidence.

12.12 Label derivation and target construction

Public EEG labels indicate the intended task class. They do not directly say whether the system should accept, wait, request evidence, defer, fallback, enter no-action, or declare unsafe. Layer 6 therefore uses four target-construction routes. Each route must produce a target card that records the source labels, allowed future information, action costs, leakage guard, and claim boundary.

Route	How the target is created	Use and limitation
Decoder-correctness target	If the baseline prediction matches the true task label, the evidence is marked reliable; otherwise unreliable.	Trains the evidence-quality estimator. Correctness is not the same as clinical readiness and must not be exposed as a test-time feature.
Cost-based oracle pseudo-label	For each decision point, compute the declared cost of legal actions using current prediction, true label, future-window evidence when allowed for target construction, fallback availability, delay, request cost, wrong-accept cost, unsafe cost, and optional stress/embodiment terms. The lowest-cost action becomes the pseudo-label.	Trains supervised Adaptive-IHARQ. This is an oracle-style computational label, not a human readiness label. Cost weights must be reported and sensitivity-tested.

Deferral target	Compare direct acceptance with fallback, safer model family, request-more-evidence route, no-action safe mode, or simulated supervisor proxy. Label deferral when routing has lower expected cost than direct acceptance.	Trains learning-to-defer and provides a strong baseline. Abstention refuses to predict; deferral routes to another action path.
Simulation reward	Layer 7 and optional Layer 9 provide reward terms for correct accepted action, wrong accepted action, waiting delay, evidence request burden, fallback burden, unsafe event, task success, late assist, over-assist, under-assist, and endpoint error.	Trains contextual bandit and RL policies. Reward design can bias results and must remain simulation-bounded.

Example cost-based pseudo-label logic is: if the current prediction is correct and confidence/stability are high, label **accept**; if current evidence is weak but future evidence becomes reliable with acceptable delay, label **request_evidence**; if fallback is more reliable than the current model under the current regime, label **defer** or **fallback**; if evidence is contradictory and RegimeRisk is high, label **unsafe** or **no_action**. The implementation must save both the final label and the per-action cost vector so reviewers can inspect why a label was assigned.

12.13 Cost, reward, and utility framework

Layer 6 is a policy layer; therefore cost and reward definitions are central. Every policy experiment must load a versioned cost/reward file and save a cost/reward card.

A transparent base reward model should be declared before any bandit/RL or reward-based pseudo-label experiment. A default simulation reward can be written as:

$$R_t = r_{\text{correct}} \mathbb{I}[\text{correct accept}] - c_{\text{wrong}} \mathbb{I}[\text{wrong accept}] - c_{\text{wait}} n_{\text{wait}} - c_{\text{request}} n_{\text{request}} - c_{\text{fallback}} \mathbb{I}[\text{fallback}] - c_{\text{unsafe}} \mathbb{I}[\text{unsafe}] \quad (1)$$

The optional embodiment term $c_{\text{emb}} e_{\text{emb}}$ may represent endpoint error, command failure, over-assist/under-assist, or another simulated embodiment penalty. It must be omitted when embodiment is inactive. This formula is not a clinical utility function; it is a transparent computational reward used for offline and simulated policy comparison.

Action	Cost components	Notes
accept	Wrong-accept penalty or correct-accept reward.	May be class-aware, context-aware, and stress-weighted.
wait	Time delay cost.	Prevents infinite waiting.
request_evidence	Evidence cost plus delay cost.	Must obey maximum request budget.
combine_evidence	Compute/evidence delay cost.	Preserves the Layer 4 canonical token and is useful only if combined evidence improves reliability.

defer	Fallback burden plus opportunity cost.	Target depends on deferral route.
fallback	Conservative-action cost.	Should be lower than wrong accept but not free.
no_action	Missed-opportunity cost.	Can be safe but may reduce task completion.
unsafe	Safety-event penalty.	Should dominate ordinary penalties.

Reward documentation must include the formula, cost weights, penalty hierarchy, stress multipliers, optional embodiment terms, maximum evidence requests, terminal conditions, sensitivity grid, random seed, simulator version, and policy version. Sensitivity analysis should vary wrong-accept penalty, waiting cost, fallback cost, unsafe penalty, request cost, stress multiplier, and optional endpoint-error penalty. A policy that only wins under one arbitrary reward setting should be reported cautiously.

12.14 Submodule structure and responsibilities

Submodule	Purpose	Status
6.0 Rule-based IHARQ-lite reference policy	Provides the interpretable, minimum viable readiness-control baseline and reason-code reference before learned policy components are introduced.	Core baseline
6.1 Evidence-quality estimator	Predict whether current decoded evidence is likely reliable and estimate failure probability from uncertainty, agreement, stability, and RegimeRisk features.	Core
6.2 Supervised Adaptive-IHARQ	Learn multi-action readiness decisions from declared cost-based pseudo-labels.	Core
6.3 Learning-to-defer	Learn when to route away from direct prediction toward fallback, safer model, request route, no-action, or supervisor proxy in simulation.	Core baseline/supporting policy
6.4 Contextual bandit policy	Adapt thresholds, actions, decoder trust, or deferral route in simulation using immediate rewards.	Stretch C simulation component
6.5 Reinforcement-learning readiness policy	Learn sequential readiness actions when consequences are delayed in Layer 7 simulation.	Stretch-only

12.15 Submodule 6.0: Rule-based IHARQ-lite reference policy

Purpose and bottleneck. The rule-based IHARQ-lite reference policy provides the interpretable, minimum viable readiness-control baseline. It is not replaced by learned policies; it remains the main explainable reference policy and the source of reason-code distributions for policy comparison. Decoder confidence and selective prediction alone may not capture evidence stability, agreement, fallback state, unsafe state, request budget, or RegimeRisk warnings. IHARQ-lite turns these signals into a transparent action before learned policies are introduced.

Scientific role. This submodule anchors the policy-learning ladder. It protects the thesis from the criticism that learned Adaptive-IHARQ is compared only to weak thresholds, and it protects the system from black-box policy overreach by preserving a human-auditable baseline.

Engineering responsibility. It applies declared thresholds and rule ordering, emits reason codes, produces accepted-prediction and evidence-request logs, records fallback/unsafe flags, and exports a baseline controller card and dashboard-ready reason-code distributions.

Inputs. Calibrated confidence, entropy, margin, stability/agreement features, IHARQ evidence fields, optional `RegimeStateRecord`, threshold registry, request budget, fallback availability, unsafe flag, stop-loss status, and action-cost configuration.

Outputs. `IHARQDecisionRecord` where owned by Layer 4, Layer 6 baseline `PolicyActionRecord` views where used for policy comparison, reason codes, accepted-prediction logs, evidence-request logs, fallback/unsafe flags, baseline controller card, and dashboard-ready reason-code distributions.

Internal processing logic. Apply threshold and rule checks in a declared order: unsafe/stop-loss override; invalid or missing evidence check; confidence and uncertainty check; stability/agreement check; RegimeRisk check; request-budget check; fallback availability check; and final `accept`, `wait`, `request_evidence`, `combine_evidence`, `fallback`, or policy-only `no_action` decision. The exact ordering, thresholds, reason-code registry, and Layer 4-to-policy action map must be saved as configuration.

Consumers and connections. Layer 6.1 can use IHARQ-lite outcomes and reason codes as features or comparison groups. Layers 6.2–6.5 use it as a baseline. Layer 10 renders its reason-code distributions and controller card. Layer 4 remains the canonical owner of IHARQ evidence verification; Layer 6.0 is the policy-comparison view of that interpretable logic.

Evaluation. Compare IHARQ-lite against accept-all decoding, calibrated confidence thresholding, entropy/margin rejection, selective prediction, longer-window decoding, multi-window voting, evidence-quality thresholding, supervised Adaptive-IHARQ, and learning-to-defer. Required checks include matched coverage, matched cost, action distribution, wrong-accept rate, request burden, fallback burden, unsafe override compliance, and stress-conditioned behavior.

Failure modes and limitations. IHARQ-lite may be brittle across subjects, sessions, decoders, regimes, and stress conditions. It may over-request evidence or under-accept useful predictions. These limitations motivate learned policy alternatives, but learned alternatives must still beat this baseline under fair comparison.

Reproducibility and thesis evidence. Save threshold registry entries, rule ordering, legality masks, reason-code registry, request budgets, fallback registry, action-cost file, policy-action exports, reason-code distributions, and baseline controller card.

12.16 Submodule 6.1: Evidence-quality estimator

Purpose and bottleneck. The evidence-quality estimator predicts whether the current decoded evidence is likely reliable. It addresses the bottleneck that calibrated confidence and entropy alone may not identify high-confidence wrong predictions, unstable windows, model disagreement, channel instability, or RegimeRisk-related deterioration. It is the simplest learned component of Layer 6 and should be implemented before multi-action policy learning.

Scientific role. This submodule bridges classic uncertainty analysis and readiness policy learning. It tests whether evidence reliability can be estimated from calibration, uncertainty, agreement, stability, signal-quality, and regime features.

Engineering responsibility. It builds a feature table, trains a reliability estimator, calibrates the estimator if needed, saves `EvidenceQualityRecord` outputs, and exports reliability-vs-coverage curves and explanation reports.

Inputs. Calibration features such as calibrated confidence and reliability-bin membership; uncertainty features such as entropy, margin, variation ratio, and probability gap; agreement features such as model-family agreement, ensemble disagreement, and channel-subset agreement; stability features such as time-window consistency and previous-window agreement; RegimeRisk features such as regime label, trust score, risk trend, and stop-loss warning; and signal-quality features such as artifact flags, channel masks, and dropout indicators.

Targets. The primary target is `reliable=1` if the decoder prediction equals the true task label and `reliable=0` otherwise. Optional targets include high-confidence wrong prediction, stable-correct versus stable-wrong category, wrong-accept risk, and failure probability under declared stress.

Models. Start with interpretable models: logistic regression, calibrated random forest, gradient-boosted trees, and small MLP only if simpler models are insufficient. Monotonic or constrained models are preferred where feasible.

Outputs. Evidence-quality score, predicted failure probability, calibrated reliability score, threshold candidates, feature importance or explanation report, reliability-vs-coverage curve, high-confidence wrong detection table, and `EvidenceQualityRecord`.

Consumers and connections. Layer 4 can use evidence-quality thresholds in named ablations; Layer 6 uses the score as policy context; Layer 7 uses the score as state; Layer 10 renders evidence-quality views and reliability cards. Feedback to Layer 3 is limited to diagnostic warnings about calibration drift or high-confidence failure clusters; it must not retune test thresholds silently.

Evaluation. Evaluate AUROC/AUPRC, calibration of the quality score, reliability-vs-coverage, accepted-trial accuracy under quality thresholds, high-confidence wrong detection, subject-wise generalization, dataset-wise transfer, regime-wise degradation, and stress-conditioned behavior.

Failure modes and limitations. The estimator may learn dataset-specific artifacts, under-detect rare failures, overfit subject identity, or treat decoder correctness as equivalent to readiness. It must be reported as an evidence reliability estimator, not as proof of clinical safety.

Reproducibility and thesis evidence. Save feature lists, target-generation files, estimator configuration, seeds, split IDs, model artifacts, prediction files, calibration plots, reliability-vs-coverage figures, feature-importance reports, and an evidence-quality card. Thesis outputs should include reliability-vs-coverage plots, high-confidence wrong tables, and a short limitation statement about decoder-correctness targets.

12.17 Submodule 6.2: Supervised Adaptive-IHARQ

Purpose and bottleneck. Supervised Adaptive-IHARQ learns multi-action readiness decisions from declared cost-based pseudo-labels. It addresses the bottleneck that rule-based IHARQ can be interpretable but brittle and may not capture nonlinear interactions among calibrated uncertainty, evidence stability, fallback availability, RegimeRisk, action history, and stress context.

Scientific role. This submodule tests whether learned readiness actions improve wrong-accept avoidance, evidence efficiency, and policy utility over rule-based IHARQ-lite, static thresholds, selective prediction, and deferral baselines.

Engineering responsibility. It constructs pseudo-label files, trains a multi-action policy, applies action-legality masks, calibrates action probabilities where possible, logs `PolicyActionRecord` outputs, and exports matched-cost and matched-coverage comparisons.

Inputs. `ReadinessContextRecord`, `EvidenceQualityRecord`, `IHARQDecisionRecord`, `RegimeStateRecord`, optional `StressProfileRecord`, cost-based pseudo-labels, action-legality masks, threshold registry entries, and train/validation/test split manifests.

Action set. The core action set is `accept`, `wait`, `request_evidence`, `combine_evidence`, `defer`, `fallback`, `no_action`, and `unsafe`. The Layer 4-compatible actions `accept`, `wait`, `request_evidence`, `combine_evidence`, `fallback`, and `unsafe` must preserve their canonical token identity, while `defer` and `no_action` are policy-only routes. If an `uncertain` source state from Layer 4 is used by this submodule, it must be preserved in `source_iharq_action` and mapped through the declared legality mask rather than treated as a new learned alias. If data is limited, reduce to `accept`, `request_evidence`, `fallback`, and `unsafe`; the reduced action space must be logged.

Models. Start with multinomial logistic regression, calibrated gradient-boosted classifier, shallow MLP, cost-sensitive classifier, or hierarchical classifier. A useful hierarchy is `safe/unsafe` first, then `accept` versus route away, then `request/defer/fallback/no-action`.

Training protocol. The protocol must define subject/session separation, within-subject versus cross-subject conditions, calibration budget, label budget, pseudo-label generation file, cost configuration file, feature normalization, model calibration method, action class imbalance handling, random seeds, and validation-based model selection.

Outputs. Learned policy artifact, action probabilities, expected costs, reason codes, `PolicyActionRecord`, action-distribution table, matched-cost comparison table, context-ablation table, failure-case log, and Adaptive-IHARQ controller card.

Consumers and connections. Layer 7 consumes policy actions for simulation. Layer 8 consumes stress-conditioned policy outputs. Layer 9 may consume only simulation-compatible gating records. Layer 10 consumes action distributions, cost curves, ablation tables, and controller cards. Layer 4 remains the interpretable baseline and should not be overwritten.

Evaluation. Required questions are: does the learned policy reduce wrong accepts; does it preserve useful coverage; does it reduce unnecessary waiting; does it improve over IHARQ-lite at matched coverage and matched cost; does RegimeRisk context help; does it generalize across subjects/datasets; and does it survive stress conditions. Required baselines are `accept-all` decoder, confidence threshold, entropy rejection, margin rejection, selective prediction, longer-window decoding, multi-window voting, probability averaging, ordinary ensemble, IHARQ-lite, evidence-quality threshold, and learning-to-defer.

Failure modes and limitations. It can overfit pseudo-label rules, collapse to fallback or

wait, over-accept wrong predictions, inherit arbitrary cost weights, or appear stronger because of unfair coverage. It must report coverage, cost, action distribution, and sensitivity analysis. It must not claim that pseudo-labels are real readiness labels.

Reproducibility and thesis evidence. Save pseudo-label tables, per-action cost vectors, cost-model card, policy configuration, feature schema, seeds, model artifacts, action records, matched comparison tables, stress-conditioned curves, and controller card. Thesis evidence should include action-distribution figures, wrong-accept versus coverage tables, matched-cost comparisons, and context-ablation results.

12.18 Submodule 6.3: Learning-to-defer

Purpose and bottleneck. Learning-to-defer decides when the system should route a case away from direct prediction toward another decision source. It addresses the bottleneck that abstention alone says “do not predict” but does not specify the safer or more useful route.

Scientific role. This submodule is both a policy component and a criticism-resistant baseline. Reviewers may ask whether Adaptive-IHARQ is necessary or whether the system only needs to learn when to defer. Including learning-to-defer makes that comparison explicit. If Adaptive-IHARQ beats deferral at matched cost, its complexity is justified. If deferral performs similarly, the thesis can report that simpler routing is sufficient in some regimes.

Engineering responsibility. It defines deferral targets, trains a deferral classifier or router, computes direct-versus-defer expected costs, logs `DeferralRecord`, and compares deferral with Adaptive-IHARQ, thresholds, and IHARQ-lite.

Inputs. Calibrated confidence, entropy, margin, evidence-quality score, IHARQ action and reason code, RegimeRisk score, model trust scores, fallback availability, fallback expected cost, direct expected cost, stress state, action history, and optional supervisor-proxy availability in simulation.

Deferral targets. Valid targets are conservative fallback decoder, safer model family with higher current trust, request-more-evidence route, no-action safe controller, safe hold, and simulated supervisor proxy. The supervisor proxy is simulation-only and must be marked as oracle/proxy.

Outputs. `DeferralRecord`, deferral probability, target source, direct-versus-defer cost comparison, deferral burden curve, accepted accuracy after deferral, deferred-case accuracy, and matched-cost comparison with Adaptive-IHARQ.

Consumers and connections. Layer 7 applies deferral routes in simulation. Layer 9 receives only hold/no-action/fallback-compatible routes through Layer 7 or policy records. Layer 10 renders deferral burden, target distributions, and matched-cost comparisons. Layer 6.2 uses learning-to-defer as a baseline and possible subpolicy.

Evaluation. Metrics include direct prediction accuracy, deferred-case accuracy, deferral rate, fallback burden, wrong-accept reduction, matched-coverage risk, matched-cost utility, benefit under high RegimeRisk, stress behavior, and failure cases where deferral is overused.

Failure modes and limitations. The policy can over-defer, hide poor accuracy behind low coverage, rely on an unrealistic fallback, or overuse a simulated supervisor proxy. Deferral should not be described as free safety. It carries cost and burden.

Reproducibility and thesis evidence. Save deferral target generation files, fallback registry, direct/defer cost table, deferral policy configuration, seeds, `DeferralRecord` logs, burden curves, matched comparisons, and deferral controller card. Thesis evidence should include deferral-rate

versus wrong-accept plots and a direct comparison with Adaptive-IHARQ.

12.19 Submodule 6.4: Contextual bandit policy

Purpose and bottleneck. The contextual bandit module is the first online/adaptive policy component. It is appropriate when each readiness action produces an immediate or near-immediate reward in the Layer 7 simulated environment. It addresses the bottleneck that the best threshold, action mode, decoder trust choice, or deferral route may change across regimes and stress conditions.

Scientific role. This submodule tests whether simple online adaptation helps under changing context without requiring full sequential RL. It should be implemented before RL and compared against the best fixed policy.

Engineering responsibility. It defines context-action-reward tuples, chooses bandit algorithms, logs exploration, updates action-selection parameters only within training/simulation rules, and saves reward/regret curves.

Context-action-reward tuple. Context equals calibrated uncertainty plus evidence quality plus IHARQ state plus RegimeRisk plus stress state plus previous action summary. Action equals readiness action, threshold mode, decoder trust action, or deferral route. Reward equals correct-action reward minus wrong-accept penalty, delay penalty, fallback penalty, request penalty, and unsafe penalty, with optional embodiment penalty.

Bandit action modes. Valid modes include conservative/balanced/aggressive threshold selection, readiness action selection, decoder-family trust selection, deferral-route selection, and embodiment route selection in simulation-only demo mode.

Algorithms. Start with epsilon-greedy over discrete modes, UCB-style selection, Thompson sampling, or contextual linear bandit. Use lightweight neural bandits only if simpler methods are insufficient and data volume supports them.

Outputs. Bandit policy artifact, exploration logs, action-probability traces, cumulative reward curve, cumulative regret curve against oracle or best-fixed policy, wrong-accept-over-time curve, action-distribution timeline, and bandit controller card.

Consumers and connections. Layer 7 provides rewards and transitions. Layer 8 supplies stress contexts. Layer 9 may provide optional endpoint-error reward terms. Layer 10 renders reward/regret, exploration burden, and policy stability dashboards.

Evaluation. Evaluate cumulative reward, cumulative regret, wrong-accept rate over time, adaptation under regime shifts, adaptation under stress profiles, exploration burden, policy stability across random seeds, and comparison against fixed thresholds, IHARQ-lite, supervised Adaptive-IHARQ, and learning-to-defer.

Failure modes and limitations. The bandit may over-explore unsafe actions, chase reward noise, collapse to conservative fallback, or overfit a stress schedule. It must obey action legality rules and unsafe overrides. It is a simulated online adaptation mechanism, not a clinically deployed adaptive BCI controller.

Reproducibility and thesis evidence. Save bandit configuration, reward formula, random seeds, exploration schedule, train/validation/test simulated sessions, action logs, reward/regret curves, stress-conditioned results, and controller card. Thesis evidence should include reward/regret figures and comparisons to non-bandit baselines.

12.20 Submodule 6.5: Reinforcement-learning readiness policy

Purpose and bottleneck. The RL readiness policy is the most stretched Layer 6 component. It is justified only when readiness consequences are delayed: repeated waiting can reduce wrong actions but increase late assistance; repeated evidence requests can improve reliability but increase delay; repeated wrong accepts can worsen simulated task outcome; aggressive actions under stress can cause unsafe events; excessive fallback can reduce task completion.

Scientific role. RL tests whether sequential policy learning adds value beyond supervised readiness policies and contextual bandits inside a declared simulation. It is not required for the core thesis claim and should remain stretch-only unless results are stable and well-bounded.

Engineering responsibility. It defines the RL state, action space, reward hierarchy, transition interface, training/evaluation split, baseline policies, unsafe penalties, and controller card. It runs only after Layer 7 simulation and reward logging are stable.

State. RL state should include `ReadinessContextRecord`, recent action history, evidence-request count, `RegimeRisk` state, stress profile state, fatigue-like or drift-like simulated variables, optional embodiment state, previous reward, and outcome summary.

Action space. Start small: `accept`, `request_evidence`, `fallback`, and `no_action`. Expand only after stable results to the full action set: `accept`, `wait`, `request_evidence`, `combine_evidence`, `defer`, `fallback`, `no_action`, and `unsafe`. `combine_evidence` preserves the Layer 4 token name; `defer` and `no_action` remain policy-only action routes.

Reward design. Correct timely accept receives positive reward. Wrong accept receives a large negative penalty. Unsafe event receives the largest negative penalty. Waiting and request evidence receive small to moderate negative costs. Fallback receives mild to medium negative cost. Late assist and endpoint error receive optional penalties. Task success receives terminal reward.

Algorithms. Start with tabular or discretized Q-learning for a toy environment. Use DQN-style policy only if state dimensionality and data volume support it. Actor-critic and offline RL are future/stretch items and require stronger logging and leakage controls.

Outputs. RL policy artifact, transition logs, reward curves, unsafe-event curves, waiting burden, fallback burden, policy stability summaries, stress-conditioned performance, and RL controller card.

Consumers and connections. Layer 7 is the primary environment. Layer 8 supplies stress conditions. Layer 9 can supply optional embodiment outcomes. Layer 10 reports RL as simulation-only with limitation statements.

Evaluation. Evaluate held-out simulated sessions, stress-conditioned performance, reward sensitivity, unsafe event count, wrong-accept rate, waiting/request/fallback burden, task success, regret or comparison against best-fixed policy where possible, and baseline comparisons against thresholds, IHARQ-lite, supervised Adaptive-IHARQ, learning-to-defer, and contextual bandit.

Failure modes and limitations. RL may overfit simulator quirks, exploit reward loopholes, produce unstable action distributions, or appear impressive only under arbitrary reward settings. It must be presented as simulation-only feasibility evidence. It must not be described as real patient-safe control.

Reproducibility and thesis evidence. Save environment configuration, transition schema, reward model, seeds, algorithm hyperparameters, training curves, validation curves, held-out results, stress results, reward sensitivity tables, failure-case logs, and controller card. Thesis evidence should place RL in a stretch or appendix section unless it becomes robust enough for

the main evaluation chapter.

12.21 Feedforward and loopback contracts

Layer 6 has a feedforward path and several bounded feedback signals. Feedback loops are allowed only when they have a scientific and engineering reason, a trigger, an action, a safety constraint, and evaluation logic.

The normal feedforward path is: Layers 2–3 produce prediction, calibration, and uncertainty records; Layer 4 produces IHARQ decisions and evidence-quality features; Layer 5 produces `RegimeStateRecord` context and, when independently consumed, linked `TrustStateRecord` trust timelines; Layer 6 builds `ReadinessContextRecord`; Layer 6 emits `PolicyActionRecord` and `DeferralRecord`; Layer 7 applies actions in simulation and returns `TransitionRecord`, `RewardTraceRecord`, `EpisodeSummaryRecord`, `PolicyRolloutRecord`, `EnvironmentDiagnosticRecord`, `SafetyEventRecord` where materialized, and optional `PolicyUpdateTrace`; Layer 8 conditions replay with stress profiles; Layer 9 optionally returns embodiment outcomes; Layer 10 renders policy dashboards, controller cards, ablation tables, and limitation statements.

Source	Target	Signal and trigger	Action	Safety and evaluation rule
Layer 6	Layer 4	Diagnostic threshold/cost warning when learned policy repeatedly disagrees with IHARQ in high-confidence/high-risk cases.	Run a named ablation or diagnostic review of IHARQ thresholds and reason codes.	Must not silently retune Layer 4 on test data; report as diagnostic or validation-only feedback.
Layer 6	Layer 5	Policy outcome summary when repeated fallback, unsafe, or wrong-accept events cluster temporally in simulation.	Layer 5 may update simulated trust timeline or produce a regime-diagnostic warning.	Must distinguish decoder instability from policy-induced instability; no clinical fatigue claim.
Layer 6	Layer 7	Readiness action, expected cost, exploration flag, and action legality mask.	Environment applies action and returns transition/reward.	Request/wait loops are bounded by budget; unsafe overrides dominate reward optimization.
Layer 7	Layer 6	Reward, next state, safety event, latency, and outcome.	Bandit or RL updates in training/simulation mode.	Held-out evaluation policies must not update unless explicitly in online-evaluation protocol.

Layer 8	Layer 6	Stress type, intensity, and schedule.	Policy is evaluated under clean, mild, moderate, severe, and combined stress.	Stress is simulated and must be reported as such.
Layer 9	Layer 6	Optional endpoint error, command status, embodiment safety proxy, SafetyEventRecord link, and Layer9RewardTrace link where enabled.	Embodiment-aware policy may include optional reward terms, gating evaluation, post-hoc policy diagnostics under declared simulation.	Must remain simulation-only, must not expose future outcomes to frozen policies, and must not claim real-body control.
Layer 6	Layer 10	Policy logs, ablation tables, reward/regret curves, and controller cards.	Dashboard renders inspectable policy behavior.	Dashboard must not modify experimental records without creating a new run and configuration hash.

12.22 Simulation, StressForge, and embodiment integration

Layer 6 operates in three modes.

Mode 1: offline policy evaluation. The policy is trained or evaluated on saved public EEG records. No simulated transition consequences are used except for declared target construction. This mode supports the evidence-quality estimator, supervised Adaptive-IHARQ, learning-to-defer, and static policy ablations.

Mode 2: simulated closed-loop readiness. The policy interacts with Layer 7 repeated sessions. It selects actions and receives transition outcomes. This mode supports contextual bandit, RL readiness policy, reward/regret curves, stress-conditioned policy comparison, and sequential failure analysis.

Mode 3: optional embodiment-aware simulation. The policy action gates or modulates a Layer 9 MyoSuite/OpenSim/static-replay proxy command through the Layer 9 adapter interface. This mode supports endpoint-error reward terms, safety-event penalties, command-status diagnostics, optional **Layer9RewardTrace** files, simulated action-level readiness consequences, and embodiment controller cards. It is stretch-only, simulation-only, and must not treat **Layer9RewardTrace** files as clinical utility.

StressForge-Lite integration should test delay, dropout, noisy intent, weakness, drift, fatigue-like degradation, and combined stress. The policy concern under delay is late action; under dropout it is fallback/request behavior; under noisy intent it is wrong-accept avoidance; under weakness it is optional embodiment reward behavior; under drift it is RegimeRisk-aware adaptation; under fatigue-like degradation it is conservative temporal behavior; and under combined stress it is safety and interpretability under worst-case simulation.

12.23 Evaluation and ablation design

Ablation is central because Layer 6 adds complexity. The thesis must show what the complexity buys and where it does not help. After the Ablation and Evaluation Architecture merge, Layer 6 owns the adaptive-policy slice of the global ablation ladder: *A6* when learned evidence-quality scoring is tested beyond IHARQ rules, *A8* for learning-to-defer, *A9* for supervised Adaptive-IHARQ, *A10* for contextual-bandit adaptation in simulation, and *A11* for reinforcement-learning readiness policy in simulation. The layer may also participate in *A12* stress-conditioned policy comparison and *A13* embodiment-aware policy comparison, but those results remain stress-proxy or simulation-only.

Global step	System variant	Question answered
A0/L6-A0	Accept-all decoder.	What happens with no readiness policy?
A2/L6-A1	Confidence threshold.	Is simple confidence rejection enough?
A3/L6-A2	Entropy/margin rejection.	Are standard uncertainty baselines enough?
A3/L6-A3	Selective prediction.	What is the risk-coverage baseline?
A4/L6-A4	Longer-window decoder.	Is waiting equivalent to using longer windows, and what delay burden is introduced?
A4/L6-A5	Multi-window voting, probability averaging, or ordinary ensemble.	Is evidence accumulation just ordinary ensembling?
A5/L6-A6	IHARQ-lite.	Does interpretable evidence verification help before learned policy logic?
A6/L6-A7	IHARQ plus evidence-quality estimator.	Does learned evidence quality help beyond rules without leakage?
A8/L6-A8	Learning-to-defer.	Is routing to fallback enough?
A9/L6-A9	Supervised Adaptive-IHARQ.	Does learned multi-action readiness improve over rules and deferral?
A10/L6-A10	Contextual bandit.	Does immediate-reward adaptation help in simulation under frozen reward and held-out evaluation?
A11/L6-A11	RL policy.	Does sequential policy learning help under delayed consequences without simulator overfitting?
A12/L6-A12	Stress-conditioned policy comparison.	Which policy survives delay, dropout, drift, noisy intent, weakness, fatigue-like degradation, and combined stress?
A13/L6-A13	Embodiment-aware policy comparison.	Do readiness decisions improve simulated action-level outcomes while preserving simulation-only boundaries?

Required comparison styles are matched coverage, matched wrong-accept rate, matched cost, subject-wise comparison, dataset-wise comparison, calibration-budget comparison, regime-wise

comparison, stress-wise comparison, clean versus stressed comparison, training/adaptation versus frozen-policy evaluation, held-out simulated session comparison, reward-sensitivity comparison, and offline versus simulated comparison.

The evaluation package must also include a policy-stage matrix, context-level ablation matrix, action-legality matrix, cost/reward sensitivity matrix, policy-to-layer dependency matrix, training/evaluation partition matrix, and claim-boundary matrix. These matrices are not decorative; they demonstrate that each policy family consumes declared records, respects the same legality rules, is compared against fair baselines, preserves decision-time leakage boundaries, and stays within its claim boundary.

Layer 6 comparisons must be matched by dataset, subject, session, trial/window identity where feasible, split ID, label-budget ID, model family, seed, calibration ID, threshold registry, IHARQ configuration, RegimeRisk configuration when active, policy training split, target-construction route, action vocabulary, legality rules, cost configuration, reward configuration, stress profile, stress seed, embodiment mode, and dashboard export configuration. A policy result that is not matched on the required fields must be marked diagnostic-only and cannot support the adaptive-readiness claim.

Success criteria should be modest and metric-based. For core Layer 6, success means the evidence-quality estimator detects unreliable evidence better than raw confidence and improves reliability-vs-coverage; supervised Adaptive-IHARQ or learning-to-defer improves wrong-accept/cost trade-offs over simple baselines at matched coverage or matched cost; and all outputs are reproducible and claim-safe. For stretch Layer 6, success means contextual bandit or RL shows stable simulation-only gains under held-out sessions, held-out stress seeds, frozen reward, and reward sensitivity, without unsafe-event inflation.

Negative results are valid. If the evidence-quality estimator helps but Adaptive-IHARQ does not, learned reliability may be useful while multi-action pseudo-labels remain weak. If learning-to-defer matches Adaptive-IHARQ, simpler routing may be sufficient. If contextual bandit helps only under drift/stress, online adaptation is conditionally useful. If RL performs poorly, delayed policy learning may be infeasible or reward design may be too artificial. If RL overfits simulation, keep it as stretch-only limitation evidence. If policy ranking changes under small cost/reward changes, the claim must be downgraded to reward-sensitive. Each such outcome should create a negative-result note linked to the same run ID, global ablation ID, matched-comparison key, and dashboard/card export.

12.24 Metrics

Layer 6 should report readiness metrics, policy metrics, safety-oriented metrics, and simulation/embodiment metrics.

Metric group	Metrics
Readiness metrics	Accepted-trial accuracy, coverage, wrong-accept rate, wait rate, request rate, fallback rate, deferral rate, unsafe rate, evidence cost, decision latency.
Policy metrics	Expected cost, reward, regret, action entropy, policy stability, action distribution, feature reliance, exploration burden, reward sensitivity, cost-weight sensitivity, matched-cost delta, matched-coverage delta, training/adaptation curve, frozen-policy held-out score, and policy-stage comparison.

Safety-oriented metrics	Unsafe event count, high-confidence wrong accepted rate, stop-loss compliance, safety override rate, stress unsafe rate.
Simulation metrics	Task success, late assist, over-assist, under-assist, transition reward, terminal reward, repeated-failure count.
Embodiment metrics	Endpoint error, command smoothness proxy, simulated movement success, embodiment safety proxy, over-assist/under-assist under embodiment.

Every metric must specify denominator, grouping, direction, record fields, and claim role in the metric dictionary. For example, wrong-accept rate should be reported among accepted actions and also with the declared global denominator where relevant, while coverage should be reported among eligible decision points. Deferral rate should be separated from fallback rate. Unsafe declarations should not be mixed with real safety validation. Subject-level and session-level summaries should be reported where possible; pooled-only outputs must be labeled as weakened or exploratory.

12.25 Reproducibility and implementation artifacts

Layer 6 must be one of the most reproducible parts of the thesis because policy results can change with small choices in cost weights, reward formulas, target construction, and action legality.

Required logs include policy configuration, feature list, target-construction route, cost/reward weights, action vocabulary, action legality rules, random seed, train/validation/test split, policy model version, policy action records, deferral records, reward/regret logs, controller cards, failure cases, and dashboard export.

Recommended configuration structure:

```
configs/policies/evidence_quality_estimator.yaml
configs/policies/supervised_adaptive_iharq.yaml
configs/policies/learning_to_defer.yaml
configs/policies/contextual_bandit.yaml
configs/policies/rl_readiness_policy.yaml
configs/policies/action_space.yaml
configs/policies/cost_model.yaml
configs/policies/reward_model.yaml
configs/policies/legality_rules.yaml
configs/policies/matched_comparison.yaml
configs/policies/metric_dictionary.yaml
configs/policies/claim_boundary.yaml
```

Recommended record structure:

```
records/policy_records/evidence_quality/
records/policy_records/supervised_adaptive_iharq/
records/policy_records/deferral/
records/policy_records/contextual_bandit/
```

```

records/policy_records/rl/
records/policy_records/ablations/
records/policy_records/matched_comparisons/
records/policy_records/negative_result_notes/
records/policy_records/claim_evidence/
records/policy_records/controller_cards/

```

Recommended source-code structure:

```

src/adaptive_policy/evidence_quality/
src/adaptive_policy/supervised_adaptive_iharq/
src/adaptive_policy/learning_to_defer/
src/adaptive_policy/contextual_bandit/
src/adaptive_policy/rl_readiness/
src/adaptive_policy/cost_reward/
src/adaptive_policy/action_space/
src/adaptive_policy/policy_logging/
src/adaptive_policy/policy_cards/

```

Repository naming is normalized to the existing Stretch C structure. Earlier dossier-style names such as `policy_learning/` are treated as conceptual aliases for the repository locations already used by this architecture: `configs/policies/`, `records/policy_records/`, and `src/adaptive_policy/`. New implementations should not create a second parallel policy-learning tree unless the repository architecture is intentionally revised in a later propagation pass.

The reproducible command sequence for Layer 6 should build `ReadinessContextRecord` files from upstream artifacts, validate schemas and leakage rules, run IHARQ-lite baseline exports, train the evidence-quality estimator, generate cost-based pseudo-labels, train supervised Adaptive-IHARQ, train/evaluate learning-to-defer, run contextual bandit simulation, run RL simulation only when Layer 7 is stable, generate policy ablations, and export controller cards.

Traceability rule: every `PolicyActionRecord` and `DeferralRecord` must be traceable backward to the exact `ReadinessContextRecord`, cost/reward config, action-space config, trained policy artifact, seed, split, and source records; it must also be traceable forward to transition outcomes, stress outcomes, embodiment outcomes where applicable, dashboard figures, and thesis tables.

12.26 Dashboard, cards, and thesis outputs

Layer 6 should produce inspectable outputs for Layer 10. Required dashboard views include adaptive policy overview, evidence-quality page, Adaptive-IHARQ page, learning-to-defer page, contextual bandit page, RL controller page, policy ablation page, matched-comparison page, cost/reward sensitivity page, negative-result page, and controller-card export page.

Required thesis-visible policy artifacts include:

Matrix or diagram	Purpose
Policy-stage matrix	Maps the pre-6.0 prerequisite check, P0/6.0–P5/6.5 policy ladder, and optional 6.E embodiment-aware extension to inputs, outputs, baselines, metrics, status, and claim boundary.

Context-level ablation matrix	Shows which context groups are used and tested: decoder, calibration, evidence, regime, history, stress, and embodiment.
Action-legality matrix	Shows which actions are legal under unsafe flag, stop-loss, request budget, fallback availability, stress state, replay-window availability, and embodiment mode.
Cost/reward sensitivity matrix	Shows which policy conclusions survive different cost/reward weights.
Policy-to-layer dependency matrix	Maps Layer 6 inputs and outputs to Layers 2–5 and 7–10.
Claim-boundary matrix	Maps policy outputs to allowed claims and forbidden claims.
Runtime sequence diagram	Shows a trial/window moving through context building, legality mask, policy action, simulation transition, and logging.
Feedback-loop diagram	Shows IHARQ/RegimeRisk to policy context, Layer 7 reward to bandit/RL update, and dashboard inspection without test-set mutation.

Every learned policy should have a controller card with policy identity, inputs, action space, legality rules, target or reward construction, training protocol, baselines, global/local ablation IDs, matched-comparison key, metrics, negative-result notes, limitations, failure cases, lineage, and claim boundary. Cards should explicitly state whether the policy is core, supporting, optional, stretch, or future work.

The thesis should place Layer 6 evidence in figures and tables such as reliability-vs-coverage curves, action-distribution histograms, wrong-accept versus coverage tables, matched-cost comparisons, deferral burden curves, context-ablation tables, reward/regret curves, stress-policy curves, and controller cards in the appendix.

12.27 Implementation phases and readiness levels

Layer 6 should follow this chronology:

Phase	Implementation task	Exit condition
Pre-6.0	Prerequisite checks: verify prediction, calibration, uncertainty, threshold-registry, IHARQ, RegimeRisk, conditional trust-state, split, and schema records.	ReadinessContextRecord can be built without hidden variables, and any independent trust-timeline feature links to a valid TrustStateRecord .
P0 / 6.0	Export rule-based IHARQ-lite reference policy and reason-code baseline.	Baseline PolicyActionRecord views, reason-code distributions, action-legality matrix, and baseline controller card exist.

P1 / 6.1	Train evidence-quality estimator using decoder correctness and reliability features.	EvidenceQualityRecord , predicted failure probabilities, and reliability-vs-coverage curves exist.
P2 / 6.2a	Define cost model and generate pseudo-labels for supervised Adaptive-IHARQ.	Pseudo-label files, per-action cost vectors, and cost-model cards exist.
P2 / 6.2b	Train supervised Adaptive-IHARQ and compare to IHARQ-lite and rejection baselines.	PolicyActionRecord and matched-cost/matched-coverage comparisons exist.
P3 / 6.3	Implement learning-to-defer.	DeferralRecord and deferral-vs-Adaptive-IHARQ comparisons exist.
P4 / 6.4	Train or tune contextual bandit after Layer 7 simulation is stable.	Reward/regret curves and non-bandit comparisons exist.
P5 / 6.5	Implement RL only after simulation and reward design are stable.	RL is evaluated on held-out simulated sessions and stress profiles with controller card.
6.E optional extension	Add embodiment-aware policy terms if Layer 9 is active.	Simulated action outcomes and embodiment-aware controller cards exist, while future embodiment outcomes remain excluded from frozen decision-time context.

Layer 6 maturity should be tracked as L6-R0 concept; L6-R1 boundary defined; L6-R2 context contract defined; L6-R3 target routes defined; L6-R4 evidence-quality estimator ready; L6-R5 supervised policy ready; L6-R6 deferral ready; L6-R7 simulation policy ready; L6-R8 RL stretch ready; and L6-R9 thesis-ready. Layer 6 should not be defended as a major adaptive-policy contribution before L6-R5. RL should not be defended as more than stretch evidence before L6-R8.

12.28 Failure modes and defensive design

Failure mode	Risk	Defensive design
Artificial-label weakness	Reviewers reject pseudo-labels as not real readiness labels.	Declare target derivation, compare target routes, label them computational proxies, and include limitations.

Cost-model arbitrariness	Results depend on chosen weights.	Save cost cards, run sensitivity analysis, report cost/reward sensitivity matrices, and avoid overclaiming single-setting wins.
Policy collapse	Learned policy always waits, always falls back, or always accepts.	Monitor action distribution, coverage, fallback burden, no-action burden, and cost balance.
Over-abstention or over-deferral	Policy appears safe but becomes useless.	Report coverage, task success, deferral burden, missed-opportunity cost, and negative-result notes.
Over-aggressive accept	Policy improves coverage by accepting too many wrong actions.	Report wrong-accept rate, high-confidence wrong accepted rate, unsafe penalties, and stress unsafe rate.
Simulator overfitting	Bandit/RL exploits environment quirks.	Use held-out simulated sessions, held-out stress seeds, simple baselines, frozen-policy evaluation, and reward sensitivity.
Hidden leakage	Future/test labels leak into policy context.	Separate feature context from target construction and run validation scripts.
Unmatched comparison	A policy appears better because data, costs, rewards, stress, or embodiment settings differ.	Require matched-comparison keys; mark unmatched runs diagnostic-only and block claim-evidence export.
Metric ambiguity	Reward or cost gains cannot be interpreted.	Require metric dictionary entries with denominator, direction, grouping, and claim role.
Complexity bloat	Layer becomes too large for thesis execution.	Use staged policy ladder and core/stretch separation.
Claim overreach	Policy sounds clinically validated.	Use controller cards, simulation-only language, and Layer 0 claim-safety rules.

Reviewer-facing criticism handling is part of the defensive design:

Criticism or validity threat	Required Layer 6 response
IHARQ is just thresholding.	Keep IHARQ-lite as the interpretable baseline and compare it to learned evidence quality, Adaptive-IHARQ, and deferral under matched coverage and matched cost.
Adaptive-IHARQ labels are artificial.	State that public EEG lacks real readiness labels; document pseudo-label construction and report it as computational proxy supervision.

The policy leaks labels or future information.	Maintain decision-time context rules, target/context/evaluation file separation, leakage audits, split manifests, and validation scripts.
RL is unjustified.	Keep RL stretch-only and simulation-only; require simpler baselines, reward sensitivity, held-out simulated sessions, disabled-update audit for frozen evaluation, and <code>PolicyUpdateTrace</code> for simulation-authorized updates.
The reward is arbitrary.	Declare the reward formula and perform sensitivity analysis across cost weights, stress multipliers, optional embodiment penalties, and any Layer 9 reward-trace terms; mark all embodiment rewards as simulation-only.
The policy collapses to fallback, waiting, or unsafe.	Report action distributions, coverage, request burden, fallback burden, missed-opportunity cost, and failure-case logs.
The dashboard hides policy failures.	Require controller cards, failure-case tables, action-distribution views, matched-comparison status, negative-result notes, and immutable run/configuration hashes.
A learned policy improves only in training/adaptation curves.	Separate training/adaptation curves from frozen-policy held-out evaluation and use the frozen result for thesis claims.

12.29 Layer 6 outputs

Layer 6 outputs evidence-quality scores, predicted failure probabilities, IHARQ-lite baseline policy views, supervised readiness actions, deferral decisions, contextual-bandit policies, RL policies, policy action logs, deferral logs, simulation-authorized `PolicyUpdateTrace` files, reward/regret curves, Layer 6 `EvaluationOutcomeRecord` files, policy ablation results, matched-comparison reports, context-ablation tables, action-legality matrices, cost/reward sensitivity tables, metric-dictionary entries, negative-result notes, claim-evidence matrix entries, failure-case logs, and controller cards.

Layer 6 feeds Layer 7 with `PolicyActionRecord`, `DeferralRecord`, learned policy identifiers, action legality masks, expected costs, and simulation-only flags. It feeds Layer 8 with policy outputs for stress-conditioned evaluation. It feeds Layer 9 with embodiment-compatible gating records through Layer 7 or explicit policy records, never with real-body safety claims. It feeds Layer 10 with policy logs, action distributions, reward/regret curves, Layer 6 `EvaluationOutcomeRecord` files, ablation tables, matched-comparison status, negative-result notes, failure cases, claim-evidence entries, and controller cards.

12.30 Definition of done

Layer 6 is complete only when the former Layer 19 policy-learning dossier has been absorbed into this layer, the pre-6.0 prerequisite check, P0/6.0–P5/6.5 policy escalation ladder, and optional 6.E embodiment-aware extension are staged and defined, the action space is staged and defined; `ReadinessContextRecord` is fully specified; conditional `TrustStateRecord` intake is validated when independent trust-timeline features are enabled; input validation and leakage prevention exist; target-construction routes are documented; cost and reward models are versioned; the

evidence-quality estimator is implemented and evaluated; supervised Adaptive-IHARQ is compared against thresholds, selective prediction, IHARQ-lite, ensembles, and deferral; learning-to-defer has matched-cost and matched-coverage comparisons; contextual bandit is evaluated only in declared simulation; RL is clearly stretch-only and simulation-only; StressForge evaluation is included where relevant; embodiment terms are optional and bounded; `PolicyActionRecord`, `DeferralRecord`, and Layer 6 `EvaluationOutcomeRecord` files are saved; reward/regret curves, policy ablation tables, matched-comparison reports, metric dictionaries, negative-result notes, and claim-evidence entries exist; controller cards are generated; failure modes and limitations are documented; no clinical-control or patient-safety overclaim is made; and every Layer 6 claim is traceable to a table, figure, record, curve, card, or limitation statement.

13 Layer 7: Simulated Closed-Loop Readiness Environment

13.1 Problem solved by Layer 7

Offline classification metrics do not reveal the sequential consequences of readiness decisions. Layers 2–6 can measure decoder predictions, calibration quality, uncertainty, evidence reliability, IHARQ decisions, RegimeRisk state, and adaptive policy actions. However, those records remain mostly isolated unless the architecture can ask what happens when a readiness policy repeatedly accepts, waits, requests more evidence, combines evidence, defers, falls back, holds no action, or blocks unsafe action over time. Layer 7 solves this bottleneck by turning isolated readiness decisions into simulated closed-loop readiness sessions with state, actions, transition consequences, rewards, delays, stress variables, regime updates, optional embodiment responses, and complete transition logs.

Layer 7 is therefore the experimental bridge between offline EEG reliability evaluation and simulated decision-consequence evaluation. It does not train the decoder, calibrate probabilities, define primary IHARQ rules, or claim clinical closed-loop control. Instead, it creates a declared environment in which readiness policies can be compared under matched replay sessions, matched stress profiles, matched reward configurations, matched evidence-request budgets, and matched optional embodiment adapters.

The thesis role of Layer 7 is to support a safe but stronger claim than ordinary decoder benchmarking:

A layered readiness architecture can be evaluated not only through offline classification, calibration, and uncertainty metrics, but also through simulated decision-consequence metrics such as wrong-accept rate, evidence-request burden, fallback rate, late-assist rate, unsafe-event rate, reward, regret, and stress-conditioned policy degradation under declared simulation assumptions.

This layer protects the thesis against the criticism that the architecture is only a static classifier wrapper. It also protects against the criticism that waiting, evidence request, fallback, deferral, and unsafe-blocking are merely verbal claims by forcing those actions to produce logged costs and consequences. If Layer 7 were removed, IHARQ BenchGuard could still report decoder accuracy, confidence, calibration, IHARQ actions, RegimeRisk scores, and policy outputs, but it could not evaluate the temporal cost of waiting, the benefit of requesting evidence, the harm of accepting uncertain evidence, the value of fallback, the effect of repeated decision dynamics, contextual bandit behavior, reinforcement-learning behavior, stress-conditioned closed-loop behavior, or readiness-gated embodiment behavior.

Claim-safety rule: Layer 7 may claim simulated closed-loop readiness-policy evidence, transition-level consequence analysis, stress-conditioned policy comparison, and optional proxy embodiment gating. It must not claim real clinical safety, real patient readiness, real BCI-FES validation, real rehabilitation benefit, real muscle activation control from public motor-imagery EEG, or deployment-ready closed-loop medical control.

13.2 Engineering responsibility, scope, and boundaries

The engineering responsibility of Layer 7 is to define and execute a reproducible simulated environment with explicit state, action, transition, reward, episode, stress, regime, embodiment, logging, and policy-comparison contracts. It receives structured records from upstream reliability and policy layers, exposes a legal readiness context to policies, applies selected actions under declared transition rules, computes transparent rewards and costs, updates optional regime and stress state, emits readiness-gated commands to optional embodiment adapters, and logs every step as auditable records for Layers 6, 8, 9, and 10.

In scope. Layer 7 includes repeated readiness sessions; session and episode initialization; replay cursor management; observable-state assembly; hidden-state declaration; policy-output normalization; action validation; action application; wait/request/combine logic; simulated consequences; reward and cost calculation; RegimeRisk coupling through explicit records; StressForge coupling through declared stress profiles; optional embodiment coupling through an adapter; transition logging; policy-comparison harnesses; sanity checks; reward sensitivity; environment cards; and dashboard-ready exports.

Out of scope. Layer 7 does not train EEG decoders, compute primary calibration, define the original IHARQ rule set, compute primary RegimeRisk features, independently generate stress profiles, train policies as a model layer, implement MyoSuite/OpenSim internals, create final dashboard layouts, or make clinical validation claims. Decoder training belongs to Layer 2; uncertainty and selective prediction belong to Layer 3; evidence verification belongs to Layer 4; RegimeRisk definition belongs to Layer 5; policy construction belongs to Layer 6; stress-profile generation belongs to Layer 8; embodiment simulation belongs to Layer 9; and final reporting belongs to Layer 10.

Core modules. The minimum core implementation includes the EnvironmentConfig Registry, SessionBuilder, ReplayCursor, StateAssembler, PolicyAdapter, ActionValidator, ActionApplier, EvidenceRequestManager, RewardEngine, TransitionLogger, PolicyComparisonHarness, and EnvironmentValidator.

Supporting modules. Supporting components include the RewardSensitivityRunner, SanityCheckRunner, EpisodeSummaryBuilder, PolicyRolloutExporter, SafetyEventExtractor, and CardGenerator.

Stretch modules. RegimeCoupler, StressCoupler, the PolicyUpdateHook for trace-safe contextual-bandit/RL updates, RL environment wrappers, EmbodimentGateway, MyoSuite adapter, OpenSim replay adapter, and dashboard live-inspection hooks are Stretch C components. They are valuable, but they must be introduced only after clean deterministic replay and transition logging are stable.

Expected implementation artifacts. Layer 7 should produce configuration files, record schemas, transition logs, session manifests, reward traces, safety-event logs, policy-comparison tables, stress-sensitivity tables, sanity-check reports, environment cards, rollout cards, reproducibility commands, and thesis figures.

13.3 Layer identity and functional position

Layer 7 should be interpreted as a simulated environment, not as a normal classifier, not as a dashboard, not as a clinical controller, and not as a hidden post-processing rule. It has three simultaneous identities:

Identity	Meaning
Experimental environment	Provides a controlled setting for comparing accept-all, threshold, selective prediction, IHARQ-lite, learning-to-defer, supervised Adaptive-IHARQ, contextual bandit, and RL policies under identical replay sessions.
Consequence simulator	Assigns declared consequences to accept , wait , request_evidence , combine_evidence , fallback , preserved-source uncertain , policy-only defer , no_action , and unsafe actions while preserving the distinction between source tokens, policy aliases, and simulator commands.
Logging and reproducibility engine	Writes transition-level records so policy behavior, reward curves, failures, stress exposure, regime state, and embodiment outcomes can be audited and reproduced.

The formal layer contract is summarized below.

Field	Layer 7 value
Layer number	Layer 7.
Layer name	Simulated Closed-Loop Readiness Environment.
Architectural plane	Simulation, sequential decision, and optional embodiment plane.
Primary function	Run repeated readiness sessions in which readiness actions have simulated consequences.
Primary upstream layers	Layer 1 data windows, Layer 2 predictions, Layer 3 calibration and uncertainty, Layer 4 IHARQ and evidence quality, Layer 5 RegimeRisk, Layer 6 adaptive readiness policies, Layer 8 stress profiles and stress schedules when active, and Layer 9 embodiment records when active.
Primary downstream layers	Layer 6 policy-learning evaluation and simulation-authorized update traces, Layer 8 stress-conditioned summaries, Layer 9 readiness-gated command execution, Layer 10 dashboard/cards/reproducibility, and Layer 0 claim-safety governance.
Primary consumed records	WindowRecord, PredictionRecord, CalibrationRecord, UncertaintyRecord, EvidenceQualityRecord, IHARQDecisionRecord, RegimeStateRecord, ReadinessContextRecord, PolicyActionRecord, StressProfileRecord, StressScheduleRecord, and optional EmbodimentStateRecord/EmbodimentOutcomeRecord records when Layer 9 is active.

Primary produced records	<code>TransitionRecord</code> , <code>ClosedLoopSessionRecord</code> , <code>EpisodeSummaryRecord</code> , <code>PolicyRolloutRecord</code> , <code>RewardTraceRecord</code> , <code>EnvironmentDiagnosticRecord</code> , <code>SafetyEventRecord</code> , and optional <code>PolicyUpdateTrace</code> .
Core thesis role	Converts offline reliability estimates into simulated decision-consequence evidence.
Claim boundary	Supports simulation-bounded policy comparison; does not validate a real clinical closed-loop BCI controller.

The functional graph rule is strict: Layer 7 must not create hidden experimental data. Every state element, policy action, reward, transition, safety event, and plotted summary must be traceable to an upstream record, configuration file, random seed, or declared simulation rule.

13.4 Input contracts

Layer 7 receives primary inputs, auxiliary inputs, metadata inputs, evaluation-only inputs, proxy inputs, and stretch inputs. The environment must validate these records before each rollout.

Input category	Records and fields	Use inside Layer 7
Primary replay input	<code>WindowRecord</code> : dataset identifier, subject identifier, session identifier, trial identifier, window identifier, start time, end time, split identifier, true label for scoring, channel mask, artifact flags.	Defines replay cursor, episode identity, chronological order, and outcome reference. True labels are not visible to ordinary policies and are used only for reward/outcome computation.
Primary decoder input	<code>PredictionRecord</code> : model identifier, model family, predicted class, probability vector or logits, prediction timestamp, trial/window identifier.	Supplies decoded intent, prediction history, coverage, correctness proxy after scoring, and accepted-command candidate.
Primary reliability input	<code>CalibrationRecord</code> and <code>UncertaintyRecord</code> : calibrated probability, confidence, entropy, margin, disagreement, reliability bin, ECE/Brier/NLL summaries, conformal-set size if used.	Supplies observable reliability state and selective-prediction features.

Primary readiness input	EvidenceQualityRecord and IHARQDecisionRecord : evidence-quality score, predicted failure probability, stability features, canonical Layer 4 IHARQ action token, optional action/display label, reason code, evidence-request count, fallback status, unsafe status, accepted prediction, and action cost.	Supplies rule-based baseline actions, evidence trust, action reasons, and policy-comparison anchors while preserving Layer 4 token identity.
Primary policy input	PolicyActionRecord : policy identifier, policy family, selected policy action, optional source IHARQ action token, action-origin or alias-map reference, action probability, selected threshold, expected cost/reward, exploration flag, legality mask, reason code, action-space version, cost/reward model identifiers, training stage, simulation-only flag, and claim boundary.	Supplies the action chosen by the policy under evaluation and exposes whether the action came from a fixed policy, supervised policy, deferral policy, contextual-bandit policy, RL policy, or simulation-only extension.
Optional simulation-update input	policy_update_config : update enabled/disabled flag, update family, training-or-evaluation mode, frozen-policy flag, exploration mode, update budget, simulator identifier, and configuration hash.	Enables contextual-bandit/RL update hooks only inside declared simulation. In frozen-policy evaluation, update feedback is disabled and PolicyUpdateTrace may appear only as an explicit disabled-update audit artifact, not as a policy mutation log.
Soft temporal input	RegimeStateRecord : regime label, RegimeRisk score, risk trend, drift/fatigue/dropout flags, model trust score, stricter-evidence recommendation, stop-loss warning.	Supplies temporal trust context and optional loopback updates. Can be mocked for minimal development.
Stretch stress input	StressProfileRecord and StressScheduleRecord : stress type, intensity, target record/field, injection point, seed, schedule type, per-window or per-step stress values, active component stressors, and delay/dropout/noise/drift/fatigue/weakness settings.	Controls stress-conditioned replay and robustness evaluation. Generated by Layer 8 rather than invented inside Layer 7.

Stretch embodiment input	<code>EmbodimentStateRecord</code> , <code>EmbodimentOutcomeRecord</code> , optional <code>SafetyEventRecord</code> , optional <code>Layer9RewardTrace</code> , and <code>Layer9EmbodimentDemoManifest</code> pointer: embodiment mode, simulated state, endpoint error, command response, safety event, over-assist/under-assist proxy, command timeline, and reward-trace identifier where enabled.	Supplies optional Layer 9-owned embodiment consequences when Layer 9 is active. Can be off or mocked; future embodiment outcomes remain hidden from frozen-policy decisions.
Metadata input	Dataset card, model card, split manifest, schema version, configuration hashes, run identifier, random seed, environment version.	Enables traceability and reproducibility.
Evaluation-only input	True label, oracle policy, future-window correctness, stress ground truth, simulator diagnostic state.	Used only for scoring, oracle baselines, sanity checks, or declared evaluation. Must not leak into policy-visible state.

Before execution, Layer 7 runs input validation.

Validation type	Rule
Identifier consistency	dataset, subject, session, trial, and window identifiers must align across records.
Split consistency	Training, validation, and test identifiers must not be mixed unless the experiment explicitly declares transfer or cross-split replay.
Time consistency	Window order must be monotonic inside a session unless randomized replay is explicitly configured with a seed.
Probability validity	Probabilities must be finite and normalized, or the record must explicitly mark them as logits/raw scores.
Label validity	True labels, predicted classes, and action mappings must use the same standardized label map.
Action validity	Selected actions must belong to the declared readiness action vocabulary, preserve any source Layer 4 token, distinguish policy-only aliases from simulator commands, and obey budget limits.
Update-mode validity	Frozen-policy evaluation cannot mutate policy parameters, thresholds, action preferences, replay order, or reward weights. Any simulation-authorized contextual-bandit/RL update must emit a <code>PolicyUpdateTrace</code> with mode, step, reward, next-state, seed, and simulation-only fields.
Stress validity	Stress intensity, schedule, target linkage, and seed must match declared Layer 8 <code>StressProfileRecord</code> and <code>StressScheduleRecord</code> entries.

Reward validity	Reward terms must be traceable to a declared reward configuration.
Embodiment validity	Embodiment fields must appear only when Layer 9 is active or mocked.
Leakage audit	Policy-visible state must not include future labels, future correctness, or hidden variables unless an oracle diagnostic experiment is explicitly declared.

13.5 Dependency logic

Layer 7 has hard, soft, optional, conceptual, data, model, metric, and software dependencies. The minimum implementation should be able to run with mocked records, but thesis-level claims require real outputs from upstream layers.

Dependency	Type	Minimum required?	Can be mocked?	Reason
WindowRecord	Hard data dependency	Yes	Yes for unit tests	Defines replay order, episode identity, and state index.
PredictionRecord	Hard model-output dependency	Yes	Yes	Supplies decoded intent and correctness candidate.
CalibrationRecord and UncertaintyRecord	Hard reliability dependency	Yes	Yes	Supplies confidence, entropy, margin, calibrated probability, and selective flags.
EvidenceQualityRecord and IHARQDecisionRecord	Hard readiness dependency	Yes	Yes	Supplies rule-based baseline action, reason code, and evidence-control state.
PolicyActionRecord	Hard policy-comparison dependency	Yes	Yes	Supplies the action to apply. In minimal mode, accept-all, threshold, or IHARQ-lite can generate it.
PolicyUpdateTrace schema and policy-update configuration	Optional/stretch simulation-update dependency	No for frozen evaluation; yes for bandit/RL training runs	Yes for unit tests	Defines when reward/transition feedback may update a policy and how every update event is logged without leaking into held-out evaluation.

RegimeStateRecord	Soft-to-hard temporal dependency	Mockable minimum; required for full Stretch C	Yes	Supplies temporal trust, risk trend, and stop-loss context.
StressProfileRecord and StressScheduleRecord	Optional/stretch stress dependency	No for clean replay; yes for StressForge experiments	Yes	Supplies the stress definition, schedule linkage, per-window/per-step stress values, injection targets, and perturbation parameters.
EmbodimentStateRecord	Optional/stretch embodiment dependency	No	Yes	Supplies proxy body/action consequences when Layer 9 is active.
Reward configuration	Hard metric dependency	Yes	No for valid results	Defines costs and rewards; without it, policy comparisons are arbitrary.
Policy configuration	Hard software/config dependency	Yes	No for valid results	Declares policy list, versions, adapters, and matched-comparison settings.
Environment configuration	Hard reproducibility dependency	Yes	No for valid results	Declares state visibility, hidden variables, transition mode, replay mode, episode rules, and seeds.
Layer 0 claim boundary	Conceptual dependency	Yes	No	Prevents simulation results from being presented as clinical validation.

The minimum runnable path is:

WindowRecord + PredictionRecord + CalibrationRecord/UncertaintyRecord + IHARQDecisionRecord → deterministic repeated session replay → TransitionRecord + EpisodeSummaryRecord + policy-comparison metrics.

The full Stretch C path is:

ReadinessContextRecord + RegimeStateRecord + PolicyActionRecord + StressProfileRecord + StressScheduleRecord + optional EmbodimentStateRecord/EmbodimentOutcomeRecord → stress-conditioned, policy-learning-ready, embodiment-aware closed-loop sessions → dashboard-ready transitions, reward traces, safety events, optional PolicyUpdateTrace files, rollout cards, and reproducibility cards.

13.6 Environment state

The Layer 7 state must be explicit. It must distinguish **observable state**, which the policy may use, from **hidden environment state**, which the simulator may use to generate consequences but which must not be exposed to ordinary policies.

State category	Contents	Visible to policy?
Replay state	Current subject/session/trial/window, time index, replay cursor, split identifier, request budget.	Partly; true label is hidden and scoring-only.
Decoder state	Predicted class, probability vector, calibrated probability, confidence, entropy, margin, model disagreement.	Yes.
Evidence state	Evidence history, request count, stability score, combined evidence, channel stability, time-window stability.	Yes.
IHARQ state	Last IHARQ action, reason code, fallback status, unsafe status, evidence-control status.	Yes.
Regime state	RegimeRisk score, regime label, trend, model trust, drift/fatigue/dropout flags, stop-loss warning.	Yes when Layer 5 is active; mockable otherwise.
Stress state	Delay, dropout, noisy-intent, drift, weakness, fatigue-like, or combined stress profile.	Visible or partially visible according to environment configuration.
Policy-history state	Previous actions, wait count, request count, fallback count, wrong-accept history when legally observable.	Yes, except future/outcome-only fields.
Embodiment state	Command state, endpoint proxy, safety flag, over-assist/under-assist proxy, safe-hold state.	Yes when Layer 9 is active, according to adapter configuration.
Hidden simulation state	Latent fatigue, latent drift, hidden noise level, latent attention proxy, motor-response-quality proxy.	No, except oracle/diagnostic experiments.

The recommended observable **ReadinessContextRecord** includes calibrated confidence, entropy, margin, model disagreement, time-window stability, channel stability, evidence-quality score, predicted failure probability, IHARQ action candidate, IHARQ reason code, evidence-request count, previous action, previous wait count, previous fallback count, current RegimeRisk score, regime label, drift/fatigue/dropout risk flags, stress profile label and intensity when visible, and embodiment status when active.

Hidden variables may include latent fatigue, latent drift, latent attention, latent motor-response quality, and hidden noise level. All hidden variables must be declared in `environment_config.yaml`. They cannot be invented silently during analysis. True labels

and future-window outcomes are hidden scoring variables and may be used only for reward computation, oracle baselines, or post-hoc evaluation.

13.7 Episode and session design

Layer 7 should default to **session episodes** because its purpose is repeated decision evaluation. Trial-level episodes are useful for debugging, but they do not fully test fatigue-like, drift-like, evidence-request, or temporal trust behavior.

Episode unit	Meaning	Best use
Trial episode	One trial or one evidence sequence.	Debugging, fast sanity checks, evidence-request unit tests.
Session episode	Many trials/windows for one subject/session.	Default Layer 7 evaluation, RegimeRisk, stress, delay, and policy dynamics.
Subject episode	Full subject-level replay across sessions.	Subject robustness and personalization analysis.
Stress episode	One session under one declared stress profile.	Layer 8 robustness evaluation.
Embodiment episode	Simulated action task with readiness gating.	Layer 9 integration and demonstration.

Each episode initializes dataset identifier, subject identifier, session identifier, split identifier, policy name, policy version, stress profile identifier, environment configuration identifier, reward configuration identifier, random seed, replay order, initial regime state, initial evidence history, action budget, and embodiment mode. Episode termination can occur at the end of a session, after an unsafe stop, after the maximum delay is exceeded, after the maximum evidence-request count is exceeded, after an embodiment failure proxy, or after a fixed horizon is reached. The termination rule must be fixed before policy comparison.

Layer 7 should support five replay modes: clean replay, stress-conditioned replay, counterfactual replay, oracle replay, and embodiment replay. Clean replay uses original prediction and uncertainty records. Stress-conditioned replay applies declared StressForge perturbations. Counterfactual replay runs different policies over identical evidence sequences. Oracle replay uses true labels only for diagnostic upper/lower-bound comparisons. Embodiment replay sends accepted or fallback commands through the Layer 9 adapter.

13.8 Action space and action validity

Layer 7 uses a normalized action vocabulary aligned with Layer 4 IHARQ and Layer 6 policy learning. Its validator distinguishes three different concepts: the source Layer 4 canonical action token, the Layer 6 selected policy action, and the Layer 7 simulator command or consequence. Display forms such as **request evidence** and **combine evidence** may appear in figures, but stored Layer 4 actions remain `request_evidence` and `combine_evidence`. Because `uncertain` is a canonical Layer 4 stored token rather than an executable embodiment command, Layer 7

must preserve it as the source token and map it through declared action configuration to a legal simulator consequence.

Action	Meaning in Layer 7	Consequence type
<code>accept</code>	Use the current predicted intent.	Immediate correct-accept reward or wrong-accept cost.
<code>wait</code>	Delay without explicitly consuming a new evidence window.	Delay cost, possible future information if the replay cursor advances by configuration.
<code>request_evidence</code>	Acquire or evaluate another window/evidence item; display label may be Request evidence .	Evidence cost, delay cost, possible reliability improvement, possible request failure under dropout.
<code>combine_evidence</code>	Combine current and prior evidence according to a declared rule; display label may be Combine evidence .	Possible confidence improvement, complexity/delay penalty, invalid if no usable history exists.
<code>fallback</code>	Route to a safe fallback, conservative controller, abstention, or proxy decision path.	Fallback cost and reduced wrong-accept risk.
<code>uncertain</code>	Preserved Layer 4 source token for undecidable evidence; Layer 7 maps it through declared action configuration to <code>wait</code> , <code>request_evidence</code> , <code>fallback</code> , <code>no_action</code> , or blocked <code>unsafe</code> rather than treating it as an executable command.	Configured undecidable-state consequence; no direct embodiment command.
<code>defer</code>	Layer 6 policy-only route to another decision source; may route to a simulated supervisor/expert proxy, fallback, safer model, no-action route, or request path when declared.	Deferral cost and proxy outcome; simulation-only.
<code>no_action</code>	Layer 6/Layer 7 policy or simulator alias for hold or do nothing.	Missed-action or late-assist cost, but possible safety benefit.
<code>unsafe</code>	Block action and mark unsafe state or stop-loss.	Safety event, high-severity log, possible protective reward/cost depending on ground-truth risk.

Action validity rules prevent arbitrary behavior. Evidence requests cannot continue indefinitely. `combine_evidence` is invalid without prior evidence. Fallback is invalid if no fallback definition exists. `uncertain` is valid only as a preserved Layer 4 source token and must be mapped to a declared legal simulator consequence before application. `defer` is invalid without a declared deferral target, such as fallback, safe no-action, request path, safer model, or simulated supervisor

proxy. Layer 9 cannot execute an intent command for `wait`, `request_evidence`, `uncertain`, `defer`, `no_action`, or `unsafe`. Every `unsafe` action must create a `SafetyEventRecord`. Invalid actions are either mapped to a configured safe fallback or blocked and logged as local `ActionValidationRecord` fields inside the transition diagnostics.

Policy outputs are normalized through the **PolicyAdapter**. Accept/reject threshold baselines map to `accept`, `no_action`, or `fallback`. IHARQ-lite preserves Layer 4 tokens directly, including `request_evidence`, `combine_evidence`, and `uncertain`; any non-executable `uncertain` source token is then mapped to a declared simulator consequence without overwriting the stored Layer 4 value. Supervised Adaptive-IHARQ maps class outputs to readiness actions while preserving `source_iharq_action` when available. Learning-to-defer maps predict/defer outputs to `accept`, `fallback`, `defer`, or `no_action`. Contextual bandit arms, RL action identifiers, and simulator command aliases are mapped through a versioned action map in `policy_config.yaml`; they must not overwrite stored Layer 4 action values.

13.9 Transition logic

The transition function is the heart of Layer 7. Every step must be explainable and reproducible.

1. The **StateAssembler** exposes an observable `ReadinessContextRecord` and preserves hidden scoring state separately.
2. The active policy selects an action and emits a `PolicyActionRecord`.
3. The **PolicyAdapter** normalizes the action to the Layer 7 vocabulary.
4. The **ActionValidator** checks legality, budgets, fallback availability, stress constraints, and embodiment constraints.
5. The **ActionApplier** applies the validated action and creates a `ConsequenceRecord`.
6. The **EvidenceRequestManager** updates evidence history, replay cursor, wait count, request count, and combine state.
7. The **StressCoupler**, when active, updates stress state according to the Layer 8 profile, formal `StressScheduleRecord`, and seed.
8. The **RegimeCoupler**, when active, updates or forwards `RegimeStateRecord` according to declared Layer 5 rules.
9. The **EmbodimentGateway**, when active, sends readiness-gated commands to Layer 9 and receives `EmbodimentStateRecord`.
10. The **RewardEngine** computes reward and cost decomposition using declared reward configuration.
11. The **TransitionLogger** writes the `TransitionRecord`, `RewardTraceRecord`, diagnostics, and safety events.

The step-level contract is:

$$state_t \rightarrow action_t \rightarrow validated\ action_t \rightarrow consequence_t \rightarrow stress_{t+1} \rightarrow regime_{t+1} \rightarrow state_{t+1} \rightarrow reward_t \rightarrow transition\ log.$$

Layer 7 should begin with deterministic transitions: chronological replay, fixed reward tables, fixed stress schedules, and rule-based regime updates. Stochastic variants are allowed only when seeds, distributions, and random draws are logged. Deterministic baselines are required before contextual bandit or RL results are considered thesis evidence.

Every transition should be recoverable as a sentence: at step t , policy P selected action A because the observable context had confidence x , risk y , request count z , and stress state s ; the environment assigned reward r because outcome O occurred under reward configuration R .

13.10 Reward, cost, and consequence model

Layer 7 rewards readiness quality, not merely classification correctness. A correct but excessively delayed action, an over-aggressive wrong accept, a useless evidence request, and a safe fallback under poor evidence are different consequences and must be scored differently.

The reward model penalizes wrong accepted actions, unsafe states, excessive delay, excessive evidence requests, unnecessary fallback, over-assistance when embodiment is active, late assistance, unstable assistance, and missed feasible correct actions. It rewards correct timely accepted actions, correct conservative fallback when evidence is poor, appropriate no-action or unsafe blocking when risk is high, reduced wrong-accept rate, and stable embodied response when Layer 9 is active.

Reward/cost term	Meaning
correct_accept_reward	Positive reward for accepting a correct intent in time.
wrong_accept_cost	Large penalty for accepting the wrong predicted intent.
wait_cost	Penalty for delaying action without necessarily adding evidence.
evidence_request_cost	Penalty for consuming another window/evidence source.
combine_cost	Small complexity or delay penalty for evidence combination.
fallback_cost	Penalty for not acting directly, lower than wrong unsafe action if fallback is useful.
defer_cost	Penalty for supervisor/proxy routing in declared simulation.
unsafe_event_cost	Highest penalty when an unsafe event occurs; protective unsafe blocking may be scored separately.
no_action_cost	Penalty for holding action when a useful action was feasible; lower than wrong action in high-risk states.
late_assist_cost	Penalty for correct but delayed assistance.
over_assist_cost	Penalty for unnecessary or excessive simulated assistive action.
embodiment_error_cost	Endpoint, instability, smoothness, or command-response proxy penalty when Layer 9 is active.

A transparent reward template is:

```
reward = correct_accept_reward - wrong_accept_cost - wait_cost -
evidence_request_cost - combine_cost - fallback_cost - defer_cost -
```

```
no_action_cost - unsafe_event_cost - late_assist_cost - over_assist_cost -
embodiment_error_cost.
```

The exact numerical values must be declared in `reward_config.yaml`. The thesis should run sensitivity analysis over wrong-accept penalty, delay penalty, fallback penalty, evidence-request penalty, unsafe-event penalty, and embodiment error penalty. If policy rankings change under small reward changes, the result should be reported as reward-sensitive rather than overclaimed as universal superiority.

13.11 Evidence-request and temporal-decision logic

Layer 7 exists partly because evidence accumulation has time consequences. The environment must distinguish **wait** from **request evidence**. **wait** delays action but does not necessarily consume new evidence. **request evidence** explicitly consumes or evaluates another window/evidence source and therefore produces both a delay cost and an evidence cost.

When a policy requests evidence, the environment must define which next window is requested, whether the request consumes time, whether the request can fail under dropout or stress, whether the new evidence can be combined, how many requests are allowed, and what happens when the maximum request count is reached. The default post-limit action should be configured as fallback, no-action, or unsafe block, not silently ignored.

Temporal metrics include mean decision delay, evidence requests per decision, maximum-request violation rate, late-accept rate, early-wrong-accept rate, delay-reward trade-off, and request-success rate. These metrics allow Layer 10 to show whether a policy is safe because it is genuinely better or only because it avoids acting.

13.12 RegimeRisk coupling

Layer 7 consumes RegimeRisk state from Layer 5 through clean interfaces and may return simulated outcomes for future regime updates. It must not duplicate Layer 5's definition of temporal trust.

Layer 7 receives regime label, RegimeRisk score, risk trend, volatility-like feature, drawdown-like feature, drift-risk flag, fatigue-risk flag, dropout-risk flag, model trust score, stricter-evidence recommendation, and stop-loss warning. These values can be included in the observable state according to environment configuration.

Layer 7 may send recent accepted correctness, wrong-accept events, unsafe events, fallback frequency, evidence-request frequency, delay patterns, stress-conditioned outcomes, and embodiment instability to Layer 5. This is a declared loopback, not an invisible intervention.

Loop source	Loop target	Signal	Trigger	Safety/evaluation rule
Layer 7 TransitionRecord	Layer 5 RegimeRisk update	Recent wrong accepts, unsafe events, fallback bursts, delays, stress-conditioned failures.	End of transition, episode, or configured rolling window.	Used only in declared simulation episodes; must not use future test labels to alter offline decisions.

Layer 5 RegimeStateRecord	Layer 7 observable state	Risk score, trend, stop-loss warning, stricter-evidence recommendation.	New episode, transition, or risk update.	Policy can use it only if the experiment declares regime-visible state.
Layer 7 safety event	Layer 5 stop-loss context	Repeated unsafe or wrong-accept cluster.	Safety threshold exceeded.	Stop-loss must be logged as environment intervention, not hidden reward manipulation.

Regime-aware scenarios should include stable regime, noisy regime, drift-risk regime, fatigue-risk regime, dropout-risk regime, and unsafe regime. The goal is to test whether policies become appropriately conservative under temporal trust degradation without claiming real physiological fatigue detection.

13.13 StressForge coupling

Layer 8 generates stress profiles and formal **StressScheduleRecord** schedules; Layer 7 applies them during repeated sessions. Stress must be configured before the rollout and must be matched across policies.

Stress type	Layer 7 application point
Delay	Adds time cost to wait, request, accept, fallback, or embodiment-command execution.
Dropout	Removes windows, channels, evidence sources, or model outputs according to the declared dropout profile.
Noisy intent	Perturbs prediction confidence, increases disagreement, or changes evidence-quality features according to configuration.
Weakness	Affects Layer 9 embodiment response or simulated task success; should not be presented as real muscle weakness.
Drift	Changes reliability over time and feeds RegimeRisk state through declared rules.
Fatigue-like degradation	Gradually worsens evidence quality, delay, or response reliability as a proxy variable.
Combined stress	Applies multiple stressors under separate logged schedules and seeds.

Stress schedules are represented by **StressScheduleRecord** and may be clean/no-stress, constant, ramp, burst, periodic, stochastic, or scenario-scripted. Combined stress is represented by the stress family and active component stressors inside the schedule record, not by stale step/cyclic schedule vocabulary. For fair comparison, Policy A, Policy B, and Policy C must see the same subject/session replay, stress profile, stress schedule, stress seed, reward configuration, maximum evidence-request count, fallback definition, and embodiment mode. If this condition is violated, policy comparisons are diagnostic only.

13.14 Policy-evaluation protocol

Layer 7 is the environment in which multiple policy families are compared. Only the policy should change during a matched comparison.

Policy	Role inside Layer 7
Accept-all decoder	Naive baseline with maximum coverage and likely higher wrong-accept risk.
Confidence threshold	Simple selective-prediction baseline.
Entropy/margin rejection	Stronger uncertainty baseline.
IHARQ-lite	Interpretable rule-based evidence-control baseline.
Learning-to-defer	Strong deferral baseline for uncertain cases.
Supervised	Learned multi-action readiness policy from declared pseudo-labels.
Adaptive-IHARQ	Online/adaptive immediate-reward policy in simulation.
Contextual bandit	Delayed-consequence policy in simulation only.
RL policy	Diagnostic upper/lower-bound comparison only; not a deployable policy.
Oracle policy	

For every comparison, Layer 7 fixes dataset, subject/session replay, split, model predictions, calibration records, uncertainty records, initial regime state, **StressProfileRecord**, **StressScheduleRecord**, reward configuration, maximum evidence-request count, random seed, fallback definition, and embodiment mode. Policy comparison outputs include reward curve, regret curve where applicable, wrong-accept curve, coverage curve, evidence-cost curve, fallback curve, unsafe-event curve, stress-sensitivity curve, per-subject ranking, and controller card.

RL is Stretch C only. It should be presented as simulation-only policy-learning feasibility, not as a deployed controller. RL should begin with a small action space: accept versus wait; then accept/wait/fallback; then accept/request-evidence/fallback; then the full IHARQ action vocabulary; and only later embodiment-aware decisions.

13.15 Frozen-policy evaluation versus simulation-authorized updates

Layer 7 must distinguish two execution modes that are often confused in policy-learning architectures.

Execution mode	Allowed behavior	Required records and claim boundary
----------------	------------------	-------------------------------------

Frozen-policy evaluation	The policy is loaded, receives only the observable <code>ReadinessContextRecord</code> , emits a <code>PolicyActionRecord</code> , and is not allowed to change thresholds, weights, action preferences, exploration schedules, reward settings, or replay order during held-out evaluation. Rewards and outcomes are scoring signals only.	Requires <code>TransitionRecord</code> , <code>RewardTraceRecord</code> , <code>EpisodeSummaryRecord</code> , and policy-comparison summaries. <code>PolicyUpdateTrace</code> is omitted unless a disabled-update audit entry is intentionally saved. Claims may compare fixed policies under matched simulation.
Simulation-authorized training/update	Contextual-bandit or RL policies may receive reward and next-state feedback and update only inside a declared simulation-training or simulation-adaptation run. The update route must be enabled by configuration before execution and must not be mixed with held-out frozen evaluation.	Every policy mutation must emit a <code>PolicyUpdateTrace</code> containing policy identity, context, action, reward, next state, update step, exploration mode, training/evaluation mode, frozen-policy flag, simulator ID, seed, configuration hash, and <code>simulation_only=true</code> . Claims are limited to simulation-only policy-learning behavior.

The recommended protocol is train-or-adapt first under declared simulation, freeze the resulting policy artifact, and then evaluate that frozen artifact on held-out subjects, held-out sessions, held-out stress seeds, or held-out stress profiles. Training curves, adaptation curves, and frozen-evaluation curves must be plotted and interpreted separately. A policy that improves while it is still updating cannot be directly compared with a frozen baseline unless the comparison is explicitly labeled as an online-adaptation experiment.

This boundary also controls feedback to Layer 6. During frozen-policy evaluation, Layer 7 may send transition, reward, and failure summaries back to Layer 6 only for offline analysis and future reruns. During simulation-authorized training, Layer 7 may provide step-wise reward and next-state feedback to the `PolicyUpdateHook`; the resulting update is valid only if it is traceable through `PolicyUpdateTrace`. Any missing trace, mismatched frozen-policy flag, or unlogged policy mutation invalidates the bandit/RL result.

13.16 Embodiment gateway

Layer 7 connects to Layer 9 through the Layer 9 embodiment session/controller interface, not by directly depending on MyoSuite, OpenSim, static-replay files, or adapter internals. The default mode is off. Development can use a mock embodiment adapter before MyoSuite or OpenSim integration. Layer 7 may request an embodiment step and consume Layer 9-owned response identifiers, but Layer 9 remains responsible for record intake validation, readiness-context assembly, command mapping, safety gating, simulator/static-replay execution, transition/safety logging, reward-trace creation where enabled, and dashboard-compatible export packaging.

Mode	Description	Use
Off	No embodiment; Layer 7 evaluates decisions only.	Core closed-loop evaluation.
Mock embodiment	Simple proxy outcome model for command success, endpoint error, or safe hold.	Early development and unit tests.
MyoSuite adapter	Gym-like simulated assistive-action task.	Stretch C primary embodiment branch.
OpenSim replay adapter	Optional visualization or biomechanical replay of selected commands.	Optional branch; not the first training environment.

Readiness-to-command mapping is declared. `accept` sends the mapped intent command. `combine_evidence` sends a mapped command only after the combined evidence resolves to an accepted stable intent. `wait` and `request_evidence` hold command execution. `fallback` sends conservative command or safe fallback. Policy-only `no_action` holds position. `unsafe` triggers safe stop or blocked action. Layer 7 may consume endpoint error, simulated success/failure, instability event, over-assist proxy, under-assist proxy, delay consequence, weakness response, safe-hold event, command smoothness proxy, `SafetyEventRecord` links, `Layer9RewardTrace` links, command-timeline links, endpoint-error-curve links, and demo-manifest identifiers from Layer 9. These outputs affect reward only through declared reward terms and must be logged as simulation-only embodiment evidence.

13.17 Submodule architecture

Layer 7 is decomposed into testable submodules. The following table defines their role, dependencies, outputs, consumers, and defensive evaluation logic.

Submodule	Status	Responsibility and inputs	Outputs and consumers	Evaluation and failure modes
EnvironmentConfig Registry	Core	Stores environment, reward, episode, action, evidence, stress, embodiment, and policy-comparison settings from YAML/JSON.	Config hash and validation report for all Layer 7 modules and Layer 10.	Check schema completeness, hash stability, and missing-field failures.
SessionBuilder	Core	Builds replay sessions from <code>WindowRecord</code> , split manifest, subject/session metadata, and policy/stress configuration.	<code>ClosedLoopSessionBuilder</code> and session manifest for <code>ReplayCursor</code> and Layer 10.	Replay identifiers, splits, or session lengths are inconsistent.

ReplayCursor	Core	Advances through windows/trials according to chronological, randomized, stress, or request-driven replay mode.	Current replay index and cursor state for StateAssembler and EvidenceRequest-Manager.	Check no future leakage, deterministic replay with same seed, and request-bound behavior.
StateAssembler	Core	Combines prediction, uncertainty, calibration, IHARQ, evidence-quality, RegimeRisk, stress, history, and optional embodiment fields.	ReadinessContext for policy selection and logging.	Check if hidden fields leak or required observable fields are missing.
PolicyAdapter	Core	Converts threshold, IHARQ, supervised, deferral, bandit, and RL outputs into normalized actions.	PolicyActionRecord for ActionValidator and Layer 10.	Check action-map versioning, invalid arms, and probability/expected-cost consistency.
ActionValidator	Core	Checks legality of action under current state, budgets, fallback availability, deferral target, preserved uncertain source-token mapping, and embodiment mode.	Local ActionValidationRecords; fields embedded in TransitionRecord and EnvironmentDiagnosticRecord; valid action or safe substitute.	Logs invalid actions; prevents unlimited requests, unmapped states, invalid deferrals, and illegal embodiment commands.
ActionApplier	Core	Applies accepted action to state and determines direct consequence using scoring-only true labels where allowed.	Local ConsequenceRecord; fields embedded in TransitionRecord and RewardTraceRecord for reward, regime, stress, and logging.	Failure modes include wrong label map, untraceable consequence, or unlogged fallback.

EvidenceRequestManager	Core	Handles <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , evidence history, request count, delay, and max-request termination.	Updated evidence state, replay cursor, preserved action tokens, and delay metrics.	Check max request bound, combine-without-history errors, and request failures under dropout.
RewardEngine	Core	Computes reward and cost decomposition from state, action, consequence, stress, regime, and embodiment outputs.	RewardTraceRecord and reward components for Layer 6 and Layer 10.	Risk of arbitrary reward; requires sensitivity analysis and config traceability.
PolicyUpdateHook	Stretch	Receives context, action, reward, next state, and update configuration for contextual-bandit/RL training mode only.	Optional PolicyUpdateTrace updated simulation-policy artifact pointer, and disabled-update audit entries for Layers 6 and 10.	Disabled during frozen-policy evaluation; fails if a policy mutates without a trace, if updates use hidden labels directly, or if training and held-out evaluation are mixed.
TransitionLogger	Core	Writes complete transition records, safety events, reward traces, and diagnostics.	TransitionRecord , SafetyEventRecord , diagnostic logs for Layers 6/10.	Logging completeness invariant: every step must have one transition record.
PolicyComparisonHandler	Core	Runs matched policies over identical sessions, stress profiles, seeds, budgets, and reward configurations.	PolicyRolloutRecord , policy-comparison table, ablation outputs.	Fails if policies receive mismatched sessions, seeds, or stress schedules.
EnvironmentValidator	Core	Runs invariants, leakage checks, reward consistency checks, and sanity experiments.	EnvironmentDiagnosticBlock , sanity-check report.	BlockRecord , thesis-level claims if invariants fail.

StressCoupler	Stretch	Applies Layer 8 stress profiles to evidence, delay, dropout, drift, fatigue-like, weakness, and combined stress state.	Updated stress state and stress-conditioned outcomes for Layer 8/10.	Requires matched seeds; unfair stress schedules invalidate policy comparison.
RegimeCoupler	Stretch/core for full C	Consumes and updates RegimeStateRecord from Layer 5 using declared transition outcomes.	Next regime state and regime-aware summaries.	Must not silently override policy actions; all loopbacks logged.
EmbodimentGatewayStretch		Sends readiness-gated commands to the Layer 9 static-replay/MyoSuite/OpenFit adapter path and receives saved proxy outcomes.	EmbodimentStateRecord , EmbodimentOutcomeRecord , embodiment-linked TransitionRecord pointers, Layer9RewardTrace where enabled, and safe-hold events.	Prevents overclaiming and illegal command execution.
CardGenerator	Supporting	Converts transition summaries into environment, rollout, stress, safety, reward, and reproducibility cards.	Markdown/JSON cards for Layer 10 and thesis appendix.	Fails if cards lack config hashes, seeds, limitations, or schema versions.

The recommended implementation order is EnvironmentConfig Registry, SessionBuilder, ReplayCursor, StateAssembler, PolicyAdapter, ActionValidator, ActionApplier, EvidenceRequestManager, RewardEngine, TransitionLogger, PolicyComparisonHarness, EnvironmentValidator, then the PolicyUpdateHook when bandit/RL training is enabled, then StressCoupler, RegimeCoupler, EmbodimentGateway, and CardGenerator. Stress, regime, policy-update, and embodiment coupling should not be implemented before clean deterministic replay and frozen-policy evaluation work.

13.18 Output contracts and record schemas

Layer 7 must output auditable records. Summary metrics are not enough because policy behavior must be traceable to individual transitions.

Output	Description	Main consumer
--------	-------------	---------------

TransitionRecord	One record per environment step: state hash, selected policy action, optional source IHARQ action token, validated Layer 7 action, legality, consequence, next state, reward, done flag, stress state, regime state, embodiment response, safety event.	Layers 6, 9, 10.
ClosedLoopSessionRecord	Session-level metadata: dataset, subject, policy, stress profile, seed, replay mode, episode length, configuration hashes.	Layer 10 and reproducibility appendix.
EpisodeSummaryRecord	Aggregate performance for one rollout/session, including global/local ablation IDs, metric dictionary ID, matched-comparison key, and diagnostic-only flag where applicable.	Layer 10 and thesis result tables.
PolicyRolloutRecord	Full behavior of one policy under one session and stress profile, including frozen/update mode and claim boundary.	Layer 6, ablation scripts, Layer 10.
RewardTraceRecord	Reward and cost decomposition over time.	Layer 6 bandit/RL modules and Layer 10 dashboard.
EnvironmentDiagnosticRecord	Validation warnings, failed transitions, invalid actions, reward anomalies, leakage checks, unmatched comparison warnings, missing Layer 9 safety-event links, missing reward-trace links when claimed, and mismatched demo-manifest identifiers.	Debugging and reproducibility package.
SafetyEventRecord	Unsafe states, blocked actions, wrong accepted actions, over-assist events, stop-loss triggers, safe-hold events, unsupported-command blocks, and Layer 9 embodiment safety events when active.	Layer 0, Layer 9, Layer 10, thesis discussion.
PolicyUpdateTrace	Optional artifact emitted only when simulation-authorized contextual-bandit/RL updates are enabled: policy identity, policy family, context ID, state summary, action, reward, next-state ID, update step, exploration mode, training-or-evaluation mode, frozen-policy flag, simulator ID, seed, configuration hash, matched-comparison key, and simulation-only boundary.	Layer 6, Layer 10, ablation scripts, and reproducibility package.

Layer 7 evaluation summary	Run-level record containing global/local ablation IDs, policy family, reward version, stress profile, embodiment mode, matched-comparison status, metric dictionary ID, dashboard export ID, negative-result note, and claim-evidence pointer.	Evaluation scripts, Layer 10, thesis results.
----------------------------	--	---

Minimum TransitionRecord fields should include `transition_id`, `session_id`, `step_index`, `dataset_id`, `subject_id`, `trial_id`, `window_id`, `policy_name`, `policy_version`, `state_hash`, `source_iharq_action`, `policy_action`, `validated_readiness_action`, `simulator_command`, `action_source`, `action_probability`, `previous_regime`, `previous_stress`, `accepted_prediction`, `correct_if_accepted`, `evidence_request_count`, `delay_cost`, `evidence_request_cost`, `combine_cost`, `fallback_cost`, `defer_cost`, `no_action_cost`, `wrong_accept_cost`, `unsafe_cost`, `late_assist_cost`, `over_assist_cost`, `embodiment_cost`, `reward`, `global_ablation_id`, `local_ablation_id`, `matched_comparison_key`, `metric_dictionary_id`, `policy_update_mode`, `policy_update_trace_id` when applicable, `next_regime`, `next_stress`, `done`, `reason_code`, `embodiment_state_id` when active, `safety_event_id` when active, `layer9_reward_trace_id` where enabled, `demo_manifest_id` where available, `config_hash`, `schema_version`, and `random_seed`.

Layer 7 should save records in simple reproducible formats. CSV is acceptable for small summary files; Parquet is preferred for large transition logs; YAML/JSON should be used for configurations.

```
results/layer7_closed_loop/
|-- configs/
|   |-- environment_config.yaml
|   |-- reward_config.yaml
|   |-- action_config.yaml
|   |-- evidence_config.yaml
|   |-- episode_config.yaml
|   '-- policy_comparison_config.yaml
|-- sessions/
|   |-- closed_loop_sessions.csv
|   '-- session_manifest.csv
|-- transitions/
|   |-- transitions_clean.parquet
|   |-- transitions_stress_delay.parquet
|   |-- transitions_stress_drift.parquet
|   '-- transitions_combined_stress.parquet
|-- rewards/
|   |-- reward_traces.csv
|   '-- reward_decomposition.csv
|-- policy_updates/
|   |-- policy_update_traces.parquet
|   '-- update_disabled_audit.csv
|-- evaluation/
|   |-- matched_comparison_report.csv
|   |-- metric_dictionary_snapshot.yaml
```

```

|   |-- negative_result_notes.csv
|   '-- claim_evidence_entries.json
|-- summaries/
|   |-- episode_summary.csv
|   |-- policy_comparison.csv
|   |-- safety_events.csv
|   '-- stress_sensitivity.csv
|-- diagnostics/
|   |-- validation_report.json
|   '-- invariant_checks.json
'-- cards/
    |-- closed_loop_environment_card.md
    |-- policy_rollout_card.md
    |-- stress_conditioned_policy_card.md
    |-- safety_event_card.md
    '-- reward_configuration_card.md

```

Every record should include schema version, configuration hash, random seed, environment version, policy version, reward version, stress profile identifier, embodiment mode, global/local ablation ID where evaluated, matched-comparison key, metric dictionary ID, training/evaluation mode where relevant, frozen-policy flag where relevant, claim boundary, diagnostic-only flag where relevant, and run timestamp.

13.19 Metrics and evaluation design

Layer 7 must evaluate consequence-aware readiness behavior. After the Ablation and Evaluation Architecture merge, Layer 7 owns the simulated closed-loop evaluation slice for global A10–A11 and participates in A12–A13 when StressForge and embodiment are active. It must preserve the distinction between training/adaptation runs and frozen-policy held-out evaluation. Training curves can diagnose learning, but thesis claims should be based on frozen-policy, matched, held-out simulation results.

Core metrics include total reward, mean reward per step, episode return, wrong-accept rate, unsafe-event rate, task-success rate, mean decision delay, late-assist rate, fallback rate, evidence-request rate, over-request rate, no-action rate, coverage, effective coverage, stress degradation, regime sensitivity, and matched-cost or matched-reward deltas against non-learning policies.

Safety-oriented metrics include high-confidence wrong-accept rate, unsafe-block precision, unsafe-block recall, stop-loss activation rate, stop-loss compliance, wrong accept under drift, wrong accept under fatigue-like degradation, wrong accept under dropout, unsafe event under combined stress, and safe-hold burden. Policy-learning metrics include reward/regret curve, cumulative reward, episode return, action distribution, exploration rate, policy stability, generalization across subjects, generalization across held-out sessions, and degradation under held-out stress profiles. For contextual-bandit and RL experiments, Layer 7 must report training/adaptation curves separately from frozen-policy evaluation curves, and final policy comparisons should prioritize held-out frozen-policy results over in-training reward curves. If Layer 9 is active, embodiment-aware metrics include endpoint error, successful-assist rate, over-assist rate, under-assist rate, safe-hold rate, embodied wrong-command rate, command-timeline consistency, and optional reward-term decomposition.

Baselines should include accept-all decoder, confidence threshold, entropy/margin rejection, selective-prediction baseline, IHARQ-lite, IHARQ + RegimeRisk, learning-to-defer, supervised

Adaptive-IHARQ, contextual bandit, RL policy, frozen non-learning policies, shuffled-reward or random-policy controls where useful, and oracle diagnostic policy. Success criteria are not one-dimensional. A policy is stronger only if it improves the declared trade-off among wrong accepts, reward, delay, fallback burden, evidence burden, unsafe events, stress degradation, and claim-safe interpretability under matched conditions. A policy that achieves low wrong-accept rate by never acting should be reported as conservative or low-coverage, not as universally superior.

Layer 7 comparisons must be matched by dataset, subject, session, replay order, trial/window identity where feasible, split, model family, seed, calibration ID, IHARQ configuration, RegimeRisk configuration, action vocabulary, legality rules, evidence-request budget, reward configuration, cost configuration, stress profile, stress schedule, stress seed, simulator mode, embodiment mode, and policy frozen/update mode. If any required match fails, the result must be labeled diagnostic-only and cannot support a claim-evidence matrix row except as a limitation or failure note.

Failure interpretation must be explicit. High reward with high unsafe events indicates reward mis-specification. Low wrong-accept with high no-action or fallback rate indicates excessive conservatism. Strong clean performance with severe stress degradation indicates lack of robustness. Bandit/RL improvement that disappears under held-out subjects, held-out sessions, held-out stress seeds, or frozen-policy evaluation indicates simulator overfitting or update leakage. Policy ranking that changes under small reward-weight changes indicates reward sensitivity. Every such result should create a negative-result note linked to run ID, global ablation ID, matched-comparison key, reward configuration, dashboard export, and claim boundary.

13.20 Ablation framework

Layer 7 should enforce a ladder of matched ablations. Its local labels are retained for implementation convenience, but dashboard pages, thesis tables, and export manifests should also expose the global A0–A13 IDs defined by the Ablation and Evaluation Architecture.

Variant	Description	Purpose
A0 / L7-A0 decoder accept-all	Always accepts decoder prediction.	Naive baseline and forced-classification reference.
A2 / L7-A1 confidence threshold	Accepts only above threshold.	Simple registered-threshold baseline.
A3 / L7-A2 selective prediction	Uses entropy, margin, calibration, or conformal rejection.	Stronger uncertainty/selective-prediction baseline from Layer 3.
A5 / L7-A3 IHARQ-lite	Rule-based <code>accept/wait/request_evidence/combine_evidence/fallback/unsafe</code> logic with display aliases kept separate.	Interpretable evidence baseline from Layer 4.
A7 / L7-A4 IHARQ + RegimeRisk	Adds temporal trust context from Layer 5.	Tests value of RegimeRisk under matched replay and stress schedules.
A9 / L7-A5 supervised Adaptive-IHARQ	Learned policy from declared proxy labels.	Tests learned readiness actions under proxy supervision.

A8 / L7-A6 learning-to-defer	Routes uncertain cases to fallback/proxy.	Tests deferral baseline.
A10 / L7-A7 contextual bandit	Learns immediate-reward action in simulation and emits <code>PolicyUpdateTrace</code> whenever updates are enabled.	Tests online adaptation under declared reward while separating training/update runs from frozen evaluation.
A11 / L7-A8 RL policy	Learns delayed-consequence policy in simulation and emits <code>PolicyUpdateTrace</code> for simulation-authorized updates.	Tests Stretch C environment only, with update traces and frozen-policy held-out evaluation kept separate.
A12–A13 / L7-A9 full stress + embodiment	Adds StressForge and Layer 9 adapter.	Tests full Stretch C demonstration under stress-proxy and embodiment-proxy boundaries.

Required layer-removal ablations include removing RegimeRisk, removing evidence request, removing combine behavior, removing fallback, disabling unsafe blocks, removing stress profiles, replacing learned policy with non-learning policy, disabling policy updates during frozen evaluation, removing reward sensitivity, removing embodiment, and replacing learned update with random or oracle diagnostic controls where useful. These ablations answer whether temporal trust, evidence accumulation, fallback routing, stress testing, reward assumptions, policy learning, and embodiment coupling add measurable insight or only complexity.

Each Layer 7 ablation must save global ablation ID, local ablation ID, policy family, policy frozen/update mode, matched-comparison key, reward/cost configuration ID, stress profile ID, stress schedule ID, embodiment mode ID, metric dictionary ID, `TransitionRecord` pointers, `EpisodeSummaryRecord` pointers, dashboard view ID, card/report pointer, claim-boundary statement, and negative-result note when applicable.

13.21 Validation, sanity checks, and reproducibility

Simulation layers are academically risky if they are arbitrary, leaky, or unfair. Layer 7 therefore must run invariants before results are accepted.

Invariant	Meaning
No future leakage	Policy cannot see future labels, future correctness, or future windows except through declared request logic.
Reward consistency	Same state/action/outcome under same seed and reward configuration gives same reward.
Replay consistency	Same session configuration gives same replay order.
Action legality	Invalid actions are blocked or mapped to a safe fallback and logged.
Maximum request bound	Evidence requests cannot exceed declared maximum.
Stress reproducibility	Same stress seed, profile, and <code>StressScheduleRecord</code> reproduce the same stress timeline.

Policy fairness	All policies are evaluated under matched data, split, session order, stress, seed, budget, reward, update mode, and embodiment conditions.
Logging completeness	Every environment step has a <code>TransitionRecord</code> .
Update trace completeness	Every simulation-authorized bandit/RL policy mutation has one <code>PolicyUpdateTrace</code> ; frozen-policy evaluation has no mutations and records the disabled-update mode when audited.
Claim provenance	Every result is tagged as simulation-derived, stress-derived, proxy-derived, or embodiment-proxy-derived.
Evaluation lineage	Every thesis-ready result has a global/local ablation ID, matched-comparison key, metric dictionary ID, dashboard export ID, and claim-evidence pointer.

Sanity-check experiments should include a perfect oracle policy, random policy, accept-all policy, always-wait policy, always-fallback policy, no-stress versus stress comparison, zero wrong-accept penalty, and very high wrong-accept penalty. Expected behavior should be documented: the oracle should outperform non-oracle policies, random should perform poorly, accept-all should have high coverage but high wrong-accept risk, always-wait should accumulate delay cost, always-fallback should reduce wrong accept but increase fallback cost, stress should degrade or alter behavior, zero wrong-accept penalty should make policies aggressive, and very high wrong-accept penalty should make them conservative.

A minimum reproducibility command should be documented as:

```
python run_layer7_closed_loop.py \
  --environment configs/layer7/environment_config.yaml \
  --reward configs/layer7/reward_config.yaml \
  --policy configs/layer7/policy_comparison_config.yaml \
  --stress configs/layer8/stress_clean.yaml \
  --seed 42
```

The reproducibility package must contain environment configuration, reward configuration, stress configuration, `StressProfileRecord` and `StressScheduleRecord` paths where stress is active, policy configuration, policy-update configuration when used, session manifest, transition records, optional `PolicyUpdateTrace` files, episode summaries, policy-comparison tables, matched-comparison reports, metric-dictionary snapshot, negative-result notes, claim-evidence entries, sanity-check report, dashboard export, closed-loop environment card, and command manifest for each experiment.

13.22 Dashboard and thesis artifacts

Layer 7 is dashboard-ready only if Layer 10 can render its behavior without reverse engineering. Required views include closed-loop session timeline, policy comparison view, matched-comparison view, evidence-request view, stress-conditioned view, RegimeRisk-aware view, reward decomposition view, safety-event view, training-versus-frozen-evaluation view, negative-result view, and embodiment view.

Layer 7 should generate or feed the following cards: closed-loop environment card, policy roll-out card, policy-update trace card when bandit/RL updates are enabled, stress-conditioned

controller card, safety event card, reward configuration card, and reproducibility card. Each card should state assumptions, state visibility, action space, reward configuration, stress profile, stress schedule, seed, schema version, global/local ablation IDs, matched-comparison key, metric dictionary ID, supported claims, unsupported claims, negative-result notes where relevant, and limitations.

Recommended thesis figures include a Layer 7 state-action-transition diagram, closed-loop session timeline, policy-comparison architecture, stress-coupled transition diagram, embodiment-gated action diagram, and `TransitionRecord` schema diagram.

13.23 Failure modes and defensive design

Layer 7 has several specific risks.

Simulation arbitrariness. The environment may define consequences without enough justification. The defense is transparent reward tables, deterministic replay baselines, reward sensitivity, and explicit assumption cards.

Policy overfitting to the simulator. Bandit or RL policies may exploit simulation artifacts. The defense is comparison against simple baselines, held-out subjects, held-out sessions, held-out stress profiles, multiple stress seeds, and honest simulation-only framing.

Unlogged policy mutation. A contextual-bandit or RL policy may update during what is supposed to be a frozen evaluation, or it may update without an auditable trace. The defense is an explicit frozen-policy flag, disabled update hooks during held-out evaluation, mandatory `PolicyUpdateTrace` emission for simulation-authorized updates, and failure of any run with mismatched update mode.

True-label leakage. A policy may accidentally see information that would not be available at decision time. The defense is a strict separation between observable state and hidden state, plus state-field audits.

Reward domination. A conclusion may depend entirely on arbitrary cost weights. The defense is reward sensitivity analysis and reporting of unstable policy rankings.

Unfair stress coupling. Different policies may see different stress schedules. The defense is fixed stress seeds and matched `StressProfileRecord`/`StressScheduleRecord` pairs for every policy comparison.

Embodiment overclaiming. The simulated embodiment branch may be presented as clinical validation or real assistive-device readiness. The defense is adapter-based wording, proxy labels, Layer 0 claim boundaries, Layer 9 `Layer9EmbodimentDemoManifests`, safety-event logs, reward-trace warnings, and explicit statement that Layer 9 is optional simulation owned by the embodiment layer, not by the closed-loop environment.

Excessive complexity. The environment may become too complex before the core replay works. The defense is staged readiness levels and a minimum deterministic implementation before Stretch C features.

Unmatched simulation comparison. A policy may appear superior because the replay order, reward version, update mode, stress seed, or embodiment mode changed. The defense is a mandatory matched-comparison report; unmatched results are diagnostic-only and cannot support the claim-evidence matrix except as limitations.

Hidden negative result. A bandit, RL, or stress-conditioned policy may fail, overfit, over-abstain, or increase unsafe events. The defense is mandatory negative-result notes linked to run

ID, ablation ID, reward configuration, dashboard export, and claim boundary.

13.24 Implementation roadmap and readiness levels

Layer 7 should be implemented in phases.

Phase	Goal	Exit condition
L7-0 contract design	Define state, action, transition, reward, logging, and validation contracts.	Mock transition validates and saves.
L7-1 clean deterministic replay	Replay public EEG windows/trials with accept-all, threshold, and IHARQ-lite policies.	Transition and episode records exist for clean replay.
L7-2 evidence dynamics	Implement <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , delay, and max request count.	Evidence-request, combine-evidence, and delay metrics are generated with canonical action-token lineage.
L7-3 RegimeRisk coupling	Include <code>RegimeStateRecord</code> and log regime-aware decisions.	Stable versus risky regime comparisons exist.
L7-4 policy comparison harness	Run threshold, IHARQ-lite, learning-to-defer, and Adaptive-IHARQ under matched conditions.	Policy-comparison table and rollout cards exist.
L7-5 StressForge integration	Evaluate policies under declared <code>StressProfileRecord</code> / <code>StressScheduleRecord</code> pairs.	Stress-sensitivity curves exist.
L7-6 contextual bandit environment	Enable immediate-reward adaptation in declared simulation with trace-safe updates.	Reward/regret curves, <code>PolicyUpdateTrace</code> files, disabled-update audits, and frozen-policy comparisons exist.
L7-7 RL environment	Enable delayed-consequence learning in bounded simulation with trace-safe updates.	Episode returns, action distributions, <code>PolicyUpdateTrace</code> files, frozen-policy evaluation curves, and safety metrics exist.
L7-8 embodiment gateway	Connect readiness actions to the Layer 9 static-replay/MyoSuite/OpenSim adapter path.	<code>EmbodimentStateRecord</code> , <code>EmbodimentOutcomeRecord</code> , and embodiment cost terms exist when Layer 9 is active.
L7-9 dashboard and cards export	Make Layer 7 inspectable and thesis-ready.	Environment, rollout, stress, safety, reward, and reproducibility cards exist.

Readiness levels should be used to limit claims. L7-RL0 means Layer 7 is only an idea. L7-RL1 means scope and claim boundaries are stated. L7-RL2 means state/action/transition/reward schemas exist. L7-RL3 means clean replay works. L7-RL4 means evidence dynamics work. L7-RL5 means RegimeRisk state is included. L7-RL6 means matched policy comparison works. L7-RL7 means StressForge profiles and schedules alter sessions reproducibly. L7-RL8 means bandit/RL can train or evaluate inside the environment with simulation-authorized updates

logged through `PolicyUpdateTrace` and frozen-policy evaluation kept separate. L7-RL9 means Layer 9 adapter can receive readiness-gated commands. L7-RL10 means metrics, ablations, cards, limitations, and reproducibility package are complete.

Layer 7 should not support major thesis claims before L7-RL6. It should not support Stretch C policy-learning claims before L7-RL8. It should not support embodiment-related claims before L7-RL9.

13.25 Layer 7 thesis evidence produced

Layer 7 produces the evidence that the architecture can evaluate readiness decisions as sequential policies rather than isolated predictions. Its thesis evidence includes transition-level logs, reward and cost curves, optional `PolicyUpdateTrace` files for simulation-authorized bandit/RL updates, policy-comparison tables, matched-comparison reports, metric-dictionary snapshots, negative-result notes, claim-evidence entries, stress-sensitivity plots, evidence-request/delay trade-off plots, safety-event tables, RegimeRisk-conditioned behavior summaries, embodiment-gated command summaries when active, environment cards, policy rollout cards, and reproducibility manifests.

The strongest defensible conclusion from Layer 7 is not that IHARQ BenchGuard is clinically ready. The strongest defensible conclusion is that IHARQ BenchGuard provides a transparent, reproducible, simulation-bounded, matched-comparison-aware environment for testing how readiness policies behave when their decisions have declared temporal, safety, fallback, stress, policy-learning, reward, and optional embodiment consequences.

14 Layer 8: StressForge-Lite Stress Generator

14.1 Problem solved by Layer 8

A reliability architecture should not only be evaluated on clean offline conditions. Clean offline evaluation can make a readiness system look stronger than it is because the evidence stream is complete, timing is idealized, decoder confidence is unperturbed, the simulated action response is not weakened, and temporal degradation is absent. StressForge-Lite solves this bottleneck by generating controlled, declared, bounded, and reproducible stress conditions so that IHARQ, RegimeRisk, Adaptive-IHARQ, learning-to-defer, contextual bandit, reinforcement-learning policies, Layer 7 simulated closed-loop sessions, Layer 9 embodiment proxies, and Layer 10 dashboard artifacts can be evaluated under explicit difficulty.

The thesis role of Layer 8 is to turn robustness from a vague statement into a measurable, auditable, and claim-bounded artifact. The layer allows the thesis to report not only clean performance, but also where performance degrades, which modules detect degradation, which policies remain stable, which actions become unsafe or costly, and which stress families expose the system’s failure envelope. This directly supports the Stretch C contribution that reliability-aware readiness control should be evaluated under declared uncertainty, delay, dropout, noisy intent, temporal drift, fatigue-like degradation, weakness, and combined stress rather than only under ideal offline replay.

Layer 8 protects the thesis against several criticisms:

- the criticism that the architecture is merely a clean-condition decoder benchmark;

- the criticism that IHARQ, RegimeRisk, Adaptive-IHARQ, deferral, bandit, or RL behavior is only useful when evidence is easy;
- the criticism that stress evaluation is ad hoc, hidden in preprocessing, or not reproducible;
- the criticism that the dashboard shows robustness plots without traceable stress records;
- the criticism that simulated embodiment success is being overclaimed as clinical or real-device validation.

If Layer 8 were removed, IHARQ BenchGuard could still report decoder metrics, calibration metrics, IHARQ decisions, regime-risk scores, policy outputs, simulated transitions, and embodiment proxy outputs. However, it could not defend any systematic robustness claim under delayed decisions, missing evidence, noisy intent, weak simulated response, drift, fatigue-like degradation, or combined stress. It also could not produce stress sensitivity curves, stress cards, policy stress-survival tables, stress-conditioned failure taxonomy, or matched-stress ablations.

For MERGE_RECORD_22, Layer 8 locally owns the StressForge robustness-evaluation slice of the global A0–A13 ablation ladder. Its primary ablation ownership is A12, with explicit participation in A13 when Layer 9 embodiment consequences are evaluated under stress. This does not turn Layer 8 into a detached global evaluator; it means that stress schedules, matched-stress comparison reports, stress-conditioned metric-dictionary entries, Layer 8 `EvaluationOutcomeRecords`, negative-result notes, dashboard/export lineage, and claim-evidence entries are produced closest to the stress generator while remaining visible to the system-level Ablation and Evaluation Architecture.

Claim-safety rule: Layer 8 may claim controlled simulated robustness evaluation, stress-conditioned policy comparison, failure-envelope discovery, stress sensitivity analysis, and reproducible stress-card reporting. It must not claim real clinical robustness, real patient fatigue modeling, real electrode failure validation, real device safety, real rehabilitation benefit, or deployment-ready BCI-FES control.

14.2 Engineering responsibility, scope, and boundaries

The engineering responsibility of Layer 8 is to generate declared stress profiles, stress schedules, injection parameters, per-step stress states, stressed record views, stress-conditioned metrics, sensitivity curves, stress cards, and failure-taxonomy entries. It must do this through explicit interfaces rather than by silently modifying raw datasets, labels, train/test splits, decoder training code, policy training code, or embodiment simulator internals.

Layer 8 is a cross-cutting stress infrastructure layer. It is downstream from public-data anchoring, decoder prediction, calibration, uncertainty, IHARQ evidence verification, RegimeRisk temporal trust monitoring, adaptive readiness policies, and the Layer 7 simulated closed-loop environment. It is upstream of stress-conditioned policy evaluation, RegimeRisk stress response analysis, embodiment stress demonstration, dashboard views, stress cards, ablation tables, and failure taxonomy.

In scope. Layer 8 includes stress-family definition, stress-profile registry, intensity ladder, schedule generation, seed-controlled perturbation, injection adapters, immutable-clean-record preservation, stressed-view creation, stress-state logging, consistency checks, matched-seed stress comparison across policies, stress-conditioned metric collection, single-stressor experiments, curated combined-stress scenarios, dashboard-ready exports, and stress-card generation.

Out of scope. Layer 8 does not train EEG decoders, create new clinical labels, change true evaluation labels, alter train/test splits, silently recalibrate on test labels, choose the best model

under stress, replace Layer 7 transition logic, implement MyoSuite/OpenSim internals, optimize clinical treatment strategies, or prove real-world patient safety. It is primarily an evaluation and robustness-exposure layer, not a hidden model-improvement method.

Core thesis modules. The core implementation should include the stress registry, scenario builder, schedule engine, injection adapter, consistency guard, run orchestrator, metric collector, StressProfileRecord writer, and Layer 8 evaluation-outcome writer for A12 stress evidence.

Supporting modules. Supporting modules include stress-card generation, dashboard exporters, failure-taxonomy writer, configuration hashing, matched-seed manifest generation, and stress-run validation reports.

Optional Stretch C modules. Optional modules include richer combined-stress scenario scripts, held-out stress profiles for policy generalization, stress-aware RegimeRisk replay, embodiment-specific weakness/delay stress, and stress-conditioned RL reward ablations.

Future-work modules. Future work includes real online BCI stress acquisition, real electrode-quality monitoring, clinical fatigue labels, patient-specific musculoskeletal weakness modeling, hardware-in-the-loop delays, and deployment safety testing. These must not be presented as completed thesis evidence unless actual data and validation are obtained.

14.3 Layer 8 identity and module status table

Identity field	Layer 8 specification
Formal name	Layer 8: StressForge-Lite Stress Generator.
Architectural role	Controlled stress-condition generator for robustness, safety-envelope, failure-mode, and stress-sensitivity evaluation.
Scientific role	Tests whether the reliability architecture behaves sensibly when clean offline assumptions are violated by declared simulated stress.
Engineering role	Generates stress profiles, stress schedules, injection records, stressed views, per-step stress states, stress-conditioned outputs, and reproducibility artifacts.
Primary upstream records	DatasetRecord, WindowRecord, PredictionRecord, CalibrationRecord, UncertaintyRecord, IHARQDecisionRecord, RegimeStateRecord, ReadinessContextRecord, PolicyActionRecord, and TransitionRecord.
Primary downstream consumers	Layer 7 simulated closed-loop readiness environment; Layer 6 policy comparison; Layer 5 RegimeRisk stress response; Layer 9 embodiment adapter; Layer 10 dashboard, cards, and reproducibility layer.
Primary outputs	StressProfileRecord, StressScheduleRecord, intensity maps, stressed record pointers, stress-conditioned metrics, stress sensitivity curves, stress cards, stress-run manifests, stress-failure taxonomy entries, Layer 8 EvaluationOutcomeRecords, matched-stress comparison reports, metric-dictionary entries, negative-result notes, dashboard/export lineage, and claim-evidence entries.

Claim boundary	StressForge-Lite supports simulated robustness evaluation. It does not validate clinical safety or real patient/device behavior.
Implementation status target	Thesis-ready status requires validated stress schemas, runnable single-stressor experiments, curated combined-stress scenarios, matched policy comparisons, stress cards, sensitivity curves, and reproducible manifests.

14.4 Input contracts

Layer 8 receives structured records rather than informal arrays. Every stress run must identify the clean record source, dataset protocol, model, policy, environment, embodiment mode if active, stress profile, random seed, and configuration hash.

Input class	Required inputs	Purpose and boundary
Primary inputs	WindowRecord, PredictionRecord, CalibrationRecord, UncertaintyRecord, IHARQDecisionRecord, RegimeStateRecord, ReadinessContextRecord, PolicyActionRecord, TransitionRecord.	Provide the clean evidence, decision, regime, policy, and transition basis to be stressed. Clean records must remain immutable.
Auxiliary inputs	Dataset ID, subject ID, session ID, split ID, trial/window IDs, model ID, policy ID, environment ID, embodiment mode, action vocabulary, reward configuration.	Preserve traceability and make stress runs comparable across subjects, splits, policies, and environments.
Metadata inputs	Stress-family registry, intensity ladder, schedule type, random seed, configuration file, configuration hash, code version, software environment, run timestamp.	Make stress generation reproducible and auditable.
Evaluation-only inputs	Ground-truth class labels, correctness indicators, clean baseline metrics, clean TransitionRecords, endpoint-error references, failure labels if manually annotated, global ablation ID A12, local Layer 8 stress-ablation ID, matched-comparison group ID, metric-dictionary ID, and claim-evidence ID.	Used only for evaluation after action generation; must not leak future correctness into policy observations or stress scheduling.

Proxy inputs	Simulated weakness parameters, simulated delay parameters, proxy embodiment state, endpoint response, safe-hold flags, over-assist/under-assist proxy.	Used only in declared simulation or embodiment-proxy modes. They must not be described as real patient physiology.
Stretch inputs	Held-out stress profiles, scenario scripts, stress generalization folds, stress-aware RegimeRisk replay settings, RL stress-reward ablation settings.	Support stronger Stretch C experiments after core single-stressor behavior is stable.

Layer 8 may run in three modes:

- **record-only mode**, where it produces stress schedules, `StressProfileRecords`, and stressed views without executing a full simulation;
- **Layer 7 simulation mode**, where it injects stress into repeated readiness sessions and logs stress-conditioned `TransitionRecords`;
- **Layer 9 embodiment-proxy mode**, where it injects embodiment-side delay or weakness into simulated action consequences while keeping EEG/evidence stress separate from musculoskeletal response stress.

14.5 Output contracts

Layer 8 outputs must be directly consumable by downstream layers. Every output must be tied to a stress run ID, seed, profile ID, affected records, metric table, and dashboard/card artifact.

Output artifact	Meaning	Downstream consumer
<code>StressProfileRecord</code>	Formal record of stress family, intensity, schedule, seed, target layer, target field, injection point, claim boundary, and output links, including Layer 9 embodiment-stress injection points when weakness, delay, endpoint noise, command delay, or safe-hold stress is active.	Layers 7, 9, 10; thesis methods appendix.
Stress registry	Versioned list of allowed stress families, parameter ranges, injection targets, guardrails, and validation rules.	Layer 8 runner; Layer 10 dashboard; reproducibility package.

StressScheduleRecord	Per-window or per-step stress value, schedule type, ramp/burst/periodic/stochastic state, and active component stressors, including embodiment-specific schedule fields consumed by Layer 9 L9.10 when active.	Layer 7 transition loop; Layer 9 adapter; metric collector.
Stressed record view	Pointer to a stressed copy or view of a clean record, never an overwrite of the original clean record.	Layer 7 replay; Layer 5/6 stress-aware evaluation; audit reports.
Stress run manifest	Dataset, split, model, policy, environment, stress config, random seed, run status, and output paths.	Experiment controller; reproducibility layer.
Stress-conditioned metrics	Accuracy drop, confidence shift, entropy shift, wrong-accept rate, wait/request/fallback/unsafe rates, reward drop, regret, task success, late-assist rate, endpoint error where available.	Layer 10 dashboards; thesis evaluation chapter.
Stress sensitivity curves	Metric-versus-intensity figures for each stress family and policy.	Thesis figures; dashboard stress view.
Policy stress-survival table	Clean/mild/moderate/severe comparison for IHARQ-lite, Adaptive-IHARQ, deferral, bandit, and RL where implemented.	Policy evaluation; ablation chapter.
Stress failure taxonomy	Failure entries grouped by stress family, intensity, module, reason code, and observed consequence.	Layer 10 failure taxonomy; discussion chapter.
Stress card	Standardized report containing stress purpose, injection point, affected modules, metrics, failures, limitations, and reproducibility fields.	Layer 10 cards; appendix.
Layer 8 EvaluationOutcomeRecord	A12 stress-evaluation record linking stress run, clean baseline, ablation-step ID, matched-comparison ID, metric-dictionary ID, stress metrics, warning flags, diagnostic-only status, dashboard/card/export lineage, and claim-evidence pointer.	Layer 10 ablation page, stress page, claim-evidence matrix, thesis tables.
Matched-stress comparison report	Table or JSON/Parquet report proving that policies, datasets, subjects, seeds, clean baselines, stress schedules, and intensity ladders were matched before a stress robustness comparison is treated as thesis-ready.	Layer 10 ablation validation; reproducibility package.

Stress metric-dictionary entries	Definitions for stress sensitivity, robustness drop, stress survival, stress-conditioned wrong accept, stress-conditioned reward drop, endpoint-error degradation where active, and stress-card completeness.	Metric engine; Layer 10; thesis methods appendix.
Stress negative-result note	Required note when a stressor causes failure, a method survives only a hand-picked schedule, stress matching fails, or a stress-conditioned improvement disappears under held-out stress.	Dashboard limitation view; failure taxonomy; discussion chapter.
Stress claim-evidence entry	Claim-safe linkage from A12 figures/tables/cards to source records, metrics, warnings, and unsupported claims.	Layer 0 claim-safety gate; Layer 10 claim-evidence view.

14.6 StressProfileRecord schema

The central data contract of Layer 8 is the **StressProfileRecord**. Without this record, stress tests become undocumented perturbations. With it, stress becomes reproducible, comparable, dashboard-ready, and academically defensible.

Field group	Required fields
Identity	stress_profile_id, stress_run_id, stress_family, stress_name, version, config_hash, random_seed.
Scope	dataset_id, subject_id, session_id, split_id, model_id, policy_id, environment_id, embodiment_mode.
Intensity	intensity_label, intensity_numeric, parameter_bounds, calibrated_reference, expected_difficulty_level.
Schedule	schedule_type, start_step, end_step, duration, ramp_rate, burst_length, periodicity, probability.
Injection target	target_layer, target_record, target_field, injection_point, pre_injection_value_pointer, post_injection_value_pointer.
Stress semantics	architectural_meaning, affected_modules, expected_failure_signature, claim_boundary.
Safety flags	invalid_trial_flag, impossible_state_flag, unsafe_simulation_flag, no_action_required_flag.
Output links	stress_schedule_id, transition_record_ids, stress_metric_id, stress_curve_id, stress_card_id, evaluation_outcome_id, matched_comparison_id, metric_dictionary_id, negative_result_note_id, claim_evidence_id, and dashboard_export_id.

Minimum schema:

```
stress_profile_id: string
```

```

stress_run_id: string
stress_family: delay | dropout | noisy_intent | weakness | drift |
               fatigue_like | combined
intensity_label: clean | mild | moderate | severe | declared_extreme
intensity_numeric: float
schedule_type: constant | ramp | burst | periodic | stochastic |
               scenario_scripted
random_seed: integer
config_hash: string
target_layer: Layer1 | Layer2 | Layer3 | Layer4 | Layer5 | Layer6 |
              Layer7 | Layer9
target_record: WindowRecord | PredictionRecord | CalibrationRecord |
               UncertaintyRecord | ReadinessContextRecord |
               IHARQDecisionRecord | RegimeStateRecord | PolicyActionRecord |
               TransitionRecord | EmbodimentStateRecord
target_field: string
injection_point: before_policy | after_policy | before_reward |
                 before_embodiment | after_transition
expected_affected_modules: list
claim_boundary: simulated_stress_only
output_links: stress_schedule | stress_metrics | stress_card |
               sensitivity_curve | evaluation_outcome |
               matched_comparison_report |
               metric_dictionary_entry | negative_result_note |
               claim_evidence_entry | dashboard_export

```

14.7 Stress ontology and stress profiles

Layer 8 defines stress through three independent dimensions: stress family, intensity, and schedule. This prevents vague claims such as “noise was added” or “difficult cases were tested.” Every stressor must specify architectural meaning, injection point, parameter family, affected modules, expected failure signature, validation rule, and claim boundary.

Stress type	Architectural meaning	Primary injection target	Expected affected modules
Delay	The decision or assistive action arrives late. This tests the cost of late readiness or late simulated assistance.	Layer 7 transition timing; Layer 9 command application timing.	IHARQ, policy layer, RL reward, late-assist metrics, dashboard.
Dropout	Some windows, channels, model-family outputs, or evidence sources become unavailable.	WindowRecord, evidence feature vector, channel mask, model-family output.	Evidence quality, RegimeRisk, request-evidence behavior, fallback behavior.
Noisy intent	Class evidence becomes more ambiguous, contradictory, or less separable without changing true labels.	PredictionRecord probabilities/logits or declared input-window perturbation.	Decoder outputs, calibration, uncertainty, IHARQ, Adaptive-IHARQ.

Weakness	Simulated musculoskeletal response is weaker, slower, or less effective after a readiness-gated command.	Layer 9 embodiment adapter outcome response.	Embodiment layer, RL reward, endpoint-error metrics, safety proxies.
Drift	Evidence distribution changes gradually or abruptly over simulated time.	Readiness context, confidence distribution, model trust, evidence features.	RegimeRisk, calibration monitoring, policy adaptation, stop-loss warnings.
Fatigue-like degradation	Performance worsens gradually across a session as a proxy stressor.	Latent simulation variable affecting evidence quality or action response.	RegimeRisk, policy layer, StressForge dashboard, failure taxonomy.
Combined stress	Multiple stressors occur together under a declared scenario.	Layer 7 and/or Layer 9 multi-target schedule.	Full system robustness, stress-survival tables, safety envelope, failure taxonomy.

Ontology rule: stress families must not be named only by intuitive labels. Delay is not the same as policy waiting; dropout is not the same as noisy intent; weakness is not the same as noisy neural evidence; drift is not the same as fatigue-like degradation. Each stress family must state where it enters the architecture and how it is measured.

14.8 Intensity ladder and schedule framework

Intensity label	Layer 8 meaning	Guardrail
Clean	No stress; baseline comparison condition.	Must be included for every dataset, model, policy, and environment used in stress evaluation.
Mild	Small degradation expected to be recoverable by reliability logic.	Do not tune all methods only for mild stress.
Moderate	Meaningful degradation that should distinguish robust from fragile policies.	Primary thesis stress level for most stress-conditioned claims.
Severe	Strong degradation used to expose failure envelopes.	Report failures honestly; survival is not required for all methods.

Declared extreme	Upper-bound or break-point stress used to find failure modes.	Must be labeled as stress-envelope exploration, not realistic validation.
------------------	---	---

Schedule type	When to use	Example
Constant	Stress is fixed across the entire simulated session.	Twenty percent evidence dropout for all steps.
Ramp	Stress gradually increases.	Fatigue-like degradation grows from mild to severe.
Burst	Stress appears briefly.	Short dropout burst or noisy-intent interval.
Periodic	Stress repeats with a pattern.	Delayed action every k steps.
Stochastic	Stress occurs with seeded probability.	Random channel dropout with fixed seed.
Scenario-scripted	Stress follows an interpretable session story.	Clean start, drift after midpoint, dropout near the end.

The per-window or per-step realization of a declared stress profile is the `StressScheduleRecord`. It must link `stress_schedule_id`, `stress_profile_id`, `stress_run_id`, `time_index` or `window_id`, `active_stress_flag`, `intensity_value`, `component_stressor_state`, `target_record_id`, `target_field`, `random_seed`, and `schedule_hash` so that every stressed value can be replayed and traced back to its clean source.

The stress phase must also be explicit: training, validation, testing, simulation-only, or embodiment-only. Evaluation stress must not become hidden data augmentation. If stress is used during policy training, the matching validation and held-out stress profiles must be declared separately.

14.9 Stress injection architecture

Stress injection must be modular and transparent. StressForge-Lite must not scatter perturbations across decoder scripts, policy scripts, simulator code, and dashboard notebooks. It should expose a small engine whose submodules can be tested independently.

Submodule	Engineering responsibility	Primary artifact
Stress registry	Defines all stress families, parameter ranges, affected layers, allowed schedules, default guardrails, and claim boundaries.	<code>stress_registry.yaml</code> .
Scenario builder	Combines stress family, intensity, schedule, dataset, split, model, policy, environment, and embodiment settings into a run configuration.	<code>stress_scenario_config.yaml</code> .
Schedule engine	Generates per-step or per-window stress values with fixed seeds and declared temporal pattern.	<code>stress_schedule.parquet</code> .

Injection adapter	Applies stress to selected target records while preserving clean records.	Stressed record view and audit log.
Consistency guard	Checks impossible labels, illegal actions, train/test leakage, invalid simulator states, and missing trace fields.	<code>stress_validity_report.json</code> .
Run orchestrator	Executes clean and stressed runs with matched policies, seeds, and record sources.	<code>stress_run_manifest.csv</code> .
Metric collector	Computes stress-conditioned metrics, degradation tables, robustness rankings, and sensitivity curves.	<code>stress_metrics.parquet</code> .
Stress-card generator	Summarizes stress definition, parameters, outcomes, failures, limitations, and reproducibility.	<code>stress_card.md</code> or <code>stress_card.pdf</code> .
Failure-taxonomy writer	Adds stress-conditioned failure entries with reason codes and affected modules.	<code>stress_failure_taxonomy.csv</code> .
Dashboard exporter	Produces Layer 10-ready tables and figure paths without mutating results.	<code>dashboard_exports/stress/</code> .

Recommended source structure:

```

src/stressforge/
|-- __init__.py
|-- registry.py
|-- scenario_builder.py
|-- schedule_engine.py
|-- injection_adapters.py
|-- consistency_guard.py
|-- stress_runner.py
|-- metrics.py
|-- stress_cards.py
|-- failure_taxonomy.py
|-- dashboard_exporter.py
configs/stressforge/
|-- delay.yaml
|-- dropout.yaml
|-- noisy_intent.yaml
|-- weakness.yaml
|-- drift.yaml
|-- fatigue_like.yaml
|-- combined_stress.yaml
records/stress_records/
|-- stress_profile_records.parquet
|-- stress_schedule.parquet
|-- stressed_record_pointers.parquet
|-- stress_run_manifest.csv
|-- stress_validity_report.json
cards/stress_cards/
figures/stress_sensitivity/

```


14.10 Module dossiers

14.10.1 Submodule 8.1: Stress registry

Purpose and bottleneck. The stress registry prevents stress testing from becoming informal and irreproducible. It lists allowed stress families, parameter ranges, schedule types, injection targets, validation rules, and claim boundaries.

Scientific role. It defines the thesis ontology of simulated difficulty and ensures that delay, dropout, noisy intent, weakness, drift, fatigue-like degradation, and combined stress are separable hypotheses rather than vague perturbations.

Engineering responsibility. Maintain versioned registry files, map stress names to implementation adapters, validate allowed parameter ranges, and expose registry entries to the scenario builder and dashboard.

Inputs. Stress-family definitions, allowed intensity labels, schedule templates, target-layer declarations, parameter bounds, and guardrails.

Outputs. Versioned stress registry, registry hash, registry validation report, and allowed-stress table for the thesis appendix.

Processing logic. Parse registry files; validate each family has family name, target records, parameters, intensity defaults, schedule compatibility, failure signature, and claim boundary; reject incomplete stress definitions.

Dependencies and consumers. Depends conceptually on Layers 3–9 record schemas. Consumed by the scenario builder, injection adapter, consistency guard, metric collector, Layer 10 dashboard, and reproducibility package.

Evaluation and evidence. Evaluation is registry-completeness: every stress family must have parameters, target, schedule, guardrails, and output metrics. Thesis evidence includes a stress ontology table and stress-family appendix.

Failure modes and limitations. Registry entries may be too broad, unrealistic, or under-specified. The defense is to keep the main thesis claims on clean, mild, and moderate declared stress and label severe/extreme stress as failure-envelope exploration.

14.10.2 Submodule 8.2: Scenario builder

Purpose and bottleneck. The scenario builder converts registry definitions into executable stress runs. It solves the bottleneck that policies can be compared unfairly if each policy sees a different stress schedule, seed, or clean record source.

Scientific role. It enforces matched-stress comparisons across IHARQ-lite, threshold baselines, Adaptive-IHARQ, learning-to-defer, contextual bandit, RL, and optional embodiment modes.

Engineering responsibility. Build run configurations that bind dataset, split, model, policy, environment, stress family, intensity, schedule, seed, and output path.

Inputs. Registry entry, dataset/split IDs, subject/session IDs, model/policy IDs, Layer 7 environment configuration, optional Layer 9 embodiment configuration, random seed, and experiment phase.

Outputs. Scenario configuration, run ID, config hash, expected output paths, and stress-run manifest entry.

Processing logic. Expand high-level stress scenario into concrete per-run configuration; assign deterministic seeds; create clean-control and stressed-run pairs; register matched comparison groups; prevent mixing incompatible splits or policy phases.

Dependencies and consumers. Depends on DatasetRecord, model registry, policy registry, environment configuration, and stress registry. Consumed by the schedule engine, run orchestrator, and Layer 10 reproducibility view.

Evaluation and evidence. Scenario reproducibility is evaluated by regenerating identical run manifests from the same config and seed. Thesis evidence includes matched-stress comparison tables.

Failure modes and limitations. Scenario files may accidentally confound stress with model selection or policy tuning. The defense is to log clean and stressed pairs, split IDs, model IDs, policy IDs, and held-out stress profiles.

14.10.3 Submodule 8.3: Schedule engine

Purpose and bottleneck. The schedule engine defines when and how strongly stress is active. It prevents phrases such as “random dropout” from hiding unrepeatable perturbations.

Scientific role. It makes stress exposure measurable through stress duration, intensity distribution, active windows, ramp slopes, burst timing, stochastic probabilities, and scenario scripts.

Engineering responsibility. Generate per-step or per-window stress schedules for constant, ramp, burst, periodic, stochastic, and scenario-scripted stress.

Inputs. Scenario configuration, intensity label, numeric parameters, schedule type, session length, random seed, and optional phase boundaries.

Outputs. StressScheduleRecord, active-stress mask, per-step intensity, schedule hash, and schedule summary.

Processing logic. Create a deterministic schedule; align it to window IDs or transition steps; store active/inactive flags; compute exposure statistics; preserve enough information to replay the schedule exactly.

Dependencies and consumers. Depends on Layer 7 transition indexing or Layer 2/3 window indexing. Consumed by injection adapters, run orchestrator, metric collector, and dashboard.

Evaluation and evidence. Schedule correctness is evaluated by seed reproducibility, exposure counts, intensity-distribution checks, and visual schedule plots. Thesis evidence includes schedule diagrams for representative stress profiles.

Failure modes and limitations. Stochastic schedules may create imbalanced exposure by chance; burst schedules may align with only easy or hard windows. The defense is to report exposure counts and optionally use matched schedule folds.

14.10.4 Submodule 8.4: Injection adapter

Purpose and bottleneck. The injection adapter applies stress without destroying the clean record. It solves the danger that stress perturbations become hidden preprocessing changes.

Scientific role. It creates explicit stressed views that support causal-style comparisons: the clean record source is fixed while the stress condition is changed.

Engineering responsibility. Apply stress to declared target records and fields, create stressed

copies or views, attach stress-state pointers, and preserve pre-injection values through pointers or audit logs.

Inputs. Clean target record, stress schedule, stress parameters, injection point, target field, and consistency-guard rules.

Outputs. Stressed record view, injection audit log, stress-state annotations, and links to `StressProfileRecord`.

Processing logic. Select target record at the correct runtime point; apply delay, dropout, probability smoothing, logit noise, drift, fatigue-like degradation, or embodiment weakness according to adapter rules; save post-injection pointer; reject label-changing or illegal modifications unless explicitly configured as a separate label-noise analysis.

Dependencies and consumers. Depends on record schemas for `WindowRecord`, `PredictionRecord`, `CalibrationRecord`, `UncertaintyRecord`, `ReadinessContextRecord`, `TransitionRecord`, and `EmbodimentStateRecord`. Consumed by Layer 7, Layer 5/6 stress-aware analyses, Layer 9, metric collector, and audit reports.

Evaluation and evidence. Evaluate by clean-record immutability checks, target-field diff reports, per-family sanity tests, and downstream metric traceability. Thesis evidence includes stress audit examples.

Failure modes and limitations. The adapter may change the wrong field, create impossible states, overwrite clean data, or alter labels. The defense is immutable clean storage, pre/post pointers, schema checks, and consistency-guard rejection.

14.10.5 Submodule 8.5: Consistency guard

Purpose and bottleneck. Stress can invalidate experiments if it creates impossible labels, illegal actions, future leakage, or simulator states that cannot be interpreted.

Scientific role. The guard protects stress-conditioned claims from being rejected as artifacts of invalid experimental design.

Engineering responsibility. Validate stress configs, schedules, stressed records, policy visibility, label integrity, split integrity, and embodiment-proxy constraints.

Inputs. Scenario config, registry rules, target record schema, train/validation/test split, policy-observation schema, action legality rules, and optional embodiment constraints.

Outputs. Validity report, rejected-run list, warnings, leakage flags, impossible-state flags, and dashboard-readable quality status.

Processing logic. Check that clean records are not overwritten; true labels are not changed; policies do not see future stress labels or future correctness; stress is matched across policy comparisons; embodiment weakness does not change EEG labels; simulator states remain within declared bounds.

Dependencies and consumers. Depends on Layer 0 claim-safety governance, Layer 1 split definitions, Layer 6 policy visibility contracts, Layer 7 environment invariants, and Layer 9 embodiment constraints. Consumed by run orchestrator and Layer 10.

Evaluation and evidence. Evaluation includes negative tests that deliberately try to violate label integrity, split integrity, future visibility, and illegal action rules. Thesis evidence includes a stress validity checklist.

Failure modes and limitations. The guard may miss semantic invalidity if the stress is

technically legal but scientifically unrealistic. The defense is claim-boundary language and stress cards that state what each stressor does and does not represent.

14.10.6 Submodule 8.6: Run orchestrator

Purpose and bottleneck. The run orchestrator executes clean and stressed runs under comparable conditions. It solves unfair policy comparisons caused by different seeds, schedules, record sources, or stopping rules.

Scientific role. It enables matched stress-conditioned evaluation across baseline thresholds, IHARQ-lite, supervised Adaptive-IHARQ, learning-to-defer, contextual bandit, RL, and optional embodiment variants.

Engineering responsibility. Launch clean baseline runs, single-stressor runs, combined-stress runs, held-out stress runs, and embodiment-proxy stress runs using explicit manifests.

Inputs. Scenario config, clean records, stress schedule, policy config, Layer 7 environment config, optional Layer 9 adapter config, seed, and output directories.

Outputs. Transition logs, stress-run manifest, run status, error logs, metric-ready records, and reproducibility metadata.

Processing logic. For each matched comparison group, load clean records; apply stress through adapters; execute Layer 7 transition loop or record-only evaluation; attach stress state to every step; run all policies under identical schedules; save outputs before metric aggregation.

Dependencies and consumers. Depends on Layers 1–7 for core stress runs and Layer 9 for embodiment-proxy runs. Consumed by metric collector, dashboard, and thesis evaluation scripts.

Evaluation and evidence. Run reproducibility is evaluated by deterministic reruns and manifest comparison. Thesis evidence includes run counts, completed/failed runs, and matched comparison tables.

Failure modes and limitations. Runs may fail due to simulator instability, missing records, or incompatible policy actions. Failures must be logged as failed runs rather than silently removed.

14.10.7 Submodule 8.7: Metric collector

Purpose and bottleneck. Stress evaluation is not useful unless it produces interpretable degradation evidence. The metric collector turns stressed runs into quantitative curves, tables, and failure signatures.

Scientific role. It supports the thesis claim that the architecture’s robustness envelope can be measured through clean-to-stress degradation, policy stress-survival, RegimeRisk response, and simulated consequence metrics.

Engineering responsibility. Compute stress exposure, decoder/calibration degradation, readiness behavior, policy robustness, RegimeRisk response, simulation consequences, embodiment consequences, and reporting completeness.

Inputs. Clean metrics, stressed records, stress schedules, TransitionRecords, PolicyActionRecords, RegimeStateRecord rows, EmbodimentStateRecords, and ground-truth evaluation labels.

Outputs. Stress metrics table, sensitivity curves, robustness rankings, stress-survival table, failure counts, and dashboard exports.

Processing logic. Group metrics by dataset, subject, model, policy, stress family, intensity, schedule, and seed; compute differences from clean baseline; compute confidence/entropy/margin shifts; compute wrong-accept, wait, request, fallback, unsafe, reward, regret, and embodiment metrics; generate intensity curves.

Dependencies and consumers. Depends on Layer 2–7 records, optional Layer 9 records, and Layer 10 plotting/export conventions. Consumed by dashboards, thesis evaluation, and ablation analysis.

Evaluation and evidence. Metric validity is evaluated by recomputation from raw records, clean-baseline agreement, missingness checks, and sensitivity-curve sanity. Thesis evidence includes stress-conditioned tables and figures.

Failure modes and limitations. Aggregates may hide subject-specific failures or rare unsafe events. The defense is to report subject/session breakdowns, failure taxonomy, and event-level logs.

14.10.8 Submodule 8.8: Stress-card generator and failure-taxonomy writer

Purpose and bottleneck. Stress experiments can become difficult to audit if only plots are shown. Cards and taxonomy entries make each stress profile interpretable and reproducible.

Scientific role. They translate technical stress runs into thesis-ready evidence with limitations and claim boundaries.

Engineering responsibility. Generate stress cards, failure entries, dashboard metadata, and appendix-ready summaries.

Inputs. StressProfileRecord, run manifest, metrics, schedule summary, validity report, failure logs, and dashboard figure paths.

Outputs. Stress card, stress-failure taxonomy table, stress-dashboard export, and reproducibility appendix entries.

Processing logic. For each stress profile, write identity, purpose, injection point, affected modules, metrics, observed failures, claim boundary, config path, code path, input records, and output records.

Dependencies and consumers. Depends on metric collector, consistency guard, and Layer 10 reporting conventions. Consumed by dashboard, thesis appendix, and discussion chapter.

Evaluation and evidence. Card completeness is evaluated by required-field coverage. Thesis evidence includes stress cards and failure-taxonomy tables.

Failure modes and limitations. Cards may become cosmetic if they do not link to exact records. The defense is mandatory output links, config hashes, and record IDs.

14.11 Per-stressor specialization

14.11.1 Delay stress

Delay stress tests the cost of late decisions or late simulated assistance. It should be applied in the Layer 7 transition loop and, when embodiment is active, in the Layer 9 command application

step.

Item	Layer 8 requirement
Required parameters	<code>decision_delay_steps</code> , <code>assistive_action_delay_steps</code> , <code>delay_distribution</code> , <code>maximum_delay</code> , <code>schedule_type</code> .
Affected outputs	Late-assist rate, reward penalty, task success, wait/request trade-off, endpoint delay where embodiment is active.
Internal logic	Queue the selected policy action or assistive command for a declared number of steps before application; log both policy decision time and applied-action time.
Dependencies	Layer 7 transition indexing and optional Layer 9 command timing.
Failure modes	Delay can be confused with policy wait/request actions. It can also make episodes incomparable if stopping rules are not fixed.
Guardrail	Policy wait/request is chosen by the policy; external delay is imposed by StressForge and must be recorded separately.

14.11.2 Dropout stress

Dropout stress removes or masks evidence sources such as windows, channels, model-family outputs, or stability features. It tests whether evidence verification, RegimeRisk, and fallback logic are resilient to missing information.

Item	Layer 8 requirement
Required parameters	<code>dropout_target</code> , <code>dropout_probability</code> , <code>dropout_block_length</code> , <code>channel_mask_strategy</code> , <code>window_missingness_strategy</code> .
Affected outputs	Evidence quality score, fallback rate, request-evidence rate, unsafe rate, RegimeRisk dropout label, missingness burden.
Internal logic	Apply masks or missing-view pointers to selected records; preserve original clean records; attach missingness reason code.
Dependencies	Layer 1/2 window and channel metadata; Layer 4 evidence-quality logic; Layer 5 temporal trust features.
Failure modes	Dropout may accidentally remove labels, destroy alignment, or produce unequal exposure across policies.
Guardrail	Raw clean records must remain stored; dropout creates stressed views or stressed copies with explicit masks.

14.11.3 Noisy-intent stress

Noisy-intent stress makes class evidence more ambiguous or contradictory. It may perturb prediction distributions, logits, evidence features, or input windows, but the injection point must be declared.

Item	Layer 8 requirement
Required parameters	<code>noise_target</code> , <code>ambiguity_level</code> , <code>class_confusion_matrix</code> , <code>probability_smoothing</code> , <code>logit_noise_scale</code> .
Affected outputs	Confidence, entropy, margin, high-confidence wrong rate, accepted-action accuracy, IHARQ request/unsafe behavior.
Internal logic	Smooth probabilities, perturb logits, or introduce declared class-ambiguity patterns while keeping true labels unchanged.
Dependencies	Layer 2 prediction outputs, Layer 3 calibration/uncertainty records, Layer 4 IHARQ evidence logic.
Failure modes	Stress can become label noise if true labels are silently changed; probability perturbation can break calibration interpretation if not logged.
Guardrail	Noisy intent affects evidence quality, not the ground-truth evaluation target.

14.11.4 Weakness stress

Weakness stress belongs primarily to the embodiment side. It simulates weaker, slower, or less successful musculoskeletal response after a readiness-gated command. It should not be used to imply real patient weakness validation.

Item	Layer 8 requirement
Required parameters	<code>response_gain</code> , <code>action_slowdown</code> , <code>endpoint_noise</code> , <code>weakness_schedule</code> , <code>affected_body_proxy</code> .
Affected outputs	Endpoint error, success/failure event, over-assist, under-assist, reward, embodiment card.
Internal logic	Modify simulated response gain, action latency, or endpoint outcome after the readiness action is selected.
Dependencies	Layer 9 adapter, Layer 7 transition consequences, optional RL reward terms.
Failure modes	Weakness may be mistaken for noisy EEG intent or real clinical motor weakness.
Guardrail	Weakness stress changes simulated action outcome, not EEG labels or decoder evidence.

14.11.5 Drift stress

Drift stress changes the evidence distribution over time. It is essential for testing RegimeRisk and adaptive policy context because a static threshold may work early but fail later.

Item	Layer 8 requirement
Required parameters	<code>drift_type</code> , <code>drift_start</code> , <code>drift_rate</code> , <code>drift_direction</code> , <code>abrupt_or_gradual</code> , <code>target_feature_group</code> .
Affected outputs	RegimeRisk score, trust scores, stop-loss warnings, calibration degradation, policy adaptation, temporal failure lead time.
Internal logic	Apply gradual or abrupt shifts to declared confidence, feature, trust, or evidence distributions while preserving traceability to clean values.
Dependencies	Layer 3 uncertainty outputs, Layer 5 temporal trust monitoring, Layer 6 policy context, Layer 7 sequential replay.
Failure modes	Artificial drift may be confused with real cross-session shift or with fatigue-like degradation.
Guardrail	Drift must be described as simulated distribution change unless real distribution-shift labels are available.

14.11.6 Fatigue-like degradation stress

Fatigue-like degradation gradually worsens evidence reliability or simulated response across a session. It is a proxy stressor and should be described as fatigue-like, not clinical fatigue.

Item	Layer 8 requirement
Required parameters	<code>degradation_start</code> , <code>degradation_slope</code> , <code>maximum_degradation</code> , <code>affected_evidence_features</code> , <code>recovery_allowed</code> .
Affected outputs	Temporal degradation slope, wrong-accept trend, stop-loss activation, fallback burden, request-evidence burden.
Internal logic	Increase ambiguity, missingness, delay, or simulated response degradation over time according to a declared slope.
Dependencies	Layer 5 temporal trust, Layer 6 adaptive policy context, Layer 7 episode horizon, optional Layer 9 embodiment response.
Failure modes	The stressor can be overclaimed as clinical fatigue or confounded with ordinary drift.
Guardrail	Always use “fatigue-like” unless real fatigue measurements or labels exist.

14.11.7 Combined stress

Combined stress applies multiple declared stressors together. It is valuable for failure-envelope analysis but dangerous if it becomes uncontrolled or uninterpretable.

Item	Layer 8 requirement
Required parameters	Component stressors, component intensities, component schedules, target records, interaction expectation, combined profile ID.
Affected outputs	Policy stress-survival, unsafe/fallback behavior, reward degradation, stress-failure taxonomy, dashboard safety-envelope view.
Internal logic	Compose single-stressor schedules only after each component is validated individually; log all components and their interactions.
Dependencies	Valid single-stressor behavior, Layer 7 transition loop, optional Layer 9 adapter, metric collector.
Failure modes	Failure attribution becomes impossible if all severe stressors are combined without rationale.
Guardrail	Combined stress must be curated, not arbitrary full factorial explosion, unless explicitly labeled as failure-envelope search.

14.12 Combined-stress design protocol

Combined stress should be implemented after single-stressor behavior is established. The default thesis design should use curated scenarios rather than a full combinatorial grid.

Scenario	Components	Purpose
C-S1: Robustness baseline	Mild dropout plus mild noisy intent.	Tests whether evidence verification tolerates moderate uncertainty.
C-S2: Temporal reliability stress	Gradual drift plus fatigue-like degradation.	Tests RegimeRisk and policy adaptation over time.
C-S3: Action consequence stress	Decision delay plus simulated weakness.	Tests whether policies balance timely action and safe fallback in embodiment.
C-S4: Safety envelope stress	Moderate drift plus dropout burst plus noisy intent.	Tests unsafe/fallback behavior under acute difficulty.
C-S5: Declared extreme stress	Severe combined stress.	Finds breaking point and failure taxonomy; not used as main performance claim.

Combined-stress reporting must state whether the expected interaction is additive, multiplicative, compensatory, or exploratory. EEG/evidence stress and embodiment-response stress must remain distinct in the records and cards.

14.13 Dependency logic

Dependency class	Layer 8 dependencies	Mocking or readiness rule
------------------	----------------------	---------------------------

Hard dependencies	Dataset/split identity from Layer 1; prediction records from Layer 2; uncertainty/calibration records from Layer 3 when evidence stress is used; IHARQ or policy records when decision stress is evaluated; Layer 7 transition loop for closed-loop stress.	Cannot run thesis-grade stress evaluation without these records. Minimal unit tests can use synthetic records with the same schema.
Soft dependencies	RegimeRisk records from Layer 5, Adaptive-IHARQ and learning-to-defer from Layer 6, dashboard exports from Layer 10.	Can be mocked or omitted for early Layer 8 development, but required for full Stretch C evidence.
Optional dependencies	Contextual bandit, RL readiness policy, MyoSuite/OpenSim adapter, embodiment endpoint metrics, video/screenshot generation.	Optional Stretch C branches; absence should not block core StressForge evaluation.
Conceptual dependencies	Layer 0 claim-safety governance, immutability of clean records, separation of EEG/evidence stress from embodiment stress, no future leakage to policies.	Cannot be mocked away in thesis writing; these are design constraints.
Data dependencies	Window IDs, trial IDs, labels for evaluation, probability/logit outputs, channel metadata, session order, transition indices, policy action logs.	Development can use reduced sample records, but full results require traceable IDs.
Metric dependencies	Clean baseline metrics, stress exposure counts, wrong-accept rate, wait/request/fallback/unsafe rates, reward/regret, late-assist, endpoint error where active.	Metrics must be recomputable from saved records.
Software dependencies	Python stressforge package, config parser, deterministic random generator, parquet/csv writer, plotting scripts, dashboard exporter.	Can be implemented with simple scripts first; production web infrastructure is unnecessary.

The minimum upstream outputs that must exist before Layer 8 can run in full thesis mode are: dataset/split manifest, prediction records, calibration/uncertainty records, IHARQ or policy action records, Layer 7 transition configuration, stress registry, scenario config, and clean baseline run manifest.

14.14 Cross-layer contracts

Connection	Contract requirement	Reason
------------	----------------------	--------

Layer 1 to Layer 8	Dataset and split identity must be attached to every stress run.	Prevents mixing stress results across incompatible protocols.
Layer 2 to Layer 8	Decoder/model IDs must remain fixed during stress evaluation.	Prevents stress from becoming hidden model selection.
Layer 3 to Layer 8	Calibration and uncertainty records may become stress-aware but must not be silently recalibrated on test labels.	Prevents leakage and unfair recalibration.
Layer 4 to Layer 8	IHARQDecisionRecord action tokens, reason codes, evidence-quality fields, and source-record identifiers must be linkable to Layer 8 stress-run IDs and stressed-view annotations during replay; clean Layer 4 records are not required to contain stress state before Layer 8 runs.	Explains whether accept, wait, request, fallback, or unsafe decisions change under stress without requiring upstream Layer 4 to invent StressForge state.
Layer 5 to Layer 8	RegimeRisk receives stress state only through declared current/past context and may detect drift, dropout, or fatigue-like conditions.	Tests whether temporal trust monitoring reacts correctly without future leakage.
Layer 6 to Layer 8	Policies must be evaluated under identical stress schedules and seeds within a matched comparison group.	Makes Adaptive-IHARQ, deferral, bandit, and RL comparisons fair.
Layer 7 to Layer 8	The closed-loop environment must expose stress state at each transition and must not invent stress internally.	Connects stress to action consequence, reward, and reproducibility.
Layer 9 to Layer 8	Embodiment stress must be declared separately from EEG/evidence stress and must return Layer 9-owned endpoint-error, safe-hold, unsupported-command, safety-event, command-timeline, and demo-manifest links when those outputs are produced.	Avoids confusing weak simulated movement with noisy neural intent and lets Layer 10 trace stress-conditioned embodiment evidence without treating Layer 8 as the simulator owner.
Layer 10 to Layer 8	Dashboard reads stress records, metrics, curves, and cards but does not mutate them.	Maintains reproducibility and auditability.

14.15 Feedforward and loopback behavior

The main feedforward path is:

Clean upstream records + stress registry + scenario configuration + **StressScheduleRecord** generation → stressed views and stress-state logs → Layer 7 and/or Layer 9 execution → stress-conditioned metrics, sensitivity curves, stress cards, and dashboard-ready failure taxonomy.

Layer 8 also supports limited loopback behavior, but only where the loop has a scientific and engineering reason. StressForge must not create arbitrary feedback loops that let policies see future stress outcomes or tune themselves on test labels.

Source	Target	Signal	Trigger	Safety and evaluation rule
Layer 8 schedule engine	Layer 7 transition loop	Current stress state, intensity, active component stressors, delay/dropout/noise flags.	Each transition step or window.	Policy may receive only allowed current/past stress state according to the declared observation schema.
Layer 7 TransitionRecord	Layer 8 metric collector	Stress-conditioned action, reward, wrong accept, fallback, unsafe event, late assist, request burden.	End of transition, episode, or run.	Used for evaluation and metrics; not used to alter past policy decisions.
Layer 8 drift/fatigue schedule	Layer 5 RegimeRisk	Current/past drift or degradation proxy and observed temporal instability.	Rolling-window update.	Used only in declared simulation mode; future true labels are not exposed.
Layer 5 RegimeRisk	Layer 8 stress analysis	Risk score, trust shift, stop-loss warning, regime label under stress.	End of transition or rolling window.	Evaluates whether RegimeRisk reacts to stress; does not change stress schedule unless scenario explicitly allows adaptive stress.
Layer 8 embodiment stress	Layer 9 L9.10 stress embodiment injector and adapter path	Weakness, slowdown, endpoint-noise, action-delay, safe-hold, or command-smoothness parameters.	Before embodiment command application or static replay step.	Must be logged as simulated embodiment proxy, linked to Layer 9 safety/demo artifacts, and not described as real patient weakness.
Layer 9 EmbodimentStateRecord / EmbodimentOutcomeRecord	Layer 8 metric collector	Endpoint error, success/failure event, safe hold, over-assist, under-assist, stress sensitivity, and action delay.	End of embodiment step or episode.	Used for embodiment stress metrics and cards; does not validate real clinical control.

Layer 10 dashboard	Layer 8 records	Read-only inspection request.	User opens stress view or exports card.	Dashboard may filter and visualize but must not mutate stress records.
-----------------------	--------------------	----------------------------------	---	--

An adaptive-stress loop may be introduced only as future work or a clearly declared stress-envelope experiment. In that case, the source, target, adaptation rule, allowed visibility, and evaluation split must be stated before the experiment runs.

14.16 Layer 8 metrics and evaluation architecture

The most important Layer 8 evidence is not just final accuracy under stress. The layer must show how decisions, uncertainty, safety proxies, delays, fallbacks, rewards, regime scores, and embodiment outcomes change as stress intensity increases.

Layer 8’s local evaluation slice is A12 in the global ladder. A12 evidence is thesis-ready only when clean and stressed runs are matched by dataset, split, subject/session, model, policy, seed policy, stress profile, schedule, intensity ladder, evaluation horizon, metric dictionary, and claim boundary. Stress results that fail this matching can still be shown as diagnostic failure-envelope evidence, but they cannot support a robustness-improvement claim.

Local A12 slice	Comparison owned by Layer 8	Required evidence artifact
A12.0	Clean reference condition	Clean baseline run manifest, clean stress profile or explicit no-stress marker, metric dictionary, and dashboard lineage.
A12.1	Single-stressor robustness	Matched clean-versus-stress comparison for delay, dropout, noisy intent, weakness, drift, fatigue-like degradation, or a declared single family.
A12.2	Combined-stress robustness	Curated scenario rationale, component-stressor exposure table, matched schedule report, and stress-failure taxonomy.
A12.3	Held-out stress generalization	Held-out schedule/profile report and negative-result note if the policy only survives development stress.
A12.4	Module-under-stress contribution	Stress without IHARQ, stress without RegimeRisk, threshold-only stress, deferral stress, and contextual-bandit/RL stress where implemented.
A12.5	Stress-to-embodiment bridge	Paired clean/stress Layer 9 links when endpoint error, weakness response, action delay, safe hold, or unsafe-event evidence is reported.

Metric group	Metrics
--------------	---------

Stress exposure	Number of stressed trials/windows/steps, stress duration, intensity distribution, affected records, active component stressors.
Decoder/calibration degradation	Accuracy drop, confidence shift, entropy shift, margin shift, high-confidence wrong rate, calibration shift under stress.
Readiness behavior	Accepted-action accuracy, wrong-accept rate, wait rate, request-evidence rate, fallback rate, unsafe rate, evidence-combine rate.
Policy robustness	Reward drop, regret under stress, action-switching rate, policy stability, threshold/action switching, deferral burden, fallback burden.
RegimeRisk response	Risk-score rise, trust-score shift, lead time before failure, stop-loss activation, regime-label transitions.
Simulation consequence	Task success, late-assist rate, over-assist, under-assist, unsafe-event count, no-action burden, reward contribution.
Embodiment consequence	Endpoint error, command success, safe-hold frequency, weakness response, simulated action smoothness proxy where available.
Reporting completeness	Stress-card completeness, stress config completeness, manifest completeness, figure traceability, reproducibility status.

Evaluation question	Required comparison
Does the method survive clean-to-stress degradation?	Clean versus mild versus moderate versus severe for each major policy and stress family.
Is IHARQ only a threshold method?	IHARQ under stress versus confidence threshold, entropy rejection, margin rejection, longer-window, voting, and ensemble baselines.
Does RegimeRisk matter under stress?	Policy without RegimeRisk context versus policy with RegimeRisk context, especially under drift and fatigue-like degradation.
Does learning help under stress?	IHARQ-lite versus supervised Adaptive-IHARQ versus learning-to-defer versus contextual bandit/RL where implemented.
Does embodiment stress change conclusions?	Closed-loop without embodiment versus MyoSuite/OpenSim proxy stress runs.
Where does the system fail?	Failure taxonomy by stress family, intensity, affected module, reason code, and simulated consequence.

Baselines. Required baselines include clean replay, threshold-only readiness, entropy rejection, margin rejection, longer-window aggregation, majority or probability voting, ensemble disagreement where available, IHARQ-lite without stress-aware context, policy without RegimeRisk context, and fallback/no-action conservative policies.

Ablations. Required ablations include no-stress baseline, single-stressor runs, combined-stress runs, held-out stress profiles where policies learn from stress, stress without RegimeRisk, stress without IHARQ, stress with threshold-only policy, stress with deferral baseline, stress with contextual bandit/RL where implemented, stress with and without embodiment, and stress-card/dashboard disabled only for reproducibility checks. Each A12 ablation must produce or explicitly waive a Layer 8 **EvaluationOutcomeRecord**, matched-stress report, metric-dictionary entry, negative-result note where applicable, dashboard/export lineage, and claim-evidence entry.

Success criteria. Layer 8 is successful if all stress profiles are declared and reproducible; clean records remain immutable; policies can be compared under matched schedules; stress sensitivity curves exist for main metrics; failure modes are logged rather than hidden; RegimeRisk and policies are evaluated under drift/fatigue-like conditions; embodiment stress is clearly separated from evidence stress; and stress cards state limitations.

Failure interpretation. A method failing under severe or declared-extreme stress is not automatically a thesis failure. It may reveal a useful boundary. A method failing under clean or mild stress indicates a stronger architecture problem. A method improving only because it sees future stress outcomes indicates leakage and must be rejected. A method that survives only one hand-picked stress schedule must be downgraded unless it generalizes to held-out schedules.

14.17 Ablation and criticism-defense matrix

Ablation	Question answered
Clean baseline only	What happens if no stress is used?
Single stressor only	Which isolated stressors harm which modules?
Stress without RegimeRisk	Does temporal trust actually help under drift or fatigue-like stress?
Stress without IHARQ	Does evidence verification reduce wrong accepts or fallback mistakes?
Stress with threshold-only policy	Is the architecture just rejection or thresholding?
Stress with deferral baseline	Is deferral enough compared with Adaptive-IHARQ?
Stress with bandit/RL	Does adaptive policy learning improve simulated stress response or overfit the simulator?
Stress with and without embodiment	Does readiness gating matter when simulated action consequences are included?
Held-out stress schedules	Does a learned policy generalize beyond the stress schedule used during development?

Likely criticism	Layer 8 defense
The stress profiles are artificial.	The thesis should explicitly state that they are declared simulated stress profiles and claim controlled robustness evaluation, not real-world clinical validation.
The stress generator unfairly favors one method.	Use matched stress schedules, fixed seeds, same clean records, and stress-card disclosure for every method.

The combined stress results are uninterpretable.	Start with single stressors and add only curated combined scenarios with explicit rationale.
StressForge is only data augmentation.	Separate training augmentation from evaluation stress. Layer 8 is primarily a robustness-evaluation layer.
RL overfits stress schedules.	Use held-out stress profiles, held-out sessions, baseline comparisons, reward ablations, and controller cards.
Dashboard stress plots are cosmetic.	Dashboard must read saved StressProfileRecords and export thesis-ready stress cards and curves.
Embodiment stress implies clinical validation.	Label weakness, delay, endpoint error, and safe hold as simulated embodiment proxies only.

14.18 Dashboard, cards, and reporting requirements

Layer 8 must produce artifacts that Layer 10 can inspect without reverse engineering. The stress dashboard view should include stress overview, sensitivity curves, policy-under-stress comparison, RegimeRisk-under-stress timeline, embodiment stress view, and failure-taxonomy view.

Stress card section	Required content
Stress identity	Stress profile ID, run ID, family, intensity, schedule, seed, version, and config hash.
Purpose	The bottleneck or failure condition tested by this stress profile.
Injection point	Target layer, target record, target field, runtime injection point, and pre/post pointer.
Affected modules	Which layers are expected to respond and which outputs should change.
Metrics	Key stress-conditioned metrics, sensitivity curves, and comparison tables.
Observed failures	Main failure modes, reason codes, unsafe/fallback events, and affected modules.
Claim boundary	Statement that this is simulated stress evaluation, not clinical or real-device validation.
Reproducibility	Exact config path, code path, input records, output records, random seed, environment version, and dashboard export path.

Recommended Layer 10 stress views:

- stress overview table with stress profiles, intensities, seeds, schedules, target layers, and affected records;
- stress sensitivity curves by policy, dataset, subject, stress family, and intensity;
- policy-under-stress comparison for IHARQ-lite, Adaptive-IHARQ, learning-to-defer, contextual bandit, and RL;
- RegimeRisk-under-stress timeline for risk score, trust shift, stop-loss behavior, drift, dropout, and fatigue-like degradation;

- embodiment stress view for endpoint error, simulated weakness response, safe hold, unsupported-command blocks, unsafe events, command timelines, `Layer9RewardTrace` files where enabled, demo-manifest links, and action delay;
- failure-taxonomy view grouped by stress family, intensity, module, reason code, and simulated consequence.

14.19 Reproducibility requirements

Every stress choice must be defined in configuration, seeded, logged, versioned, and carded. The stress result should be traceable from thesis figure back to metric table, run manifest, `StressProfileRecord`, `StressScheduleRecord`, scenario config, registry version, clean record ID, and original dataset split.

Required configuration files:

- `configs/stressforge/stress_registry.yaml`;
- `configs/stressforge/delay.yaml`;
- `configs/stressforge/dropout.yaml`;
- `configs/stressforge/noisy_intent.yaml`;
- `configs/stressforge/weakness.yaml`;
- `configs/stressforge/drift.yaml`;
- `configs/stressforge/fatigue_like.yaml`;
- `configs/stressforge/combined_stress.yaml`;
- `configs/stressforge/matched_comparison_groups.yaml`.

Required output files:

- `records/stress_records/stress_profile_records.parquet`;
- `records/stress_records/stress_schedule.parquet`;
- `records/stress_records/stressed_record_pointers.parquet`;
- `records/stress_records/stress_run_manifest.csv`;
- `records/stress_records/stress_validity_report.json`;
- `metrics/stress_metrics.parquet`;
- `figures/stress_sensitivity/*.pdf`;
- `cards/stress_cards/*.md`;
- `reports/stress_failure_taxonomy.csv`;
- `records/evaluation_outcomes/layer8_stress_evaluation_outcomes.parquet`;
- `reports/matched_comparisons/layer8_matched_stress_comparisons.csv`;

- `metrics/metric_dictionary_layer8_stress.yaml`;
- `notes/negative_results/layer8_stress_negative_results.md`;
- `manifests/claim_evidence/layer8_stress_claim_evidence.json`;
- `dashboard_exports/stress/`.

Seeds should be stored per stress run and per schedule. Code version, package versions, dataset split hash, model ID, policy ID, and environment ID should appear in the run manifest. If GPU nondeterminism affects RL or embodiment experiments, the card must state that exact reproducibility is limited and provide deterministic record-level reruns where possible.

14.20 Readiness levels and implementation roadmap

Level	Meaning for Layer 8	Exit condition
L8-R0 Idea	StressForge-Lite exists only as a concept.	Stress families listed.
L8-R1 Scope defined	Stress families and claim boundaries are documented.	Scope/non-scope section exists.
L8-R2 Contracts defined	StressProfileRecord and StressScheduleRecord schemas exist.	Schemas validate independently.
L8-R3 Single stressors runnable	Delay, dropout, noisy intent, weakness, drift, and fatigue-like profiles can run independently on sample records.	Single-stressor audit logs exist.
L8-R4 Simulation integrated	Layer 7 transitions include stress state and stress-conditioned metrics.	TransitionRecords include stress run IDs.
L8-R5 Policy comparable	All implemented policies can be compared under matched stress schedules.	Policy stress-survival table exists.
L8-R6 Regime/embodiment aware	RegimeRisk and Layer 9 can consume stress state correctly.	Stress-aware regime and embodiment records exist.
L8-R7 Evaluated	Stress sensitivity curves and stress cards exist for selected experiments.	Thesis figures/cards generated.
L8-R8 Reproducible	Stress configs, seeds, manifests, and outputs can be regenerated.	Rerun hash/check report passes.
L8-R9 Thesis-ready	Layer 8 supports claims, limitations, ablations, dashboard views, and failure taxonomy.	Evaluation chapter can cite Layer 8 artifacts.

Recommended implementation order:

1. define stress ontology and registry;
2. define **StressProfileRecord** and **StressScheduleRecord** schemas;
3. implement clean-run baseline manifest;
4. implement single-stressor schedules;

5. connect stress state to Layer 7 **TransitionRecords**;
6. connect stress state to **RegimeRisk** and policy evaluation;
7. implement stress metrics and sensitivity curves;
8. implement curated combined-stress scenarios;
9. connect embodiment stress to Layer 9 adapter;
10. generate Layer 10 dashboard views and stress cards.

14.21 Failure modes and defensive design

Layer 8 failure mode	Why it is dangerous	Required defense
Stress is too vague	The reader cannot know what was actually tested.	Define stress family, intensity, schedule, target, and seed.
Stress is too unrealistic	Reviewers may reject the robustness claim.	Use claim-safe language and separate illustrative extremes from main evaluation.
Stress is hidden in preprocessing	Downstream results become impossible to reproduce.	Keep clean records immutable and save stressed views separately.
Stress changes labels	Evaluation becomes invalid.	Never alter true labels unless the experiment is explicitly label-noise analysis.
Policy sees future stress labels	Creates leakage and inflated performance.	Expose only allowed current/past stress state to policy.
Combined stress is uncontrolled	Failures cannot be attributed.	Use single-stressor baselines and curated combinations.
Stress is tuned per method	Comparisons become unfair.	Use matched schedules and seeds across methods.
A12 matching fails	A stress result may look stronger than it is because clean/stress records, seeds, policies, horizons, or intensity ladders are not matched.	Mark the result diagnostic-only, save a matched-comparison failure report, and attach a negative-result or limitation note.

Embodiment stress is overclaimed	Could imply clinical validation or real assistive-device performance.	Label weakness/delay/endpoint error as simulated embodiment proxy only, require Layer 9 demo-manifest and safety-event links when embodiment stress is reported, and route all dashboard claims through Layer 0 and Layer 10 warnings.
Dashboard mutates results	Results are no longer traceable.	Dashboard must read saved records and export artifacts only.

14.22 Layer 8 thesis evidence

The thesis chapter should use Layer 8 to produce the following evidence:

- a stress ontology table defining the seven stress families;
- clean-to-stress degradation curves for main policies and datasets;
- stress sensitivity curves for accepted-action accuracy, wrong-accept rate, request/fallback/unsafe rates, reward, and RegimeRisk response;
- policy stress-survival tables across clean, mild, moderate, severe, and declared-extreme conditions;
- RegimeRisk response plots under drift and fatigue-like degradation;
- embodiment-proxy stress plots for delay and weakness where Layer 9 is active;
- failure taxonomy entries grouped by stress family, intensity, module, and reason code;
- stress cards in the appendix documenting assumptions, parameters, limitations, seeds, and output paths;
- A12 Layer 8 `EvaluationOutcomeRecords`, matched-stress reports, metric-dictionary entries, negative-result notes, dashboard/export lineage, and claim-evidence entries proving that stress-conditioned claims are matched and bounded.

The correct final interpretation is that StressForge-Lite exposes the robustness envelope of IHARQ BenchGuard under declared simulated stress. It strengthens, but does not clinically validate, the argument that reliability-aware readiness architectures should be judged by their behavior under imperfect evidence, delayed action, temporal degradation, and simulated action consequences.

15 Layer 9: MyoSuite/OpenSim Embodiment Demo

15.1 Layer 9 thesis role and problem statement

Layer 9 is the simulated embodiment bridge of IHARQ BenchGuard. It demonstrates how readiness-gated neural-intent decisions can be translated into declared proxy assistive-action commands and evaluated as simulated musculoskeletal consequences. The bottleneck solved by this layer is that an otherwise strong reliability architecture may remain confined to decoder-level or decision-level metrics. Accuracy, calibration, uncertainty, IHARQ decisions, RegimeRisk labels, policy actions, and StressForge robustness are all necessary, but they do not by themselves show what those decisions would mean for an action-facing assistive interface. Layer 9 makes the downstream consequence visible while preserving strict simulation-only claim boundaries.

The central thesis role of Layer 9 is to support the claim that reliability-aware readiness decisions are not merely abstract classifier filters; they can change the simulated action-level behavior of an assistive system. It contributes to the thesis by converting canonical readiness tokens `accept`, `wait`, `request_evidence`, `combine_evidence`, `fallback`, `uncertain`, and `unsafe` into logged no-action, hold, conservative-assist, blocked-command, or executed-command events inside a bounded embodiment interface without rewriting those source tokens as adapter statuses. The scientific value is therefore not clinical validation. The value is a reproducible proof-of-concept showing how readiness, uncertainty, RegimeRisk, policy learning, and stress exposure affect downstream simulated action outcomes.

Layer 9 protects the thesis against several likely criticisms. It prevents the criticism that the system stops at offline classification and never shows downstream consequences. It prevents the criticism that adaptive readiness policy has no action-level interpretation. It prevents the criticism that StressForge-Lite produces only artificial perturbation tables without showing how those perturbations could affect an embodied loop. It also protects the work from overclaiming by making the embodiment path explicit, proxy-labeled, simulation-only, adapter-based, and reproducibly logged.

If Layer 9 were removed, the core reliability thesis would still be valid through Layers 1–8 and Layer 10, but the architecture would lose its simulated assistive-action consequence layer, the A13 embodiment ablation, the `EmbodimentStateRecord` and `EmbodimentOutcomeRecord` evidence streams, and the clearest bridge between readiness decisions and rehabilitation-oriented future work. Layer 9 is therefore a Stretch C demonstration layer, not a required clinical validation layer and not the central algorithmic contribution.

For `MERGE_RECORD_22`, Layer 9 locally owns the A13 embodiment-proxy evaluation slice of the global A0–A13 ablation ladder. It also participates in A12 when StressForge perturbations are evaluated through embodiment-side outcomes. This local ownership means that Layer 9 must produce embodiment `EvaluationOutcomeRecords`, matched embodiment comparison reports, metric-dictionary entries, negative-result notes, dashboard-compatible export lineage, and claim-evidence entries for A13 claims. It does not make MyoSuite/OpenSim evidence clinical; it makes the simulation-only action-consequence evidence explicit, comparable, and auditable.

Claim-safety rule: Layer 9 is a simulated embodiment proof-of-concept. It does not validate real clinical BCI control, real patient rehabilitation, real exoskeleton safety, real stimulation safety, or direct EEG-to-muscle actuation. It demonstrates only that the proposed readiness architecture can be connected to a simulated musculoskeletal action environment through declared proxy mappings, adapter contracts, stress profiles, safety gates, and logged outcome metrics.

15.2 Layer 9 research identity

The research problem for Layer 9 is that a readiness architecture can reduce unsafe or unreliable decoder-level decisions, but without downstream embodiment analysis it remains unclear how those readiness decisions affect action-level consequences in a simulated assistive setting. The correct research framing is therefore:

Layer 9 evaluates whether readiness-gated decisions reduce simulated unsafe-action events, wrong-accept execution, delay costs, and stress-induced action failures compared with simpler action-execution policies, while explicitly treating all intent-to-command mappings as proxy mappings.

Layer 9 supports the following research questions.

ID	Research question
RQ-L9.1	Can readiness gating reduce simulated unsafe-action events compared with accept-all or confidence-only action execution?
RQ-L9.2	Do IHARQ, RegimeRisk, and adaptive policy decisions improve simulated action-level outcomes under declared proxy mappings?
RQ-L9.3	Does stress exposure, such as delay, dropout, weakness, noisy intent, drift, or fatigue-like degradation, reveal the value of readiness-gated embodiment beyond clean offline metrics?
RQ-L9.4	Can embodiment outcome metrics provide useful simulation-only reward or feedback signals for contextual bandit and reinforcement-learning policies?
RQ-L9.5	Can MyoSuite and optional OpenSim replay be integrated through a simulator-agnostic adapter while preserving reproducibility and claim-safety boundaries?

The corresponding hypotheses are limited and testable.

ID	Hypothesis
H-L9.1	Readiness-gated embodiment will produce fewer unsafe or inappropriate simulated actions than accept-all embodiment.
H-L9.2	RegimeRisk-aware gating will behave more conservatively during unstable or stressed regimes than confidence-only gating.
H-L9.3	StressForge-conditioned embodiment will expose failure modes that are not visible in clean offline decoder metrics.
H-L9.4	Embodiment outcome metrics can support policy comparison as simulation-only reward signals, not as patient-benefit evidence.
H-L9.5	A simulator adapter keeps the architecture valid even if OpenSim is used only for optional replay rather than policy training.

Layer 9 is best framed as a system-integration, evaluation, and reproducibility contribution. It defines the embodiment bridge, adds simulated action-level outcome metrics, produces `EmbodimentStateRecord` and `EmbodimentOutcomeRecord` records plus transition logs, supplies reward terms for simulation-only policy comparison, and generates transparent embodiment cards for Layer 10.

15.3 Layer 9 engineering identity, boundaries, and scope

The primary engineering responsibility of Layer 9 is to receive readiness and policy records, map accepted proxy intent into an allowed simulated command, enforce readiness safety logic, apply the command through a simulator adapter, compute simulated movement outcomes, and log every transition for evaluation, dashboarding, and reproducibility.

The key boundary rule is that Layer 9 must never bypass readiness control. Every simulated movement command must pass through a readiness decision, a safety gate, a declared command mapping, a simulator adapter, an outcome calculator, and a transition logger. Decoder predictions alone must not directly step MyoSuite or OpenSim.

Scope category	Layer 9 responsibility
In scope	Readiness-gated simulated command execution; proxy EEG-intent mapping; simulator adapter contracts; static replay; MyoSuite simple-task embodiment; optional OpenSim replay; stress-conditioned embodiment; outcome and reward computation; <code>EmbodimentStateRecord</code> , <code>EmbodimentOutcomeRecord</code> , and <code>TransitionRecord</code> logging; embodiment-card export.
Out of scope	Clinical validation; real exoskeleton control; real functional electrical stimulation; direct inference of muscle activation from public EEG motor-imagery labels; hidden policy retraining inside the simulator; untracked dashboard recomputation; replacing Layers 4–8 as the core reliability architecture.
Core components	Session controller, adapter interface, proxy command mapper, readiness safety gate, transition logger, outcome calculator, <code>EmbodimentStateRecord</code> writer, and <code>EmbodimentOutcomeRecord</code> writer.
Supporting components	Stress-condition injector, reward bridge, embodiment card exporter, static replay adapter, mapping validator, schema validator, and safety reason-code registry.
Optional / stretch components	MyoSuite stress branch, OpenSim replay adapter, comparative embodiment report, generated screenshots/videos, and advanced reward ablation.
Future-work components	Real-time simulator coupling, richer biomechanical control, clinical subject-specific modeling, hardware-in-the-loop control, and patient-facing validation. These remain outside the current thesis claims.

Expected implementation artifacts include `src/layer9_embodiment/`, `configs/layer9_embodiment/embodiment_config.yaml`, `configs/layer9_embodiment/simulator_config`, `configs/layer9_embodiment/command_mapping.yaml`, `configs/layer9_embodiment/stress_embedding_c`, `configs/layer9_embodiment/safety_rules.yaml`, `configs/layer9_embodiment/reward_config.yaml`, `configs/layer9_embodiment/logging_schema.yaml`, `records/embodiment/embodiment_state_records.p`, `records/embodiment/embodiment_outcome_records.parquet`, `records/embodiment/transition_records`, `records/embodiment/safety_event_log.csv`, `figures/embodiment/`, and `cards/embodiment/embodiment_card.md`. Evaluation-facing implementation artifacts additionally include `records/evaluation_outcomes/layer9_embodiment_evaluation_outcomes.parquet`, `reports/matched_comparisons/layer9_matched_embodiment_comparisons.csv`,

`metrics/metric_dictionary_layer9_embodiment.yaml`, `notes/negative_results/layer9_embodiment_notes`, and `manifests/claim_evidence/layer9_embodiment_claim_evidence.json`.

15.4 Detailed embodiment scope, non-scope, and allowed claims

Layer 9 is in scope when it implements, evaluates, or documents the following items:

- proxy mapping from public EEG motor-imagery labels to simulated intent commands;
- readiness safety gating before any simulator step;
- static replay, MyoSuite simple-task embodiment, stress-conditioned MyoSuite embodiment, and optional OpenSim replay/visualization;
- EmbodimentStateRecord, EmbodimentOutcomeRecord, TransitionRecord, and SafetyEventRecord logging;
- endpoint error, command success, blocked-command rate, unsafe-event rate, no-action/fallback burden, wrong-accept-to-action rate, delay cost, over-assist, under-assist, instability, smoothness proxy, stress sensitivity, and reward decomposition where enabled;
- dashboard-ready embodiment summaries, embodiment cards, `Layer9EmbodimentDemoManifests`, and reproducibility packages;
- A13 MyoSuite/OpenSim demo ablation and related stress-conditioned embodiment comparisons.

Layer 9 is out of scope when it would require or imply:

- clinical validation, clinical safety, clinical rehabilitation efficacy, or medical-device readiness;
- real patient control, real FES control, real exoskeleton control, or hardware-in-the-loop control;
- direct EEG-to-muscle actuation or subject-specific musculoskeletal parameter inference;
- patient-specific OpenSim validation, implanted-interface claims, or online controller deployment;
- hidden dashboard recomputation of simulator outcomes;
- simulator outcomes being used to correct EEG ground truth, decoder predictions, calibration labels, or IHARQ correctness.

Allowed claims are therefore limited to the following:

1. The architecture can connect readiness-gated neural-intent decisions to simulated assistive-action consequences through declared proxy mappings.
2. Readiness gating can be evaluated against accept-all, confidence-only, IHARQ-gated, RegimeRisk-aware, adaptive-policy, and stress-conditioned embodiment baselines.

3. Stress-conditioned embodiment can expose failure modes under declared simulated perturbations.
4. Embodiment outcomes can provide simulation-only reward/cost terms for policy comparison.
5. Layer 10 can inspect and reproduce the embodiment results using saved records, cards, figures, logs, and manifests.

15.5 Stakeholders and concerns

Stakeholder	Primary concern addressed by Layer 9
Thesis examiner	Needs to see that the embodiment branch is scientifically bounded, non-clinical, and not overclaimed.
Supervisor or reviewer	Needs traceable evidence that readiness gating affects simulated action outcomes rather than merely classifier metrics.
Implementer	Needs adapter interfaces, schemas, validation rules, safety rules, logging requirements, testable modules, and repository artifacts.
Policy-learning researcher	Needs reward and outcome signals that are useful for simulation-only comparison without being misrepresented as patient benefit.
Dashboard user	Needs saved records, command timelines, endpoint-error curves, safety-event tables, embodiment cards, warnings, and reproduction commands.
Future maintainer	Needs modular adapters, versioned command mappings, optional OpenSim behavior, fallback-safe E0/E1 modes, and configuration hashes.

15.6 Layer 9 position in the full architecture graph

Layer 9 is downstream of the reliability and policy layers, parallel to the closed-loop simulation where embodiment is active, downstream of StressForge-Lite when stress profiles are enabled, and upstream of Layer 10 reporting. It may also feed simulation-only outcome and reward signals back to Layer 6 and Layer 7 when the experimental mode explicitly permits embodiment-informed policy comparison.

Upstream source	What Layer 9 receives
Layer 1: Public-data protocol	Dataset IDs, subject IDs, session IDs, trial/window records, labels, split IDs, channel metadata, preprocessing provenance, and WindowRecord references.
Layer 2: Decoder spine	Prediction class, probability/logit information, model ID, decoder ID, ensemble identity, and correctness helpers indirectly through downstream records.
Layer 3: Calibration and uncertainty	Calibrated confidence, entropy, margin, uncertainty summaries, selective flags, risk-coverage context, and threshold registry references.

Layer 4: IHARQ evidence verification	IHARQDecisionRecord with canonical stored tokens accept , wait , request_evidence , combine_evidence , fallback , uncertain , and unsafe , plus display labels, reason codes, costs, and rule paths.
Layer 5: RegimeRisk temporal trust	RegimeStateRecord with regime label, RegimeRisk score, trust score, drift-risk, dropout-risk, fatigue-like risk, trusted set, stop-loss status, and downstream recommendation.
Layer 6: Adaptive readiness policy	PolicyActionRecord, DeferralRecord, selected policy action, action probability, legality mask, selected threshold, reward estimate, action cost, and simulation-only flag.
Layer 7: Closed-loop readiness environment	Simulated session state, TransitionRecord structure, reward-loop context, delayed consequences, repeated-session identity, and environment reset boundaries.
Layer 8: StressForge-Lite	StressProfileRecord with stress family, intensity, target record/field, delay, dropout, weakness, noisy-intent parameters, drift, fatigue-like degradation, combined-stress label, seed, and injection point, plus StressScheduleRecord with per-step or per-window stress timing and active component stressors.

Downstream consumer	What it consumes from Layer 9
Layer 6 policy layer	Simulation-only reward and outcome terms for policy comparison or training when the experiment mode explicitly enables embodiment feedback.
Layer 7 simulator	Embodiment response, next-state effects, delay consequences, and safety events when Layer 9 is active inside the closed-loop simulation.
Layer 10 dashboard	EmbodimentStateRecord, EmbodimentOutcomeRecord, TransitionRecord, endpoint-error curves, command timelines, safety-event logs, stress-conditioned outcomes, screenshots/videos if generated, and embodiment-card data.
Thesis evaluation chapter	A13 ablation results, action-level metrics, stress-conditioned embodiment comparisons, and baseline comparisons.
Thesis limitations section	Proxy mapping limitations, simulation-only boundary, non-clinical scope, OpenSim optionality, and simulator-dependence risks.

The feedforward path is:

EEG trial/window → decoder prediction → calibrated uncertainty → IHARQ evidence decision → RegimeRisk context → Adaptive policy action → readiness safety gate → proxy intent-command mapping → stress-conditioned simulator state → MyoSuite/OpenSim/static-replay adapter → simulator step → simulated movement outcome → outcome metric and reward computation → EmbodimentStateRecord → TransitionRecord → dashboard view, embodiment card, thesis figure, and optional simulation-only policy feedback.

The dependency-direction rule is strict:

Policy and readiness records may control embodiment behavior. Embodiment may produce outcomes, reward terms, safety events, TransitionRecords, dashboard summaries, and cards. Embodiment must not silently alter decoder predictions, calibration outputs, IHARQ decisions, RegimeRisk records, StressProfileRecords, dashboard summaries, or thesis claims.

The only backward-facing signal allowed from Layer 9 is an explicitly configured simulation-only outcome or reward signal to Layer 6 and Layer 7. That signal may support policy comparison, contextual-bandit evaluation, reinforcement-learning experiments, or reward ablations. It must not update EEG labels, decoder outputs, calibration models, IHARQ rule correctness, RegimeRisk labels, or clinical interpretations.

15.7 Embodiment modes and maturity ladder

The original E0–E5 embodiment modes are preserved and expanded into a maturity ladder. The ladder is designed so that the thesis remains valid even if the physical-simulation branch is incomplete. Higher modes add evidence but do not erase lower-mode validity.

Mode	Name	Implementation requirement	Required inputs	Outputs	Thesis value and boundary
E0	No embodiment	Use only offline and simulated closed-loop readiness results.	Layers 1–8 and Layer 10 records; no simulator required.	No EmbodimentStateRecord or EmbodimentOutcomeRecord ; optional E0 Layer9EmbodimentDemoManifest explaining absence.	Preserves thesis validity if MyoSuite/OpenSim setup fails. It supports reliability claims but not action-level outcome claims.
E1	Static replay	Replay readiness decisions as command/no-command timelines without physics.	WindowRecord, PredictionRecord, IHARQDecisionRecord, RegimeStateRecord, PolicyActionRecord, command mapping, safety rules.	Command timeline, safety-event log, replay manifest, preliminary embodiment card.	Minimum debugging and dashboard bridge; shows readiness-to-command behavior without musculoskeletal claims.
E2	MyoSuite simple task	Connect readiness-gated proxy intent to a stable simple MyoSuite task such as pose, reach, or hand proxy movement.	E1 inputs plus MyoSuite environment config, task target, initial state, simulator seed.	EmbodimentStateRecord , EmbodimentOutcomeRecord , TransitionRecord , endpoint-error curve, outcome summary, and Layer9RewardTrace if enabled.	Preferred first physical-simulation demo; supports action-level simulated consequence claims only.
E3	MyoSuite with stress	Apply Layer 8 stress profiles and schedules only through declared embodiment-side injection points such as command timing, command dropout or safe hold, amplitude/weakness, response delay, endpoint/target shift, or fatigue-like response degradation; evidence-side noisy-intent, dropout, drift, or fatigue-like stress remains represented through upstream records and transition context.	E2 inputs plus StressProfileRecord, StressScheduleRecord, and stress_embedding_config.yaml .	Stress-conditioned EmbodimentOutcomeRecord entries, stress-conditioned outcome table, stress-event log, paired clean/stress curves, and updated embodiment card.	Tests whether readiness gating remains useful under declared simulated perturbations. Stress is not clinical pathology.
E4	OpenSim replay/visualization	Replay selected command timelines, movement profiles, trajectories, or exported assets through OpenSim-style visualization or plausibility analysis.	Selected E1–E3 command timelines or replay assets plus OpenSim replay config.	Replay visualization, screenshots/video manifest if generated, qualitative biomechanical-style summaries where feasible.	Optional non-blocking visualization; does not validate subject-specific biomechanics or policy training.

E5	Comparative embodiment report	Compare no embodiment, static replay, MyoS uite, stress-conditioned MyoS uite, and optional OpenSim replay.	Matched outputs from implemented E0–E4 modes.	Comparative report, A13 tables, embodiment-card set, Layer9EmbodimentDemoManifest set, limitations statement, and reproducibility package.	Synthesizes quantitative and qualitative evidence while preserving limitations.
----	----------------------------------	--	--	---	---

Each mode must specify a clear exit condition. E1 exits when every readiness state can be converted into a logged command/no-command timeline. E2 exits when the selected MyoSuite task can be reset, stepped, logged, and reproduced with finite state values. E3 exits when every stress-conditioned run has a paired clean baseline. E4 exits when replay outputs are clearly labeled optional and non-clinical. E5 exits when A13 tables, figures, cards, manifests, and limitations are complete.

15.8 Internal module decomposition

Layer 9 decomposes the embodiment subsystem into implementation modules. The former detailed embodiment dossier is now absorbed here; module IDs therefore use the Layer 9 namespace rather than a detached P20 namespace. Each module must be testable separately and traceable through records.

ID	Module	Status	Primary responsibility
L9.1	Embodiment session controller	Core	Loads configs, validates run identifiers, selects E0–E5 mode, initializes adapters, controls reset/step/close lifecycle, and writes the run manifest.
L9.2	Record intake and validator	Core	Validates upstream records, config hashes, allowed readiness vocabulary, mapping coverage, stress-profile availability, simulator state recoverability, and evaluation-only field separation.
L9.3	Readiness context assembler	Core	Combines IHARQDecisionRecord, RegimeStateRecord, PolicyActionRecord, DeferralRecord, uncertainty summaries, and stress flags into a single ReadinessContextRecord for embodiment.
L9.4	Proxy command mapper	Core	Maps normalized labels and proxy intent classes to simulator-compatible commands using versioned <code>command_mapping.yaml</code> ; unsupported labels become safe hold/no-action.
L9.5	Readiness safety gate	Core	Enforces canonical readiness tokens <code>accept</code> , <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , <code>fallback</code> , <code>uncertain</code> , and <code>unsafe</code> , plus separate adapter statuses such as unsupported mapping, simulator error, and stop-loss, before any simulator step.
L9.6	Simulator adapter interface	Core	Defines simulator-agnostic methods for static replay, MyoSuite, and OpenSim replay without embedding policy logic inside simulator code.
L9.7	Static replay adapter	Supporting / minimum bridge	Implements E1 command timeline replay without physics for debugging, dashboarding, and fallback-safe evidence.
L9.8	MyoSuite adapter	Supporting / preferred Stretch C branch	Implements E2–E3 gym-style task reset, step, state extraction, command application, outcome calculation, and artifact export.

L9.9	OpenSim replay adapter	Optional	Implements E4 replay/visualization from selected command timelines or movement assets without becoming a required policy-training environment.
L9.10	Stress embodiment injector	Supporting	Applies declared <code>StressProfileRecord</code> and <code>StressScheduleRecord</code> effects only to Layer 9-owned embodiment-side injection points such as command timing, command dropout or safe hold, amplitude/weakness, response delay, endpoint/target shift, fatigue-like response degradation, and combined-stress schedules; evidence-side noisy-intent, dropout, drift, or fatigue-like stress is consumed through upstream records rather than re-injected by Layer 9.
L9.11	Outcome and reward bridge	Core for evaluation; supporting for policy learning	Computes action-level outcome metrics, materializes <code>EmbodimentOutcomeRecord</code> entries, and optional simulation-only reward terms for Layer 6/Layer 7 feedback when enabled.
L9.12	Transition and safety logger	Core	Writes <code>EmbodimentStateRecord</code> , <code>EmbodimentOutcomeRecord</code> , <code>TransitionRecord</code> , <code>SafetyEventRecord</code> , source CSVs, schemas, file hashes, and warnings.
L9.13	Dashboard/card/export packager	Supporting	Produces Layer 10-ready summaries, <code>EmbodimentCard</code> artifacts, command timelines, endpoint-error curves, reward decomposition plots, <code>Layer9EmbodimentDemoManifest</code> files, and reproduction commands.

The internal call order is:

L9.1 session controller → L9.2 record validator → L9.3 readiness context assembler → L9.5 readiness safety gate → L9.4 proxy command mapper → L9.10 stress injector if enabled → L9.6/L9.7/L9.8/L9.9 adapter branch → L9.11 outcome/reward bridge → L9.12 transition and safety logger → L9.13 dashboard/card/export packager.

The safety gate is intentionally placed before simulator stepping. Decoder predictions and policy actions must never call `step()` directly.

15.9 Module cards and responsibilities

The following module cards define the purpose, bottleneck, scientific role, engineering role, inputs, outputs, dependencies, consumers, feedforward behavior, feedback behavior, evaluation, failure modes, reproducibility requirements, thesis evidence, and limitations for each module. They are written compactly to preserve implementation usability.

15.9.1 L9.1 Embodiment session controller

Purpose. Starts, resets, runs, closes, and manifests embodiment sessions for E0–E5.

Bottleneck addressed. Without a controller, embodiment experiments become ad hoc scripts with hidden settings, inconsistent seeds, and untraceable artifacts.

Scientific role. Ensures that every embodiment claim is tied to a declared mode, branch, seed, configuration hash, simulator version, and claim boundary.

Engineering responsibility. Load `embodiment_config.yaml`, select mode, resolve paths, instantiate adapters, enforce run lifecycle, attach identifiers, and fail closed when required records are missing.

Inputs. Embodiment configuration, simulator configuration, command mapping, safety rules, upstream record paths, stress profile path and stress schedule path where enabled, policy record path, seed, and output root.

Outputs. `Layer9EmbodimentDemoManifest`, run manifest, mode declaration, resolved config snapshot, environment metadata, and error manifest if execution is blocked.

Dependencies. Hard dependencies are configuration files, allowed mode vocabulary, record schema versions, and run identifier generation. MyoSuite and OpenSim are optional branch dependencies and must not block E0/E1.

Consumers. All downstream modules, Layer 10 reproducibility page, embodiment card, A13 evaluation scripts, and thesis appendix.

Feedforward. Provides mode, branch, seed, and paths to validators and adapters.

Feedback. Receives fatal errors from validators/adapters and converts them into explicit blocked-run manifests rather than silent failures.

Evaluation. Unit tests for config loading, seed determinism, branch selection, and missing-file behavior.

Failure modes. Hidden defaults, changed configs, simulator import errors, missing outputs, and inconsistent mode labels.

Reproducibility. Must write config hash, command used, software environment, seed, branch, mode, and record schema version.

Thesis evidence. Reproduction manifest supporting the claim that embodiment results are traceable.

Limitations. A correct controller does not guarantee simulator validity; it guarantees only traceable execution.

15.9.2 L9.2 Record intake and validator

Purpose. Checks that upstream records and configuration files are complete, compatible, and safe to use before any simulator event.

Bottleneck addressed. Embodiment is scientifically weak if it consumes missing, stale, mismatched, or evaluation-leaked fields.

Scientific role. Protects against leakage and traceability criticism by separating online decision inputs from evaluation-only labels.

Engineering responsibility. Validate required IDs, schemas, allowed actions, mapping coverage, simulator branch availability, stress profile declarations, finite simulator state assumptions, and no hidden recomputation.

Inputs. DatasetRecord, WindowRecord, LabelMapRecord, SplitRecord, PredictionRecord, CalibrationRecord, UncertaintyRecord, IHARQDecisionRecord, RegimeStateRecord, PolicyActionRecord, DeferralRecord, StressProfileRecord and StressScheduleRecord where enabled, and configuration files.

Outputs. Validation report, accepted record index, rejected record list, missing-field list, unsupported-mapping list, leakage warning list, and run-blocking reason codes.

Dependencies. Hard dependencies include schema version, identity fields, allowed readiness vocabulary, mapping ID, simulator ID, seed, and config hash. Soft dependencies include optional OpenSim assets and visualization files.

Consumers. Session controller, readiness context assembler, command mapper, safety gate, Layer 10 reproducibility page, and failure taxonomy.

Feedforward. Sends only validated records and validity flags to later modules.

Feedback. On invalid inputs, sends a blocked-run signal to the session controller and a warning record to Layer 10.

Evaluation. Schema-validation tests, missing-ID tests, illegal-action tests, unsupported-mapping tests, and ground-truth leakage tests.

Failure modes. Accepting mismatched subjects/sessions, allowing unsupported labels, letting evaluation labels influence policy actions, and losing configuration provenance.

Reproducibility. Must save validation reports and exact input file hashes.

Thesis evidence. Demonstrates that embodiment results are based on declared upstream records rather than manual simulator scripting.

Limitations. Validation cannot prove that proxy mappings are physiologically correct; it proves only that mappings are declared, versioned, and safe.

15.9.3 L9.3 Readiness context assembler

Purpose. Creates a single embodiment-ready context from reliability, uncertainty, temporal trust, deferral, policy, and stress records.

Bottleneck addressed. The simulator needs one coherent context, but readiness information is distributed across Layers 3–8.

Scientific role. Makes it possible to analyze how specific readiness reasons, RegimeRisk states, uncertainty profiles, and policy actions affect simulated action outcomes.

Engineering responsibility. Join records by run, dataset, subject, session, trial, window, time index, model ID, policy ID, stress profile ID, stress schedule ID, and horizon.

Inputs. CalibrationRecord, UncertaintyRecord, EvidenceQualityRecord, IHARQDecisionRecord, RegimeStateRecord, PolicyActionRecord, DeferralRecord, TransitionRecord context, StressProfileRecord, and StressScheduleRecord.

Outputs. ReadinessContextRecord or equivalent in-memory validated context with action, reason codes, uncertainty summary, regime score, policy metadata, deferral status, fallback status, stop-loss status, and stress context.

Dependencies. Requires aligned time index and identity keys. It can mock missing policy records during E1/E2 baseline tests by using an explicit baseline policy ID.

Consumers. Safety gate, command mapper, stress injector, outcome/reward bridge, logger, dashboard.

Feedforward. Sends context to safety gate and mapping modules before simulator stepping.

Feedback. Does not update upstream records. It may return join-error diagnostics to the validation module.

Evaluation. Join-consistency tests, record-alignment tests, missing-policy tests, and stress-context pairing tests.

Failure modes. Off-by-one window alignment, mixing trial IDs, confusing IHARQ action with policy action, and losing deferral/fallback status.

Reproducibility. Must log join keys, context schema version, and any mocked baseline policy route.

Thesis evidence. Supports causal interpretability of command timelines and safety-event timelines.

Limitations. The context is a record assembly artifact, not a new estimator.

15.9.4 L9.4 Proxy command mapper

Purpose. Converts normalized public EEG labels and proxy intent classes into declared simulated commands.

Bottleneck addressed. Public motor-imagery labels are not direct muscle commands. Without a declared mapping, embodiment could look arbitrary or misleading.

Scientific role. Preserves the claim boundary by explicitly labeling intent-to-action mappings as proxies.

Engineering responsibility. Load and validate `command_mapping.yaml`; map original label, normalized label, and intent class to simulator-compatible command; mark unsupported mappings; return safe hold/no-action when mapping is missing or ambiguous.

Inputs. LabelMapRecord, normalized label, predicted class, intended readiness action, mapping ID, simulator branch, and mapping configuration.

Outputs. Mapped command, mapping type, supported/unsupported flag, command allowed flag, command block reason, mapping rationale, simulator compatibility flag, and mapping version.

Dependencies. Hard dependency is a valid command mapping for accepted commands. Unsupported mappings are allowed only if they fail closed to no-action.

Consumers. Safety gate, simulator adapter, outcome logger, dashboard mapping table, embodiment card, failure taxonomy.

Feedforward. Sends mapped command or no-action to the adapter only after safety approval.

Feedback. Unsupported mappings return warnings to validation and Layer 10; they must not force a command.

Evaluation. Mapping coverage tests, unsupported-label tests, branch-compatibility tests, and command/no-command timeline inspection.

Failure modes. Arbitrary mapping, unsupported label forced into movement, branch-incompatible command, and mapping version drift.

Reproducibility. Must save mapping ID, mapping file hash, mapping type, and rationale for every executed or blocked command.

Thesis evidence. Provides defensible trace from EEG proxy label to simulated action command.

Limitations. A proxy command is an architectural demonstration, not evidence of real efferent motor control.

15.9.5 L9.5 Readiness safety gate

Purpose. Enforces the rule that simulated movement occurs only when readiness and mapping conditions allow it.

Bottleneck addressed. If decoder predictions directly step the simulator, the entire IHARQ/RegimeRisk/policy architecture is bypassed.

Scientific role. Operationalizes the thesis claim that reliability-aware readiness decisions affect downstream simulated consequences.

Engineering responsibility. Apply safety state-machine rules, block unsafe/aggressive actions, generate reason codes, select hold/no-op/fallback behavior, and ensure every simulator event has a safety-gate result.

Inputs. ReadinessContextRecord, IHARQDecisionRecord, PolicyActionRecord, DeferralRecord, RegimeStateRecord, stop-loss status, command-mapping result, safety rules, and stress flags.

Outputs. SafetyGateResult, allowed behavior, forbidden behavior, command execution permission, block reason, fallback/no-action decision, and SafetyEventRecord fields.

Dependencies. Hard dependencies are allowed readiness vocabulary, safety rules, reason-code registry, and mapping status. Soft dependencies are policy-specific cost thresholds and RegimeRisk recommendations.

Consumers. Simulator adapter, transition logger, safety-event log, dashboard warnings, A13 unsafe-event metrics.

Feedforward. Sends safe mapped command or no-op/fallback to adapter.

Feedback. Sends safety events to Layer 10 and optional simulation-only policy feedback to Layers 6/7 when enabled.

Evaluation. State-machine tests, blocked-command tests, aggressive-command bypass tests, and stop-loss tests.

Failure modes. Unsafe state executes command, fallback becomes aggressive, unsupported mapping passes, or stop-loss is ignored.

Reproducibility. Must log readiness state, safety state, safety reason code, rule version, and command decision.

Thesis evidence. Blocked-command tables, unsafe-event statistics, and wrong-accept-to-action reductions.

Limitations. This is a simulated safety gate; it is not a medical-device safety certification.

15.9.6 L9.6 Simulator adapter interface

Purpose. Provides a simulator-agnostic contract for static replay, MyoSuite, and OpenSim replay.

Bottleneck addressed. A one-off simulator script would be hard to test, hard to reproduce, and difficult to compare across branches.

Scientific role. Makes the embodiment claim about a readiness-to-action interface rather than a specific simulator trick.

Engineering responsibility. Define method names, preconditions, outputs, failure behavior, logging obligations, and claim-safety constraints.

Inputs. Session configuration, validated records, safety-gate result, mapping result, stress profile, simulator config, reward config, and target state.

Outputs. Simulator state, next state, outcome, reward, render artifacts, error flags, and transition log entries.

Dependencies. Base adapter is a hard dependency. MyoSuite and OpenSim packages are optional branch dependencies.

Consumers. MyoSuite adapter, OpenSim replay adapter, static replay adapter, outcome calculator, logger, dashboard.

Feedforward. Takes safe commands into branch execution and passes state/outcome to logging.

Feedback. Simulator errors return explicit failure signals to the controller and safety logger.

Evaluation. Mock adapter tests, deterministic reset tests, branch interface tests, and non-finite-state tests.

Failure modes. Hard-coded policy logic, hidden state mutation, missing logs, simulator crash, and incompatible state representations.

Reproducibility. Must record simulator version, environment name, initial state reference, target reference, seed, and branch.

Thesis evidence. Shows that the subsystem is modular and not tied to an unverifiable demo.

Limitations. Adapter compatibility does not make simulator dynamics clinically valid.

15.9.7 L9.7 Static replay adapter

Purpose. Implements E1 as a physics-free command timeline replay.

Bottleneck addressed. MyoSuite/OpenSim installation or stability problems should not block thesis-level embodiment traceability.

Scientific role. Provides the minimum evidence that readiness decisions become command/no-command timelines.

Engineering responsibility. Replay time-indexed readiness decisions, mapped commands, blocked commands, and reason codes without simulator dynamics.

Inputs. Validated readiness records, command mapping, safety-gate results, time index, and mode E1 configuration.

Outputs. Command timeline CSV/figure, safety-event log, no-action/fallback burden, replay manifest, and preliminary embodiment card.

Dependencies. No physics simulator dependency. Requires record schemas and plotting/export utilities.

Consumers. Dashboard, embodiment card, A13.1 static replay baseline, failure taxonomy.

Feedforward. Sends replay artifacts to Layer 10 and A13 evaluation.

Feedback. Does not feed rewards to policy learning unless explicitly configured as a non-physics debugging reward, which must be labeled separately.

Evaluation. Timeline consistency checks, safety state counts, mapping coverage, and human-inspectable plots.

Failure modes. Treating replay as physics, losing event order, and hiding blocked commands.

Reproducibility. Must save source CSV and plotting command.

Thesis evidence. Shows command consequences at the decision level even without musculoskeletal simulation.

Limitations. Provides no physical outcome metrics such as endpoint error.

15.9.8 L9.8 MyoSuite adapter

Purpose. Implements the preferred first physical-simulation branch for E2 and E3.

Bottleneck addressed. The architecture needs an action-level simulated environment that is more interaction-friendly than pure visualization.

Scientific role. Supports bounded evidence that readiness gating can change simulated task outcomes under clean and stressed proxy conditions.

Engineering responsibility. Initialize selected MyoSuite environment, reset with seed, extract state, apply mapped commands, apply stress, step, compute endpoint/success metrics, export renders if generated, and log all transitions.

Inputs. MyoSuite environment spec, task target, initial state, command mapping, safety result, stress profile where enabled, reward config, seed, and horizon.

Outputs. EmbodimentStateRecord, EmbodimentOutcomeRecord, TransitionRecord, endpoint-error curve, command-success table, safety-event log, Layer9RewardTrace where enabled, and optional screenshots/video manifest.

Dependencies. Optional software dependency on MyoSuite/MuJoCo stack. Required task-choice criteria are stable simulation, clear proxy mapping, endpoint/success metric, feasible stress injection, and reproducible seed control.

Consumers. Layer 10 dashboard, A13.2–A13.7 evaluation, reward bridge, controller card, embodiment card, thesis figures.

Feedforward. Produces state/outcome/reward artifacts for evaluation and reporting.

Feedback. May send reward and safety events to Layers 6/7 only under simulation-only policy-feedback mode.

Evaluation. Deterministic reset tests, finite-state tests, baseline accept-all comparison, IHARQ-gated comparison, stress-pair comparison, and reward ablation.

Failure modes. Installation failure, unstable task, invalid command, simulator crash, missing target, non-finite state, reward anomaly, and overcomplicated environment.

Reproducibility. Must record environment name, simulator version, seed, initial/target state references, command mapping ID, and config hashes.

Thesis evidence. Endpoint-error curves, command-success tables, unsafe-event rates, stress sensitivity, and A13 comparisons.

Limitations. MyoSuite results are simulated proxy outcomes and do not prove rehabilitation efficacy.

15.9.9 L9.9 OpenSim replay adapter

Purpose. Implements optional E4 replay/visualization or plausibility analysis.

Bottleneck addressed. OpenSim visuals can help communicate biomechanical direction, but they can also create false clinical authority if not bounded.

Scientific role. Provides optional qualitative or plausibility-oriented replay evidence, not primary policy-training evidence.

Engineering responsibility. Consume selected command timelines, movement profiles, trajectories, or replay assets; produce replay visualization and manifest; record limitations.

Inputs. E1–E3 command timelines or movement assets, OpenSim replay config, model/replay asset path, seed or deterministic replay settings, and selected trial/session identifiers.

Outputs. Replay visualization, screenshots/video manifest if generated, limited biomechanical-style summaries where feasible, replay limitations statement, and optional E5 comparative entries.

Dependencies. OpenSim installation and assets are optional and non-blocking. The thesis remains valid through E1/E2 if OpenSim is infeasible.

Consumers. Embodiment card, thesis qualitative demo, E5 comparative report, dashboard `Layer9EmbodimentDemoManifest`.

Feedforward. Produces visualization artifacts and replay summaries only.

Feedback. Must not provide reward signals for core policy training unless a separate simulation-only replay experiment is explicitly declared and bounded.

Evaluation. Replay consistency, artifact generation, limitation labeling, and non-blocking fallback tests.

Failure modes. Visuals imply clinical validation, missing assets block thesis, replay is mistaken for patient-specific biomechanics, and outputs lack provenance.

Reproducibility. Must save replay asset hash, OpenSim version, command timeline hash, and generated artifact paths.

Thesis evidence. Optional illustrative evidence and limitations, not required algorithmic evidence.

Limitations. No subject-specific biomechanics validation, no muscle activation inference, no medical-device safety claim.

15.9.10 L9.10 Stress embodiment injector

Purpose. Applies declared StressForge-Lite profiles to embodiment behavior.

Bottleneck addressed. Stress profiles are not meaningful for embodiment unless their injection point and effect on command/state/outcome are explicit.

Scientific role. Tests whether readiness gating changes action-level outcomes under declared simulated perturbations.

Engineering responsibility. Implement declared embodiment-side command delay, command dropout or safe hold, command perturbation, amplitude/weakness scaling, response delay, endpoint/target shift, fatigue-like response degradation, and combined stress when Layer 8 explicitly targets Layer 9; evidence-side noisy-intent, dropout, drift, or fatigue-like stress must arrive through upstream records and transition context rather than hidden Layer 9 re-injection.

Inputs. StressProfileRecord, StressScheduleRecord, `stress_embedding_config.yaml`, step index, simulator branch, command, state, target, seed, and clean baseline pairing.

Outputs. Stress-applied command or state modification, stress-event fields, schedule-step lineage, affected simulator fields, paired clean/stress run identifiers, and stress-conditioned metrics.

Dependencies. Requires Layer 8 StressProfileRecord, StressScheduleRecord, and seed. Can be mocked with no-stress profile and no-stress schedule for clean baselines.

Consumers. MyoSuite adapter, outcome calculator, safety logger, A13.7 stress-conditioned embodiment, dashboard stress page, stress card.

Feedforward. Applies declared perturbations before simulator step or outcome computation according to injection-point config.

Feedback. Produces stress outcome signals for simulation-only policy comparison when enabled.

Evaluation. Clean/stress pairing tests, intensity schedule tests, StressScheduleRecord replay tests, deterministic stress replay, and combined-stress comparison.

Failure modes. Stress not logged, stress not paired with clean baseline, stress interpreted clinically, and stress injection changes hidden upstream decisions.

Reproducibility. Must log stress profile ID, stress schedule ID, family, intensity, schedule type, injection point, seed, affected fields, profile hash, and schedule hash where feasible.

Thesis evidence. Stress sensitivity tables, stress-conditioned endpoint-error curves, and failure taxonomy.

Limitations. Stress is simulated perturbation, not disease, fatigue diagnosis, or real limb weakness.

15.9.11 L9.11 Outcome and reward bridge

Purpose. Computes action-level outcome metrics and optional simulation-only reward terms.

Bottleneck addressed. Policy learning needs outcome signals, but reward design can easily become arbitrary or overclaimed.

Scientific role. Converts embodiment consequences into measurable quantities for A13 and optional policy comparison.

Engineering responsibility. Compute endpoint error, command success, unsafe-event rate, blocked-command rate, wrong-accept-to-action rate, no-action burden, fallback burden, delay cost, over-assist, under-assist, instability, smoothness proxy, stress sensitivity, `EmbodimentOutcomeRecord` entries, and reward decomposition.

Inputs. Simulator state, target state, safety event, command, ground-truth correctness for evaluation-only analyses, reward config, stress context, and transition context.

Outputs. `EmbodimentOutcomeRecord` fields, outcome metrics, reward total, reward-success term, reward-safety term, reward-delay term, reward-fallback term, reward-smoothness term, reward-stress term, reward ablation table, and reward trace.

Dependencies. Requires outcome schema and reward config. Evaluation-only wrong-accept analysis may use ground truth labels but must not influence online actions.

Consumers. Layer 6/7 simulation-only policy comparison, Layer 10 dashboard, A13 tables, thesis results chapter.

Feedforward. Sends metrics and rewards to logger and dashboard.

Feedback. Sends reward/outcome/safety signals to policy layers only if `reward_feedback_enabled=true` or equivalent is declared.

Evaluation. Metric sanity checks, reward ablation, accept-all baseline comparison, confidence-only comparison, stress comparison, and reward hacking inspection.

Failure modes. Reward hacking, over-weighted success, hidden clinical interpretation, endpoint metric mismatch, and unlogged reward changes.

Reproducibility. Must save outcome record IDs, reward config hash, all decomposed terms, and reward-ablation settings.

Thesis evidence. A13 reward/cost trade-off, endpoint-error curves, and policy-comparison tables.

Limitations. Reward is not patient benefit; it is a simulation-only cost function.

15.9.12 L9.12 Transition and safety logger

Purpose. Saves every embodiment event, state, command, safety decision, outcome, reward, and artifact reference.

Bottleneck addressed. Without logging, embodiment becomes an unverifiable demo.

Scientific role. Converts simulation behavior into auditable evidence.

Engineering responsibility. Write `EmbodimentStateRecord`, `EmbodimentOutcomeRecord`, `TransitionRecord`, `SafetyEventRecord`, source CSVs, warnings, schema versions, file hashes, and failure manifests.

Inputs. Context, mapping result, safety result, simulator state, next state, `EmbodimentOutcomeRecord` fields, reward, stress fields, branch metadata, and artifact paths.

Outputs. Records in CSV/Parquet/JSONL including `EmbodimentOutcomeRecord`, command timeline source table, endpoint-error source table, safety-event log, `Layer9RewardTrace` where enabled, `Layer9EmbodimentDemoManifest`, and data dictionary.

Dependencies. Requires logging schema, output path, run ID, record version, and config hash.

Consumers. Layer 10 dashboard, A13 evaluation, failure taxonomy, reproducibility package,

thesis appendix.

Feedforward. Sends saved records to dashboard and evaluation scripts.

Feedback. Logging failures must stop or mark the run invalid; no hidden continuation after a critical logging failure.

Evaluation. Schema tests, append-only tests, missing-field tests, file-hash tests, and dashboard-read tests.

Failure modes. Missing provenance, overwritten records, unlogged simulator errors, dashboard-incompatible schema, and silent file mismatch.

Reproducibility. Must guarantee traceability from input artifacts to output artifacts.

Thesis evidence. The record backbone for all embodiment results.

Limitations. Logs record simulated behavior; they do not establish real-world safety.

15.9.13 L9.13 Dashboard/card/export packager

Purpose. Converts saved embodiment records into Layer 10-ready summaries, figures, cards, manifests, and thesis artifacts.

Bottleneck addressed. Dashboard reporting can become untracked recomputation unless it reads saved records only.

Scientific role. Ensures that embodiment conclusions are inspectable, reproducible, and limitation-aware.

Engineering responsibility. Export command timelines, endpoint-error curves, safety-event timelines, action distributions, blocked-command tables, reward decomposition plots, screenshots/video manifests, embodiment cards, and reproduction commands.

Inputs. Saved `EmbodimentStateRecord`, `EmbodimentOutcomeRecord`, `TransitionRecord`, `SafetyEventRecord`, outcome summaries, reward traces, config hashes, and artifact paths.

Outputs. `DashboardMetricRecord`-compatible summaries, `EmbodimentCard`, `Layer9EmbodimentDemoManifest`, figure paths, table paths, limitation text, and reproduction command.

Dependencies. Requires saved records; must not recompute hidden simulator outcomes.

Consumers. Layer 10 embodiment page, ablation page, failure taxonomy page, reproducibility page, thesis appendix.

Feedforward. Sends dashboard-ready exports and card artifacts.

Feedback. Dashboard warnings may trigger documentation updates but must not modify original results.

Evaluation. Dashboard-read tests, card-completeness tests, figure/source-data consistency tests, and claim-warning tests.

Failure modes. Hidden recomputation, missing limitation text, inconsistent metrics, unlinked figures, and missing reproduction command.

Reproducibility. Must save source-record paths, figure paths, file hashes, card version, and `Layer9EmbodimentDemoManifest` version.

Thesis evidence. Embodiment card and A13 result presentation.

Limitations. Dashboard clarity does not increase scientific validity beyond the saved records.

15.10 Formal input contract

Inputs are grouped by source, requirement level, semantic role, validity constraints, and timing. Evaluation-only inputs may be used for offline analysis but must not leak into online policy action selection.

Source	Inputs	Level	Semantic role	Validity constraints and consumer
Layer 1	DatasetRecord, WindowRecord, LabelMapRecord, SplitRecord, PreprocessingRecord, ArtifactFlagRecord, Layer1Manifest, dataset/subject/session/trial/window IDs, normalized labels.	Primary	Defines public-data anchor and proxy-intent labels.	IDs must be present; labels must be normalized; mapping limitations must be declared. Consumed by validator, context assembler, mapper, logger.
Layer 2	PredictionRecord, predicted class, model ID, decoder ID, probability/logit information, ensemble identity where available.	Primary	Supplies neural-intent candidate before readiness gating.	Must match dataset/window IDs; must not directly step simulator. Consumed by context assembler and evaluation.
Layer 3	CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, calibrated confidence, entropy, margin, threshold references, risk-coverage context.	Primary / auxiliary	Supplies confidence and uncertainty context.	Must be generated before readiness action; threshold IDs must match policy config. Consumed by readiness context and safety/evaluation.

Layer 4	EvidenceQualityRecordPrimary and IHARQDecision-Record with canonical stored tokens <code>accept</code> , <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , <code>fallback</code> , <code>uncertain</code> , and <code>unsafe</code> , plus optional display labels, reason codes, action cost, evidence request count, fallback status, and rule path.		Supplies evidence-verification decision.	Action must use Layer 4 canonical vocabulary; reason codes required. Layer 9 may map tokens to safety-gate states or simulator commands, but must not store adapter statuses as IHARQ actions. Consumed by safety gate and logger.
Layer 5	RegimeStateRecord with regime label, RegimeRisk score, risk trend, trust scores, drift-risk, fatigue-like risk, dropout risk, stop-loss status, and downstream recommendation.	Primary / auxiliary	Supplies temporal trust and stop-loss context.	Time alignment required; high-risk stop-loss must constrain execution. Consumed by safety gate and outcome analysis.
Layer 6	PolicyActionRecord, DeferralRecord, policy ID, action probability, selected threshold, legality mask, expected cost/reward, simulation-only flag, learning-to-defer route.	Primary for policy variants; optional for fixed baselines	Supplies adaptive readiness action or deferral policy.	Policy ID and legality mask required when policy route is enabled. Consumed by context assembler, safety gate, A13.
Layer 7	Simulated session state, TransitionRecord structure, environment reset boundary, episode horizon, reward-loop context, delayed consequences.	Auxiliary / conceptual	Defines closed-loop evaluation timing.	Episode boundary and horizon required for feedback experiments. Consumed by session controller and reward bridge.

Layer 8	StressProfileRecord with family, intensity, target record/field, delay, dropout, weakness, noisy intent, drift, fatigue-like degradation, combined-stress label, seed, and injection point, plus StressScheduleRecord with schedule ID, schedule type, per-step/per-window values, and active component stressors.	Optional / stress-primary	Supplies declared perturbations.	Stress profile and schedule must be versioned and paired with clean baseline. Consumed by stress injector and dashboard.
Layer 9 configuration	embodiment_config.yaml, simulator_config.yaml, command_mapping.yaml, stress_embedding_config.yaml, safety_rules.yaml, reward_config.yaml, logging_schema.yaml.	Hard configuration	Defines mode, simulator, mapping, stress, reward, safety, and logs.	Config hash required; unsupported mappings fail closed. Consumed by all Layer 9 modules.
Simulator branch	MyoSuite environment spec, OpenSim replay asset if used, static replay adapter config, simulator version, task target, initial state, endpoint/target reference, seed.	Branch-specific	Defines execution environment.	MyoSuite/OpenSim missing must not block E0/E1; state must be finite and recoverable. Consumed by adapters.
Ground truth labels and correctness helpers	True labels, correctness flags, wrong-accept tags, and outcome attribution helpers.	Evaluation-only	Used for offline wrong-accept analysis and A13 interpretation.	Must not influence online policy action selection, safety gating, or command mapping. Consumed only by evaluation scripts.

Evaluation registry and claim evidence	Global ablation ID A13, local E0–E5 mode ID, matched-comparison group ID, metric-dictionary ID, negative-result-note ID where applicable, dashboard export ID, and claim-evidence ID.	Evaluation- Binds only	embodiment outcomes to the A13 ablation ladder and to Layer 0 claim boundaries.	Must be attached after actions are fixed; missing lineage downgrades A13 results to diagnostic-only.
--	---	------------------------	---	--

15.11 Input validation rules

Before any simulator step, the validator must enforce the following rules:

1. Every run must have dataset ID, subject ID, session ID where available, trial ID, window ID, model ID, policy ID or baseline-policy ID, mapping ID, stress-profile ID plus stress-schedule ID where stress is active or explicit clean profile/schedule identifiers, simulator ID, seed, configuration hash, record version, and simulated time index.
2. Every source IHARQ readiness action must come from the Layer 4 canonical vocabulary: `accept`, `wait`, `request_evidence`, `combine_evidence`, `fallback`, `uncertain`, and `unsafe`. Adapter statuses such as unsupported mapping, simulator error, hold/no-op command, and stop-loss/high-RegimeRisk state must be logged as Layer 9 safety-gate or command-status fields, not as replacement IHARQ actions.
3. Every accepted intent class must be mapped by `command_mapping.yaml`; otherwise it is explicitly marked unsupported.
4. Unsupported mappings must default to no-action or safe hold.
5. Unsafe, uncertain, fallback, wait, request-evidence, unsupported, and stop-loss decisions must not execute aggressive assistive commands.
6. Stress profiles must be versioned, declared, seeded, and attached to the run manifest.
7. Simulator state must be finite, valid, and recoverable.
8. OpenSim assets are optional and must not block E1/E2 execution.
9. No command may execute without a logged readiness decision, safety-gate result, mapping result, and transition record.
10. Evaluation-only labels and correctness flags must not be accessible to policy action selection or safety-gate decision logic.

A validation failure has one of three outcomes: fail closed and produce no-action/blocked-command record; fail the run with an explicit manifest error; or downgrade to a lower maturity mode such as E1 when the configured fallback policy allows it.

15.12 Formal output contract

Every output has a downstream consumer and a reproducibility purpose.

Output	Format	Consumer	Purpose
EmbodimentStateRecord	CSV/Parquet/JSON	Layers 6, 7, 10; A13; thesis figures	Main state, command, safety, simulator, outcome, reward, and provenance record.
EmbodimentOutcomeRecord	CSV/Parquet/JSON	Layers 6, 7, 10; A13; simulation summaries	Step- or episode-level outcome record containing endpoint error, command success or failure, over/under-assist flags, safety-event pointer, transition pointer, command-timeline pointer, and optional screenshot or reward-trace pointer.
TransitionRecord	CSV/Parquet/JSON	Layers 6, 7, 10; policy comparison	State-action-next-state-reward log for contextual bandit and RL experiments when enabled.
SafetyEventRecord or safety_event_log.csv	CSV/JSONL	Failure taxonomy, dashboard, embodiment card	Records blocked commands, unsafe decisions, fallback events, no-action events, over-assist, under-assist, simulator errors, and reason codes.
Command timeline	Source CSV plus PDF/PNG	Dashboard embodiment page, card, A13.1	Shows canonical readiness tokens, policy-only no-action/hold states, and executed commands over time without conflating source IHARQ actions with command statuses.
Endpoint-error curve	Source CSV plus PDF/PNG	Thesis figures, dashboard, A13.2–A13.7	Shows simulated action-level outcome over time.
Outcome summary / aggregate table	CSV/JSON/LaTeX	Evaluation chapter, dashboard	Aggregates EmbodimentOutcomeRecord entries into endpoint error, command success, unsafe event rate, no-action rate, fallback rate, delay cost, over-assist, under-assist, smoothness proxy, and stress sensitivity.

Stress-conditioned outcome table	CSV/LaTeX/Markdown	Layer 8/Layer 9 reports and A13.7	Compares clean, delay, dropout, weakness, noisy intent, drift, fatigue-like degradation, and combined-stress outcomes.
Layer9RewardTrace	CSV/Parquet plus figure	Layers 6/7 policy comparison and dashboard	Shows reward total and decomposed success, safety, delay, fallback, smoothness, and stress terms when feedback is enabled.
EmbodimentCard	Markdown/PDF/HTML	Layer 10, thesis appendix, examiner review	Documents claim boundary, dataset/provenance, mapping, simulator setup, stress profile, metrics, figures, failures, reproducibility command, and limitations.
Layer9EmbodimentDemo	JSON/YAML/Markdown	Reproducibility package and appendix	Lists screenshots/videos if any, plot paths, source-record paths, config hashes, seeds, simulator versions, commands used, and file hashes.
DashboardMetricRecord compatible export	CSV/JSON/Parquet	Layer 10 embodiment, ablation, failure taxonomy, and reproducibility pages	Enables dashboard reading without hidden recomputation.
Layer 9 EvaluationOutcomeRecord	CSV/Parquet/JSON	Layer 10 ablation page, embodiment page, claim-evidence matrix, thesis tables	Records A13 local/global ablation IDs, E0–E5 mode, source records, matched-comparison ID, metric dictionary ID, embodiment metrics, warning flags, diagnostic-only status, negative-result pointer, dashboard/export lineage, and claim-evidence pointer.

Matched embodiment comparison report	CSV/JSON/Markdown	A13 evaluator, Layer 10, reproducibility package	Proves that no-embodiment, static replay, MyoSuite, stress-conditioned MyoSuite, optional OpenSim replay, accept-all, confidence-only, IHARQ-gated, RegimeRisk-aware, and adaptive-policy variants are matched on source records, mapping version, simulator config, stress profile where relevant, seed, horizon, and logging schema.
Embodiment metric-dictionary entries	YAML/JSON	Metric engine, Layer 10, thesis methods appendix	Defines endpoint error, command success, blocked unsupported command rate, no-action/safe-hold burden, unsafe-event rate, reward terms, stress-conditioned embodiment outcome, adapter failure, and demo-manifest completeness.
Embodiment negative-result note	Markdown/JSON	Dashboard limitation view, failure taxonomy, discussion chapter	Documents adapter failure, simulator instability, reward hacking, unmatched comparison, stress-pair failure, or lack of embodiment benefit without hiding the result.
Embodiment claim-evidence entry	JSON/CSV	Layer 0 claim-safety gate, Layer 10 claim-evidence view	Links A13 figures/tables/cards to simulation-only warnings, unsupported clinical claims, source records, and limitations.

15.13 EmbodimentStateRecord schema

The `EmbodimentStateRecord` is the main traceable state, command, safety, simulator, outcome, reward, and provenance record for Layer 9.

Field group	Required fields
Identity	<code>record_id</code> , <code>run_id</code> , <code>dataset_id</code> , <code>subject_id</code> , <code>session_id</code> , <code>trial_id</code> , <code>window_id</code> .

Provenance	model_id, decoder_id, policy_id, mapping_id, stress_profile_id, simulator_id, simulator_version, config_hash, seed.
Readiness context	source_iharq_action as the preserved Layer 4 token, optional iharq_action_label, policy_action, deferral_status, regime_label, regime_risk_score, uncertainty_summary, evidence_request_count, and fallback_status.
Command mapping	intent_class, normalized_label, mapped_command, mapping_type, command_allowed, command_block_reason, unsupported_mapping_flag, safety_gate_state, and adapter_status.
Simulator state	sim_time, state_vector_ref, target_state_ref, endpoint_position, target_position, simulator_branch, environment_name.
Outcome	endpoint_error, success_flag, delay_steps, over_assist_flag, under_assist_flag, instability_flag, command_success.
Safety	unsafe_event, fallback_event, no_action_event, blocked_command_event, safety_reason_code.
Reward	reward_total, reward_success_term, reward_safety_term, reward_delay_term, reward_fallback_term, reward_smoothness_term, reward_stress_term.
Logging	timestamp, simulated_step_index, record_version, notes, warning_flags, ablation_step_id, matched_comparison_id, metric_dictionary_id, negative_result_note_id, claim_evidence_id, and dashboard_export_id.

The `TransitionRecord` extends the Layer 7 state-action-next-state-reward structure by linking `state_ref`, `source_iharq_action`, `policy_action`, `validated_readiness_action`, `mapped_command`, `safety_gate_result`, `next_state_ref`, `reward_total`, `done_flag`, `stress_profile_id`, `embodiment_record_id`, and `simulation_only_flag`. It is valid for policy learning only when the run manifest explicitly enables simulation-only feedback.

15.14 EmbodimentOutcomeRecord schema

The `EmbodimentOutcomeRecord` is the companion outcome artifact for `EmbodimentStateRecord`. It materializes simulated action-level consequences at step, window, episode, or comparison granularity so that endpoint-error curves, command-success tables, reward traces, safety-event summaries, A13 comparisons, and Layer 10 embodiment views have a named source record rather than a loose outcome summary.

Field group	Required fields
Identity	outcome_record_id, run_id, embodiment_session_id, mode_id, episode_id, step_index, and record_version.
Source lineage	embodiment_record_id, transition_record_id, safety_event_id, readiness_context_record_id, policy_action_record_id, deferral_record_id, stress_profile_id, stress_schedule_id, command_timeline_id, and endpoint_curve_id.

Command and branch context	<code>source_iharq_action</code> , <code>policy_action</code> , <code>mapped_command</code> , <code>command_allowed</code> , <code>command_block_reason</code> , <code>simulator_branch</code> , <code>simulator_id</code> , <code>simulator_version</code> , <code>mapping_id</code> , and <code>config_hash</code> .
Outcome metrics	<code>endpoint_error</code> , <code>command_success</code> , <code>success_flag</code> , <code>delay_steps</code> , <code>over_assist_flag</code> , <code>under_assist_flag</code> , <code>instability_flag</code> , <code>smoothness_proxy</code> , and <code>stress_sensitivity</code> .
Safety and burden	<code>unsafe_event_flag</code> , <code>blocked_command_flag</code> , <code>fallback_event_flag</code> , <code>no_action_event_flag</code> , <code>wrong_accept_to_action_flag</code> for evaluation-only analysis, <code>safety_reason_code</code> , and <code>accepted_missingness_reason</code> where applicable.
Reward and feedback	<code>reward_trace_id</code> , <code>reward_total</code> , <code>reward_success_term</code> , <code>reward_safety_term</code> , <code>reward_delay_term</code> , <code>reward_fallback_term</code> , <code>reward_smoothness_term</code> , <code>reward_stress_term</code> , and <code>reward_feedback_enabled</code> .
Evaluation and exports	<code>ablation_step_id</code> , <code>matched_comparison_id</code> , <code>metric_dictionary_id</code> , <code>negative_result_note_id</code> , <code>evaluation_outcome_id</code> , <code>dashboard_export_id</code> , <code>claim_evidence_id</code> , <code>screenshot_manifest_id</code> , and <code>demo_manifest_id</code> .
Claim boundary	<code>simulation_only_flag</code> , <code>proxy_mapping_flag</code> , <code>non_clinical_warning</code> , <code>diagnostic_only_flag</code> , and <code>warning_flags</code> .

The record may aggregate a single step, a full episode, or a matched A13 comparison slice, but it must always point back to the `EmbodimentStateRecord` and `TransitionRecord` evidence that produced it. It must never replace source readiness tokens, safety-gate results, or simulator logs.

15.15 Simulator adapter interface

The adapter interface prevents simulator-specific code from bypassing readiness logic. Every branch must implement the following conceptual methods.

Method	Purpose and output	Failure behavior and logging obligation
<code>initialize(session_config)</code>	Configure branch instance and load declared mode, environment, mappings, safety rules, logging schema, and claim boundary.	Fail closed if required config is absent; log simulator ID, version, branch, and config hash.
<code>reset(seed, initial_state, target_state)</code>	Reset simulator or replay state to declared initial and target states.	If reset fails, write simulator-error event and prevent outcome claims.
<code>validate_mapping(command_mappings)</code>	Check compatibility of all mapped commands.	Unsupported commands are marked unsupported; they do not become forced commands.

<code>get_state()</code>	Return current state, target reference, endpoint state, and safety flags.	Non-finite or missing state triggers blocked-run or safe hold depending on mode.
<code>apply_readiness_action()</code> <code>apply_readiness_action(safety_rules)</code>	Convert readiness decision to allowed embodiment behavior before command stepping.	Every action produces a safety-gate result and reason code.
<code>map_command(intent_class, mapping_id)</code>	Convert proxy intent to mapped command, fallback, or no-action.	Missing or ambiguous mapping returns unsupported/no-action.
<code>apply_stress(stress_profile, step_index)</code>	Apply declared timing, dropout, weakness, noise, drift, fatigue-like, or combined stress.	Stress effect must be logged with profile ID, seed, intensity, schedule, and injection point.
<code>step(mapped_command_or_action)</code>	Execute static replay, MyoSuite, or OpenSim replay by one step/event.	Step requires safety gate and mapping result; decoder predictions cannot directly call this method.
<code>compute_outcome(state, target_state)</code>	Compute endpoint error, success, instability, delay, over-assist, under-assist, or branch-specific proxy outcome.	Invalid outcome triggers warning and prevents success claim.
<code>compute_reward(outcome, safety_event, reward_config)</code>	Compute simulation-only reward terms when enabled.	Missing reward config disables feedback or uses declared no-reward mode.
<code>log_transition(transition_record)</code>	Log <code>EmbodimentStateRecord</code> , <code>EmbodimentOutcomeRecord</code> , and <code>TransitionRecord</code> with provenance.	Logging failure marks run invalid; no unlogged command is accepted.
<code>render_or_export_artifacts()</code>	Generate optional plots, screenshots, video manifest, or replay artifacts.	Artifacts must have paths and hashes; visuals are not outcome substitutes.
<code>close()</code>	Release simulator resources and finalize manifest.	Close failures are logged but must not alter previous records.

The adapter state machine is:

```

UNINITIALIZED → INITIALIZED → RESET_READY → STATE_AVAILABLE →
DECISION_RECEIVED → SAFETY_CHECKED → COMMAND_MAPPED → STRESS_APPLIED
→ STEP_EXECUTED → OUTCOME_COMPUTED → TRANSITION_LOGGED → EXPORT_READY
→ CLOSED.

```

The ordering is claim-safety critical. `apply_readiness_action()` and the readiness safety gate must occur before `step()`. Decoder predictions must never directly call `step()`.

15.16 Proxy command mapping contract

Public EEG motor-imagery labels are not direct muscle commands. They are proxy intent classes mapped to simulated actions for architecture demonstration. The mapping file must be

explicit, versioned, and inspectable.

Mapping field	Required meaning
<code>original_label</code>	Dataset-specific label or event code.
<code>normalized_label</code>	Layer 1 normalized label used throughout the architecture.
<code>intent_class</code>	Proxy intent class consumed by readiness and mapping modules.
<code>supported_status</code>	Supported, unsupported, visualization-only, rest/no-action, or fallback.
<code>mapping_type</code>	Direct proxy mapping, weak proxy mapping, unsupported mapping, no-action/rest mapping, fallback mapping, or visualization-only mapping.
<code>mapped_simulator_command</code>	Branch-compatible command, action vector reference, hold/no-op command, or replay marker.
<code>simulator_branch_command</code>	Static replay, MyoSuite, OpenSim replay, or incompatible.
<code>fallback_behavior</code>	Safe hold, no-action, conservative fallback, or blocked command.
<code>mapping_rationale</code>	Why the mapping is reasonable as a proxy demonstration.
<code>known_limitation</code>	Why the mapping is not a direct real-world biomechanical or clinical command.
<code>safety_constraint</code>	Constraint that prevents aggressive or unsupported execution.
<code>mapping_version</code> and <code>mapping_id</code>	Versioned identity used in logs, cards, A13 tables, and reproduction commands.

Mapping categories are interpreted as follows:

Category	Meaning	Safety behavior
Direct proxy mapping	Dataset intent maps cleanly to a simple simulator command such as left/right/hold/reach proxy.	May execute only under accepted readiness and supported branch.
Weak proxy mapping	Mapping is plausible for demonstration but semantically loose.	Must be flagged in card and evaluated carefully.
Unsupported mapping	No defensible simulator action exists.	Must produce safe hold/no-action, never forced motion.
No-action/rest mapping	Label naturally maps to no movement or hold.	Executes hold/no-op and logs no-action semantics.
Fallback mapping	Conservative command used when policy selects fallback.	Must be declared and non-aggressive.
Visualization-only mapping	Used only for replay or display, not action control.	Cannot feed policy reward unless separately declared.

Example mapping rules are: left-hand motor imagery may map to a left reach or left hand-open proxy; right-hand motor imagery may map to a right reach or right hand-open proxy; feet imagery may map to a lower-limb proxy only if the simulator task supports it; rest maps to no-action or hold-position; **uncertain** maps to no-action or safe hold; **fallback** maps to conservative assistance or safe hold; **unsafe** maps to blocked movement and a safety event.

These examples are not universal defaults; the versioned command mapping file remains the source of truth for each dataset, simulator branch, and experiment, and command outcomes remain separate from stored Layer 4 action tokens.

15.17 Readiness safety gate and safety state machine

The original safety logic is preserved and expanded into a state machine. The embodiment adapter must obey the readiness decision while keeping readiness tokens separate from adapter statuses. If the policy action or preserved Layer 4 token is **unsafe**, **uncertain**, **fallback**, **wait**, **request_evidence**, **combine_evidence** pending resolution, or if the adapter reports unsupported mapping, simulator error, stop-loss, high RegimeRisk, or no-action, the simulator must not execute an aggressive assistive command. This is important because the demo should visually and metrically show the value of readiness gating.

State	Allowed behavior	Forbidden behavior	Simulator action	Log and dashboard warning
Accept	Apply mapped assistive command if mapping is supported and safety checks pass.	Direct decoder-to-step execution; unsupported forced mapping.	Execute mapped command.	L9_ACCEPT_EXECUTED; command executed.
Wait	Hold current state or no-op for one step.	Aggressive command while waiting.	Hold/no-op.	L9_WAIT_NOOP; delay event.
Request evidence	Hold/no-op while another evidence window is evaluated.	Movement before evidence result.	Hold/no-op.	L9_REQUEST_EVIDENCE_HOLD; evidence request event.
Combine evidence	Wait until combined decision is available; act only if accepted.	Execute partial or unresolved combined evidence.	Pending hold; then mapped command only if accepted.	L9_COMBINE_PENDING; combine event.
Fallback	Apply declared conservative fallback command or safe hold.	Treat fallback as full assistive command.	Conservative fallback or hold.	L9_FALLBACK_SAFE_HOLD; fallback event and card warning.
Uncertain	Use no-action or safe hold.	Movement under uncertainty.	Hold/no-op.	L9_UNCERTAIN_NO_ACTION; uncertain event.
Unsafe	Block movement and record safety event.	Any aggressive movement.	Block/hold.	L9_UNSAFE_BLOCKED; safety warning.
Unsupported mapping	Block command or map to no-action.	Forced invented command.	No-action/hold.	L9_MAPPING_UNSUPPORTED; mapping warning.
Simulator error	Stop, hold, or fail run; prevent success claim.	Continue unlogged execution.	Stop/hold.	L9_SIMULATOR_ERROR; error manifest.

Stop-loss / high RegimeRisk	Enforce conservative behavior, no-action, or block according to <code>safety_rules.yaml</code> .	Ignore temporal trust risk.	Hold/no- op/fallback.	L9_STOPLOSS_BLOCK; risk warning.
-----------------------------------	---	--------------------------------	--------------------------	-------------------------------------

The safety invariant is:

Unsafe, uncertain, unsupported, fallback, wait, request-evidence, simulator-error, and stop-loss states must not execute aggressive assistive commands. They may hold, no-op, request more evidence, apply conservative fallback, block movement, or terminate the run with a recorded reason.

For every loopback behavior, the required fields are: source, target, signal, trigger, action, safety constraint, evaluation logic, and logging requirement. The allowed loopbacks are listed below.

Loopback	Source/target	Signal	Trigger and action	Safety and evaluation constraint
Embodiment-to-policy	Layer 9 to Layers 6/7	Reward, outcome, safety event	Enabled only in simulation-only policy-feedback mode; writes TransitionRecord.	Does not update decoder labels or clinical claims; evaluated by reward ablation and held-out policy comparison.
Embodiment-to-dashboard	Layer 9 to Layer 10	Saved records, cards, figures	Always enabled after records exist; dashboard reads records.	Dashboard must not recompute hidden simulator outcomes; tested by file-hash/source-table checks.
Validation-to-controller	Validator to session controller	Blocked-run or downgrade signal	Missing required fields, invalid mapping, simulator unavailable.	Fail closed or downgrade to allowed mode; manifest explains result.
Safety-to-failure-taxonomy	Safety gate/logger to Layer 10	Safety reason codes	Blocked, unsafe, fallback, unsupported, stop-loss, or simulator error.	Used for failure analysis, not clinical safety certification.

15.18 Feedforward and feedback-loop behavior

Layer 9 uses feedforward processing for normal execution and carefully bounded feedback for policy evaluation. It must not introduce arbitrary feedback loops.

Loop	Source	Target	Signal	Purpose and safety constraint
------	--------	--------	--------	-------------------------------

Readiness-to-embodiment	Layers 4/6	Layer 9	Readiness action	Controls whether action is executed. Unsafe and uncertain decisions must not execute aggressive commands.
Regime-to-embodiment	Layer 5	Layer 9	Risk score and regime label	Makes embodiment conservative under unstable regimes. The signal can reduce execution, but must be logged.
Stress-to-embodiment	Layer 8	Layer 9	StressProfileRecord and StressScheduleRecord	Tests action outcomes under declared stress timing and intensity. Stress is simulation-only and must not be interpreted clinically.
Embodiment-to-policy	Layer 9	Layers 6/7	Reward, outcome, safety event	Supports policy comparison or training only in explicitly declared simulation modes. Rewards do not represent patient benefit.
Embodiment-to-dashboard	Layer 9	Layer 10	EmbodimentStateRecord, EmbodimentOutcomeRecord, safety-event logs, and card data	Enables observability, records plots, and failure taxonomy without recomputing hidden results.
Claim-safety loop	Layer 0	Layers 9/10	Boundary warnings	Prevents overclaiming in embodiment text, dashboard, cards, and thesis discussion.

Each loop must document source, target, signal, trigger, action, safety constraint, evaluation logic, risk, and claim boundary. For example, the embodiment-to-policy loop is triggered only when the experiment configuration enables simulation reward feedback. Its action is to write reward-augmented TransitionRecords for Layer 6 or Layer 7. Its safety constraint is that reward terms remain simulation-only and must not update decoder labels or clinical claims. Its evaluation is reward/regret comparison, held-out sessions, stress generalization, and reward ablation.

15.19 MyoSuite branch design

MyoSuite is the preferred first implementation target for the embodiment subsystem because it is more suitable for machine-learning-style interaction and action-level metrics than a pure replay visualization branch. In this architecture, MyoSuite should begin with the simplest stable task that supports clear proxy mapping, deterministic resets, interpretable endpoint or success metrics, feasible stress injection, and tractable logging.

Supported modes are E2 and E3. The initial task should be a simple pose, reach, hand proxy, or other stable interpretable task. The task is acceptable only if it satisfies the following criteria:

1. the simulator can reset deterministically with a declared seed;
2. the target state is explicit and logged;
3. the endpoint or success metric is interpretable;
4. the proxy command mapping is simple enough to defend;
5. stress injection can be applied to declared embodiment-side command timing, command dropout or safe hold, amplitude/weakness, response delay, endpoint/target shift, or fatigue-like response degradation;
6. non-finite simulator states, crashes, or invalid commands are detected and logged;
7. the task does not become a core thesis dependency that invalidates Layers 1–8 if unavailable.

The MyoSuite adapter responsibilities are reset, step, state extraction, command application, outcome calculation, reward logging, stress application, artifact export, and transition logging. Required MyoSuite metrics are endpoint error, success flag, unsafe-event rate, blocked-command rate, no-action burden, fallback burden, delay cost, over-assist flag, under-assist flag, smoothness proxy where available, and stress sensitivity.

MyoSuite failure handling must cover installation failure, unstable task, invalid command, simulator crash, missing target, non-finite state, reward anomaly, and render failure. The defensive rule is to downgrade to E1 static replay or E0 no embodiment when necessary, rather than changing thesis claims or hiding missing artifacts.

15.20 OpenSim branch design

OpenSim is optional and non-blocking. It is framed as replay, visualization, or limited plausibility analysis rather than the primary policy-training loop. It may support E4 and E5, but it is not required for the core Stretch C validity.

The OpenSim branch may consume selected command timelines, movement profiles, trajectories, or replay assets. It may produce replay visualization, screenshots, video manifests, and biomechanical-style summaries where feasible. It must not claim clinical biomechanics validation, patient-specific muscle activation inference, real rehabilitation efficacy, medical-device safety, or subject-specific motor control.

If OpenSim integration is infeasible, the thesis remains valid through E1 static replay and E2 MyoSuite simple-task embodiment. The non-blocking design prevents a complex visualization tool from becoming the failure point for the reliability and policy-learning thesis.

15.21 StressForge-Lite integration

Layer 8 StressProfileRecord and StressScheduleRecord affect embodiment only through declared simulation-only injection points owned by Layer 9. Stress may affect command timing, command dropout or safe hold, command amplitude/weakness, response delay, endpoint/target shift, fatigue-like performance degradation, or combined stress according to the active schedule record when the stress profile targets the embodiment path. Evidence-side noisy-intent, dropout, drift, or fatigue-like stress may influence embodiment only indirectly through the upstream readiness records, policy action, transition context, or declared Layer 8 stress lineage.

For every stress-conditioned run, the following fields are required:

Field	Requirement
Stress profile ID	Must be stored in EmbodimentStateRecord, TransitionRecord, outcome table, embodiment card, and manifest.
Stress schedule ID	Must be stored when stress is active and must link the embodiment event to the Layer 8 per-step or per-window schedule used for injection.
Stress family	Must identify delay, dropout, weakness, noisy intent, drift, fatigue-like degradation, or combined stress.
Intensity and schedule type	Must define how strongly stress is injected and which formal schedule pattern controls timing.
Injection point	Must specify command, timing, state, response, target, or outcome-level perturbation.
Seed	Must enable deterministic replay of stress effects.
Affected simulator fields	Must identify which command/state/response fields were modified.
Clean baseline pairing	Each stress run must be paired with a matching clean run where feasible.
Dashboard warning	Must state that stress is simulated and not clinical pathology.

Stress must not rewrite upstream prediction, calibration, IHARQ, RegimeRisk, or policy records. It modifies only declared embodiment behavior in the simulator/replay branch.

15.22 Policy-learning and closed-loop integration

Layer 9 can connect to Layer 6 and Layer 7 in two modes.

Mode	Allowed behavior	Forbidden behavior
Evaluation-only embodiment	Replay or execute readiness decisions in simulator and report outcomes without feeding rewards back into training.	Do not update policy weights or claim learning from embodiment outcomes.
Simulation-only policy-feedback embodiment	Write outcomes, rewards, and safety events to TransitionRecords for contextual bandit or RL policy comparison when explicitly enabled.	Do not treat reward as patient benefit; do not update decoder labels, calibration outputs, IHARQ correctness, or clinical claims.

Reward weights must be declared in `reward_config.yaml`. Reward ablation must be available to detect reward hacking or metric imbalance. All policy-learning use must be labeled simulation-only. If embodiment reward is used, the run manifest, controller card, embodiment card, and dashboard must display that policy feedback was enabled.

15.23 Reward and outcome logic

Outcome metrics should be computed even when reward feedback is disabled. Reward terms are optional and must be decomposed.

Metric or term	Meaning	Use
Endpoint error	Distance or task-specific deviation from target.	Main action-level outcome curve.
Command success	Whether the mapped command achieved the task-specific success condition.	Command-success table and A13 comparison.
Unsafe event rate	Fraction of events marked unsafe or blocked for safety.	Safety analysis and failure taxonomy.
Blocked command rate	Fraction of commands blocked by safety gate, unsupported mapping, or stop-loss.	Shows readiness-gating consequence.
Wrong-accept-to-action rate	Accepted wrong predictions that became simulated actions; uses ground truth only for offline evaluation.	Core A13 safety-relevance metric.
No-action burden	Frequency/cost of hold/no-op decisions.	Trade-off against unsafe execution.
Fallback burden	Frequency/cost of fallback route.	Measures conservative policy cost.
Delay cost	Cost of <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , and command-delay states.	Policy and stress comparison.
Over-assist flag	Command exceeds declared conservative behavior or target direction.	Failure taxonomy and reward penalty.
Under-assist flag	Command is too weak or fails to help when action is allowed.	Outcome analysis and stress effect.
Instability flag	Non-finite, oscillatory, or unstable simulated response.	Simulator defensive design.
Smoothness proxy	Approximate jerk, command changes, or state-change smoothness where available.	Optional reward and quality metric.
Stress sensitivity	Degradation from clean to stress-conditioned runs.	A13.7 and StressForge linkage.
Success reward	Positive reward for task success.	Optional policy-feedback term.
Wrong-accept penalty	Negative reward for wrong accepted action.	Optional policy-feedback term and ablation.
Unsafe-event penalty	Negative reward for unsafe/blocked/aggressive event.	Optional policy-feedback term.
Delay/fallback/no-action penalty	Cost for waiting, fallback, or no-action where appropriate.	Trade-off shaping, must be ablated.
Smoothness or stress robustness term	Optional term for smooth behavior or robust behavior under stress.	Must be declared and not overinterpreted.

Every reward term must be declared, versioned, logged, and ablated. Reward design must not be presented as clinical benefit. It is a simulation-only cost function used for policy comparison.

15.24 Layer 9 metrics

Layer 9 should never report only a single metric. At minimum, it should report endpoint error, command success, unsafe events, blocked unsafe commands, no-action/fallback burden, stress sensitivity, and limitations.

Metric group	Metrics
Core embodiment metrics	Endpoint error, command success rate, unsafe event rate, no-action rate, fallback rate, delay cost, over-assist rate, under-assist rate, action smoothness proxy, stress sensitivity.
Readiness-to-embodiment metrics	Wrong-accept-to-action rate, blocked unsafe command rate, readiness action distribution, risk-conditioned execution rate, evidence-request effect, fallback effect.
Policy-learning metrics	Embodiment reward, regret where applicable, reward-safety trade-off, stress generalization, held-out session performance, reward ablation result.
Reproducibility metrics	Trace completeness, schema pass rate, config-hash stability, seed reproducibility, simulator version capture, mapping coverage.

Success criteria are deliberately bounded. Layer 9 succeeds if readiness-gated variants reduce unsafe or inappropriate simulated actions compared with accept-all execution, maintain traceable command timelines, produce interpretable action-level metrics, preserve claim boundaries, and remain reproducible. Failure does not invalidate the core architecture; it indicates that embodiment mapping, simulator choice, reward design, or policy conservatism needs improvement.

15.25 Embodiment data flow and operational flow

The complete operational sequence is:

```

EEG trial/window
-> WindowRecord
-> PredictionRecord
-> CalibrationRecord and UncertaintyRecord
-> EvidenceQualityRecord and IHARQDecisionRecord
-> RegimeStateRecord
-> PolicyActionRecord and/or DeferralRecord
-> ReadinessContextRecord
-> readiness safety gate
-> proxy command mapper
-> StressProfileRecord and StressScheduleRecord attachment if enabled
-> simulator adapter selection: static replay, MyoSuite, or OpenSim replay
-> simulator reset/step or replay event
-> outcome calculator
-> reward bridge if enabled
-> EmbodimentStateRecord
-> EmbodimentOutcomeRecord
-> TransitionRecord
-> safety event log

```

- > command timeline and endpoint-error source tables
- > DashboardMetricRecord-compatible summaries
- > embodiment card
- > A13 evaluation tables
- > reproducibility package

Timing/order constraints are:

1. The simulator step may occur only after readiness action, safety-gate result, command mapping result, StressProfileRecord/StressScheduleRecord attachment if enabled, and transition identity are available.
2. Ground-truth labels may be attached only for offline evaluation fields after the policy action and safety result are already fixed.
3. Dashboard exports read saved records only.
4. Feedback to Layer 6/Layer 7 is disabled unless the run config explicitly enables simulation-only reward feedback.

15.26 Evaluation and A13 ablation linkage

The main A13 question is:

Do readiness-gated decisions improve simulated action-level outcomes compared with simpler execution policies under clean and stressed conditions?

Required A13 baselines and variants are:

A13 variant	Evaluation role
E0 no embodiment	Shows that core readiness results remain valid without embodiment.
E1 static replay	Tests whether readiness decisions produce sensible command/no-command timelines.
Accept-all embodiment	Tests what happens when every decoder decision becomes a simulated action.
Confidence-only embodiment	Tests whether a simple confidence threshold is sufficient.
IHARQ-gated embodiment	Tests whether evidence verification improves simulated outcomes or reduces unsafe events.
RegimeRisk-aware embodiment	Tests whether temporal trust reduces risky simulated actions under unstable regimes.
Adaptive-policy embodiment	Tests whether learned readiness improves outcome/cost trade-offs.
Stress-conditioned embodiment	Tests robustness under delay, dropout, weakness, noisy intent, drift, fatigue-like degradation, and combined stress.
Optional OpenSim replay	Tests whether replay adds useful qualitative insight without becoming a clinical claim.

For matched comparison, each variant must use the same dataset/split, PredictionRecords, readiness records, stress profile and stress schedule where relevant, command mapping version, simulator config, seed, evaluation horizon, and logging schema. Success criteria include lower wrong-accept-to-action rate, lower unsafe-event rate, acceptable no-action/fallback burden, improved endpoint-error or command-success trade-off, reduced stress sensitivity, and reproducible records. Failure interpretation must remain bounded: poor embodiment performance indicates limitations of proxy mapping, readiness policy, stress handling, simulator choice, or reward design; it does not prove clinical harm or clinical inefficacy.

Every A13 result must be materialized as a Layer 9 **EvaluationOutcomeRecord** supported by **EmbodimentOutcomeRecord** entries where an embodied mode executed, or explicitly marked absent with an accepted-missingness reason. Thesis-ready A13 evidence requires a matched embodiment comparison report, metric-dictionary entry, dashboard/card/export lineage, claim-evidence entry, and negative-result note whenever a comparison fails, an adapter branch fails, a simulator run becomes unstable, a reward term dominates the result, or the result is only diagnostic. The A13 ladder is local to Layer 9, but its IDs must remain compatible with the global A0–A13 ladder exposed by Layer 10.

15.27 Dashboard and reporting linkage

Layer 10 consumes saved Layer 9 outputs. The dashboard must read saved records only and must not recompute hidden simulator outcomes or silently change results.

The embodiment dashboard page should display:

- embodiment mode E0–E5;
- simulator branch: static replay, MyoSuite, or optional OpenSim replay;
- command mapping ID, policy ID, stress profile ID, stress schedule ID, simulator ID, seed, and config hash;
- command timeline;
- endpoint-error curve;
- **EmbodimentOutcomeRecord** outcome table;
- safety-event timeline;
- action distribution;
- no-action/fallback burden;
- blocked-command table;
- reward decomposition where applicable;
- screenshots/video manifest if generated;
- claim-safety warnings;
- **EmbodimentCard** export;
- **Layer9EmbodimentDemoManifest**;
- exact reproduction command.

The embodiment card must contain claim boundary, dataset/provenance, mapping, simulator setup, stress profile, stress schedule, metrics, metric-dictionary references, matched-comparison status, figures, failures, negative-result notes where applicable, reproducibility command, limitations, claim-evidence ID, and file hashes.

15.28 Reproducibility and artifact management

Required configuration files are:

File	Purpose
<code>configs/layer9_embodiment/embodiment_config.yaml</code>	Middleware, metrics, seed, horizon, output paths, feedback flag, and claim boundary.
<code>configs/layer9_embodiment/simulator_config.yaml</code>	Simulation by MySim, or OpenSim setup.
<code>configs/layer9_embodiment/command_mapping.yaml</code>	Hand mapping command and mapping with mapping type, rationale, limits, and fallback behavior.
<code>configs/layer9_embodiment/stress_embedding_config.yaml</code>	Stress embedding config. Schedules affect commands, timing, response, target, state, or outcome.
<code>configs/layer9_embodiment/reward_config.yaml</code>	Reward config, and lights, enable/disable flags, and ablation settings.
<code>configs/layer9_embodiment/safety_config.yaml</code>	Safety config, and safety behavior and reason-code mapping.
<code>configs/layer9_embodiment/logging_schema.yaml</code>	Logging schema for Record, EmbodimentOutcomeRecord, TransitionRecord, and SafetyEventRecord schema.

Required implementation folders are:

```
src/layer9_embodiment/adapters/
src/layer9_embodiment/mapping/
src/layer9_embodiment/safety/
src/layer9_embodiment/stress/
src/layer9_embodiment/outcomes/
src/layer9_embodiment/logging/
src/layer9_embodiment/cards/
```

Required output folders are:

```
records/embodiment/
records/embodiment/embodiment_state_records.parquet
records/embodiment/embodiment_outcome_records.parquet
records/embodiment/transition_records.parquet
records/embodiment/safety_event_log.csv
records/embodiment/reward_traces/
records/embodiment/demo_manifests/
figures/embodiment/
cards/embodiment/
results/layer9_embodiment/
manifests/embodiment/
```

```

manifests/embodiment/layer9_embodiment_demo_manifest.json
records/evaluation_outcomes/layer9_embodiment_evaluation_outcomes.parquet
reports/matched_comparisons/layer9_matched_embodiment_comparisons.csv
metrics/metric_dictionary_layer9_embodiment.yaml
notes/negative_results/layer9_embodiment_negative_results.md
manifests/claim_evidence/layer9_embodiment_claim_evidence.json

```

Required reproducibility metadata are run ID, seed, config hash, mapping ID, simulator version, stress profile ID, stress schedule ID where stress is active, policy ID, dataset/split identifiers, record schema version, environment version, command used to reproduce the run, and file hashes.

A reproduction command should follow the form:

```

python run_layer9_embodiment.py \
  --embodiment-config configs/layer9_embodiment/embodiment_config.yaml \
  --simulator-config configs/layer9_embodiment/simulator_config.yaml \
  --mapping configs/layer9_embodiment/command_mapping.yaml \
  --policy-records records/policy/policy_action_records.parquet \
  --stress-profile records/stress_records/stress_profile_records.parquet \
  --stress-schedule records/stress_records/stress_schedule.parquet \
  --output-dir results/layer9_embodiment/

```

15.29 Failure modes and defensive design

Failure mode	Risk and cause	Detection method	Defensive design	Output/test
Demo is overclaimed	Readers may infer clinical validation from visuals.	Claim-audit checklist and card warnings.	Repeat simulation-only boundary in Layer 0, Layer 9, dashboard, and card.	Embodiment card warning.
Proxy mapping looks arbitrary	Mapping rationale is unclear.	Mapping coverage and rationale review.	Versioned command mapping with category and limitation fields.	Mapping validation report.
Simulator complexity blocks thesis	MyoSuite/OpenSim installation or stability fails.	Branch smoke tests.	E0/E1 fallback and OpenSim non-blocking rule.	Mode manifest.
Safety gate is bypassed	Decoder predictions directly step simulator.	Adapter call-order tests.	Require safety result before step.	State-machine unit test.
Reward hacking	Policy optimizes cosmetic reward.	Reward ablation and term inspection.	Decomposed rewards and held-out comparison.	Reward ablation table.

OpenSim visuals imply false authority	Visualization looks clinical.	Claim-safety review.	Label OpenSim as optional re-play/visualization.	Replay limitations statement.
Dashboard recomputes hidden results	Dashboard becomes separate result generator.	Source-table hash checks.	Dashboard reads saved records only.	Dashboard-read test.
Missing provenance	Results cannot be reproduced.	Manifest completeness check.	Require config hashes, seeds, versions, and file hashes.	Layer9EmbodimentDem
Stress not paired with clean baseline	Stress effect cannot be interpreted.	Pairing validation.	Require clean baseline pairing where feasible.	Stress-conditioned table.
Unsupported labels forced into commands	Invalid actions executed.	Mapping validator and safety-gate tests.	Unsupported labels produce no-action/hold.	Unsupported mapping log.
Simulator errors unlogged	False success claims.	Exception and non-finite-state tests.	Stop/hold and write simulator-error manifest.	Safety event log.
Metrics become cosmetic	Only screenshots are reported.	A13 output checklist.	Require endpoint, success, safety, burden, stress, reward, and limitations outputs.	A13 tables and figures.
A13 matching fails	Embodiment comparisons can look meaningful even when variants use different records, mappings, simulator settings, stress profiles, seeds, or horizons.	Matched embodiment comparison report and diagnostic-only downgrade.	Require matched source records or explicit diagnostic label plus negative-result note.	A13 comparison report and claim-evidence matrix.

15.30 Implementation roadmap and definition of done

Stage	Name	Definition
L9-0	Claim boundary and contracts	Document simulation-only scope, input/output contracts, schema versions, and safety invariants.
L9-1	Static replay E1	Generate readiness command timelines and safety-event logs without physics.
L9-2	Command mapping validation	Validate mapping coverage, unsupported behavior, branch compatibility, rationale, and mapping hash.

L9-3	Safety gate tests	Prove that unsafe, uncertain, fallback, wait, request, unsupported, and stop-loss states do not execute aggressive commands.
L9-4	MyoSuite simple task E2	Run a stable MyoSuite task with endpoint/success metrics, <code>EmbodimentStateRecord</code> , <code>EmbodimentOutcomeRecord</code> , <code>TransitionRecord</code> , and <code>Layer9EmbodimentDemoManifest</code> .
L9-5	Stress-conditioned MyoSuite E3	Pair clean and stress-conditioned runs for delay, dropout, weakness, noisy intent, drift, fatigue-like degradation, and combined stress where feasible.
L9-6	Reward bridge to policy learning	Enable simulation-only reward feedback with decomposed terms and reward ablation.
L9-7	Optional OpenSim replay E4	Produce optional replay/visualization manifest and limitations statement.
L9-8	Comparative embodiment report E5	Produce A13 tables, curves, cards, and limitations across implemented modes.
L9-9	Dashboard/card/reproducibility completion	Export dashboard-ready records, embodiment card, <code>Layer9EmbodimentDemoManifest</code> , source tables, figures, hashes, and reproduction command.

Layer 9 is thesis-ready only when it defines the simulation-only claim boundary; documents upstream and downstream dependencies; specifies input and output contracts; defines adapter methods; defines proxy mapping; defines safety gate and state-machine behavior; specifies E0–E5 maturity levels; distinguishes MyoSuite and OpenSim roles; connects to StressForge-Lite, Layer 6, Layer 7, and Layer 10; defines `EmbodimentStateRecord`, `EmbodimentOutcomeRecord`, and `TransitionRecord` linkage; specifies metrics and A13 ablations; produces Layer 9 `EvaluationOutcomeRecords`, matched embodiment reports, metric-dictionary entries, negative-result notes where applicable, dashboard/export lineage, and claim-evidence entries; includes failure modes; lists reproducibility artifacts; and prevents clinical overclaiming.

15.31 Traceability and claim-safety checklist

For every input, the implementation must record where it originates, which record or config contains it, whether it is required, optional, evaluation-only, or stretch-only, what validity checks apply, which submodule consumes it, and what happens if it is missing or invalid.

For every output, the implementation must record which submodule produces it, which schema it follows, which layer consumes it, which dashboard page uses it, which evaluation metric uses it, which thesis claim it supports, and which claim would become invalid if it were absent.

For every feedback path, the implementation must record source, target, signal, trigger, allowed use, forbidden use, claim boundary, and logging requirement.

The final claim-safety checklist is:

1. The MyoSuite/OpenSim branch is labeled simulated proxy embodiment.
2. Public EEG remains the experimental anchor.

3. OpenSim is optional and non-blocking.
4. MyoSuite is a Stretch C demonstration branch, not a thesis-critical dependency.
5. Decoder predictions never directly control simulator steps.
6. Policy outputs never bypass readiness safety gating.
7. Unsupported labels map to no-action or safe hold.
8. Stress profiles are declared, seeded, and paired with clean baselines where feasible.
9. Embodiment rewards are simulation-only and not patient benefit.
10. Dashboard reads saved records and does not recompute hidden outcomes.
11. A13 remains the embodiment ablation linkage.
12. No clinical validation, rehabilitation efficacy, medical-device readiness, or real hardware-control claim is introduced.

15.32 Layer 9 readiness levels and definition of done

Level	Name	Meaning
L9-RL0	Concept only	Layer 9 is mentioned but not specified.
L9-RL1	Claim boundary defined	Simulation-only and non-clinical scope are explicit.
L9-RL2	Contracts defined	Inputs, outputs, mapping file, and adapter interface are specified.
L9-RL3	Static replay ready	E1 command timeline and safety gate work without physics.
L9-RL4	MyoSuite simple task ready	E2 runs and logs embodiment outcomes.
L9-RL5	Stress integration ready	E3 stress conditions affect embodiment outcomes.
L9-RL6	Reward bridge ready	Outcomes support simulation-only policy reward/comparison.
L9-RL7	Optional OpenSim replay ready	E4 outputs exist if feasible.
L9-RL8	A13 evaluation complete	Baselines and ablations are reported.
L9-RL9	Thesis-ready	Cards, figures, limitations, and reproducibility package are complete.

Layer 9 should not be presented as a strong Stretch C contribution below L9-RL4. It should not be defended as an evaluated embodiment contribution below L9-RL8.

Layer 9 is thesis-ready when the architecture states the simulation-only claim boundary; separates Layer 9 from Layer 10 reporting; documents upstream and downstream dependencies; defines the adapter interface, mapping file, safety gate, schemas, and outputs; produces static replay and preferably MyoSuite logs; keeps OpenSim optional; attaches stress profiles where relevant; saves `EmbodimentStateRecord`, `EmbodimentOutcomeRecord`, and `TransitionRecord`

records; includes accept-all, confidence-only, IHARQ-gated, RegimeRisk-aware, adaptive policy, and stress-conditioned comparisons where implemented; reports endpoint error, command success, unsafe events, and no-action/fallback burden; generates an `EmbodimentCard` and `Layer9EmbodimentDemoManifest`; and provides a reproduction command.

15.33 Layer 9 thesis evidence

The thesis chapter should use Layer 9 to produce the following evidence:

- a Layer 9 identity and claim-boundary table;
- an E0–E5 embodiment maturity ladder;
- a simulator-adapter contract and state-machine figure;
- a command mapping table with mapping validity categories and limitations;
- a safety behavior table for `accept`, `wait`, `request_evidence`, `combine_evidence`, `fallback`, `uncertain`, `unsafe`, and separate unsupported-adapter decisions;
- command timelines comparing accept-all, confidence-only, IHARQ-gated, RegimeRisk-aware, and adaptive policy variants;
- endpoint-error and command-success curves for static replay or MyoSuite where implemented;
- blocked-command and unsafe-event statistics;
- stress-conditioned embodiment tables linked to Layer 8;
- A13 ablation results from A13.0 through the implemented highest mode;
- Layer 9 `EvaluationOutcomeRecords` supported by `EmbodimentOutcomeRecord` entries where applicable, matched embodiment comparison reports, metric-dictionary entries, negative-result notes, dashboard/export lineage, and claim-evidence entries for thesis-used A13 claims;
- an `EmbodimentCard` and `Layer9EmbodimentDemoManifest` documenting configuration, mapping, simulator setup, metrics, matched-comparison status, failure cases, negative-result notes where applicable, limitations, claim-evidence ID, and reproduction command.

The correct final interpretation is that Layer 9 demonstrates a bounded simulated path from reliability-aware neural-intent readiness to action-level consequence. It strengthens the architecture by showing how readiness control can influence simulated assistive-action behavior, while explicitly not claiming clinical validation, real patient safety, or direct EEG-to-muscle control.

16 Layer 10: Dashboard, Cards, and Reproducibility Layer

16.1 Layer 10 identity, thesis role, and bottleneck

Layer 10 is the cross-layer observability, reporting, card-generation, artifact-export, and reproducibility layer of IHARQ BenchGuard. It sits at the end of the runtime architecture, but its scientific role is not cosmetic. It is the layer that turns the full Stretch C system into an

inspectable thesis artifact: every result must be connected to saved records, every figure and table must be traceable to a configuration and run, every card must state its assumptions and limitations, and every dashboard view must support a real research inspection task.

Layer property	Layer 10 specification
Layer name	Dashboard, Cards, and Reproducibility Layer.
Architectural type	Cross-layer scientific observability and reproducibility layer.
Runtime status	Mostly downstream and read-only; allowed to write derived dashboard metrics, cards, figures, tables, logs, and manifests with provenance.
Primary purpose	Make the complete Stretch C architecture inspectable, explainable, reproducible, claim-safe, and thesis-ready.
Main consumers	Thesis author, supervisor, examiner, reviewer, future reader, future engineer, and future researcher.
Main inputs	Canonical records from Layers 1–9, including <code>RegimeStateRecord</code> and conditional <code>TrustStateRecord</code> rows for independent trust timelines, Layer 0 claim-safety labels, experiment summaries, split manifests, run registries, configuration files, evaluation records, Layer 7 closed-loop artifacts such as <code>TransitionRecord</code> , <code>ClosedLoopSessionRecord</code> , <code>EpisodeSummaryRecord</code> , <code>PolicyRolloutRecord</code> , <code>RewardTraceRecord</code> , <code>EnvironmentDiagnosticRecord</code> , <code>SafetyEventRecord</code> where materialized, and optional <code>PolicyUpdateTrace</code> , and Layer 9-owned embodiment export artifacts such as <code>EmbodimentStateRecord</code> , <code>EmbodimentOutcomeRecord</code> , <code>TransitionRecord</code> embodiment linkages, <code>SafetyEventRecord</code> or <code>safety_event_log.csv</code> , <code>Layer9RewardTrace</code> where enabled, command timelines, endpoint-error curves, <code>Layer9EmbodimentDemoManifest</code> , and dashboard-compatible exports.
Main outputs	Dashboard views, <code>DashboardMetricRecords</code> , <code>DashboardViewRecords</code> , <code>CardRecords</code> , static plots, ablation tables, embodiment evidence summaries, failure-taxonomy reports, validation logs, export logs, artifact manifests, <code>ReproducibilityManifests</code> , thesis-ready figures, thesis-ready tables, and repository release package indexes.
Core thesis contribution	Scientific transparency: the architecture is not only a classifier pipeline, but a modular, auditable, reproducible research system.

The bottleneck addressed by Layer 10 is result fragmentation. A Stretch C architecture contains public-data protocols, decoders, calibration, uncertainty, selective prediction, IHARQ decisions, temporal trust monitoring, adaptive readiness policy, simulated sessions, optional policy-update traces for simulation-authorized bandit/RL runs, StressForge stress conditions, and a MyoSuite/OpenSim embodiment demonstration. Without a dedicated observability and reproducibility layer, these components can appear as disconnected experiments. Layer 10 solves this by collecting their records into a structured inspection surface and by exporting thesis artifacts with explicit lineage.

After the detailed MyoSuite/OpenSim/static-replay dossier has been merged back into Layer

9, Layer 10 treats embodiment as a Layer 9-owned evidence stream. It may consume Layer 9 dashboard-ready summaries, embodiment cards, command timelines, endpoint-error curves, safety-event logs, reward-trace outputs, `Layer9EmbodimentDemoManifests`, simulator-version metadata, and reproduction commands. It must not re-create a detached embodiment runtime namespace or imply that simulator logic is owned by Layer 10.

Layer 10 supports the central thesis claim that reliability in low-calibration neural intent systems should be evaluated as a modular evidence-control architecture, not only as a final decoder accuracy score. It supports this claim by exposing the intermediate evidence chain: dataset and split choices, prediction quality, calibration quality, uncertainty behavior, IHARQ canonical action tokens and reason codes, `RegimeRisk` warnings, policy actions, stress conditions, simulated transitions, embodiment outcomes, and failure categories.

Layer 10 protects the thesis against several predictable criticisms:

- **“The architecture is too complex.”** Layer 10 exposes ablations, matched-coverage comparisons, matched-cost comparisons, and module contribution summaries.
- **“The results are not reproducible.”** Layer 10 exports run registries, configuration manifests, environment descriptions, source-record lineage, and artifact manifests.
- **“The dashboard is decorative.”** Layer 10 uses a record-only principle and links every dashboard page to a thesis question, evaluation metric, card, artifact, or limitation.
- **“The policy, stress, or embodiment results are overclaimed.”** Layer 10 attaches simulation-only, proxy-label, public-data, incomplete-coverage, and clinical non-claim warning flags.
- **“The system hides negative results.”** Layer 10 centralizes failure taxonomy, missing-record reports, failed validation checks, and cases where a module does not help.

If Layer 10 were removed, the project would still be able to run individual experiments, but it would lose its evidence-control surface. The thesis would have weaker traceability from claims to records, weaker reproducibility, weaker artifact governance, weaker failure analysis, and weaker defense of architectural complexity.

16.2 Stakeholders and thesis inspection questions

Layer 10 is designed around thesis inspection questions rather than generic plotting. Its detailed dashboard subsystem must let each stakeholder trace a result from claim to source record and must make missingness, limitation flags, and non-thesis-ready status visible instead of hiding them.

Stakeholder	Primary need	Representative Layer 10 questions
Thesis author	Debugging, figure generation, artifact completeness, record validation, reproducibility checks.	Which dataset and split produced this result? Which record is missing? Which figure is stale? Which ablation is incomplete? Which card cannot yet be marked thesis-ready?

Supervisor	Coherence of architecture and support for research claims.	Are the thesis claims supported by records? Are public-data, proxy, simulation, stress, and embodiment claims separated? Are negative results and failure modes visible?
Examiner or reviewer	Traceability from claims to figures, tables, cards, configs, and source records.	Which claim does this figure support? Which source records and configuration IDs generated it? Was threshold selection performed on the correct split? Is the evidence public-data, simulated, proxy, stress-test, embodiment-demo, or reporting-only?
Future reproducer	Manifest completeness, commands, paths, seeds, versions, record IDs, and environment metadata.	Which command rebuilds this table? Which run ID and seed generated this metric? Which package versions and checksums are required? Which records must be downloaded or regenerated?
Future engineer or researcher	Module boundaries, failure modes, missingness behavior, and extension points.	Which backend module owns validation? Which page consumes <code>RegimeStateRecord</code> or linked <code>TrustStateRecord</code> rows? Where should a new card template be added? Which dependency can be mocked?

The dashboard page architecture must answer whether confidence was calibrated; whether a threshold was selected without leakage; why IHARQ accepted, waited, requested evidence, combined evidence, fell back, marked uncertain, or marked unsafe; whether the operating regime was stable or risky; which policy action was selected and under which cost or reward assumptions; what happened under StressForge stress conditions; what happened in Layer 9 MyoSuite/OpenSim/static-replay simulation; which A0–A13 ablation step supports the contribution; which failure modes appeared; which thesis figure or table uses each metric; and which claim becomes invalid if a source record is missing.

16.3 Engineering responsibility, scope, and implementation boundary

The technical responsibility of Layer 10 is to read canonical records, validate their schemas and lineage, summarize them into dashboard-ready metrics, render inspection views, generate standardized cards, export thesis artifacts, and assemble reproducibility manifests. It is an observability and artifact-production layer, not a training, inference, calibration, policy-learning, stress-generation, or simulator-control layer.

16.3.1 In-scope responsibilities

Responsibility	Layer 10 behavior
Dashboard views	Provide interactive or static inspection of datasets, decoders, calibration, uncertainty, selective prediction, IHARQ, RegimeRisk, adaptive policies, learning-to-defer, bandit/RL policies, Layer 7 closed-loop session timelines, policy rollouts, reward decompositions, environment diagnostics, optional policy-update traces, StressForge, Layer 9-owned embodiment exports, ablations, cards, and failure taxonomy.
Card generation	Generate standardized dataset, protocol, decoder, calibration, IHARQ, RegimeRisk, policy, deferral, bandit, RL, closed-loop environment, policy rollout, policy-update trace, reward-configuration, StressForge, Layer 9 embodiment, safety, limitation, and failure-taxonomy cards.
Thesis figure export	Export static figures for the thesis, such as low-calibration curves, reliability diagrams, decision-flow timelines, RegimeRisk timelines, policy ablation plots, closed-loop session timelines, policy-rollout timelines, bandit/RL training-adaptation curves, frozen-policy evaluation curves, reward-decomposition/reward-regret plots, environment-diagnostic warning summaries, stress sensitivity curves, Layer 9 command timelines, endpoint-error plots, unsafe-event tables, and reward-decomposition plots where enabled.
Table export	Export CSV, Parquet, Markdown, or LaTeX-ready tables for metrics, ablations, artifacts, validation checks, and reproducibility summaries.
Ablation inspection	Expose the A0–A13 ablation ladder and identify where architectural modules improve, fail, or add cost without benefit.
Failure taxonomy reporting	Connect high-confidence wrong cases, unsafe events, fallback logs, stress failures, Layer 9 safety-event logs, embodiment failures, reward-trace anomalies, missing <code>Layer9EmbodimentDemoManifests</code> , and missing records to thesis limitations.
Reproducibility manifests	Record configs, seeds, environment assumptions, run IDs, record paths, artifact paths, Layer 7 transition, closed-loop session, episode-summary, policy-rollout, reward-trace, environment-diagnostic, and safety-event paths, optional <code>PolicyUpdateTrace</code> paths, disabled-update audit paths, Layer 9 demo-manifest paths, command-mapping IDs, simulator adapter branches, safety-event log paths, reward-trace paths, source layers, warning flags, and reproduction commands.
Artifact completeness checks	Verify that expected records, cards, tables, figures, logs, manifests, and Layer 9 embodiment export bundles exist and are linked to source records.

Claim-safety warnings	Surface Layer 0 scope boundaries: public-data limits, proxy-label warnings, simulation-only warnings, Layer 9 embodiment-demo-only warnings, real-actuation non-claims, and clinical non-claims.
Repository packaging	Help assemble a reproducibility package containing records, cards, manifests, exported figures, exported tables, configs, environment files, and reproduction instructions.

16.3.2 Out-of-scope responsibilities

Out-of-scope behavior	Reason
Training decoders	Belongs to Layer 2.
Calibrating probabilities	Belongs to Layer 3.
Selecting thresholds for experimental evaluation	Belongs to Layer 3 and must be recorded before Layer 10 displays the result.
Making IHARQ decisions	Belongs to Layer 4.
Computing RegimeRisk state	Belongs to Layer 5.
Learning adaptive readiness policies	Belongs to Layer 6.
Running closed-loop simulated sessions	Belongs to Layer 7.
Mutating policies or enabling bandit/RL update hooks from the dashboard	Belongs to Layer 6 policy logic and Layer 7 simulation-authorized update execution; Layer 10 may only inspect <code>PolicyUpdateTrace</code> files and disabled-update audits.
Generating stress profiles or mutating records with stress	Belongs to Layer 8.
Running MyoSuite/OpenSim/static-replay simulator logic, recomputing adapter outputs, or changing Layer 9 reward/outcome records	Belongs to Layer 9.
Creating stronger scientific claims than the records support	Controlled by Layer 0 and thesis validity rules.
Producing hidden untracked results	Violates the record-only and reproducible-export principles.

16.3.3 Core, supporting, optional, stretch, and future components

Status	Components	Interpretation
--------	------------	----------------

Core	Record loader, schema validator, record indexer, metric summarizer, card generator, export manager, reproducibility manifest builder, warning engine, audit logger.	Required for a thesis-grade Layer 10. Without these, the layer cannot claim reproducibility or traceability.
Supporting	Dashboard view renderer, filtering interface, static HTML or notebook report, artifact completeness table, card completeness table.	Strongly recommended because they make inspection practical, but can be implemented simply.
Optional	Interactive Streamlit, Dash, Gradio, or web dashboard; video/screenshot export and demo-manifest indexing for the Layer 9 embodiment demo.	Useful for communication, but not required if static reports and exported artifacts are complete.
Stretch	Full artifact browser, drill-down from aggregate plots to trial-level records, optional drill-down from bandit/RL curves to <code>PolicyUpdateTrace</code> entries, automated claim-to-artifact mapping, dashboard snapshots for every thesis figure.	Valuable if time permits; not needed for the minimum defensible thesis.
Future work	Public web release, user authentication, collaborative review UI, live experiment monitoring, continuous-integration dashboard.	Outside the thesis unless implementation capacity is high.

The implementation boundary is strict: Layer 10 may summarize, filter, visualize, validate, and export records. It may compute descriptive summaries from saved records. It may write derived artifacts with provenance. It must not silently train decoders, preprocess raw EEG, create hidden splits, calibrate probabilities, select thresholds, compute IHARQ decisions, compute RegimeRisk states, learn Adaptive-IHARQ policies, learn deferral policies, run contextual-bandit or RL simulations, generate StressForge profiles, execute MyoSuite/OpenSim/static-replay simulations, modify upstream records, edit metrics manually, enable policy-update hooks, mutate bandit/RL policies, overwrite upstream records, or hide failed checks. If a recomputation is intentionally triggered from the dashboard, it must be explicit, configuration-backed, assigned a new run ID, written as a new output, and recorded in the manifest.

Claim-safety rule: Layer 10 artifacts are reporting evidence, not new experimental evidence. A dashboard figure, page, table, or card can support a thesis claim only when its source-record lineage, warning flags, configuration identity, split identity, and run identity are complete.

16.4 Input contracts

Layer 10 receives a broad set of inputs. They are grouped by role so that the dashboard does not confuse scientific evidence, metadata, proxy labels, and optional stretch signals.

Input class	Records and files	Layer 10 use
Primary inputs	DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, DatasetCard, ProtocolCard, ValidationReport, Layer1Manifest, layer1_ablation_readiness_11_manifest, PredictionRecord, ModelRegistryRecord, BaselineMetricRecord, LowCalibrationCurveRecord, SubjectProfileRecord, EnsembleControlRecord, Layer2ReadinessReport, ModelCard, BenchmarkCard, CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, SplitIntegrityReport, ReliabilityAuditReport, HighConfidenceWrongTable, CalibrationDashboardMetricRecord, ReliabilityCard, EvidenceQualityRecord, IHARQDecisionRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, Layer 4 EvaluationOutcomeRecord, RegimeStateRecord, conditional TrustStateRecord, PolicyActionRecord, TransitionRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, RewardTraceRecord, EnvironmentDiagnosticRecord, SafetyEventRecord where materialized, and optional PolicyUpdateTrace.	Main scientific dashboard views, ablations, figures, tables, and cards, including Layer 2 decoder-readiness/card artifacts, Layer 3 calibration/reliability report artifacts, and Layer 4 IHARQ deci- sion/evidence/combination/fallback/unsafe/ablation artifacts, as read-only upstream outputs; TrustStateRecord is required only when trust-state plots, trusted-set evolution, or per-target trust trends are rendered from independent Layer 5 records, and Layer 7 records are read only for closed-loop session timelines, policy-rollout cards, reward decomposition/regret views, and environment-diagnostic warning panels.

Auxiliary inputs	EvidenceQualityRecord, ReadinessContextRecord, DeferralRecord, model registry, decoder registry, threshold registry, policy registry.	Explain policy decisions, evidence quality, threshold origins, fallback behavior, and readiness-context provenance.
Metadata inputs	Split manifests, preprocessing reports, configuration files, seed logs, run registry, environment files, version files, data-source notes, license notes, layer1_ablation_readiness_l1_warnings, Layer2ReadinessReport, SplitIntegrityReport, LeakageWarningRecord, CalibrationEligibilityTable, CalibrationDashboardMetricRecord, and matched-comparison key completeness reports.	Reproducibility, validation, leakage checks, artifact lineage, Layer 2 downstream-readiness status, Layer 3 calibration/split legality, ablation-readiness warnings, and thesis appendix material.

Evaluation-only inputs	Correctness labels, task labels, held-out evaluation targets, cost configuration, reward terms, <code>TransitionRecord</code>, <code>RewardTraceRecord</code>, <code>EpisodeSummaryRecord</code>, <code>PolicyRolloutRecord</code>, <code>ClosedLoopSessionRecord</code>, <code>EnvironmentDiagnosticRecord</code>, <code>SafetyEventRecord</code> where materialized, <code>Layer9RewardTrace</code> entries where enabled, reward-decomposition tables, regret logs, optional <code>PolicyUpdateTrace</code> files, disabled-update audit entries, ablation summaries, matched-coverage groups, matched-cost groups, matched-stress groups, matched embodiment groups, A0 raw-decoder tables, A1 calibration deltas, A2 threshold comparisons, A3 selective-prediction/uncertainty summaries, A4 ordinary-combination controls and <code>EnsembleControlRecord</code>, A12 stress-evaluation outcomes, A13 embodiment-evaluation outcomes, <code>HighConfidenceWrongTable</code>, <code>Layer3MatchedOperatingPointTable</code>, <code>MatchedComparisonReports</code>, <code>MetricDictionaryEntry</code> snapshots or IDs, and <code>NegativeResultNotes</code>.	Evaluation plots, policy-update trace cards, Layer 7 closed-loop timelines, policy rollout cards, reward-decomposition/reward-regret views, environment-diagnostic warning panels, Layer 8 stress robustness views, Layer 9 reward/outcome summaries, training/adaptation versus frozen-evaluation curves, A0–A13 evaluation-slice views, high-confidence-wrong audit views, matched operating-point tables, and ablation tables; must not be displayed as policy inputs unless marked as evaluation-only.
Proxy inputs	Fallback or expert-proxy records, simulator success labels, embodiment endpoint error, <code>SafetyEventRecord</code> or <code>safety_event_log.csv</code>, safety-event proxies, proxy evidence-quality targets.	Must carry proxy-label and simulation-only warning flags; used for analysis, not clinical claims or real-actuation safety claims.

Stretch inputs	StressProfileRecord , StressScheduleRecord , EmbodimentStateRecord , EmbodimentOutcomeRecord , TransitionRecord embodiment linkages, SafetyEventRecord or safety_event_log.csv , command timelines, endpoint-error curves, Layer9RewardTrace entries where enabled, Layer9EmbodimentDemoManifest , dashboard-compatible Layer 9 exports, simulator screenshots, video references, embodiment configuration, command-mapping configuration, and simulator version files.	StressForge, stress-schedule inspection, Layer 9 MyoSuite/OpenSim/static-replay embodiment page, simulation-only pages, embodiment cards, and reproducibility manifests.
Layer 0 inputs	Claim ledger, claim-safety labels, public-data limitations, simulation-only flags, clinical non-claim text, thesis-scope boundaries, evaluation-mode labels, ablation-step IDs, MetricDictionaryEntry references, MatchedComparisonReports , dashboard/card/export lineage, NegativeResultNotes , and ClaimEvidenceMatrix entries.	Warning engine, safety cards, limitation summaries, claim-to-artifact mapping, ablation-lineage validation, diagnostic-only labels, and thesis-safe evaluation wording.

The following upstream outputs must exist before Layer 10 can run meaningfully: at least one dataset/split manifest from Layer 1, at least one prediction table from Layer 2, at least one configuration manifest and run ID, and a claim-safety scope definition from Layer 0. Other views can be marked incomplete until their corresponding upstream layers have produced records.

16.4.1 Canonical records consumed by Layer 10

Record	Produced by	Layer 10 use
DatasetRecord	Layer 1	Dataset/protocol page, dataset cards, split summaries, source/license notes, public-data limitations, and matched-comparison grouping for ablations.
WindowRecord	Layer 1	Trial/window inspection, preprocessing summary, artifact flags, label-budget analysis.

SplitRecord	Layer 1	Split-membership inspection, leakage checks, calibration/adaptation budget reporting, matched-comparison grouping, and reproducibility manifests.
PreprocessingRecord	Layer 1	Preprocessing-summary views, preprocessing-version validation, normalization-scope checks, and dataset/protocol card provenance.
LabelMapRecord	Layer 1	Original-to-normalized label inspection, task-family reporting, proxy-intent boundary warnings, and label-map version lineage.
ArtifactFlagRecord	Layer 1	Data-quality warnings, missing/bad-channel reports, artifact/short-trial limitation summaries, and dashboard missingness panels.
DatasetCard	Layer 1	Read-only dataset scope, source/citation, inclusion/exclusion, public-data limitations, allowed/forbidden claims, and dataset-card export lineage.
ProtocolCard	Layer 1	Read-only protocol scope, split/preprocessing/windowing IDs, leakage controls, low-calibration budget summaries, downstream-consumer notes, and diagnostic-readiness warnings.
ValidationReport	Layer 1	Layer 1 schema validity, metadata completeness, split disjointness, leakage status, label-budget integrity, comparison-key completeness, and pass/warning/fail dashboard state.
Layer1Manifest	Layer 1	Layer 1 output-path index, protocol-version lineage, validation-status display, ablation-readiness linkage, and reproducibility package entry.
layer1_ablation_readiness	layer1_v1.json	Ablation-readiness panels, matched-comparison key completeness warnings, diagnostic-only downgrade indicators, A0–A4 evaluation-lineage checks, and reproducibility package entry.
PredictionRecord	Layer 2	Decoder benchmark view, low-calibration curves, subject difficulty profiles, model comparisons, confusion summaries, A0 raw-decoder evidence views, and A4 ordinary-combination-control lineage.

ModelRegistryRecord	Layer 2	Decoder/model identity inspection, model-card lineage, score-type and class-order checks, artifact-path validation, and model-family grouping.
BaselineMetricRecord	Layer 2	Baseline accuracy/balanced-accuracy/F1 summaries, seed or confidence-interval display, and A0 decoder-baseline thesis tables.
LowCalibrationCurveRecord	Layer 2	Low-label and low-calibration curves, budget-aware comparison views, and calibration-budget limitation reporting.
SubjectProfileRecord	Layer 2	Subject/session difficulty profiles, low-label summaries, RegimeRisk diagnostic context, and subject-level dashboard drill-downs.
EnsembleControlRecord	Layer 2	A4 ordinary-combination-control tables, majority-vote/probability-averaging/longer-window comparisons, ensemble-disagreement summaries, delay-burden notes, and ensemble-control card lineage.
FailureCaseIndex	Layer 2	Decoder-level failure-case dashboard, failed/skipped run visibility, difficult-subject/session drill-downs, raw high-confidence wrong candidate review, and thesis failure-analysis pointers.
Layer2ReadinessReport	Layer 2	Downstream compatibility/readiness panel for prediction, registry, budget, score-schema, class-order, subject-profile, and ensemble-control coverage; blocks silent dashboard use of incomplete decoder outputs.
ModelCard	Layer 2 records rendered by Layer 10	Decoder/model card views, registry linkage, score-type/class-order warnings, baseline/low-calibration summaries, failed-run status, and model limitation exports.
BenchmarkCard	Layer 2 records rendered by Layer 10	Decoder benchmark card views, A0/A4 baseline lineage, low-calibration/subject-profile/ensemble-control summaries, matched-comparison keys, and diagnostic-only warnings.
CalibrationRecord	Layer 3	Reliability diagrams, ECE, Brier score, NLL summaries, calibration cards, A1 calibration deltas, selective reliability analysis, and metric-dictionary linkage.
UncertaintyRecord	Layer 3	Confidence histograms, entropy/margin distributions, high-confidence wrong cases, selective-prediction baselines.

SelectivePredictionRecord	Layer 3	Risk-coverage curves, retained accuracy, rejection-rate summaries, threshold baselines, matched-coverage comparisons, matched-cost comparisons, A2 threshold comparisons, A3 selective-prediction summaries, and negative-result notes.
ThresholdRegistryRecord	Layer 3	Threshold origin audits, allowed-consumer checks, leakage-status checks, threshold cards or appendix tables.
SplitIntegrityReport	Layer 3	Calibration/threshold split-legality panels, leakage-lockout warnings, validation/test separation evidence, and reproducibility-readiness views.
ReliabilityAuditReport	Layer 3	Reliability-audit page inputs, calibration-card warnings, high-confidence failure summaries, group-wise reliability checks, and negative-result linkage.
LeakageWarningRecord	Layer 3	Visible leakage-warning panels, claim-safety blockers, protocol-repair pointers, and dashboard warning propagation for calibration or threshold-selection violations.
CalibrationEligibilityTable	Layer 3	Calibration-budget/eligibility views, pooled-only or sparse-budget warnings, method-eligibility filters, and threshold-selection legality checks.
HighConfidenceWrongTable	Layer 3	High-confidence-wrong case page, failure-case filters, offline failure-analysis links for Layers 4/6, and evaluation-only warning coverage.
CalibrationDashboardMetricRecord	Layer 3	Plot-ready Layer 3 metric exports, bin-table and risk-coverage lineage, source-record pointers, card references, export manifests, and reliability-dashboard backend inputs.
ReliabilityCard	Layer 3 records rendered by Layer 10	Calibration, uncertainty, selective-prediction, threshold, high-confidence-error, and group-wise reliability card views with source-record lineage and claim boundaries.
EvidenceQualityRecord	Layers 4 and 6	Evidence-quality views, quality-estimator cards, readiness-target analysis, policy-context explanation.

IHARQDecisionRecord	Layer 4	IHARQ flow view, canonical action-token validation, display-label rendering, reason-code analysis, evidence-request counts, fallback and unsafe status, uncertain status, accepted-trial accuracy, action-cost summaries, and policy/simulation handoff lineage.
CombinedEvidenceRecord	Layer 4	Request/combine timelines, combination eligibility audits, before/after evidence summaries, delay/evidence-cost plots, and final-combined-decision lineage.
FallbackDecisionRecord	Layer 4	Fallback route tables, fallback availability and cost summaries, fallback success/failure evaluation where scored, embodiment-compatible hold/no-action/stop route lineage, and defensive-design warnings.
UnsafeEventRecord	Layer 4	Unsafe-event tables, blocked-action summaries, RegimeRisk/stress context warnings, simulation/proxy boundary labels, and claim-safety cards.
IHARQAblationRecord	Layer 4	A5/A6/A7 IHARQ ablation views, matched-comparison checks, metric-dictionary linkage, diagnostic-only warnings, and negative-result notes.
IHARQTraceRecord	Layer 4	Rule-path drilldowns, threshold/source-feature trace inspection, trace-completeness validation, figure/table lineage, and reproducibility-manifest linkage.
RegimeStateRecord	Layer 5	RegimeRisk timeline, trust-score summary, drift/fatigue/dropout risk panels, stop-loss warnings, temporal failure maps, risk-to-error alignment, and links to TrustStateRecord rows where emitted.
TrustStateRecord	Layer 5	Per-target trust-score timelines, trust-band/trend plots, trusted-set evolution, trust-update reason-code tables, trust-allocation drill-downs, and source RegimeStateRecord lineage when independent trust timelines are used.
ReadinessContextRecord	Layers 3–8	Adaptive policy context inspection, feature provenance, leakage and evaluation-only field checks.
DeferralRecord	Layer 6	Learning-to-defer views, fallback routing, wait/request patterns, matched-cost and matched-coverage analysis.

PolicyActionRecord	Layer 6	Policy comparison, action probabilities, thresholds, legality masks, reward estimates, reason codes.
TransitionRecord	Layers 7 and 9	Closed-loop session view, reward/regret curves, delayed consequences, safety events, embodiment-linked outcomes, Layer 9 embodiment transition linkage, and linkage to PolicyUpdateTrace through policy_update_trace_id where present.
ClosedLoopSessionRecord	Layer 7	Closed-loop session timeline, replay/session metadata, frozen-versus-update mode display, stress/session filters, reproducibility manifest, and simulation-only warning coverage.
EpisodeSummaryRecord	Layer 7	Episode return, cost, safety-event, stress, and policy-family summaries for A10/A11/A12 evaluation views, ablation tables, and thesis simulation summaries.
PolicyRolloutRecord	Layer 7	Policy rollout card/view, action-sequence drill-down, transition/reward-trace pointers, evidence-request and delay burden, safety-event linkage, and reproducibility manifest.
RewardTraceRecord	Layer 7	Reward decomposition, reward/regret dashboard, reward-sensitivity view, policy-update audit linkage, frozen/update separation, and simulation-only reward warning coverage.
EnvironmentDiagnosticRecord	Layer 7	Environment-diagnostic warning panels for invalid actions, failed transitions, reward anomalies, leakage checks, unmatched comparisons, missing safety-event links, missing reward-trace links, and disabled-update audit failures.
PolicyUpdateTrace	Layers 6 and 7	Optional simulation-authorized bandit/RL update trace card, update-completeness audit, training/adaptation versus frozen-policy evaluation separation, disabled-update audit, and simulation-only warning coverage.
StressProfileRecord	Layer 8	StressForge view, stress profile inspection, stress intensity, sensitivity curves, stress-conditioned ablations, A12 stress-test lineage, and Layer 9 stress-conditioned embodiment linkages.

StressScheduleRecord	Layer 8	StressForge schedule inspection, intensity timelines, per-window/per-step stress lineage, matched-stress validation, Layer 7 stress-conditioned session views, and Layer 9 stress-conditioned embodiment linkages.
EmbodimentStateRecord	Layer 9	Layer 9 MyoSuite/OpenSim/static-replay demo view, embodiment mode, simulator adapter branch, command mapping, endpoint error, assistive command timeline, outcome fields, reward linkage, unsafe-event status, and non-clinical warning coverage.
EmbodimentOutcomeRecord	Layer 9	Endpoint-error, command-success/failure, over/under-assist, safety-event pointer, transition pointer, command-timeline pointer, optional screenshot/reward-trace linkage, and A13 outcome-summary provenance.
SafetyEventRecord or safety_event_log.csv	Layers 7 and 9	Unsafe simulated event table, blocked-command analysis, safety-gate activation counts, no-action/fallback burden, closed-loop safety-event views, embodiment safety-event views, failure taxonomy entries, and safety-card evidence.
Layer9RewardTrace or reward-trace export	Layer 9	Simulation-only reward decomposition, reward/outcome bridge inspection, policy-learning feedback audit where enabled, and reward-hacking warning checks.
Layer9EmbodimentDemoManifest and dashboard-compatible Layer 9 export	Layer 9	Demo reproducibility, simulator version lineage, command-mapping configuration, screenshot/video references, exported endpoint-error curves, command timelines, and artifact completeness checks.
layer1_ablation_readiness	layer1_v1.json	Public-data/protocol ablation-readiness warnings, matched-comparison key completeness, and diagnostic-only downgrade explanations.
Layer3MatchedOperatingPoint	Layer 3	Matched-coverage and matched-cost threshold/selective-prediction comparisons, A1–A3 fairness checks, and threshold-sufficiency or calibration-failure notes.

ClaimEvidenceMatrix	Layer 0 and Layer 10	Claim-to-artifact mapping, evidence-mode labels, ablation-step linkage, MetricDictionaryEntry references, warning inheritance, and thesis-safe wording checks.
ExperimentSummaryRecord	And evaluation scripts	Ablation page, aggregate metrics, matched-cost, matched-coverage, matched-stress, matched-embodiment, and contribution summaries.
EvaluationOutcomeRecord	Layers 4–9 and evaluation scripts	Layer-local evaluation outcomes for A5–A13, including Layer 8 A12 stress evidence and Layer 9 A13 embodiment evidence; used by ablation, claim-evidence, negative-result, card, and export views.
MatchedComparisonReport	Layers 1–9 and evaluation scripts	Validation that compared variants share required records, keys, seeds, stress schedules, cost/reward definitions, embodiment mappings, simulator configs, horizons, and metric-dictionary references.
MetricDictionaryEntry	Evaluation/configuration scripts	Metric definitions, directions, aggregation units, required fields, ablation IDs, warning rules, and claim boundaries for dashboard-safe interpretation.
NegativeResultNote	Layer-local evaluation scripts	Visible explanation of failed, null, unmatched, reward-sensitive, stress-sensitive, adapter-failed, or diagnostic-only results.
DashboardMetricRecord	Layer 10	Dashboard-ready metric registry, ablation/matched-comparison/metric-dictionary lineage, diagnostic-only status, export references, warning flags, card references, and artifact lineage.

16.4.2 Required metadata and validation rules

Every record consumed by Layer 10 should include the following metadata where applicable: record ID, record type, producing layer, producing module or script, dataset ID, subject ID, session ID, trial ID, window ID, split ID, model ID, policy ID, policy-update mode, policy-update trace ID, closed-loop session ID, episode ID, policy-rollout ID, reward-trace ID, environment-diagnostic ID, stress profile ID, stress schedule ID, embodiment run ID, embodiment mode, simulator adapter branch, command-mapping ID, safety-event ID, reward-trace ID, demo-manifest ID, ablation-step ID, matched-comparison ID, metric-dictionary ID, operating-point ID, diagnostic-only status, negative-result note ID, claim-evidence ID, configuration ID, random seed, run ID, timestamp or simulated time index, schema version, code version, and checksum or hash where feasible.

Layer 10 validates inputs before producing views or cards.

Validation check	Purpose
Schema validation	Confirm required fields, types, enumerations, valid column names, and allowed action-token vocabularies.
Action-token and alias validation	Ensure <code>IHARQDecisionRecord.action</code> uses only the Layer 4 canonical tokens <code>accept</code> , <code>wait</code> , <code>request_evidence</code> , <code>combine_evidence</code> , <code>fallback</code> , <code>uncertain</code> , and <code>unsafe</code> ; render human-readable labels through a read-only mapping; flag hyphenated request-evidence labels, bare combine aliases, <code>defer</code> , <code>no_action</code> , hold, unsupported mapping, simulator error, or unsafe-stop values when they appear in the wrong stored field.
ID consistency check	Ensure dataset, subject, session, trial, window, model, policy, stress profile, stress schedule, and run IDs align across records.
Split consistency check	Ensure training, validation, calibration, selection, and test records are not mixed incorrectly.
Metric range check	Detect impossible values such as negative accuracy, confidence above 1, invalid probability sums, or impossible endpoint-error units.
Missingness report	Identify missing records, missing columns, missing cards, missing figures, and incomplete artifact chains.
Version compatibility check	Ensure records were generated by compatible pipeline, schema, and configuration versions.
Threshold leakage check	Ensure threshold selection records are tied to the correct split and allowed consumers.
Claim-safety flag check	Ensure public-data, proxy-label, simulation-only, embodiment-demo, diagnostic-only, and clinical non-claim flags are present when required.
Ablation-lineage check	Ensure A0–A13 evaluation-slice artifacts carry ablation-step IDs, metric-dictionary entries, matched-comparison source-record IDs, Layer 1 ablation-readiness status where relevant, Layer 8/Layer 9 local evaluation ownership where relevant, and Layer 0 claim-evidence boundaries.
Layer 7 closed-loop lineage check	Ensure <code>ClosedLoopSessionRecord</code> , <code>EpisodeSummaryRecord</code> , <code>PolicyRolloutRecord</code> , <code>RewardTraceRecord</code> , <code>EnvironmentDiagnosticRecord</code> , <code>TransitionRecord</code> , <code>SafetyEventRecord</code> , and optional <code>PolicyUpdateTrace</code> identifiers agree before closed-loop timelines, reward-decomposition panels, policy-rollout cards, or environment-diagnostic warnings are marked thesis-ready.

A12/A13 stress-embodiment lineage check	Ensure Layer 8 A12 stress outcomes and Layer 9 A13 embodiment outcomes expose matched-stress or matched-embodiment status, StressProfileRecord and StressScheduleRecord lineage where stress is active, metric dictionaries, negative-result notes where applicable, claim-evidence IDs, simulation/stress/embodiment warnings, and dashboard/export lineage before thesis-ready export.
Negative-result visibility check	Ensure calibration failure, threshold sufficiency, selective-prediction sufficiency, and unmatched-comparison downgrades are exported as visible limitation/evaluation evidence rather than silently omitted.
Policy-update mode check	Ensure frozen-policy evaluation records contain no unlogged policy mutation, simulation-authorized bandit/RL updates have corresponding PolicyUpdateTrace entries, and disabled-update audit entries are visible when update hooks are intentionally off.
Layer 9 embodiment export check	Ensure EmbodimentStateRecord, EmbodimentOutcomeRecord, TransitionRecord embodiment linkage, SafetyEventRecord or safety-event log, Layer9RewardTrace where enabled, Layer9EmbodimentDemoManifest, command-mapping ID, simulator version, dashboard-compatible export, and required simulation-only warnings are mutually consistent before thesis-ready export.

16.5 Output contracts

Layer 10 produces derived, thesis-facing artifacts. Each output must have a downstream consumer and an audit trail.

Output type	Examples	Primary consumers
Dashboard views	Dataset, decoder, calibration, threshold, IHARQ, RegimeRisk, policy, deferral, bandit/RL, policy-update trace, StressForge, Layer 9 embodiment, A12/A13 evaluation-lineage, ablation, failure taxonomy, cards/export.	Researcher, supervisor, examiner, debugging workflow.

DashboardViewRecords	Page-level view metadata with page ID, panel IDs, filters, source	Page renderer, card generator, export manager, artifact manifest, reproducibility manifest.
Layer 6 policy-matrix exports	DashboardMetricRecords, evaluation-lineage references, warning banners, completeness state, export references, and thesis usage.	
	Policy-stage matrix, context-level ablation matrix, action-legality matrix, cost/reward sensitivity matrix, policy-to-layer dependency matrix, and claim-boundary matrix.	Policy chapter, Layer 6 controller cards, ablation chapter, examiner-facing claim-safety review.
DashboardMetricRecords	View-level metrics with filters, source records, ablation/matched-comparison/metric-dictionary lineage, diagnostic-only status, warning flags, card references, and export references.	Dashboard renderer, export manager, artifact manifest, thesis tables.
CardRecords and card files	Dataset, protocol, decoder, calibration, IHARQ, RegimeRisk, Adaptive-IHARQ, learning-to-defer, contextual bandit, RL, policy-update trace, StressForge, Layer 9 embodiment evidence, safety, failure-taxonomy cards, each with card metadata, source-record lineage, warning flags, limitations, artifact paths, and thesis usage.	Thesis methodology/results chapters, appendix, repository release, examiner, cards/export page, artifact manifest, and ReproducibilityManifest.
Static figures	Reliability diagrams, low-calibration curves, risk timelines, decision-flow plots, reward/regret curves separated by training/adaptation versus frozen-policy evaluation mode, policy-update trace completeness figures where applicable, stress sensitivity curves, Layer 9 endpoint-error figures, command timeline figures, unsafe-event summaries, and reward-decomposition plots where enabled.	Thesis chapters, paper figures, supervisor review.

Tables	Metrics tables, ablation tables, artifact completeness tables, card completeness tables, validation tables, run-registry summaries.	Thesis tables, appendix, repository release.
Logs	Validation logs, missingness logs, warning logs, export logs, card-generation logs, Layer 9 safety-event export checks, disabled-update audit logs, and policy-update trace validation logs.	Debugging, auditability, reproducibility appendix.
Manifests	Artifact manifest, ReproducibilityManifest , environment manifest, configuration manifest, run registry, Layer 9 demo-manifest index, and dashboard-compatible export index.	Future reproducer, examiner, repository release, thesis reproducibility appendix, and final demonstration evidence package.
Failure taxonomy	Failure categories, frequencies, representative examples, affected layers, mitigation or interpretation.	Discussion, limitations, threats-to-validity chapter.
Package index	List of files needed to reproduce the main results and regenerate thesis artifacts.	Repository release and artifact review.

16.5.1 DashboardMetricRecord schema

The `DashboardMetricRecord` is the Layer 10 record that represents dashboard-ready information. It does not replace raw records. It connects a view, metric, filter context, warning flags, card references, export references, and lineage.

Field	Meaning
<code>metric_record_id</code>	Stable unique ID for the dashboard metric record.
<code>view_name</code>	Dashboard page or view name.
<code>metric_name</code>	Name of the displayed metric.
<code>metric_level</code>	Dataset, subject, session, trial, window, closed-loop session, episode, policy rollout, reward trace, policy, policy-update, stress, embodiment, or aggregate level.
<code>metric_value</code>	Numeric, categorical, vector, table pointer, or plot-data pointer.
<code>metric_units</code>	Units or interpretation, such as accuracy, ECE, cost, regret, milliseconds, endpoint-error units, or count.
<code>time_axis</code>	Trial, session, run, or simulated time axis if applicable.
<code>dataset_filter</code>	Dataset filter used to compute the metric.
<code>subject_filter</code>	Subject filter used to compute the metric.
<code>session_filter</code>	Session filter used to compute the metric.
<code>model_filter</code>	Decoder/model filter used to compute the metric.

<code>policy_filter</code>	Policy or controller filter used to compute the metric.
<code>policy_update_filter</code>	Optional filter for frozen-policy evaluation, disabled-update audit, simulation-training, or simulation-adaptation mode when <code>PolicyUpdateTrace</code> records are present.
<code>stress_filter</code>	Stress profile, stress schedule, stress family, intensity, schedule type, or active stress-component filter.
<code>embodiment_filter</code>	Optional filter for Layer 9 embodiment mode, simulator adapter branch, mapping ID, command family, safety state, endpoint-error family, or demo run.
<code>reward_trace_filter</code>	Optional filter for Layer 7 <code>RewardTraceRecord</code> or Layer 9 reward-trace ID, reward component, simulation-only feedback mode, frozen/update mode, reward configuration, or reward/outcome bridge configuration.
<code>source_records</code>	IDs or paths of records used to compute the metric.
<code>source_layers</code>	Layers that produced the underlying records.
<code>ablation_step_id</code>	Optional global A0–A13 ablation-step ID connected to the metric.
<code>local_ablation_id</code>	Optional layer-local ablation ID, such as Layer 8 A12 or Layer 9 A13 local evaluation identifier.
<code>matched_comparison_id</code>	Optional <code>MatchedComparisonReport</code> identifier or path used to establish fair comparison lineage.
<code>metric_dictionary_id</code>	Optional <code>MetricDictionaryEntry</code> identifier, snapshot ID, or metric-dictionary path used to define the metric.
<code>diagnostic_only_status</code>	Thesis-ready, diagnostic-only, exploratory, or blocked status when matching, lineage, warning, or metric-definition requirements are incomplete.
<code>negative_result_note_id</code>	Optional <code>NegativeResultNote</code> identifier for failed, null, unmatched, or limitation-bound results.
<code>claim_evidence_id</code>	Optional <code>ClaimEvidenceMatrix</code> entry ID linking the metric to an allowed thesis claim or limitation.
<code>config_id</code>	Configuration ID used to generate the derived metric.
<code>run_id</code>	Source experiment run or dashboard-export run.
<code>warning_flags</code>	Claim-safety, data-quality, simulation-only, proxy-label, leakage, or missingness warnings.
<code>card_references</code>	Cards connected to this metric.
<code>export_references</code>	Figures, tables, or reports generated from this metric.

16.5.2 DashboardViewRecord schema

The `DashboardViewRecord` is the Layer 10 record that represents dashboard-view metadata. It is not a new scientific result and it does not replace `DashboardMetricRecord`; it records how saved metrics and source records are assembled into an inspectable page or panel, which warnings were shown, which filters were applied, and whether the page is complete enough for thesis use.

Field	Meaning
<code>view_record_id</code>	Stable unique ID for the dashboard view record.

<code>page_name</code>	Dashboard page name, such as dataset, decoder, calibration, IHARQ, RegimeRisk, policy, closed-loop session, policy rollout, reward decomposition, environment diagnostics, stress, embodiment, ablation, failure taxonomy, or cards/export.
<code>panel_id</code>	Stable identifier for the page panel, figure slot, table slot, timeline, card slot, or drill-down component.
<code>inspection_question</code>	The scientific or reproducibility question the page or panel is intended to answer.
<code>metric_record_ids</code>	<code>DashboardMetricRecord</code> identifiers used by the page or panel.
<code>source_records</code>	IDs or paths of upstream records reachable through the page or panel lineage.
<code>filters</code>	Dataset, subject, session, model, policy, policy-update, stress, embodiment, reward-trace, run, or time-window filters applied in the view.
<code>view_state</code>	Complete, incomplete, blocked, stale, non-thesis-ready, or thesis-ready state.
<code>warning_flags</code>	Claim-safety, data-quality, simulation-only, proxy-label, leakage, missingness, stale-artifact, or unresolved-lineage warnings displayed in the view.
<code>completeness_status</code>	Required records and panels present, missing, blocked, or not applicable.
<code>export_references</code>	Figure, table, card, screenshot, static-report, or dashboard-bundle artifacts generated from the view.
<code>claim_references</code>	Thesis claim IDs or limitation IDs connected to the view.
<code>evaluation_lineage_references</code>	Optional references to A0–A13 IDs, layer-local ablation IDs, <code>MatchedComparisonReports</code> , <code>MetricDictionaryEntry</code> snapshots or IDs, <code>NegativeResultNotes</code> , and <code>ClaimEvidenceMatrix</code> entries needed by the view.
<code>config_id</code>	Dashboard or export configuration used to create the view.
<code>run_id</code>	Dashboard rendering or export run ID, plus source experiment run IDs where applicable.
<code>checksum</code>	Optional checksum of the rendered view, snapshot, or serialized view metadata.

A `DashboardViewRecord` may be regenerated from the same source records and dashboard configuration. If the source records, filters, warning flags, or export settings change, a new view record or versioned view record must be produced rather than silently overwriting thesis-ready evidence.

16.5.3 Artifact identity contract

Every exported artifact must satisfy the following contract: artifact ID, artifact type, artifact title, source records, source layers, filters, generating script or dashboard page, configuration ID, run ID, random seed where applicable, ablation-step ID where applicable, matched-comparison ID where applicable, metric-dictionary ID or snapshot where applicable, diagnostic-only status

where applicable, negative-result note ID where applicable, claim-evidence ID where applicable, export path, export timestamp, thesis chapter or figure/table reference, claim supported, warning flags, known limitations, and reproduction status.

Recommended ID examples include FIG_CAL_001_reliability_diagram, FIG_IHARQ_001_decision_flow, FIG_REGIME_001_risk_timeline, FIG_L9_001_endpoint_error, TAB_L9_001_safety_events, TAB_REPRO_001_artifact_manifest_summary, CARD_L9_001_embodiment_evidence, and CARD_SAFETY_001_claim_safety_summary.

16.6 Dependency logic

Layer 10 has many dependencies, but not all have the same strength.

Dependency type	Dependencies	Development implication
Hard dependencies	Layer 0 claim-safety scope; Layer 1 dataset/split records; Layer 2 prediction records; configuration files; run IDs; record schemas.	Required for a meaningful minimum Layer 10. Without these, the layer can only display placeholder status.
Soft dependencies	Layer 3 calibration/uncertainty; Layer 4 IHARQ; Layer 5 RegimeRisk; Layer 6 policy; Layer 7 transition and optional PolicyUpdateTrace; Layer 8 stress; Layer 9 embodiment records, safety-event logs, reward traces, command timelines, endpoint-error curves, and Layer9EmbodimentDemoManifests.	Pages can be marked incomplete until upstream records exist. The dashboard should show missingness rather than fail silently.
Optional dependencies	Streamlit/Dash/Gradio, static HTML generation, notebook reports, video screenshots, advanced artifact browser.	Can be mocked or replaced by static exports. The thesis does not require a production web application.
Conceptual dependencies	Layer 0 scope boundaries, ablation ladder, claim-to-artifact mapping, failure taxonomy, reproducibility principles.	Required for academic defensibility, even if the implementation is simple.
Data/model dependencies	Public dataset versions, preprocessing configs, model registry, decoder IDs, threshold registry, policy IDs, policy-update configuration IDs, stress profile IDs, Layer 9 embodiment mode IDs, command-mapping IDs, simulator adapter branch IDs, simulator version IDs, and demo-manifest IDs.	Must be recorded in manifests to prevent ambiguous figures, ambiguous policy-update curves, ambiguous embodiment outputs, or unreproducible tables.

Metric dependencies	Accuracy, balanced accuracy, ECE, Brier score, NLL, retained accuracy, rejection rate, high-confidence wrong rate, fallback rate, unsafe rate, reward, regret, policy-update trace completeness, frozen/update separation pass rate, endpoint error, command success/failure, blocked-command rate, no-action/fallback burden, reward decomposition, safety-event coverage, and artifact completeness.	Must be computed from saved records, not hidden dashboard logic.
Software dependencies	Python version, package versions, dashboard framework version, plotting library versions, schema validation library, serialization formats.	Must be captured in the environment manifest and reproducibility guide.
Mockable dependencies	Synthetic records, toy prediction tables, toy stress profiles, miniature Layer 9 embodiment records, toy safety-event logs, dummy reward traces, <code>Layer9EmbodimentDemoManifests</code> , and dummy cards.	Useful for testing Layer 10 before the full pipeline is finished.

16.7 Internal module architecture

Layer 10 should be implemented as a small set of clear internal modules. These modules are allowed to share a repository area, but their responsibilities should remain separate.

Every Layer 10 backend module must document its module name, purpose, bottleneck addressed, scientific role, engineering responsibility, inputs, outputs, internal processing logic, dependencies, consumers, feedforward links, feedback signals, evaluation method, failure modes, reproducibility requirements, thesis evidence produced, and limitations. Core modules are the loader, validator, indexer, metric summarizer, view model builder, warning engine, export manager, and manifest builder. Supporting modules include page rendering, audit logging, and card templating. Optional modules include a fully interactive Streamlit, Dash, Gradio, or web dashboard. Future-work modules may include richer drill-down UI, automated supervisor-review checklists, or repository-release packaging assistants, but these must remain reporting tools rather than experimental engines.

Internal module	Technical responsibility	Status
-----------------	--------------------------	--------

Record loader	Load canonical records, configs, run registries, optional <code>PolicyUpdateTrace</code> files, disabled-update audit entries, Layer 9 <code>Layer9EmbodimentDemoManifests</code> , safety-event logs, reward traces, dashboard-compatible embodiment exports, and existing artifacts from storage.	Core.
Schema validator	Validate record contracts, ranges, IDs, split usage, warning flags, update-mode legality, <code>PolicyUpdateTrace</code> completeness, Layer 9 embodiment export-bundle completeness, and compatibility.	Core.
Record indexer	Index records by dataset, subject, session, trial, window, model, policy, policy-update trace, stress, embodiment run, command-mapping ID, safety-event ID, reward-trace ID, demo-manifest ID, and run ID.	Core.
Metric summarizer	Compute descriptive summaries from saved records, including separated training/adaptation and frozen-evaluation policy curves where applicable, Layer 9 endpoint-error, command-timeline, safety-event, and reward-decomposition summaries, and emit <code>DashboardMetricRecords</code> .	Core.
View model builder	Convert <code>DashboardMetricRecords</code> and indexed records into <code>DashboardViewRecords</code> with filters, panels, warning banners, completeness states, export references, and inspection questions.	Core.
View renderer	Render dashboard pages, plots, timelines, tables, cards/export panels, and drill-down views from <code>DashboardViewRecords</code> without changing upstream evidence.	Supporting or core depending on implementation.
Card generator	Generate standardized cards, including policy-update trace cards when optional update artifacts exist, Layer 9 embodiment evidence cards, and card-completeness reports.	Core.
Export manager	Export thesis figures, tables, cards, screenshots, and static report bundles.	Core.

Manifest builder	Generate artifact, reproducibility, environment, run, and Layer 9 demo-manifest indexes.	Core.
Warning engine	Apply claim-safety, simulation-only, proxy-label, leakage, incomplete-coverage, and clinical non-claim warnings.	Core.
Audit logger	Record which artifacts were generated from which records, configs, filters, scripts, and versions.	Core.

16.7.1 Module 10.1: Record Loader and Canonical Artifact Reader

Purpose. Load the saved records and configuration artifacts produced by Layers 1–9. This module solves the bottleneck of scattered outputs by creating a single read-only entry point for Layer 10.

Scientific role. It ensures that dashboard results are based on official pipeline outputs rather than ad hoc files.

Engineering responsibility. Read CSV, Parquet, JSON, YAML, and figure/card manifests from configured folders; normalize paths; preserve source identities; and pass records to the validator without mutating them.

Inputs. Canonical records, optional `PolicyUpdateTrace` files, disabled-update audit files, Layer 9 embodiment export bundles, safety-event logs, reward traces, `Layer9EmbodimentDemoManifests`, configuration files, split manifests, run registry, existing cards, existing figures, existing tables, and environment metadata.

Outputs. Loaded record batches, source-file inventory, missing-file inventory, and record provenance table.

Internal processing logic. Discover expected folders; load files by record type; attach path, checksum, schema version, and run ID; detect duplicates; and pass loaded batches to schema validation.

Dependencies. File organization convention, schema registry, run registry, storage format readers, and Layer 0 claim-safety file.

Consumers. Schema validator, record indexer, manifest builder, warning engine, dashboard views, and card generator.

Feedforward connections. Sends loaded records to the validator and sends inventory metadata to the artifact completeness table.

Feedback or loopback. If expected records are missing, emits a missing-record signal to the relevant upstream layer owner or experiment controller. It does not fabricate missing records.

Evaluation method. Unit tests using miniature valid and invalid repositories; missing-file tests; duplicate detection tests; checksum tests.

Failure modes. Silent path mismatch, duplicated run IDs, partially loaded record groups, stale files, and inconsistent naming. The defense is explicit inventory logs and visible missingness reports.

Reproducibility requirements. The loader configuration must be saved, and all loaded files

should be recorded with path and checksum where feasible.

Thesis evidence produced. Artifact inventory and record availability table for the reproducibility appendix.

Limitations and risks. The loader cannot guarantee that upstream experiments were correctly run; it can only validate the existence and structure of their saved outputs.

16.7.2 Module 10.2: Schema Validator and Completeness Auditor

Purpose. Check whether records satisfy the expected contracts before they are visualized or exported.

Bottleneck addressed. Prevents invalid, incomplete, or incompatible records from becoming thesis figures.

Scientific role. Protects the validity of the evidence chain by exposing missingness, leakage risks, incompatible versions, and missing warning flags.

Engineering responsibility. Validate required fields, types, enumerations, canonical Layer 4 action tokens and read-only display-label mappings, ID consistency, split consistency, metric ranges, schema versions, claim-safety flags, policy-update legality for optional bandit/RL update traces, and Layer 9 embodiment export-bundle consistency.

Inputs. Loaded records, schema registry, valid ID registry, split manifests, threshold registry, optional `PolicyUpdateTrace` files, policy-update configuration metadata, claim-safety rules, and configuration metadata.

Outputs. Validation report, pass/fail table, missingness report, schema-error log, warning-flag coverage report, policy-update mode audit, disabled-update audit status, Layer 9 embodiment export-readiness status, and artifact readiness status.

Internal processing logic. Apply per-record schema checks; cross-check IDs across record types; verify split legality; detect missing warning flags; summarize errors by layer and run; and mark records as usable, incomplete, or blocked.

Dependencies. Record schemas, Layer 0 claim-safety labels, split manifests, and run registry.

Consumers. Record indexer, warning engine, card generator, export manager, and thesis reproducibility appendix.

Feedforward connections. Valid records move to indexing and summarization. Invalid records are visible in the dashboard and blocked from thesis-ready export unless explicitly marked exploratory.

Feedback or loopback. Sends a schema-failure signal to the producing layer, with record type, failing fields, affected run ID, and required repair action.

Evaluation method. Schema tests, contract tests against sample outputs from Layers 1–9, missing-record tests, and warning-flag tests.

Failure modes. Overly strict validators can block useful exploratory results; overly loose validators can allow invalid figures. The mitigation is severity levels: error, warning, incomplete, exploratory, and thesis-ready.

Reproducibility requirements. Validation version, schema version, and validation logs must be saved.

Thesis evidence produced. Validation pass-rate table, missingness table, and claim-warning

coverage table.

Limitations and risks. Schema validity is not the same as scientific correctness. A record can pass schema checks and still reflect a weak experiment; evaluation interpretation remains necessary.

16.7.3 Module 10.3: Record Indexer and Lineage Resolver

Purpose. Build cross-layer links between records so that aggregate figures can be traced back to datasets, subjects, sessions, windows, models, policies, stress profiles, stress schedules, simulator runs, and configs.

Bottleneck addressed. Prevents the architecture from becoming a collection of unconnected tables.

Scientific role. Enables drill-down from thesis claims to source evidence and makes subject/session/stress-specific failure analysis possible.

Engineering responsibility. Create indexes over identity fields, join compatible records, resolve one-to-many and many-to-one relationships, link `TransitionRecord` entries to optional `PolicyUpdateTrace` IDs, and maintain lineage without overwriting upstream data.

Inputs. Validated records, ID registry, run registry, config registry, artifact registry, and card registry.

Outputs. Cross-record index, lineage graph, record-to-view mapping, record-to-card mapping, and source-record lookup service.

Internal processing logic. Index by dataset, subject, session, trial, window, model, calibration budget, threshold ID, IHARQ decision ID, policy ID, transition ID, stress profile ID, stress schedule ID, embodiment run ID, embodiment mode, simulator adapter branch, command-mapping ID, safety-event ID, reward-trace ID, demo-manifest ID, and artifact ID. Resolve lineage chains such as `PredictionRecord` → `CalibrationRecord` → `IHARQDecisionRecord` → `PolicyActionRecord` → `TransitionRecord` → `EmbodimentStateRecord` → Layer 9 safety-event, reward-trace, and demo-manifest exports.

Dependencies. Consistent IDs, record schemas, artifact identity standard, and run registry.

Consumers. Metric summarizer, dashboard pages, card generator, export manager, manifest builder, and audit logger.

Feedforward connections. Supplies indexed records and lineage chains to dashboards, cards, and artifact exports.

Feedback or loopback. Emits an ID-inconsistency signal when records cannot be linked because of mismatched identifiers.

Evaluation method. Join consistency tests, lineage completeness tests, duplicate-ID tests, and manual spot checks on representative artifacts.

Failure modes. Broken lineage, ambiguous joins, many-to-many explosion, stale artifact references, and inconsistent run IDs. The defense is explicit lineage-status fields and blocked thesis-ready export when lineage is unresolved.

Reproducibility requirements. Lineage graph or lookup tables must be exported so figures can be traced after the dashboard is closed.

Thesis evidence produced. Figure-to-record and table-to-record traceability reports.

Limitations and risks. Trial-level lineage may be unavailable for some public datasets or baseline implementations; such cases must be marked as aggregate-only.

16.7.4 Module 10.4: Metric Summarizer and DashboardMetricRecord Builder

Purpose. Convert validated records into descriptive summaries that the dashboard, cards, figures, and tables can use.

Bottleneck addressed. Avoids scattering metric calculations across dashboard pages and export scripts.

Scientific role. Ensures consistent definitions of accuracy, balanced accuracy, ECE, Brier score, NLL, rejection rate, retained accuracy, fallback rate, unsafe rate, closed-loop session completeness, episode return, policy-rollout behavior, reward, regret, reward decomposition, environment-diagnostic count, policy-update trace completeness, frozen/update separation, stress sensitivity, Layer 9 endpoint error, command success/failure, blocked-command rate, no-action/fallback burden, safety-event rate, and reward decomposition.

Engineering responsibility. Compute summaries from saved records only; emit DashboardMetricRecords with filters, source records, warnings, and export references.

Inputs. Indexed records, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, RewardTraceRecord, EnvironmentDiagnosticRecord, optional PolicyUpdateTrace files, Layer9RewardTrace files, safety-event logs, command timelines, endpoint-error outputs, metric configuration, cost configuration, reward/update-mode configuration, ablation definitions, stress definitions, stress schedules, and warning flags.

Outputs. DashboardMetricRecords, summary tables, plot-ready data, closed-loop session timeline metrics, policy-rollout summaries, reward-decomposition/reward-regret summaries, environment-diagnostic warning counts, separated training/adaptation versus frozen-evaluation curves, policy-update trace completeness tables where applicable, Layer 9 endpoint-error summaries, command-timeline summaries, safety-event tables, reward-decomposition summaries where enabled, ablation tables, and derived metric logs.

Internal processing logic. Apply deterministic metric functions; group by configured filters; separate simulation-training/adaptation metrics from frozen-policy evaluation metrics; attach source-record IDs; attach warning flags; create aggregate and subgroup summaries; and pass outputs to view renderer, cards, and export manager.

Dependencies. Valid records, metric definitions, source-record lineage, and configuration files.

Consumers. View renderer, card generator, export manager, ablation page, and thesis tables.

Feedforward connections. Sends dashboard-ready metrics to all visualization and reporting components.

Feedback or loopback. If a metric cannot be computed because required fields are missing, it emits a completeness signal to the validator and producing layer.

Evaluation method. Snapshot tests, deterministic recomputation tests, metric-range tests, and independent metric spot checks.

Failure modes. Inconsistent metric definitions, hidden filters, unmarked subgroup imbalance, and accidental recomputation from raw data. The mitigation is centralized metric definitions and explicit filter recording.

Reproducibility requirements. Metric configuration, code version, filters, source records,

and random seeds where relevant must be recorded.

Thesis evidence produced. Metrics tables, ablation summaries, dashboard metric registry, and figure-ready data.

Limitations and risks. Aggregated summaries can hide subject-level failure. Therefore Layer 10 must support subgroup and drill-down views where feasible.

16.7.5 Module 10.5: View Model Builder, Renderer, and Scientific Observability Pages

Purpose. Convert saved records and `DashboardMetricRecords` into `DashboardViewRecords`, then render dashboard pages and static views that let a reader inspect what happened, why it happened, and which records support it.

Bottleneck addressed. Makes the architecture understandable without manually opening dozens of logs.

Scientific role. Converts structured evidence into inspectable visual arguments while preserving claim-safety warnings and source-record links.

Engineering responsibility. Build page-specific view models; assign page and panel IDs; attach filters, warning banners, completeness states, record drill-down links, card/export references, and inspection questions; then render tables, plots, timelines, cards, filters, export buttons, and completeness indicators.

Inputs. `DashboardMetricRecords`, indexed records, `ClosedLoopSessionRecord`, `PolicyRolloutRecord`, `RewardTraceRecord`, `EnvironmentDiagnosticRecord`, optional `PolicyUpdateTrace` indexes, Layer 9 dashboard-compatible exports, `Layer9EmbodimentDemoManifests`, card registry, artifact manifest, validation report, warning flags, and page specifications.

Outputs. `DashboardViewRecords`, interactive dashboard pages, static HTML reports, notebook reports, screenshots, serialized view metadata, or dashboard export bundles.

Internal processing logic. Load page specification; apply filters; create `DashboardViewRecords`; render required views, including Layer 7 closed-loop session timelines, policy rollout views, reward-decomposition panels, and environment-diagnostic warnings; display warning flags; expose source-record lineage; separate training/adaptation displays from frozen-policy evaluation displays; treat Layer 9 embodiment views as read-only exports rather than simulator controls; record completeness and blocked-export state; and pass selected views to the export manager.

Dependencies. Dashboard framework or static report generator, plot libraries, metric records, warning engine, and lineage resolver.

Consumers. Researcher, supervisor, examiner, card generator, export manager, thesis writing process.

Feedforward connections. Sends `DashboardViewRecords`, selected views, panel metadata, and plot specifications to the card generator, export manager, audit logger, and manifest builder.

Feedback or loopback. If a page cannot render because records are missing, it shows the missingness and points to the required upstream record rather than hiding the page.

Evaluation method. Snapshot tests, page completeness tests, visual spot checks, export consistency tests, and supervisor/examiner usability inspection.

Failure modes. Cosmetic pages without scientific value, hidden filters, misleading aggregation, and warning banners not shown. The defense is the page specification checklist.

Reproducibility requirements. Every rendered export must be linked to source records, filters, config, and export date.

Thesis evidence produced. Architecture overview screenshots, dashboard-derived thesis figures, and reviewer inspection views.

Limitations and risks. A dashboard view improves inspection but does not itself prove a scientific claim; the claim is supported only by the underlying records and evaluation design.

16.7.6 Module 10.6: Card Generator and Card Completeness Checker

Purpose. Generate standardized cards that summarize datasets, protocols, decoders, calibration, IHARQ, RegimeRisk, policies, deferral, bandit/RL, closed-loop environments, policy rollouts, optional policy-update traces, reward configurations, stress, embodiment, safety, limitations, and failure taxonomy.

Bottleneck addressed. Prevents important modules from being summarized inconsistently or without limitations.

Scientific role. Makes each major object in the architecture independently understandable and claim-safe.

Engineering responsibility. Fill card templates from records and metrics; attach source-record lineage; attach warning flags; check required fields; export cards as Markdown, HTML, PDF, or thesis appendix material.

Inputs. Card templates, card registry, indexed records, optional `PolicyUpdateTrace` files, `DashboardMetricRecords`, warning flags, source-record lineage, and thesis location mapping.

Outputs. `CardRecords`, card files, card completeness report, card registry, and card-to-thesis mapping.

Internal processing logic. Select card type; load required source records; fill the `CardRecord` universal card schema; insert metrics, plots, interpretations, limitations, warning flags, reproduction info, and thesis usage; validate card completeness; export.

Dependencies. Card templates, source records, metric summarizer, warning engine, export manager, and manifest builder.

Consumers. Thesis methodology/results/discussion chapters, appendix, supervisor, examiner, future researcher, dashboard cards/export page.

Feedforward connections. Sends generated cards to export manager and manifest builder.

Feedback or loopback. Emits card-incomplete signals when required records, metrics, limitations, or warning flags are missing.

Evaluation method. Card-generation tests, template tests, completeness tests, warning-flag tests, and manual review of representative cards.

Failure modes. Superficial cards, missing limitations, missing source records, unmarked simulation-only claims, and inconsistent metrics. The defense is the `CardRecord` universal card schema and card completeness gate.

Reproducibility requirements. Each card must record card ID, version, generated date, generating script, source records, config ID, run ID, environment, command, artifact path,

warning flags, and thesis location.

Thesis evidence produced. Dataset cards, protocol cards, reliability cards, controller cards, policy-update trace cards where relevant, stress cards, embodiment cards, safety cards, and failure-taxonomy cards.

Limitations and risks. Cards support but do not replace thesis explanation. A card should summarize evidence and limitations, not act as an unsupported conclusion.

16.7.7 Module 10.7: Export Manager for Thesis Figures and Tables

Purpose. Export static figures, tables, card bundles, and report bundles in formats suitable for thesis writing and repository release.

Bottleneck addressed. Prevents manual, untraceable figure creation.

Scientific role. Ensures every thesis figure and table has a known source, filter, configuration, and warning status.

Engineering responsibility. Generate deterministic exports; assign artifact IDs; record filters and source records; create export logs; and write figure/table paths to the artifact manifest.

Inputs. View specifications, plot-ready metrics, summary tables, optional policy-update trace summaries, card outputs, artifact naming rules, thesis location mapping, and warning flags.

Outputs. PNG/PDF/SVG figures, CSV/Parquet/Markdown/LaTeX-ready tables, separated training/adaptation and frozen-evaluation curve exports, policy-update trace card bundles where relevant, dashboard screenshots, export logs, and artifact manifest entries.

Internal processing logic. Receive an export request; validate source metric records; apply selected filters; render with fixed export settings; assign artifact ID; write file; compute checksum; update manifest; and write export log.

Dependencies. Metric summarizer, view renderer, card generator, artifact manifest schema, plotting libraries, and filesystem conventions.

Consumers. Thesis manuscript, appendix, repository release, supervisor review, examiner review.

Feedforward connections. Sends artifact references to manifest builder and card generator.

Feedback or loopback. If an export lacks source-record lineage or warning flags, it is blocked from thesis-ready status.

Evaluation method. Figure-generation tests, export determinism tests, checksum tests, snapshot tests, and artifact-lineage tests.

Failure modes. Untracked manual edits, nondeterministic exports, hidden filters, stale figures, and mismatched thesis references. The mitigation is export logs and artifact IDs.

Reproducibility requirements. All export scripts, configurations, filters, source records, versions, and paths must be recorded.

Thesis evidence produced. Thesis-ready figure bundle, table bundle, and artifact index.

Limitations and risks. Formatting quality does not equal scientific validity. The export manager must not hide negative or incomplete results.

16.7.8 Module 10.8: Reproducibility Manifest Builder

Purpose. Assemble the reproducibility package that lets another person understand and rerun the research pipeline.

Bottleneck addressed. Prevents the thesis from depending on informal memory of commands, seeds, configs, and file paths.

Scientific role. Supports external auditability and makes results portable across time, machines, and readers.

Engineering responsibility. Generate artifact manifest, `ReproducibilityManifest`, environment manifest, run registry summary, configuration index, source-record index, and reproduction guide.

Inputs. Run registry, configuration files, optional policy-update configuration files, Layer 9 `Layer9EmbodimentDemoManifests`, command-mapping configs, simulator-version metadata, safety-event log paths, reward-trace paths, dashboard-compatible Layer 9 export paths, environment files, source-record inventory, optional `PolicyUpdateTrace` paths, artifact manifest entries, export logs, validation logs, disabled-update audit logs, warning flags, and thesis location mapping.

Outputs. `artifact_manifest.yaml`, `ReproducibilityManifest` serialized as `reproducibility_manifest.yaml` or equivalent JSON/YAML, `run_registry.yaml`, environment summary, optional policy-update trace index, reproduction guide, package index, and validation report.

Internal processing logic. Collect run IDs, config IDs, seeds, dataset versions, code versions, source paths, artifact paths, card paths, figure paths, table paths, dashboard version, warnings, limitations, and reproduction commands; then write machine-readable and human-readable manifests.

Dependencies. Loader, validator, export manager, audit logger, environment capture, and repository organization.

Consumers. Future researcher, thesis appendix, repository release, examiner, and reproducibility reviewer.

Feedforward connections. Sends manifest references to the cards/export page and thesis reproducibility appendix.

Feedback or loopback. If a figure, table, or card cannot be traced to source records and configs, the manifest marks it incomplete and sends an artifact-completeness signal to the export manager.

Evaluation method. Manifest tests, path-existence tests, source-record coverage tests, seed/config coverage tests, and dry-run reproduction tests on a small subset.

Failure modes. Missing configs, undocumented seeds, unavailable dataset versions, stale environment files, and incomplete artifact paths. The defense is an explicit known-limitations section.

Reproducibility requirements. The manifest itself must be versioned, generated from a saved configuration, and included in the repository release.

Thesis evidence produced. Reproducibility appendix and repository release package.

Limitations and risks. Public datasets may require external download steps. Layer 10 should record source notes and download instructions but cannot guarantee future external availability.

16.7.9 Module 10.9: Warning Engine and Claim-Safety Gate

Purpose. Apply claim-safety labels consistently across dashboards, cards, figures, tables, and manifests.

Bottleneck addressed. Prevents public-data, proxy-label, simulation-only, stress-test, RL, or embodiment outputs from being overinterpreted.

Scientific role. Enforces the thesis boundary between computational evidence, simulated evidence, proxy evidence, and clinical claims.

Engineering responsibility. Load Layer 0 claim-safety rules; attach warning flags to records and artifacts; block or mark exports that lack required limitation labels; and generate safety/limitation cards.

Inputs. Claim ledger, Layer 0 scope rules, record metadata, page type, card type, artifact type, simulation flags, proxy-label flags, stress profile flags, and embodiment flags.

Outputs. Warning flags, warning banners, safety cards, limitation summaries, blocked-export reports, and claim-to-artifact mapping.

Internal processing logic. Match records and artifacts to warning rules; attach flags such as `public_data_limitation`, `healthy_subject_limitation` where applicable, `offline_or_pseudo_online`, `proxy_label_target`, `simulation_only`, `stress_test_only`, `embodiment_demo_only`, `reward_design_dependence`, `incomplete_ablation_coverage`, `missing_source_records`, `unresolved_lineage`, `non_thesis_ready`, `no-real-actuation`, `no-clinical-rehabilitation-claim`, `policy-update simulation-only`, and `clinical non-claim`; verify that cards and exports include these flags, that `Layer9RewardTrace` files are not presented as clinical improvement, and that bandit/RL update traces are not presented as frozen-policy evidence. If any warning is active, preserve it in page metadata, card metadata, figure/table metadata, export logs, and manifest entries.

Dependencies. Layer 0 governance, metadata completeness, artifact identity, card schema, export manager, and manifest builder.

Consumers. All dashboard pages, card generator, export manager, manifest builder, thesis limitations and threats-to-validity sections.

Feedforward connections. Sends warning flags to views, cards, exports, and manifests.

Feedback or loopback. If an artifact attempts to support an overstrong claim, the warning engine signals Layer 0 claim-safety governance and blocks thesis-ready status until the claim is weakened or evidence is added.

Evaluation method. Warning-flag tests, simulation-only export tests, proxy-label tests, clinical non-claim tests, and manual claim-safety audit.

Failure modes. Missing warnings, too many generic warnings, false sense of safety, inconsistent limitation text, and unmarked policy-update mode mixing. The defense is card-level and artifact-level warning requirements plus policy-update mode checks.

Reproducibility requirements. Warning-rule version and claim ledger version must be saved in the manifest.

Thesis evidence produced. Claim-safety summary, safety and limitation cards, and threats-to-validity support.

Limitations and risks. Warning labels reduce overclaiming but cannot replace careful thesis writing.

16.7.10 Module 10.10: Audit Logger and Artifact Registry

Purpose. Record the provenance of every Layer 10 output and maintain a registry of generated artifacts.

Bottleneck addressed. Prevents figures, cards, and tables from becoming detached from the records that produced them.

Scientific role. Enables audit trails from thesis claims to records and from records to modules.

Engineering responsibility. Log artifact generation events, source-record IDs, filters, configs, seeds, script versions, warning flags, export paths, and thesis usage.

Inputs. Export events, card-generation events, validation events, manifest events, source-record lineage, and configuration metadata.

Outputs. Audit log, artifact registry, figure-to-record mapping, table-to-record mapping, card-to-record mapping, and completeness status.

Internal processing logic. Intercept export and card-generation events; assign or validate artifact IDs; write append-only logs; update registry; compute completeness; and expose the registry through the cards/export page.

Dependencies. Export manager, card generator, manifest builder, record indexer, and warning engine.

Consumers. Reproducibility manifest, dashboard cards/export page, thesis appendix, future reproducer, examiner.

Feedforward connections. Sends registry entries to manifest builder and completeness dashboards.

Feedback or loopback. If an artifact is generated without required lineage, the audit logger marks it non-thesis-ready and requests regeneration through the export manager.

Evaluation method. Audit-log tests, append-only behavior tests, artifact ID uniqueness tests, and traceability tests.

Failure modes. Incomplete logging, duplicate artifact IDs, stale registry entries, and manually edited figures outside the export path. The mitigation is artifact checksums and explicit non-thesis-ready status for unregistered files.

Reproducibility requirements. Audit logs and artifact registry must be included in the reproducibility package.

Thesis evidence produced. Artifact manifest summary and traceability appendix.

Limitations and risks. Audit logs can become large; the thesis should summarize them while releasing machine-readable versions in the repository.

16.8 Dashboard page specifications

The dashboard may be implemented as Streamlit, Dash, Gradio, a Jupyter notebook, static HTML, or a collection of deterministic static reports. It does not need to be a production web application. Its academic purpose is observability, debugging, communication, and reproducibility.

Each page should answer a specific thesis inspection question, read from saved records, expose filters, attach warning flags, and export artifacts with lineage.

16.8.1 Project overview and claim-safety page

Purpose. Show the architecture overview, layer list, core/stretch/future status, public-data boundaries, simulation limitations, and claim-safety status.

Main inputs. Claim ledger, layer metadata, artifact manifest, card registry, Layer 0 scope labels, validation summary, and reproducibility manifest.

Required views. Architecture status table, layer-to-record table, claim-to-artifact summary, warning-flag coverage, artifact completeness, and reproducibility readiness.

Required exports. Architecture overview screenshot, claim-safety card, limitation summary, reproducibility status table, and `CARD_SAFETY_001`.

16.8.2 Dataset and split page

Purpose. Show datasets, subjects, sessions, task labels, preprocessing, artifact flags, label budgets, and fixed train/validation/test or calibration/test splits.

Main inputs. `DatasetRecord`, `WindowRecord`, `SplitRecord`, `PreprocessingRecord`, `LabelMapRecord`, `ArtifactFlagRecord`, `DatasetCard`, `ProtocolCard`, `ValidationReport`, `Layer1Manifest`, `layer1_ablation_readiness_l1_v1.json`, public-data source notes, dataset cards, and protocol cards.

Required views. Dataset summary table, subject/session distribution, task-label distribution, calibration-budget summary, split checks, leakage warnings, missingness report, and artifact summary.

Required exports. Dataset card, protocol card, split table, preprocessing report table or figure, and public-data limitation note.

16.8.3 Decoder baseline page

Purpose. Show baseline decoder performance, low-calibration curves, subject difficulty, and model-family comparisons.

Main inputs. `PredictionRecord`, `ModelRegistryRecord`, `BaselineMetricRecord`, `LowCalibrationCurveRecord`, `SubjectProfileRecord`, `EnsembleControlRecord`, `FailureCaseIndex`, `Layer2ReadinessReport`, `ModelCard`, `BenchmarkCard`, and dataset/split metadata.

Required views. Accuracy and balanced accuracy by dataset/subject/session, model-family comparison, low-calibration performance curves, subject difficulty profile, confusion summaries where applicable, and model registry table.

Required exports. Decoder card, baseline comparison table, low-calibration figure, subject difficulty figure, and model-family summary.

16.8.4 Calibration, uncertainty, and selective-prediction page

Purpose. Show whether decoder confidence is trustworthy and whether simple thresholding is a sufficient baseline for reliability.

Main inputs. `CalibrationRecord`, `UncertaintyRecord`, `PredictionRecord`,

SelectivePredictionRecord, ThresholdRegistryRecord, SplitIntegrityReport, ReliabilityAuditReport, CalibrationEligibilityTable, HighConfidenceWrongTable, CalibrationDashboardMetricRecord, Layer3MatchedOperatingPointTable, ReliabilityCard, and correctness labels for evaluation-only display.

Required views. Reliability diagrams, ECE, Brier score, NLL summaries, confidence histograms, entropy and margin distributions, high-confidence wrong cases, risk-coverage curves, retained accuracy, rejection rate, threshold origins, allowed consumers, and leakage status.

Required exports. Calibration card, uncertainty summary table, reliability figure, risk-coverage figure, threshold registry table, and high-confidence wrong-case report.

16.8.5 IHARQ decision-flow page

Purpose. Show canonical Layer 4 IHARQ decisions over time using display labels for `accept`, `wait`, `request_evidence`, `combine_evidence`, `fallback`, `uncertain`, and `unsafe` without rewriting the stored action tokens.

Main inputs. IHARQDecisionRecord, EvidenceQualityRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, Layer 4 EvaluationOutcomeRecord, CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, and RegimeStateRecord where available.

Required views. Canonical action-token distribution, display-label distribution, unknown-alias warning table, reason-code timeline, evidence-request count, request/combine trace, fallback rate and route table, unsafe-event table, uncertain rate, accepted-trial accuracy, action-cost summary, Layer 4 ablation table, trace-completeness view, and comparison with confidence/entropy/margin rejection baselines.

Required exports. IHARQ reliability card, decision-flow plot, reason-code table, request/combine report, fallback and unsafe event report, trace-completeness report, Layer 4 ablation table, and matched-coverage comparison figure.

16.8.6 RegimeRisk timeline page

Purpose. Show temporal trust, regime labels, risk trends, volatility-like behavior, drawdown-like behavior, failure-risk score, trust scores, trusted set, and stop-loss warnings.

Main inputs. RegimeStateRecord, conditional TrustStateRecord rows where independent trust timelines are exported, PredictionRecord, IHARQDecisionRecord, StressProfileRecord and StressScheduleRecord where available, and TransitionRecord where available.

Required views. RegimeRisk timeline, trust-score trends, trust-state drill-downs where TrustStateRecord rows exist, volatility-like features, drawdown-like features, drift/fatigue/dropout risk panels, risk-to-error alignment, early-warning lead-time analysis where applicable, trusted-set evolution, stop-loss warning table, and reason-code table.

Required exports. RegimeRisk card, risk timeline figure, regime-shift report, temporal trust summary, and stop-loss warning appendix table.

16.8.7 Adaptive policy page

Purpose. Inspect supervised Adaptive-IHARQ, evidence-quality estimation, policy context, threshold selection, legality masks, and readiness action decisions.

Main inputs. ReadinessContextRecord, EvidenceQualityRecord, DeferralRecord, PolicyActionRecord, TransitionRecord, cost configuration, and policy registry.

Required views. Policy action distribution, cost-sensitive decision comparison, threshold selection behavior, evidence-quality prediction performance, legality mask summary, policy stability across datasets/subjects/sessions/stress conditions, supervised policy versus rule-based IHARQ comparison, and the Layer 6 policy-matrix bundle: policy-stage matrix, context-level ablation matrix, action-legality matrix, cost/reward sensitivity matrix, policy-to-layer dependency matrix, and claim-boundary matrix.

Required exports. Adaptive-IHARQ controller card, policy comparison table, decision-cost figure, action-distribution figure, evidence-quality estimator report, and Layer 6 policy-matrix export bundle.

16.8.8 Learning-to-defer page

Purpose. Show whether the system benefits from deferring, waiting, requesting evidence, routing to fallback, or choosing safe state.

Main inputs. DeferralRecord, PolicyActionRecord, EvidenceQualityRecord, fallback/expert-proxy records, and cost configuration.

Required views. Deferral rate, fallback burden, matched-coverage comparison, matched-cost comparison, wrong-accept reduction, deferral reason codes, simple-threshold comparison, and proxy-label warning banner.

Required exports. Learning-to-defer card, deferral curve, fallback burden table, matched-cost comparison figure, and limitation note for proxy fallback labels.

16.8.9 Contextual bandit and RL page

Purpose. Show online or sequential adaptive decision behavior in the simulated environment while preserving simulation-only boundaries.

Main inputs. PolicyActionRecord, TransitionRecord, optional PolicyUpdateTrace, disabled-update audit entries, reward logs, regret logs, stress profiles, stress schedules, controller configuration, policy-update configuration, exploration flags, and held-out session records.

Required views. Reward curves, regret curves, action-probability evolution, exploration flag analysis, policy-update trace completeness, disabled-update audit status, explicit separation between simulation-training/adaptation curves and frozen-policy evaluation curves, held-out session performance, stress-conditioned policy performance, reward-weight ablation, cost/reward sensitivity matrix view, action-legality matrix view, policy-to-layer dependency matrix view, and simulation-only warning banner.

Required exports. Contextual-bandit card, RL controller card, policy-update trace card where updates are enabled or audited, reward/regret figures, separated training/adaptation and frozen-evaluation curve exports, disabled-update audit table, reward-design limitation statement, and controller comparison table.

16.8.10 StressForge page

Purpose. Show declared stress profiles and stress sensitivity curves without implying that synthetic stress equals real clinical or hardware degradation.

Main inputs. StressProfileRecord, StressScheduleRecord, TransitionRecord, PolicyActionRecord, RegimeStateRecord, EmbodimentStateRecord where applicable, Layer 8 EvaluationOutcomeRecords, matched-stress comparison reports, stress metric-dictionary entries, negative-result notes, dashboard/export lineage, and claim-evidence entries.

Required views. Stress type and intensity distribution, clean/mild/moderate/severe/combined stress comparison, delay/dropout/noisy-intent/weakness/drift/fatigue-like views, A12 local/global ablation IDs, matched-stress status, diagnostic-only downgrades, metric definitions, policy robustness under stress, held-out stress generalization where available, stress-induced failure categories, and stress schedule inspection.

Required exports. StressForge card, stress sensitivity curves, stress failure taxonomy, stress-conditioned ablation table, Layer 8 A12 evaluation-outcome export, matched-stress report export, stress negative-result note, stress claim-evidence export, and stress limitation note.

16.8.11 Layer 9 MyoSuite/OpenSim/static-replay embodiment page

Purpose. Show how readiness-gated neural intent maps to simulated assistive-action outcomes in the Layer 9-owned embodiment subsystem. The page is an observability surface over saved Layer 9 outputs, not a simulator controller and not a detached runtime layer.

Main inputs. EmbodimentStateRecord, EmbodimentOutcomeRecord, TransitionRecord embodiment linkage, PolicyActionRecord, IHARQDecisionRecord, RegimeStateRecord, StressProfileRecord, StressScheduleRecord where stress is active, SafetyEventRecord or safety_event_log.csv, Layer9RewardTrace where enabled, command timelines, endpoint-error curves, dashboard-compatible Layer 9 export files, simulator screenshots or video references, Layer9EmbodimentDemoManifest, Layer 9 EvaluationOutcomeRecords, MatchedComparisonReports for embodiment comparisons, MetricDictionaryEntry snapshots or IDs for embodiment metrics, NegativeResultNotes, dashboard/export lineage, and ClaimEvidenceMatrix entries.

Required views. E0–E5 embodiment maturity status, A13 local/global ablation IDs, simulator adapter branch, static-replay/MyoSuite/OpenSim branch label, command-mapping ID, matched-embodiment status, diagnostic-only downgrades, simulated action state, assistive command timeline, endpoint error, command success/failure, unsafe events, blocked-command and no-action safety behavior, weakness-stress response, readiness-gating effect, controller source, reward-decomposition view where enabled, demo-manifest completeness, and simulation-only/non-clinical warning banner.

Required exports. Layer 9 embodiment evidence card, demo screenshot or video reference, endpoint-error figure, command timeline figure, unsafe-event table, reward-decomposition figure where enabled, demo-manifest summary, dashboard-compatible embodiment export bundle, Layer 9 A13 evaluation-outcome export, matched embodiment comparison export, embodiment negative-result note, embodiment claim-evidence export, and simulation-only limitation statement.

16.8.12 Ablation and comparison page

Purpose. Prove that architectural complexity is useful where it is useful, and expose where it is not.

Main inputs. All experiment summaries plus `EvaluationOutcomeRecords`, `PredictionRecord`, `CalibrationRecord`, `SelectivePredictionRecord`, `IHARQDecisionRecord`, `RegimeStateRecord`, `PolicyActionRecord`, `TransitionRecord`, `ClosedLoopSessionRecord`, `EpisodeSummaryRecord`, `PolicyRolloutRecord`, `RewardTraceRecord`, `EnvironmentDiagnosticRecord`, `StressProfileRecord`, `StressScheduleRecord`, `EmbodimentStateRecord`, `EmbodimentOutcomeRecord` where embodied mode executed, `MatchedComparisonReports`, `MetricDictionaryEntry` snapshots or IDs, `NegativeResultNotes`, and `ClaimEvidenceMatrix` entries.

Required views. A0–A13 ablation ladder, Layer 6 policy-stage matrix, context-level ablation matrix, action-legality matrix, cost/reward sensitivity matrix, policy-to-layer dependency matrix, claim-boundary matrix, Layer 7 closed-loop session timeline, policy rollout comparison, reward-decomposition/reward-regret comparison, environment-diagnostic warning summary, matched-coverage comparison, matched-cost comparison, matched-stress comparison, matched-embodiment comparison, A10/A11 frozen-policy versus update-enabled simulation separation, A12 stress contribution summary, A13 embodiment contribution summary, module contribution summary, cases where a module does not help, result stability across subjects and sessions, and negative-result notes.

Required exports. Ablation table, Layer 6 policy-matrix bundle, Layer 7 closed-loop timeline export, policy-rollout card/export, reward-decomposition or reward-regret figure, environment-diagnostic warning export, architecture contribution figure, module usefulness card, matched-cost figure, matched-coverage figure, matched-stress figure/table, matched-embodiment figure/table, claim-evidence export, and negative-result note where needed.

16.8.13 Failure taxonomy page

Purpose. Organize errors and limitations into a defensible taxonomy.

Main inputs. Failure logs, high-confidence wrong cases, fallback logs, unsafe logs, Layer 7 `EnvironmentDiagnosticRecords`, missing reward-trace or safety-event link reports, stress failure logs, embodiment unsafe events, validation errors, missingness logs, and limitation cards.

Required views. Failure category list, frequency by dataset/model/policy/stress, representative examples, affected layers, mitigation or interpretation, claim-safety status, and thesis limitation mapping.

Required exports. Failure-taxonomy card, failure table, limitation appendix material, and discussion-section evidence.

16.8.14 Cards and export page

Purpose. Generate standardized cards, thesis figures, thesis tables, and reproducibility manifests.

Main inputs. All records, artifact registry, card templates, export configuration, claim ledger, validation reports, and warning flags.

Required views. Card completeness dashboard, artifact export status, missing artifact warn-

ings, figure-to-record mapping, table-to-record mapping, manifest preview, and thesis usage mapping.

Required exports. All cards, figure bundle, table bundle, reproducibility manifest, artifact index, validation report, export logs, and package index.

16.9 Cards architecture

Cards are standardized, compact, reusable research artifacts. They make each dataset, protocol, model, policy, policy-update trace, risk monitor, stress profile, embodiment demo, safety boundary, and failure category understandable without requiring the reader to inspect the full codebase. Cards support thesis writing; they do not replace it.

Card type	Purpose
Dataset card	Summarizes dataset source, subjects, sessions, labels, preprocessing, split membership, license/source notes, and public-data limitations.
Protocol card	Summarizes splits, calibration budgets, leakage controls, evaluation procedure, and low-calibration protocol.
Decoder card	Summarizes model family, training setup, low-calibration behavior, performance, and limitations.
Calibration card	Summarizes calibration method, reliability metrics, uncertainty behavior, high-confidence wrong cases, and selective-prediction baselines.
IHARQ reliability card	Summarizes evidence rules, actions, reason codes, evidence requests, fallback, uncertain, and unsafe behavior.
RegimeRisk card	Summarizes risk features, regime timelines, trust scores, trusted set, stop-loss behavior, and temporal limitations.
Adaptive-IHARQ controller card	Summarizes supervised policy inputs, targets, action space, costs, threshold behavior, and comparisons.
Learning-to-defer card	Summarizes deferral policy, fallback routing, matched-cost results, matched-coverage results, and proxy limitations.
Contextual-bandit card	Summarizes context, action space, reward/regret, exploration, held-out behavior, and simulation-only limits.
RL controller card	Summarizes environment, reward design, state/action space, delayed consequences, held-out performance, and simulation-only limits.
Layer 6 policy-matrix card	Summarizes the policy-stage, context-ablation, action-legality, cost/reward sensitivity, policy-to-layer dependency, and claim-boundary matrices required after the Layer 6 policy-learning merge.
Closed-loop environment card	Summarizes <code>ClosedLoopSessionRecord</code> , environment assumptions, replay/session metadata, reward/action/evidence configuration hashes, stress and embodiment mode where active, deterministic replay status, and simulation-only boundaries.

Policy rollout card	Summarizes PolicyRolloutRecord , policy identity, action sequences, transition pointers, reward-trace pointers, evidence-request/delay behavior, safety-event linkage, frozen/update mode, and claim boundaries.
Policy-update trace card	Summarizes optional PolicyUpdateTrace files, update mode, update-step completeness, disabled-update audit status, exploration mode, reward/next-state linkage, frozen-policy evaluation separation, and simulation-only limits.
Reward configuration card	Summarizes RewardTraceRecord lineage, reward terms, cost components, reward version, sensitivity settings, reward-hacking warnings, and simulation-only limits.
StressForge card	Summarizes stress type, intensity, schedule, seed, target layer, affected records, and stress-conditioned outcomes.
Layer 9 embodiment evidence card	Summarizes E0–E5 mode, simulator branch, command mapping, endpoint error, command success/failure, unsafe events, safety-gate behavior, Layer9RewardTrace where enabled, demo-manifest status, and non-clinical boundaries.
Safety and limitation card	Summarizes claim-safety rules, scope boundaries, public-data limitations, proxy-label limits, and clinical non-claims.
Failure-taxonomy card	Summarizes failure categories, representative examples, affected modules, mitigations, and interpretation.

16.9.1 CardRecord schema and universal card contract

Every **CardRecord** should contain: card ID, card type, card version, generated date, generating script, object summarized, source records, source layers, configuration ID, run ID, main metrics, main figures or table references, interpretation, thesis claim supported or limitation represented, warning flags, limitations, reproduction command, artifact paths, thesis usage, completeness status, and generation timestamp. The human-readable card file is the rendered artifact; the **CardRecord** is the machine-readable card registry entry that makes the card traceable. A card is complete only if required source records exist, schema validation passes, key metrics are present, limitations are explicit, warning flags are present, reproduction information exists, the generated artifact path is recorded, and thesis location is assigned or marked as unused.

Layer 6 controller cards must cross-link to the policy-matrix bundle whenever they summarize supervised Adaptive-IHARQ, learning-to-defer, contextual bandit, or RL results. The matrix bundle is complete only when it identifies the policy stage, context feature groups, legal/illegal actions, cost/reward setting, layer dependency, and claim boundary for every policy family included in a thesis figure or table.

16.10 Traceability framework

Layer 10 should make the following chain inspectable:

Thesis claim → figure/table/card ID → CardRecord when the artifact is a card → exported artifact → dashbo

16.10.1 Record-to-view traceability

Source record	Dashboard views
DatasetRecord	Dataset and split page, cards/export page, claim-safety page.
WindowRecord	Dataset page, preprocessing view, trial/window inspection.
SplitRecord	Dataset and split page, leakage checks, ablation-readiness panels, cards/export page, and reproducibility manifest.
PreprocessingRecord	Dataset and split page, preprocessing view, protocol cards, artifact lineage, and reproducibility manifest.
LabelMapRecord	Dataset and split page, label-normalization view, proxy-intent warning panels, claim-safety page, and cards/export page.
ArtifactFlagRecord	Dataset and split page, missingness report, data-quality warning panels, failure taxonomy page, and limitation cards.
DatasetCard	Dataset and split page, cards/export page, public-data limitation panels, claim-safety page, and thesis appendix exports.
ProtocolCard	Dataset and split page, leakage/protocol page, cards/export page, ablation-readiness panels, and reproducibility manifest.
ValidationReport	Dataset and split page, validation/readiness panels, leakage warnings, diagnostic-only downgrade views, and reproducibility manifest.
Layer1Manifest	Project overview page, dataset and split page, ablation-readiness panels, reproducibility page, and package index.
layer1_ablation_readiness_Ablation	Ablation-readiness panels, A0–A4 matchability checks, diagnostic-only warning banners, cards/export page, and reproducibility manifest.
PredictionRecord	Decoder page, calibration page, selective-prediction page, ablation page.
ModelRegistryRecord	Decoder page, model-family comparison, model/decoder cards, artifact registry, and reproducibility manifest.
BaselineMetricRecord	Decoder page, A0 raw-decoder table, baseline comparison table, ablation page, and thesis tables.
LowCalibrationCurveRecord	Decoder page, low-calibration curve view, low-label budget appendix, ablation page, and thesis figures.
SubjectProfileRecord	Decoder page, subject difficulty profile, RegimeRisk diagnostic views, subject/session drill-downs, and cards/export page.
CalibrationRecord	Calibration page, IHARQ page, cards/export page.
UncertaintyRecord	Calibration page, IHARQ page, failure taxonomy page.
SelectivePredictionRecord	Calibration/selective-prediction page, IHARQ baseline comparison, ablation page.

ThresholdRegistryRecord	Selective-prediction page, threshold audit table, leakage check, cards/export page.
IHARQDecisionRecord	IHARQ page, policy page, simulated session page, ablation page.
RegimeStateRecord	RegimeRisk page, IHARQ page, policy page, stress page, embodiment page.
TrustStateRecord	RegimeRisk page, policy page, cards/export page, and reproducibility manifest when independent trust timelines are produced.
PolicyActionRecord	Adaptive policy page, learning-to-defer page, bandit/RL page, stress page, embodiment page.
TransitionRecord	Closed-loop, policy, stress, embodiment, and ablation views, plus policy-update trace linkage where <code>policy_update_trace_id</code> exists.
ClosedLoopSessionRecord	Closed-loop session timeline, replay/session metadata panel, stress/session filter, reproducibility manifest, and simulation-only warning banner.
EpisodeSummaryRecord	Episode-return view, reward/cost summary, safety-event summary, A10/A11/A12 ablation views, thesis simulation tables, and cards/export page.
PolicyRolloutRecord	Policy rollout view, policy rollout card, action-sequence drill-down, transition/reward-trace pointers, evidence-request/delay burden, and reproducibility manifest.
RewardTraceRecord	Reward-decomposition view, reward/regret dashboard, reward-sensitivity panel, policy-update audit linkage, cards/export page, and reproducibility manifest.
EnvironmentDiagnosticRecord	Environment-diagnostic warning panel, failure taxonomy page, missingness report, validation report, claim-safety page, and cards/export page.
PolicyUpdateTrace	Contextual bandit/RL page, policy-update trace card, closed-loop page, ablation page, cards/export page, and reproducibility manifest.
StressProfileRecord	StressForge page, ablation page, embodiment page.
StressScheduleRecord	StressForge page, stress schedule inspection, ablation page, closed-loop session page, embodiment page, cards/export page, and reproducibility manifest.
EmbodimentStateRecord	Layer 9 embodiment page, failure taxonomy page, cards/export page, reproducibility manifest.
EmbodimentOutcomeRecord	Layer 9 embodiment page, A13 outcome summaries, endpoint-error/command-success panels, ablation page, failure taxonomy page, and cards/export page.
SafetyEventRecord or <code>safety_event_log.csv</code>	Layer 9 embodiment page, failure taxonomy page, safety cards, cards/export page.
Layer9RewardTrace where enabled	Layer 9 embodiment page, contextual bandit/RL page where simulation-only reward feedback is audited, reward-decomposition panels, cards/export page, and reproducibility manifest.
Layer9EmbodimentDemoManifests, and dashboard-compatible export	Layer 9 embodiment page, cards/export page, reproducibility manifest, artifact registry.

16.10.2 View-to-card and artifact-to-thesis traceability

Dashboard view	Generated cards	Thesis usage
Dataset and split page	Dataset cards, protocol cards.	Methodology chapter and appendix.
Decoder baseline page	Decoder cards.	Baseline results chapter.
Calibration and uncertainty page	Calibration cards.	Reliability analysis chapter.
IHARQ flow page	IHARQ reliability cards.	Evidence-verification chapter.
RegimeRisk page	RegimeRisk cards.	Temporal trust chapter.
Adaptive policy page	Adaptive-IHARQ controller cards.	Adaptive policy chapter.
Learning-to-defer page	Learning-to-defer cards.	Deferral and fallback analysis.
Contextual bandit/RL page	Contextual-bandit cards, RL controller cards, policy-update trace cards where applicable, policy rollout cards, closed-loop environment cards, and reward configuration cards.	Simulated adaptive policy chapter or appendix, with training/adaptation curves separated from frozen-policy evaluation curves and reward-decomposition/frozen-policy evidence kept distinct.
StressForge page	StressForge cards.	Robustness and stress-test chapter.
Layer 9 embodiment page	Layer 9 embodiment evidence cards, safety cards, and demo-manifest summaries.	Stretch C demonstration chapter or appendix.
Failure taxonomy page	Failure-taxonomy cards, safety cards.	Discussion and threats-to-validity chapter.
Cards/export page	All cards and reproducibility manifests.	Reproducibility appendix and repository release.

16.11 Feedforward and loopback behavior

The normal feedforward flow is:

Layers 1–9 records → Layer 10 loader → validator → indexer → metric summarizer → views/cards/exports →

Layer 10 loopbacks are diagnostic and governance-oriented. They do not alter upstream records directly.

Source	Target	Signal and trigger	Action	Safety and evaluation logic
--------	--------	--------------------	--------	-----------------------------

Record loader	Producing layer or experiment controller	Missing expected record, missing config, or missing run ID.	Mark page/card incomplete and request upstream regeneration.	No fabricated records; evaluated by missing-record tests and completeness report.
Schema validator	Producing layer	Schema failure, invalid metric range, incompatible version, or split leakage.	Block thesis-ready export and report failing fields.	Invalid records remain visible but marked unusable; evaluated by contract tests.
Record indexer	Producing layer or run registry	ID mismatch or unresolved lineage chain.	Mark artifact lineage unresolved and request ID repair or mapping file.	No ambiguous joins for thesis figures; evaluated by lineage completeness tests.
Warning engine	Layer 0 claim-safety governance	Artifact lacks required public-data, proxy-label, simulation-only, or clinical non-claim flag.	Block thesis-ready status until warning is added or claim is weakened.	Prevents overclaiming; evaluated by warning-flag tests.
Export manager	Card generator or metric summarizer	Requested artifact lacks source records, filters, or config ID.	Refuse thesis-ready export and log regeneration requirement.	Prevents untracked manual figures; evaluated by artifact contract tests.
Contextual bandit/RL page	Layer 7 and Layer 6 policy artifacts	A bandit/RL curve mixes simulation-training/adaptation with frozen-policy evaluation, or an update event lacks <code>PolicyUpdateTrace</code> .	Mark the curve invalid for thesis-ready export and emit a policy-update audit finding.	Prevents hidden online learning claims; evaluated by policy-update mode checks and trace-completeness tests.
Ablation page	Experiment controller	Required ablation step missing or unmatched comparison unavailable.	Mark ablation ladder incomplete and list missing variant.	Avoids false complexity claims; evaluated by ablation completeness table.

Failure taxonomy page	Earlier layers and discussion chapter	Repeated failure category, unsafe event, or high-confidence wrong case.	Route to limitation analysis and potential module improvement.	Diagnostic only; does not claim clinical safety; evaluated by failure coverage.
Layer 9 plus Layer 0	Unsafe simulated event, unmarked stress condition, or overstrong embodiment claim.	Add simulation-only warning, safety note, and failure-taxonomy entry.	Keeps Layer 9 embodiment claims bounded; evaluated by safety-card and embodiment-card completeness.	

16.12 Evaluation design for Layer 10

Layer 10 is evaluated as an observability and reproducibility component. Its success is not measured by improving decoder accuracy directly. It is measured by whether it makes the architecture inspectable, traceable, reproducible, and claim-safe.

16.12.1 Baselines

Baseline	Question answered
Manual scattered plots	Does Layer 10 reduce untracked and inconsistent reporting compared with manually produced figures?
Single final metrics table	Does Layer 10 expose subject/session/stress/failure details that a final table hides?
No cards	Do standardized cards improve completeness of assumptions, limitations, and reproduction information?
No manifest	Does the reproducibility manifest improve traceability from thesis figures to records and configs?
No warning engine	Do claim-safety warnings reduce the risk of overclaiming simulation, proxy, or public-data results?

16.12.2 Ablations

Layer 10 should expose its own ablations in addition to the system-level A0–A13 ladder:

- dashboard without record validation;
- dashboard without card generation;
- dashboard without artifact manifest;
- dashboard without warning engine;

- exports without source-record lineage;
- StressForge page without A12 matched-stress reports, metric dictionaries, negative-result notes, or claim-evidence lineage;
- Layer 9 embodiment page without A13 matched embodiment reports, metric dictionaries, negative-result notes, or claim-evidence lineage;
- bandit/RL page without separated training/adaptation and frozen-policy evaluation curves;
- policy-update trace card disabled for update-enabled runs;
- cards without limitations;
- static reports versus interactive dashboard, if both exist.

16.12.3 Metrics and success criteria

Metric	Meaning	Success criterion
Artifact completeness	Percentage of expected cards, figures, tables, logs, and manifests generated.	High for core thesis artifacts; incomplete items explicitly listed.
Card completeness	Percentage of cards with identity, source records, metrics, limitations, warnings, and reproduction info.	All thesis-used cards complete.
Policy-matrix completeness	Percentage of required Layer 6 matrices generated and linked to the corresponding controller cards, dashboard pages, and thesis exports.	All thesis-used policy results have policy-stage, context-ablation, action-legality, cost/reward sensitivity, dependency, and claim-boundary matrices.
Config reproducibility	Percentage of artifacts linked to config files and run IDs.	All thesis figures and tables linked.
Figure traceability	Percentage of thesis figures linked to source records and source layers.	All thesis-used figures traceable.
Record validation pass rate	Percentage of records passing schema and compatibility checks.	Failures visible and interpreted.
Missingness visibility	Whether missing records and failed checks are exposed.	Missingness must never be hidden.
Claim-warning coverage	Percentage of simulation/proxy/public-data artifacts with warning flags.	All applicable artifacts flagged.
Policy-update trace completeness	Percentage of simulation-authorized bandit/RL policy mutation events with a corresponding <code>PolicyUpdateTrace</code> .	All enabled update events traced; disabled-update runs explicitly audited when relevant.

A12 stress-lineage completeness	Percentage of thesis-used stress views with Layer 8 <code>EvaluationOutcomeRecord</code> , <code>StressProfileRecord</code> , <code>StressScheduleRecord</code> , matched-stress report, metric dictionary, negative-result note where applicable, and claim-evidence entry.	All thesis-used A12 stress figures and tables pass or are marked diagnostic-only.
A13 embodiment-lineage completeness	Percentage of thesis-used embodiment views with Layer 9 <code>EvaluationOutcomeRecord</code> , matched embodiment report, metric dictionary, negative-result note where applicable, and claim-evidence entry.	All thesis-used A13 embodiment figures and tables pass or are marked diagnostic-only.
Frozen/update separation pass rate	Percentage of bandit/RL dashboard exports that keep simulation-training/adaptation curves separate from frozen-policy evaluation curves.	All thesis-used bandit/RL figures pass.
Export determinism	Whether repeated exports from same records/configs produce identical files or controlled differences.	Deterministic for static figures and tables.
Usability inspection	Whether a supervisor/examiner can follow claim-to-artifact lineage.	Manual inspection should pass for major claims.

Failure interpretation must be honest. Low artifact completeness means the thesis is not yet reproducibility-ready. Low warning coverage means claim-safety is weak. Low traceability means figures should not be used as thesis evidence until repaired. A dashboard that looks polished but lacks lineage, validation, or warnings should be considered a failed Layer 10.

16.13 System-level ablation ladder exposed by Layer 10

Layer 10 must make the following ablation ladder visible because Stretch C is intentionally complex.

Step	System variant	Layer 10 responsibility
A0	Raw decoder	Show baseline before calibration or reliability control, with Layer 1 matched-comparison keys and Layer 0 claim boundaries.
A1	Calibrated decoder	Show whether calibration improves reliability visibility beyond raw scores, including calibration deltas and negative results.
A2	Simple thresholding / confidence rejection	Show whether simple confidence-threshold or matched-coverage rejection is enough under validation-only threshold selection.

A3	Uncertainty and selective prediction	Show whether entropy, margin, uncertainty, and risk-coverage baselines solve wrong accepts before IHARQ is credited.
A4	Ordinary combination controls	Show whether longer windows, multi-window voting, probability averaging, or ordinary ensembles already explain the apparent gain.
A5	IHARQ-lite	Show explicit evidence-verification value.
A6	IHARQ plus evidence-quality estimator	Show learned quality scoring beyond rules.
A7	IHARQ plus RegimeRisk	Show temporal trust contribution.
A8	Learning-to-defer	Show whether routing to fallback helps.
A9	Supervised Adaptive-IHARQ	Show learned readiness over rules and deferral.
A10	Contextual bandit	Show online threshold/action adaptation in simulation only when <code>PolicyUpdateTrace</code> completeness and frozen-evaluation separation are visible.
A11	RL policy	Show sequential policy learning under delayed consequences with trace-safe update logs and separate frozen-policy evaluation curves.
A12	StressForge stress tests	Show stress robustness across variants.
A13	Layer 9 MyoSuite/OpenSim/static-replay embodiment demo	Show simulated action-level outcome effects, safety-event behavior, endpoint-error/command-timeline evidence, optional reward-trace summaries, and simulation-only warnings.

16.14 Reproducibility architecture

Layer 10 must make it possible for another person to understand, inspect, and rerun the research pipeline. The reproducibility package connects code, data manifests, configurations, records, figures, cards, and thesis claims.

16.14.1 Required configuration files

Recommended configuration groups are:

- dataset and split configs;
- preprocessing configs;
- decoder and baseline configs;
- calibration and selective-prediction configs;
- IHARQ rule and evidence configs;
- RegimeRisk configs;
- Adaptive-IHARQ, deferral, bandit, and RL policy configs;

- policy-update configuration files and disabled-update audit configs when contextual-bandit/RL updates are enabled or audited;
- StressForge configs;
- Layer 9 MyoSuite/OpenSim/static-replay adapter configs, command-mapping configs, simulator configs, safety-gate configs, and `Layer9EmbodimentDemoManifest` files;
- dashboard, card, export, and manifest configs;
- claim-safety and warning-rule configs.

16.14.2 Expected output files

Layer 10 should produce or index the following files:

- `records/dashboard_metric_records/`;
- `records/dashboard_view_records/`;
- `records/card_records/`;
- `manifests/layer1_ablation_readiness_l1_v1.json`;
- A0–A4 evaluation-slice exports, including raw-decoder summaries, ordinary-combination-control tables, matched-coverage and matched-cost operating-point tables, `ClaimEvidenceMatrix` views, `MetricDictionaryEntry` snapshots or IDs, and `NegativeResultNotes`;
- optional `records/policy_update_traces/` or indexed `results/layer7_closed_loop/policy_updates/` files when bandit/RL updates are enabled or audited;
- `cards/` with subfolders for dataset, protocol, decoder, calibration, IHARQ, regime, trust-state when independent trust timelines are exported, policy, stress, Layer 9 embodiment evidence, safety, and failure taxonomy cards; Layer 6 policy-matrix cards may live under the existing policy-card family;
- indexed Layer 9 embodiment outputs such as `records/embodiment/`, `logs/layer9_safety_events/` or `records/embodiment/safety_events/`, `results/layer9_embodiment/`, `manifests/embodiment/`, `figures/embodiment/`, `cards/embodiment/`, `Layer9RewardTrace` exports where enabled, command timelines, endpoint-error curves, and `Layer9EmbodimentDemoManifest` files;
- `figures/dashboard_exports/` and `figures/thesis_ready/`;
- `thesis_tables/`, including Layer 6 policy-stage, context-ablation, action-legality, cost/reward sensitivity, policy-to-layer dependency, and claim-boundary matrices;
- `logs/validation_logs/` and `logs/export_logs/`;
- `manifests/artifact_manifest.yaml`;
- `manifests/reproducibility_manifest.yaml`;
- `manifests/run_registry.yaml`;
- dashboard static export or notebook report when interactive deployment is not used.

16.14.3 ReproducibilityManifest schema and fields

The `ReproducibilityManifest` should include: project name, project version, commit hash if available, environment ID, Python/package versions, dataset versions and source notes, configuration files, policy-update configuration files when used, `StressProfileRecord` and `StressScheduleRecord` paths where stress is active, Layer 9 embodiment configuration files, command-mapping IDs, simulator adapter branch, simulator-version metadata, `Layer9EmbodimentDemoManifest` paths, safety-event log paths, `Layer9RewardTrace` paths where enabled, Layer 1 ablation-readiness manifest path, matched-comparison report paths, metric-dictionary paths, claim-evidence matrix paths, negative-result note paths, random seeds, run IDs, record paths, optional `TrustStateRecord` paths when independent trust timelines are produced, optional `PolicyUpdateTrace` paths, disabled-update audit paths, `DashboardMetricRecord` paths, `DashboardViewRecord` paths, `CardRecord` paths, figure paths, table paths, card artifact paths, dashboard export path, artifact checksums where feasible, known limitations, warning-rule version, claim ledger version, and reproduction steps.

16.14.4 Repository organization

Human-readable Layer 10 artifacts are pages, cards, figures, reports, and thesis tables. Machine-readable Layer 10 artifacts are records, schemas, manifests, configs, logs, and checksums. Thesis-facing artifacts are only those that pass validation, lineage, claim-safety, and deterministic-export checks.

A recommended repository organization is:

```
IHARQ-BenchGuard/
  configs/
    dashboard/
    export/
    cards/
  src/
    dashboard/
    reporting_cards/
    artifact_export/
    record_validation/
  records/
    dataset_records/
    window_records/
    split_records/
    preprocessing_records/
    label_map_records/
    artifact_flag_records/
    layer1_manifests/
    prediction_records/
    model_registry_records/
    baseline_metric_records/
    low_calibration_curve_records/
    subject_profile_records/
    calibration_records/
    uncertainty_records/
    selective_prediction_records/
```



```
threshold_registry_records/
iharq_records/
regime_state_records/
trust_state_records/
policy_records/
policy_update_traces/
transition_records/
closed_loop_session_records/
episode_summary_records/
policy_rollout_records/
reward_trace_records/
environment_diagnostic_records/
stress_records/
embodiment/
  embodiment_state_records.parquet
  embodiment_outcome_records.parquet
  transition_records.parquet
  safety_event_log.csv
  reward_traces/
  demo_manifests/
dashboard_metric_records/
dashboard_view_records/
card_records/
figures/
  dashboard_exports/
  embodiment/
  thesis_ready/
cards/
  dataset_cards/
  protocol_cards/
  decoder_cards/
  calibration_cards/
  iharq_cards/
  regime_cards/
  trust_state_cards/
  policy_cards/
  policy_update_trace_cards/
  stress_cards/
  embodiment_cards/
  safety_cards/
  failure_taxonomy_cards/
dashboard_app/
thesis_tables/
manifests/
  artifact_manifest.yaml
  reproducibility_manifest.yaml
  run_registry.yaml
  layer1_ablation_readiness_l1_v1.json
  claim_evidence_matrix.json
  metric_dictionary.json
  embodiment/
```

```

logs/
  validation_logs/
  policy_update_audits/
  layer9_safety_events/
  export_logs/

```

This organization makes clear that Layer 10 consumes all records but does not own the experimental logic of earlier layers.

16.15 Testing and quality assurance

Layer 10 must be tested as a serious software and research component. Dashboard QA metrics include artifact completeness, card completeness, record-validation pass rate, figure traceability, table traceability, config reproducibility, warning coverage, missingness visibility, export determinism, manifest completeness, claim-to-artifact coverage, source-record lineage coverage, blocked-export correctness, and supervisor/examiner inspection pass rate.

Test type	Purpose
Schema tests	Validate that records contain required fields, valid types, legal enumerations, and valid ranges.
Contract tests	Ensure Layer 10 can consume records produced by Layers 1–9.
Snapshot tests	Ensure dashboard summaries and exported tables remain stable when inputs do not change.
Figure-generation tests	Ensure expected figures are generated from test records and tied to source records.
Card-generation tests	Ensure cards contain required sections, warnings, limitations, and reproduction fields.
Manifest tests	Ensure source records, configs, seeds, environment, artifact paths, and warning flags are recorded.
Warning-flag tests	Ensure simulation-only, proxy-label, public-data, diagnostic-only, and clinical non-claim warnings appear where required.
Ablation-lineage tests	Ensure Layer 1 ablation-readiness warnings, A0–A4 ablation-step IDs, matched-comparison IDs, metric-dictionary references, and claim-evidence matrix entries appear in thesis-used exports.
Policy-update trace tests	Ensure update-enabled bandit/RL runs expose <code>PolicyUpdateTrace</code> completeness, disabled-update audits are visible, and frozen-policy evaluation exports contain no hidden mutation.
Missing-record tests	Ensure the dashboard reports missing records instead of crashing or hiding gaps.
Export determinism tests	Ensure repeated exports from the same records and configs are reproducible.
Usability inspection	Ensure pages help supervisors and examiners understand results and lineage.

Layer 10 is ready only when canonical record schemas have validators, every dashboard page has a specification, every card type has a template, optional

`TrustStateRecord` files have dashboard/export handling when produced, Layer 7 `TransitionRecord`, `RewardTraceRecord`, `EpisodeSummaryRecord`, `PolicyRolloutRecord`, `EnvironmentDiagnosticRecord`, and `SafetyEventRecord` files have dashboard/export handling when produced, optional `PolicyUpdateTrace` files have dashboard/export handling when produced, bandit/RL training-adaptation curves are separated from frozen-policy evaluation curves, the Layers 0–3 evaluation slice has ablation-lineage, matched-comparison, metric-dictionary, negative-result, and claim-evidence exports where applicable, the Layers 4–7 evaluation slice has local/global ablation lineage and frozen/update separation where applicable, the Layers 8–9 A12/A13 evaluation slices have matched-stress or matched-embodiment reports, metric dictionaries, negative-result notes where applicable, dashboard/export lineage, and claim-evidence entries, figure exports record source files and filters, the reproducibility manifest is generated, warning flags appear for simulation-only, stress-proxy, embodiment-proxy, diagnostic-only, and proxy-label artifacts, missing records are visible, thesis figures can be traced to records, card completeness reaches the chosen threshold, and the dashboard does not mutate upstream records.

16.16 Layers 4–7 evaluation-lineage dashboard addendum

The Layers 4–7 propagation adds a mandatory Layer 10 observability contract. Layer 10 must consume Layer 4–7 `EvaluationOutcomeRecord` files, matched-comparison reports, metric dictionaries or snapshots, negative-result notes, claim-evidence entries, dashboard/card/export lineage, diagnostic-only labels, and controller-card lineage where applicable. These are evaluation-only and reporting-only inputs; they must not be displayed as decision-time policy inputs and must not be used by the dashboard to retune thresholds, alter IHARQ rules, reclassify `RegimeRisk` states, update policies, rerun simulations, or repair missing matches without a new explicit run.

Dashboard area	Added required view	Validation and warning behavior
IHARQ decision-flow page	A5 IHARQ-lite local/global ablation IDs, A6/A7 participation flags, matched-comparison reports, metric-dictionary entries, negative-result notes, and claim-evidence entries	Unmatched IHARQ comparisons remain visible but must be marked diagnostic-only.
RegimeRisk timeline page	A7 regime-aware versus regime-unaware matched comparisons, shuffled/random risk negative controls, metric dictionaries, negative-result notes, and claim-evidence entries	Missing negative controls or failed matching blocks temporal-trust improvement wording.

Adaptive policy page	A6, A8, A9, A10, A11, A12, and A13 local/global ablation IDs where applicable; matched cost, coverage, reward, and stress comparisons; controller-card lineage; and cost/reward sensitivity summaries	Reward-sensitive or unmatched policy rankings must carry diagnostic-only or limitation warnings.
Contextual bandit and RL page	A10/A11 frozen-policy held-out evaluation reports, update-enabled trace separation, disabled-update audits, metric-dictionary snapshots, and negative-result notes	Training/adaptation curves must not be merged with frozen-policy evaluation curves.
Ablation and comparison page	A0–A13 ladder with Layers 4–7 local IDs, matched-comparison keys, metric dictionary IDs or snapshots, negative-result notes, claim-boundary labels, and diagnostic-only flags	Unmatched comparisons may be shown for diagnosis but cannot be styled as thesis-ready improvements.
Cards and exports	Layer 4–7 evaluation cards or card sections linked to source records, metric definitions, comparison keys, negative-result notes, and claim-evidence entries	Export is blocked or downgraded if required lineage fields are absent.

Layer 10 is thesis-ready only when the existing Layers 0–3 evaluation slice and the newly propagated Layers 4–7 evaluation slice are both visible in dashboard records, cards, figures, tables, export manifests, reproducibility manifests, and claim-evidence views. The added readiness condition is: every Layer 4–7 thesis-used evaluation view must expose local/global ablation IDs, matched-comparison keys, metric-dictionary entries or snapshots, negative-result notes, claim-evidence entries, frozen/update separation where applicable, and diagnostic-only labels where matching fails.

16.17 Layers 8–10 evaluation-lineage dashboard addendum

The MERGE_RECORD_22 Layers 8–10 merge adds the final local evaluation-ownership slice for StressForge robustness, Layer 9 embodiment-proxy consequences, and Layer 10 read-only dashboard/reproducibility validation. Layer 10 must consume these artifacts as saved evaluation evidence: Layer 8 A12 `EvaluationOutcomeRecords`, matched-stress comparison reports, stress metric-dictionary entries, stress negative-result notes, stress claim-evidence entries, Layer 9 A13 `EvaluationOutcomeRecords`, matched embodiment comparison reports, embodiment metric-dictionary entries, embodiment negative-result notes, embodiment claim-evidence entries, and dashboard/card/export lineage. These artifacts are reporting inputs only; Layer 10 may inspect,

filter, aggregate, card, and export them, but it must not mutate stress schedules, simulator settings, command mappings, reward weights, policy choices, or missing comparison keys.

Dashboard area	Added required view	Validation and warning behavior
StressForge page	A12 local/global ablation IDs, clean/stress pairing, single-stressor and combined-stress slices, held-out stress status where relevant, matched-stress reports, metric dictionaries, negative-result notes, and claim-evidence entries	Unmatched stress comparisons, hand-picked stress survival, or missing held-out stress evidence remain visible but must be marked diagnostic-only or limitation-bound.
Layer 9 embodiment page	A13 local/global ablation IDs, E0–E5 mode, simulator branch, command-mapping ID, matched embodiment report, metric dictionaries, negative-result notes, demo-manifest status, and claim-evidence entries	Missing simulator lineage, unmatched mappings, unstable adapter runs, reward-dominated findings, or absent Layer9EmbodimentDemoManifests block thesis-ready embodiment improvement wording.
Ablation and comparison page	A12 and A13 rows in the A0–A13 ladder with matched-stress and matched-embodiment status, local layer owners, metric definitions, negative-result notes, and claim boundaries	Stress and embodiment evidence can support only controlled robustness or simulation-only action-consequence claims; clinical, rehabilitation, real-device, and real patient-safety claims stay blocked.
Cards and exports	StressForge cards, embodiment evidence cards, A12/A13 evaluation-outcome exports, matched-comparison exports, metric-dictionary exports, negative-result note exports, and claim-evidence exports	Export is blocked or downgraded if lineage, warnings, source records, comparison keys, metric definitions, or limitation statements are missing.
Reproducibility page	Bidirectional links from thesis figures/tables to stress configs, stress schedules, Layer 9 configs, command mappings, simulator versions, seeds, records, cards, and claim-evidence entries	The page reports missingness rather than reconstructing or repairing absent records.

Layer 10 is thesis-ready for the MERGE_RECORD_22 Layers 8–10 slice only when A12 and A13 artifacts appear in dashboard records, cards, figures, tables, export manifests, reproducibility manifests, and claim-evidence views. The added readiness condition is: every thesis-used A12/A13 view must expose local/global ablation IDs, matched-comparison keys, metric-dictionary entries or snapshots, negative-result notes where applicable, claim-evidence entries, simulation/stress/embodiment warning flags, and diagnostic-only labels where matching fails.

16.18 Implementation roadmap and maturity model

Phase	Tasks	Exit condition
L10-0	Define input record schemas, <code>DashboardMetricRecord</code> , <code>DashboardViewRecord</code> , artifact identity standard, card templates, and manifest schema.	Layer 10 knows what it expects and exports.
L10-1	Build record loader, validator, missingness report, and record index by dataset, subject, session, model, policy, stress, and run ID.	Available records can be inspected without visualization.
L10-2	Build core summary tables for dataset, decoder, calibration, IHARQ, RegimeRisk, policy, optional policy-update trace, stress, and embodiment records.	Non-interactive summaries exist for all produced layers.
L10-3	Implement <code>DashboardViewRecord</code> generation, dashboard pages with filters, warnings, drill-down where feasible, and links to source records and cards.	The dashboard can inspect all major results and serialize page-level view provenance.
L10-4	Implement universal card schema, card templates, card completeness checks, limitations, and warning flags.	Required cards can be generated from saved records.
L10-5	Implement figure/table/card export, optional policy-update trace card/export handling, artifact manifest, reproducibility manifest, and export logs.	Every exported artifact can be traced to records and configs.
L10-6	Assign thesis figure/table IDs, map artifacts to chapters, generate appendix material, and write reproduction instructions.	Layer 10 outputs are thesis-manuscript ready.

Level	Name	Meaning
L10-MRL-0	Concept	Dashboard, cards, and reproducibility are only mentioned.
L10-MRL-1	Role defined	Purpose, scope, and non-scope are defined.
L10-MRL-2	Record contracts defined	Required input records, <code>DashboardMetricRecord</code> , and <code>DashboardViewRecord</code> are specified.

L10-MRL-3	View specs defined	Dashboard pages, required views, <code>DashboardViewRecord</code> fields, and page inspection questions are specified.
L10-MRL-4	Card templates defined	All card types and schemas are defined.
L10-MRL-5	Loader implemented	Saved records can be loaded and validated.
L10-MRL-6	Dashboard implemented	Core pages are usable and read from records.
L10-MRL-7	Cards generated	Standardized cards are produced with limitations and provenance.
L10-MRL-8	Reproducibility package generated	Manifests, exports, configs, and artifact indexes exist.
L10-MRL-9	Thesis-ready	Figures, tables, cards, traceability, limitations, and reproduction instructions are manuscript-ready.

Layer 10 should reach at least L10-MRL-4 before implementation begins, L10-MRL-8 before thesis claims are treated as reproducible, and L10-MRL-9 before final manuscript submission.

16.19 Failure modes, defensive design, and validity threats

Failure mode	Risk	Defense
Dashboard becomes cosmetic	It looks impressive but does not support scientific claims.	Link every page to records, metrics, claims, cards, ablations, limitations, or reproducibility artifacts.
Dashboard becomes a second experiment engine	It recomputes hidden logic and creates untracked results.	Enforce record-only reading; allow recomputation only through explicit, versioned configs and new run IDs.
Cards become superficial summaries	Cards lack limitations, source records, or reproduction commands.	Enforce universal card schema and card completeness checks.
Reproducibility package is incomplete	Future readers cannot reproduce results.	Generate manifests, validation logs, artifact indexes, environment files, and known-limitations reports.
Simulation claims are overstated	Layer 7, Layer 8, Layer 9, contextual bandit, or RL outputs are interpreted as clinical proof.	Attach simulation-only and clinical non-claim warnings to all relevant views, cards, and artifacts.
Policy-update traces are hidden or mixed with evaluation	Online/adaptive learning appears better because training/adaptation curves are confused with frozen-policy evaluation, or policy mutations lack auditability.	Require <code>PolicyUpdateTrace</code> completeness for update-enabled runs, show disabled-update audits, and keep training/adaptation exports separate from frozen-evaluation exports.

Missing records are hidden	The thesis appears more complete than it is.	Show missingness and mark incomplete pages/cards/artifacts explicitly.
Ablation evidence is weak	Architectural complexity is not defended.	Make A0–A13 visible and include matched-cost, matched-coverage, and matched-stress comparisons.
Manual figure editing breaks lineage Threshold leakage is not surfaced	Thesis figures no longer match saved records. Selective-prediction or threshold claims may rely on invalid split usage.	Use export manager, checksums, artifact IDs, and audit logs. Cross-check ThresholdRegistryRecord against allowed selection splits, display leakage warnings, and block thesis-ready selective-prediction claims until repaired.
Policy reward leakage is not surfaced	Policy-learning claims may use future or test-only information not allowed by protocol.	Validate reward configuration, policy-update mode, and transition linkage; emit reward-leakage warnings and require rerun with a valid reward protocol.
Record IDs do not join across layers	Figures and cards may combine incompatible datasets, subjects, runs, stress conditions, or embodiment branches.	Use lineage resolver unresolved-link reports, mark affected pages incomplete, and repair upstream identifiers or manifest mappings.
Warning flags are missing	Simulation, proxy, stress, or embodiment artifacts may appear stronger than their evidence permits.	Require warning flags in page metadata, cards, exports, logs, and manifests; block unsafe export when warning metadata is absent.

Layer 10 has its own limitations. It can prove traceability of artifacts, but it cannot prove that every upstream scientific decision was optimal. It can expose failure categories, but it cannot eliminate them. It can reproduce saved experiments, but public datasets and external simulator versions may change. It can support claim safety, but careful thesis writing must still avoid unsupported clinical, deployment, or real-patient claims.

16.20 Expected thesis evidence from Layer 10

Layer 10 should produce the following thesis-facing evidence:

- architecture observability overview figure;
- dataset and protocol cards for the methodology chapter;
- decoder and low-calibration comparison tables;
- calibration reliability diagrams and uncertainty summaries;

- IHARQ decision-flow plots and reason-code tables;
- RegimeRisk timelines and temporal trust summaries;
- Adaptive-IHARQ and learning-to-defer policy comparison tables;
- Layer 6 policy-stage, context-level ablation, action-legality, cost/reward sensitivity, policy-to-layer dependency, and claim-boundary matrices;
- contextual bandit and RL reward/regret figures with simulation-only warnings, policy-update trace cards where applicable, Layer 7 closed-loop session timelines, policy roll-out cards, reward-decomposition views from `RewardTraceRecord`, environment-diagnostic warning panels from `EnvironmentDiagnosticRecord`, and separated training/adaptation versus frozen-policy evaluation curves;
- StressForge stress sensitivity curves and stress-conditioned ablation tables;
- Layer 9 MyoSuite/OpenSim/static-replay endpoint-error and command timeline figures, safety-event tables, `EmbodimentOutcomeRecord`-grounded outcome summaries, reward-decomposition plots from `Layer9RewardTrace` where enabled, `Layer9EmbodimentDemoManifest` summaries, and embodiment non-claim warnings;
- A0–A13 ablation ladder table;
- failure-taxonomy card and limitation appendix material;
- artifact manifest, run registry, reproducibility manifest, and reproduction guide.

The strongest version of Layer 10 demonstrates that every result comes from a saved record, every dashboard page has a `DashboardViewRecord` with inspection question, filters, warning flags, and completeness state, every figure comes from a traceable artifact, every card has limitations and source records, optional `PolicyUpdateTrace` files are traceable when produced, Layer 6 policy matrices are generated for thesis-used policy results, Layer 7 closed-loop session timelines, policy rollout cards, reward-decomposition views, safety-event views, and environment-diagnostic warning panels are grounded in `TransitionRecord`, `ClosedLoopSessionRecord`, `EpisodeSummaryRecord`, `PolicyRolloutRecord`, `RewardTraceRecord`, `SafetyEventRecord` where materialized, and `EnvironmentDiagnosticRecord`, Layer 9 embodiment evidence is consumed only through saved Layer 9 records, safety-event logs, reward traces, `Layer9EmbodimentDemoManifests`, and dashboard-compatible exports, every dashboard page supports a research inspection task, every ablation is visible, every simulation-only claim is marked, bandit/RL training-adaptation evidence is separated from frozen-policy evaluation evidence, and every major thesis claim can be traced to records, figures, tables, cards, and limitations.

17 Cross-Layer Functional Module Architecture

This section restates IHARQ BenchGuard Stretch C by functional responsibility rather than by revised layer number. The layer view explains the conceptual progression from claim-safety governance to public-data anchoring, decoding, calibration, IHARQ verification, RegimeRisk monitoring, adaptive policy learning, simulation, stress testing, embodiment, and dashboard reproducibility. The cross-layer module view explains how those responsibilities should be implemented as a modular research codebase. It defines how records move across modules, how feedback is controlled, how claim-safety rules are propagated, how evaluation claim-evidence

gates and matched ablation comparisons inherit source-record lineage, and how the dashboard inspects saved artifacts without silently recomputing results.

The central architectural point is that Stretch C is not a monolithic EEG classifier script. The public EEG decoder is the measurement source, but the thesis contribution is the surrounding functional graph: calibrated uncertainty, IHARQ-style evidence verification, temporal trust monitoring, adaptive readiness policy learning, simulated closed-loop evaluation, StressForge-Lite stress testing, Layer 9-owned MyoSuite/OpenSim/static-replay proxy embodiment, and dashboard-based reproducibility. This section therefore treats the system as a record-first and artifact-first graph of independent modules connected through explicit records, manifests, versioned configurations, reason codes, logs, and cards. The former detailed P20 embodiment namespace is not an active runtime module in this cross-layer view; its useful implementation content is represented as Layer 9 Module H internals and as Layer 10 read-only dashboard/reproducibility consumption.

17.1 Purpose of the Cross-Layer Module View

The layer view alone is insufficient for implementation because several implementation responsibilities cut across layer boundaries. For example, a `PredictionRecord` originates in Layer 2, but it is consumed by calibration in Layer 3, evidence verification in Layer 4, temporal trust monitoring in Layer 5, adaptive policy learning in Layer 6, simulation in Layer 7, stress-conditioned analysis in Layer 8, embodiment in Layer 9, and dashboard inspection in Layer 10. Similarly, claim-safety rules originate in Layer 0, but they must appear in every simulator card, embodiment card, policy report, dashboard warning, and thesis-facing table.

The cross-layer module view is necessary because Stretch C is a functional graph. Its modules must remain independently testable and replaceable while sharing common records and controlled feedback signals. This view supports code organization, experiment control, reproducibility, dashboarding, ablation design, and thesis explanation by making each module's ownership, inputs, outputs, forbidden dependencies, quality attributes, and evidence obligations explicit. After the Layers 0–3 evaluation-slice merge, this view also records that Module A supplies public-data/protocol matchability, Module B supplies A0 raw-decoder evidence and A4-style ordinary-combination controls, Module C supplies A1 calibration, A2 thresholding, and A3 uncertainty/selective-prediction comparisons, and Layer 0/Module I together enforce claim-evidence lineage for those results. After the Layers 8–10 propagation, this view also records that Module G supplies the nearest A12 StressForge robustness-evaluation records, matched-stress comparison reports, stress metric dictionaries, negative-result notes, dashboard/export lineage, and claim-evidence entries; Module H supplies the nearest A13 embodiment-proxy evaluation records and matched embodiment comparison artifacts; and Module I consumes those artifacts read-only for dashboard, card, export, and reproducibility validation without becoming the producer of stress or embodiment evidence.

Without this view, the architecture could collapse into a single script that loads EEG data, trains a model, computes metrics, and then retrofits reliability claims. That would weaken the thesis because it would obscure protocol leakage, threshold selection, policy tuning, stress injection, simulation-only boundaries, and reproducibility provenance. The cross-layer module view prevents that collapse by requiring every result to pass through declared records, every decision to be reason-coded, every feedback loop to be justified, and every thesis claim to be traceable to artifacts.

17.2 Stakeholders and Concerns for the Cross-Layer View

Stakeholder	Architectural concerns addressed by this view
Thesis examiner or reviewer	Scientific novelty beyond baseline classification; strict claim discipline; reproducible evaluation; visible ablations; defensible separation between public EEG evidence, simulated closed-loop behavior, stress testing, and non-clinical embodiment demonstration.
Implementer	Clear module boundaries; stable interfaces; record schemas; configuration files; artifact paths; unit and integration tests; accepted failure states; reproducible run identifiers.
Experiment designer	Fixed dataset protocols; leakage-safe splits; low-calibration budgets; model baselines; calibration splits; threshold-validation rules; ablation ladders; stress profiles; matched-cost and matched-coverage comparisons.
Reliability researcher	Calibration quality; uncertainty features; high-confidence wrong predictions; evidence quality; temporal trust; regime instability; decision risk; failure taxonomies.
Policy-learning researcher	Pseudo-label generation; deferral targets; legality rules; cost and reward versions; staged policy escalation; contextual bandit and RL evaluation as simulation-only; separation between offline labels and online rewards.
Dashboard user	Traceable dashboard views; source-record drilldown; cards; failure cases; plots; warning summaries; reproducibility exports; no hidden recomputation or silent mutation.
Future maintainer	Versioned configs; module independence; extensible registry design; rerun logic; provenance; compatibility checks; deterministic replay where feasible.

17.3 Cross-Layer Architectural Principles

1. **Record-first communication.** Modules exchange declared records, not hidden Python objects or implicit global variables.
2. **Explicit record ownership.** Each record type has one canonical producer module and declared consumer modules.
3. **Immutable protocol versions.** Dataset, split, preprocessing, label-map, and low-calibration-budget versions become immutable after downstream consumption; changes require a new protocol version and a new manifest.
4. **No hidden downstream feedback into data protocol decisions.** Decoder accuracy, IHARQ outcomes, policy rewards, embodiment outcomes, or dashboard interactions must not modify Layer 1 protocols without an explicit new protocol record.
5. **Test-set lockout.** Calibration, threshold selection, IHARQ rule tuning, policy selection, reward-weight selection, and stress-profile selection must not use held-out test outcomes.
6. **Reason-coded decisions.** IHARQ actions, RegimeRisk warnings, policy actions, stress injections, embodiment gates, dashboard warnings, and claim-safety messages must carry machine-readable reason codes.

7. **Deterministic replay where feasible.** Fixed seeds, dataset versions, code versions, config hashes, and manifest records must allow reruns to reproduce saved records or explicitly explain nondeterminism.
8. **Configuration hashing and manifest provenance.** Every run artifact must record configuration hash, code version, environment summary, seed, dataset IDs, split IDs, model IDs, policy IDs, stress-profile IDs, and simulator IDs where applicable.
9. **Claim-safety propagation.** Layer 0 claim boundaries must appear in module outputs, cards, dashboard pages, exported figures, and thesis tables.
10. **Dashboard read-only behavior.** The dashboard reads saved records and manifests. It may request a new explicit run, but it must not silently recompute, retune, rewrite, or relabel experimental records.
11. **Simulation and embodiment labeling.** Layer 7, Layer 8, Layer 9, contextual bandit, and RL outputs must be labeled simulation-only or proxy-only. They must not be described as clinical validation, patient safety proof, real FES control, real exoskeleton validation, or validated fatigue diagnosis.
12. **Ablation readiness.** Every major cross-layer claim must have an ablation path, a matched comparison, a figure, a table, a card, or a failure-case audit. For the Layers 0–3 evaluation slice, this means A0 raw-decoder evidence, A1 calibration evidence, A2 threshold evidence, A3 uncertainty/selective-prediction evidence, and A4 ordinary-combination controls must remain matched by Layer 1 protocol keys and bounded by Layer 0 claim-evidence gates.

17.4 Module Taxonomy and Layer Mapping

Module	Primary function	Layers	Main records consumed	Main records produced	Mode	Claim boundary
A	Data and protocol engine	0–1, supports all	Public dataset files, dataset configs, protocol configs, Layer 0 evaluation claim boundaries	DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, DatasetCard, ProtocolCard, ValidationReport, Layer1Manifest, layer1_ablation_readiness_comparison_key completeness status	Offline; governance-facing	Public-data protocol only; no decoder-dependent protocol changes; unmatched ablation comparisons are diagnostic only.

B	Decoder and prediction engine	2	A records, decoder configs, budget configs, matched-comparison keys	PredictionRecord , OfflineModelRegistryRecord , BaselineMetricRecord , LowCalibrationCurveRecord , SubjectProfileRecord , A0 raw-decoder evidence, A4 ordinary-combination controls, downstream-readiness validation artifacts	Measurement source only; no readiness decision; ordinary voting/average window controls remain baselines, not IHARQ evidence, and Layer 9 use is bounded to provenance input compatibility. Reliability evidence; no threshold tuning on test data; threshold/selection sufficiency remains a bounded result, not clinical safety.
C	Calibration and uncertainty engine	3	PredictionRecord , split metadata, calibration configs, matched operating-point requirements	CalibrationRecord , OfflineUncertaintyRecord , BenchmarkSelectivePredictionRecord , ThresholdRegistry , facing matched-coverage/matched-cost operating-point tables, A1–A3 comparison summaries, negative-result notes	

D	Evidence verification engine	4, receives E context	PredictionRecord, EvidenceQualityRecord, Offline CalibrationRecord, IHARQDecisionRecord, IndivUncertaintyRecord, CombinedEvidenceCellRecord, ThresholdRegistry, FeedbackDecisionRecord, ArtifactFlagRecord, UnsafeEventRecord, optional IHARQAblationRecord, RegimeStateRecord, IHARQTraceRecord, and lo-Layer 4 cal/precomputed EvaluationOutcomeRecord, window-history, matched-channel-subset-comparison stability, and reports, metric-model-agreement dictionary views entries, negative-result notes, claim-evidence entries, dashboard trace exports, and reproducibility lineage	Readiness decision over evidence; diagnostic evidence-verification proxy only not clinical safety validation.
E	Temporal regime and trust engine	5, interacts with 4/6/7/8/9	WindowRecord, RegimeStateRecord, Offline PredictionRecord, conditional temporal CalibrationRecord, TrustStateRecord, replay or UncertaintyRecord, regime_timeliness.csv, SelectivePredictionRecords.csv, EvidenceQualityRecord, loss_warnings.csv, IHARQDecisionRecord, trust-score optional tables, trusted StressProfileRecord, risk trends, optional threshold-TransitionRecord, adjustment logs, optional risk-error EmbodimentStateRecord, adjustment tables, policy/action Layer 5 history, and EvaluationOutcomeRecord, subject difficulty matched-comparison and profiles negative-control reports, negative-result notes, claim-evidence entries, dashboard-ready regime tables, and RegimeCard lineage	Temporal operating-risk proxy not real fatigue diagnosis or clinical state inference.

F	Adaptive policy engine	6, interacts with 4/5/7/8/9	ReadinessContext , D/E records, cost/reward configs, pseudo-labels, rewards	PolicyRecord , ActionRecord , DeferralRecord , policy artifacts, controller cards	Offline learning and simulation-only adaptation	Policy learning over declared records; bandit/RL claims are simulation only.
G	Simulation and stress engine	7–8	A–F records, stress configs, seeds, optional policy-update config	TransitionRecord , optional PolicyUpdateTrace , disabled-update audits, StressProfileRecord , StressScheduleRecord , stress metrics, reward logs	Simulated online	Closed-loop, policy-update, and stress evaluation only; policy mutation is allowed only in declared simulation update mode and no real-world therapeutic outcome claim is allowed.
H	Embodiment adapter engine	9	D/E/F/G records, ReadinessContext Layer 9 embodiment configs, command mapping, readiness/safety-gate config, simulator state, optional stress profile and stress schedule	EmbodimentStateRecord , EmbodimentOutcomeRecord , Embodiment-Record , linked TransitionRecord pointers, SafetyEventRecord or safety-event log, Layer9RewardTrace where enabled, command timeline, endpoint-error curve, Layer9EmbodimentDemoManifest , dashboard-compatible export	Record , Reward , simulated embodiment	Proxy demonstration only; no real FES/exoskeleton patient-specific biomechanics, medical-device readiness, or patient validation.

I	Dashboard and reporting engine	10, reads all	All saved records, cards, manifests, logs, optional PolicyUpdateTrace files, disabled-update audits, claim-evidence matrix entries, metric dictionaries, matched-comparison reports, negative-result notes, Layer 1 ablation-readiness warnings, and Layer 9 embodiment bundles including safety-event logs, reward traces, command timelines, endpoint-error curves, Layer9EmbodimentDemoManifests , and dashboard-compatible exports	DashboardMetricRecord with policy-update, embodiment, and reward-trace filters; DashboardViewRecord ; CardRecord ; ReproducibilityManifest ; figure/table manifests; claim-to-artifact maps; A0–A4 evaluation-slice lineage exports; optional policy-update trace cards; Layer 9 embodiment evidence cards and exports	Dashboard-Inspection facing and thesis export only; no silent mutation, hidden recomputation, untracked policy update, simulator control from dashboard views, or claim-boundary removal from evaluation artifacts.
---	--------------------------------	---------------	--	--	--

Modules D, E, F, G, and H may participate in feedback, but only through declared records and only under the allowed feedback rules in this section. Module H is the single cross-layer functional home for the Layer 9 embodiment subsystem: its internals correspond to L9.1–L9.13, including record intake, readiness-context assembly, command mapping, safety gating, static replay, MyoSuite/OpenSim adapters, stress injection, outcome/reward bridging, transition/safety logging, and dashboard/card/export packaging. Module I is intentionally read-only with respect to experimental records and consumes Layer 9 embodiment outputs only as saved artifacts. Module A is intentionally upstream-protected.

17.5 Standard Module Specification Template

Every cross-layer module specification in Stretch C should follow the same template.

Specification field	Required content
---------------------	------------------

Identity and architectural position	Module name, related layers, architectural type, core responsibility, and downstream consumers.
Problem addressed	Scientific or engineering bottleneck solved by the module and what would fail if it did not exist.
Inputs	Required records, optional records, configuration files, external tool dependencies, and evaluation-only fields that must not be used during training, calibration, thresholding, or policy selection.
Outputs	Primary records, secondary records, cards, plots, manifests, dashboard exports, logs, warnings, and accepted missingness markers.
Internal processing logic	Major processing steps, validation rules, deterministic rules, learned components, split controls, leakage controls, and error-handling behavior.
Interface contract	Schema expectations, required identifiers, versioning, config hashes, seed handling, provenance fields, and accepted failure states.
Dependencies	Hard dependencies, soft dependencies, optional dependencies, forbidden dependencies, accepted feedback, and feedback never allowed.
Quality attributes	Reproducibility, traceability, modularity, robustness, inspectability, extensibility, comparability, and claim safety.
Failure modes and defensive design	Likely failure modes, detection logic, reporting logic, downstream reaction, and required warnings or card entries.
Evaluation evidence	Unit tests, integration tests, sanity checks, ablations, plots, tables, cards, dashboard views, and reproducibility manifests.
Thesis-facing contribution	Claim supported, claim not supported, and where outputs appear in the thesis.

17.6 Detailed Cross-Layer Module Specifications

17.6.1 Module A: Data and Protocol Engine

Aspect	Specification
Identity and position	Module A owns the public-data foundation and protocol contract. It corresponds primarily to Layer 1 and is governed by Layer 0. It is an offline, governance-facing module consumed by every downstream module.
Problem addressed	It prevents the thesis from depending on ad-hoc dataset loading, inconsistent label maps, hidden preprocessing, or decoder-dependent protocol choices. Without it, low-calibration results, calibration claims, IHARQ decisions, and policy comparisons would not be reproducible or leakage-safe.

Inputs	Public EEG motor-imagery datasets; dataset registry entries; dataset adapters; metadata normalization rules; label-map configuration; preprocessing registry; artifact-flag rules; split definitions; low-calibration budget definitions; trial/window-generation parameters; claim-safety rules from Layer 0. Evaluation-only fields such as held-out test outcomes must not influence split construction or preprocessing decisions.
Outputs	<code>DatasetRecord</code> , <code>WindowRecord</code> , <code>SplitRecord</code> , <code>PreprocessingRecord</code> , <code>LabelMapRecord</code> , <code>ArtifactFlagRecord</code> , <code>DatasetCard</code> , <code>ProtocolCard</code> , <code>ValidationReport</code> , <code>Layer1Manifest</code> , and <code>layer1_ablation_readiness_l1_v1.json</code> . It also exports split files, dataset-card tables, protocol-card tables, warning logs, comparison-key completeness summaries, diagnostic-only ablation warnings, and manifest hashes.
Internal processing logic	Register datasets; validate subject/session/run structure; normalize metadata; map labels into the thesis label ontology; generate trial and window records; apply declared preprocessing; flag artifacts without using downstream performance; create reproducible splits and low-calibration budgets; validate that each record has dataset, subject, session, run, trial, window, split, preprocessing, label-map, and protocol identifiers where appropriate.
Interface contract	All records must include dataset ID, protocol version, adapter version, code version, config hash, source dataset version or download fingerprint, split ID, seed where applicable, and claim-safety boundary. Missing metadata must be represented explicitly rather than silently imputed.
Dependencies	Hard dependencies: public dataset files, dataset adapters, protocol configs, Layer 0 claim-safety rules. Soft dependencies: MOABB-style loaders or equivalent public-data tooling. Optional dependencies: additional public datasets. Forbidden dependencies: decoder performance, calibration outcomes, IHARQ decisions, RegimeRisk signals, policy rewards, stress outcomes, embodiment outcomes, or dashboard interactions. Mockable dependencies: dataset adapters can be replaced by synthetic toy datasets for unit tests.
Consumers and feedforward	Module B consumes windows, splits, labels, preprocessing records, and budgets. Modules C–I consume identities, manifests, and cards for traceability. Module I consumes the cards and validation reports.
Feedback and loopback	No ordinary downstream feedback is allowed to mutate Module A outputs. A protocol issue discovered downstream may trigger a new protocol version, but the original consumed protocol remains immutable and auditable.
Quality attributes	Reproducibility, leakage prevention, traceability, comparability, deterministic replay, claim safety, and codebase modularity.

Failure modes and defensive design	Risks include inconsistent labels, hidden global normalization, split drift, missing subject identifiers, untracked preprocessing, and undocumented artifact rejection. Detection uses schema validation, split-integrity tests, provenance checks, and dataset-card completeness checks. Downstream modules must fail closed when required identifiers are missing.
Evaluation evidence	Dataset-card completeness reports; protocol-card tables; split-integrity tests; low-budget reproducibility tests; leakage checks; adapter smoke tests; validation reports; comparison-key completeness reports; ablation-readiness warnings; dashboard data-protocol page.
Thesis-facing contribution	Supports the claim that all later reliability and policy experiments are grounded in fixed public-data protocols. It does not support clinical deployment claims or decoder superiority claims by itself. Outputs appear in the methodology chapter, dataset/protocol appendix, and reproducibility package.

17.6.2 Module B: Decoder and Baseline Measurement Engine

Aspect	Specification
Identity and position	Module B is the decoder and baseline measurement engine for Layer 2. It trains and evaluates classical, Riemannian, deep, optional compact SSL, and ensemble baselines under fixed protocols and low-calibration budgets.
Problem addressed	It provides the measurement source that later reliability layers inspect. Without it, calibration, IHARQ, RegimeRisk, Adaptive-IHARQ, stress evaluation, and dashboard reports would have no traceable prediction evidence.
Inputs	<code>DatasetRecord</code> , <code>WindowRecord</code> , <code>SplitRecord</code> , <code>PreprocessingRecord</code> , <code>LabelMapRecord</code> , <code>ArtifactFlagRecord</code> , model configs, training/validation/test split manifests, label-budget configs, seed configs, and optional pretrained compact SSL checkpoints. Test labels may be used only for final evaluation metrics, not for model selection.
Outputs	<code>PredictionRecord</code> , trained model artifacts, <code>ModelRegistryRecord</code> , run manifests, failed-run logs, <code>BaselineMetricRecord</code> , <code>LowCalibrationCurveRecord</code> , cross-subject or subject-wise summaries, <code>SubjectProfileRecord</code> , <code>EnsembleControlRecord</code> , <code>FailureCaseIndex</code> , A0 raw-decoder evidence, A4 ordinary-combination control summaries, delay-burden notes for longer-window controls, ensemble-disagreement-ready outputs, <code>ModelCard</code> , <code>BenchmarkCard</code> , and <code>Layer2ReadinessReport</code> .

Internal processing logic	Load fixed windows and splits; instantiate declared baselines; train under budget constraints; evaluate on declared splits; save logits, probabilities, scores, canonical <code>predicted_class</code> , evaluation-only <code>label_true</code> , <code>probability_vector</code> , <code>score_type</code> , <code>class_order</code> , model IDs, split IDs, <code>split_role</code> , budget IDs, <code>timestamp_or_order</code> where replay is required, seed IDs, and failure states; produce disagreement-ready records for ensembles and later evidence verification.
Interface contract	<code>PredictionRecord</code> must include record IDs linking back to dataset, subject, session, trial, window, split, <code>split_role</code> , budget, model, model family, preprocessing, <code>score_type</code> , <code>class_order</code> , <code>timestamp_or_order</code> where replay is needed, seed, config hash, and code version. <code>ModelRegistryRecord</code> must preserve schema version, label-map version, artifact path, score type, class order, and optional ensemble membership through <code>model_registry_schema.yaml</code> . Model artifacts must be registered even when a run fails; failures must be explicit records, not missing rows.
Dependencies	Hard dependencies: Module A records, decoder configs, training environment. Soft dependencies: EEG machine-learning libraries, Riemannian toolkits, deep-learning frameworks. Optional dependencies: compact SSL encoders and ensembles. Forbidden dependencies: IHARQ decisions, policy rewards, stress-test outcomes, embodiment outcomes, dashboard edits, and post-test threshold tuning. Mockable dependencies: toy windows and dummy classifiers for integration tests.
Consumers and feedforward	Module C consumes probabilities/logits/scores and model-registry schema fields. Module D consumes predictions and agreement features. Module E consumes temporal prediction histories. Module F consumes prediction-derived contexts. Module G consumes prediction histories for simulated replay where configured. Module H consumes only bounded provenance/readiness-input evidence and must not treat Layer 2 records as embodiment-safety validation. Module I consumes metrics, model registry artifacts, subject profiles, low-calibration curves, and baseline records.
Feedback and loopback	No downstream module may alter training splits or model selection after test inspection. Downstream failure analysis may propose a new explicitly versioned experiment, not mutate the old run.
Quality attributes	Comparability across baselines, reproducibility, traceability, robustness to failed runs, modular baseline replacement, and inspectability.

Failure modes and defensive design	Risks include invalid split use, inconsistent preprocessing, seed drift, uncalibrated score ambiguity, failed deep runs, hidden alias fields, missing <code>split_role</code> , missing <code>score_type</code> , missing <code>class_order</code> , orphan predictions, and ensemble records with mismatched windows. Detection uses split checks, schema validation, model registry checks, downstream-readiness validation, and run-failure logs. Downstream modules must ignore incomplete predictions unless failure states are explicitly supported.
Evaluation evidence	Baseline accuracy/balanced-accuracy/F1 where appropriate; low-label curves; cross-subject curves; subject difficulty profiles; model registry tables; A0 raw-decoder tables; A4 voting/averaging/longer-window/ensemble controls; matched-comparison readiness checks; failed-run summaries; reproducibility reruns; decoder smoke tests.
Thesis-facing contribution	Supports the claim that the reliability architecture is evaluated over traceable baseline measurement sources. It does not itself make readiness or safety decisions. Outputs appear in baseline tables, low-calibration figures, subject-profile figures, and dashboard decoder pages.

17.6.3 Module C: Calibration and Uncertainty Engine

Aspect	Specification
Identity and position	Module C is the bridge from raw prediction to reliability evidence. It corresponds to Layer 3 and is consumed by IHARQ, RegimeRisk, Adaptive-IHARQ, simulation, and dashboard modules.
Problem addressed	Raw classifier probabilities are not reliable evidence by default. Without Module C, IHARQ would confuse confidence with calibrated reliability, threshold comparisons would be uncontrolled, and high-confidence wrong predictions would remain hidden.
Inputs	<code>PredictionRecord</code> , <code>DatasetRecord</code> , <code>WindowRecord</code> , <code>SplitRecord</code> , model registry entries, calibration split, threshold-validation split, label map, logits/probability vectors or declared score proxies, optional ensemble metadata, and calibration configs. Test labels are evaluation-only and must not tune calibrators or thresholds.
Outputs	<code>CalibrationRecord</code> , <code>UncertaintyRecord</code> , <code>SelectivePredictionRecord</code> , <code>ThresholdRegistryRecord</code> , <code>SplitIntegrityReport</code> , <code>ReliabilityAuditReport</code> , reliability diagrams, expected calibration error summaries, Brier score summaries, negative log-likelihood summaries where valid, risk-coverage curves, matched-coverage/matched-cost operating-point tables, high-confidence wrong tables, A1–A3 comparison summaries, negative-result notes, and dashboard metric records.

Internal processing logic	Validate split separation; fit calibration methods on declared calibration data; compute calibrated probabilities; compute entropy, margin, maximum probability, confidence, disagreement, calibration residual summaries, and selective-prediction decisions; select thresholds only on validation or calibration-selection splits; audit high-confidence wrong cases on evaluation splits; produce reliability records with selection provenance.
Interface contract	Calibration and threshold records must include model ID, split ID, calibration split ID, threshold-selection split ID, calibrator type, calibrator version, threshold version, metric version, config hash, seed, and forbidden-field audit.
Dependencies	Hard dependencies: Modules A and B, calibration configs, split metadata. Soft dependencies: calibration libraries and plotting tools. Optional dependencies: ensemble predictions and stress metadata for stratified audits. Forbidden dependencies: held-out test outcomes for fitting, threshold selection, IHARQ tuning, or policy tuning. Mockable dependencies: synthetic probability records with known calibration behavior.
Consumers and feedforward	Module D consumes calibrated uncertainty; Module E consumes uncertainty trajectories; Module F consumes policy contexts and selective baselines; Module I consumes plots, audit tables, and threshold registries.
Feedback and loopback	Module C does not accept policy or embodiment feedback. If calibration failure is detected, a new calibrator version or threshold registry version must be created; prior outputs remain auditable.
Quality attributes	Test-set lockout, traceability, reliability interpretability, calibration comparability, threshold discipline, and high-confidence-error auditability.
Failure modes and defensive design	Risks include tuning on test data, using uncalibrated scores as probabilities, selecting thresholds after seeing final outcomes, missing high-confidence wrong cases, and inconsistent threshold definitions across models. Detection uses split-integrity reports, threshold-provenance checks, calibration-card warnings, and dashboard warnings.
Evaluation evidence	Reliability diagrams; ECE/Brier/NLL tables where appropriate; high-confidence wrong tables; A1 calibration deltas; A2 threshold comparisons; A3 risk-coverage and selective-prediction baselines; matched operating-point tables; calibration ablations; negative-result notes; split-integrity reports.
Thesis-facing contribution	Supports the claim that later readiness decisions are based on calibrated and audited evidence rather than raw classifier confidence. It does not prove clinical reliability or real-time deployment safety. Outputs appear in the calibration chapter, reliability audit appendix, and dashboard reliability pages.

17.6.4 Module D: Evidence Verification Engine

Aspect	Specification
Identity and position	Module D is the IHARQ evidence verification engine for Layer 4. It is decoder-agnostic, post-calibration, and decision-focused.
Problem addressed	Calibration alone does not define readiness. Module D addresses the bottleneck of deciding whether current evidence is sufficient, unstable, contradictory, incomplete, or unsafe for simulated downstream action. Without it, the architecture would reduce to ordinary thresholding or selective prediction.
Inputs	PredictionRecord, CalibrationRecord, UncertaintyRecord, ThresholdRegistryRecord, ArtifactFlagRecord, optional RegimeStateRecord, IHARQ rule configs, claim-safety constraints, and local/precomputed helper views for window history, channel-subset stability, and model agreement. These helper views are implementation conveniences derived from declared upstream records and must not be promoted into new canonical upstream records. Evaluation-only true labels may be used for offline scoring, not for online decision formation.
Outputs	EvidenceQualityRecord, IHARQDecisionRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, Layer 4 EvaluationOutcomeRecord, matched-comparison reports, metric-dictionary entries, negative-result notes, claim-evidence entries, reason-code distributions, action-cost tables, request/combine logs, fallback logs, unsafe-action logs, dashboard trace exports, reproducibility lineage, IHARQ decision-flow plots, and IHARQ cards. Decisions include accept, wait, request evidence, combine, fallback, uncertain, and unsafe.
Internal processing logic	Validate calibrated evidence, threshold-registry provenance, action vocabulary validity, reason-code completeness, and evaluation-only field separation; compare uncertainty, agreement, channel-subset stability, window-history stability, and model-family agreement features against declared rules or learned evidence-quality scores; combine multi-window evidence where allowed; request additional evidence when evidence is insufficient; fallback to conservative modes when disagreement or regime risk is high; emit reason-coded decisions, fallback/unsafe records, traces, warnings, matched-comparison keys, metric-dictionary links, diagnostic-only flags, and claim-evidence pointers.
Interface contract	Decision, evidence, combination, fallback, unsafe, trace, ablation, and evaluation records must include dataset/window/model/calibration IDs, source prediction/evidence IDs, evidence feature versions, IHARQ config version, threshold-registry version, optional regime-state ID, action code, action-vocabulary version, reason-code hierarchy, request/combine/fallback/unsafe status where applicable, matched-comparison key, metric-dictionary ID or snapshot, diagnostic-only flag where applicable, decision timestamp or replay index, dashboard trace fields, reproducibility pointers, and claim-boundary tag.

Dependencies	Hard dependencies: Modules B and C. Soft dependencies: Module E regime context. Optional dependencies: ensemble disagreement and multi-window evidence. Forbidden dependencies: test-label-based decision tuning, simulation reward tuning unless explicitly part of a later Module F/G policy experiment, and dashboard edits. Mockable dependencies: synthetic uncertainty and disagreement sequences.
Consumers and feedforward	Module E consumes decisions as temporal events; Module F consumes decisions as rule baselines and contexts; Module G consumes readiness decisions for simulated sessions; Module H consumes gates; Module I consumes records and cards.
Feedback and loopback	Accepts declared feedback from Module E as regime-aware conservatism signals. It must not accept reward feedback from Module G directly; policy adaptation belongs to Module F.
Quality attributes	Inspectability, reason-coded behavior, modularity, decoder agnosticism, conservative failure handling, action-vocabulary validity, matched-comparison traceability, metric-dictionary linkage, reproducibility lineage, dashboard traceability, and claim safety.
Failure modes and defensive design	Risks include reducing IHARQ to a single confidence threshold, missing disagreement, requesting evidence indefinitely, using future labels, hiding unsafe states, omitting fallback/combination traces, losing matched-comparison keys, or promoting diagnostic-only variants to thesis-ready evidence. Detection uses action-vocabulary checks, reason-code coverage tests, threshold-vs-IHARQ ablations, fallback/no-fallback and unsafe/no-unsafe ablations, request/combine/fallback/unsafe log completeness checks, trace-record validation, metric-dictionary linkage checks, and failure-case cards.
Evaluation evidence	IHARQ versus threshold/selective-prediction baselines; confidence-only, entropy-only, margin-only, longer-window, voting, averaging, ensemble, fallback-only, no-stability, no-fallback, no-unsafe, no-combination, and regime-unaware ablations; action distribution tables; accepted-trial accuracy; wrong-accept rate; risk-coverage and matched-cost comparisons; high-confidence wrong interception analysis; evidence-request, combination, fallback, and unsafe-event burden; Layer 4 <code>EvaluationOutcomeRecord</code> exports; <code>IHARQAblationRecord</code> tables; <code>IHARQTraceRecord</code> drilldowns; metric-dictionary-linked matched-comparison reports; negative-result notes; dashboard trace exports; and failure-case audit.
Thesis-facing contribution	Supports the claim that Stretch C adds an explicit evidence-verification layer between decoding and action readiness through saved decision, trace, fallback, combination, unsafe-event, ablation, matched-comparison, and evaluation-outcome artifacts. It does not support real patient safety, device certification, or clinical safety claims. Outputs appear in IHARQ methodology, ablation tables, decision-flow figures, matched-comparison reports, reliability cards, dashboard trace views, claim-evidence entries, and reproducibility bundles.

17.6.5 Module E: Temporal Regime and Trust Engine

Aspect	Specification
Identity and position	Module E is the RegimeRisk temporal trust monitor for Layer 5. It operates across windows, trials, sessions, stress conditions, and simulated transitions.
Problem addressed	Single-window reliability may miss temporal degradation, volatility-like instability, drawdown-like reliability loss, model trust shifts, or repeated near-failure patterns. Without Module E, IHARQ and policy learning would lack temporal operating-state context.
Inputs	WindowRecord, PredictionRecord, CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, EvidenceQualityRecord, IHARQDecisionRecord, subject/session histories, policy/action history, subject difficulty profiles, optional StressProfileRecord, optional TransitionRecord, optional EmbodimentStateRecord, optional stress variables, optional embodiment outcomes, and RegimeRisk configuration. It must treat true labels as evaluation-only except for offline retrospective metrics.
Outputs	RegimeStateRecord, conditional TrustStateRecord, regime_timelines.csv, trust_scores.csv, stop_loss_warnings.csv, trust-score tables, trusted sets, risk trends, volatility scores, drawdown scores, drift/fatigue/dropout risk fields, failure-risk scores, fallback recommendations, stricter-evidence recommendations, no_action recommendations, unsafe-state recommendations, stop-loss status, regime transition logs, threshold-adjustment logs, risk-error alignment tables, Layer 5 EvaluationOutcomeRecord, matched-comparison reports, shuffled/random-risk negative-control reports, negative-result notes, metric-dictionary entries, claim-evidence entries, dashboard-ready regime tables, RegimeCard lineage, and dashboard timelines.
Internal processing logic	Aggregate sliding-window evidence with causal temporal cutoffs; monitor drift-risk proxies, uncertainty volatility, disagreement instability, model trust shifts, subject-difficulty effects, repeated wrong-accept patterns, stress/transition context where present, embodiment-proxy context where present, and stop-loss conditions; produce temporal summaries, trust allocations, conservative feedback adapters, threshold-adjustment logs, risk-error alignment tables, shuffled/random-risk negative-control comparisons, and dashboard-ready timeline exports for IHARQ/policy modules; mark all fatigue-like outputs as proxy operating-state signals, not diagnoses.

Interface contract	Regime, trust, timeline, stop-loss, evaluation, and RegimeCard-lineage artifacts must include subject/session/run IDs, time-window or replay-index range, source record IDs, aggregation horizon, regime config version, trust-state schema version where exported, policy/action-history references where used, stress-profile ID if present, simulator-run ID if present, embodiment-state ID if present, stop-loss trigger/cooldown/recovery state where applicable, matched-comparison key, metric-dictionary ID or snapshot, shuffled/random-risk control ID where applicable, reason codes, dashboard export references, and claim-boundary tag.
Dependencies	Hard dependencies: Modules B–D for prediction, uncertainty, and decision histories. Soft dependencies: Module G transitions and StressForge summaries. Optional dependencies: Module H proxy embodiment outcomes. Forbidden dependencies: clinical fatigue labels unless explicitly public and within claim-safe evaluation; dashboard edits; protocol mutation. Mockable dependencies: synthetic temporal sequences.
Consumers and feedforward	Module D consumes regime-aware IHARQ conservatism, trust-source restrictions, and active stop-loss status; Module F consumes temporal trust and risk state as policy context; Module G consumes regime, trust, stop-loss, and recovery state in simulated sessions; Module H consumes embodiment-gating recommendations and unsafe/no-action recommendations; Module I consumes RegimeStateRecord, conditional TrustStateRecord, regime_timelines.csv, trust_scores.csv, stop_loss_warnings.csv, Layer 5 EvaluationOutcomeRecord, matched-comparison/negative-control reports, and RegimeCard lineage.
Feedback and loopback	Permitted loopback to Module D: high regime risk, trusted-set shrinkage, or active stop-loss can increase conservatism, tighten evidence requirements within declared bounds, restrict trusted evidence sources, or trigger request/fallback/no-action/unsafe recommendations. Permitted feed to Module F: trust state and stop-loss state become policy context. Outputs must never rewrite Module A–C records or tune thresholds from held-out outcomes.
Quality attributes	Temporal inspectability, robustness under nonstationarity, conservative handling, traceability, and claim-safe wording.
Failure modes and defensive design	Risks include overclaiming real fatigue diagnosis, unstable regime labels, hindsight bias, confusing stress-induced artifacts with physiology, and hiding stop-loss triggers. Detection uses proxy-warning checks, temporal ablations, stress-stratified analyses, and dashboard timeline audits.

Evaluation evidence	Regime timelines; trust-shift plots; trust-score tables; trusted-set summaries; stop-loss event tables; drift-risk proxy summaries; risk-error alignment tables; threshold-adjustment logs; Layer 5 <code>EvaluationOutcomeRecord</code> exports; <code>RegimeRisk</code> ablations; shuffled/random-risk negative-control reports; matched-comparison reports; stress-conditioned temporal summaries; dashboard-ready regime tables; <code>RegimeCard</code> outputs; negative-result notes; and failure cards.
Thesis-facing contribution	Supports the claim that readiness reliability is monitored over time rather than only at isolated windows through saved regime-state, trust-state, timeline, trust-score, stop-loss, evaluation-outcome, matched-comparison, negative-control, and <code>RegimeCard</code> -lineage artifacts. It does not diagnose real fatigue or clinical state. Outputs appear in temporal trust chapters, stress analysis, dashboard timelines, <code>RegimeRisk</code> cards, limitation cards, claim-evidence entries, and reproducibility bundles.

17.6.6 Module F: Adaptive Policy Engine

Aspect	Specification
Identity and position	Module F is the Layer 6 learned readiness policy family. It includes rule baseline comparison, evidence-quality estimation, supervised Adaptive-IHARQ, learning-to-defer, selective-decision comparison, contextual bandit adaptation, and simulation-only reinforcement learning.
Problem addressed	Fixed IHARQ rules may be too rigid across subjects, models, stress states, and temporal regimes. Module F studies whether policy learning can improve readiness trade-offs under declared cost, reward, and deferral assumptions. Without it, Stretch C would not evaluate adaptive readiness beyond static rule logic.
Inputs	<code>ReadinessContextRecord</code> , <code>EvidenceQualityRecord</code> , <code>IHARQDecisionRecord</code> , <code>RegimeStateRecord</code> , calibrated uncertainty features, action-space configuration, legality rules, cost model, reward model, pseudo-labels, deferral targets, transition rewards from Module G where applicable, and optional embodiment outcomes from Module H. Test labels are not allowed for policy selection or reward tuning.
Outputs	<code>PolicyActionRecord</code> , <code>DeferralRecord</code> , updated <code>EvidenceQualityRecord</code> where appropriate, learned policy artifacts, controller cards, action distributions, reward/regret curves, matched-cost tables, matched-coverage tables, policy ablation tables, failure-case logs, and dashboard policy views.

Internal processing logic	Implement staged escalation: P0 rule-based IHARQ-lite baseline; P1 evidence-quality estimator; P2 supervised Adaptive-IHARQ; P3 learning-to-defer; P4 contextual bandit with immediate simulation rewards; P5 RL only after Module G is stable and replayable. Validate legality rules, freeze cost/reward versions, log action reasons, compare against matched baselines, and label all bandit/RL claims as simulation-only.
Interface contract	Policy records must include context IDs, source evidence IDs, regime IDs, policy ID, policy family, action-space version, cost/reward version, training split ID, validation split ID, seed, config hash, simulator ID if applicable, and reason/action codes.
Dependencies	Hard dependencies: Modules C–E for contexts and evidence; Module A/B identities for traceability. Soft dependencies: Module G for simulated rewards. Optional dependencies: Module H proxy outcomes. Forbidden dependencies: clinical labels, hidden reward changes, test-set tuning, dashboard edits, and direct mutation of decoder/calibration outputs. Mockable dependencies: toy readiness contexts and toy reward functions.
Consumers and feedforward	Module G consumes policy actions in simulated sessions; Module H consumes readiness-gated actions; Module I consumes policy artifacts and comparison tables.
Feedback and loopback	Receives rewards/regrets from Module G for bandit/RL experiments. May receive optional embodiment outcomes from Module H as simulation-only analysis. Must not feed learned policy outputs back into Module A/B/C protocol or calibration decisions.
Quality attributes	Comparability, staged complexity, cost transparency, deterministic replay, extensibility, failure-case auditability, and claim safety.
Failure modes and defensive design	Risks include treating pseudo-labels as clinical labels, reward hacking, simulator overfitting, unfair baseline comparisons, action illegality, and unstable RL. Detection uses matched-cost and matched-coverage tables, reward-version checks, held-out simulation evaluation, action-distribution audits, and controller cards.
Evaluation evidence	Adaptive-IHARQ ablations; learning-to-defer comparisons; selective-decision baselines; bandit reward/regret curves; RL vs non-RL baselines; policy failure logs; controller cards; dashboard policy views.
Thesis-facing contribution	Supports the claim that readiness policies can be compared and adapted under explicit, reproducible, simulation-bounded assumptions. It does not prove real-world therapeutic benefit or clinical decision-making. Outputs appear in policy-learning chapters, simulation-only evaluation sections, and dashboard controller cards.

17.6.7 Module G: Simulation and Stress Engine

Aspect	Specification
Identity and position	Module G combines Layer 7 simulated closed-loop readiness sessions with Layer 8 StressForge-Lite stress profiles. It is the simulation environment for repeated policy evaluation and stress-conditioned robustness analysis.
Problem addressed	Offline predictions do not expose delayed consequences, repeated decision burden, stress sensitivity, or policy behavior across simulated sessions. Without Module G, bandit/RL evaluation and stress-conditioned readiness claims would be unsupported.
Inputs	Clean records from Modules A–E, <code>PolicyActionRecord</code> , <code>ReadinessContextRecord</code> , stress-profile configs, stress schedules, stress seeds, reward configs, transition configs, simulator state configs, optional policy-update configuration for contextual-bandit/RL simulation runs, and optional embodiment-proxy settings.
Outputs	<code>TransitionRecord</code> , optional <code>PolicyUpdateTrace</code> files, disabled-update audit entries, <code>StressProfileRecord</code> , <code>StressScheduleRecord</code> , stress-conditioned metrics, reward/outcome logs, delayed-consequence summaries, replay manifests, sensitivity curves, stress cards, policy-update trace cards where applicable, and dashboard stress/simulation views.
Internal processing logic	Define state, action, transition, reward, stress schedule, delayed consequences, replay logic, and optional policy-update mode; instantiate clean or stressed sessions; inject only declared stress profiles; execute policy actions; record transitions and outcomes; when contextual-bandit/RL updates are enabled, emit one <code>PolicyUpdateTrace</code> per simulation-authorized policy mutation; when frozen-policy evaluation is active, record disabled-update status rather than mutating policy state; compute stress-conditioned metrics; export rewards/regrets and update traces to Module F only under declared policy-learning experiments.
Interface contract	Transition and stress records must include simulator-run ID, state ID, action ID, policy ID, stress-profile ID, stress family, intensity, target, injection point, schedule, seed, reward version, transition version, source record IDs, simulation-only claim tag, optional policy-update mode, and optional <code>TransitionRecord.policy_update_trace_id</code> linkage. Any emitted <code>PolicyUpdateTrace</code> must include policy ID, context/action/reward/next-state references, update step, exploration mode, training-or-evaluation mode, frozen-policy flag, simulator ID, seed, config hash, and simulation-only tag.
Dependencies	Hard dependencies: Modules A–F for clean records, contexts, decisions, and policy actions. Soft dependencies: stress libraries or synthetic perturbation tools. Optional dependencies: Module H embodiment adapter. Forbidden dependencies: hidden stress changes, post-test reward tuning, clinical outcome claims, and dashboard mutation. Mockable dependencies: deterministic toy simulator and fixed stress sequences.

Consumers and feedforward	Module F consumes rewards/regrets and optional <code>PolicyUpdateTrace</code> records for declared bandit/RL update analysis; Module E consumes stress/transition summaries for temporal summaries; Module H consumes simulated readiness-gated commands; Module I consumes transitions, optional policy-update traces, disabled-update audits, stress metrics, environment cards, policy-update trace cards, and plots.
Feedback and loopback	Permitted feedback to Module F is reward/regret under versioned simulation. Permitted feedback to Module E is transition/stress summaries. Simulation results must not alter Layer 1 protocols or be framed as real-world outcomes.
Quality attributes	Replayability, stress traceability, robustness evaluation, comparability, deterministic seeding, and claim safety.
Failure modes and defensive design	Risks include silently invented stress conditions, inconsistent seeds, reward hacking, overfitting to simulator dynamics, and conflating simulation reward with therapeutic outcome. Detection uses stress-profile registry checks, replay tests, reward-version audits, sensitivity analyses, and simulation-only warnings.
Evaluation evidence	End-to-end smoke tests; stress-conditioned curves; clean vs stressed comparisons; reward/outcome logs; replay validation; stress ablations; simulation cards.
Thesis-facing contribution	Supports simulation-bounded evaluation of readiness policies under declared closed-loop and stress conditions. It does not validate real device safety or therapy. Outputs appear in simulation and stress chapters, dashboard stress views, and limitation cards.

17.6.8 Module H: Embodiment Adapter Engine

Aspect	Specification
Identity and position	Module H is the bounded, Layer 9-owned MyoSuite/OpenSim/static-replay proxy embodiment bridge. It maps readiness-gated intent into declared simulated assistive-action commands through the integrated L9.1–L9.13 subsystem rather than through any independent P20 runtime namespace.
Problem addressed	The thesis needs a concrete downstream demonstration of readiness-gated intent without claiming real clinical control. Without Module H, embodiment remains abstract; with it, downstream consequences can be visualized, safety-gated, logged, exported, and inspected as simulation-only Layer 9 evidence.

Inputs	PolicyActionRecord, DeferralRecord, IHARQDecisionRecord, RegimeStateRecord, ReadinessContextRecord, TransitionRecord, StressProfileRecord, command-mapping config, readiness/safety-gate config, simulator adapter config, simulator state, embodiment mode, seed, and optional evaluation-only correctness/outcome helpers.
Outputs	EmbodimentStateRecord, EmbodimentOutcomeRecord, embodiment-linked TransitionRecord pointers, SafetyEventRecord or safety_event_log.csv, Layer9RewardTrace where enabled, command timeline, endpoint-error curve, adapter logs, outcome summaries, screenshots or plot manifests where appropriate, Layer9EmbodimentDemoManifest, dashboard-compatible export bundle, EmbodimentCard, and dashboard embodiment views generated downstream by Module I.
Internal processing logic	Validate incoming records through the Layer 9 record-intake validator; assemble the readiness context; map accepted or deferred intent to a declared proxy command; apply the readiness safety gate and safety state machine; select static replay, MyoSuite, or OpenSim replay adapter; inject declared StressForge context where configured; compute endpoint, command-success, burden, and optional reward terms; write transition and safety logs; package dashboard/card/export artifacts with non-clinical warnings.
Interface contract	Embodiment records and bundles must include source IHARQ, regime, policy, deferral, transition, stress, and readiness-context IDs; embodiment session/run ID; command-mapping version; adapter interface version; simulator or static-replay version; seed or nondeterminism explanation; readiness/safety-gate version; endpoint-metric version; reward-trace version where enabled; screenshot/plot/demo-manifest IDs; dashboard-export ID; and simulation-only/non-clinical claim warning.
Dependencies	Hard dependencies: Modules D–G for evidence decisions, policy actions, regime state, readiness context, transition context, and stress-conditioned simulation context. Soft dependencies: MyoSuite/OpenSim installation. Optional dependencies: static replay when full physics simulation is infeasible. Forbidden dependencies: real patient data, real FES control claims, real muscle activation inference, real exoskeleton safety claims, patient-specific biomechanics, medical-device readiness claims, dashboard mutation, and treating reward traces as clinical reward. Mockable dependencies: deterministic static command replay adapter.
Consumers and feedforward	Module I consumes Layer 9 dashboard-compatible exports, embodiment cards, Layer9EmbodimentDemoManifests, figures, endpoint-error curves, command timelines, safety-event logs, Layer9RewardTrace files where enabled, and warnings. Modules F and G may consume optional proxy outcomes only for declared simulation-only policy analysis or logged outcome summaries.

Feedback and loopback	Optional outcomes may feed Module F/G only as simulation-only evidence under declared experiment modes. Safety-event logs may block or mark later embodiment steps. They must not be used to claim clinical validation, tune public-data protocols, or alter frozen-policy evaluation.
Quality attributes	Claim safety, demonstrability, traceability, modular adapter design, simulator substitutability, deterministic replay where feasible, safety-gate auditability, dashboard-readiness, and failure transparency.
Failure modes and defensive design	Risks include overclaiming embodiment, simulator installation failure, stale simulator outputs, unsafe command mapping, readiness-gate bypass, mismatched action semantics, missing safety-event logs, missing <code>Layer9EmbodimentDemoManifests</code> , missing reward-trace provenance, and missing warnings. Detection uses record-intake validation, adapter tests, safety-gate logs, command-mapping checks, simulator-version checks, claim-warning checks, export-bundle completeness checks, and embodiment card validation.
Evaluation evidence	Record-intake validation tests; readiness-context checks; static replay tests; MyoSuite/OpenSim smoke tests where feasible; adapter interface tests; safety-event logs; command timeline plots; endpoint-error summaries; reward-trace decomposition where enabled; demo-manifest checks; dashboard-compatible export checks; embodiment cards with non-clinical warnings.
Thesis-facing contribution	Supports a bounded demonstration that readiness decisions can be routed to a simulated assistive-action proxy and inspected downstream. It does not support real clinical control, real FES, real exoskeleton deployment, medical-device readiness, patient-specific biomechanics, or patient validation. Outputs appear in the Layer 9 embodiment demonstration chapter and as Layer 10 read-only embodiment evidence pages/cards.

17.6.9 Module I: Dashboard and Reporting Engine

Aspect	Specification
Identity and position	Module I is the cross-layer observability, card-generation, thesis-export, and reproducibility engine for Layer 10. It reads all saved records and produces thesis-facing artifacts.
Problem addressed	A complex reliability architecture is not defensible if results cannot be inspected, traced, and reproduced. Without Module I, the thesis would contain disconnected plots and tables without source-record lineage.

Inputs	All saved records, manifests, configs, logs, cards, plots, failure cases, model registries, threshold registries, policy artifacts, stress profiles, Layer 9 embodiment bundles, optional <code>PolicyUpdateTrace</code> files, disabled-update audit entries, policy-update configuration metadata, claim-safety warnings, claim-evidence matrix entries, metric dictionaries, matched-comparison reports, negative-result notes, and Layer 1 ablation-readiness warnings. Layer 9-specific inputs include <code>EmbodimentStateRecord</code>, <code>EmbodimentOutcomeRecord</code>, embodiment-linked <code>TransitionRecord</code> pointers, <code>SafetyEventRecord</code> or safety-event logs, reward traces, command timelines, endpoint-error curves, <code>Layer9EmbodimentDemoManifests</code>, simulator-version metadata, dashboard-compatible exports, and embodiment evidence cards.
Outputs	<code>DashboardMetricRecord</code> with <code>policy_update_filter</code>, <code>embodiment_filter</code>, and <code>reward_trace_filter</code> where applicable; <code>DashboardViewRecord</code>; <code>CardRecord</code>; <code>ReproducibilityManifest</code>; figure manifests; table manifests; warning summaries; ablation summaries; claim-to-artifact maps; A0–A4 evaluation-slice lineage exports; failure-taxonomy exports; audit views; optional policy-update trace cards; Layer 9 embodiment evidence cards; embodiment export-bundle indexes; disabled-update audit tables; separated training/adaptation versus frozen-policy evaluation exports; thesis-ready plots; thesis-ready tables; and reproducibility packages.
Internal processing logic	Load saved records; validate schemas and provenance; validate optional <code>PolicyUpdateTrace</code> completeness when update-enabled runs are present; validate Layer 9 embodiment bundle completeness when embodiment pages are generated; keep simulation-training/adaptation curves separate from frozen-policy evaluation curves; keep <code>Layer9RewardTrace</code> files labeled as simulation-only; generate read-only views; aggregate cards; export figures/tables with source IDs; summarize warnings and limitations; build reproducibility package; block hidden recomputation unless a new explicit run ID and config version are created.
Interface contract	Every dashboard view must include source record IDs, artifact IDs, run IDs, config hashes, code version, generation timestamp, and claim-boundary warnings where relevant. Views that involve contextual-bandit/RL updates must also preserve <code>TransitionRecord.policy_update_trace_id</code> linkage where available, <code>DashboardMetricRecord.policy_update_filter</code>, disabled-update audit status, and a clear separation between simulation-training/adaptation and frozen-policy evaluation. Views that involve Layer 9 embodiment must preserve <code>DashboardMetricRecord.embodiment_filter</code>, <code>DashboardMetricRecord.reward_trace_filter</code> where applicable, safety-event-log linkage, demo-manifest linkage, simulator-version metadata, and explicit simulation-only/non-clinical warnings. Exported figures and tables must carry a manifest that traces back to records.

Dependencies	Hard dependencies: all upstream saved records and manifests. Soft dependencies: dashboard framework and plotting/reporting libraries. Optional dependencies: static HTML or notebook exports. Forbidden dependencies: silent mutation of experimental records, hidden metric recomputation, split rewriting, threshold retuning, policy reruns without versioning, and untracked manual figure edits. Mockable dependencies: small fixture record bundles.
Consumers and feedforward Feedback and loopback	Thesis author, examiner, implementer, dashboard user, and reproducibility reviewer consume Module I artifacts. Module I can report missingness, warnings, and failed validations to all modules. It cannot silently edit the source records. A rerun request must become an explicit versioned experiment.
Quality attributes	Observability, traceability, reproducibility, failure-case auditability, claim safety, and thesis defensibility.
Failure modes and defensive design	Risks include dashboard recomputation drift, cards inconsistent with records, missing provenance, cosmetic-only views, manual figure divergence, hidden policy updates, accidental mixing of simulation-training/adaptation curves with frozen-policy evaluation curves, incomplete Layer 9 embodiment bundles, missing safety-event logs, missing <code>Layer9EmbodimentDemoManifests</code> , stale simulator metadata, and overinterpreting reward traces as clinical reward. Detection uses export manifests, card-schema validation, source-record checksums, <code>PolicyUpdateTrace</code> completeness checks, disabled-update audit checks, Layer 9 embodiment bundle checks, reward-trace filter checks, and warning summaries.
Evaluation evidence	Dashboard smoke tests; record-loader tests; card completeness checks; figure/table manifest checks; policy-update trace-card checks where applicable; frozen/update curve-separation checks; Layer 9 embodiment page/export-bundle checks; safety-event-log/demo-manifest checks; reproducibility package validation; end-to-end thesis export test.
Thesis-facing contribution	Supports the claim that the architecture is inspectable and reproducible. It does not prove that upstream scientific choices are optimal. Outputs appear throughout the thesis as figures, tables, cards, appendices, and reproduction instructions.

17.7 Cross-Layer Interface Contract Matrix

Every record must contain enough identity fields to trace any plot, card, table, warning, or failure case back to dataset, subject, session, run, trial, window, model, split, policy, stress profile, simulator run, config, seed, and code version where applicable.

Record	Producer	Consumers	Required identifiers	Versioning	Risk and dashboard visibility
--------	----------	-----------	----------------------	------------	-------------------------------

DatasetRecord	A	B-I	dataset, subject, session, run, protocol, source fingerprint	Dataset/protocol version	leakage if changed after downstream use; visible in dataset pages.
WindowRecord	A	B-I	dataset, subject, session, trial, window, label-map, preprocessing	Windowing and preprocessing version	Window drift affects all metrics; visible in protocol drilldown.
SplitRecord	A	B-I	split, subject/session partition, budget, seed	Split version immutable	Leakage risk; visible in split-integrity views.
PreprocessingRecord	A	B-I	preprocessing pipeline, parameters, fit scope, split scope	Preprocessing version	Hidden global normalization risk; visible in protocol cards.
LabelMapRecord	A	B-I	label ontology, dataset label, mapped label	Label-map version	Inconsistent labels invalidate comparisons; visible in dataset cards.
ArtifactFlagRecord	A	B-I	artifact rule, trial/window, flag, reason	Artifact-rule version	Performance-dependent rejection risk; visible in validation reports.
Layer1Manifest	A	B-I	protocol, run, validation, limitation IDs, ablation-readiness status	Manifest version	Reproducibility and matchability risk; visible in protocol pages.
layer1_ablation_readiness_11_B-I, json	A	B-I, json	dataset, subject, session, run, trial, window, split, preprocessing, label-map, budget, seed, protocol	Manifest/artifact version	Unmatched comparisons become diagnostic-only; visible in Layer 10 ablation-readiness warnings.

PredictionRecord B	C–I, with H/Layer 9 limited to provenance/readiness input compatibility	run, model, split, split_role , budget, window/trial, timestamp_or_order , predicted_class , probability_vector /logits/scores, score_type , class_order , evaluation-only label_true , seed, A0/A4 comparison key fields	Prediction schema and model/run version	Score misuse, hidden alias, class-order mismatch, unmatched-baseline, and replay-lineage risk; visible in decoder pages, A0/A4 ablation views, simulated replay provenance, and dashboard lineage.
ModelRegistryRecord B	C–I where model identity, score schema, replay, or dashboard lineage is required	model, decoder, family, artifact path, training split, split_role , label_map_version , score_type , class_order , seed, config/code version, optional ensemble membership	Registry schema version and model_registry_version	Missing or stale model identity in dashboard; calibration, replay, and dashboard traceability; visible in model cards and reproducibility manifests.
BaselineMetricRecord B	I, evaluation scripts, ablation/reporting sections	model, dataset, subject/session where applicable, split role, label budget, metric dictionary entry, seed/CI summary, matched-comparison key	Metric and baseline-run version	Unmatched or split-ambiguous baselines become diagnostic-only; visible in baseline tables and A0/A4 ablation views.

LowCalibrationCurveRecord	C diagnostic analysis where needed, I, evaluation/reporting sections	model, dataset, label budget, split role, aggregation level, seed summary, comparison key, curve version	Low-calibration protocol version	Budget or split mismatch weakens low-calibration claims; visible in low-calibration curves and diagnostic reports.
SubjectProfileRecord	E, F where allowed, I	subject/session, baseline performance, difficulty metrics, low-label summaries, aggregation version, source prediction/metric IDs	Subject-profile schema version	Split-inconsistent aggregation risks false heterogeneity claims; visible in subject-difficulty and policy-context diagnostics.
CalibrationRecord	D–I	calibrator, model, calibration split, config hash	Calibrator version	Test-tuning risk; visible in reliability pages.
UncertaintyRecord	D–I	uncertainty feature set, model, window, calibration ID	Metric version	Feature mismatch risk; visible in uncertainty views.
SelectivePredictionRecord	D/F/I	threshold, coverage, accept/reject state, validation split, matched operating-point ID, A2/A3 comparison ID	Threshold version	Threshold discipline and threshold-sufficiency overclaiming risk; visible in risk-coverage and matched-threshold views.
Layer3MatchedOperatingPointTable	D, E, I, evaluation scripts	matched coverage/cost, source prediction/calibration IDs, threshold registry ID, split role, metric dictionary ID	Table/artifact version	Validates A1–A3 fairness; visible in Layer 10 calibration/selection prediction ablation views.

ThresholdRegistryRecord	D/F/I	threshold ID, metric, selection split, selection criterion	Registry version	Test-set lockout risk; visible in audit reports.
EvidenceQualityRecord	E/F/G/H/I	evidence score, feature version, source predictions, decision context	Evidence model/rule version	Overfitting or opaque scoring risk; visible in IHARQ and controller cards.
IHARQDecisionRecord	E/F/G/H/I	action code, action-vocabulary version, reason-code hierarchy, source evidence IDs, threshold-registry ID, optional regime ID, re- quest/combine/fallback/unsafe status where applicable, matched- comparison key where evaluated, claim-boundary tag	IHARQ config version	Threshold- only collapse, action-token drift, missing reason-code, or hidden un- safe/fallback state risk; visible in decision-flow, IHARQ trace, and reliability- card views.
CombinedEvidenceRecord	D/F/G/H/I, evaluation scripts	source window/source IDs, combination method, before/after confidence, entropy, and margin, combined class or vote summary, eligibility reason, final decision pointer, delay/evidence cost, config hash, matched- comparison key where evaluated	Layer 4 evidence- combination schema version	Unsupported evidence fusion or unlogged delay-cost risk; visible in combine-log views, IHARQ traces, and ablation tables.

FallbackDecisionRecord	F/G/H/I, failure taxonomy, evaluation scripts	fallback route, reason, target, availability, cost, selected fallback state, evaluation outcome where available, source decision/evidence IDs, compatibility mode, claim boundary	Layer 4 fallback schema version	Hidden fallback, unavailable fallback target, or incomparable fallback-only baseline risk; visible in fallback-log views and defensive-design cards.
UnsafeEventRecord	G/H/I, failure taxonomy, thesis safety discussion	unsafe trigger, unsafe reason, blocked action, source evidence/decision IDs, RegimeRisk context, stress context, recommended conservative outcome, severity, simulation/proxy boundary label, warning text	Layer 4 unsafe-event schema version	Missing unsafe-event provenance or clinical-safety overclaim risk; visible in unsafe-action logs, warning panels, and limitation cards.
IHARQAblationRecord	I, evaluation scripts, ablation/reporting sections	local/global ablation ID, ablated variant, source decision/evidence IDs, decisions and metrics, matched-comparison key, metric-dictionary ID, diagnostic-only flag, negative-result note ID, claim-evidence pointer	Layer 4 ablation schema version	Unmatched or undocumented IHARQ ablations become diagnostic-only; visible in A5–A7 ablation views and thesis tables.

IHARQTraceRecord	D	I, reproducibility package, debugging workflows, thesis figures/tables	rule path, thresholds, feature values, reason-code hierarchy, input file paths/hashes, configuration hash, metric-dictionary ID, dashboard trace fields, figure/table lineage, reproducibility pointers	Layer 4 trace schema version	Non-reproducible decision path or missing traceability risk; visible in IHARQ trace drilldowns and reproducibility manifests.
Layer 4 EvaluationOutcomeRecord	D and Record condition scripts	I, ablation/reporting sections, thesis tables	dataset, subject/session, model, threshold or operating point, ablation step, matched-comparison key, metric-dictionary ID, source IHARQ record IDs, seed/CI summary, diagnostic-only flag, negative-result note ID	Layer 4 evaluation-outcome schema version	Unsupported evidence-verification claim or split-ambiguous metric risk; visible in IHARQ evaluation tables, matched-comparison reports, and claim-evidence matrices.
RegimeStateRecord	E	D/F/G/H/I	subject/session/run, temporal horizon, replay-index range, risk score, trust-score summary, optional trust-state links, stop-loss status, regime reason codes, source IHARQ/prediction/uncertainty records, stress/simulator/proxy tags where present, claim-boundary tag	Regime config version	Fatigue-overclaim, temporal-leakage, or unlinked regime-state risk; visible in timelines, RegimeCards, stop-loss views, and reproducibility manifests.

TrustStateRecord E	F/I and optional D/G/H via linked regime context	source regime ID, trust target type/ID, trust score, trust trend/band, trusted-set membership, update source, decay/recovery state, source records, reason codes, provenance	Trust-state schema version	Dangling trust-timeline, unidentified trust target, or unsupported trusted-set restriction risk; visible in trust-state plots, policy context checks, RegimeCards, and reproducibility manifests when independent trust timelines are exported.
regime_timelines.Esv	I, RegimeCard, thesis figures, matched-comparison reports, and reproducibility packages	time index or replay index, regime label, risk score, risk trend, recovery marker, transition marker, source record IDs, stress/session references where applicable, config hash, claim-boundary tag	Regime-timeline artifact version	Temporal-ordering, stale timeline, or unlinked source-record risk; visible in regime timeline views and temporal-failure audits.
trust_scores.csv E	D/F/I and optional G/H via linked regime context, RegimeCard, thesis temporal-trust summaries	trust target type/ID, score, trend, band, trusted-set membership, update source, decay/recovery state, source RegimeStateRecord or TrustStateRecord IDs, reason codes, provenance	Trust-score artifact version	Unidentified trust allocation or unsupported source restriction risk; visible in trust-state views, policy context checks, and RegimeCards.

stop_loss_warningE.csv		D/F/G/H/I, failure taxonomy, claim-safety reporting, RegimeCard	warning status, trigger, severity, affected source IDs, recommendation, cooldown/recovery state, timestamp or simulated step, simulation/proxy boundary label, source evidence IDs	Stop-loss warning schema version	Missing conservative-action route or hidden stop-loss trigger risk; visible in stop-loss views, warning panels, and RegimeCards .
Layer 5 EvaluationOutcomeRecord	E and Record scripts	I, ablation/reporting sections, thesis tables	dataset, subject/session, horizon, regime configuration, trust configuration where applicable, stop-loss setting, ablation or negative-control ID, matched-comparison key, metric-dictionary ID, source regime/trust records, seed/CI summary, diagnostic-only flag, negative-result note ID	Layer 5 evaluation-outcome schema version	Temporal-trust, shuffled-control, or risk-error-alignment claim becomes unsupported without matched evaluation lineage; visible in RegimeRisk evaluation tables and claim-evidence matrices.
RegimeCard lineage	E produces validated source lineage; I renders/exports read-only	Thesis/dashboard users, reproducibility package	source RegimeStateRecord IDs, optional TrustStateRecord IDs, timeline/trust/stop-loss artifact IDs, matched-comparison and negative-control IDs where applicable, limitations, warning tags, config hashes, export IDs	RegimeCard lineage/schema version	RegimeRisk card overclaim or missing limitation risk; visible in card index, dashboard cards/export page, and thesis limitation tables.

ReadinessContextRecord	D/E	F/G/I	context features, source evidence, regime, stress, legality state	Context schema version	Hidden feature leakage risk; visible in policy views.
DeferralRecord	F	G/H/I	deferral target, reason, cost, source context	Policy/cost version	Deferral target ambiguity risk; visible in controller cards.
PolicyActionRecord	F	G/H/I	policy, context, action, reason, cost/reward version	Policy version	Simulation-only policy risk; visible in action distributions.
PolicyUpdateTrace	F/G	F/G/I	policy, context, action, reward, next-state, update step, update mode, exploration mode, frozen/evaluation flag, source transition	Policy-update schema version	Hidden policy mutation risk; visible in update-trace cards and frozen/update separation views.
TransitionRecord	G	E/F/H/I	simulator run, state, action, reward, next state, seed, optional policy update mode, optional policy update trace ID	Transition/reward version	Reward hacking or unlogged update risk; visible in simulation views and policy-update trace cards.
StressProfileRecord	F	E/F/H/I	stress family, target, intensity, injection point, schedule linkage, seed	Stress-profile version	Silent stress drift risk; visible in stress cards.
StressScheduleRecord	G	G/H/I	schedule ID, parent stress profile ID, schedule type (clean/no-stress, constant, ramp, burst, periodic, stochastic, or scenario-scripted), per-step/per-window values, active component stressors, seed	Stress-schedule version	Unmatched schedule risk; visible in schedule inspection, stress cards, and reproducibility manifests.

EmbodimentStateRecord	F/G/I	simulator state, embodiment session, command mapping, readiness/safety gate, endpoint metrics, stress context, warning	Adapter/simulator replay version	Clinical-overclaim risk; visible with non-clinical warning and Layer 9 provenance.
SafetyEventRecord or safety_event_log.csv	H/I/F/G where declared	safety gate, blocked command, unsafe/fallback reason, source policy/IHARQ/regime/stress IDs, response taken	Safety-gate/log schema version	Bypassed or missing safety control risk; visible in embodiment pages, warning panels, and failure taxonomy.
Layer9EmbodimentDemoManifest	I/thesis exports	embodiment run ID, adapter branch, simulator/static-replay version, command timeline, endpoint-error curve, safety logs, reward-trace pointer where enabled, export bundle path	Demo-manifest version	Incomplete or stale embodiment bundle risk; visible in embodiment card and reproducibility package.
DashboardMetricRecord	Dashboard pages, DashboardViewRecords, thesis figures, cards, export manifests, and final demonstration evidence	metric, source records, aggregation config, split/stress/policy/embodiment filters, optional policy update filter, optional reward trace filter, A0–A13/local ablation IDs, matched-comparison ID, metric-dictionary ID, diagnostic-only status, negative-result note ID, claim-evidence ID, warning flags, and export references	Metric/export version	Mixed training/evaluation, hidden aggregation, missing matched-comparison or metric-dictionary linkage, unsupported diagnostic-to-claim promotion, or embodiment/reward overclaim risk; visible in metric views and export manifests.

DashboardViewRecord		Thesis/dashboard users, static exports, cards/export page, reproducibility manifest, and final thesis artifact inventory	page/view ID, panel IDs, source IDs, source-record IDs, filter state, evaluation-lineage references, warning banners, completeness state, diagnostic-only state, figure/table IDs, export references, and generation config	View/export version	Recompute drift, hidden filtering, incomplete lineage, or unsupported claim-ready display risk; visible through serialized view records and export manifests.
CardRecord	I and source modules	Thesis/dashboard users	card type, source records, limitations, warnings	Card schema version	Missing limitations risk; visible in card index.
ReproducibilityManifest		Future maintainer/reviewer	run IDs, configs, code, environment, seeds, artifacts	Manifest version	Non-reproducibility risk; exported package root.

17.8 Feedback Loops and Permitted Cross-Layer Interactions

Feedback is permitted only when it has a scientific and engineering reason, is represented as a record, and does not violate split discipline or claim-safety boundaries.

Source	Target	Signal	Trigger	Action	Safety/evaluation rule
E	D	Regime-aware conservatism, trust score, stop-loss status	High temporal risk, drawdown, volatility, trusted-set shrinkage	IHARQ increases conservatism, requests evidence, falls back, or emits unsafe	Must be reason-coded and compared against non-regime IHARQ.
E	F	Temporal trust as policy context	Policy context construction	Policy conditions actions on regime state	Must not tune policy on test outcomes.
F	G	Policy actions	Simulated session step	Environment executes action and logs transition	Policy ID and cost/reward version required.

G	F	Rewards, regrets, transitions, and optional PolicyUpdateTrace entries	Bandit/RL experiment only	Policy update, offline comparison, or frozen-policy evaluation	Simulation-only; reward version frozen; policy mutation allowed only in declared simulation-update mode and must be traceable.
G	E	Stress/transition summaries	Simulated run completed or replayed	Temporal summaries include stress/transition context	Must be labeled simulated.
H	F/G/I	Optional proxy embodiment outcomes, safety-event logs, Layer9RewardTrace files where enabled, command timelines, endpoint-error curves, and Layer9EmbodimentDemoManifests	Layer 9 embodiment demo enabled and readiness/safety gate satisfied or blocked	Simulation-only policy analysis, outcome logging, dashboard observability, or explicit block/warning response	Must not support clinical validation, public-protocol mutation, hidden policy updates, or dashboard-side simulator control.
I	All	Warnings, missingness, validation failures	Dashboard/report generation	Human-readable report or explicit rerun request	Read-only; no silent mutation.

Forbidden feedback paths are equally important:

- Decoder performance must not alter Layer 1 splits, windows, label maps, or preprocessing without a new protocol version.
- Test-set outcomes must not tune calibration, thresholds, IHARQ rules, policy actions, reward weights, stress profiles, or embodiment gates.
- Dashboard interactions must not rewrite source records, regenerate hidden metrics, or silently relabel artifacts.
- Embodiment results must not be used to claim clinical validation, real muscle activation inference, real FES behavior, or exoskeleton safety.
- Simulation rewards must not be described as real-world therapeutic outcomes.

17.9 Runtime Scenarios

Scenario	Trigger	Records consumed	Records produced and checks	Dashboard outputs
Offline low-calibration benchmark run	New fixed protocol and decoder config	A records, model configs, budget configs	B/C records, baseline tables, calibration records, split-integrity checks	Decoder tables, low-label curves, reliability diagrams.
IHARQ readiness decision run	Calibrated predictions available	PredictionRecord, CalibrationRecord, UncertaintyRecord, thresholds	EvidenceQualityRecord, IHARQDecisionRecord, reason-code and threshold-vs-IHARQ checks	IHARQ decision flow, action distributions, failure cases.
Regime-aware readiness run	Temporal sequence or session replay available	B/C/D histories, optional stress summaries	RegimeStateRecord, conditional TrustStateRecord, regime-aware D records; stop-loss, trust-lineage, and proxy-warning checks	Regime timelines, trust-state plots, conservative-action summaries.
Adaptive-IHARQ supervised policy run	Readiness contexts and pseudo-label rules frozen	C/D/E records, cost model, pseudo-label config	F policy artifacts, PolicyActionRecord, matched-cost checks	Policy tables, action distributions, controller card.
Learning-to-defer comparison	Deferral targets and cost model declared	C/D/E context, fallback options, deferral labels	DeferralRecord, matched coverage/cost tables	Deferral curves, burden summaries, failure cases.

Contextual bandit or RL simulation run	Replayable Module G environment validated	A–F records, reward config, seeds, simulator config, optional policy-update config	TransitionRecord , rewards/regrets, optional PolicyUpdateTrace , disabled-update audits, simulation-only checks, frozen/update separation checks	Reward/regret plots, unsafe-event summaries, simulation card, policy-update trace card where applicable, separated training/adaptation and frozen-evaluation curves.
StressForge-Lite stress-conditioned run	Declared stress profile and stress schedule selected	Clean records, stress config, seeds, policy/decision records	StressProfileRecord , StressScheduleRecord , stressed transitions, sensitivity curves; stress-profile and schedule-registry checks	Stress records, stress-conditioned metrics, sensitivity figures.
MyoSuite/OpenSim/replay embodiment demo run	Readiness-gated simulated command enabled or intentionally blocked by Layer 9 safety gate	D/E/F/G records, ReadinessContextRecord , command mapping, safety-gate config, simulator/static-replay state, optional stress profile and stress schedule	EmbodimentStateRecord , EmbodimentOutcomeRecord , embodiment-linked TransitionRecord pointers, SafetyEventRecord or safety-event log, Layer9RewardTrace where enabled, command timeline, endpoint-error curve, Layer9EmbodimentDashboard compatible export; non-clinical warnings	Layer 9 embodiment card, command timeline, endpoint plots, unsafe-event summary, demo-manifest index, and Manifest , read-only embodiment page.

End-to-end thesis export run	Thesis export requested	All saved records, manifests, cards, logs	DashboardViewRecord, CardRecord, ReproducibilityManifest; source-record checksums	Figures, tables, Manifest; failure taxonomy, reproducibility package.
------------------------------	-------------------------	---	---	---

17.10 Cross-Layer Quality Attribute Scenarios

Quality attribute	Source	Stimulus	Environment/artifact	Response	Measure
Reproducibility	Future maintainer	Rerun a saved experiment	Same records/configs/seeds	Recreate equivalent records or report nondeterminism	Manifest match or explained delta.
Leakage prevention	Experiment designer	Attempt threshold tuning on test split	Module C/F configs	Validation blocks run and emits warning	Zero accepted leakage runs.
Traceability	Thesis examiner	Inspect a figure	Exported plot	Figure manifest traces to source records	Complete record lineage.
Threshold discipline	Reliability researcher	Add new threshold	Threshold registry	Requires selection split and version	Registry completeness.
Deterministic replay	Implementer	Replay simulated session	Module G	Same stress schedule and transitions where deterministic	Seeded replay match.
Robustness under stress	Reliability researcher	Apply declared stress profile	Module G records	Stress-conditioned metrics and sensitivity curves produced	Complete stress cards.
Dashboard read-only behavior	Dashboard user	Change view filters, including <code>policy_update_filter</code> where present	Module I	View changes only; source records unchanged, policy-update traces not mutated, and training/adaptation curves not merged into frozen-policy evaluation	Checksums unchanged and frozen/update separation checks pass.

Simulator claim safety	Examiner	Read simulation/RL output	Modules G/H/I	Simulation-only warnings visible	Warning coverage 100%.
Module extensibility	Implementer	Add a decoder or policy	Modules B/F	New registry entry without schema break	Existing tests pass.
Failure-case auditability	Examiner	Ask why a wrong accept occurred	D/E/F/I records	Reason codes and source evidence shown	Complete failure card.
Configuration versioning	Future maintainer	Modify reward or stress config	Modules F/G	New version/hash and run ID required	No over-written runs.
Ablation comparability	Thesis reviewer	Compare IHARQ vs threshold	Modules C/D/F/I	Matched split/cost/coverage table generated	Matched-comparison present.

17.11 Cross-Layer Failure Modes and Defensive Design

Failure mode	Affected modules	Detection method	Prevention mechanism	Required warning/card entry	Thesis implication
Protocol leakage	A–I	Split-integrity tests and protocol manifests	Immutable protocol versions and test lockout	Protocol leakage warning	Invalidates affected benchmark claims.
Hidden global normalization	A/B/C	Preprocessing fit-scope audit	Split-scoped preprocessing records	Preprocessing warning	Threatens low-calibration validity.
Split drift across modules	A–F/I	Split ID mismatch checks	Shared SplitRecord references	Split drift card	Invalidates comparisons.
Inconsistent label mapping	A/B/C	Label-map validation	Single LabelMapRecord owner	Label-map warning	Invalidates class-level results.
Missing provenance	All	Manifest completeness check	Required IDs and config hashes	Missing provenance card	Weakens reproducibility.
Threshold tuned on test data	C/D/F	Threshold-selection split audit	Registry requires selection split	Threshold leakage warning	Invalidates reliability/selection claims.
IHARQ reduced to simple thresholding	D	Reason-code and ablation audit	Multi-feature IHARQ rules and threshold baseline	IHARQ simplification card	Weakens novelty claim.
RegimeRisk overclaimed as fatigue diagnosis	E/I	Claim-boundary text check	Proxy-only labels and warnings	RegimeRisk non-diagnosis warning	Prevents clinical overclaim.

Policy labels treated as clinical labels	F/I	Controller-card audit	Pseudo-label and cost-model versioning	Policy-label limitation card	Prevents clinical-policy overclaim.
RL overfitting simulator reward	F/G	Held-out simulation and reward audit	Frozen reward versions, baseline comparisons, and explicit separation of training/adaptation from frozen evaluation	RL overfit warning	Limits RL claims to simulation.
Unlogged or mixed policy updates	F/G/I	PolicyUpdateTrace completeness check, disabled-update audit, and dashboard curve-separation check	Policy mutation only in declared simulation-update mode; frozen evaluation must remain immutable	Policy-update trace warning	Invalidates adaptive-learning or frozen-evaluation claims until rerun or relabeled.
Stress profiles silently changed	G/I	Stress-profile hash mismatch	Versioned stress registry	Stress drift warning	Invalidates stress comparisons.
Embodiment demo overclaimed or incomplete	H/I	Claim-warning validation, safety-event-log presence check, demo-manifest completeness check, simulator-version check, reward-trace warning check	Simulation-only embodiment cards, blocked incomplete export bundles, and non-clinical reward/embodiment warnings	Non-clinical embodiment warning and incomplete-bundle warning	Protects thesis scope and prevents unsupported Layer 9 embodiment evidence from reaching final exports.
Dashboard recomputation drift	I	Source checksum and export manifest checks	Read-only dashboard design	Recompute drift warning	Protects figure integrity.
Untracked random seeds	B/F/G/H	Seed-field validation	Required seed or nondeterminism explanation	Seed missing warning	Weakens reproducibility.
Cards inconsistent with records	I	Card-record checksum checks	Card generation from saved records only	Card inconsistency warning	Weakens thesis evidence.

17.12 Cross-Layer Evaluation and Ablation Plan

The cross-layer module architecture supports evaluation at module, integration, and thesis-claim levels. Every major claim should be backed by a table, plot, card, manifest, ablation, or failure-case analysis.

Evaluation layer	Required checks	Thesis evidence produced
------------------	-----------------	--------------------------

Module-level tests	Schema validation, config parsing, seed handling, accepted failure states, toy data tests, comparison-key completeness tests, and claim-evidence gate tests	Test summary table and implementation appendix.
Integration tests	A→B→C smoke test; C→D IHARQ run; D/E→F policy context; F→G simulation; G→H embodiment; all→I dashboard export	Integration-test manifest and dashboard smoke-test card.
Record schema validation	Required identifiers, version fields, config hashes, provenance fields, claim-boundary tags, ablation-step IDs where applicable, metric dictionary IDs, matched-comparison IDs, and diagnostic-only labels	Record contract table, claim-evidence matrix, and reproducibility manifest.
Protocol lock tests	Split lockout, preprocessing scope, label-map immutability, downstream mutation blocks	Protocol integrity report.
Calibration and uncertainty audits	Reliability diagrams, ECE/Brier/NLL where valid, A1 calibration deltas, A2 threshold comparisons, A3 risk-coverage curves, matched operating-point tables, negative-result notes, and high-confidence wrong audit	Calibration figures, reliability audit tables, matched-threshold views, and negative-result notes.
IHARQ comparisons	IHARQ vs confidence threshold, IHARQ vs selective prediction, IHARQ with/without agreement/stability/regime context	IHARQ ablation table and reason-code figures.
RegimeRisk ablations	With/without temporal trust, with/without stop-loss, stress-stratified regime summaries	Regime timelines and temporal trust ablation.
Adaptive-IHARQ ablations	Rule baseline, evidence-quality estimator, supervised policy, learning-to-defer, matched-cost and matched-coverage comparisons	Policy ablation ladder and controller cards.

Bandit/RL simulation-only comparisons	Bandit/RL vs non-learning policies under frozen rewards and held-out simulated sessions, with update-enabled traces separated from frozen-policy evaluation	Reward/regret plots, PolicyUpdateTrace completeness summaries where applicable, disabled-update audits, and simulation-only limitations.
Stress-conditioned evaluations	Clean vs declared stress profiles, sensitivity curves, stress family/intensity analysis	StressForge cards and stress-conditioned tables.
Embodiment checks	Layer 9 record-intake validation, readiness-context assembly checks, static replay, adapter smoke tests, safety-event logs, endpoint metrics, command timelines, endpoint-error curves, demo-manifest checks, reward-trace checks where enabled, claim-warning checks	Embodiment demonstration figures, non-clinical cards, unsafe-event summaries, command-timeline plots, and reproducibility-visible Layer 9 export bundles.
Dashboard/reproducibility validation	Record-loader tests, export manifests, figure/table source lineage, card consistency, reproducibility package completeness	Thesis export package and reproducibility appendix.

17.13 Implementation Artifact Map

Module	Source folders	Configs	Records/artifacts	Tests and dashboard views
A	src/data_protocol, src/dataset_adapters	configs/datasets, configs/protocols	records/dataset_records, records/window_records, manifests/layer1, manifests/layer1_adapter_readiness	Dataset adapter tests, split tests, comparison-key adapter_readiness, protocol-card and ablation-readiness views.
B	src/decoders, src/baselines	configs/decoders, configs/budgets	records/prediction_models, model registry, A0/A4 baseline-control summaries	Decoder smoke tests, low-label dashboard, baseline tables, A0 raw-decoder and A4 ordinary-combination views.

C	src/calibration_uncertainty	configs/calibration_records/calibration_records, tests, configs/thresholds, threshold registry, split-lock tests, reliability plots, matched threshold tests, reliability operating-point dashboard. tables, A1–A3 summaries, negative-result notes
D	src/iharq	configs/iharq records/iharq_records, IHARQ rule tests, evidence records, decision-flow reason-code logs dashboard.
E	src/regime_risk	configs/regime_risk records/regime_state, Temporal display conditional tests, trust-state records/trust_state, Invariants, trust_scores.csv, stop-loss tests, stop_loss_warnings, regime dashboard, and regime and trust-state timelines views.
F	src/adaptive_policy	configs/policies, records/policy_records, Policy tests, configs/rewards policy artifacts, deferral tests, controller cards reward-version tests, policy dashboard.
G	src/simulation, src/stressforge	configs/stressforge, records/transition_records, Simulink replay configs/simulation, records/stress_records, stress cards, reward logs stress dashboard.
H	src/layer9_embodiment	configs/layer9_embodiment, Record-intake command-mapping configs, state, outcome, tests, adapter safety-gate embodiment- static-replay tests, configs, linked transition, safety-gate tests, simulator/static-safety-event-log, demo-manifest replay configs reward-trace, checks, command-timeline, embodiment endpoint-error-dashboards-export curve, checks, demo-manifest, adapter-log, and dashboard-compatible export artifacts

I	src/dashboard, src/reporting_cards, dashboard_app	configs/dashboard, export configs	records/dashboard_ cards, figures, thesis_tables, logs, reproducibility manifests, optional policy-update trace cards/exports, Layer 9 embodiment evidence cards/export indexes	Dashboard records, tests, card validation, export-manifest checks, policy-update trace-card checks, frozen/update separation checks, Layer 9 embodiment page and export-bundle checks.
---	---	--------------------------------------	--	--

17.14 Layers 4–7 Evaluation-Lineage Propagation Addendum

The MERGE_RECORD_22 Layers 4–7 merge creates a real cross-layer consistency obligation because the evaluation evidence for IHARQ, RegimeRisk, adaptive readiness policies, and simulated closed-loop policies is now locally owned by Modules D–G while still participating in the global A0–A13 ladder. This addendum does not create a new runtime module and does not replace the Ablation and Evaluation Architecture. It strengthens the existing Module D, E, F, G, and I contracts so that their evaluation outputs remain explicit, matchable, dashboard-visible, reproducible, and claim-bounded.

Module	Local evaluation ownership	Required propagated artifacts	Claim-safety rule
D / Layer 4	A5 IHARQ-lite; participation in A6 evidence-quality and A7 RegimeRisk-aware IHARQ	Layer 4 EvaluationOutcomeRecord IHARQ matched-comparison report, metric-dictionary entry, negative-result note where applicable, dashboard/export lineage, and claim-evidence entry	If IHARQ is not matched against threshold, selective-prediction, or ordinary controls, the result is visible but diagnostic-only.
E / Layer 5	A7 temporal-trust and regime-aware versus regime-unaware evaluation	Layer 5 EvaluationOutcomeRecord matched RegimeRisk report, shuffled/random risk negative-control report, metric-dictionary entry, negative-result note, and claim-evidence entry	RegimeRisk supports temporal trust characterization only; shuffled-control failure or unmatched comparison blocks improvement claims.

F / Layer 6	A6 evidence-quality, A8 learning-to-defer, A9 supervised Adaptive-IHARQ, A10 contextual bandit, A11 RL, and A12/A13 participation	Layer 6 EvaluationOutcomeRecord matched cost/coverage/reward/stress reports, training/adaptation versus frozen-policy separation evidence, controller-card lineage, metric dictionary, negative-result note, and claim-evidence entry	Policy claims remain computational, proxy-label, and simulation-bounded; unmatched or reward-sensitive rankings are diagnostic-only unless explicitly bounded.
G / Layer 7	A10/A11 simulation-policy held-out evaluation and A12/A13 participation	Layer 7 EvaluationOutcomeRecord frozen-policy held-out reports, update-mode separation records, metric-dictionary snapshots, matched-comparison keys, negative-result notes, diagnostic-only labels, and claim-evidence entries	Frozen held-out simulation is the claim-supporting mode; update-enabled adaptation curves are explanatory unless separately frozen, held out, and matched.
I / Layer 10	Read-only dashboard, card, and export visibility for the above artifacts	Matched-comparison views, negative-result views, metric-dictionary exports, claim-evidence views, and diagnostic-only warnings	Module I may inspect, filter, export, and card these artifacts, but it must not repair missing matches by hidden recomputation.

All Layers 4–7 evaluation artifacts that participate in A5–A11 or in Layers 4–7-supported A12–A13 evidence must carry: global ablation ID, local layer ablation ID, source-record IDs, matched-comparison key, metric-dictionary ID or snapshot, dashboard/card/export lineage, negative-result-note pointer where applicable, claim-boundary label, and diagnostic-only flag when any required match fails. These fields extend the cross-layer record contract without changing the runtime ownership of the underlying IHARQ, RegimeRisk, policy, or simulation records.

The existing cross-layer consistency checklist is therefore strengthened as follows: every Layers 4–7 evaluation-slice artifact must carry its global/local ablation IDs, matched-comparison key, metric-dictionary ID or snapshot, dashboard/card/export lineage, negative-result note where applicable, claim-boundary label, and diagnostic-only flag when matching fails. This requirement is substantive because it governs claim eligibility, not merely wording.

17.15 Layers 8–10 Evaluation-Lineage Propagation Addendum

The MERGE_RECORD_22 Layers 8–10 propagation creates a real cross-layer consistency obligation because the evaluation evidence for StressForge robustness and Layer 9 embodiment-proxy consequences is now locally owned by Modules G and H while still participating in the global A0–A13 ladder. This addendum does not create a new runtime module and does not

replace the Ablation and Evaluation Architecture. It strengthens the existing Module G, H, and I contracts so that their evaluation outputs remain explicit, matchable, dashboard-visible, reproducible, and claim-bounded.

Module	Local evaluation ownership	Required propagated artifacts	Claim-safety rule
G / Layer 8	A12 StressForge robustness evaluation and A13 participation when stress is carried into embodiment outcomes	Layer 8 EvaluationOutcomeRecord, records/evaluation_outcomes/layer8, reports/matched_comparisons/layer8, metrics/metric_dictionary_layer8.yml, notes/negative_results/layer8, dashboard/export lineage, and manifests/claim_evidence/layer8	Stress evidence supports bounded stress evidence, and stress evidence supports bounded stress evidence. The result to diagnostic-only. Embodiment evidence supports bounded stress evidence, and stress evidence supports bounded stress evidence. The result to diagnostic-only and never supports clinical or real-device claims.
H / Layer 9	A13 embodiment-proxy evaluation and A12 participation when stress-conditioned embodiment outcomes are evaluated	Layer 9 EvaluationOutcomeRecord, records/evaluation_outcomes/layer9, reports/matched_comparisons/layer9, metrics/metric_dictionary_layer9.yml, notes/negative_results/layer9, dashboard/export lineage, and manifests/claim_evidence/layer9	Embodiment evidence supports bounded stress evidence, and stress evidence supports bounded stress evidence. The result to diagnostic-only and never supports clinical or real-device claims.
I / Layer 10	Read-only dashboard, card, export, and reproducibility visibility for A12/A13 artifacts	StressForge page, embodiment page, ablation/comparison page, cards/export page, reproducibility page, A12/A13 warning flags, diagnostic-only labels, and claim-to-artifact maps	Module I may inspect, filter, aggregate, card, and export A12/A13 artifacts, but it must not repair missing matches, mutate stress schedules, rewrite embodiment logs, retune reward weights, or remove claim-boundary labels.

All Layers 8–10 evaluation artifacts that participate in A12 or A13 must carry: global ablation ID, local layer ablation ID, source-record IDs, matched-comparison key, metric-dictionary ID or snapshot, dashboard/card/export lineage, negative-result-note pointer where applicable, claim-boundary label, and diagnostic-only flag when any required match fails. These fields extend the cross-layer record contract without changing the runtime ownership of StressForge generation, embodiment adaptation, or dashboard rendering.

The existing cross-layer consistency checklist is therefore strengthened as follows: every thesis-used Layer 8 A12 stress artifact and Layer 9 A13 embodiment artifact must carry its local/global ablation IDs, matched-comparison key, metric-dictionary ID or snapshot, dashboard/card/export lineage, negative-result note where applicable, claim-boundary label, and diagnostic-only flag when matching fails. This requirement is substantive because it governs claim eligibility, not merely wording.

17.16 Cross-Layer Consistency Checklist

Before a Stretch C experiment is treated as thesis-ready, the following checklist must pass.

- Every module has an owner, inputs, outputs, dependencies, forbidden dependencies, accepted failure states, and dashboard-facing artifacts.
- Every record has a producer, at least one consumer, required identifiers, version fields, and a provenance path.
- Every permitted feedback loop is declared with source, target, signal, trigger, action, safety constraint, and evaluation logic.
- Every major threshold has a selection split, threshold version, selection criterion, and test-set lockout proof.
- Every policy has an action-space version, legality rules, cost/reward version, seed, and controller card.
- Every stress profile has seed, intensity, target, injection point, schedule, stress family, and profile version.
- Every thesis-used Layer 8 A12 stress evaluation has a Layer 8 `EvaluationOutcomeRecord`, matched-stress comparison report, stress metric-dictionary entry, negative-result note where applicable, dashboard/export lineage, claim-evidence entry, and diagnostic-only status when matching fails.
- Every embodiment result carries a simulation-only and non-clinical warning, a Layer 9 owner, and a demo-manifest or explicit accepted-missingness marker.
- Every thesis-used Layer 9 A13 embodiment evaluation has a Layer 9 `EvaluationOutcomeRecord`, matched embodiment comparison report, embodiment metric-dictionary entry, negative-result note where applicable, dashboard/export lineage, claim-evidence entry, and diagnostic-only status when matching fails.
- Every Layer 9 embodiment dashboard artifact is read-only and traceable to saved `EmbodimentStateRecord`, `EmbodimentOutcomeRecord`, safety-event log, command timeline, endpoint-error curve, `Layer9RewardTrace` where enabled, and `Layer9EmbodimentDemoManifest`.
- Every dashboard figure and table is traceable to saved records through a figure/table manifest.
- Every Layers 0–3 evaluation-slice artifact carries an evidence mode, ablation-step ID where applicable, metric dictionary entry, matched-comparison lineage, negative-result note where applicable, and Layer 0 claim boundary.

- Where contextual-bandit/RL policy updates are enabled or audited, every policy mutation is represented by `PolicyUpdateTrace`, every disabled-update state is recorded, and dashboard exports keep training/adaptation curves separate from frozen-policy evaluation curves.
- Every thesis claim has evidence and a claim boundary; simulation and proxy claims are labeled honestly.
- No module silently changes another module's artifacts.
- The section remains consistent with Layer 0 through Layer 10, the Stretch C scope, the record-first design, the functional graph, and the claim-safety rule.

The strongest implementation of this cross-layer architecture is therefore not the one with the most complex model. It is the one in which every module can be tested independently, every record can be traced, every decision has a reason code, every feedback loop is justified, every stress or embodiment condition is declared, every dashboard artifact is read-only and reproducible, and every thesis claim is bounded by the evidence that the architecture actually produces.

18 Detailed Input-Output Proximity Map

This section defines the implementation-facing contract map for IHARQ BenchGuard under the Stretch C scope. It places each module close to the records it consumes and produces, identifies immediate and delayed dependencies, separates feed-forward evidence flow from feedback control flow, and clarifies how each output becomes auditable thesis evidence. The map is not merely a table of inputs and outputs. It is the traceability bridge between the conceptual architecture, the implementation modules, the experiment controller, the ablation design, the dashboard, and the final thesis artifacts. After the Layers 0–3 evaluation-slice merge, it also records how public-data/protocol matchability, raw decoder baselines, calibration/threshold/selective-prediction comparisons, and claim-evidence gating remain proximate to the records that justify them.

The layer addresses a central bottleneck of this thesis architecture: a reliability system can become scientifically weak if records, thresholds, policy actions, stress perturbations, simulated outcomes, and dashboard figures are connected only implicitly. This proximity map prevents that weakness by specifying which module owns each record, why each record is proximate to a decision, which boundary checks must pass before the next record is trusted, which feedback paths are permitted, and which claims are allowed from each artifact. If this section were removed, the architecture would still contain strong individual layers, but it would lack a single inspectable contract proving that data movement, decision logic, evaluation evidence, and scope control are coherent across the whole system.

The layer supports the thesis claim that IHARQ BenchGuard is not only a decoder benchmark but a record-first reliability and readiness architecture. It protects against the criticism that the system is a loose collection of modules by showing how public EEG records, calibrated uncertainty, IHARQ decisions, temporal trust states, adaptive policy actions, simulated transitions, stress profiles, Layer 9-owned embodiment states/safety events/reward traces/`Layer9EmbodimentDemoManifests`, Layer 10 dashboard views, cards, plots, and reproducibility artifacts are connected through declared interfaces.

18.1 Purpose of the Proximity Map

The proximity map exists for eight implementation and thesis reasons.

1. It prevents ambiguity about which module owns each record and which module is only a consumer.
2. It prevents hidden dependencies between layers, especially downstream tuning that could leak into dataset splits, calibration thresholds, policy labels, stress definitions, or dashboard exports.
3. It makes feed-forward evidence flow, feedback control flow, and reporting flow explicit rather than leaving them as informal implementation choices.
4. It supports reproducible implementation by linking records to configuration files, seeds, code versions, storage locations, validation gates, and manifests.
5. It supports ablation design by making the minimum set of records required for each comparison visible, including the Layer 1 matched-comparison keys, Layer 2 A0/A4 baseline evidence, Layer 3 A1–A3 operating-point tables, and Layer 0 claim-evidence entries required before an ablation result can support thesis wording.
6. It supports dashboard traceability by requiring every card, plot, table, and warning to point back to saved source records.
7. It supports failure analysis and claim-safety auditing by identifying boundary failures before they become thesis claims.
8. It improves thesis readability by linking modules, records, figures, tables, cards, and appendices into one coherent architectural graph.

Within the full architecture, this section is a core architectural layer rather than an appendix. It is the layer that turns Layer 0–Layer 10 and the cross-layer modules into an implementable interface specification. It also separates core responsibilities from optional Stretch C extensions: the public-data benchmark, decoder records, calibration records, IHARQ records, `RegimeStateRecord`/conditional `TrustStateRecord` temporal-trust records, and dashboard artifacts are core; contextual bandit, reinforcement learning, StressForge-Lite, and MyoSuite/OpenSim are Stretch C modules that must remain simulation-only and claim-bounded.

18.2 Proximity-Map Principles

An input and output are treated as proximate when the architecture has a scientific or engineering reason to place them next to each other. Proximity is therefore not the same as file adjacency or code adjacency. It is an interface property. The following proximity dimensions govern this section.

Proximity dimension	Architectural definition
------------------------	--------------------------

Functional proximity	An input is proximate when it is directly required for the module's primary function. For example, <code>WindowRecord</code> , decoder configuration, and split membership are functionally proximate to <code>PredictionRecord</code> .
Temporal proximity	An input is proximate when it belongs to the same trial, window, session, subject, regime interval, stress episode, policy step, or simulated control step as the output. This prevents accidental aggregation across incompatible time scales.
Semantic proximity	An input is proximate when its meaning is transformed into the output's meaning. Raw decoder scores become calibrated probabilities; uncertainty and stability become evidence-quality features; evidence quality and regime context become readiness decisions.
Validation proximity	An input is proximate when it must be checked before the output can be trusted. Calibration cannot consume predictions without confirming score availability, split identity, calibration-split ownership, and model identity.
Decision proximity	An input is proximate when it directly affects accept, wait, request-evidence, combine, fallback, defer, unsafe, threshold-change, or policy-action decisions.
Feedback proximity	An input is proximate when it comes from a downstream monitoring, policy, simulation, stress, or embodiment module and intentionally modifies future readiness behavior through a declared, logged, bounded, and reproducible loop.
Reporting proximity	An input or output is proximate when it appears in cards, plots, dashboards, ablation tables, matched-comparison reports, negative-result notes, failure taxonomy entries, export manifests, or reproducibility artifacts.
Claim-safety proximity	An input or output is proximate when it constrains what the thesis is allowed to claim. Public EEG records, decoder baselines, calibration/uncertainty records, selective-prediction records, simulation records, embodiment records, policy reward records, dashboard cards, metric dictionaries, matched-comparison reports, negative-result notes, and claim-evidence matrix entries must remain semantically distinct, and evaluation artifacts must inherit the Layer 0 claim-evidence gate before they support thesis claims.

These principles define the layer boundary. The proximity map does not train decoders, calibrate probabilities, choose policies, run simulations, generate stress, or execute embodiment. It specifies the contracts that make those operations auditable, reproducible, and scientifically bounded.

18.3 Global Feed-Forward Record Chain

The main forward path begins with public EEG and ends with thesis artifacts. Each step transforms a record into a more decision-relevant or reporting-relevant record while preserving provenance. Not every experiment will instantiate every record. A minimum implementation may stop after baseline decoding and calibration; a target implementation may stop after IHARQ and RegimeRisk; the maximum Stretch C implementation defines the full path through adaptive policy, simulation, StressForge-Lite, optional embodiment, dashboarding, and repro-

ducibility packaging.

Raw public EEG and metadata → DatasetRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, DatasetCard, ProtocolCard, ValidationReport, Layer1Manifest, saved split manifests, and layer1_ablation_readiness_l1_v1.json → WindowRecord with parent-trial lineage fields → PredictionRecord, ModelRegistryRecord, BaselineMetricRecord, LowCalibrationCurveRecord, SubjectProfileRecord, EnsembleControlRecord, FailureCaseIndex, Layer2ReadinessReport, ModelCard, BenchmarkCard, A0 raw-decoder summaries, and A4 ordinary-combination controls → CalibrationRecord, UncertaintyRecord, SplitIntegrityReport, LeakageWarningRecord, CalibrationEligibilityTable, A1 calibration deltas, and metric-dictionary entries → SelectivePredictionRecord, ThresholdRegistryRecord, Layer3MatchedOperatingPointTable, matched-coverage/matched-cost operating-point tables, A2/A3 comparison summaries, HighConfidenceWrongTable, CalibrationDashboardMetricRecord, NegativeResultNotes, ReliabilityAuditReport, and ReliabilityCard → EvidenceQualityRecord, IHARQDecisionRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, Layer 4 EvaluationOutcomeRecords, IHARQ matched-comparison reports, metric-dictionary entries, negative-result notes, claim-evidence entries, and dashboard-ready IHARQ trace exports → RegimeStateRecord, conditional TrustStateRecord, regime_timelines.csv, trust_scores.csv, stop_loss_warnings.csv, regime transition logs, threshold-adjustment logs, risk-error alignment tables, Layer 5 EvaluationOutcomeRecords, RegimeRisk matched-comparison reports, shuffled/random-risk negative-control reports, metric-dictionary entries, negative-result notes, claim-evidence entries, dashboard-ready regime tables, and RegimeCard exports → ReadinessContextRecord → PolicyActionRecord, DeferralRecord → TransitionRecord, RewardTraceRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, EnvironmentDiagnosticRecord, SafetyEventRecord where materialized, optional PolicyUpdateTrace → StressProfileRecord, StressScheduleRecord, stressed-record pointers, Layer 8 A12 EvaluationOutcomeRecords, matched-stress comparison reports, stress metric-dictionary entries, stress negative-result notes, and stress claim-evidence entries → EmbodimentStateRecord, EmbodimentOutcomeRecord, embodiment-linked TransitionRecord pointers, SafetyEventRecord or safety-event log, command timeline, endpoint-error curve, Layer9RewardTrace where enabled, Layer9EmbodimentDemoManifest, dashboard-compatible embodiment export, Layer 9 A13 EvaluationOutcomeRecords, MatchedComparisonReports, embodiment MetricDictionaryEntry entries, NegativeResultNotes, and embodiment ClaimEvidenceMatrix entries → DashboardMetricRecord with optional policy_update_filter, embodiment_filter, reward_trace_filter, stress-evaluation filters, A12/A13 ablation-lineage filters, matched-comparison IDs, metric-dictionary references, diagnostic-only status, negative-result references, and claim-evidence linkage; DashboardViewRecord with page/panel/filter/completeness/export lineage; claim-evidence matrix entries, metric dictionaries, matched-comparison reports, policy-update trace cards where applicable, Layer 8 stress evidence cards, Layer 9 embodiment evidence cards, separated training/adaptation versus frozen-policy evaluation exports, A12/A13 warning panels, cards, plots, ablation tables, failure taxonomy, export manifests,

and `ReproducibilityPackage`.

At the Layer 2 boundary, `PredictionRecord` is a cross-layer measurement record rather than a private decoder array. Its downstream scope covers calibration and uncertainty in Layer 3, evidence verification in Layer 4, temporal trust in Layer 5, policy-context construction in Layer 6, simulated replay in Layer 7, stress replay in Layer 8, bounded Layer 9 provenance/readiness-input compatibility, and Layer 10 dashboard/reproducibility views. The record must therefore carry canonical `split_role`, `timestamp_or_order` where replay is used, `score_type`, `class_order`, `model_id`, `decoder_id`, `model_family`, `label_budget_id`, `predicted_class`, `probability_vector` or declared score fallback, `seed`, `config hash`, and evaluation-only `label_true` where labels are legally available. Local aliases such as `y_true`, `y_pred`, and `prob_vector` are implementation details only and must not become unmapped cross-layer fields. Layer 2 companion outputs such as `ModelRegistryRecord`, `BaselineMetricRecord`, `LowCalibrationCurveRecord`, `SubjectProfileRecord`, `EnsembleControlRecord`, `FailureCaseIndex`, and `Layer2ReadinessReport` must remain close enough to each `PredictionRecord` to support A0/A4 baselines, downstream schema validation, dashboard cards, and reproducibility without rerunning hidden decoder scripts.

At the Layer 3 boundary, calibration, uncertainty, threshold, and selective-prediction artifacts are evidence-preparation records rather than readiness actions. `CalibrationRecord` and `UncertaintyRecord` must preserve links to source `PredictionRecord` and `ModelRegistryRecord` entries; `SelectivePredictionRecord` and `ThresholdRegistryRecord` must preserve selection split, evaluation split, target constraint, allowed consumers, leakage status, and matched operating-point lineage. `SplitIntegrityReport`, `LeakageWarningRecord`, `CalibrationEligibilityTable`, `ReliabilityAuditReport`, `HighConfidenceWrongTable`, `CalibrationDashboardMetricRecord`, `Layer3MatchedOperatingPointTable`, and `ReliabilityCard` are reporting and validation artifacts that make calibration/threshold sufficiency, calibration failure, and high-confidence wrong cases visible without turning them into decision-time ground-truth features.

This chain is directional for evidence production. It does not mean that every downstream artifact is allowed to modify upstream evidence. Public EEG records, `SplitRecord` instances, saved split manifests, and Layer 1 cards/reports remain stable. Calibration thresholds remain tied to the validation source that created them. IHARQ and policy decisions can use prior information and declared context, but they cannot retroactively change decoder records. Dashboard views are read-only interpretability surfaces unless an explicit new experimental run is created. Layer 7 policy-update traces are an optional branch of the simulation transition path: they are produced only when contextual-bandit/RL update mode is explicitly enabled or when disabled-update auditing is requested. In frozen-policy evaluation, `TransitionRecord.policy_update_trace_id` is absent or points only to a disabled-update audit entry, never to a policy mutation. Layer 9 embodiment artifacts are similarly a Layer 9-owned optional branch, not a detached or retired runtime path: static replay, MyoSuite, and OpenSim evidence reaches Layer 10 only as saved embodiment records, safety-event logs, `Layer9RewardTrace` files where enabled, command timelines, endpoint-error curves, `Layer9EmbodimentDemoManifests`, and dashboard-compatible exports.

18.4 Control and Feedback Paths

Feedback is separated from ordinary feed-forward data flow. A feedback path is allowed only when it has a declared source, target, signal, trigger, action, safety constraint, and evaluation logic. The default rule is that records move forward and remain immutable. Feedback may

influence future decisions, future thresholds within permitted adaptation rules, simulation states, or future policy actions, but it must not silently rewrite prior records.

Source	Target	Signal and trigger	Action	Safety and reproducibility constraint	Evaluation
Layer 2 validation	Layer 1 protocol / experiment controller	Split-leakage, sparse-budget, missing split-ID, label-budget leakage, or downstream-readiness failure	Regenerate or repair split/budget manifests before final training, or mark the run invalid/diagnostic only	No test-outcome-driven protocol repair; write Layer2ReadinessReport , failed-run status, and where applicable LeakageWarningRecord	Downstream compatibility, readiness-validator pass
Layer 2 prediction logger	Layer 3 compatibility check	Missing probability/logit, label-only outputs, malformed score schema, missing score_type , or class_order mismatch	Mark score type, route to declared fallback, repair export, or exclude the model with documented reason	No hidden inference of decoder state; all excluded or fallback models remain visible in registry/cards	Probability/logit availability, Model-Registry completeness, and calibration eligibility coverage.
Layer 2 ensemble builder	Layers 3–4 and Layer 10	Aligned model-family predictions permit ordinary voting, probability averaging, confidence-threshold, longer-window, or multi-window controls	Export EnsembleControl and matched A4 control summaries	Do not conflate Delion model combination with IHARQ evidence verification or calibrated evidence quality	A4 ordinary-combination control coverage and matched-comparison audits.
Layer 3 split/leakage guard	Layer 1 / experiment controller / Layer 2	Calibration, threshold-validation, adaptation, and final-test roles overlap invalidly, or a prediction file lacks legal split lineage	Stop final evaluation, write SplitIntegrityReport and LeakageWarningRecord , and require corrected upstream records before final claims	No repair using final test labels; exploratory runs remain warning-tagged	Leakage pass rate, calibration eligibility, and diagnostic-only down-grade count.

Layer 3 input validator	Layer 2	Prediction records contain malformed probabilities, missing IDs, hidden aliases, score/class-order mismatch, or unsupported label-only fallback	Repair Layer 2 export, mark score-only/invalid, or exclude affected model with registry-visible reason	Evaluation-only labels and correctness helpers must not become decision-time features	Schema validation, score availability, and source-record linkage checks.
Layer 3 reliability and threshold audit	Layers 4, 6, 10, and experiment runner	Sparse calibration, high-confidence failure clusters, coverage collapse, or unregistered threshold request	Emit ReliabilityAuditReports , HighConfidenceWrongfile , operating-point warnings, or fail fast on unknown thresholds	Test-derived Reports are wrongfile, analysis; unregistered thresholds cannot support final IHARQ/policy/dashboards claims	Risk-coverage, matched-cost/coverage, threshold-registry coverage, and high-confidence-wrong audits.
RegimeRisk	IHARQ	Rising RegimeStateRecord , trust degradation, stop-loss, instability, drift-risk, fatigue-like proxy trend, dropout risk, or trusted-set restriction	Tighten evidence requirements, adjust request-evidence limits, restrict trusted evidence sources, raise wait/fallback/no-action/unsafe sensitivity, or annotate IHARQ context	Only prior or current temporal information may be used; no future labels; all changes logged in IHARQDecisionRecord , IHARQTraceRecord , and Layer 5 warning/timeline artifacts	Compare IHARQ with and without regime context Record , matched coverage/cost.
RegimeRisk	Adaptive policy	Regime features become part of ReadinessContextRecord	Provide temporal trust, Record , trusted set, stop-loss, no-action/unsafe constraints, and volatility-like features to policy	Features must be computed before the policy action time and versioned by regime config, with TrustStateRecord lineage when independent trust timelines are used	Policy ablation with and without regime features.

IHARQ	RegimeRisk	Repeated uncertain, unsafe, fallback, request-evidence, or high-risk reason codes	Update temporal trust estimation and regime summaries	IHARQ events are inputs to rolling summaries only; they do not alter past predictions	Regime timelines and temporal replay checks.
Policy	Closed-loop simulation	PolicyActionRecord at simulated policy step	Execute simulated accept/wait/request/fallback/legality/unsafe action and produce state transition	Simulation identity, reward mask, and seed must be recorded	Reward, regret, shirder, unsafe-event, and transition analyses.
StressForge-Lite	Closed-loop simulation	Declared StressProfileRecord and StressScheduleRecord	Apply declared Record , dropout, noisy-intent, Record noisiness, drift, fatigue-like degradation, or combined-stress conditions according to the profile and schedule while preserving clean source records	Stress profile must point to clean source records and must be reproducible by seed and schedule	Clean vs stressed sensitivity curves and stress cards.
StressForge-Lite	Layer 9 embodiment path	Declared StressProfileRecord and StressScheduleRecord with embodiment-side stress parameters	Apply Record noisiness, slowdown, Record endpoint noise, action delay, safe-hold, or command-smoothness stress through the Layer 9 stress injector and adapter path	Embodiment stress must remain separate from EEG/evidence stress, must be logged as simulation-only proxy behavior, and must link to EmbodimentStateRecords , EmbodimentOutcomeRecords , safety events, and Layer9EmbodimentDenial	A13 stress-conditioned embodiment outcomes, unsafe-event summaries, endpoint-error plots, and DenialManifests embodiment stress cards.

Closed-loop simulation	Bandit/RL policy	RewardTraceRecord , TransitionRecord or evaluate with optional TransitionRecord optional policies under PolicyUpdateTrace , disabled-update audits, regret, and outcome summaries	Train, adapt, or evaluate contextual policies under declared reward/cost definitions	Training, validation, and test simulated sessions must be separated; reward must not be retrofitted after evaluation; frozen evaluation must not update the policy; simulation-authorized updates require one trace per mutation.	
Embodiment adapter	Policy/simulation	EmbodimentStateRecord , EmbodimentOutcomeRecord , TransitionRecord , SafetyEventRecord , or safety-event log, command success, endpoint error, command timeline, Layer9RewardTrace where enabled, and Layer9EmbodimentDemoManifest	EmbodimentStateRecord , EmbodimentOutcomeRecord , TransitionRecord , SafetyEventRecord , or safety-event log, command success, endpoint error, command timeline, Layer9RewardTrace where enabled, and Layer9EmbodimentDemoManifest	Outcomes must be labeled simulated/proxy-only; reward traces remain simulation-only; dashboard views must not control simulators or become clinical validation evidence	Layer 9 embodiment demonstration panels, unsafe-event summaries, endpoint-error plots, command timelines, demo-manifest index, and non-clinical limitation cards.
Dashboard	Human interpretation	Dashboard cards, plots, warning panels, drilldowns, export manifests	Support inspection, thesis reporting, and debugging decisions	Dashboard must not silently mutate experimental records, thresholds, labels, or policies	Export-manifest validation and figure-source checks.

Arbitrary feedback loops are forbidden. For example, poor test accuracy must not change the split manifest; dashboard filtering must not remove failed cases from thesis tables; embodiment success must not relabel public EEG predictions; and simulation rewards must not be described

as real-world therapeutic outcomes.

18.5 Expanded Module-Level Proximity Table

The following table is the central module-level contract. It includes both the revised layer modules and the cross-layer functional modules A–I. Rows marked as core are required for the main thesis evidence. Rows marked as Stretch C are required only for the maximum implementation path. Rows marked as reporting or governance operate across the full system.

Module / layer	Plane	Main responsibility	Closest upstream inputs and granularity	Core transformation and closest outputs	Consumers and feedback	Validation, failures, thesis artifacts
Layer 0: Claim-Safety and Scope Governance	Governance	Defines allowed and forbidden scientific claims; audits public-data, simulation, embodiment, policy, dashboard, ablation, metric-dictionary, matched-comparison, negative-result, and final-demonstration language.	Research scope, dataset provenance, simulation declarations, embodiment mode, cards, figures, thesis claims, MetricDictionaryEntry, MatchedComparison, NegativeResult, and ClaimEvidenceMatrix entries; report-level granularity.	Converts evidence provenance into claim-boundary decisions, warning tags, diagnostic-only or invalid-claim labels, ap-proved/limited claim text, Reports, statements, warning-inheritance rules, and claim-evidence matrix entries.	Consumed by all layers and Module I; feedback to human thesis writing only, not experimental records.	Gates clinical overclaim, unlabeled simulation, missing limitations, unmatched comparisons, ambiguous metrics, and unsupported thesis wording; produces claim-safety warnings, limitation statements, negative-result linkage, and thesis-scope audit.

Layer 1: Public- Data and Protocol Anchor / Module A	Measurement and gov- ernance	Owns public EEG ingestion, dataset identity, protocol versioning, split construction, label mapping, preprocessing scope, artifact/channel- quality flags, valida- tion/leakage auditing, dataset/protocol cards, and low- calibration/label- budget protocols.	Raw public EEG, metadata, labels, channel maps, dataset configs, protocol configs, Layer 0 claim constraints; dataset/session/run/ granularity.	Transforms raw files into DatasetRecord , SplitRecord , WindowRecord , PreprocessingRecord , LabelMapRecord , ArtifactFlagRecord , DatasetCard , ProtocolCard , ValidationReport , Layer1Manifest , saved split manifests, and layer1_ablation_readiness_11_manifest . trial identity is preserved through parent-trial lineage fields in WindowRecord .	Consumed by decoders, calibration, IHARQ, RegimeRisk, policies, stress display, Layer 9 embodiment map- ping/provenance checks, and Layer 10 readiness_11_manifest . board/reproducibility views; no downstream module may silently alter it.	Gates dataset identity, sub- ject/session/ labels, split leakage, channel metadata, task labels, preprocess- ing scope, label- budget mitigations artifact flags, validation status, card limitation text, and ablation- readiness status; failures produce protocol warnings, dataset/protoc cards, ValidationR Layer1Manifest and layer1_abla
---	------------------------------------	--	--	--	--	--

Layer 2: Decoder and Baseline Measure- ment Spine / Module B	Measurement Trains and evaluates sanity, classical, Riemannian, compact deep, optional compact SSL/foundation-sub- style, and ensemble- control baselines under fixed Layer 1 protocols.	WindowRecord with parent-trial lineage fields, DatasetRecord , SplitRecord , PreprocessingRecord , LabelMapRecord , into reproducible ject/session/channel metadata, split/budget configs, decoder/model configs, held-out labels for evaluation only; win- dow/trial/model/benchmark granularity.	PredictionRecord , ModelRegistryRecord , BaselineMetricRecord , LowCalibrationCurvesRecord , SubjectProfileRecord , EnsembleControlRecord , FailureCaseIndex , trained model artifacts, model-family dis- agreement tables, model/benchmark cards, training/inference/failures, run logs, and Layer2ReadinessReport	Does not simply output predictions; transforms windowed EEG and fixed configuration into reproducible records, artifacts, model-family dis- agreement tables, model/benchmark cards, training/inference/failures, run logs, and bounded	Consumed by Layer 3 calibra- tion/uncertainty, Layer 4 IHARQ feature con- struction, Layer 5 RegimeRisk , Layer 6 Identity- enabled , replay, stress bounded Layer 9 provenance/readiness; input compatibil- ity, Layer 10 dash- board/cards/reports and A0/A4 base- line/evaluations tables.	Gates model identity, used, pre- processing fit scope, training split, split_role , label_budg score_type , class_order , timestamp_o where replay is required, probability/lo score availability, registry path, failed/skippe status, and hidden alias aliasing; failures include prediction without prohibited orphan predictions, split mismatch, missing score schema, hidden alias fields, and hidden preprocess- ing leakage; artifacts include baseline tables, low- calibration curves, model registry records, ensemble- control summaries, model/bench cards, and
---	--	--	--	---	---	--

Layer 3: Calibration, Uncertainty, and Selective Prediction / Module C	Evidence	Calibrates decoder scores, computes uncertainty, audits reliability, defines selective-prediction and threshold baselines, and exports matched operating-point evidence.	PredictionRecord, ModelRegistryRecord, DatasetRecord, WindowRecord, calibration split, threshold-validation split, split manifest, calibration configs, uncertainty-feature configs, optional ensemble metadata, and evaluation-only labels/correctness fields; prediction/window/trial/model/budget granularity.	Does not simply compute uncertainty; transforms raw scores into CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, SplitIntegrityReport, LeakageWarningRecord, CalibrationEligibilityTable, ReliabilityAuditReport, HighConfidenceWrongTable, Layer3MatchedOperatingPointTable, CalibrationDashboardsMetricRecord, reliability diagrams, risk-coverage models/budget negative-result notes, and reliability-card inputs.	Consumed by Layer 4 IHARQ, optional Layer 5 summaries, Layer 6 ABQ/policy comparison, Layer 10 eligibilityTable, cards/responsibilityTable, settingsPointTable, dashboardMetricRecord, threshold registries may feed future decisions only under declared validation and allowed-consumer rules.	Gates calibration split, threshold-validation split, probability/score format, model-registry match, product-type, class_order, test coverage, and reason for missing probabilities; artifacts include reliability diagrams, ECE/Brier/N
--	----------	--	---	---	--	--

Layer 4: IHARQ Evidence Verification / Module D	Evidence and decision	Performs decoder- agnostic, post- calibration evidence verification with accept, wait, request- evidence, combine, fallback, uncertain, and unsafe actions.	PredictionRecord , Does not retrain CalibrationRecord , the decoder; UncertaintyRecord , transforms ThresholdRegistry , post-calibration optional evidence into RegimeStateRecord , EvidenceQualityReasoning, ArtifactFlagRecord , IHARQDecisionRecord, and and lo- CombinedEvidenceGate , cal/precomputed FallbackDecisionDashboard , window-history, UnsafeEventRecord , evaluation channel-subset- IHARQAblationRecords , and stability, and IHARQTraceRecord , repro- model- Layer 4 ducibility agreement EvaluationOutcomeRecords ; views; deci- accepted- sion/window/trial prediction logs, granularity. re- quest/combine/fallback/tighten logs, action costs, and reason-code distributions.	Consumed by RegimeRisk, policy context, reasoning, modiment Gate , Dashboard , evaluation scripts, and repro- ducibility records; feedback from RegimeRisk back/tighten require- ments or stop-loss routing, but only with explicit logging.	Gates calibrated confidence, entropy, margin, agreement, stability, threshold source, reason- code complete- ness, lo- cal/global ablation ID, matched- comparison key, metric- dictionary linkage, diagnostic- only status, and claim boundary; artifacts include IHARQ decision plots, reason- code tables, trace records, re- quest/combine logs, and matched baseline ablations.
---	-----------------------------	---	--	--	--

Layer 5: RegimeRisk Temporal Trust Monitoring / Module E	Regime	Estimates temporal operating regime, trust state, instability, drift-risk, fatigue-like proxy degradation, dropout risk, and stop-loss warnings.	Recent WindowRecord, PredictionRecord, CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, EvidenceQualityRegimeRecord, IHARQDecisionRecord, optional StressProfileRecord, TransitionRecord, EmbodimentStateRecords, policy/action history, and subject- difficulty profiles; regime- interval/session granularity.	Does not replace IHARQ; transforms recent evidence and temporal simulation records into RegimeStateRecord, conditional TrustStateRecord, and score tables, trusted records, risk trends, regime_timelines for stop_loss_warnings context, threshold- adjustment logs, risk-error alignment tables, feedback logs, Layer 5 EvaluationOutcomeRecords, and RegimeCard outputs.	Consumed by IHARQ, Adaptive- IHARQ, simulation, embodi- ment, dashboard, evaluation scripts, and thesis reporting; for regime context, forward and bounded feedback to IHARQ, Layer 6 Records, context, Layer 7 simulation state, Layer 9 simulated command gating, and Layer 10 read-only views.	Gates temporal ordering, rolling- window length, session identity, no future in- formation, aggrega- tion rules, simula- tion/proxy tags, matched- comparison keys, shuffled/rand risk negative controls, metric- dictionary linkage, and claim boundary; artifacts include regime timelines, trust-state plots, stop-loss views, risk-error alignment tables, matched- comparison reports, negative- result notes, Regime- Cards, and temporal- failure audits.
---	--------	--	--	---	--	--

Layer 6: Adaptive Readiness Policy Layer / Module F	Policy	Owens the merged policy-learning plane: P0/6.0 IHARQ-lite reference policy, P1/6.1 evidence-quality estimation, P2/6.2 supervised Adaptive-IHARQ, P3/6.3 learning-to-defer, P4/6.4 contextual bandit, and P5/6.5 RL readiness policy.	ReadinessContextRecord , IHARQ records, RegimeStateRecord , conditional TrustStateRecord where independent trust timelines are enabled, pseudo-labels, deferral targets, legality masks, action-space versions, cost/reward configs, action history, stress variables, optional PolicyUpdateTrace , policy-step granularity.	Record , make unrestricted autonomous clinical decisions; transforms declared context into expanded PolicyActionRecord , DeferralRecord , optional simulation-authorized PolicyUpdateTrace , learned-policy artifacts, action distributions, policy-matrix exports, reward/regret summaries, and controller cards.	Consumed by simulation, StressForge-conditioned evaluation, embodiment gate, dashboard; re-wards/transitions may train bandit/RL only under declared simulation splits and must preserve frozen-policy evaluation separation.	Gates context completeness, action-space legality, cost/reward version, training/evaluation separation, pseudo-label provenance, update-trace completeness, and artifacts include controller cards, action distributions, deferral curves, policy-stage/action-legality/cost-sensitivity matrices, and reward/regret plots.
--	--------	---	--	--	---	---

Layer 7: Simulated Closed- Loop Readiness Environ- ment / Module G	Simulation	Provides simulation- only state/action/reward dynamics for repeated readiness sessions and delayed consequences.	PolicyActionRecord, transforms DeferralRecord, policy actions ReadinessContextRecord replay state, prior context, optional StressProfileRecord, stress optional EmbodimentStateRecord, safety- links, simulator config, reward config, and optional policy-update config; policy- step/session granularity.	End, transforms policy actions TransitionRecord, train- ing/evaluation, RewardTraceRecording, success ClosedLoopSessionRecord, stress Epoch/Step/Safety/Random PolicyRolloutRecords, Record/Safety/AgentRecord/ safety-event logs, simulated session summaries, and optional PolicyUpdateTrace, only within files for declared bandit/RL update runs.	Consumed by bandit/RL train- ing/evaluation, SuccessForge StressForge records, adapters, dashboard; feedback to policy is re- ward/transition only within simulation and must be trace-linked when policy mutation occurs.	Gates envi- ronment identity, action semantics, reward definition, seed, session separation, update- enabled versus frozen- evaluation mode, policy- update trace com- pleteness, reward- trace linkage, ses- sion/rollout/ complete- ness, and environmen- t diagnostic warnings; artifacts include simulation curves, reward summaries, session timelines, policy- rollout cards, closed-loop environ- ment cards, disabled- update audits, policy- update traces, and simulation- only controller cards.
---	------------	---	---	---	---	--

Layer 8: StressForge- Lite Stress Generator / Module G	Simulation and stress	Creates bounded, reproducible stress profiles for delay, dropout, noisy intent, weakness, drift, fatigue-like degradation, and combined stress.	Clean records, target record IDs, stress family, intensity, injection point, schedule, seed, pol- icy/environment IDs; stress-episode granularity.	Transforms clean record references into StressProfileRecord , StressScheduleRecord , stressed-record pointers, stress- conditioned metrics, sensitivity curves, and stress cards.	Consumed by simulation, RegimeRisk , RegimeRisk , evaluation, embodi- ment, dashboard; feedback occurs through stress- conditioned transitions and reward outcomes only.	Gates clean- record pointer, stress family, intensity, injection point, schedule, seed, and version; failures include un- traceable stressed data or post-hoc stress changes; artifacts include stress sensitivity curves and Stress- Cards.
---	-----------------------------	---	--	--	---	--

Layer 9:	Simulation/OpenSim	Embodiment	PolicyActionRecord, does not DeferralRecord, validate real IHARQDecisionRecord, consistency; RegManifestStateRecord, transforms ReadinessContext, policy decisions optional and traceable TransitionRecord, Layer 1 StressProfileRequest, provenance into Layer 1 EmbodimentStateRecord, initial provenance EmbodimentOutcomeRecord, records embodiment- including linked DatasetRecord, TransitionRecord WindowRecord, pointers, SplitRecord, SafetyEventRecord PreprocessingRecord, safety-event LabelMapRecord, log, endpoint ArtifactFlagRecord, command and success/failure, Layer1Manifest, command intent-timeline, command Layer9RewardTrace mapping, readiness/safety gate, simulator/static-replay state, embodiment export, and mode, safety demonstration rules; panels. embodiment-step granularity.	Consumed by Layer 10 dash-board/reproduction simulation outcome analysis, and Record, Record, evaluation; policy only as simulation/proxy outcomes under declared modes.	Gates record intake, capability, and mapping, simulation-only scope, embodiment mode, readiness gate, safety block, simulator/static replay version, Layer 1 prove-nance/flag complete-ness, EmbodimentO complete-ness, Layer9Reward labeling where enabled, Layer9Embo complete-ness, and non-clinical warning; artifacts include embodi-ment cards, unsafe-event summaries, endpoint-error plots, command timelines, demo panels, and Layer 10 read-only embodi-ment pages.
----------	--------------------	------------	---	--	---

Layer 10: Dashboard, Cards, and Reproducibility / Module I	Reporting	Converts validated records into read-only dashboard views, standardized non-Layer-1 cards, plots, tables, failure taxonomy, policy-update trace cards where applicable, read-only rendering/export of Layer 1 DatasetCard and ProtocolCard , Layer 1 ablation-readiness lineage, Layer 9 embodiment evidence cards, and reproducibility package.	All validated records, manifests, configs, warning tags, source checksums, optional PolicyUpdateTrace files, disabled-update audit entries, Layer1Manifest , layer1_ablation_realtime_v1.json , MetricDictionary references, MatchedComparisonReports , NegativeResultNoClaimEvidenceMatrix entries, and Layer 9 embodiment artifacts including EmbodimentStateModelCard , EmbodimentOutcomeReliabilityCard , SafetyEventRecordRegimeCard , or safety-event logs, Layer9RewardTrace where enabled, command timelines, endpoint-error curves, Layer9EmbodimentDataManifest , simulator-version metadata, dashboard-compatible exports; dashboard/report granularity.	Transforms saved records into DashboardMetricRecord with optional policy_update_filter , embodiment_filters and reward_trace_filters ablation/matched-comparison/metric-dictionary/negative-experimental lineage, diagnostic-only Reports , DashboardViewRecord with page/panel/filter/completeness/metadata, rendered Layer 1 DatasetCard/ProtocolCard views, ModelCard , ReliabilityCard , RegimeCard , ControllerCard , policy-update trace cards, StressCard , EmbodimentCard , figures, tables, Layer 1 validation and readiness warning/readiness panels, Layer 9 embodiment export indexes, separated training/adaptation versus frozen-policy evaluation exports, FailureTaxonomy , and ReproducibilityPackage .	Consumed by thesis writing and inspection; dashboard feedback is interpretability and must not mutate experimental records, policy-update traces, Layer 1 protocol completeness/export negative-result visibility, claim-evidence linkage, claim warnings, Layer 1 ablation-readiness status, policy-update trace completeness, disabled-update audit status, frozen/update curve separation, Layer 9 safety-event/ Embodiment completeness, Layer9Reward labeling where enabled, and embodiment non-clinical warnings; failures include	Gates artifact completeness, versioning, provenance, source-record lineage, metric-dictionary linkage, matched-comparison linkage, diagnostic-only status, negative-result visibility, claim-evidence linkage, claim warnings, Layer 1 ablation-readiness status, policy-update trace completeness, disabled-update audit status, frozen/update curve separation, Layer 9 safety-event/ Embodiment completeness, Layer9Reward labeling where enabled, and embodiment non-clinical warnings; failures include
--	-----------	--	--	---	--	--

18.6 Record-Level Dictionary and Ownership Map

The following dictionary normalizes record names while preserving the existing document terminology. The existing `SplitRecord` remains the canonical split object; saved split manifests are run-level storage bundles that point to one or more `SplitRecord` instances and their leakage checks, not separate canonical interfaces. Layer 1 does not define a separate `ProtocolRecord` or `TrialRecord`: protocol identity is carried through `ProtocolCard`, `Layer1Manifest`, `SplitRecord`, and `PreprocessingRecord`, while trial identity and parent-trial lineage are carried inside `WindowRecord`. Similarly, `ModelRegistryRecord`, `BaselineMetricRecord`, `LowCalibrationCurveRecord`, `SubjectProfileRecord`, `EnsembleControlRecord`, `Layer2ReadinessReport`, `ReliabilityAuditReport`, `SplitIntegrityReport`, `Layer3MatchedOperatingPointTable`, `CalibrationDashboardMetricRecord`, `CombinedEvidenceRecord`, `FallbackDecisionRecord`, `UnsafeEventRecord`, `IHARQAblationRecord`, `IHARQTraceRecord`, `EvaluationOutcomeRecord`, `regime_timelines.csv`, `trust_scores.csv`, `stop_loss_warnings.csv`, `ClosedLoopSessionRecord`, `EpisodeSummaryRecord`, `PolicyRolloutRecord`, `RewardTraceRecord`, `EnvironmentDiagnosticRecord`, `EmbodimentOutcomeRecord`, `SafetyEventRecord`, `DashboardMetricRecord`, and `DashboardViewRecord` are explicit record-level or reporting-artifact names for artifacts already implied by the decoder, calibration/uncertainty, IHARQ, RegimeRisk, simulation, Layer 9 embodiment, and dashboard sections. The proximity map treats the former P20 embodiment dossier as retired; all embodiment records in this dictionary are Layer 9-owned and Layer 10-consumed.

Record/artifact	Owner	Created by / consumed by	Granularity	Mandatory and optional fields	Validation, storage, dashboard, claim relevance
<code>DatasetRecord</code>	Layer 1 / A	Created by data protocol; consumed by all experimental layers and dashboard.	Dataset/subject/ session/run	dataset ID, source, task labels, channels, sampling rate, preprocessing ID, split ID, protocol version, license, scope tag; optional notes and exclusions.	Validate identity and metadata; store in <code>records/dataset_rec</code> visible in <code>DatasetCard</code> ; anchors public-data claim boundary.

DatasetCard	Layer 1 / A	Created by dataset-card generator from Layer 1 records and Layer 0 constraints; consumed by Layer 10 and thesis chapters.	Dataset/protocol/ Mandatory	dataset identity, source/citation, task description, subject/session coverage, recording/channel details, label summary, inclusion rationale, limitations, allowed/forbidden claims, known risks, warning status, and source-record links.	Validate card-record consistency and public-data limitation coverage; store in cards/dataset_cards. Layer 10 may render/export but must not rewrite protocol content.
ProtocolCard	Layer 1 / A	Created by protocol-card generator from SplitRecord, PreprocessingRecord, WindowRecord, ValidationReport, Layer1Manifest, and Layer 0 constraints; consumed by Layer 10 and thesis chapters.	Protocol/run/ Mandatory	protocol ID/version, protocol type, split logic, low-calibration budget, preprocessing ID, windowing ID, leakage controls, downstream consumers, limitations, diagnostic-readiness warnings, and source-record links.	Validate protocol-card consistency, immutability, leakage-control visibility, and no downstream mutation; store in cards/protocol_cards. prevents protocol leakage and public-data overclaiming.

SplitRecord	Layer 1 / A	Created by split and protocol manager; consumed by Layers 2–6 and 8–10, with saved split manifests used only as storage bundles for one or more split records and leakage checks.	Subject/session/ manifest	Mandatory split ID, protocol name, split level, train/validation/tests/ calibration/adaptation membership, label_budget_id , calibration budget, seed, stratification rule, leakage-check status, and split version.	Validate split exclusivity, selection checks, and label-budget matchability; store in records/split_reco and saved split-manifest bundles; visible in reproducibility package; critical to no-leakage and matched-comparison claims.
WindowRecord	Layer 1 / A	Created by windowing engine; consumed by decoders, calibration/uncertainty, IHARQ/RegimeRisk/policy layers, stress replay, Layer 9 embodiment provenance checks, and dashboard.	Window/trial	Mandatory window ID, trial ID, parent trial ID, window-role tag, start/end, segment pointer, overlap group, normal-ized/original label, split membership, label_budget_id , preprocessing ID/version, channel mask, config hash; optional artifact flags.	Validate label mapping, split membership, leakage-safe overlap grouping, preprocessing-version lineage, and artifact flags; store in records/window_reco supports baseline, failure, embodiment-provenance, and dashboard lineage plots.

PreprocessingRecord	Layer 1 / A	Created by preprocessing registry; consumed by Layers 2, 3, and 8–10.	Protocol/run/lineage	Mandatory preprocessing ID, preprocessing version, filtering/resampling/channel preprocessing configs, normalization fit scope, artifact policy, implementation script, and config hash.	Validate train-only fitting scope, versioned preprocessing/epoch/metadata/lineage, and config hash; store in records/preprocessing ; visible in protocol cards, dashboard lineage, Layer 9 provenance, and reproducibility exports.
LabelMapRecord	Layer 1 / A	Created by task and label ontology mapper; consumed by Layers 0, 2, 7, 9, and 10.	Dataset/protocol/label	Mandatory label-map ID/version, dataset ID, original label, normalized label, task family, protocol inclusion, exclusion reason, rest/no-action policy, and proxy-intent note.	Validate proxy-intent limitations and label-map versioning; store in records/label_map_r ; visible in Dataset-Card/ProtocolCard and command-boundary warnings; not clinical intent evidence.
ArtifactFlagRecord	Layer 1 / A	Created by artifact and channel-quality annotator; consumed by Layers 2, 4, 5, and 8–10.	Dataset/subject/session/flag/window	Mandatory ID/type, affected dataset/subject/session/flag IDs, severity, action, source, and notes.	Validate miss- ingness/artifact pool/cyano/di/window downstream action; store in records/artifact_fi ; visible in failure taxonomy, Layer 9 safety/provenance checks, and dashboard limitation panels.

Layer1Manifest	Layer 1 / A	Created by provenance and manifest builder; consumed by every downstream layer, Layer 10, and reproducibility package.	Protocol/run	Mandatory manifest ID, protocol version, dataset IDs, paths to Layer 1 records/cards/reports and validation-report path, ablation-readiness path, config files, config hash, seed, code version, output paths, generation timestamp, validation status, ablation-readiness status, matched-comparison key completeness, and limitations.	Validate artifact existence, status consistency, and comparison-key completeness; store in records/reproducibility ; visible in artifact lineage and reproducibility package.
layer1_ablation_readiness	Layer 1 / A	Created by validation auditor and manifest builder; consumed by evaluation scripts, Layer 10, and thesis evaluation/reporting sections.	Protocol/run/readiness	Mandatory readiness status, matched-comparison key completeness, missing-key warnings, diagnostic-only downgrade reasons, validation-report links, and protocol-version provenance.	Validate A0–A4 public-data/protocol matchability; store in manifests/ ; visible in Layer 10 readiness/warning panels; reporting artifact only, not a runtime record.

ValidationReport	Layer 1 / A	Created by data validation and leakage auditor; consumed by all layers, Layer 10, evaluation scripts, thesis chapters, and reproducibility packaging.	Protocol/run/Mandatory	schema validity, metadata completeness, distribution summaries, split disjointness, leakage checks, duplicate/window-lineage checks, train-only preprocessing fitting status, label-budget integrity, comparison-key completeness, repeated-generation reproducibility status, and pass/warning/fail state.	Validate whether Layer 1 is downstream-ready; store in reports/layer1_val visible in Layer 10 validation/readiness panels; blocks thesis-grade downstream claims when failures are unresolved.
SubjectProfile	Layer 2 / B	Created by decoder evaluation; consumed by Layer 5 RegimeRisk diagnostics, Layer 6 policy-context analysis where allowed, and Layer 10 dashboard.	Subject/sessionMandatory	subject ID, session ID where available, source prediction/metric IDs, difficulty metrics, split-consistent baseline performance, low-label summaries, aggregation version, and split_role ; optional subject/session instability notes.	Validate split-consistent aggregation and source-record lineage; store in prediction summaries; visible in subject difficulty profiles; supports reliability heterogeneity claims without implying clinical phenotype.

Decoder configuration artifacts	Layer 2 / B	Created by experiment controller; consumed by decoder training/evaluation, ModelRegistryRecord , Layer 10 model views, and reproducibility manifests.	Model/run	Mandatory decoder family, hyperparameters, preprocessing dependency, seed, split/budget policy link, config hash, and fixed/selected value declarations; optional search notes.	Validate frozen config before test and registry linkage; store in configs/decoders and configs/layer2_exp visible in ModelCard ; prevents model identity ambiguity without creating a separate canonical ModelConfigRecord .
ModelRegistryRecord	Layer 2 / B	Created by decoder engine; consumed by calibration, IHARQ, RegimeRisk/policy provenance where needed, Layer 7 simulated replay, Layer 8 stress replay, and dashboard.	Model/run	Mandatory schema version, model ID, decoder ID, family, artifact path, training split, split_role , label_map_version , score_type , class_order , seed, config/code version, failed-run status, and model_registry_schema_yaml reference; optional ensemble_memberships	Validate registry path, split role, label-map version, score type, class order, artifact existence, and failed-run status; store in model registry; visible in ModelCard ; required for policy and traceability and replay compatibility.

PredictionRecord	Layer 2 / B	Created by decoder; consumed by Layers 3–8, by Layer 9 only as bounded provenance/readiness-input context, and by Layer 10.	Prediction/window/trial schema version, record ID, dataset/subject/session IDs, trial/window IDs, split ID, split_role , label_budget_id , model ID, decoder ID, predicted_class , probability_vector or declared logits/decision-score fallback, score_type , class_order , timestamp_order where replay is required, evaluation-only label_true where available, seed, and config hash; optional helper features only when mapped from local aliases.	Validate model identity, score schema, class order, split role, tempo-ral/replay order, and evaluation-only label separation; store in records/prediction visible in dashboard baseline plots, simulated replay lineage, and Layer 9 provenance/readiness-input checks; measurement evidence only, not embodiment-safety validation.
BaselineMetricRecord	Layer 2 / B	Created by evaluation; consumed by Layer 10 dashboard, evaluation scripts, and ablation/thesis-reporting sections.	Model/dataset/subject/metric ID, model ID, dataset/subject/session where applicable, accuracy/balanced accuracy/F1 as applicable, label_budget_id , split_role , aggregation level, metric dictionary entry, CI/seed summary, comparison key, and diagnostic-only flag where matching fails; optional statistical tests.	Validate metric level, split role, matched-comparison keys, seed aggregation, and metric dictionary linkage; store in thesis tables or records/baseline_m visible in baseline tables; supports decoder baseline claims.

LowCalibrationCurveRecords	Layer 2 / B	Created by low-label evaluation; consumed by Layer 10, thesis/evaluation artifacts, and optional Layer 3 budget-aware diagnostic analysis.	Model/dataset	Mandatory label_budget_id, model ID, dataset ID, performance values, seed summary, split_role, aggregation level, comparison key, and curve version; optional confidence intervals and budget-failure notes.	Validate budgets, split role, no subject leakage, and comparison-key completeness; store in curve tables or records/low_calibration; visible as low-calibration curves; supports low-calibration bottleneck claims without changing downstream threshold rules.
EnsembleControlRecords	Layer 2 / B	Created by ensemble comparison builder; consumed by Layers 3–4, Layer 10, and evaluation scripts.	Model/dataset/window/trial/control	Mandatory control type, source model IDs, aggregation rule, matched-comparison key, coverage/cost where applicable, delay-burden note, label_budget_id, split_role, seed, and config hash; optional disagreement features and longer-window metadata.	Validate alignment of source predictions, model-family membership, split/budget matching, and no hidden IHARQ logic; store in records/ensemble_records; visible in benchmark cards and A4 ordinary-combination controls.

FailureCaseIndex	Layer 2 / B	Created by decoder evaluation and logging; consumed by Layers 4–5, Layer 10, and defensive-design reporting.	Prediction/model/subject/session	Validate source prediction IDs, model ID, error category, raw-confidence helper where legal, subject/session grouping, failed/skipped-run link, and diagnostic-only status; optional difficult-subject note.	Validate evaluation-only target separation and source-record linkage; store with failure/debug reports; visible in failure views; supports limitation analysis without becoming a decision-time feature.
Layer2ReadinessReport	Layer 2 / B	Created by downstream readiness validator; consumed by Layers 3–10, Layer 10 readiness panels, evaluation scripts, and reproducibility manifests.	Run/consumer/monitor	Monitors coverage of prediction, registry, score-schema, class-order, label-budget, subject-profile, ensemble-control, and consumer-compatibility requirements, missing-field warnings, diagnostic-only downgrade reasons, source paths, and schema versions.	Validate downstream loader compatibility and hidden-field absence; store in reports/layer2_readiness or run manifests; visible in dashboard readiness panels; prevents downstream layers from inferring hidden decoder state.
ModelCard	Layer 2 export / Layer 10 render	Produced from ModelRegistryRecord , BaselineMetricRecord , failed-run logs, and decoder configs; consumed by thesis and dashboard.	Model/family/mandatory model	Mandatory model identity, registry path, split/budget lineage, score type, class order, metrics, limitations, failed/skipped status, and source-record links.	Validate card-record consistency; store in cards/model_cards ; reporting artifact only, not a new experimental result.

BenchmarkCard	Layer 2 export / Layer 10 render	Produced from baseline metrics, low-calibration curves, subject profiles, ensemble controls, and readiness reports; consumed by thesis and dashboard.	Benchmark/calibration	Mandatory baseline table links, low-calibration links, subject-profile summaries, ensemble-control summaries, warning flags, matched- comparison keys, and diagnostic-only status.	Validate benchmark source lineage; store in cards/layer2_benchmark
CalibrationRecords	Layer 3 / C	Created by calibration engine; consumed by Layers 4 and 6, optional Layer 5 summaries, Layer 10, and reliability cards.	Prediction/validation	Flowchart source prediction/model IDs, raw and calibrated probabilities/confidence, calibration method and scope, calibration split ID, reliability-bin membership, correctness for evaluation-only scoring, Brier/NLL components, ECE-style summaries, and config hash.	Validate calibration split, probability format, model-registry match, no test leakage, and evaluation-only field separation; store in records/calibration

UncertaintyRecords	Layer 3 / C	Created by uncertainty engine; consumed by Layers 4 and 6, optional Layer 5 summaries, and Layer 10.	Prediction/winning model	Model/utility linked prediction/calibration IDs, confidence, entropy, normalized entropy, margin, top-1/top-2 classes, variation ratio, uncertainty profile, selective-accept flag, and config hash; optional disagreement and conformal set size.	Validate calibrated/uncalibrated distinction, source linkage, and label-free decision-time eligibility; store with calibration records; visible in uncertainty plots; supports high-confidence-wrong audit.
SelectivePredictions	Layer 3 / C	Created by selective baseline engine; consumed by Layers 4 and 6, Layer 10, and ablation scripts.	Threshold/model/dataset	Model/dataset/budget threshold ID, uncertainty feature, selection split, evaluation split, coverage, rejection rate, retained accuracy/error/risk, high-confidence wrong rate, class/subject/session coverage, calibration budget, and operating-point lineage.	Validate no test-set threshold choice, matched coverage/cost keys, and group-wise coverage disclosure; store in selective tables; visible in risk-coverage curves; baseline for IHARQ claims.
ThresholdRegistry	Layer 3 / C	Created by threshold registry; consumed by Layers 4, 6, 10, and experiment runner.	Threshold/run/model	Model/utility threshold type, feature, value/rule, selected-on split, selection rule, target constraint, applicable models, allowed consumers, version, leakage status, rationale, and fairness note.	Validate selection provenance, allowed-consumer use, and no unregistered threshold in final runs; store in <code>threshold_registry</code> visible in threshold audit; central to claim defensibility.

SplitIntegrityReport	Layer 3 / C	Created by split/leakage guard; consumed by experiment controller, Layer 10, and thesis/reproducibility views.	Split/run/eligibility	Mandatory split membership checks, calibration/threshold/test disjointness, leakage warnings, nested-CV legality where used, eligibility tables, invalid-run flags, and final-claim eligibility state.	Validate split legality before calibration/threshold selection; store with Layer 3 reports; visible in reproducibility/readiness views; blocks thesis-grade claims when invalid.
LeakageWarningReport	Layer 3 / C	Created by split/leakage guard when invalid overlap or hidden leakage is detected; consumed by experiment controller, Layer 1/protocol repair workflow, Layer 10, and claim-safety audits.	Warning/run/split	Mandatory warning ID, offending split/membership condition, affected source IDs, severity, required action, final-evaluation block status, and exploratory-only tag.	Validate warning inheritance into dashboards and claim evidence; store with split-integrity reports; prevents test-derived calibration or threshold claims.
CalibrationEligibilityTable	Layer 3 / C	Created by calibration eligibility guard; consumed by calibrators, threshold search, Layer 10, and evaluation scripts.	Model/dataset/budget	Mandatory method eligible/ineligible status, missing class or sparse-budget flags, allowed calibration methods, fallback method, split/legal-use status, and invalid-run reason.	Validate sparse-budget and method-selection legality; store with split-integrity reports; prevents hidden method tuning on final test data.

ReliabilityAuditReport/ C	Created by reliability audit; consumed by the- sis/dashboard, reliability cards, and negative-result reporting.	Model/dataset/Model	Mandatory reliability diagrams, ECE/Brier/NLL summaries, confidence histograms, high-confidence wrong cases, group-wise audits, risk-coverage summaries, source records, and warning flags; optional subgroup analyses.	Validate source linkage and metric dictionary references; store in reports; visible in ReliabilityCard ; supports uncertainty contribution without safety guarantees.
HighConfidenceWrongTable/ C	Created by reliability audit/error miner; consumed by Layers 4 and 6 for offline analysis only, Layer 10, and thesis limitations.	Prediction/error/Model	Mandatory source prediction/calibration/uncertainty IDs, class-pair error, confidence threshold, subject/session/model family/budget grouping, correctness source, and diagnostic-only status.	Validate evaluation-only certainty separation; store with reliability reports; visible in high-confidence-error dashboard views; not a decision-time policy feature.
CalibrationDashboardMetricRecord	Created by dashboard/card exporter; consumed by Layer 10 dashboard backend, card generator, and reproducibility manifests.	View/report/Model	Mandatory plot-ready metrics, bin tables, risk-coverage curve data, filters, warning flags, card references, source-record IDs, and export lineage.	Validate source-record lineage and config hash; store with dashboard export records; visible in calibration/uncertainty coverage views.

ReliabilityCard	Layer 3 export / Layer 10 render	Produced from calibration, uncertainty, selective- prediction, threshold, high- confidence-error, and group-wise reliability artifacts; consumed by thesis and dashboard appendices.	Card/report	Mandatory source records, warnings, diagnostic-only status, claim boundaries, calibra- tion/uncertainty summaries, threshold lineage, and reliability plots/tables.	Validate card-source consistency; store with cards/dashboard exports; reporting artifact only, not a new scientific source.
EvidenceQualityRecord	Layer 4 / D and Layer 6 / F	Created by IHARQ evidence verification or learned evidence-quality estimation; consumed by IHARQ, RegimeRisk, policy, simulation, dashboard, and evaluation scripts.	Decision/window	Mandatory evidence-quality score, source IDs, confidence, entropy, margin, stability/signal- quality/disagreement features, missing-evidence flags, and decision- time/evaluation- only partition status; optional target construction and predicted failure probability when learned.	Validate feature availability, target provenance, source-record lineage, and no evaluation-only leakage; store in evidence records; visible in IHARQ/policy/regime cards; supports evidence- verification and learned- readiness claims without becoming clinical safety evidence.

IHARQDecisionRecord	Layer 4 / D	Created by IHARQ; consumed by RegimeRisk, policy, simulation, embodiment, dashboard, and evaluation scripts.	Decision/window	Mandatory canonical action token, reason code/text, accepted prediction if any, evidence request count, combined-evidence summary, fall-back/unsafe/uncertain flags, thresholds, action cost, rule path, config hash, seed, claim boundary, and source IDs; optional regime context and evaluation lineage fields.	Validate reason-code completeness, calibrated inputs, action vocabulary, threshold registry linkage, ablation IDs where evaluated, and downstream alias mapping; store in <code>records/iharq_recon</code> visible in reason-code figures and dashboard traces; readiness proxy only.
CombinedEvidenceRecord	Layer 4 / D	Created by the evidence combiner; consumed by IHARQ, policy, simulation, dashboard, and reproducibility.	Decision/window group	Mandatory source window/source IDs, combination method, before/after confidence, entropy, margin, combined class or vote summary, eligibility reason, final combined decision pointer, delay/evidence-cost fields, config hash, and provenance.	Validate source-window compatibility, combination eligibility, temporal proximity, and traceability; store with IHARQ records; visible in request/combine timelines and IHARQ ablation views.

FallbackDecisionRecord	Layer 4 / D	Created by fallback router; consumed by policy, simulation, embodiment, dashboard, and failure taxonomy.	Decision/fallback route	Mandatory fallback route, reason, target, availability, cost, selected state, source decision/evidence IDs, compatibility mode, claim boundary, and evaluation outcome where scored.	Validate fallback target existence, cost model, simulation/proxy tag, and source decision lineage; store with IHARQ records; visible in fallback burden views and defensive-design tables.
UnsafeEventRecord	Layer 4 / D	Created by unsafe-state monitor when evidence verification materializes an unsafe event; consumed by simulation, embodiment, dashboard, claim-safety cards, and failure taxonomy.	Decision/unsafe event	Mandatory unsafe trigger, reason, blocked action, relevant evidence IDs, RegimeRisk/stress context where available, recommended conservative outcome, severity, simulation/proxy boundary label, and warning text.	Validate unsafe trigger provenance, action blocking, scope label, and no clinical-safety overclaim; store with IHARQ/failure records; visible in unsafe-action logs and safety discussion.
IHARQAblationRecord	Layer 4 / D	Created by ablation and matched-comparison exporter; consumed by evaluation scripts, dashboard, and thesis ablation chapters.	Ablation run / decision group	Mandatory ablated variant, global/local ablation IDs, source decision IDs, matched-comparison key, metric-dictionary ID, metric outputs, diagnostic-only flag, negative-result note, and claim-evidence pointer.	Validate matched coverage/cost/action-cost lineage and variant definition; store in evaluation outcomes; visible in A5/A6/A7 IHARQ baseline tables.

IHARQTraceRecord	Layer 4 / D	Created by decision logger; consumed by dashboard, reproducibility packages, debugging, and thesis figures.	Decision/trace	Mandatory rule path, thresholds, feature values, reason-code hierarchy, input file paths/hashes, configuration hash, dashboard trace fields, metric-dictionary ID, figure/table lineage, and source IDs.	Validate trace completeness and source-record accessibility; store in IHARQ trace exports; visible in dashboard drilldowns and reproducibility manifests.
EvaluationOutcomeRecord	Layer 4 and Layer 5 evaluation exporters	Created by Layer 4 and Layer 5 evaluation exporters and other evaluated layers; consumed by Layer 10, evaluation scripts, thesis tables, and claim-evidence matrix generation.	Evaluation unit / ablation group	Mandatory source-record IDs, metric values, aggregation unit, global/local ablation ID, matched-comparison key, metric-dictionary ID, diagnostic-only flag, dashboard/export lineage, negative-result note, and claim-evidence pointer; optional stress/policy/regime/embodiment identifiers.	Validate source lineage, matching, metric definitions, warning inheritance, and claim boundary before thesis use; store in records/evaluation diagnostic-only if matching fails.

RegimeStateRecord	Layer 5 / E	Created by RegimeRisk; consumed by IHARQ, policy, simulation, embodiment, dashboard, and evaluation scripts.	Regime inter-val/session	Mandatory record/time identity, regime label/probability, regime risk score, risk trend, volatility and drawdown scores, drift/fatigue-like/dropout risk scores, failure-risk score, trust-score summary, trusted set, stop-loss status, recommendation, reason codes, and provenance; optional trust-state links, stress/transition/policy/embodiment IDs, recovery status, and evaluation labels.	Validate temporal ordering, rolling-window scope, no future information, proxy/simulation/evaluation tags, and source-record lineage; store in records/regime_state visible in regime timelines and RegimeCards; temporal trust proxy only.
TrustStateRecord	Layer 5 / E	Created by RegimeRisk when independent trust timelines are exported; consumed by policy, dashboard, and optional IHARQ/simulation/embodiment references through linked RegimeStateRecord fields.	Trust target over regime inter-val/session	Mandatory source RegimeStateRecord ID, dataset/subject/session/target/entity identity, trust target type/ID, trust score, trend, band, trusted-set membership, update source, decay/recovery state, reason codes, and provenance.	Validate source-regime lineage, causal update cutoff, target/window/time identity, update source, and simulation/evaluation tag separation; store in records/trust_state or run-local results/regime_risk where exported; visible in trust-state plots; not a clinical diagnosis.

<code>regime_timeline.csv</code>	Layer 5 / E	Created by regime classifier and feedback adapter; consumed by Layer 10, RegimeCards, thesis figures, and matched-comparison reports.	Session/regime interval	Mandatory time index, regime label, risk score, risk trend, recovery/transition marker, source record IDs, config hash, stress/proxy tag where applicable, and claim boundary.	Validate temporal ordering and source linkage; store under results/regime_risk visible in regime timeline views and temporal-failure audits.
<code>trust_scores.csv</code>	Layer 5 / E	Created by trust allocator; consumed by IHARQ where allowed, policy context, Layer 10 trust-state views, RegimeCards, and thesis summaries.	Trust target / time interval	Mandatory trust target type/ID, score, trend, band, trusted-set membership, update source, source regime/trust record IDs, reason codes, and provenance.	Validate target identity, update source, causal cutoff, and linkage to TrustStateRecord when exported; store under results/regime_risk not a clinical trust guarantee.
<code>stop_loss_warnings.csv</code>	Layer 5 / E	Created by stop-loss module; consumed by IHARQ, policy, simulation, embodiment, dashboard, failure taxonomy, and claim-safety reporting.	Warning/event	Mandatory warning status, trigger, severity, affected source IDs, recommendation, cooldown/recovery state, timestamp or simulated step, and simulation/proxy boundary label.	Validate trigger provenance and conservative-action routing; store under results/regime_risk visible in stop-loss views and RegimeCards.
<code>ReadinessContextRecord</code>	Layer 6 / F	Assembled from Layers 2–8; consumed by policies.	Policy step	Mandatory policy-time features, identity fields, IHARQ/regime context, action history, legal action mask; optional stress/embodiment state.	Validate completeness and separation of evaluation-only targets; store in policy records; visible in controller cards; governs learned-policy claims.

PolicyActionRecord	Layer 6 / F	Created by policy engine; consumed by simulation, embodiment, dashboard.	Policy step	Mandatory policy ID, policy family, context ID, source evidence IDs, action, selected threshold where used, action probability or score, legality mask, reason code, expected cost/reward, action-space version, cost-model ID, reward-model ID, training stage, config hash, claim boundary, and simulation-only flag; optional exploration flag.	Validate action legality, source-context hash, cost/reward config, action-space version, and claim boundary; store in records/policy_records ; visible in action distributions and policy-matrix exports; not autonomous clinical control.
DeferralRecord	Layer 6 / F	Created by learning-to-defer/policy; consumed by simulation, embodiment, dashboard.	Policy step	Mandatory defer decision, target, reason, fallback availability, cost comparison; optional supervisor-proxy flag.	Validate deferral target existence; store with policy records; visible in deferral curves; supports routing not diagnosis.
ClosedLoopSessionRecord	Layer 7 / G	Created by environment session runner; consumed by dashboard, reproducibility appendices, and thesis simulation summaries.	Session/episode	Mandatory dataset/session identity where available, policy ID, stress profile ID, seed, replay mode, episode length, environment version, configuration hashes, update/frozen mode, and claim boundary.	Validate split/session separation, environment identity, seed, config hashes, and update/frozen mode; store in results/layer7_closedloop ; visible in session timelines and environment cards; simulation-only.

EpisodeSummaryRecord	Layer 7 / G	Created by rollout summarizer; consumed by Layer 10, evaluation scripts, and thesis result tables.	Rollout/session summary	Mandatory global/local ablation IDs, metric-dictionary ID, matched-comparison key, diagnostic-only flag where applicable, reward/cost summary, safety-event counts, policy family, stress/embodiment mode, and claim-evidence pointer.	Validate matched-comparison linkage and diagnostic-only status; store in results/layer7_closed-loop visible in policy-comparison and matched-comparison views; supports only declared simulation claims.
PolicyRolloutRecord	Layer 7 / G	Created by policy rollout logger; consumed by Layer 6 policy analysis, ablation scripts, dashboard, and reproducibility manifests.	Policy/session profile	Mandatory policy ID/version, session ID, stress profile/schedule ID where applicable, frozen/update mode, transition IDs, reward-trace IDs, evidence-request/delay behavior, safety-event links, optional embodiment linkage, and claim boundary.	Validate action sequence completeness, transition/reward linkage, update-mode separation, and optional embodiment links; store with closed-loop rollout records; visible in policy rollout cards and comparison views.

RewardTraceRecorder	Layer 7 / G	Created by reward/cost decomposer; consumed by bandit/RL, policy analysis, dashboard, ablations, and reproducibility manifests.	Policy step/episode	Mandatory reward value, reward terms, reward version, source transition ID, action, state/next-state linkage, seed, config hash, update/frozen mode, and simulation-only warning; optional regret estimate and embodiment cost terms.	Validate declared reward version, trace-to-transition linkage, no post-hoc reward mutation, and reward-hacking warnings; store with Layer 7 reward traces; visible in reward/regret and reward-decomposition plots; simulation-only and distinct from Layer9RewardTrace.
EnvironmentDiagnosticRecord	Layer 7 / G	Created by environment validator; consumed by debugging, reproducibility, dashboard warning panels, and claim-safety review.	Run/session/manifestation group	Mandatory validation warnings, invalid actions, failed transitions, reward anomalies, leakage checks, unmatched-comparison warnings, missing safety-event/reward-trace links, missing or mismatched demo-manifest IDs, and disabled-update audit status.	Validate before dashboard/thesis use; store under Layer 7 diagnostics; visible as warning panels and environment-diagnostic cards; blocks unsupported simulation claims when unresolved.

PolicyUpdateTrace	Layer 6 / F and Layer 7 / G	Created only by simulation-authorized contextual-bandit/RL update execution or disabled-update audits; consumed by policy analysis, dashboard, ablation scripts, and reproducibility manifests.	Policy update step	Mandatory policy ID, policy family, context ID, state summary, action, reward, next-state ID, update step, exploration mode, training-or-evaluation mode, frozen-policy flag, simulator ID, seed, config hash, and simulation-only flag.	Validate update-enabled versus frozen-evaluation mode, TransitionRecord .po linkage, disabled-update audit status, and no mutation during frozen-policy evaluation; store in records/policy_updates or results/layer7_close visible in policy-update trace cards and Layer 6 policy-matrix exports.
TransitionRecord	Layer 7 / G and Layer 9 / H	Created by simulation and optionally extended or linked by embodiment; consumed by bandit/RL only when simulation feedback is explicitly enabled, and by Layer 10 dashboard/ablation views.	Policy step/episode or embodiment-linked step	Mandatory previous state, validated readiness action, optional source Layer 4 IHARQ action token, next state, reward pointer, done flag, environment ID, stress ID where active, embodiment outcome/state pointer where active, and policy-update trace ID only for authorized update/audit modes.	Validate environment identity, reproducible seed, update/frozen mode, action-token separation, and embodiment linkage; store in Layer 7 transition records or in records/embodiment , for Layer 9-linked transitions; visible in simulation and embodiment curves; not real-world outcome.

StressProfileRecord	Layer 8 / G	Created by StressForge; consumed by Layer 7 simulation, Layer 5 RegimeRisk summaries where stress context is evaluated, Layer 9 embodiment stress injection, and Layer 10 read-only dashboard/card/reproducibility views.	Stress episode/run	Mandatory stress run/profile IDs, stress family/name/version, intensity label/numeric value, injection point, target layer/record/field, pre/post stressed-value pointers, seed, schedule linkage, clean-baseline pairing, claim boundary, and output links; optional combined-stress label and invalid/impossible/unsafe simulation flags.	Validate clean-record pointer, target-record legality, seed replayability, and claim boundary; store in <code>records/stress_reco</code> visible in StressCard and A12/A13 lineage views; synthetic/proxy stress only.
StressScheduleRecord	Layer 8 / G	Created by StressForge scheduler; consumed by Layer 7 simulation, Layer 9 stress-conditioned embodiment, Layer 10 schedule inspection, and A12/A13 matched comparisons.	Episode/time step	Mandatory parent stress profile ID, schedule type, onset/duration, per-window or per-step intensity curve, active component stressors, affected variables, timestamp or simulated step index, seed, target layer/record linkage, and clean-baseline pairing ID where applicable; optional randomization rule.	Validate replayability by seed, target compatibility, and matched-comparison linkage; store with stress profiles; dashboard stress timelines and reproducibility manifests; no biological fatigue claim.

EmbodimentStateRecord / H	Created by L9.1–L9.12 embodiment session, validator, context, mapper, safety gate, adapter, stress injector, outcome bridge, and logger modules; consumed by Layer 10 and by Layers 6/7 only under declared simulation-only policy-analysis modes.	Embodiment step	Mandatory embodiment session ID, simulator/static-replay state reference, source IHARQ/regime/policy/context/stress IDs, mapping ID, preserved source IHARQ action token, policy action where enabled, mapped command, readiness/safety-gate state, safety block reason, simulator/adapter version, endpoint state, reward terms where enabled, ablation/matched-comparison lineage, seed or nondeterminism note; optional kinematic variables.	Validate simulator version, readiness gate, action-token separation, copy/context/stress lineage, stress compatibility, and non-clinical tag; store in records/embodiment, visible in EmbodimentCard and Layer 10 embodiment page; simulation-only.
---------------------------	--	-----------------	---	---

EmbodimentOutcomeRecord	Layer 9 / H or Layer 7 Cs where applicable	Created by embodiment evaluation/outcome bridge; consumed by Layer 10 and simulation-only summaries.	Embodiment step/episode/embodiment step	Mandatory comparison error, command success/failure, over/under-assist flags, smoothness proxy, stress sensitivity, safety event pointer, source transition, command-timeline pointer, endpoint-curve pointer, accepted-missingness reason where applicable, A13 outcome provenance, diagnostic-only flag, and non-clinical claim boundary; optional screenshots and reward-trace pointer.	Validate output-source linkage, reward-feedback labeling, accepted-missingness status, and claim boundary; store in records/embodiment , visible in demo panels and A13 views; not clinical efficacy evidence.
SafetyEventRecord or safety_event_log	Layer 9 / H and Layer 7 Cs where applicable	Created by readiness/safety gates and simulation environment; consumed by Layer 10, failure taxonomy, and optional policy/simulation analysis.	Safety event / embodiment step / simulation step	Mandatory event ID, source policy/IHARQ/regime/sensor/condition IDs, blocked or allowed command, unsafe/fallback/request reason, safety state, response, timestamp/step, safety-gate version; optional simulator state pointer.	Validate gate execution and completeness; store with embodiment or simulation logs; visible in unsafe-event summaries and failure taxonomy; supports safety-control evidence but not clinical safety.

Layer9RewardTrace	Layer 9 / H	Created by the Layer 9 outcome/reward bridge when reward-enabled embodiment is configured; consumed by Layer 10 and optional simulation-only policy analysis.	Step/episode reward decomposition	Mandatory reward version, reward terms, source transition/embodiment IDs, action, state/next-state linkage, seed, config hash, reward-feedback enabled status, reward-hacking warning status, and simulation-only tag; optional burden terms.	Validate reward version and no post-hoc clinical interpretation; store with Layer 9 transition/embodiment outputs; visible through DashboardMetricRecord simulation-only. Layer 7 policy rewards remain RewardTraceRecord.
Layer9EmbodimentDemoManifest	Layer 9 / H	Created by dashboard/card/export packager; consumed by Layer 10 and reproducibility package.	Embodiment run/export bundle	Mandatory run ID, embodiment mode, adapter branch, simulator/static-replay version, command-mapping ID, config hashes, source-record paths, command timeline, endpoint-error curve, safety logs, reward-trace pointer where enabled, screenshot/video references where generated, figure/table IDs, export path, warning tags, accepted-missingness markers, and file hashes.	Validate bundle completeness, accepted-missingness status, and stale-simulator status; store with manifests; visible in embodiment export index, card/export page, and reproducibility package.

DashboardMetricRecord	10 / I	Created by dashboard metric summarizer; consumed by dashboard views, thesis figures, cards, export manifests, and reproducibility manifests.	Metric/view/export	Mandatory metric ID, source record IDs, source layers, aggregation config, filters, run/config hashes, ablation step, matched-comparison ID, metric-dictionary reference, diagnostic-only status, negative-result note reference, claim-evidence ID, warning tags, card references, and export references; optional <code>policy_update_filter</code> , <code>embodiment_filter</code> , <code>reward_trace_filter</code> , stress filter, and A12/A13 lineage filters.	Validate source lineage, frozen/update separation, stress/embodiment/relabeling, matched-comparison status, and no hidden aggregation; store in <code>records/dashboard_</code> visible in dashboard exports.
DashboardViewRecord	10 / I	Created by dashboard engine; consumed by thesis exports, card generation, artifact manifests, reproducibility manifests, and claim-safety review.	View/report	Mandatory view ID, page role, panel IDs, filters, source <code>DashboardMetricRecord</code> IDs, source records, metric definitions, evaluation-lineage references, warning banners, completeness state, diagnostic-only state, export path, thesis usage location, generation config, code version, and generation timestamp; optional user notes.	Validate read-only source lineage and export completeness; store in <code>records/dashboard_v</code> visible as dashboard view; interpretability only.

ModelCard, ReliabilityCard, RegimeCard, ControllerCard, StressCard, EmbodimentCard	Layer 10 / I	Generated from validated non-Layer-1 records; consumed by thesis and appendix. Layer 1 DatasetCard and ProtocolCard are produced by Layer 1 and rendered/exported read-only by Layer 10.	Card/report	Mandatory source records, limitations, metrics, config hashes, warning tags; optional plots.	Validate card-record consistency; store in cards ; dashboard visible; carries claim-safety text without overwriting Layer 1 card content.
FailureTaxonomy	Layer 10 / I	Created from failure audits, high-confidence wrong cases, safety events, stress failures, adapter failures, reward-dominated findings, and missingness warnings; consumed by thesis discussion and claim-safety review.	Failure class/report	Mandatory failure mode, source record, source layer, detection rule, response, frequency or severity where computed, affected claim, mitigation or limitation, diagnostic-only status, and representative examples where allowed.	Validate source cases, warning inheritance, and claim consequences; store in reports; visible in dashboard; strengthens limitation analysis without becoming a decision-time feature.
ReproducibilityPackage	Layer 10 / I	Created by export manager and reproducibility manifest builder; consumed by thesis submission, artifact review, examiner inspection, repository release, and future reruns.	Project/run	Mandatory configs, manifests, seeds, code version, environment, source records, dashboard metric/view records, figures, tables, cards, run registry, checksums, claim-evidence links, and rerun commands; optional notebooks.	Validate completeness, checksums, lineage, and simulation/stress/embodiment warning propagation; store in export package; thesis appendix; protects reproducibility claim.

18.7 Granularity Alignment Map

Many implementation errors occur because records with different granularities are mixed without explicit aggregation. The architecture therefore treats granularity as part of the interface contract. Calibration, IHARQ, RegimeRisk, policy learning, StressForge-Lite, embodiment, and dashboard reports must not silently combine dataset-level, subject-level, trial-level, window-level, decision-level, regime-level, and simulation-step-level fields.

Granularity level	Typical records	Writers and readers	Aggregation rule
Dataset-level	DatasetRecord , dataset card, protocol summary	Written by Layer 1; read by all layers and dashboard.	May be aggregated only across declared datasets with dataset IDs preserved.
Subject-level	SubjectProfileRecord , subject difficulty summaries, subject-level trust summaries	Written by Layers 2 and 5; read by policy and dashboard.	Must preserve subject identity and avoid train/test subject leakage.
Session-level	Session split entries, regime timelines, session simulation summaries	Written by Layers 1, 5, 7; read by policy, dashboard, ablations.	Must preserve session ordering for temporal analyses.

Run-level	ProtocolCard, saved split manifests for SplitRecord, ValidationReport, Layer1Manifest, layer1_ablation_readiness_l1_v1.json, model registry, reproducibility manifest	Written by experiment controller and Layers 1–10.	Defines immutable version context, validation status, and ablation-readiness status for downstream records.
Trial-level	Parent-trial lineage fields inside WindowRecord, trial predictions, trial decisions, trial outcome summaries	Written by Layers 1–4 and 7–9.	Window predictions may aggregate to trial only by declared rule and must preserve parent-trial lineage.
Window-level	WindowRecord, PredictionRecord, CalibrationRecord, UncertaintyRecord	Written by Layers 1–3; read by Layers 4–6 and dashboard.	Must not mix overlapping windows without overlap group handling.
Prediction-level	PredictionRecord, BaselineMetricRecord rows	Written by Layer 2; read by Layers 3–6.	Evaluation label can be present but must be separated from decision-time policy inputs.

Decision-level	EvidenceQualityRecord, IHARQDecisionRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, SelectivePredictionRecord decisions	Written by Layers 3–4; read by Layers 5–7, Layer 9 where simulation/proxy gating applies, evaluation scripts, and dashboard.	One decision must point to its source predic- tion/window/trial, thresh- olds, action token, reason code, ablation IDs where evalu- ated, and claim bound- ary.
Regime-interval level	RegimeStateRecord, conditional TrustStateRecord, regime_timelines.csv, trust_scores.csv, stop_loss_warnings.csv, regime transition logs, and threshold-adjustment logs	Written by Layer 5; read by Layers 4, 6, 7, 9, 10 and evaluation scripts.	Rolling sum- maries must specify window length, temporal cutoff, source- record IDs, simula- tion/proxy tags where ap- plicable, and must use no future samples.

Policy-step level	ReadinessContextRecord , expanded PolicyActionRecord , DeferralRecord , optional PolicyUpdateTrace	Written by Layer 6 and, for update execution traces, Layer 7; read by simulation, embodiment, dashboard, ablation scripts, and reproducibility manifests.	Context must be constructed before action; targets/rewards must be separated; policy mutation must be trace-linked and absent from frozen-policy evaluation except disabled-update audit entries.
Stress-episode level	StressProfileRecord , StressScheduleRecord , stressed-record pointers	Written by Layer 8; read by Layers 5, 7, 9, 10.	Every stressed record must point back to a clean record and stress schedule.

Embodiment- step level	<code>EmbodimentStateRecord</code> , <code>EmbodimentOutcomeRecord</code> , <code>SafetyEventRecord</code> or safety-event log, command timeline, endpoint-error curve, <code>Layer9RewardTrace</code> where enabled, <code>Layer9EmbodimentDemoManifest</code>	Written by Layer 9; read by Layer 10 dashboard/reproducibility and optional simulation-only policy analysis.	Simulation- only and non- clinical labels, Layer 9 owner- ship, demo- manifest linkage, and accepted- missingness markers must be attached to every output. All views must list source records, source metric records, filters, metric defini- tions, matched- comparison/diagnostic- only status where relevant, claim- evidence linkage, export refer- ences, and warnings.
Dashboard/report- level	<code>DashboardMetricRecord</code> , <code>DashboardViewRecord</code> , cards, plots, tables, <code>FailureTaxonomy</code> , export manifests, and <code>ReproducibilityPackage</code>	Written by Layer 10; read by thesis/human reviewer.	

If an implementation aggregates from a finer level to a coarser level, the aggregation function, grouping keys, split context, and excluded records must be saved. If an implementation expands a coarser context to a finer decision level, the inherited fields must be explicitly copied or referenced and must not be mistaken for measured per-window evidence.

18.8 Boundary Validation and Quality Gates

Each major boundary has a quality gate. The gate may be implemented as schema validation, config validation, unit tests, manifest checks, dashboard warnings, or experiment-controller assertions. A downstream module should fail closed when a required gate fails.

Boundary	Required quality gate	Failure response and artifact
Dataset → WindowRecord	Check dataset identity, subject/session/run labels, channel metadata, sampling rate, task labels, label-map version, split rules, <code>label_budget_id</code> , preprocessing scope/version, artifact flags, and leakage prevention.	Stop window generation or mark excluded windows; write <code>ArtifactFlagRecord</code> , <code>DatasetCard</code> warning, protocol manifest note, and Layer 1 ablation-readiness warning when matched-comparison keys are incomplete.
WindowRecord → PredictionRecord	Check model configuration, training split, <code>label_budget_id</code> , seed, preprocessing ID/version, model identity, class map, artifact flags, window overlap group, <code>score_type</code> , <code>class_order</code> , and <code>ModelRegistryRecord</code> linkage.	Block decoder run or write invalid/failed-run status; update <code>ModelCard</code> , <code>BenchmarkCard</code> , baseline test logs, and <code>Layer2ReadinessReport</code> .
PredictionRecord → CalibrationRecord / UncertaintyRecord / ThresholdRegistryRecord	Check calibration split, threshold-validation split, probability/logit/decision-score availability, <code>score_type</code> , <code>class_order</code> , model registry match, calibration eligibility, and test-set leakage.	Block calibration/threshold selection, label method as score-only or invalid, write <code>SplitIntegrityReport</code> , <code>LeakageWarningRecord</code> , <code>CalibrationEligibilityTable</code> , and <code>ReliabilityAuditReport</code> warnings.
CalibrationRecord / UncertaintyRecord / ThresholdRegistryRecord → Layer 4 IHARQ records	Check calibrated confidence, entropy, margin, agreement, stability, threshold source, model family, action vocabulary, non-code completeness, and evaluation-only field separation.	Block IHARQ decision or produce unsafe/uncertain with reason code; write <code>IHARQDecisionRecord</code> , <code>IHARQTraceRecord</code> , and where triggered <code>CombinedEvidenceRecord</code> , <code>FallbackDecisionRecord</code> , or <code>UnsafeEventRecord</code> .

Layer 4 IHARQ records → RegimeStateRecord / TrustStateRecord	Check temporal ordering, rolling-window length, session identity, subject identity, previous-decision availability, action-token validity, unsafe/fallback/request history, and aggregation logic.	Exclude invalid intervals, reset regime state, mark trust unknown, or label the comparison diagnostic-only; write <code>regime_timelines.csv</code> , <code>trust_scores.csv</code> or <code>TrustStateRecord</code> where exported, <code>stop_loss_warnings.csv</code> where triggered, and <code>RegimeCard</code> warning.
RegimeStateRecord → PolicyActionRecord	Check regime validity, readiness-context completeness, action history, cost/reward configuration, legality mask, and action-space constraints.	Block policy action or force fallback/unsafe under declared rule; write <code>ControllerCard</code> warning.
TrustStateRecord → PolicyActionRecord	Check source <code>RegimeStateRecord</code> link, causal cutoff, trust target identity, and declared trust-timeline feature use.	Block trust-timeline policy features or mark policy comparison diagnostic-only when trust-state lineage is missing or stale; write <code>TrustStateCard/ControllerCard</code> warning.
PolicyActionRecord → TransitionRecord	Check simulated environment identity, action semantics, reward definition, seed, stress condition, done-state logic, update mode, and optional <code>TransitionRecord.policy_update</code>	Stop simulation episode or mark transition invalid; write simulation manifest warning; require <code>PolicyUpdateTrace</code> if an update-enabled policy mutation occurs.
StressProfileRecord → stressed records	Check stress family, intensity, injection point, schedule, seed, clean-record pointer, and target-module compatibility.	Block stress injection or label stress profile invalid; write <code>StressCard</code> warning and preserve clean records.
PolicyActionRecord → EmbodimentStateRecord / safety-event log / Layer9EmbodimentDemoManifest	Check record-intake validity, readiness-context completeness, command mapping, simulation-only scope, embodiment mode, readiness gate, safety block, simulator/static-replay version, stress compatibility, endpoint-metric availability, reward-trace labeling, and demo-manifest completeness.	Block command, create safety-event record, run static replay only, mark accepted missingness, or block Layer 10 embodiment export; write <code>EmbodimentCard</code> warning.

All records → dashboard	Check artifact completeness, versioning, source-record lineage, config hashes, warning tags, card consistency, optional <code>PolicyUpdateTrace</code> completeness, <code>DashboardMetricRecord.policy_update_filter</code> , frozen/update curve separation, and no silent mutation of experimental records.	Hide or warn on incomplete views; export only validated figures/tables; write <code>DashboardMetricRecord</code> , <code>DashboardViewRecord</code> , <code>PolicyUpdateTrace</code> , trace card where applicable, and manifest.
----------------------------	---	---

18.9 Failure, Ambiguity, and Contradiction Map

The following table defines interface failures that must be detected before a run is considered thesis-ready. The required response is architectural, not cosmetic: the system must either block the run, mark the artifact invalid, degrade to a safe/uncertain/fallback state, or expose the limitation in the dashboard and thesis artifacts.

Failure mode	Where it occurs	Why it matters and detection	Required architectural response	Claim-safety consequence
Missing metadata	Layer 1 boundary	Subject/session/channel context cannot be trusted; detect by schema validation.	Block affected records or mark exclusions with <code>ArtifactFlagRecord</code> .	Dataset claims weakened or excluded.
Inconsistent subject/session split	Layer 1–2	Leakage invalidates low-calibration and cross-subject claims; detect split-ID conflicts.	Stop run and regenerate split manifest.	Benchmark claims invalid until fixed.
Window without valid label	Windowing	Prediction/evaluation target is ambiguous; detect label-map lookup failure.	Exclude window or assign declared rest/no-action policy if valid.	Prevents class-level overclaim.
Prediction without model identity	Layer 2–3	Calibration and dashboard cannot trace source; detect missing model ID, decoder ID, registry path, score type, or class order.	Reject prediction record, update <code>ModelRegistryRecord</code> failed/invalid status, <code>ModelCard</code> warning, and <code>Layer2ReadinessReport</code> .	Baseline evidence unusable.

Malformed score schema or hidden aliases	Layer 2–3	Calibration and uncertainty may read the wrong field; detect missing probabilities/logits/decision score declaration, class_order mismatch, or unmapped aliases such as y_true , y_pred , and prob_vector . IHARQ may overtrust raw scores; detect missing calibration method/version.	Repair Layer 2 export, mark score-only fallback, or exclude model decision-registry-visible reason; write CalibrationEligibilityTable and reliability-audit warning.	Calibration/uncertainty claims become diagnostic-only for affected IntegrityTable
Uncalibrated confidence treated as calibrated confidence	Layer 3–4	IHARQ may overtrust raw scores; detect missing calibration method/version.	Use raw-score branch with warning or block calibrated IHARQ mode.	Reliability claim limited.
Threshold chosen on test data	Layer 3–6	Selective/IHARQ/policy results become biased; detect threshold source split and ThresholdRegistryRecord	Invalidate threshold, write SplitIntegrityReport and rerun selection on validation or cross-validation split.	Affected results support thesis claims.
IHARQ decision without reason code	Layer 4	Readiness decision is not auditable; detect missing code/text.	Mark decision invalid or unsafe/uncertain with missing-reason warning.	IHARQ evidence claim weakened.
RegimeRisk using future information	Layer 5	Temporal trust becomes non-causal; detect timestamps and rolling-window sources.	Recompute using prior/current records only; flag invalid timelines.	Regime improvements invalid until rerun.
Policy trained on evaluation rewards	Layer 6–7	Learned policy overfits held-out simulation; detect reward split mismatch.	Discard policy artifact and retrain under declared train/validation/test sessions.	Policy-learning claim invalid.
Stress profile without clean-record pointer	Layer 8	Stressed results cannot be traced or compared; detect missing source IDs.	Reject stress output and preserve clean record.	Stress robustness claim unusable.

Embodiment action without readiness gate or incomplete Layer 9 bundle	Layer 9 / Layer 10	Simulated command bypasses architecture safety logic, lacks source context, lacks safety-event log, lacks <code>Layer9EmbodimentDemoManifest</code> , or carries stale simulator metadata; detect missing gate/context/manifest/export records.	Block command, write safety-event log, mark accepted missingness, or block dashboard embodiment export.	Embodiment demonstration invalid or incomplete until re-run/export repair.
Dashboard plot without source record	Layer 10	Figure cannot be reproduced; detect export manifest gap.	Hide/export-block plot until lineage is restored.	Figure cannot appear as thesis evidence.
Clinical claim implied from simulation-only output	Layers 0, 7–10	Scope violation; detect claim text/card warnings.	Rewrite claim as simulation/proxy result and add limitation.	Prevents unsupported clinical claim.
Contradictory record versions	Any boundary	Mixed configs make results non-reproducible; detect config hash conflicts.	Create separate run group or fail validation.	Reproducibility claim weakened.
Evaluation target included in decision context	Layers 4–7	Policy/IHARQ can leak ground truth; detect protected target fields.	Strip evaluation-only fields from decision-time context and rerun.	Decision-policy claim invalid if not fixed.

18.10 Proximity Map for Ablation Studies

The proximity map supports ablations because it identifies exactly which records are required to remove, replace, or compare a module. Each ablation must use matched datasets, splits, label budgets, seeds where feasible, evaluation metrics, and source-record manifests.

Ablation question	Required inputs	Expected outputs and metrics	Thesis artifact
Decoder-only versus calibrated decoder	<code>PredictionRecord</code> , <code>ModelRegistryRecord</code> , <code>BaselineMetricRecord</code> , <code>CalibrationRecord</code> , <code>SplitIntegrityReport</code> , same splits and threshold/calibration eligibility	Accuracy, ECE, Brier, NLL, high-confidence wrong rate, reliability diagrams, diagnostic-only downgrade status	Baseline table, reliability diagram, calibration summary, and readiness/split- integrity warning if applicable.

Calibrated decoder versus IHARQ	CalibrationRecord, UncertaintyRecord, ThresholdRegistryRecord, IHARQDecisionRecord	Accepted accuracy, coverage, risk, unsafe/wait/fallback counts, reason-code distribution	IHARQ ablation table and decision plots.
IHARQ without RegimeRisk versus with RegimeRisk	IHARQ records, RegimeStateRecord, TrustStateRecord	Matched coverage/cost, temporal failure rate, high-risk accepted error, wait/fallback burden	Regime-aware IHARQ table and regime timeline.
Rule-based IHARQ versus learned evidence-quality estimator	Rule IHARQ records, learned EvidenceQualityRecord, target-construction card	Reliability/failure prediction, accepted risk, coverage, calibration of evidence-quality score	Evidence-quality ablation and ReliabilityCard.
Supervised Adaptive-IHARQ versus learning-to-defer Static thresholds versus contextual bandit adaptation	ReadinessContextRecord, pseudo-labels, DeferralRecord, cost configs ThresholdRegistryRecord, bandit contexts, RewardTraceRecord, TransitionRecord	Action accuracy, cost, deferral rate, fallback burden, unsafe reduction Reward, regret, coverage, wrong-accept cost, adaptation stability	Policy ablation ladder and ControllerCard.
Bandit policy versus RL policy in simulation	Shared simulation environment, action space, reward version, held-out sessions	Episode reward, delayed unsafe events, waiting burden, fallback burden, regret if defined	Bandit reward/regret curves and controller card.
Clean conditions versus StressForge-Lite conditions	Clean source records, StressProfileRecord, StressScheduleRecord, stressed transitions	Robustness drop, stress sensitivity, recovery, failure modes, regime warnings	Simulation-only policy comparison table.
No embodiment versus simulated embodiment adapter	PolicyActionRecord, TransitionRecord, EmbodimentStateRecord, EmbodimentOutcomeRecord, SafetyEventRecord or safety-event log, command timeline, endpoint-error curve, Layer9EmbodimentDemoManifest, Layer9RewardTrace where enabled	Endpoint error, command success, safety-event count, no-action/fallback burden, command-timeline behavior, reward-term modification where enabled	Stress sensitivity curves and StressCard.
Dashboard-only reporting versus full reproducibility package	Dashboard views, cards, export manifests, configs, records, seeds	Figure traceability, rerun completeness, card-record consistency, missing artifact count	Embodiment demonstration panel, unsafe-event summary, command-timeline plot, endpoint-error plot, demo-manifest index, and non-clinical card.
			Reproducibility appendix and export manifest.

18.11 Proximity Map for Dashboard and Thesis Artifacts

Dashboard and thesis outputs are not independent scientific sources. They are read-only projections from saved records. Each artifact must trace back to exact source records, configuration hashes, split IDs, model IDs, policy IDs, stress IDs, simulator IDs where applicable, and warning tags.

Dashboard/thesis artifact	Source records	Purpose and claim boundary
Dataset cards and protocol cards	DatasetRecord, DatasetCard, ProtocolCard, SplitRecord, PreprocessingRecord, LabelMapRecord, ValidationReport, Layer1Manifest	Document public-data scope, split rules, preprocessing/windowing IDs, validation status, exclusions, limitations, allowed/forbidden claims, and diagnostic-readiness warnings without creating new experimental evidence.
Baseline tables	PredictionRecord, BaselineMetricRecord, ModelRegistryRecord, Layer2ReadinessReport	Report decoder measurement performance, schema/registry coverage, and downstream compatibility, not readiness safety.
Low-calibration curves	LowCalibrationCurveRecord, BaselineMetricRecord, split/budget configs, Layer2ReadinessReport	Show calibration-burden behavior under fixed protocols while preserving label-budget and diagnostic-only status.
Layer 2 model and benchmark cards	ModelCard, BenchmarkCard, ModelRegistryRecord, BaselineMetricRecord, LowCalibrationCurveRecord, SubjectProfileRecord, EnsembleControlRecord, failed-run logs, and Layer2ReadinessReport	Summarize decoder/model evidence and A0/A4 ordinary controls without making readiness-safety claims.
Reliability diagrams and ECE/Brier/NLL summaries	CalibrationRecord, UncertaintyRecord, ReliabilityAuditReport, CalibrationDashboardMetricRecord	Show calibration quality and uncertainty limitations.
High-confidence-wrong audits	PredictionRecord, CalibrationRecord, UncertaintyRecord, HighConfidenceWrongTable, true labels for evaluation only	Expose failure cases hidden by accuracy without turning test-derived error flags into decision-time features.
Risk-coverage curves and matched operating points	SelectivePredictionRecord, ThresholdRegistryRecord, Layer3MatchedOperatingPointTable, CalibrationDashboardMetricRecord	Baseline for selective prediction and IHARQ comparisons under matched coverage/cost and registered threshold lineage.

Layer 3 reliability cards and threshold views	ReliabilityCard, CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, SplitIntegrityReport, ReliabilityAuditReport, CalibrationDashboardMetricRecord	Package calibration, uncertainty, selective-prediction, threshold, and split-integrity evidence for read-only reporting with warning inheritance.
IHARQ decision distributions	IHARQDecisionRecord, EvidenceQualityRecord	Show readiness actions and accepted/rejected evidence behavior.
Reason-code frequency tables	IHARQDecisionRecord, policy reason logs, safety-event logs	Make decisions auditable and interpretable.
Regime timelines and trust-state plots	RegimeStateRecord, TrustStateRecord, IHARQ histories	Show temporal reliability proxy behavior, not clinical fatigue diagnosis.
Policy action distributions	ReadinessContextRecord, PolicyActionRecord, DeferralRecord	Show learned or rule-based readiness-control behavior under declared costs.
Layer 7 closed-loop timelines and rollout cards	ClosedLoopSessionRecord, PolicyRolloutRecord, TransitionRecord, RewardTraceRecord, EpisodeSummaryRecord, EnvironmentDiagnosticRecord, SafetyEventRecord, optional PolicyUpdateTrace, stress/reward/action configs, and optional embodiment links	Show simulation-bounded sequential consequences, evidence-request/delay behavior, reward decomposition, update/frozen separation, invalid-action diagnostics, safety-event lineage, and policy-rollout assumptions without treating simulated outcomes as clinical validation.
Layer 6 policy matrices	ReadinessContextRecord, expanded PolicyActionRecord, DeferralRecord, optional PolicyUpdateTrace, cost/reward configs, legality-rule configs, action-space registry, policy registry, claim-boundary warnings	Show policy-stage ownership, context feature groups, legal/illegal actions, cost/reward sensitivity, policy-to-layer dependencies, and claim boundaries after the Layer 6 policy-learning merge.
Reward/regret curves	RewardTraceRecord, TransitionRecord, PolicyRolloutRecord, EpisodeSummaryRecord, optional PolicyUpdateTrace, TransitionRecord.policy_update_training_adaptation, policy registry, and reward configuration	Report simulation-only policy learning behavior, reward/cost decomposition, and rollout-level summaries while separating training/adaptation curves from frozen-policy evaluation curves.

Policy-update trace cards and disabled-update audit tables	Optional <code>PolicyUpdateTrace</code> , <code>TransitionRecord.policy_updates</code> , <code>DashboardMetricRecord.policy_updates</code> , policy-update config, disabled-update audit entries	Prove that simulation-authorized updates are traceable and that frozen-policy evaluation remained immutable.
Stress sensitivity curves	<code>StressProfileRecord</code> , <code>StressScheduleRecord</code> , stressed-record pointers, transition metrics	Show robustness under declared synthetic stress.
Embodiment demonstration panels and Layer 9 evidence cards	<code>EmbodimentStateRecord</code> , <code>EmbodimentOutcomeRecord</code> , <code>SafetyEventRecord</code> or safety logs, command timeline, endpoint-error curve, <code>Layer9RewardTrace</code> where enabled, simulator/static-replay config, <code>Layer9EmbodimentDemoManifest</code> , dashboard-compatible export, <code>DashboardMetricRecord</code> lineage, and <code>DashboardViewRecord</code> export metadata	Visualize simulated proxy consequences through Layer 10 read-only views; never real clinical validation, real FES/exoskeleton evidence, patient-specific biomechanics, or medical-device readiness.
Failure taxonomy	Failure cases, boundary-gate logs, dashboard warnings, source records	Explain limitations, ambiguity, and contradiction handling.
Reproducibility package	All configs, manifests, seeds, records, cards, figures, tables, environment summaries	Allows inspection and rerun of thesis evidence where feasible.

18.12 Implementation Ordering Implied by the Map

The proximity map implies an implementation order. The order is dependency-driven rather than ambition-driven. Reinforcement learning, contextual bandit adaptation, StressForge-Lite, and embodiment must not be introduced before the baseline, calibration, IHARQ, and simulation contracts are stable.

1. Implement claim-safety and protocol definitions.
2. Implement dataset ingestion and split manifests.
3. Implement windowing and trial records.
4. Implement baseline decoders and prediction records.
5. Implement calibration and uncertainty records.
6. Implement selective prediction and threshold registry.
7. Implement IHARQ evidence verification.
8. Implement RegimeRisk temporal trust monitoring.

9. Implement adaptive readiness context and supervised policy baseline.
10. Implement learning-to-defer.
11. Implement contextual bandit adaptation.
12. Implement the simulated closed-loop environment.
13. Integrate StressForge-Lite.
14. Introduce RL policy only after simulation state/action/reward validity is established.
15. Introduce the embodiment adapter only after readiness and safety gating exist.
16. Build dashboard and reproducibility package throughout, but finalize exports after stable records exist.

This ordering protects the thesis from a common failure pattern: implementing the most complex policy-learning component before the measurement, uncertainty, decision, and simulation records are trustworthy. In IHARQ BenchGuard, RL is a Stretch C simulation-only module. It is scientifically meaningful only after baseline decoding, calibration, IHARQ, RegimeRisk context, policy action contracts, reward definitions, and transition records are already validated.

18.13 Claim-Safety and Scope-Control Map

The proximity map is also a claim-control mechanism. It keeps public EEG evidence, policy-learning records, simulation rewards, stress perturbations, embodiment outcomes, and dashboard artifacts semantically separated.

Claim category	Allowed and forbidden interpretation
Allowed claims	The architecture may claim that it evaluates reliability, calibration, evidence verification, temporal trust, adaptive readiness, stress robustness, and simulated embodied consequences using public EEG and declared simulated environments. It may compare decoder-only, calibrated, IHARQ, RegimeRisk-aware, deferral, bandit, RL, stress-conditioned, and simulation-only embodiment variants under declared metrics.
Not allowed claims	The architecture must not claim clinical BCI validation, real rehabilitation efficacy, patient safety, implanted control, real FES stimulation, medical-device safety, real-world therapeutic benefit, or real-time patient-ready deployment. Public healthy EEG and simulation-only evidence cannot support those claims.
How the map prevents overclaiming	Public EEG records remain separate from embodiment records; simulation outputs remain labeled as simulation outputs; policy rewards remain declared simulated rewards; stress profiles remain declared perturbations; dashboard cards expose source records and limitations; Layer 0 audits thesis language before final reporting.

The central rule is that evidence provenance determines claim strength. A `PredictionRecord` together with `ModelRegistryRecord`, `BaselineMetricRecord`, and `Layer2ReadinessReport` can support a decoder benchmark and downstream-readiness claim, not a safety

claim. A `CalibrationRecord`, `UncertaintyRecord`, `SelectivePredictionRecord`, `ThresholdRegistryRecord`, `SplitIntegrityReport`, and `ReliabilityAuditReport` can support calibration, uncertainty, threshold, and selective-prediction claims under declared split legality. An `IHARQDecisionRecord` can support a computational readiness/evidence-verification claim. A `RegimeStateRecord` can support a temporal trust proxy claim, and a linked `TrustStateRecord` can support only the narrower claim that a declared per-target trust timeline or trusted-set evolution was recorded and inspected. A `RewardTraceRecord` or `Layer9RewardTrace` can support only a simulation reward claim. An `EmbodimentOutcomeRecord`, `EmbodimentStateRecord`, `SafetyEventRecord`, endpoint-error curve, command timeline, or Layer 9 `Layer9EmbodimentDemoManifest` can support only a simulated proxy embodiment demonstration. A `DashboardViewRecord` can support traceability and interpretability, but it is not a new experimental source.

18.14 Expected Implementation Artifacts

The target implementation should produce the following artifact families as evidence that the proximity map has been respected.

Artifact family	Expected files or tables	Purpose
Configuration artifacts	Dataset configs, protocol configs, decoder configs, calibration configs, IHARQ configs, RegimeRisk configs, policy/reward configs, stress configs, embodiment configs, dashboard configs	Freeze the assumptions used to create records.

Record artifacts	records/dataset_records, Provide inspectable, replayable records/window_records, data movement across layers, records/split_records, including Layer 1 records/preprocessing_records, maintenance/flag lineage, Layer 2 records/label_map_records, baseline and readiness lineage, records/artifact_flag_records, Layer 3 calibra- records/reproducibility_metrics/uncertainty/threshold records/prediction_records, lineage, trace-safe policy records/model_registry, updates, Layer 9 embodiment records/baseline_metric_records, safety events, and records/low_calibration_cultural_records, metric filters. records/subject_profiles, records/ensemble_records, records/calibration_records, records/uncertainty_records, records/selective_prediction_records, threshold registries, split-integrity/leakage reports, records/iharq_records, records/regime_state_records, records/policy_records, records/policy_update_traces, records/transition_records, records/layer7_closed_loop/sessions, records/layer7_closed_loop/episode_summaries, records/layer7_closed_loop/policy_rollouts, records/layer7_closed_loop/reward_traces, records/layer7_closed_loop/diagnostics, records/stress_records, records/embodiment, safety-event logs, Layer9RewardTrace files where enabled, records/dashboard_metric_records, records/dashboard_view_records
Registry artifacts	Split registry, model Make ownership, versions, and registry, threshold registry, allowed consumers explicit. policy registry, reward registry, stress registry, simulator/static-replay registry, command-mapping registry, safety-gate registry

Evaluation artifacts	<code>layer1_ablation_readiness_SupportJobs</code>, thesis experiments and ablations while preserving protocol matchability, downstream-readiness status, calibration/threshold legality, matched operating points, and diagnostic-only downgrade evidence. <code>Layer2ReadinessReport</code>, baseline metrics, low-calibration curves, subject-profile summaries, <code>EnsembleControlRecord</code>, calibration summaries, <code>SplitIntegrityReport</code>, <code>ReliabilityAuditReport</code>, <code>HighConfidenceWrongTable</code>, risk-coverage curves, <code>Layer3MatchedOperatingPointTable</code>, IHARQ ablations, regime timelines, policy comparisons, stress sensitivity, embodiment metrics, command timelines, endpoint-error curves, safety-event summaries	
Reporting artifacts	<code>DatasetCard</code>, <code>ProtocolCard</code>, <code>ModelCard</code>, <code>BenchmarkCard</code>, <code>ReliabilityCard</code>, calibration/uncertainty/selective-prediction/threshold/high-confidence-error cards, <code>RegimeCard</code>, <code>ControllerCard</code>, closed-loop environment card, policy rollout card, policy-update trace card where applicable, reward configuration card, <code>StressCard</code>, <code>EmbodimentCard</code>, Layer 9 embodiment evidence cards/export indexes, failure taxonomy, dashboard views, separated training/adaptation versus frozen-policy evaluation exports, thesis tables, figures	Turn validated records into readable thesis evidence without treating reporting artifacts as new experimental results.

Reproducibility artifacts	Run manifest, <code>Layer1Manifest</code> , <code>layer1_ablation_readiness_public_data</code> , <code>Layer2ReadinessReport</code> , <code>ModelRegistryRecord</code> , <code>SplitIntegrityReport</code> , <code>CalibrationEligibilityTable</code> , threshold registry, config hashes, code version, seeds, environment summary, source-record manifest, export manifest, warning manifest, Layer 9 <code>Layer9EmbodimentDemoManifest</code>	Allow rerun, audit, and defense of results, including public-data/protocol readiness status, decoder-readiness status, and calibration/threshold legality.
---------------------------	---	--

18.15 Layers 0–3 evaluation-slice proximity addendum

The Layers 0–3 evaluation merge adds one precise proximity rule. Evaluation evidence is valid only when the artifact is close enough to the records that produced it to preserve public-data protocol identity, decoder baseline identity, calibration/threshold-selection identity, matched-comparison identity, metric-definition identity, and claim-safety identity. Layer 0 owns the evaluation claim-evidence gate; Layer 1 owns the public-data and protocol matchability keys; Layer 2 owns A0 raw-decoder evidence and A4 ordinary-combination controls; Layer 3 owns A1 calibration, A2 thresholding, and A3 uncertainty/selective-prediction comparisons; Layer 10 reads those records as reporting evidence only.

The nearest records for this slice are `DatasetRecord`, `WindowRecord`, `SplitRecord`, `PreprocessingRecord`, `LabelMapRecord`, `ArtifactFlagRecord`, `Layer1Manifest`, `layer1_ablation_readiness_l1_v1.json`, `PredictionRecord`, `ModelRegistryRecord`, `BaselineMetricRecord`, `LowCalibrationCurveRecord`, `SubjectProfileRecord`, `EnsembleControlRecord`, `Layer2ReadinessReport`, `ModelCard`, `BenchmarkCard`, `CalibrationRecord`, `UncertaintyRecord`, `SelectivePredictionRecord`, `ThresholdRegistryRecord`, `SplitIntegrityReport`, `ReliabilityAuditReport`, `HighConfidenceWrongTable`, `Layer3MatchedOperatingPointTable`, `CalibrationDashboardMetricRecord`, `ReliabilityCard`, matched-coverage/matched-cost operating-point tables, metric dictionary entries, negative-result notes, dashboard/card/export lineage, and Layer 0 claim-evidence matrix entries.

The proximity link is implemented by matching source-record IDs rather than by reusing aggregate metrics. A0, A1, A2, A3, and A4 evidence must be matched by dataset, subject/session where available, trial/window or aggregation unit, split, preprocessing version, label-map version, `label_budget_id`, seed, model family where applicable, threshold-selection split, and metric dictionary entry. If any required key is missing, the comparison may remain visible as diagnostic evidence but must not be promoted to a thesis-ready improvement claim. Negative results such as calibration failure, threshold sufficiency, or selective-prediction sufficiency remain proximate evidence and must be exported rather than hidden.

Layer 10 consumes this branch as a read-only reporting path. It may filter these artifacts by dataset, subject/session, model family, `label_budget_id`, split, ablation step, operating point, coverage, cost, threshold registry, calibration method, ablation-readiness status, and diagnostic-only status. It must not recompute metrics from anonymous arrays, retune thresholds, alter comparison groups, or remove Layer 0 limitation warnings.

18.16 Layers 4–7 evaluation-lineage proximity addendum

The MERGE_RECORD_22 Layers 4–7 propagation adds one precise proximity rule. Evaluation evidence for IHARQ, RegimeRisk, adaptive policies, and simulated closed-loop policies must stay closest to the records that generated it. It must not be stored only as a dashboard aggregate, a thesis table, or a global ablation summary. Layer 4 owns the nearest A5 IHARQ-lite evidence; Layer 5 owns the nearest A7 temporal-trust evidence; Layer 6 owns the nearest A6/A8/A9 policy-readiness evidence and participates in A10–A13; Layer 7 owns the nearest A10/A11 frozen-policy held-out simulation evidence and participates in A12–A13.

The nearest records and artifacts for this slice are EvidenceQualityRecord, IHARQDecisionRecord, RegimeStateRecord, conditional TrustStateRecord, ReadinessContextRecord, DeferralRecord, PolicyActionRecord, TransitionRecord, RewardTraceRecord, optional PolicyUpdateTrace, Layer 4–7 EvaluationOutcomeRecord files, matched-comparison reports, shuffled/random RegimeRisk negative-control reports, metric-dictionary entries or snapshots, negative-result notes, controller-card lineage, dashboard/card/export lineage, claim-evidence matrix entries, and diagnostic-only labels.

The proximity link is implemented through explicit source-record IDs and matched-comparison keys rather than through aggregate metric names. A Layers 4–7 evaluation artifact is thesis-ready only when it can point backward to its dataset, subject/session where available, trial/window or episode unit, split, model, calibration/threshold/evidence IDs, policy/action IDs where applicable, regime state where applicable, simulator/reward/stress IDs where applicable, global ablation ID, local layer ablation ID, metric-dictionary ID, dashboard/export lineage, and Layer 0 claim-boundary label. If the comparison cannot be matched, it may remain visible as diagnostic evidence but must not support an improvement claim.

Layer 10 consumes this branch as a read-only reporting path. It may filter Layers 4–7 evaluation artifacts by ablation step, policy family, regime state, cost, coverage, reward version, stress profile, update mode, frozen-policy flag, diagnostic-only status, and claim boundary. It must not silently repair missing matches, merge training/adaptation curves with frozen-policy evaluation, hide negative results, or remove the simulation-only and proxy-label limitations attached to Layers 6–7.

18.17 Layer 9 embodiment proximity addendum

The Layer 20-to-Layer 9 merge adds one precise proximity rule. The detailed MyoSuite/OpenSim/static-replay embodiment subsystem is not a separate runtime layer and not a P20-owned branch. Its nearest functional owner is Layer 9 / Module H, and its nearest downstream observability owner is Layer 10 / Module I. The closest upstream records are IHARQDecisionRecord, RegimeStateRecord, ReadinessContextRecord, PolicyActionRecord, DeferralRecord, optional TransitionRecord, and optional StressProfileRecord. The closest downstream records and artifacts are EmbodimentStateRecord, EmbodimentOutcomeRecord, SafetyEventRecord or safety_event_log.csv, command timelines, endpoint-error curves, Layer9RewardTrace files where enabled, Layer9EmbodimentDemoManifest, dashboard-compatible exports, EmbodimentCard, and Layer 10 read-only embodiment views.

The proximity link is implemented through explicit source-record IDs rather than through visual similarity or simulator success. A Layer 9 embodiment output is valid only when it can point backward to the readiness decision, policy action or deferral, regime/stress context where applicable, command-mapping version, safety-gate version, adapter/simulator/static-replay version, seed or nondeterminism note, and non-clinical claim warning. If an output cannot provide

those links, it must be marked incomplete, blocked from thesis export, or represented as accepted missingness rather than treated as valid embodiment evidence.

Layer 10 consumes this branch as a read-only reporting path. It may filter embodiment results by run, mode, stress condition, safety-event type, endpoint metric, command type, and reward-trace availability, but it must not control the simulator, rerun adapters silently, rewrite safety logs, or convert `Layer9RewardTrace` files into clinical reward. The proximity map therefore makes the Layer 9 embodiment branch observable and reproducible without weakening the public-data, simulation-only, and non-clinical scope boundaries.

18.18 Layer 7 policy-update proximity addendum

The Layer 7 propagation adds one precise proximity rule. `PolicyUpdateTrace` is not a general-purpose policy record and is not required for every simulation run. It is optional and valid only for simulation-authorized contextual-bandit/RL update runs or disabled-update audits. Its nearest upstream records are `ReadinessContextRecord`, `PolicyActionRecord`, `TransitionRecord`, and `RewardTraceRecord`, with session and rollout context supplied by `ClosedLoopSessionRecord` and `PolicyRolloutRecord`; its nearest downstream consumers are Layer 6 policy-analysis reruns, Layer 10 policy-update trace cards, dashboard frozen/update separation filters, ablation scripts, and reproducibility manifests.

The proximity link is implemented through `TransitionRecord.policy_update_trace_id` when a policy mutation occurs or when a disabled-update audit is intentionally materialized. If the simulation run is a frozen-policy evaluation, this field must be empty, null, or explicitly tied to a disabled-update audit entry. A non-null mutation trace during frozen evaluation is a validity failure, not an informative optional output.

After the Layer 6 policy-learning merge, `PolicyUpdateTrace` must also be visible in Layer 6 record proximity as a simulation-authorized update artifact rather than as an ordinary policy-decision record. The expanded `PolicyActionRecord` is the closest decision-time output; `PolicyUpdateTrace` is the closest update-time artifact. The two must be linked but not confused: the action record says what the policy chose, while the update trace says whether a simulation-authorized learner changed after receiving reward/transition evidence.

18.19 Layers 8–10 evaluation-lineage proximity addendum

The `MERGE_RECORD_22` Layers 8–10 propagation adds one precise proximity rule. A12 StressForge robustness evidence must stay closest to Layer 8 / Module G, and A13 embodiment-proxy evidence must stay closest to Layer 9 / Module H. These artifacts must not exist only as dashboard aggregates, thesis tables, or Ablation and Evaluation Architecture summary labels. Layer 10 / Module I is the nearest observability owner, but it is not the producer of stress schedules, embodiment logs, matched-comparison keys, metric dictionaries, negative-result notes, or claim-evidence entries.

For Layer 8, the nearest records and artifacts are `StressProfileRecord`, `StressScheduleRecord` where present, clean/stressed source-record pointers, Layer 8 `EvaluationOutcomeRecord`, `records/evaluation_outcomes/layer8_stress_evaluation_outcomes.parquet`, `reports/matched_comparisons/layer8_matched_stress_comparisons.csv`, `metrics/metric_dictionary_layer8_stress.yaml`, `notes/negative_results/layer8_stress_negative_1`, `dashboard/card/export` lineage, and `manifests/claim_evidence/layer8_stress_claim_evidence.json`. The closest upstream granularity is the clean source record, stress profile, stress episode, policy/simulation context where applicable, and ablation comparison key.

For Layer 9, the nearest records and artifacts are `EmbodimentStateRecord`, `EmbodimentOutcomeRecord`, embodiment-linked `TransitionRecord` pointers, `SafetyEventRecord` or `safety_event_log.csv`, command timelines, endpoint-error curves, `Layer9RewardTrace` files where enabled, `Layer9EmbodimentDemoManifest`, dashboard-compatible exports, Layer 9 `EvaluationOutcomeRecord`, `records/evaluation_outcomes/layer9_embodiment_evaluation_outcomes.parquet`, `reports/matched_comparisons/layer9_matched_embodiment_comparisons.csv`, `metrics/metric_dictionary_layer9_embodiment.yaml`, `notes/negative_results/layer9_embodiment_ne`, `dashboard/card/export` lineage, and `manifests/claim_evidence/layer9_embodiment_claim_evidence.json`. The closest upstream granularity is the readiness decision, policy action or deferral, regime/stress context where applicable, command mapping, safety-gate decision, simulator/static-replay state, and ablation comparison key.

Layer 10 consumes this branch as a read-only reporting path. It may filter A12/A13 artifacts by ablation step, stress family, stress intensity, policy family, regime state, embodiment mode, command type, endpoint metric, safety-event status, reward-trace availability, diagnostic-only status, and claim boundary. It must not silently repair missing matched comparisons, infer missing metric definitions, suppress negative-result notes, mutate stress or embodiment source artifacts, or remove simulation-only and non-clinical limitations. If any required A12/A13 proximity link is missing, the output may remain visible as diagnostic evidence but must not support an improvement, robustness, embodiment-consequence, or final-demonstration claim.

18.20 Final Consistency Checklist

Before the architecture is considered internally consistent, the following checklist must pass.

- Every module has declared inputs.
- Every module has declared outputs.
- Every output has at least one downstream consumer or thesis artifact.
- Every feedback path is explicit, logged, bounded, and reproducible.
- Every record has an owner.
- Every learned policy has a declared training source.
- Every thesis-used Layer 6 policy result has policy-stage, context-ablation, action-legality, cost/reward sensitivity, policy-to-layer dependency, and claim-boundary matrices where applicable.
- Every threshold has a validation source and test-set lockout proof.
- Every stress output points back to a clean record and a declared stress schedule.
- Every thesis-used Layer 8 A12 stress artifact points back to a Layer 8 `EvaluationOutcomeRecord`, matched-stress comparison report, stress metric-dictionary entry, negative-result note where applicable, `dashboard/export` lineage, claim-evidence entry, and diagnostic-only status when matching fails.
- Every embodiment output is labeled simulation-only and proxy-only, owned by Layer 9, and linked to readiness context, command mapping, safety gate, simulator/static-replay version, safety-event log, and `Layer9EmbodimentDemoManifest` where applicable.

- Every Layer 10 embodiment dashboard artifact points back to Layer 9 source records, metric definitions, `embodiment_filter`, `reward_trace_filter` where applicable, and non-clinical warning tags.
- Every thesis-used Layer 9 A13 embodiment artifact points back to a Layer 9 `EvaluationOutcomeRecord`, matched embodiment comparison report, embodiment metric-dictionary entry, negative-result note where applicable, dashboard/export lineage, claim-evidence entry, and diagnostic-only status when matching fails.
- Every dashboard artifact points back to source records and metric definitions.
- Every A0–A4 evaluation-slice artifact has matched-comparison keys, a metric-dictionary entry, diagnostic-only status when required, and a Layer 0 claim-evidence boundary.
- Every simulation-authorized bandit/RL policy update has an optional `PolicyUpdateTrace` when enabled, and frozen-policy evaluation contains no unlogged policy mutation.
- No module uses future information unless an experiment explicitly permits non-causal retrospective analysis and labels it as such.
- No clinical claim is inferred from public EEG, simulated rewards, StressForge-Lite outputs, or MyoSuite/OpenSim-only results.
- All non-target layers remain consistent with Layer 0 through Layer 10, the cross-layer module architecture, the record-first design, the functional graph, and the Stretch C claim-safety rule.

This completed proximity map therefore functions as the central architectural contract of IHARQ BenchGuard. It shows how the system moves from public EEG evidence to calibrated uncertainty, IHARQ readiness decisions, temporal trust, adaptive policy actions, simulated closed-loop consequences, stress-conditioned robustness, optional simulated embodiment, and thesis-ready dashboard artifacts without losing record ownership, validation discipline, reproducibility, or claim safety.

19 Retired detailed dossier: Policy-Learning Architecture merged into Layer 6

The former standalone *Layer 19: Stretch C Policy-Learning Architecture* detailed dossier has been merged into **Layer 6: Adaptive Readiness Policy Layer**. It is no longer an independent runtime layer or a separate architectural owner.

The preserved content now lives in Layer 6 as policy-plane ownership, decision-time context and leakage rules, the P0/6.0–P5/6.5 escalation ladder, the IHARQ-lite reference baseline, output-record updates, cost/reward formula, policy-update trace handling, evaluation matrices, dashboard artifacts, reproducibility scaffold, failure-mode controls, and definition-of-done requirements.

This retired note is intentionally compact. It prevents the document from carrying two competing policy-learning sections while preserving traceability for readers who saw the older Layer 19 numbering. The authoritative policy-learning specification is Layer 6.

20 Retired detailed dossier: MyoSuite/OpenSim Embodiment Architecture merged into Layer 9

The former standalone detailed dossier, previously titled **MyoSuite/OpenSim Embodiment Architecture in Detail**, has been merged into **Layer 9: MyoSuite/OpenSim Embodiment Demo**. Layer 9 now owns the embodiment subsystem, including the MyoSuite/OpenSim/static-replay modes, adapter interface, proxy command mapping contract, readiness safety state machine, record-intake validation, readiness-context assembly, EmbodimentStateRecord and TransitionRecord linkage, StressForge-Lite integration, simulation-only reward bridge, A13 embodiment ablation linkage, dashboard/card/export pathway, reproducibility artifacts, failure controls, implementation roadmap, readiness levels, thesis evidence, and claim-safety checklist.

This retired marker is retained only to preserve document history and prevent the detached dossier from being treated as an independent runtime layer. It introduces no new runtime layer, no separate record namespace, and no additional claim beyond the Layer 9 architecture. Any remaining global references to Layer 20 or P20 should be updated only through a targeted propagation pass when those sections are selected for editing.

21 Retired detailed dossier: Dashboard Architecture in Detail merged into Layer 10

The former standalone detailed dossier, previously titled **Dashboard Architecture in Detail** and corresponding to the retired Layer 21 dashboard/detail expansion, has been merged into **Layer 10: Dashboard, Cards, and Reproducibility Layer**. Layer 10 now owns the complete dashboard subsystem, including evidence-accountability framing, stakeholder inspection questions, read-only dashboard boundary, DashboardMetricRecord and DashboardViewRecord contracts, backend module-strengthening requirements, page/view generation, card/export logic, claim-to-artifact traceability, warning and blocked-export rules, dashboard QA, reproducibility package logic, implementation roadmap, maturity criteria, failure controls, and thesis-readiness checklist.

This retired marker is retained only to preserve document history and prevent the detached dashboard dossier from being treated as an independent runtime layer. It introduces no new runtime layer, no separate record namespace, and no dashboard-owned experimental authority. The authoritative dashboard, card, export, and reproducibility specification is Layer 10. Any remaining global references to the former Layer 21 or detached dashboard dossier should be updated only through a targeted propagation pass when those sections are selected for editing.

22 Ablation and Evaluation Architecture

22.1 Why ablation is central

The architecture is complex. Therefore, the thesis must prove that the complexity is useful. Ablation is not an optional extra; it is the main defense against criticism. In IHARQ BenchGuard Stretch C, the criticism to defend against is direct: a reader may ask whether the system is only a complicated wrapper around a conventional motor-imagery decoder, whether thresholding would already solve the reliability problem, whether temporal trust and adaptive policies

add measurable value, and whether simulation, stress testing, embodiment, and dashboards are scientifically disciplined rather than decorative.

This layer is the system-level evaluation contract for that criticism. It defines how the full record-first functional graph is tested, compared, ablated, reported, and bounded by claim-safety rules. Its thesis role is to turn architecture into evidence: decoder outputs become calibration evidence, calibration records become reliability evidence, IHARQ decisions become evidence-verification evidence, RegimeRisk records become temporal-trust evidence, policy actions become readiness-policy evidence, simulation transitions become sequential-consequence evidence, StressForge schedules become robustness evidence, MyoSuite/OpenSim outcomes become simulated action-consequence evidence, and dashboard exports become traceability and reproducibility evidence.

The layer addresses the bottleneck that accuracy alone cannot justify a readiness architecture. Low-calibration neural-intent decoding can fail through miscalibration, high-confidence wrong predictions, unstable windows, disagreement across evidence sources, drift-like temporal behavior, fatigue-like degradation, dropout, delay, excessive deferral, unsafe accepted actions, and untraceable figures. Without this layer, the thesis would contain modules but no fair mechanism for deciding which modules are necessary, which modules are redundant, which claims are supported, and which claims must remain proxy-only or simulation-only.

The contribution supported by this layer is therefore not a single metric. It supports the central claim that IHARQ BenchGuard can be evaluated as a reliability and readiness architecture rather than merely as a decoder benchmark. It also protects the thesis against overclaiming: public EEG records, pseudo-readiness labels, StressForge stressors, contextual bandit policies, RL policies, and MyoSuite/OpenSim embodiment can produce useful architectural evidence, but they cannot prove clinical BCI-FES safety, real rehabilitation efficacy, medical-device readiness, real patient safety, or real muscle stimulation control.

22.2 Layer responsibility, boundaries, and implementation role

The technical responsibility of this layer is to define, run, validate, and report all comparisons needed to evaluate IHARQ BenchGuard Stretch C. It does not train the base decoders, create the EEG protocol, invent clinical labels, run hidden dashboard recomputation, or convert simulated embodiment into a clinical claim. Instead, it consumes saved records from earlier layers, applies matched comparison discipline, computes declared metrics, generates ablation artifacts, creates claim-evidence mappings, and exports thesis-ready figures, tables, cards, and manifests.

The layer boundary is strict. It may compute metrics from existing records; it may group records into experiment summaries; it may run declared ablations and stress-conditioned comparisons; it may produce dashboards, cards, and reproducibility manifests. It may not tune thresholds, costs, rewards, stress schedules, or policies on the test set unless that run is explicitly labeled diagnostic and excluded from thesis claims. It may not expose evaluation-only labels to decision-time policies. It may not silently replace missing records with inferred results. It may not let a dashboard plot become evidence unless the figure can be traced to records, configs, seeds, and code versions.

The expected implementation artifacts are evaluation configuration files, run manifests, metric dictionaries, experiment summary records, ablation tables, matched-comparison tables, dashboard metric exports, failure-case audit logs, negative-result notes, cards, figure/table manifests, reproducibility manifests, and a claim-evidence matrix. The core submodules are the evaluation-mode registry, ablation-runner, matched-comparison validator, metric engine, statistical-summary engine, failure-audit engine, claim-safety validator, dashboard-export adapter, and reproducibility packager. Supporting submodules include schema validators, miss-

ingness reporters, card generators, and thesis-table exporters. Optional or Stretch C submodules include contextual-bandit comparison, RL rollout comparison, StressForge held-out stress evaluation, and MyoSuite/OpenSim embodiment outcome evaluation. Future work may include richer Bayesian hierarchical analysis or real online deployment evaluation, but those are not required for the core thesis claim.

22.3 Whole-system evaluation questions

The evaluation architecture is organized around concrete questions rather than a generic metric list:

- Does calibration improve reliability visibility beyond raw decoder output?
- Does selective prediction or simple confidence thresholding already solve the wrong-accept problem?
- Does IHARQ-lite add value beyond confidence thresholding, entropy rejection, margin rejection, longer windows, voting, averaging, and ordinary ensembles?
- Does the evidence-quality estimator improve over rule-only IHARQ under matched data, coverage, cost, and split constraints?
- Does RegimeRisk improve decisions under temporal instability, drift-like behavior, fatigue-like degradation, dropout, changing trust, or stress-conditioned sequences?
- Does learning-to-defer provide a strong sufficient baseline, or does Adaptive-IHARQ add value beyond deferral?
- Does supervised Adaptive-IHARQ improve readiness decisions under declared proxy labels and declared cost functions?
- Do contextual bandit and RL policies improve readiness only under declared simulation assumptions and frozen rewards?
- Does StressForge-Lite reveal robustness boundaries across delay, dropout, weakness, noisy intent, drift, fatigue-like degradation, and combined stress?
- Does the MyoSuite/OpenSim embodiment demo show simulated action-level consequences without implying clinical validation?
- Does the dashboard improve traceability, reproducibility, missingness visibility, and claim-safety inspection rather than merely visualizing final numbers?
- Which modules fail, add cost without benefit, or help only under narrow protocol conditions?

22.4 Evaluation modes

Evaluation results must be separated by mode. A decoder result, a pseudo-online replay result, a simulated closed-loop result, a stress-conditioned result, an embodiment result, and a dashboard completeness result do not carry the same scientific meaning.

Mode	Purpose	Required inputs	Outputs and metrics	Claim status
Offline public EEG benchmark	Evaluate decoder families, low-calibration behavior, calibration, uncertainty, and selective prediction under public protocol constraints.	DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, PredictionRecord, ModelRegistryRecord, BaselineMetricRecord, LowCalibrationCurveRecord, Layer2ReadinessReport, CalibrationRecord, UncertaintyRecord, SplitIntegrityReport, ReliabilityAuditReport, Layer 3 outputs plus IHARQDecisionRecord, EvidenceQualityRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQTraceRecord.	Decoder metrics, calibration metrics, risk-coverage curves, HighConfidenceWrongTables, baseline tables, Reliability diagrams, Layer3MatchedOperatingPointTable, and CalibrationDashboardMetricRecord reports.	Core public-EEG reliability evidence; not clinical.
Evidence-readiness evaluation	Test IHARQ accept, wait, request-evidence, combine, fallback, uncertain, and unsafe decisions.	Layer 3 outputs plus IHARQDecisionRecord, EvidenceQualityRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQTraceRecord.	Accepted accuracy, wrong-accept rate, reason-code distribution, evidence-request burden, fallback burden, IHARQ ablation table.	Core evidence-verification evidence; readiness is proxy/computational.
Temporal-trust evaluation	Evaluate whether RegimeRisk and trust states improve decisions under temporal instability.	RegimeStateRecord, TrustStateRecord, regime timelines, stop-loss logs, stress-conditioned records where present.	Regime transition summaries, high-risk accepted error, stop-loss events, early-warning lead time where valid, regime-aware versus regime-unaware comparison.	Core temporal-trust evidence; drift/fatigue labels are proxy unless real labels exist.
Adaptive-policy evaluation	Compare rule IHARQ, learned evidence-quality scoring, learning-to-defer, supervised Adaptive-IHARQ, and other policy variants.	ReadinessContextRecord, PolicyActionRecord, DeferralRecord, ControllerCard, cost configs, reward traces where applicable.	Policy reward/cost, action distribution, deferral rate, wait/request burden, fallback burden, policy stability, matched-cost comparison.	Core policy evidence for proxy readiness; no real clinical controller claim.

Simulated closed-loop evaluation	Evaluate sequential consequences of actions under replay or simulator assumptions.	TransitionRecord, RewardTraceRecord, EpisodeSummaryRecord, ClosedLoopSessionRecord, PolicyRolloutRecord, EnvironmentDiagnosticsRecord, SafetyEventRecord, and optional PolicyUpdateTrace.	Episode return, task success, delay cost, late assist, wrong simulated action, unsafe action, unsafe action count, reward decomposition, environment diagnostics, transition-level failure rate, and separated training/adaptation versus frozen-policy evaluation.	Simulation-only Stretch C evidence.
Stress-conditioned evaluation	Measure robustness under clean, single-stressor, combined-stressor, and held-out stress schedules.	StressProfileRecord, StressScheduleRecord, stress configuration, stressed transitions and stressed decision records.	Robustness drop, stress sensitivity, stress family/intensity curves, stress-specific failure taxonomy, StressCard.	Controlled stress robustness evidence; not proof of real-world physiology.
Embodiment-proxy evaluation	Test whether readiness decisions produce different simulated action consequences through the Layer 9-owned MyoSuite/OpenSim/continua replay subsystem.	EmbodimentStateRecord, EmbodimentOutcomeRecord, embodiment-linked TransitionRecord, SafetyEventRecord or safety-event log, Layer9RewardTrace files where enabled, continua and timelines, endpoint-error curves, command-mapping manifests, Layer9EmbodimentDemoManifests, adapter logs, policy actions, and transition extensions.	Endpoint error, command success, command block rate, no-action/safe-hold burden, unsafe-event rate, blocked unsupported command rate, reward-term decomposition manifests, stress-conditioned embodiment outcome, adapter failure rate, and demo-manifest completeness.	Simulated embodiment evidence only; Layer 10 may visualize it only as read-only evidence from saved Layer 9 outputs.

Dashboard and reproducibility evaluation	Validate that all figures, cards, warnings, exports, and thesis artifacts are traceable.	DashboardMetricRecord, CardRecord, DashboardViewRecord, ReproducibilityManifest, export manifests, warning flags, figure/table lineage records.	Artifact completeness, card completeness, figure traceability, record validation pass rate, claim-warning coverage, export determinism.	Reproducibility and inspection evidence; dashboard does not create new scientific evidence.
--	--	---	---	---

Every mode must declare its source records, baselines, metrics, supported claim, unsupported claim, dashboard artifact, and reproducibility artifact. If a required record is missing, the result may still be used for exploratory debugging, but it is not thesis-ready until the missingness is visible and justified.

22.5 Core ablation ladder

The global ablation ladder is A0–A13. It preserves the original ladder and expands it into a contract for fair evaluation. The shorter Layer 7 ladder should be treated as a simulation-local mapping: Layer 7 A0 decoder accept-all corresponds to global A0; Layer 7 confidence threshold corresponds to global A2, while Layer 7 entropy, margin, and selective-prediction controls correspond to global A3; Layer 7 IHARQ-lite corresponds to global A5; Layer 7 IHARQ plus RegimeRisk corresponds to global A7; Layer 7 supervised Adaptive-IHARQ corresponds to global A9; Layer 7 learning-to-defer corresponds to global A8; Layer 7 contextual bandit and RL correspond to global A10 and A11; and Layer 7 full stress plus embodiment corresponds to global A12–A13. Dashboard exposure should use the global A0–A13 IDs so that thesis figures, cards, and export manifests remain consistent.

Step	System variant	Component tested	Required comparison discipline	Claim and failure interpretation
A0	Raw decoder	Base decoder and preprocessing protocol without calibration or reliability control.	Same DatasetRecord, SplitRecord, PreprocessingRecord, low-calibration budget, subject/session set, ModelRegistryRecord, BaselineMetricRecord, LowCalibrationCurveReadiness, Layer2ReadinessReport, and seed where applicable.	Establishes the naive reference with readiness/compatibility lineage. Poor performance shows decoder difficulty; strong accuracy alone does not prove readiness.

A1	Calibrated decoder	Probability calibration and uncertainty reporting.	Compare to A0 on identical predictions or identical model family; calibration split must be separate from test; SplitIntegrityReport and CalibrationEligibilityTable must support calibration legality; threshold selection cannot use test.	Supports calibration visibility if ECE, Brier, NLL, reliability diagrams, ReliabilityAuditReport , or HighConfidenceWrongTable improve. If not, HighConfidenceWrongTable calibration split size, method, and model probability quality.
A2	Simple registered threshold	Confidence or calibrated-confidence thresholding baseline.	Must compare at matched coverage or matched wrong-accept cost; threshold registry must be frozen before test; Layer3MatchedOperatingPointTable records the matched operating points and threshold lineage.	Tests whether simple registered thresholding is sufficient. If A2 matches later layers, the thesis must report that simpler Layer3MatchedOperatingPointTable is strong under that protocol.
A3	Uncertainty and selective prediction	Entropy, margin, calibrated uncertainty, conformal-set-size proxies where used, and Layer 3 risk-coverage/selective-prediction baselines.	Compare against A0/A1/A2 using matched coverage, matched wrong-accept cost, matched split roles, metric-dictionary entries, and Layer3MatchedOperatingPointTable lineage.	Tests whether uncertainty-aware rejection or selective prediction already solves the wrong-accept problem before IHARQ is credited. Layer3MatchedOperatingPointTable later layers, narrow the claim to traceability, interpretability, or downstream-specific value.
A4	Longer-window, multi-window voting/averaging, and ordinary ensemble controls	More temporal evidence, simple evidence aggregation, probability averaging, and ordinary ensembling.	Same decoder family where feasible, same windows or declared longer-window protocol, same model set, same aggregation split, matched coverage/cost, explicit delay burden, and EnsembleControlRecord lineage.	Tests whether IHARQ only replicates longer observation, voting, averaging, or ordinary ensembles. If A4 is sufficient, IHARQ claims must be narrowed to interpretability or traceability.

A5	IHARQ-lite	Rule-based evidence verification with accept, wait, request-evidence, combine, fallback, uncertain, and unsafe decisions.	Compare against A2, A3, and A4 at matched coverage, cost, data, and split.	Supports explicit evidence verification if wrong accepts decrease or traceable decisions improve without unjustified burden. If not, analyze reason-code failure and over-conservatism.
A6	IHARQ plus evidence-quality estimator	Learned evidence-quality scoring on declared targets.	Same IHARQ inputs as A5; protected target fields excluded at decision time; training/validation/test split locked.	Tests learned evidence quality beyond rules. Failure suggests pseudo-label weakness, class imbalance, leakage, or overfitting.
A7	IHARQ plus RegimeRisk	Temporal trust context, regime state, trust state, and stop-loss behavior.	Compare A5/A6 with and without RegimeRisk under matched sequences and stress schedules.	Supports temporal-trust value if high-risk accepts decrease or degradation warnings improve. Failure may mean temporal instability is absent or RegimeRisk is mis-specified.
A8	Learning-to-defer	Deferral/routing baseline to fallback, no-action, or extra evidence.	Matched cost functions, fallback availability, coverage, and split constraints.	Tests whether routing is a sufficient simpler alternative. If A8 matches A9, the thesis should state that deferral is a strong baseline.
A9	Supervised Adaptive-IHARQ	Learned readiness policy from declared proxy labels and costs.	Compare against A5, A6, and A8 using matched cost, coverage, subject/session splits, and validation-only policy selection.	Supports adaptive readiness under proxy labels if cost and wrong-accept trade-offs improve. Failure suggests pseudo-label, cost, imbalance, or overfit issues.

A10	Contextual bandit	Immediate-reward online adaptation in simulation.	Frozen reward, held-out simulated sessions, matched context fields, no future outcome leakage, and Layer 7 evidence from TransitionRecord , RewardTraceRecord , ClosedLoopSessionRecord , EpisodeSummaryRecord , PolicyRolloutRecord , EnvironmentDiagnosticRecord , SafetyEventRecord , and optional PolicyUpdateTrace .	Simulation-only evidence for immediate adaptation. If improvement is reward-specific or environment-diagnostic checks fail, downgrade to reward-sensitivity or diagnostic-only observation.
A11	RL policy	Sequential policy learning under delayed consequences.	Same simulator, reward version, stress schedules, action space, safety constraints, held-out rollouts, RewardTraceRecord decomposition, PolicyRolloutRecord lineage, ClosedLoopSessionRecord , replay metadata, and EnvironmentDiagnosticRecord checks as non-learning baselines.	Simulation-only evidence for delayed consequences. Failure, unstable ranking, missing update/frozen separation, or unresolved environment diagnostics indicates simulator overfitting, reward fragility, or diagnostic-only EnvironmentDiagnosticRecord .
A12	StressForge stress tests	Delay, dropout, weakness, noisy intent, drift, fatigue-like degradation, and combined stress.	Compare clean, single-stressor, combined-stressor, and held-out stress; stress registry and seeds must be versioned.	Supports robustness-boundary analysis, not real physiological validation. Large degradation is a valuable vulnerability finding.

A13	Layer 9 MyoSuite/OpenSim/static replay embodiment demo	Simulated static-level consequences of readiness decisions through the Layer 9-owned L9.1–L9.13 embodiment subsystem.	Compare readiness-gated action against no embodiment, E1 static replay, accept-all action, confidence-only gating, IHARQ-gated action, RegimeRisk-aware gating, adaptive-policy action, no-action/safe-hold behavior, unsupported- command blocking, matched stress where relevant, and optional OpenSim replay where implemented. Required evidence includes <code>EmbodimentStateRecord</code> , <code>EmbodimentOutcomeRecord</code> , embodiment-linked <code>TransitionRecord</code> , safety-event log, command timeline, endpoint-error curve, <code>Layer9RewardTrace</code> where enabled, command-mapping manifest, <code>Layer9EmbodimentDemoManifest</code> , and dashboard- compatible Layer 9 export.	Simulated embodiment evidence only. Technical failure preserves E0/E1 no-embodiment or static replay modes, keeps embodiment claims optional, and prevents Layer 10 embodiment cards from implying clinical validation.
-----	--	--	--	---

For each ablation, the implementation must save an ablation-step ID, system-variant config, required upstream record pointers, comparison baseline, metric set, dashboard view, card/report pointer, claim boundary, and negative-result note when applicable.

22.6 Internal evaluation modules

The evaluation layer is implemented as a set of modules. Each module has a scientific role, engineering responsibility, input/output contract, dependencies, consumers, and failure behavior.

Module	Purpose and bottleneck	Inputs and processing logic	Outputs and consumers	Failure modes
Evaluation-mode registry	Prevents incompatible evidence types from being mixed. Scientifically separates offline EEG, pseudo-online replay, simulation, stress, embodiment, and dashboard evidence.	Reads evaluation configs, Layer 0 scope tags, source-record types, and run metadata; assigns mode and claim boundary.	Mode registry consumed by metric engine, dashboard, cards, and claim-evidence matrix.	Missing or wrong mode invalidates claim status and triggers claim-safety warning.
Ablation-runner	Executes or assembles <i>A0–A13</i> comparisons. Defends against complexity without contribution.	Loads ablation configs, variant IDs, baseline IDs, and matched record groups; computes variant-level outputs.	ExperimentSummaryRecord ablation tables, ablation cards, thesis tables.	Recorded variants, missing baselines, or hidden test tuning weaken or invalidate comparison.
Matched-comparison validator	Ensures fairness across data, seeds, budgets, stress, reward, and embodiment modes.	Compares dataset, subject, session, window, split, preprocessing, label-map, low-calibration budget, threshold, reward, stress, and embodiment identifiers.	Pass/fail validation report consumed by all thesis-ready outputs.	Mismatch without diagnostic label blocks thesis use.
Metric engine	Computes decoder, calibration, readiness, IHARQ, RegimeRisk, policy, simulation, stress, embodiment, dashboard, and reproducibility metrics.	Reads record fields and metric dictionary; applies deterministic metric functions.	EvaluationOutcomeRecord DashboardMetricRecord metric tables, figures.	Undefined record, missing field, or pooling-only reporting creates metric warning.

Statistical-summary engine	Prevents overinterpretation of small or pooled differences.	Aggregates by subject/session/split; computes mean, variance, confidence intervals or credible intervals where feasible; runs paired comparisons when valid.	Statistical appendix, uncertainty-aware tables, sensitivity summaries.	No uncertainty estimate weakens confirmatory claim but may remain exploratory.
Failure-audit engine	Converts negative results and errors into thesis evidence.	Reads unsafe events, wrong accepts, reason codes, stress failures, adapter failures, missingness, and dashboard warnings.	Failure taxonomy, audit logs, negative-result notes, limitation statements.	Hidden failures invalidate or weaken the relevant claim.
Claim-safety validator	Enforces Layer 0 boundaries.	Checks mode, data provenance, proxy labels, simulation flags, stress flags, embodiment flags, and dashboard warning coverage.	Claim-warning records, boundary statements, card warnings, invalid-claim flags.	Unsupported clinical or deployment wording blocks figure/card export.
Dashboard-export adapter	Provides Layer 10 with read-only evaluation artifacts.	Converts saved evaluation outputs into dashboard records, figures, table manifests, filters, and warnings.	Dashboard pages, export manifests, thesis artifact pointers.	Silent recomputation or missing lineage blocks dashboard evidence.
Reproducibility packager	Makes evaluation rerunnable and auditable.	Collects configs, seeds, code version, schema versions, record hashes, output hashes, and figure/table manifests.	Reproducibility Missing, reproducibility card, repository package.	Missing seed, config hash, or record pointer prevents thesis-ready status.

Core modules are the evaluation-mode registry, ablation-runner, matched-comparison validator, metric engine, claim-safety validator, and reproducibility packager. Supporting modules are the statistical-summary engine, failure-audit engine, dashboard-export adapter, schema validators, missingness reporters, and card generators. Optional Stretch C modules are the bandit/RL evaluator, StressForge held-out-stress evaluator, and MyoSuite/OpenSim embodiment evaluator.

22.7 Matched-comparison discipline

All comparisons must be matched wherever possible by dataset, subject, session, trial identity, window identity, `SplitRecord`, low-calibration budget, `PreprocessingRecord`, `LabelMapRecord`, decoder family, seed, calibration split, threshold-selection split, policy-training split, stress profile, stress schedule, reward configuration, cost configuration, embodiment mode, and dashboard export configuration.

No learned policy, threshold, reward setting, stress schedule, or dashboard selection may be tuned on the test set unless explicitly labeled as diagnostic and excluded from thesis claims. Confirmatory comparisons must use locked training, validation, calibration, adaptation, and test roles. Evaluation-only fields such as true correctness, future reward, future stress outcome, and oracle readiness labels must not be visible to decision-time policies.

The following comparison rules are mandatory:

- IHARQ must be compared against selective prediction at matched coverage or matched wrong-accept cost.
- Longer windows must be compared with explicit delay burden, not only accuracy.
- Voting and averaging must be compared under the same window pool and model family where feasible.
- RegimeRisk-aware variants must be compared against regime-unaware variants under identical temporal sequences and stress schedules.
- Adaptive-IHARQ must be compared against rule-based IHARQ-lite and learning-to-defer under matched costs and fallback availability.
- Contextual bandit and RL variants must be compared against non-learning policies under frozen rewards and held-out simulated sessions.
- StressForge results must include clean, single-stressor, combined-stressor, and held-out stress settings when feasible.
- MyoSuite/OpenSim/static-replay embodiment results must compare readiness-gated action against no embodiment, static replay, accept-all, confidence-only, IHARQ-gated, RegimeRisk-aware, adaptive-policy, no-action or safe-hold, unsupported-command blocking, stress-conditioned variants, and optional OpenSim replay where relevant.
- A13 evidence is thesis-ready only when Layer 9 artifacts include the `EmbodimentStateRecord`, `EmbodimentOutcomeRecord`, embodiment-linked `TransitionRecord` pointers, safety-event log, command timeline, endpoint-error curve, `Layer9RewardTrace` where enabled, command-mapping manifest, `Layer9EmbodimentDemoManifest`, and dashboard-compatible export required by Layer 10.
- Dashboard figures must be compared only through saved source records and export manifests, never through manual plot edits.

Matched-cost, matched-coverage, matched-stress, matched-delay, and matched-reward comparisons should be reported separately. A method that reduces wrong accepts by never acting is conservative, not universally superior. A method that improves reward while increasing unsafe events indicates reward mis-specification. A method that improves only under one hand-picked stress schedule or reward weight must be downgraded to a narrow diagnostic finding.

22.8 Input contract

The evaluation layer receives primary inputs, auxiliary inputs, metadata inputs, evaluation-only inputs, proxy inputs, and Stretch C inputs. Each input must declare whether it is allowed in training, validation, calibration, adaptation, test, dashboard display, or post-hoc analysis. A field that is evaluation-only may be used to score a policy, but not to choose the action at decision time.

Source	Inputs	Required fields and validity constraints	Claim invalid if missing or unvalidated
Layer 0 governance	Claim boundaries, scope tags, simulation-only warnings, public-data limitations, clinical non-claim rules.	Each run requires scope tag, allowed claim class, prohibited claim class, proxy/simulation/embodiment flags, and warning policy. These are metadata inputs and dashboard-visible.	Claim-safety and limitation statements become invalid; clinical overclaim risk.
Layer 1 data/protocol	DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, Layer1Manifest.	Dataset ID, subject/session ID, trial/window ID, split ID, preprocessing hash, label map, artifact flags, and manifest version. These are primary and metadata inputs.	Public-EEG benchmark claim and matched comparison become invalid.
Layer 2 decoder	PredictionRecord, ModelRegistryRecord, LowCalibrationCurveRecord, SubjectProfileRecord, BaselineMetricRecord, Layer2ReadinessReport, EnsembleControlRecord, ModelCard, and BenchmarkCard.	Prediction probabilities or scores, predicted label, and label for evaluation only, model ID, seed, training budget, subject/session references, score schema, class order, readiness/compatibility status, and A4 ordinary-control lineage.	Decoder baseline, low-calibration curve, subject-difficulty, ordinary-combination-control, and downstream-readiness evidence become invalid.
Layer 3 calibration and uncertainty	CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, SplitIntegrityReport, ReliabilityAuditReport, HighConfidenceWrongTable, Layer3MatchedOperatingPointRecord, and CalibrationDashboardMetricRecord.	Calibration method, calibration split, uncertainty scores, thresholds, selective decision, split-integrity status, reliability-bin summaries, HighConfidenceWrongTable, Layer3MatchedOperatingPointRecord, and CalibrationDashboardMetricRecord. Flags, matched points, and dashboard-export lineage. Thresholds must be validation-selected.	Calibration, selective prediction, threshold legality, reliability-audit, and matched operating-point claims become invalid.

Layer 4 IHARQ	IHARQDecisionRecord, EvidenceQualityRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord.	Decision action, reason code, evidence sources, quality score, fallback route, unsafe flag, trace ID, decision-time visible fields. Proxy labels must be marked.	Evidence-verification and IHARQ-versus-threshold claims become invalid.
Layer 5 RegimeRisk	RegimeStateRecord, TrustStateRecord, RegimeCard, temporal trust summaries, stop-loss logs.	Regime score, trust score, transition marker, warning trigger, stop-loss event, temporal window definition, stress/proxy label status.	Temporal-trust and RegimeRisk claims become invalid.
Layer 6 policy	ReadinessContextRecord, PolicyActionRecord, DeferralRecord, ControllerCard, RewardTraceRecord.	Context fields, visible-feature list, action, policy ID, cost ID, proxy-label source, deferral route, reward trace where applicable.	Adaptive-IHARQ, learning-to-defer, and policy comparison claims become invalid.
Layer 7 simulation	TransitionRecord, RewardTraceRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, EnvironmentDiagnosticRecord, SafetyEventRecord, and optional PolicyUpdateTrace.	State, action, next state, reward decomposition, safety event, session ID, episode ID, rollout ID, environment-diagnostic record, simulator version, held-out split, seed, update/frozen mode, and disabled-update audit status. Future fields cannot be decision-visible.	Bandit/RL and sequential-consequence claims become invalid or diagnostic-only when these records are missing, unmatched, or inconsistent.
Layer 8 stress	StressProfileRecord, StressScheduleRecord, StressCard, stress configuration.	Stress family, intensity, schedule, seed, affected fields, clean/stressed pairing, held-out status. Stress labels are proxy unless otherwise documented.	Stress robustness and stress sensitivity claims become invalid.

Layer 9 embodiment	EmbodimentStateRecord, Embodiment mode, EmbodimentOutcomeRecord, command, embodiment-linked readiness-context ID, TransitionRecord, block/safe-hold status, SafetyEventRecord or endpoint, simulator safety-event log, branch/version, adapter Layer9RewardTrace error, reward-trace ID where enabled, where enabled, EmbodimentCard, safety-event status, command timeline, demo-manifest endpoint-error curve, completeness, and Layer9EmbodimentDemoManifest, inflation-only warning. dashboard-compatible export, adapter logs, and endpoint metrics.	Embodiment demonstration claim becomes invalid, optional-only, or dashboard-hidden until required Layer 9 artifacts and warnings exist.
Layer 10 dashboard and repro- ducibility	DashboardMetricRecord, Source record pointers, CardRecord, config hashes, figure IDs, DashboardViewRecord, card IDs, warning status, ReproducibilityManifest, export hash, dashboard export manifest, version. Dashboard reads warning flags, saved records only. figure/table lineage records.	Traceability, card completeness, figure reproducibility, and dashboard inspection claims become invalid.

Primary inputs are dataset, split, preprocessing, prediction, calibration, uncertainty, IHARQ, regime, policy, transition, stress, embodiment, and dashboard records. Auxiliary inputs include subject profiles, baseline summaries, cards, and audit reports. Metadata inputs include IDs, schema versions, config hashes, code versions, and seeds. Evaluation-only inputs include true labels, correctness, oracle outcomes, future rewards, and final episode outcomes. Proxy inputs include pseudo-readiness labels, fatigue-like or drift-like stress indicators, and simulator-derived outcome labels. Stretch inputs include contextual bandit, RL, StressForge, and MyoSuite/OpenSim records.

22.9 Output contract

The evaluation layer sends artifacts to the dashboard, thesis tables, figures, cards, appendices, repository manifests, and claim-safety layer. Every output must have a downstream consumer and missingness behavior.

Output	Content	Consumer	Missingness and failure state
ExperimentSummaryRecord	Record ID, experiment ID, ablation step ID, dataset, subject/session set, variant config, metric summary, claim tags.	Thesis results chapter, dashboard ablation page, reproducibility package.	Missing record blocks thesis-ready status for that run.

EvaluationOutcome	Metric by dataset, subject, session, model, policy, stress profile, embodiment mode, and ablation step.	Metric engine, thesis tables, cards, dashboard filters.	Missing group-level values must be visible; pooled-only outputs are weakened.
Ablation comparison tables	A0–A13 comparisons, baselines, matched fields, primary/secondary metrics, limitation notes.	Main thesis ablation table and dashboard overview.	Unmatched or incomplete comparison is diagnostic only.
Matched-comparison tables	Matched coverage, cost, stress, reward, delay, and embodiment comparisons.	Examiner-facing evidence and policy/regime/stress pages.	If matching fails, table must show mismatch reason and exclude unsupported claim.
Metric dictionary	Metric name, definition, record fields, direction, primary/secondary role, claim status, limitations.	Implementation tests, thesis methods, dashboard legends.	Undefined metric cannot appear as evidence.
Figure/table manifest	Artifact ID, source records, config hash, seed, code version, output hash, thesis location.	Thesis figures, appendices, dashboard export page.	Figure cannot be used if lineage is absent.
Failure taxonomy and audit logs	Failure categories, case IDs, source records, reason codes, unsafe events, stress failures, adapter failures.	Failure-mode section, discussion, dashboard failure page.	Hidden failures invalidate robustness claims.
Negative-result notes	Module did not improve, added burden, was unstable, or helped only narrowly.	Discussion and claim-evidence matrix.	Missing note risks overclaiming; result should be flagged.
DashboardMetricRepor	Dashboard-ready metric values, filters, warning flags, and source pointers.	Layer 10 dashboard pages.	Dashboard hides or warns on missing source records.
Cards	Reliability card, regime card, controller card, stress card, embodiment card, reproducibility card, ablation card.	Supervisor/examiner inspection, thesis appendix, dashboard cards page.	Incomplete cards carry visible warning and cannot independently support claims.
ReproducibilityManifest	Manifest configs, code version, seeds, schema versions, record hashes, outputs, figures, tables, cards.	Repository architecture, implementation pipeline, reproducibility appendix.	Missing manifest means result is not thesis-ready.

Claim-evidence matrix	Thesis claim, ablation, record, metric, figure, dashboard view, card, limitation, invalidation condition.	Final thesis argument and claim-safety governance.	Claim is weakened or removed if required evidence is missing.
-----------------------	---	--	---

Each output must include claim-safety warnings where appropriate. Every dashboard/reporting artifact must preserve source-record identity. If an artifact cannot be generated, the system should produce a visible failure state rather than silently omitting the artifact.

22.10 Metrics architecture

Metrics should be grouped by level, with explicit direction, record fields, and claim role.

Metric family	Metrics	Records used	Use and limitations
Decoder metrics	Accuracy, balanced accuracy, F1, subject-wise performance, session-wise performance where available, low-label curves, model-family comparison, subject difficulty profiles, failed-run status, readiness/compatibility status, and ordinary-combination controls.	PredictionRecord, ModelRegistryRecord, BaselineMetricRecord, LowCalibrationCurveReport, SubjectProfileRecords, Layer2ReadinessReport, and EnsembleControlRecord	Primary for A0 and A4 baseline/control reporting. Higher generally better, but decoder performance or readiness compatibility alone does not prove readiness.
Calibration metrics	ECE, Brier score, NLL, reliability diagrams, high-confidence wrong rate, calibration-bin analysis, split-integrity status, calibration eligibility, dashboard-export lineage, and calibration drift/degradation summaries where applicable.	CalibrationRecord, UncertaintyRecord, ReliabilityAuditReport, SplitIntegrityReport, CalibrationEligibilityTable, HighConfidenceWrongTable, and CalibrationDashboardMetricRecord	Primary for A1. Lower ECE/Brier/NLL is better; reliability diagrams are interpretive; calibration does not guarantee safe action.
Selective/readiness metrics	Risk-coverage, accepted-trial accuracy, retained accuracy, wrong-accept rate, rejection rate, wait rate, request-evidence rate, combine rate, fallback rate, uncertain rate, unsafe rate, delay burden, evidence-request burden, matched coverage/cost, and action-cost summary.	SelectivePredictionRecord, ThresholdRegistryRecord, Layer3MatchedOperationalPointTable, IHARQDecisionRecord, and PolicyActionRecord	Primary for A2, A3, A5, A8, A9. Interpreted with coverage and cost; low risk by no-action is not automatically better.

IHARQ metrics	Reason-code distribution, stability-check pass/fail rate, channel-subset stability contribution, model-family agreement contribution, evidence-combination success, fallback routing success, unsafe-state detection counts, IHARQ versus threshold matched-coverage and matched-cost performance.	IHARQDecisionRecord, EvidenceQualityRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQTraceRecord.	Primary for A5 and A6. Supports evidence-modification claims; reason codes can reveal redundancy or over-conservatism.
RegimeRisk metrics	RegimeRisk score usefulness, regime transition summaries, degradation detection, drift/fatigue-like warning behavior, early-warning lead time where valid, trust-score usefulness, stop-loss event counts, regime-aware versus regime-unaware comparison.	RegimeStateRecord, TrustStateRecord, stop-loss logs, stress records.	Primary for A7. Labels are proxy unless real temporal-state labels exist; usefulness should be tied to downstream decisions.
Policy metrics	Reward, regret, cost, deferral rate, fallback burden, wait burden, request burden, action distribution, policy stability across subjects, learned policy versus rule baseline, Adaptive-IHARQ versus learning-to-defer, bandit/RL versus fixed policy under simulation-only conditions.	ReadinessContextRecord, PolicyActionRecord, DeferralRecord, RewardTraceRecord, PolicyRolloutRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, and optional PolicyUpdateTrace.	Primary for A8–A11. Reward is only meaningful under frozen cost/reward conditions; regret may be unavailable outside bandit settings; update-enabled training/adaptation curves must remain separated from frozen-policy evaluation.
Simulation metrics	Task success, late assist, wrong simulated action, delay cost, over-assist, under-assist, unsafe action, safety-event count, reward decomposition, stress sensitivity, episode return, environment-diagnostic warning count, policy-rollout completeness, transition-level failure rate.	TransitionRecord, RewardTraceRecord, EpisodeSummaryRecord, ClosedLoopSessionRecord, PolicyRolloutRecord, EnvironmentDiagnosticRecord, and optional SafetyEventRecord, PolicyUpdateTrace.	Primary for A10–A12. Simulation-only; do not translate directly to clinical use; failed diagnostic or update/frozen-separation checks downgrade the result.

StressForge metrics	Stress family sensitivity, stress intensity sensitivity, single-stressor versus combined-stressor degradation, held-out stress generalization, stress-specific failure taxonomy, matched stress schedule reproducibility.	StressProfileRecord , Primary for A12. StressScheduleRecord , Supports stressed decision and transition records.	robustness-boundary discovery, not real physiological validation.
Embodiment metrics	Endpoint error, command success, command block rate, no-action safety, simulated weakness response, safe-hold behavior, over-assist and under-assist flags, action smoothness proxy where available, unsafe-event rate, blocked unsupported command rate, reward-term decomposition where enabled, command-timeline consistency, endpoint-error-curve completeness, demo-manifest completeness, embodiment adapter failure rate, and simulation-only warning coverage.	EmbodimentStateRecord , Primary for A13. EmbodimentOutcomeRecord , Simulation-only and optional; TransitionRecord , technical adapter failure should not invalidate core EEG reliability claims and should be surfaced through Layer 10 Layer9RewardTrace where enabled, command timeline, endpoint-error curve, Layer9EmbodimentDemoManifest , adapter logs, and transition extensions.	
Dashboard/reproducibility metrics	Ability completeness, card completeness, config reproducibility, figure traceability, record validation pass rate, missingness visibility, claim-warning coverage, export determinism, supervisor/examiner inspection usability.	DashboardMetricRecord , Primary for Layer 10 reproducibility evidence. CardRecord , DashboardViewRecord , ReproducibilityManifest , Dashboard metrics export manifest.	audit evidence; they do not create new scientific performance.

A metric may be primary for one ablation and diagnostic for another. The metric dictionary must state whether higher or lower is better, whether the metric supports a core claim, Stretch C claim, or diagnostic claim, and which known limitations apply. When a metric depends on proxy labels, simulation outcomes, or stress schedules, the output table and dashboard view must carry that warning.

22.11 Statistical and robustness analysis

The evaluation must not rely only on pooled window-level averages. Subject-level aggregation is required wherever subject IDs exist. Session-level aggregation should be reported where sessions exist. For confirmatory comparisons, the evaluation should report mean, variance, confidence

intervals, credible intervals, or another declared uncertainty summary appropriate to the sample size and design. Paired comparisons should be used when the same subject/session/split supports both variants. If the available data cannot support a strong statistical test, the result should be reported as descriptive or exploratory rather than overinterpreted.

Sensitivity analysis is required for thresholds, costs, reward weights, stress intensity, label budgets, and major policy choices. Low-calibration budget curves should show how performance and reliability change as calibration data decreases. Stress-conditioned analysis should report clean, single-stressor, combined-stressor, and held-out stress results where feasible. Policy comparisons should include reward sensitivity and cost sensitivity. RegimeRisk analysis should include conditions where temporal instability is present and conditions where it is weak or absent.

Per-subject and per-session failure analysis is especially important. A method that improves the mean while harming difficult subjects should not be presented as unambiguously better. A method that helps only one decoder family or one dataset should be marked as narrow. A method that changes ranking under small reward changes should be described as reward-sensitive. Confirmatory thesis results and exploratory diagnostics must be separated in tables, dashboards, and cards.

22.12 Claim-evidence matrix

Every thesis claim must be linked to records, metrics, ablations, figures, dashboard views, cards, and limitations. The claim-evidence matrix is the bridge from evaluation output to the thesis argument.

Thesis claim	Required ablation	Evidence	Dashboard/card artifact	Boundary
Calibration improves reliability visibility.	A0 versus A1.	ECE, Brier, NLL, reliability diagrams, <code>SplitIntegrityReport</code> , <code>ReliabilityAuditReport</code> , <code>HighConfidenceWrongTable</code> , and <code>CalibrationDashboardMetricRecord</code> lineage from <code>CalibrationRecord</code> and <code>UncertaintyRecord</code> .	Decoder/calibration page and <code>ReliabilityCard</code> .	Public EEG calibration behavior; not clinical readiness.
IHARQ is more than thresholding.	A2, A3, A4 versus A5.	Matched coverage/cost, accepted accuracy, wrong-accept rate, reason-code distribution, evidence-request analysis.	IHARQ-versus-threshold page and ablation card.	Computational/proxy readiness decisions.

Evidence-quality learning adds value beyond rules.	A5 versus A6.	Evidence-quality score calibration, failure prediction, accepted risk, coverage, leakage audit.	Reliability card and evidence-quality ablation table.	Depends on declared proxy labels and split discipline.
RegimeRisk adds temporal trust value.	A5/A6 versus A7.	Regime transition summaries, high-risk accepted errors, stop-loss events, stress-conditioned temporal comparison.	RegimeRisk page and regime card.	Drift/fatigue-like evidence is proxy unless real labels exist.
Learning-to-defer is a strong baseline or alternative.	A5/A6 versus A8.	Deferral rate, cost, fallback burden, retained accuracy, wrong-accept reduction.	Learning-to-defer page and controller card.	Deferral quality depends on fallback definition.
Adaptive-IHARQ improves readiness decisions.	A5, A6, A8 versus A9.	Matched cost/coverage, action distribution, proxy-label accuracy, cost reduction, unsafe-rate reduction.	Policy comparison page and controller card.	Proxy labels and declared cost functions only.
Bandit/RL policies are useful under simulated sequential consequences.	A10/A11 versus fixed policies.	Reward, regret, reward-trace decomposition, episode return, policy-rollout behavior, safety-event count, environment-diagnostic status, held-out simulated session performance, reward sensitivity, and separated training/adaptation versus frozen-policy evaluation.	Bandit/RL simulation page, closed-loop session timeline, policy rollout card, reward configuration card, policy-update trace card where applicable, and controller card.	Simulation-only; no deployed controller safety claim.
StressForge exposes robustness boundaries.	A12 versus clean and single/combined/held-out stress settings.	Stress sensitivity, robustness drop, stress-specific failure taxonomy, matched stress schedule reproducibility.	StressForge robustness page and StressCard.	Controlled stress evaluation; not proof of real-world robustness.

Layer 9 MyoSuite/OpenSim replay subsystem makes readiness decisions action-consequential under proxy mappings.	A13 versus no static embodiment, static replay, accept-all, confidence-only, IHARQ-gated, RegimeRisk-aware, adaptive-policy, safe-hold, stress-conditioned, and optional OpenSim variants.	Endpoint error, command success, blocked unsafe command, safe-hold behavior, unsafe-event rate, reward terms where enabled, command-timeline consistency, endpoint-error curve, demo-manifest completeness, and adapter failure rate.	Read-only Layer 10 embodiment page, embodiment card, Layer 9 Embodiment DemoManifest index, and dashboard-compatible Layer 9 export.	Simulated embodiment only; no patient manifest, habilitation efficacy, or real biomechanics claim.
Dashboard improves reproducibility and inspection.	Layer 10 validation across A0–A13.	Artifact completeness, card completeness, manifest completeness, figure traceability, warning coverage.	Cards/export page and reproducibility card.	Dashboard exposes saved evidence; it does not create evidence.

A claim is weakened or removed if its required ablation is absent, required records are missing, metrics are undefined, matching fails, evaluation-only fields leak into decision context, the dashboard artifact lacks lineage, or the limitation statement is absent.

22.13 Failure modes and negative-result interpretation

The evaluation architecture must not assume that every advanced module will outperform simpler baselines. Negative results are scientifically useful if they reveal where the architecture is unnecessary, fragile, expensive, or dependent on narrow assumptions.

Observed result	Interpretation and diagnostic action	Claim consequence
Calibration does not improve reliability metrics.	Inspect calibration split size, calibration method, model probability quality, low-label setting, and binning. Report whether probabilities remain unreliable.	Calibration claim weakened; reliability audit still valuable.
IHARQ does not outperform selective prediction.	Analyze matched coverage/cost, reason-code distribution, over-conservatism, evidence-request burden, and fallback routing.	State that simpler rejection may be sufficient under that protocol.
Longer windows or voting outperform IHARQ.	Report delay burden, aggregation assumptions, and whether IHARQ adds interpretability or traceability only.	Narrow IHARQ contribution.
Evidence-quality estimator fails.	Check pseudo-label construction, class imbalance, leakage guards, feature visibility, and overfitting.	Learned-quality claim weakened; rule IHARQ may remain.

RegimeRisk does not help.	Inspect whether temporal instability exists, whether stress profiles activate instability, and whether warnings affect decisions.	Temporal-trust claim narrowed to diagnostic monitoring.
Learning-to-defer matches	Treat deferral as a strong simpler baseline and compare cost/fallback burden.	Adaptive claim narrowed or made conditional.
Adaptive-IHARQ.		Learned readiness claim weakened; rule baseline preserved.
Adaptive-IHARQ fails.	Examine pseudo-label construction, cost weights, class imbalance, leakage guards, policy overfitting, and split discipline.	Downgrade to simulation-specific reward sensitivity.
Bandit/RL improves only under one reward setting.	Run reward sensitivity, held-out simulated sessions, and non-learning baselines.	Robustness boundaries become central; clean-only claims weakened.
StressForge changes conclusions dramatically.	Preserve the result as vulnerability evidence and update failure taxonomy.	Embodiment remains optional; core EEG reliability architecture remains evaluable.
MyoSuite/OpenSim fails technically.	Preserve E0/E1 no-embodiment or static replay modes, adapter logs, and simulation-only warnings.	Dashboard/reproducibility claim invalid until repaired.
Dashboard artifacts are incomplete or untraceable.	Block figure/card export until lineage, config hashes, and source records are restored.	

A negative-result note should be saved with the same run ID and ablation-step ID as the original result. The dashboard should expose negative results rather than hiding them.

22.14 Reproducibility and implementation linkage

Every evaluation run must be traceable from input records to thesis artifact. A result is not thesis-ready unless the following identifiers are saved: run ID, experiment ID, ablation step ID, dataset ID, subject/session IDs, split ID, low-calibration budget ID, preprocessing ID, model ID, calibration ID, threshold registry ID, IHARQ configuration ID, RegimeRisk configuration ID, policy configuration ID, reward/cost configuration ID, stress profile ID, embodiment mode ID, dashboard export ID, seed, code version, configuration hash, and record schema version.

Conceptual repository locations should align with the repository architecture:

- **configs/**: evaluation configs, ablation configs, threshold configs, policy configs, reward/cost configs, stress configs, embodiment configs, dashboard export configs.
- **records/**: source records and produced `ExperimentSummaryRecord`, `EvaluationOutcomeRecord`, `DashboardMetricRecord`, and validation reports.
- **figures/**: generated plots only, each with a figure manifest.
- **cards/**: reliability, regime, policy, stress, controller, embodiment, reproducibility, and ablation cards.
- **dashboard_app/**: read-only dashboard code and loaders.

- **logs/**: execution logs, validation logs, failure-audit logs, and warning logs.
- **thesis_tables/**: exported tables with source-record lineage.
- **tests/**: schema tests, metric tests, matching tests, leakage tests, dashboard-loader tests.
- **manifests/**: reproducibility manifests, export manifests, figure/table manifests, and claim-evidence manifests.

Traceability must be bidirectional. From a thesis figure, the examiner should be able to find source records, configs, seeds, code version, and schema version. From a record, the examiner should be able to find every figure, card, table, dashboard page, and claim that used it.

22.15 Dashboard and reporting linkage

This layer feeds Layer 10 through saved records only. The dashboard must not silently recompute experiment logic, retune thresholds, change stress schedules, change reward weights, or repair missing records. It may filter, aggregate, visualize, and export only from saved evaluation artifacts.

Required dashboard views include:

Dashboard view	Source records and filters	Plots/tables	Warnings and exports
Ablation overview	ExperimentSummaryRecord, ablation IDs, dataset/subject/model/stress filters.	A0–A13 ladder table, metric deltas, matched-comparison status.	Missing ablation, unmatched comparison, diagnostic-only export.
Decoder/calibration comparison	PredictionRecord, ModelRegistryRecord, BaselineMetricRecord, Layer2ReadinessReport, CalibrationRecord, UncertaintyRecord, SplitIntegrityReport, ReliabilityAuditReport, and HighConfidenceWrongTable, and CalibrationDashboardMetricRecord.	Accuracy/balanced accuracy, ECE/Brier/NLL, reliability diagrams, high-confidence wrong audit, matched operating-point table, and readiness/eligibility status.	Calibration split warning, pooled-only warning, downstream-readiness warning, reliability card export, and dashboard lineage export.
IHARQ versus threshold	Selective prediction and IHARQ records.	Risk-coverage, accepted accuracy, wrong-accept rate, reason-code distribution, burden table.	Matched-coverage/cost warning, ablation card export.
RegimeRisk-aware versus regime-unaware	Regime, trust, IHARQ, stress-conditioned records.	Regime timeline, high-risk accepted error, stop-loss events, stress-stratified comparison.	Proxy drift/fatigue warning, regime card export.

Policy comparison	Policy action, readiness context, deferral, cost, and reward records.	Action distribution, cost, reward, deferral/fallback burden, subject stability.	Proxy-label warning, cost-sensitivity warning, controller card export.
Learning-to-defer	Deferral records, fallback records, IHARQ records.	Deferral rate, fallback burden, retained accuracy, wrong-accept reduction.	Fallback definition warning, deferral baseline export.
Bandit/RL simulation	Transition, reward, rollout, episode, and safety-event records.	Reward/regret, episode return, unsafe events, held-out rollout comparison.	Simulation-only warning, frozen-reward warning, controller card export.
StressForge robustness	Stress profile, stress schedule, stressed decision, stressed transition records.	Clean versus stress curves, stress intensity sensitivity, failure taxonomy.	Stress proxy warning, held-out stress warning, StressCard export.
Layer 9 embodiment	Embodiment state/outcome records, embodiment-linked transitions, safety-event logs, Layer9RewardTrace files where enabled, command timelines, endpoint-error curves, Layer9EmbodimentDemoManifest files, adapter logs, and policy actions.	Endpoint error, command success, blocked command, unsafe-event rate, safe-hold burden, reward terms where enabled, adapter failure, and manifest completeness.	Simulation-only embodiment warning, embodiment card export, demo-manifest index, and dashboard-compatible Layer 9 export.
Failure taxonomy	Audit logs, unsafe events, negative-result notes, missingness reports.	Failure categories, case examples, affected modules, claim consequence.	Hidden-failure warning, discussion-table export.
Cards and export	Card records, manifests, figure/table lineage, dashboard metrics.	Card completeness, artifact lineage, export manifest status.	Export blocked if lineage is missing.
Claim-safety warnings	Layer 0 scope tags, claim-evidence matrix, warning flags.	Supported/unsupported claims, limitation map, invalidation conditions.	Clinical/deployment overclaim warning.

Each dashboard view must state its source records, main filters, main plots/tables, warning flags, exported thesis artifacts, and missingness behavior. If source records are absent, the view should show missingness explicitly rather than computing a substitute result.

22.16 Claim-safety and boundary discipline

This evaluation architecture may support the following claims:

- The architecture evaluates reliability of low-calibration neural-intent decoding on public EEG datasets.
- Calibration and uncertainty reveal reliability failures hidden by accuracy.
- IHARQ-style evidence verification can be compared against thresholds, selective prediction, longer windows, voting, averaging, and ensembles.
- RegimeRisk can be tested as temporal trust context.
- Adaptive readiness policies can be evaluated using declared proxy labels and cost functions.
- Contextual bandit and RL policies can be evaluated in simulated environments.
- StressForge can test controlled stress robustness and reveal robustness boundaries.
- MyoSuite/OpenSim can provide a simulated embodiment demonstration.
- Dashboard and cards improve traceability, inspection, missingness visibility, and reproducibility.

It must not support the following claims:

- Clinical BCI-FES validation.
- Real patient safety.
- Medical-device readiness.
- Real rehabilitation efficacy.
- Real muscle stimulation control.
- Real-time deployed online control.
- True clinical readiness labels from public EEG.
- Biological validation of fatigue or drift unless such labels exist.
- OpenSim/MyoSuite proof of real human motor outcomes.
- RL controller safety outside the declared simulator.

Every A10, A11, A12, and A13 result must carry simulation-only, stress-proxy, or embodiment-proxy warnings where appropriate. Every figure, table, card, and dashboard export must inherit these warnings.

22.17 Definition of thesis-ready evaluation

An evaluation result is thesis-ready only if all of the following conditions hold:

1. The evaluation mode is declared and compatible with the claim.
2. All required source records exist and pass schema validation.
3. Dataset, split, preprocessing, label map, budget, seed, calibration, threshold, policy, reward, stress, and embodiment identifiers are recorded where applicable.

4. Comparisons are matched or explicitly labeled diagnostic.
5. Thresholds, policies, rewards, and stress schedules are not tuned on the test set.
6. Evaluation-only fields are not visible to decision-time policies.
7. Metrics are defined in a metric dictionary and computed from saved record fields.
8. Subject-level or session-level aggregation is reported where possible.
9. Sensitivity or robustness analysis is included for thresholds, costs, rewards, stress intensity, and label budgets where relevant.
10. Negative results and failure cases are visible.
11. Dashboard artifacts read saved records only.
12. Figure/table manifests link every artifact to source records, configs, seeds, code version, and output hashes.
13. Claim-safety warnings match Layer 0 boundaries.

An evaluation is invalid or weakened if it uses untracked records, mixed splits, hidden test tuning, future leakage, unmatched baselines, unversioned stress schedules, unversioned reward functions, missing seeds without justification, dashboard recomputation drift, missing figure lineage, absent limitation statements, or unsupported clinical/deployment language.

This final checklist should be applied before any result enters the thesis results chapter, dashboard export package, cards appendix, or final architecture summary.

23 Implementation Pipeline for Stretch C

This section defines the implementation-governance pipeline for IHARQ BenchGuard under Stretch C. It is not merely a to-do list. It is the build-order layer that converts the preceding architecture into an executable, reproducible, claim-safe thesis system. The preceding layers define the scientific modules and their interfaces; this layer defines when each module is allowed to be implemented, which upstream artifacts it must consume, which downstream records it must produce, which validation gate must pass, and which thesis claim it can support.

The main bottleneck addressed by this section is **implementation credibility under maximum Stretch C scope**. A system that includes public EEG, calibration, IHARQ, RegimeRisk, learned readiness policies, learning-to-defer, simulated closed-loop control, stress generation, MyoSuite/OpenSim embodiment, dashboard exports, and final thesis evidence can easily collapse into an impressive but untraceable collection of scripts. The Implementation Pipeline section prevents that failure by enforcing a record-first, split-safe, stage-gated, reproducible, stoppable, and claim-safe pipeline.

If this section were removed, the architecture would still describe important scientific ideas, but it would not prove that those ideas can be built in a defensible order. The thesis would be vulnerable to the criticism that the system is too broad to implement, that advanced policies were introduced before stable calibration and IHARQ baselines existed, that pseudo-labels and simulation rewards were not controlled, that the dashboard generated untracked results, or that optional embodiment became a hidden blocker. The Implementation Pipeline section protects against these criticisms by making every phase produce named records, validation reports, cards, figures, and manifests that can be traced into the final thesis.

After the Layer 7 and Layer 10 propagation updates, this pipeline also treats optional contextual-bandit/RL policy mutation as a first-class implementation concern. Simulation-authorized policy updates are permitted only when a declared update configuration is active and every mutation can be traced through `PolicyUpdateTrace`. Frozen-policy evaluation remains the default evidence mode for policy comparison: update hooks must be disabled, disabled-update audit entries must be available where audited, and training/adaptation curves must not be merged with frozen-policy evaluation curves.

After the Layer 20-to-Layer 9 merge and the Layer 10, cross-layer, and proximity-map propagation updates, the pipeline also treats embodiment implementation as a Layer 9-owned branch rather than a detached Layer 20 dossier. Phase 12 implements and validates the L9.1–L9.13 embodiment subsystem; Phase 13 remains optional OpenSim replay/comparison; Phase 14 consumes saved Layer 9 embodiment artifacts through read-only Layer 10 dashboard/card/export logic.

23.1 Thesis role and contribution of the implementation pipeline

The thesis role of this section is to demonstrate that IHARQ BenchGuard is not only a conceptual architecture but also a disciplined implementation program. It supports the thesis claim that low-calibration neural-intent readiness can be studied as a modular reliability system rather than as a single decoder-accuracy benchmark. It also supports the engineering claim that each module can be implemented, evaluated, ablated, and stopped at meaningful capability tiers without invalidating the thesis.

the Implementation Pipeline section contributes to four defensibility goals:

1. **Build-order defensibility.** Repository, schemas, splits, baselines, calibration, IHARQ-lite, and RegimeRisk must exist before learned policies, simulation, stress, embodiment, or dashboard claims are presented.
2. **Evidence defensibility.** Each phase must produce records and cards that become thesis evidence rather than undocumented intermediate variables.
3. **Claim-safety defensibility.** Public-data, proxy-label, simulation-only, and non-clinical embodiment boundaries must be repeated at the relevant implementation gates.
4. **Fallback defensibility.** The thesis remains valid at C0, C1, C2, C3, or C4 depending on the highest completed and validated capability tier.

This layer therefore supports the research questions concerning reliability beyond accuracy, evidence verification, temporal trust, adaptive readiness decisions, stress behavior, and simulated embodied consequences. It does not itself introduce a new scientific model; its contribution is disciplined integration, dependency control, output traceability, and thesis-readiness governance.

23.2 Pipeline design principles

The implementation pipeline follows six non-negotiable design principles.

Record-first. Every implemented module must read canonical records and produce canonical records. Hidden arrays, notebook-only outputs, manual spreadsheets, or dashboard-only

computations are not thesis artifacts unless exported as versioned records with a run identifier and configuration hash.

Split-safe. Dataset splits, calibration splits, threshold-selection splits, adaptation splits, and held-out evaluation splits must be declared before training or tuning. Test information must not influence decoder training, calibration, threshold selection, pseudo-label construction, policy tuning, stress selection, or dashboard exports.

Stage-gated. Later phases are not thesis-ready merely because code exists. A phase becomes thesis-ready only when its inputs, outputs, validators, exit condition, failure interpretation, and thesis-evidence role are documented.

Reproducible. Each phase must be runnable from saved configurations, seeds, environment files, versioned schema definitions, and run manifests. Each figure, table, card, and dashboard export must link back to source records and configuration hashes.

Stoppable. The thesis must remain coherent if the project stops at C0, C1, C2, C3, or C4. Optional MyoSuite/OpenSim, contextual bandit, or RL branches may strengthen the work, but they must not be required for the validity of earlier empirical contributions.

Claim-safe. Public EEG supports offline reliability and simulated readiness analysis, not clinical deployment. Adaptive-IHARQ and learning-to-defer use computational proxy labels, not real clinical readiness labels. Bandit and RL policies are simulation-only. MyoSuite/OpenSim outputs are proxy embodiment demonstrations, not validation of real rehabilitation control.

23.3 Layer boundary, scope, and expected implementation artifacts

The technical responsibility of the Implementation Pipeline section is to define the implementation order, dependency gates, artifact contracts, validation checks, and thesis-traceability rules for the complete Stretch C system. It does not replace the technical definitions in earlier layers. Instead, it orchestrates their build order.

In scope:

- repository skeleton, configuration families, run registry, schema registry, and validation scripts;
- public-data ingestion, preprocessing records, split protocols, and low-calibration budgets;
- decoder training and prediction-record export;
- calibration, uncertainty, selective prediction, threshold registry, and reliability audits;
- IHARQ-lite decision records, RegimeRisk temporal-trust records, and readiness-context records;
- evidence-quality estimation, supervised Adaptive-IHARQ, learning-to-defer, contextual bandit, and RL policy phases;
- simulated closed-loop transition records, StressForge-Lite stress records, MyoSuite embodiment logs, and optional OpenSim replay outputs;
- dashboard records, static exports, cards, figures, tables, artifact manifests, and final claim-to-artifact traceability.

Out of scope:

- clinical deployment, stimulation, rehabilitation efficacy claims, real-time medical safety certification, or patient-facing control;
- changing the scientific meaning of earlier layers;
- allowing optional OpenSim, RL, or advanced dashboard work to block lower-tier thesis completion;
- treating pseudo-labels as ground-truth readiness labels;
- allowing the dashboard to silently recompute or tune results outside the record system.

The expected implementation artifacts are: configuration files under `configs/`; source modules under `src/`; validated records under `records/`; model artifacts under `models/`; figure outputs under `figures/`; cards under `cards/`; dashboard application files under `dashboard_app/`; thesis tables under `thesis_tables/`; logs under `logs/`; validation tests under `tests/`; and final traceability files under the thesis-reporting directory.

23.4 Capability tiers and thesis-valid stopping points

The pipeline is intentionally staged so that incomplete Stretch C work does not invalidate completed lower tiers. The following stopping tiers define what the thesis can honestly claim at each maturity level.

Tier	Highest required phases	Thesis-valid output if stopping here
C0	Phases 0–3	A defensible public EEG low-calibration reliability benchmark with standardized records, fixed splits, baseline decoders, calibrated uncertainty, selective-prediction baselines, reliability diagrams, and high-confidence-wrong audits.
C1	Phases 0–5	A reliability-control architecture with IHARQ-lite evidence verification, RegimeRisk temporal-trust characterization, reason-coded readiness decisions, temporal warning records, and interpretable comparisons against simple baselines.
C2	Phases 0–7	A learned readiness-policy study with evidence-quality estimation, supervised Adaptive-IHARQ, learning-to-defer, cost-sensitive pseudo-label analysis, matched-cost and matched-coverage comparisons, and explicit proxy-label limitations.

C3	Phases 0–11	A simulated closed-loop policy-learning evaluation with transition records, StressForge-Lite robustness tests, contextual bandit adaptation, optional RL policy learning, reward sensitivity analysis, and simulation-only claim tags.
C4	Phases 0–15	The maximum Stretch C artifact with Layer 9-owned simulated embodiment/static replay, optional OpenSim replay, Layer 10 read-only embodiment observability, cards, thesis figures, traceability matrix, repository manifest, and final claim-safety documentation.

C0 is the minimum empirical spine. C1 is the minimum IHARQ and RegimeRisk contribution. C2 is the minimum learned-readiness contribution. C3 is the minimum simulated closed-loop policy contribution. C4 is the complete maximum Stretch C architecture.

23.5 Phase overview matrix

Phase	Main output	Primary consumer	Minimum exit gate
0	Schemas, configs, repository skeleton, validation scripts, mock records, evaluation artifact schemas, and claim-evidence fixtures	All phases	Mock records validate, evaluation artifacts validate, and every phase has a planned output location.
1	DatasetRecord, WindowRecord, SplitRecord, dataset/protocol cards, and layer1_ablation_readiness_l1_v1.json	Decoders, calibration, stress, dashboard, evaluation, final demo	Split-safe standardized windows, split manifests, and matched-comparison key completeness reports exist.
2	PredictionRecord, checkpoints, decoder cards, baseline metrics, A0 raw-decoder tables, and A4 ordinary-combination control summaries	Calibration, IHARQ, policy, evaluation	Predictions and baseline/control evidence exist across selected datasets, subjects, splits, model families, and label budgets.
3	CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, Layer3MatchedOperatingPointTable, and negative-result notes	IHARQ, RegimeRisk, policy, dashboard, evaluation, final demo	Calibrated/uncertainty outputs, matched operating points, negative-result notes, and threshold provenance are leakage-safe and recorded.

4	IHARQDecisionRecord, Policy, simulation, rule-derived embodiment, EvidenceQualityRecord, dashboard, ablation, CombinedEvidenceRecord, reproducibility FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, Layer 4 EvaluationOutcomeRecord, and reason-code taxonomy	IHARQ-lite can be compared with confidence, entropy, margin, selective prediction, voting, averaging, and ensemble baselines using matched-comparison, trace, and claim-boundary lineage.
5	RegimeStateRecord, IHARQ, policy, conditional simulation, TrustStateRecord, embodiment, regime_timelines.csv, dashboard, evaluation trust_scores.csv, and stop_loss_warnings.csv	Regime context, trust-state links where exported, and stop-loss warnings join cleanly with decision and readiness records without split, identity, or temporal-causality mismatch.
6	ReadinessContextRecord, Deferral, simulation, EvidenceQualityRecord, dashboard, evaluation PolicyActionRecord, pseudo-label files	Adaptive-IHARQ comparisons exist with proxy-label and cost-model cards.
7	DeferralRecord, Simulation, dashboard, deferral curves, evaluation fallback-source logs	Matched-cost and matched-coverage comparisons exist against Adaptive-IHARQ and simpler policies.
8	TransitionRecord, Bandit, RL, stress, ClosedLoopSessionRecord, dashboard EpisodeSummaryRecord, PolicyRolloutRecord, RewardTraceRecord, EnvironmentDiagnosticRecord, SafetyEventRecord where materialized, optional PolicyUpdateTrace, simulated session logs, and reward/cost curves	Stable state, action, transition, reward, delay, fallback, safety-event, environment-diagnostic, and update/frozen-mode definitions exist.
9	StressProfileRecord, Policy, simulation, stress schedules, embodiment, stress-run manifests, dashboard stress cards	Matched stress profiles are reproducible and not tuned on test outcomes.
10	Bandit Evaluation, dashboard, PolicyActionRecord, optional RL baseline reward/regret curves, bandit card	Bandit policies are evaluated against non-bandit policies under clean and stressed simulation.

11	RL policy artifacts, RL PolicyActionRecord , controller cards	Evaluation, dashboard, thesis stretch analysis	RL is evaluated on held-out simulated sessions and stress profiles with simulation-only tags.
12	EmbodimentStateRecord , Layer 10 EmbodimentOutcomeRecord , dashboard/cards, thesis embodiment-linked demo, A13 evaluation, TransitionRecord , optional OpenSim SafetyEventRecord or safety-event logs, Layer9RewardTrace where enabled, command timeline, endpoint-error curve, command-mapping manifest, Layer9EmbodimentDemoManifest , dashboard-compatible Layer 9 export, and embodiment cards		Readiness-gated simulated action outcomes are logged through Layer 9 L9.1–L9.13 declared proxy mappings, outcome records, safety gates, adapter contracts, Layer9EmbodimentDemoManifest files, and simulation-only reward-trace boundaries where enabled.
13	OpenSim output manifest, screenshots or plots, replay logs, optional comparison report, and optional linkage to Layer9EmbodimentDemoManifest	Layer 10 dashboard/cards, thesis demo, future-work appendix	Outputs are documented as optional biomechanical visualization or replay, not clinical validation or an independent runtime layer.
14	DashboardMetricRecord , Thesis figures, tables, DashboardViewRecord , appendix dashboard app, static figures, cards, export manifest, metric dictionary exports, matched-comparison reports, diagnostic-only status exports, negative-result note index, and ClaimEvidenceMatrix views		Dashboard reads saved records only and every export links to source records, source metric/view records, metric definitions, matched-comparison status, diagnostic-only status, claim-evidence entries, and config hashes.
15	Claim-to-artifact traceability matrix, ClaimEvidenceMatrix , reproduction guide, final limitation map, and negative-result register	Final thesis	Every major claim maps to evidence, metric definitions, ablation lineage, negative-result interpretation, or a bounded limitation statement.

23.6 Consistent phase template

Every phase below is specified using the same implementation-governance template:

- **Purpose and bottleneck controlled:** why the phase exists and what failure it prevents.
- **Preconditions and dependencies:** which records, configs, tests, or decisions must already exist.
- **Primary inputs:** canonical records, configuration files, datasets, models, manifests, or cards consumed.
- **Core implementation tasks:** activities that belong in the phase and should not be moved downstream.
- **Outputs and consumers:** records, models, logs, cards, figures, and downstream modules served.
- **Validation gate and exit condition:** checks that must pass before the phase can be treated as complete.
- **Risks, fallback, and limitations:** what can go wrong and how the thesis remains valid if the phase is partial.
- **Thesis evidence role:** which claim, table, figure, ablation, card, or limitation the phase supports.

23.7 Detailed phase specifications

23.7.1 Phase 0: Repository, configuration, and record schema

Purpose and bottleneck controlled. Phase 0 is the hard foundation gate. It prevents the entire Stretch C system from becoming a set of undocumented scripts by making the architecture record-driven before any advanced modeling begins. It controls schema drift, inconsistent run identifiers, hidden intermediate variables, untraceable dashboard outputs, and irreproducible experiments.

Preconditions and dependencies. This phase depends conceptually on the full architecture document, the canonical record list, the repository architecture, the experiment naming policy, and the Layer 0 claim-safety boundaries. It has no data or model dependency and can be developed entirely with mock records.

Primary inputs. Full architecture specification; canonical record names; planned folder structure; seed policy; environment assumptions; schema-version policy; claim-safety vocabulary; evaluation-mode vocabulary; ablation-step vocabulary; metric-dictionary requirements; matched-comparison key requirements; naming conventions for datasets, subjects, sessions, runs, models, policies, stress profiles, and dashboard exports.

Core implementation tasks.

- Add optional `PolicyUpdateTrace` and disabled-update audit fixtures to the Phase 0 schema and mock-record suite so update-enabled and frozen-evaluation runs can be validated before Layer 7/10 implementation.

- Create the repository skeleton with separate folders for `configs/`, `src/`, `records/`, `models/`, `figures/`, `cards/`, `dashboard_app/`, `notebooks/`, `thesis_tables/`, `logs/`, and `tests/`.
- Define schema files for `DatasetRecord`, `WindowRecord`, `SplitRecord`, `PreprocessingRecord`, `LabelMapRecord`, `ArtifactFlagRecord`, `Layer1Manifest`, `PredictionRecord`, `CalibrationRecord`, `UncertaintyRecord`, `SelectivePredictionRecord`, `ThresholdRegistryRecord`, `EvidenceQualityRecord`, `IHARQDecisionRecord`, `RegimeStateRecord`, `ReadinessContextRecord`, `DeferralRecord`, `PolicyActionRecord`, `TransitionRecord`, `StressProfileRecord`, `EmbodimentStateRecord`, `SafetyEventRecord`, `Layer9RewardTrace`, `Layer9EmbodimentDemoManifest`, `DashboardMetricRecord`, `MetricDictionaryEntry`, `MatchedComparisonReport`, `ClaimEvidenceMatrix`, `Layer3MatchedOperatingPointTable`, and `NegativeResultNote`.
- Define the optional `PolicyUpdateTrace` schema and the expected disabled-update audit entry fields: run ID, policy ID, update mode, frozen-policy flag, trace ID when present, source `TransitionRecord`, reward version, seed, config hash, and simulation-only tag.
- Create configuration families for datasets, decoders, calibration, IHARQ, RegimeRisk, adaptive policies, deferral, simulation, StressForge-Lite, contextual bandit, RL, embodiment, dashboard, evaluation, cards, and reporting.
- Add a policy-update configuration family that distinguishes disabled/frozen evaluation, simulation-training, and simulation-adaptation modes.
- Define a run registry with run identifier, config hash, schema version, code version, seed list, data version, split identifier, model identifier, policy identifier, and output manifest.
- Create mock records for every major record type and require validators to pass on those mock records.
- Create validation scripts for schema compatibility, required fields, split identifiers, Layer 1 ablation-readiness status, matched-comparison key completeness, metric-dictionary completeness, threshold-registry provenance, Layer 3 matched operating-point tables, policy action legality, stress profile completeness, simulator adapter compatibility, dashboard export lineage, claim-evidence matrix linkage, negative-result note visibility, and claim-safety tags.
- Create validation scripts for `PolicyUpdateTrace` completeness, `TransitionRecord.policy_update_trace_id` linkage where present, disabled-update audits, `DashboardMetricRecord.policy_update_filter`, and frozen/update curve separation.

Outputs and consumers. Repository skeleton; schema registry; evaluation artifact schemas; config templates; run-registry template; seed policy; mock records; validation scripts; reproducibility README. Every later phase consumes these outputs. The dashboard, evaluation scripts, defensive-design register, final demonstration contract, and final thesis also consume the run registry, artifact manifest, metric dictionary template, matched-comparison template, and claim-evidence matrix template.

Validation gate and exit condition. No downstream phase may be called implemented until mock records validate, configuration hashing works, every phase has a planned output location, and the validation scripts can detect at least one deliberately malformed mock record. The exit

condition is a minimal record-first repository that can generate and validate all mock canonical records.

The Phase 0 exit condition also requires two policy-update fixtures: an update-enabled mock run that fails if `PolicyUpdateTrace` is missing, and a frozen-evaluation mock run that fails if a policy mutation occurs or the disabled-update audit is absent when audit mode is enabled. It also requires a Layers 0–3 evaluation-slice fixture: a mock Layer 1 ablation-readiness file, A0/A4 decoder-control summary, A1–A3 matched operating-point table, metric dictionary, matched-comparison report, negative-result note, and claim-evidence matrix entry must validate together before downstream phase claims are considered traceable.

Risks, fallback, and limitations. If Phase 0 is rushed, later work becomes untraceable and the dashboard may silently create results. If full automation is too large, the fallback is a minimal but strict skeleton: schemas, configs, mock records, validators, run identifiers, and output directories. This phase does not produce scientific results; it produces the reproducibility substrate for all later claims.

Thesis evidence role. Phase 0 supports the reproducibility appendix, repository architecture section, artifact manifest, and defense argument that the architecture is implementable rather than merely conceptual.

23.7.2 Phase 1: Public data and split protocol

Purpose and bottleneck controlled. Phase 1 is the empirical anchor gate. It transforms public EEG datasets into standardized, split-safe records and prevents train/test contamination, inconsistent low-calibration budgets, untracked exclusions, label-map ambiguity, and dataset-specific assumptions from leaking into downstream decoders or policies.

Preconditions and dependencies. Phase 0 schemas, dataset configuration files, run registry, preprocessing configuration, and split-manifest format must exist. Hard dependencies include public EEG dataset access, label definitions, subject/session metadata, and low-calibration budget definitions. Additional datasets are soft dependencies; one primary validated dataset can support an initial C0 path.

Primary inputs. Raw or downloaded public EEG; dataset configs; label mapping rules; channel metadata; sampling-rate metadata; preprocessing config; inclusion/exclusion rules; train/validation/test/calibration/adaptation split protocol; subject/session/run identifiers; low-calibration budget settings.

Core implementation tasks.

- Load each selected public EEG dataset through a standardized ingestion interface.
- Normalize dataset, subject, session, run, channel, task-label, sampling-rate, and trial/window metadata.
- Apply declared preprocessing and windowing rules without using test labels for tuning.
- Generate fixed `WindowRecord` and `SplitRecord` outputs for each dataset and low-calibration budget.
- Produce `DatasetRecord`, `PreprocessingRecord`, `LabelMapRecord`, `ArtifactFlagRecord`, `DatasetCard`, `ProtocolCard`, `ValidationReport`, and `Layer1Manifest` outputs.

- Produce `manifests/layer1_ablation_readiness_l1_v1.json` with matched-comparison key completeness, missing-key warnings, and diagnostic-only downgrade reasons for later A0–A4 evaluation.
- Validate channel consistency, label consistency, split disjointness, subject/session identifiers, excluded-trial accounting, and matched-comparison key completeness.
- Create dataset cards and protocol cards that state what the public data can and cannot support.

Outputs and consumers. `DatasetRecord`, `WindowRecord`, `SplitRecord`, `PreprocessingRecord`, `LabelMapRecord`, `ArtifactFlagRecord`, `DatasetCard`, `ProtocolCard`, `ValidationReport`, `Layer1Manifest`, `manifests/layer1_ablation_readiness_l1_v1.json`, dataset cards, protocol cards, low-calibration budget table, matched-comparison key completeness report, and split-leakage report. Phase 2 consumes the window and split records. Phases 3–15 consume the split identifiers, dataset cards, protocol cards, and ablation-readiness status for evaluation and claim-safety.

Validation gate and exit condition. Every downstream phase must be able to read the same `WindowRecord` and `SplitRecord` format. No split leakage, label ambiguity, untracked exclusion, or missing matched-comparison key may remain unresolved unless it is explicitly marked as diagnostic-only in `manifests/layer1_ablation_readiness_l1_v1.json`. The exit condition is that one or more selected public EEG datasets have standardized windows, fixed splits, label maps, preprocessing manifests, ablation-readiness status, and cards that validate under the Phase 0 schema.

Risks, fallback, and limitations. The main risks are train/test contamination, subject/session leakage, inconsistent label definitions, over-narrow preprocessing assumptions, and undocumented dropped trials. If multi-dataset coverage is infeasible, the fallback is one primary dataset with a fully validated split protocol before expanding to secondary datasets. Public data must be described as an offline reliability anchor, not a clinical deployment population.

Thesis evidence role. Phase 1 supports the methods chapter, dataset cards, protocol cards, low-calibration setup, reproducibility appendix, and the validity boundary that the project uses public healthy or public benchmark EEG as an empirical proxy.

23.7.3 Phase 2: Baseline decoders

Purpose and bottleneck controlled. Phase 2 is the measurement spine gate. It creates fair decoder outputs for all later reliability, calibration, IHARQ, RegimeRisk, and policy phases. It prevents unfair model comparisons, missing raw scores for calibration, inconsistent seed handling, and downstream policy work built on unstable or incomplete predictions.

Preconditions and dependencies. Phase 1 `WindowRecord` and `SplitRecord` outputs must validate. Phase 0 model configuration and seed policy must exist. Hard dependencies include selected decoder implementations and metrics. Compact SSL or deep neural decoders are optional depending on compute and availability.

Primary inputs. `WindowRecord`; `SplitRecord`; `LabelMapRecord`; `PreprocessingRecord`; model configs; seed registry; low-calibration budgets; training-resource constraints; selected baseline family list.

Core implementation tasks.

- Train classical baselines, such as CSP plus LDA or SVM where applicable.

- Train Riemannian or tangent-space baselines where data format and channel consistency allow.
- Train compact neural baselines such as EEGNet or ShallowConvNet where feasible.
- Include compact SSL or ensemble baselines only if compute, licensing, and implementation stability permit.
- Generate A0 raw-decoder evidence and A4 ordinary-combination controls, including voting, averaging, longer-window, multi-window, and ensemble summaries where available.
- Preserve training, calibration, validation, adaptation, and test split separation.
- Export per-window or per-trial predictions, logits or scores when available, predicted labels, true labels for evaluation-only fields, subject/session identifiers, model identifiers, split identifiers, and seed identifiers.
- Produce subject-wise, session-wise, dataset-wise, and low-label baseline summaries.

Outputs and consumers. `PredictionRecord`; model checkpoints; training logs; decoder cards; baseline metrics; model config hash; baseline comparison tables; A0 raw-decoder evidence tables; A4 ordinary-combination control summaries; delay-burden notes for longer-window or multi-window controls where applicable. Phase 3 consumes raw probabilities, scores, logits, and labels. Phases 4–15 consume prediction identifiers, A0/A4 evidence, and baseline summaries for IHARQ, RegimeRisk, policies, simulation, stress, dashboard, ablations, and thesis tables.

Validation gate and exit condition. Per-window or per-trial `PredictionRecord` outputs must exist for every selected dataset, subject, split, model family, seed, and label budget intended for downstream comparison. Missing logits or probabilities must be explicitly marked. A0/A4 evidence must be matched on Layer 1 comparison keys or downgraded to diagnostic-only. The exit condition is a baseline prediction spine that supports calibration, ablation comparison, and later policy records without re-running hidden model code.

Risks, fallback, and limitations. Risks include unfair comparisons, decoder overfitting, missing score fields, seed inconsistency, and excessively broad model coverage. If full model-family coverage is infeasible, the fallback minimum is one classical baseline, one Riemannian or robust classical baseline, and one compact neural baseline where feasible. Phase 2 does not claim readiness; it produces decoder evidence for later reliability analysis.

Thesis evidence role. Phase 2 supports the baseline table, low-calibration curves, subject difficulty profiles, model-family comparison, and the argument that later reliability modules operate on a fair decoder spine rather than a single cherry-picked model.

23.7.4 Phase 3: Calibration and uncertainty

Purpose and bottleneck controlled. Phase 3 is the reliability-feature gate. It converts raw decoder outputs into calibrated probabilities, uncertainty features, threshold registries, selective-prediction baselines, and reliability audits. It prevents the thesis from relying on accuracy alone or treating raw confidence as trustworthy.

Preconditions and dependencies. Phase 2 `PredictionRecord` outputs must exist. Phase 1 split manifests must identify allowed calibration and threshold-selection splits. Phase 0 calibration configs and metric dictionaries must exist. Hard dependencies include reliable score or probability fields for each eligible decoder.

Primary inputs. `PredictionRecord`; `SplitRecord`; calibration split; threshold-selection rules; metric configs; uncertainty-feature config; reliability-diagram config; seed and run meta-data.

Core implementation tasks.

- Fit calibration methods only on allowed calibration or validation splits.
- Compute calibrated probabilities, confidence, entropy, margin, variation ratio where relevant, model disagreement, Brier score, negative log likelihood, and ECE-like summaries.
- Generate reliability diagrams, confidence histograms, and high-confidence-wrong audits.
- Create selective-prediction baselines using confidence, entropy, margin, and calibrated score thresholds.
- Export A1 calibration deltas, A2 threshold comparisons, and A3 uncertainty/selective-prediction comparisons as matched-coverage or matched-cost operating-point tables.
- Record negative-result notes when calibration fails, thresholding is sufficient under a declared protocol, or selective prediction matches later modules.
- Register thresholds in `ThresholdRegistryRecord` with source split, selection criterion, metric, run identifier, and leakage status.
- Export dashboard-ready Layer 3 metric files and split-integrity reports.

Outputs and consumers. `CalibrationRecord`, `UncertaintyRecord`, `SelectivePredictionRecord`, `ThresholdRegistryRecord`, `Layer3MatchedOperatingPointTable`, reliability diagrams, high-confidence-wrong audit, matched-coverage and matched-cost operating-point tables, negative-result notes, split-integrity report, and Layer 3 dashboard metrics. Phase 4 consumes uncertainty features and thresholds. Phase 5 consumes temporal uncertainty signals. Phase 6 consumes calibrated and uncertainty features for evidence-quality estimation. Phases 14–15 and the Final Demonstration section consume the matched operating points and negative-result notes for dashboard panels, ablations, and thesis claim boundaries.

Validation gate and exit condition. Every eligible decoder variant must have calibrated outputs, uncertainty features, threshold-registry entries, selective-prediction baselines, matched operating-point exports, negative-result visibility, and leakage-safe reliability artifacts. Any decoder without valid calibration must be marked as ineligible for calibrated policy comparisons. The exit condition is a complete calibration and uncertainty record set that can be joined to predictions and A0/A4 controls without split, threshold-selection, or metric-dictionary mismatch.

Risks, fallback, and limitations. Risks include threshold tuning on test data, unreliable confidence estimates, hidden recalibration, untraceable uncertainty features, and calibration overfitting in low-label regimes. The fallback is transparent temperature or Platt-style calibration plus confidence, entropy, and margin baselines before advanced uncertainty options. Calibration improves probability interpretation but does not guarantee true readiness.

Thesis evidence role. Phase 3 supports reliability diagrams, high-confidence-wrong analysis, calibration metrics, selective prediction curves, and the thesis claim that accuracy hides risk-relevant errors in low-calibration neural-intent decoding.

23.7.5 Phase 4: IHARQ-lite evidence verification

Purpose and bottleneck controlled. Phase 4 is the interpretable readiness-control gate. It creates a non-learned IHARQ-lite reference before any learned policy. It prevents Adaptive-IHARQ from being introduced without an interpretable baseline and protects the architecture from the criticism that IHARQ is only confidence thresholding.

Preconditions and dependencies. Valid `PredictionRecord`, `CalibrationRecord`, `UncertaintyRecord`, `ThresholdRegistryRecord`, `WindowRecord`, and split identifiers must exist. Optional dependencies include channel-subset stability features, multi-window aggregation features, and model-family agreement fields.

Primary inputs. Predictions; calibrated probabilities; uncertainty features; threshold registry; window metadata; subject/session metadata; optional stability features; IHARQ rule configuration; request-evidence budget; fallback configuration.

Core implementation tasks.

- Implement reason-coded readiness actions using the canonical stored Layer 4 tokens `accept`, `wait`, `request_evidence`, `combine_evidence`, `fallback`, `uncertain`, and `unsafe`, with display labels kept separate.
- Compute time-window stability using adjacent or repeated windows only when available without future leakage.
- Compute channel-subset stability where channel metadata and computational budget allow.
- Compute model-family agreement or ensemble disagreement where multiple decoder families exist.
- Define request-evidence budgets, wait limits, fallback triggers, unsafe triggers, and any no-action behavior only as an explicit fallback or downstream policy/simulation alias rather than a new stored Layer 4 token.
- Produce decision records with reason codes, input feature provenance, threshold identifiers, action-token/display-label mapping, and action legality status.
- Compare IHARQ-lite against confidence rejection, entropy rejection, margin rejection, selective prediction, longer-window decisions, multi-window voting, probability averaging, and ensemble baselines where available.

Outputs and consumers. `IHARQDecisionRecord`; rule-derived `EvidenceQualityRecord` where relevant; `CombinedEvidenceRecord`; `FallbackDecisionRecord`; `UnsafeEventRecord`; `IHARQAblationRecord`; `IHARQTraceRecord`; Layer 4 `EvaluationOutcomeRecord`; reason-code taxonomy; evidence request logs; fallback logs; unsafe-event logs; IHARQ reliability card; decision summary tables. Phase 5 consumes IHARQ decisions for temporal trust. Phase 6 consumes IHARQ-lite decisions and evidence-quality features as an interpretable comparator and policy input. Phase 8 consumes decision actions for simulated replay through Layer 7. Phase 14 consumes decision records, trace records, ablation records, and cards.

Validation gate and exit condition. `IHARQDecisionRecord` and the associated Layer 4 trace, fallback, unsafe-event, ablation, and evaluation records must exist when their behaviors are active and must be comparable against simpler baselines at matched coverage, matched request budget, or matched cost. Each decision must include the features, threshold identifiers,

canonical action token, display-label mapping, and reason code that produced it. The exit condition is an interpretable IHARQ-lite layer that can operate on saved records only.

Risks, fallback, and limitations. Risks include uncontrolled evidence requests, unclear unsafe behavior, excessive abstention, and IHARQ being reducible to thresholding. The fallback is a transparent rule-based IHARQ-lite version using calibrated confidence, entropy, margin, and a small number of stability features. IHARQ-lite is not a clinical safety controller; it is a computational evidence-verification baseline.

Thesis evidence role. Phase 4 supports IHARQ cards, matched-baseline ablations, readiness-action summaries, reason-code analysis, and the thesis claim that evidence verification can reduce unsafe accepted decisions compared with simple thresholding under declared costs.

23.7.6 Phase 5: RegimeRisk temporal trust

Purpose and bottleneck controlled. Phase 5 is the temporal reliability gate. It monitors changing reliability across time, subject, session, stress condition, and decoder behavior. It prevents the system from assuming stationary reliability and provides context to IHARQ, adaptive policies, simulation, stress analysis, and dashboard views.

Preconditions and dependencies. Phase 3 uncertainty records and Phase 4 IHARQ decision records must exist. Phase 1 subject/session/run metadata and artifact flags must be joinable. Hard dependencies include temporal ordering and stable identifiers. Real regime labels are not required and usually will not exist.

Primary inputs. `PredictionRecord`; `CalibrationRecord`; `UncertaintyRecord`; `IHARQDecisionRecord`; `WindowRecord`; subject/session metadata; artifact flags; rolling-window configuration; RegimeRisk metric dictionary.

Core implementation tasks.

- Compute rolling features such as confidence volatility, entropy volatility, margin changes, agreement instability, error clusters, request-evidence bursts, and fallback bursts.
- Compute drawdown-like reliability degradation, stop-loss warning signals, temporal trust scores, and regime-risk summaries.
- Generate regime labels or trust states only as computational temporal-trust categories, not as medically validated physiological regimes.
- Record monitoring-window size, stride, eligible fields, and provenance for each score.
- Define recommendations to downstream IHARQ or policy modules, such as tighten threshold, request evidence, fallback, or mark unsafe.
- Export RegimeRisk timelines, trust-score summaries, stop-loss warning logs, and regime cards.

Outputs and consumers. `RegimeStateRecord`; conditional `TrustStateRecord` where independent trust timelines are consumed; `regime_timelines.csv`; `trust_scores.csv`; `stop_loss_warnings.csv`; RegimeRisk timelines; trust-score summaries; stop-loss warning logs; regime cards; and dashboard metrics. Phase 6 consumes `RegimeStateRecord` fields and conditional `TrustStateRecord` links through `ReadinessContextRecord`. Phase 8 consumes regime context for stress-response analysis. Phase 7 consumes regime context for simulated

trajectories. Phase 9 uses regime context for simulation-only embodiment gating. Phase 14 displays temporal trust and failure regions.

Validation gate and exit condition. `RegimeStateRecord` outputs and conditional `TrustStateRecord` links must join cleanly with `IHARQDecisionRecord` and `ReadinessContextRecord` using dataset, subject, session, window, split, run, and temporal-cutoff identifiers. No future correctness field may be visible to decision-time `RegimeRisk` features. The exit condition is a temporal trust layer that can summarize reliability variation without requiring true regime labels.

Risks, fallback, and limitations. Risks include overclaiming true regime detection, using future correctness circularly, noisy risk scores, excessive abstention, and confusing artifacts with physiological regimes. The fallback is to frame `RegimeRisk` as temporal trust characterization and warning rather than formal regime classification. It remains a proxy reliability-monitoring tool.

Thesis evidence role. Phase 5 supports `RegimeRisk` cards, temporal failure plots, stop-loss warning analysis, stability ablations, and the thesis claim that reliability varies over time and should be monitored rather than assumed stationary.

23.7.7 Phase 6: Evidence-quality estimator and supervised Adaptive-IHARQ

Purpose and bottleneck controlled. Phase 6 is the first learned policy gate. It introduces evidence-quality estimation and supervised Adaptive-IHARQ under proxy-label constraints. It prevents premature RL by first learning a transparent offline readiness policy from declared features, costs, legality masks, and pseudo-label construction rules.

Preconditions and dependencies. Phases 0–5 must be complete enough to provide prediction, calibration, uncertainty, IHARQ, `RegimeRisk`, and split records. Hard dependencies include `ReadinessContextRecord` schema, action-space definition, legality masks, cost-model configuration, target-construction logic, and train/validation/test separation. Optional dependencies include deferral baselines and simulation rewards, but these must not leak into decision-time features unless explicitly allowed for training targets.

Primary inputs. `PredictionRecord`; `CalibrationRecord`; `UncertaintyRecord`; `IHARQDecisionRecord`; `RegimeStateRecord`; split-safe correctness labels for evaluation or target construction; action-space file; legality-mask file; cost-model config; reward/cost files; pseudo-label generation config.

Core implementation tasks.

- Run prerequisite checks confirming that policy-input fields are separated from evaluation-only targets such as correctness, future evidence, or simulated rewards.
- Assemble `ReadinessContextRecord` entries containing calibrated confidence, entropy, margin, disagreement, stability features, `RegimeRisk` fields, subject/session metadata allowed by protocol, and action-legality fields.
- Train evidence-quality estimators that predict whether the current evidence is reliable under declared proxy definitions.
- Define action space, legality masks, cost vectors, reward components, and pseudo-label construction rules.
- Generate cost-based pseudo-labels and save target-construction cards.

- Train supervised Adaptive-IHARQ policies and evaluate sensitivity to cost weights, threshold choices, feature subsets, and subject/session splits.
- Compare against IHARQ-lite, selective prediction, longer-window strategies, voting, averaging, ensembles, confidence-only policies, and deferral baselines when available.

Outputs and consumers. `ReadinessContextRecord`; `EvidenceQualityRecord`; `PolicyActionRecord`; pseudo-label files; target-construction cards; cost-model cards; controller cards; reliability-vs-coverage curves; action-distribution reports; Adaptive-IHARQ comparison tables. Phase 7 consumes readiness contexts and policy actions. Phase 8 consumes policies for replay. Phase 14 consumes controller metrics and cards.

Validation gate and exit condition. Adaptive-IHARQ must be evaluated against simpler policy baselines under matched cost, coverage, and split conditions. Pseudo-label construction must be versioned and separated from evaluation-only leakage. The exit condition is a supervised learned readiness policy that is clearly labeled as proxy-label based and computational, not clinical.

Risks, fallback, and limitations. Risks include treating pseudo-labels as clinical ground truth, target leakage from future windows, action imbalance, arbitrary cost weights, and weak baselines. The fallback is to preserve evidence-quality estimation and supervised policy analysis as offline proxy readiness experiments if later simulation phases are delayed. Learned readiness labels are computational artifacts.

Thesis evidence role. Phase 6 supports Adaptive-IHARQ results, cost-sensitivity analysis, evidence-quality tables, controller cards, and the thesis claim that readiness decisions can be learned from calibrated uncertainty and temporal trust features under explicit proxy-label limits.

23.7.8 Phase 7: Learning-to-defer

Purpose and bottleneck controlled. Phase 7 is the routing and fallback baseline gate. It asks whether uncertain cases should be deferred, routed, waited on, or marked unsafe rather than simply rejected. It prevents the thesis from treating abstention as the only option for uncertain evidence.

Preconditions and dependencies. Phase 6 `ReadinessContextRecord`, `EvidenceQualityRecord`, and `PolicyActionRecord` outputs must exist. Deferral target definitions, fallback-source assumptions, and matched-cost evaluation rules must be declared. Real expert labels are not required; proxy or simulated fallback sources must be labeled honestly.

Primary inputs. `ReadinessContextRecord`; `EvidenceQualityRecord`; `PolicyActionRecord`; fallback/expert-proxy definitions; cost configs; coverage targets; policy comparison configs.

Core implementation tasks.

- Define the difference between defer, wait, request evidence, fallback, unsafe, and reject/no-action.
- Declare fallback or expert-proxy sources, such as longer-window evidence, ensemble consensus, oracle-for-analysis-only labels, or simulated expert proxies.
- Implement learning-to-defer baselines and deferral-aware readiness actions.

- Log fallback availability, deferral reason, deferral cost, deferred-case distribution, and post-deferral outcome where available.
- Compare at matched coverage, matched request budget, matched cost, and matched fallback burden.

Outputs and consumers. `DeferralRecord`; learning-to-defer curves; matched-cost comparison tables; matched-coverage comparison tables; fallback-source logs; deferral cards; dashboard metrics. Phase 8 consumes deferral policies for simulated trajectories. Phase 14 visualizes deferral burden and routing behavior.

Validation gate and exit condition. Learning-to-defer must be fairly compared against Adaptive-IHARQ, IHARQ-lite, selective prediction, confidence-only rejection, and accept-all baselines under matched cost or coverage. The exit condition is a deferral analysis that makes routing behavior explicit rather than hiding it inside rejection.

Risks, fallback, and limitations. Risks include calling rejection deferral without a real target, using unrealistic expert proxies, ignoring fallback burden, and overclaiming human-in-the-loop validity. The fallback is to present learning-to-defer as a strong computational baseline and analysis layer, with fallback sources labeled as proxy or simulated.

Thesis evidence role. Phase 7 supports deferral curves, matched-cost and matched-coverage analyses, fallback-burden tables, and the thesis argument that readiness systems should evaluate routing choices, not only accept/reject decisions.

23.7.9 Phase 8: Simulated closed-loop readiness

Purpose and bottleneck controlled. Phase 8 is the sequential consequence gate. It turns static prediction and decision records into repeated-session policy trajectories with state, action, transition, reward, delay, request count, fallback, and unsafe events. It prevents contextual bandit or RL claims before a stable simulator exists.

Preconditions and dependencies. Phases 4–7 must produce IHARQ, policy, and deferral actions. State, action, reward, transition, delay, and episode-boundary definitions must be declared. Hard dependencies include an environment API and held-out simulated sessions. Stress profiles are optional at this phase but become important for Phase 9 onward.

Primary inputs. `ReadinessContextRecord`; `PolicyActionRecord`; `DeferralRecord`; `IHARQDecisionRecord`; `RegimeStateRecord`; simulation configs; reward/cost files; episode definitions; delay and request-count settings; fallback and unsafe-event definitions.

Core implementation tasks.

- Implement a repeated-session replay environment that consumes saved readiness contexts and policy decisions.
- Define state fields, legal actions, transition dynamics, reward components, delay penalties, request-evidence costs, fallback costs, unsafe penalties, and episode termination rules.
- Generate `TransitionRecord`, `RewardTraceRecord`, `ClosedLoopSessionRecord`, `EpisodeSummaryRecord`, `PolicyRolloutRecord`, `EnvironmentDiagnosticRecord`, and `SafetyEventRecord` outputs where materialized for rule-based IHARQ, supervised Adaptive-IHARQ, learning-to-defer, confidence-threshold, accept-all, and other baseline policies.

- Define held-out simulated sessions for policy evaluation and avoid tuning environment parameters on final evaluation outcomes.
- Export episode summaries, rollout records, reward/cost curves, action-distribution trajectories, safety-event summaries where materialized, environment diagnostics, limitation cards, and dashboard-ready trajectory files.
- Keep Phase 8 frozen by default: replayed policies may generate transition, reward, rollout, diagnostic, and safety-event outputs, but they must not mutate policy parameters during frozen-policy evaluation.
- If a policy-update hook is configured for later Phase 10–11 experiments, Phase 8 must either disable it and write a disabled-update audit entry or reserve the `runs/<run_id>/results/layer7_closed_loop/policy_updates/` path without emitting update traces.

Outputs and consumers. `TransitionRecord`; `ClosedLoopSessionRecord`; `EpisodeSummaryRecord`; `PolicyRolloutRecord`; `RewardTraceRecord`; `EnvironmentDiagnosticRecord`; `SafetyEventRecord` where materialized; optional `PolicyUpdateTrace` only for declared simulation-update or disabled-update audit modes; simulated session logs; reward/cost curves; episode summaries; simulation limitation cards; dashboard trajectories. Phase 9 consumes transition, reward, and safety-event records for stress evaluation where relevant. Phase 10 and Phase 11 require stable transition and reward definitions. Phase 12 may consume transition and safety-event records for embodiment replay.

Validation gate and exit condition. `TransitionRecord`, `RewardTraceRecord`, `ClosedLoopSessionRecord`, `EpisodeSummaryRecord`, `PolicyRolloutRecord`, and `EnvironmentDiagnosticRecord` files must exist for all policy families selected for contextual bandit or RL comparison, with `SafetyEventRecord` files present whenever safety events are materialized, stable state/action/reward definitions, and held-out simulated sessions. The exit condition is a simulator that can replay simpler policies before any learned online policy is claimed.

Frozen-policy replay must pass a no-mutation check. Any transition that refers to `policy_update_trace_id` must have a valid corresponding `PolicyUpdateTrace`; otherwise the run is diagnostic-only and cannot support bandit/RL or frozen-policy evaluation claims.

Risks, fallback, and limitations. Risks include arbitrary simulator design, reward hacking, unclear delayed consequences, and conflating simulation outcomes with clinical outcomes. The fallback is static replay or a simple sequential simulation before attempting bandit or RL. All Phase 8 claims are simulation-only.

Thesis evidence role. Phase 8 supports simulated closed-loop curves, episode-level tables, sequential consequence analysis, and the thesis claim that readiness decisions can be evaluated by downstream action consequences rather than only per-window accuracy.

23.7.10 Phase 9: StressForge-Lite

Purpose and bottleneck controlled. Phase 9 is the robustness and criticism-test gate. It tests how decoders, IHARQ, RegimeRisk, policies, simulation, and embodiment behave under declared degradation profiles. It prevents the architecture from claiming reliability only under clean conditions.

Preconditions and dependencies. `Clean WindowRecord`, `PredictionRecord`, `ReadinessContextRecord`, and `TransitionRecord` outputs must exist. Stress-family

definitions, intensity levels, injection points, seeds, and guardrails must be declared. Hard dependencies include matched stress-run manifests and a rule that stress profiles are not chosen based on final test outcomes.

Primary inputs. Clean records; stress configs; seed registry; stress family definitions; target record types; injection-point definitions; guardrails; claim-boundary tags; selected policy and simulator outputs.

Core implementation tasks.

- Define stress families such as delay, dropout, noisy intent, weakness proxy, drift, fatigue-like degradation, and curated combined stress.
- Define intensity levels, schedules, seeds, affected fields, target records, injection points, and guardrails.
- Run clean, mild, moderate, severe, and curated combined stress profiles using matched stress-run manifests.
- Evaluate eligible policies under identical stress profiles and split conditions.
- Generate stress sensitivity curves, stress-conditioned reliability metrics, stress-conditioned policy actions, stress cards, and failure-taxonomy updates.

Outputs and consumers. `StressProfileRecord`; stress schedules; stress-run manifests; stress sensitivity curves; stress-conditioned metrics; stress cards; failure taxonomy updates. Phase 10 and Phase 11 consume stress profiles for policy evaluation. Phase 12 may consume stress profiles for embodiment demonstrations. Phase 14 displays stress behavior.

Validation gate and exit condition. Stress results must be record-driven, reproducible, matched across policies, and split-safe. The same stress profile must be applied to comparable policies without changing evaluation splits improperly. The exit condition is a minimal but defensible stress suite with documented clean and degraded comparisons.

Risks, fallback, and limitations. Risks include unfair stress comparisons, unrealistic stress definitions, stress leakage into training, and overclaiming robustness. The fallback is a small stress suite with delay, dropout, and noise before adding fatigue-like degradation, weakness, drift, or combined stress. StressForge-Lite is a controlled robustness probe, not a real clinical stress emulator.

Thesis evidence role. Phase 9 supports robustness curves, stress cards, failure taxonomy, stress-conditioned ablations, and the thesis claim that readiness architectures should be tested under degradation rather than only clean benchmark settings.

23.7.11 Phase 10: Contextual bandit

Purpose and bottleneck controlled. Phase 10 is the lightweight online-adaptation simulation gate. It evaluates immediate reward adaptation inside the declared simulated environment before full RL. It controls the risk of introducing high-variance sequential learning before simpler adaptive policies are tested.

Preconditions and dependencies. Phase 8 must provide stable `TransitionRecord` and environment definitions. Phase 9 stress profiles should be available for robustness evaluation if bandit claims include stressed conditions. Action legality masks, reward definitions, and policy configs must exist.

Primary inputs. `ReadinessContextRecord`; `TransitionRecord`; `StressProfileRecord`; action-legality masks; reward definitions; bandit policy configs; held-out subject/session definitions; baseline policy outputs.

Core implementation tasks.

- Implement contextual bandit policies for threshold selection or small action selection before attempting large action spaces.
- Define context fields, action candidates, exploration policy, reward function, regret baseline, and safety overrides.
- Log action probabilities, selected actions, rewards, regret estimates, safety overrides, and illegal-action masks.
- Emit one `PolicyUpdateTrace` for every simulation-authorized bandit update, including policy ID, context ID, action, reward, next state, update step, exploration mode, update mode, frozen-policy flag, simulator ID, seed, config hash, and `simulation_only=true`.
- Export training/adaptation curves separately from frozen-policy evaluation curves and write disabled-update audit entries when a bandit artifact is evaluated without mutation.
- Evaluate on held-out subjects, sessions, and stress profiles where available.
- Compare against non-bandit policies including IHARQ-lite, supervised Adaptive-IHARQ, learning-to-defer, static thresholds, and accept-all baselines.

Outputs and consumers. `Bandit PolicyActionRecord`; reward curves; regret curves; exploration logs; safety override logs; bandit controller card; comparison tables. Phase 11 can use bandit as a simpler online-adaptation baseline. Phase 14 consumes bandit metrics and cards.

When update mode is enabled, Phase 10 also produces optional `PolicyUpdateTrace` files under `runs/<run_id>/results/layer7_closed_loop/policy_updates/`, disabled-update audits for frozen evaluation, policy-update trace cards, and separated training/adaptation versus frozen-evaluation comparison tables for Layer 10.

Validation gate and exit condition. Bandit curves and comparisons must exist under declared clean and stressed conditions, with simulation-only claim tags and fair non-bandit baselines. The exit condition is a contextual adaptation result that improves or fails transparently relative to simpler policies.

The gate also requires complete `PolicyUpdateTrace` coverage for update-enabled runs and explicit no-mutation evidence for frozen evaluation. A bandit policy that improves only while still adapting must be labeled as online-adaptation behavior rather than frozen-policy superiority.

Risks, fallback, and limitations. Risks include large action spaces, exploration of unsafe actions, reward hacking, weak baselines, and simulator overfitting. The fallback is bandit-based threshold selection only. Bandit results are simulation-only and do not imply deployed online adaptation.

Thesis evidence role. Phase 10 supports reward/regret plots, adaptive threshold/action analysis, controller cards, and the thesis stretch claim that lightweight online adaptation can be studied under declared simulated consequences.

23.7.12 Phase 11: Reinforcement-learning policy

Purpose and bottleneck controlled. Phase 11 is the sequential policy-learning stretch gate. It is only valid after the simulator is stable and simpler policies exist. It tests whether sequential policy learning adds value under delayed consequences, stress profiles, and declared reward weights.

Preconditions and dependencies. Stable Phase 8 environment API and `TransitionRecord` outputs are mandatory. Phase 10 bandit results are strongly recommended as a simpler adaptive baseline. Phase 9 stress profiles are required for stress claims. Action legality masks, reward/cost config, held-out simulated sessions, and controller-card templates must exist.

Primary inputs. Stable transition records; simulated environment API; reward/cost config; held-out simulated sessions; action legality masks; stress profiles; simple baseline policies; RL algorithm configs.

Core implementation tasks.

- Implement simple RL algorithms first and keep the action space small.
- Declare reward weights for accuracy, accepted-trial correctness, unsafe events, request evidence, waiting, fallback, delay, and embodiment-relevant penalties where used.
- Enforce legality masks and block exploration of unsafe or undefined actions.
- Run reward ablations, seed sensitivity checks, policy stability analysis, and held-out simulated evaluation.
- Emit `PolicyUpdateTrace` for every simulation-authorized RL policy update and preserve `TransitionRecord.policy_update_trace_id` linkage where applicable.
- Freeze trained/adapted policy artifacts before held-out evaluation and export training, adaptation, and frozen-evaluation curves as separate evidence families.
- Compare against IHARQ-lite, supervised Adaptive-IHARQ, learning-to-defer, contextual bandit, static thresholds, and accept-all policies.
- Produce controller cards with reward design, action space, safety constraints, training instability, and simulation-only limitations.

Outputs and consumers. RL `PolicyActionRecord`; trained policy artifacts; reward learning curves; held-out evaluation tables; stress evaluation tables; controller cards; reward sensitivity analysis; limitation statements. Phase 14 consumes RL outputs for dashboard views. Phase 15 uses RL only as stretch evidence if validation gates pass.

When RL updates are enabled, Phase 11 additionally produces optional `PolicyUpdateTrace`, disabled-update audit entries for frozen evaluation, training/adaptation curves, frozen-evaluation tables, and policy-update trace-card inputs for Layer 10.

Validation gate and exit condition. RL must be evaluated against non-RL policies under held-out and stressed simulated conditions, and all results must be labeled simulation-only. The exit condition is not necessarily RL superiority; it is a transparent evaluation showing whether RL adds value or fails under declared constraints.

The RL gate fails if any update-enabled run lacks `PolicyUpdateTrace`, if a held-out frozen evaluation mutates the policy, if disabled-update auditing is missing when configured, or if dashboard exports mix training/adaptation curves with frozen-policy evaluation curves.

Risks, fallback, and limitations. Risks include simulator overfitting, reward hacking, unstable training, opaque policy behavior, and exaggerated claims. The fallback is to present RL as optional Stretch C or future work if bandit, supervised Adaptive-IHARQ, and learning-to-defer provide sufficient thesis evidence.

Thesis evidence role. Phase 11 supports the advanced policy-learning stretch section, reward-sensitivity figures, controller cards, and a bounded discussion of sequential policy learning under simulated readiness consequences.

23.7.13 Phase 12: MyoSuite embodiment demo

Purpose and bottleneck controlled. Phase 12 is the simulated embodiment proof-of-concept gate. It maps readiness decisions into simulated action consequences through declared proxy commands. It prevents the architecture from ending at abstract policy metrics while also preventing embodiment visuals from being mistaken for clinical validation.

Preconditions and dependencies. Phase 8 transition records and selected policy actions must exist. Phase 0 schemas for `EmbodimentStateRecord`, `EmbodimentOutcomeRecord`, `SafetyEventRecord`, `Layer9RewardTrace` where enabled, and `Layer9EmbodimentDemoManifest` must validate. Embodiment configs, command mapping rules, readiness-context assembly, record-intake validation, safety gates, simulator adapter contracts, and Layer 10 dashboard-compatible export expectations must be declared. MyoSuite installation is a software dependency for E2+; E0 no embodiment and E1 static replay remain the fallback if physics simulation is infeasible.

Primary inputs. `PolicyActionRecord`; `IHARQDecisionRecord`; `ReadinessContextRecord`; `TransitionRecord`; `StressProfileRecord` where used; proxy intent mapping; embodiment configs; readiness safety gate; command mapping manifest; simulator branch configuration; reward-trace configuration where enabled; and Layer 10 export requirements.

Core implementation tasks.

- Implement the Layer 9 session controller, record-intake validator, readiness-context assembler, proxy command mapper, readiness safety gate, simulator adapter interface, static replay adapter, MyoSuite adapter, optional OpenSim replay adapter, stress injector, outcome/reward bridge, transition/safety logger, and dashboard/card/export packager as L9.1–L9.13 responsibilities rather than as a detached Layer 20/P20 branch.
- Implement a simulator adapter that separates readiness actions from low-level simulated motor commands.
- Define a command mapping validator from proxy neural-intent decisions to allowable simulated action commands.
- Enforce a readiness safety gate so uncertain, unsafe, fallback, or no-action states cannot issue aggressive commands.
- Begin with E1 static replay before E2 stepped MyoSuite interaction.
- Implement a simple MyoSuite proxy task and log endpoint, movement, command, success, delay, safety events, command timeline, endpoint-error curve, and reward terms where enabled.
- Export embodiment cards, `Layer9EmbodimentDemoManifest`, dashboard-compatible export bundle, dashboard screenshots or figures, and non-clinical claim-safety warnings.

Outputs and consumers. `EmbodimentStateRecord`; embodiment-linked `TransitionRecord`; `SafetyEventRecord` or safety-event logs; `Layer9RewardTrace` where enabled; command timeline; endpoint-error curve; command mapping manifest; simulator-version meta-data; `Layer9EmbodimentDemoManifest`; dashboard-compatible Layer 9 export bundle; endpoint/success metrics; embodiment cards; and simulation-only warnings. Phase 13 may consume selected trajectories for OpenSim replay. Phase 14 visualizes embodiment outcomes as read-only Layer 10 evidence. Phase 15 uses embodiment only as bounded proof-of-concept evidence.

Validation gate and exit condition. Readiness decisions must produce logged simulated outcomes through declared proxy mappings and safety rules before any simulator step is treated as valid evidence. The gate fails if record intake is invalid, readiness context cannot be assembled, command mapping is unsupported, the readiness safety gate is bypassed, safety-event logs are missing, the endpoint-error curve or command timeline is absent when required, reward traces are missing when reward-enabled embodiment is claimed, or the `Layer9EmbodimentDemoManifest` cannot be traced by Layer 10. The exit condition is a readiness-gated simulated action demo with complete Layer 9 logs, manifests, dashboard-compatible export, and non-clinical warnings.

Risks, fallback, and limitations. Risks include overclaiming real motor control, brittle simulator installation, invalid command mapping, and confusing visual demos with clinical validation. The fallback is E0 no embodiment or E1 static replay if MyoSuite is infeasible. The thesis remains valid without embodiment physics if C0–C3 evidence is complete.

Thesis evidence role. Phase 12 supports simulated embodiment cards, proxy action outcome figures, safety-event logs, reward-trace summaries where enabled, command timelines, endpoint-error curves, `Layer9EmbodimentDemoManifest` files, and the stretch claim that readiness gating can be linked to simulated assistive-action consequences under declared limitations.

23.7.14 Phase 13: OpenSim replay or optional comparison

Purpose and bottleneck controlled. Phase 13 is an optional biomechanical visualization gate. It may provide additional plausibility visualization or comparison, but it must never block thesis completion. It prevents OpenSim from becoming an uncontrolled dependency or a source of overclaimed biomechanical validation.

Preconditions and dependencies. Phase 12 selected command trajectories or movement profiles should exist. OpenSim installation, model configuration, mapping limitations, and output-manifest templates are optional software dependencies. The phase is non-blocking by design.

Primary inputs. Selected command trajectories; embodiment logs; movement profile configs; OpenSim configuration; mapping limitation text; optional comparison metrics where valid.

Core implementation tasks.

- Replay selected command trajectories or simplified movement profiles.
- Document OpenSim model configuration, mapping assumptions, units, and limitations.
- Export screenshots, plots, replay logs, and optional comparison reports.
- Compare qualitatively or quantitatively only where the mapping is valid and explicitly bounded.

- Add non-clinical claim boundaries to every OpenSim card, figure, and thesis paragraph.

Outputs and consumers. OpenSim output manifest; screenshots or plots; replay logs; optional comparison report; mapping limitation card; and optional linkage to the `Layer9EmbodimentDemoManifest`. Phase 14 may display optional outputs as Layer 10 read-only evidence. Phase 15 may place them in a demo or future-work appendix.

Validation gate and exit condition. OpenSim outputs must be documented as optional plausibility visualization, not validation. The exit condition is either a documented optional replay artifact or an explicit decision to skip OpenSim without affecting thesis validity.

Risks, fallback, and limitations. Risks include installation burden, overclaiming biomechanics, poor mapping from EEG intent to movement, and blocking core thesis completion. The fallback is to skip OpenSim entirely or keep it as future work while preserving MyoSuite/static replay and dashboard outputs.

Thesis evidence role. Phase 13 supports optional visualization, appendix material, and future-work discussion. It does not support clinical or biomechanical efficacy claims.

23.7.15 Phase 14: Dashboard and cards

Purpose and bottleneck controlled. Phase 14 is the observability and reproducibility gate. It turns validated records into inspectable thesis artifacts. It prevents the dashboard from becoming cosmetic or from creating hidden results outside the record system.

Preconditions and dependencies. Valid records from completed phases must exist. `DashboardMetricRecord` and `DashboardViewRecord` schemas, export templates, card templates, figure scripts, metric-dictionary exports, matched-comparison reports, negative-result note indexes, `ClaimEvidenceMatrix` views, and source-record lineage rules must be defined. The dashboard may be interactive, static, or hybrid, but it must read saved records only.

Primary inputs. All validated records; run manifests; config hashes; metric dictionaries; matched-comparison reports; negative-result notes; `ClaimEvidenceMatrix` entries; figures; cards; limitation statements; claim-safety warnings; export templates; dashboard page configuration; and Layer 9 embodiment bundles including `EmbodimentOutcomeRecord`, `Layer9RewardTrace` where enabled, and `Layer9EmbodimentDemoManifest`.

Core implementation tasks.

- Build dashboard pages for dataset/protocol, decoder baselines, calibration/uncertainty, IHARQ, RegimeRisk, Adaptive-IHARQ, deferral, simulated closed-loop policies, StressForge-Lite, contextual bandit, RL, Layer 9 embodiment evidence, ablation, failure taxonomy, cards, and exports.
- Generate `DashboardMetricRecord` outputs that summarize dashboard metrics without recomputing hidden results and preserve ablation, metric-dictionary, matched-comparison, diagnostic-only, negative-result, claim-evidence, warning, and export lineage.
- Generate `DashboardViewRecord` outputs for page/view metadata, source metric records, filters, warning banners, completeness state, diagnostic-only state, export references, and thesis usage locations.
- Generate metric dictionary exports, matched-comparison reports, A0–A4 evaluation-slice lineage, `ClaimEvidenceMatrix` views, and negative-result note indexes for Layer 1–3 evaluation artifacts.

- Generate static thesis figures, CSV or Parquet tables, PDF/HTML/Markdown cards, screenshots, and export manifests.
- Produce dataset cards, protocol cards, decoder cards, reliability cards, regime cards, controller cards, stress cards, embodiment cards, limitation cards, and artifact cards.
- Link every figure, table, card, and screenshot to source records, configs, seeds, schema versions, code versions, and output hashes.
- Render optional policy-update trace cards, disabled-update audit tables, and policy-update validation logs when `PolicyUpdateTrace` or update-audit files exist.
- Use `DashboardMetricRecord.policy_update_filter` to keep simulation-training/adaptation curves separate from frozen-policy evaluation curves in every dashboard page, export manifest, thesis figure, and table.

Outputs and consumers. Dashboard app; `DashboardMetricRecord`; `DashboardViewRecord`; metric dictionary exports; matched-comparison reports; `ClaimEvidenceMatrix` views; negative-result note index; diagnostic-only status exports; static figures; thesis tables; cards; screenshots; CSV/Parquet exports; export manifest; artifact manifest. Phase 15 consumes all dashboard-derived thesis artifacts. Reviewers and thesis examiners consume the dashboard as an observability and reproducibility interface.

Validation gate and exit condition. The dashboard must inspect results from saved records only. Hidden recomputation is forbidden unless it creates a new run identifier, config hash, and output record. The exit condition is a traceable dashboard/export package in which every figure, table, and card links to source artifacts.

For policy-learning views, the dashboard gate additionally requires trace-card completeness, disabled-update audit visibility, `DashboardMetricRecord.policy_update_filter`, and a visible separation between simulation-training/adaptation and frozen-policy evaluation exports. For the Layers 0–3 evaluation slice, the dashboard gate additionally requires Layer 1 ablation-readiness visibility, metric-dictionary references, matched-comparison reports, Layer 3 matched operating-point exports, negative-result notes, and `ClaimEvidenceMatrix` linkage.

Risks, fallback, and limitations. Risks include cosmetic dashboards, inconsistent figures, missing provenance, untracked recomputation, and dashboard exports that disagree with thesis tables. The fallback is to generate static cards and figures from records before building a full interactive dashboard.

Thesis evidence role. Phase 14 supports the thesis figures, tables, cards appendix, reproducibility appendix, defense demo, and the claim that the architecture is inspectable and auditable.

23.7.16 Phase 15: Final thesis integration

Purpose and bottleneck controlled. Phase 15 is the manuscript traceability gate. It converts implementation artifacts into defended thesis claims and prevents a gap between architecture, experiments, results, limitations, and final writing.

Preconditions and dependencies. The highest completed capability tier must have validated records, metrics, figures, tables, cards, ablations, limitations, and manifests. Phase 14 exports should exist for all completed phases. Incomplete phases must be classified as optional, partial, stretch, or future work.

Primary inputs. Architecture sections; method descriptions; experiment configs; records; result manifests; ablation tables; metric dictionaries; matched-comparison reports; **ClaimEvidenceMatrix**; negative-result notes; stress results; cards; figures; limitations; failure taxonomy; repository manifest; reproducibility guide.

Core implementation tasks.

- Map every major thesis claim to a record, metric-dictionary entry, matched-comparison report, figure, table, card, ablation, repository artifact, negative-result note, or limitation statement.
- Write the methods, experimental design, evaluation, results, discussion, threat-to-validity, and limitations chapters using saved artifacts.
- Generate a claim-to-artifact traceability matrix and reproduction guide.
- Mark public-data, proxy-label, simulation-only, embodiment-only, optional, and future-work claims explicitly.
- Ensure that negative results, failed phases, partial phases, and fallback decisions are documented rather than hidden.

Outputs and consumers. Thesis-ready architecture, methods, evaluation, results, and discussion sections; claim-to-artifact traceability matrix; **ClaimEvidenceMatrix**; negative-result register; artifact appendix; reproduction guide; limitation statements; final claim map. The final thesis, defense slides, repository release, and appendix consume these outputs.

Validation gate and exit condition. Every major claim must be supported by an artifact or explicitly bounded by a limitation statement. The exit condition is a manuscript and artifact package where the highest completed tier is clear, evidence is traceable, and unsupported claims are removed.

Risks, fallback, and limitations. Risks include unsubstantiated claims, missing evidence, scope creep, weak connection between architecture and experiments, and overemphasis on incomplete stretch phases. The fallback is to write the thesis around the highest completed capability tier and classify unfinished phases honestly.

Thesis evidence role. Phase 15 is the final integration of all evidence into the thesis. It establishes that the architecture, implementation, experiments, dashboard, and limitations form one coherent defended contribution.

23.8 Phase dependency and record-lineage matrix

Phase	Hard upstream records	Primary downstream records	Mockable during development
0	Full architecture, record list, repository plan	Mock records, schemas, config templates, validation outputs	Yes, fully mockable

1	Phase 0 schemas and dataset configs	DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, DatasetCard, ProtocolCard, ValidationReport, Layer1Manifest, and manifests/layer1_ablation_readiness_l1_v1.json	Partially, with synthetic windows
2	WindowRecord, SplitRecord, label maps	PredictionRecord, decoder cards	Partially, with mock predictions
3	PredictionRecord, split manifests	CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord	Yes, with mock probabilities
4	Prediction, calibration, uncertainty, threshold records, artifact flags, and optional regime context	IHARQDecisionRecord, EvidenceQualityRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, and Layer 4 EvaluationOutcomeRecord	Yes, with mock feature records
5	Prediction, uncertainty, IHARQ, metadata records	RegimeStateRecord	Yes, with synthetic timelines
6	Prediction, uncertainty, IHARQ, RegimeRisk records	ReadinessContextRecord, EvidenceQualityRecord, PolicyActionRecord	Partially, but leakage checks require real split metadata
7	Readiness contexts, evidence-quality records, policy actions	DeferralRecord	Yes, with mock fallback sources
8	Policy actions, deferral records, RegimeRisk context	TransitionRecord	Yes, with toy environment
9	Clean records, transition records, stress configs	StressProfileRecord, stress-run manifests	Yes, with simple stress profiles
10	Readiness contexts, transitions, stress profiles	Bandit PolicyActionRecord, regret logs	Partially, with toy rewards
11	Stable transition records, environment API, reward config	RL policy records, controller cards	Partially, but thesis claims require held-out simulation
12	Policy actions, transitions, mapping configs	EmbodimentStateRecord, EmbodimentOutcomeRecord, safety-event logs	Yes, with static replay

13	Embodiment logs or command trajectories	OpenSim manifests, replay plots	Optional and fully skippable
14	All completed validated records, metric dictionaries, matched-comparison reports, negative-result notes, and claim-evidence entries	DashboardMetricRecord, DashboardViewRecord, cards, figures, export manifests, and diagnostic-only status exports	Yes, with mock records
15	Records, cards, figures, limitations, manifests	Claim-to-artifact matrix, reproduction guide	No, final claims require real artifacts

23.9 Dependency classes and mockability rules

Dependencies in this pipeline are classified as follows.

- **Hard dependencies** are artifacts without which a phase cannot make a valid thesis claim. Examples include schemas for every phase, fixed splits before decoder training, `PredictionRecord` before calibration, `CalibrationRecord` and `UncertaintyRecord` before IHARQ, and stable `TransitionRecord` before bandit or RL.
- **Soft dependencies** strengthen a phase but do not block a minimal valid version. Examples include multiple datasets, compact SSL baselines, channel-subset stability, full interactive dashboard views, and multi-stress combined profiles.
- **Optional dependencies** are explicitly non-blocking. OpenSim replay, advanced RL, and full physics embodiment can be skipped or classified as future work.
- **Conceptual dependencies** are design commitments that must be respected even when software is incomplete: claim-safety boundaries, proxy-label honesty, split safety, and dashboard record provenance.
- **Software dependencies** must be versioned and isolated: MOABB or dataset loaders, decoder libraries, calibration utilities, dashboard framework, MyoSuite/MuJoCo, optional OpenSim, and plotting/reporting tools.

Mock records may be used to develop validators, dashboard pages, adapter interfaces, and toy policies. They may not be used as scientific evidence. A mocked phase can reach PIRL-3, but not PIRL-6 or above.

23.10 Feedforward and loopback behavior

The normal feedforward direction is Phase 0 $\rightarrow$ Phase 1 $\rightarrow$ Phase 2 $\rightarrow$ Phase 3 $\rightarrow$ Phase 4 $\rightarrow$ Phase 5 $\rightarrow$ Phase 6 $\rightarrow$ Phase 7 $\rightarrow$ Phase 8 $\rightarrow$ Phase 9 $\rightarrow$ Phase 10 $\rightarrow$ Phase 11 $\rightarrow$ Phase 12 $\rightarrow$ Phase 13 $\rightarrow$ Phase 14 $\rightarrow$ Phase 15. However, controlled loopbacks are allowed when they have a scientific and engineering reason.

Loopback source	Target	Signal and trigger	Allowed action	Safety constraint
-----------------	--------	--------------------	----------------	-------------------

Phase 3 calibration audit	Phase 2 decoder setup	High-confidence wrong clusters, missing score fields, calibration failure	Mark decoder ineligible for calibrated policy use or rerun decoder export with required score fields	Do not tune decoder on test outcomes
Phase 4 IHARQ comparison	Phase 3 threshold registry	Excessive abstention or request burden under validation split	Adjust threshold-selection rule on allowed split and record new threshold version	No test-set threshold tuning
Phase 5 RegimeRisk warning	Phase 4 IHARQ rules	Sustained instability or temporal trust drawdown	Tighten IHARQ action rules, request evidence, fallback, or unsafe state on validation protocol	No future correctness feature in decision- time context
Phase 6 pseudo-label sensitivity	Phase 6 cost model	Action imbalance or unstable policy behavior	Revise cost weights and regenerate pseudo-labels with new version	Keep target construc- tion versioned and split-safe
Phase 8 simulation diagnostics	Phase 6 or Phase 7 policies	Undefined action, impossible transition, or unrealistic cost accumulation	Update legality masks, fallback rules, or environment constraints	Document change and rerun affected comparisons
Phase 9 stress failure	Phase 4–8 modules	Systematic failure under declared stress	Add failure-taxonomy entry, run ablation, or evaluate safer fallback policy	Do not train or tune on final stress-test outcomes
Phase 12 Layer 9 embodiment logs	Phase 8 simulator or Phase 6/7/10/11 policy masks	Unsafe command attempt, invalid command mapping, blocked action burst, missing safety-event log, missing Layer9EmbodimentDemoManifest, or inconsistent reward trace	Tighten command validator, safety gate, policy legality mask, logging contract, or reward-trace configuration	Do not claim clinical safety; maintain proxy mapping limits and preserve Layer 10 read-only dashboard consump- tion

Phase 14 dashboard lineage check	Any completed phase	Missing source record, config hash, or export manifest	Regenerate artifact from records or mark result non-thesis-ready	Dashboard cannot silently recompute hidden results
Phase 15 claim audit	Any completed phase	Claim lacks evidence or limitation	Add artifact reference, rerun evaluation, or remove/limit claim	Unsupported claims must not enter final thesis

Each loopback must produce a new run identifier or updated manifest when it changes artifacts. Loopbacks are not permission for hidden tuning. They are controlled correction pathways with explicit triggers, actions, and safety constraints.

23.11 Validation, leakage, and claim-safety gates

The implementation pipeline uses cross-cutting gates that apply across phases.

Gate	What it checks	Failure interpretation
Schema gate	Required fields, record versions, allowed value ranges, identifier consistency, output directories	The phase is not record-compatible and cannot feed downstream modules.
Split-leakage gate	Disjoint train/validation/test/calibration/adaptation splits; threshold source split; subject/session leakage	Evaluation is invalid or adaptation until split provenance is fixed.
Threshold gate	Threshold source, selection metric, allowed split, version, matched-coverage or matched-cost operating point, and test-set invisibility	IHARQ or selective-prediction results cannot be interpreted as fair.
Evaluation-artifact gate	<code>layer1_ablation_readiness_11</code> matched-comparison keys, metric dictionary entries, A0–A4 lineage, <code>Layer3MatchedOperatingPointTable</code> , negative-result notes, and <code>ClaimEvidenceMatrix</code> linkage	Evaluation artifacts are diagnostic-only or non-thesis-ready until their source-record, metric, table, comparison, and claim-evidence lineage is repaired.
Policy-input gate	Separation of decision-time features from evaluation-only targets, future correctness, or reward labels	Adaptive-IHARQ, deferral, bandit, or RL results are leakage-contaminated.
Action-legality gate	Allowed action set, masks, fallback rules, unsafe-state behavior, embodiment command blocking	Policy outputs are unsafe or undefined in simulation and embodiment replay.

Stress gate	Stress family, intensity, schedule, seed, target record, injection point, guardrails, matched-stress comparison report, stress metric dictionary, negative-result note where applicable, dashboard/export lineage, claim-evidence entry, and claim boundary	Stress comparisons are not reproducible, not fair across policies, or not thesis-ready for A12.
Simulation gate	State/action/reward/transition definitions, held-out sessions, reward version, episode boundaries	Bandit or RL claims are premature or simulation-overfit.
Policy-update trace gate	Declared update mode, PolicyUpdateTrace completeness for simulation-authorized mutations, disabled-update audit for frozen evaluation, and TransitionRecord.policy_update_trace_id linkage where present	Bandit/RL adaptation or frozen-policy evaluation claims are invalid or must be relabeled diagnostic until traceability is repaired.
Embodiment gate	Proxy mapping, readiness safety gate, adapter logs, command validation, safety-event log, command timeline, endpoint-error curve, Layer9RewardTrace where enabled, Layer9EmbodimentDemoManifest , matched embodiment comparison report, embodiment metric dictionary, negative-result note where applicable, dashboard/export lineage, claim-evidence entry, dashboard-compatible export, and non-clinical warning	Embodiment outputs are only visual demos and cannot support readiness-consequence or A13 claims unless the Layer 9 artifact bundle and evaluation-lineage bundle are complete and Layer 10 can trace them.
Dashboard lineage gate	Source records, config hashes, figure/table/card manifests, export timestamps, code versions, optional PolicyUpdateTrace references, DashboardMetricRecord.policy_update_filter , A12/A13 matched-comparison IDs, metric-dictionary IDs, negative-result-note pointers, claim-evidence pointers, diagnostic-only labels, and frozen/update separation status	Dashboard exports are not thesis evidence.

Claim-safety gate	Public-data, proxy-label, simulation-only, non-clinical, and optional-phase tags	The claim is overstated and must be rewritten or removed.
-------------------	--	---

23.12 Implementation-readiness levels

A phase is not considered complete because a script exists. The following pipeline implementation-readiness levels define maturity.

Level	Name	Meaning for the Implementation Pipeline section phases
PIRL-0	Named	The phase is named but not yet specified.
PIRL-1	Bottleneck defined	The phase explains which implementation or thesis bottleneck it controls.
PIRL-2	Contracts defined	Inputs, outputs, records, configs, and schemas are specified.
PIRL-3	Mock validated	Mock records pass validators and can be consumed by downstream placeholder code.
PIRL-4	Implemented on sample	The phase runs on a small sample or single dataset slice.
PIRL-5	Integrated	The phase reads real upstream records and produces real downstream records.
PIRL-6	Evaluated	The phase produces metrics, comparisons, cards, or dashboard views relevant to thesis evidence.
PIRL-7	Stress or ablation checked	Where relevant, the phase has ablation, stress evidence, sensitivity analysis, or failure analysis.
PIRL-8	Reproducible	The phase reruns from configs with seeds, hashes, manifests, and documented outputs.
PIRL-9	Thesis-ready	The phase maps to figures, tables, claims, cards, limitations, and repository artifacts.

Phases 0–3 should reach at least PIRL-8 before later learned policy claims are credible. Phases 4–7 should reach at least PIRL-7 if they are used as thesis contributions. Phases 10–13 may remain Stretch C or optional if the thesis already has sufficient evidence from C0, C1, C2, or C3.

23.13 Risk, fallback, and criticism-control matrix

Risk or criticism	Phases affected	Required defensive design
The project is only an EEG classifier benchmark	0–15	Show that decoders feed calibration, IHARQ, RegimeRisk, policies, simulation, stress, embodiment, dashboard, and cards through explicit records.
The pipeline is too complex to implement	0–15	Use capability tiers, stage gates, fallback stopping points, and independent record outputs.

Schema drift or hidden variables	0 onward	Require schema registry, mock records, validators, config hashes, run IDs, and record versioning.
Evaluation artifacts not traceable	0–3, 14–15	Require Layer 1 ablation-readiness, matched-comparison reports, metric dictionary entries, Layer 3 matched operating points, negative-result notes, and ClaimEvidenceMatrix linkage before A0–A4 claims are thesis-ready.
Split leakage	1–7	Require split manifests, leakage checks, threshold registry leakage status, and separation of policy inputs from evaluation-only targets.
IHARQ is just thresholding	4	Require matched baselines: confidence, entropy, margin, selective prediction, longer windows, voting, averaging, and ensembles.
RegimeRisk is overclaimed	5	Call it temporal trust characterization unless real regime labels exist; block future-correctness leakage.
Adaptive-IHARQ labels are artificial	6–7	Describe labels as proxy, cost-based, computational targets; save target-construction cards and sensitivity analyses.
RL overfits the simulator	8–11	Require held-out simulated sessions, stress profiles, simple baselines, reward ablations, controller cards, and simulation-only tags.
MyoSuite/OpenSim is clinical validation	12–13	Require proxy mapping, safety gate, command validator, non-clinical warnings, and optional OpenSim status.
Dashboard is cosmetic	14	Require dashboard to read saved records and export traceable figures, tables, cards, and manifests.
Final thesis claims are not traceable	15	Require claim-to-record/table/figure/card/limitation mapping.

23.14 Layers 4–7 evaluation-lineage pipeline addendum

The MERGE_RECORD_22 Layers 4–7 propagation strengthens the implementation pipeline by turning evaluation-lineage completeness into an implementation gate for Phases 4–8 and for later policy, stress, and embodiment participation. This does not alter the phase order and does not require a new pipeline phase. It requires Phases 4–8 to produce evaluation artifacts that can be consumed by Layer 10, the repository manifest, the failure-mode register, and the final demonstration matrix.

Phase	Added evaluation-lineage gate	Required artifacts	Failure interpretation
-------	-------------------------------	--------------------	------------------------

Phase 4	A5 IHARQ-lite and A6/A7 participation lineage	Layer 4 EvaluationOutcomeRecord IHARQ matched-comparison report, metric dictionary entry, negative-result note, claim-evidence entry	Missing match or metric lineage downgrades IHARQ evidence to diagnostic-only.
Phase 5	A7 RegimeRisk lineage	Layer 5 EvaluationOutcomeRecord regime-aware/unaware comparison, shuffled/random risk negative control, metric dictionary, negative-result note	Missing negative control or non-causal feature blocks RegimeRisk contribution claims.
Phase 6–7	A6/A8/A9 policy-readiness lineage	Layer 6 EvaluationOutcomeRecord cost/coverage/reward matched comparisons, controller-card lineage, metric dictionary, negative-result note	Reward-sensitive or unmatched policy evidence is diagnostic-only.
Phase 8	A10/A11 closed-loop simulation lineage	Layer 7 EvaluationOutcomeRecord frozen-policy held-out report, update-mode separation evidence, metric-dictionary snapshot, diagnostic-only label where needed	Mixed training/adaptation and frozen evaluation invalidates policy-comparison claims.
Phases 9–13	A12/A13 participation lineage where Layers 4–7 evidence feeds stress or embodiment branches	Stress/embodiment participation summaries tied to Layer 4–7 source records and claim boundaries	Unmatched stress or embodiment participation remains optional or diagnostic-only.
Phase 14–15	Dashboard and final synthesis of the above	Matched-comparison views, claim-evidence matrix, negative-result register, reproducibility manifest, and final demonstration matrix	Missing export lineage blocks thesis-ready final claims.

The pipeline evaluation-artifact gate therefore covers A0–A13: Layer 1 ablation-readiness, matched-comparison keys, metric dictionary entries or snapshots, Layer 3 matched operating points, Layer 4–7 EvaluationOutcomeRecord files, negative-result notes, diagnostic-only flags, dashboard/card/export lineage, and ClaimEvidenceMatrix linkage. Evaluation artifacts are diagnostic-only or non-thesis-ready until their source-record, metric, comparison, dashboard/export, and claim-evidence lineage is repaired.

23.15 Layers 8–10 evaluation-lineage pipeline addendum

The MERGE_RECORD_22 Layers 8–10 propagation strengthens the implementation pipeline by turning A12/A13 evaluation-lineage completeness into an implementation gate for Phase 9, Phase 12, Phase 14, and Phase 15. This does not alter the phase order and does not require a new pipeline phase. It requires StressForge and embodiment evaluation artifacts to be produced by their nearest owning layers before Layer 10 renders them or the final thesis demonstration cites them.

Phase	Added evaluation-lineage gate	Required artifacts	Failure interpretation
Phase 9	A12 StressForge robustness lineage	records/evaluation_outcomes/layer8_stress_evaluation_outcomes/layer8_matched_stress_comparisons/layer8_matched_stress_metrics/metric_dictionary_layer8_stress.yaml, notes/negative_results/layer8_stress_negative_results.m where applicable, and to manifests/claim_evidence/layer8_stress_claim_evidence.j	Missing stress evaluation artifacts or lineage blocks
Phase 12	A13 embodiment-proxy lineage	records/evaluation_outcomes/layer9_embodiment_evaluation_outcomes/layer9_matched_embodiment_comparisons/layer9_embodiment_metrics/metric_dictionary_layer9_embodiment.yaml, notes/negative_results/layer9_embodiment_negative_results.m where applicable, and boundary manifests/claim_evidence/layer9_embodiment_claim_evidence.j	Missing embodiment evaluation artifacts or lineage blocks
Phase 14	Dashboard and reproducibility rendering of A12/A13	Read-only StressForge, embodiment, ablation/comparison, card/export, warning, and reproducibility views over the required Phase 9 and Phase 12 artifacts	Missing dashboard/export lineage blocks thesis-ready A12/A13 exports.
Phase 15	Final claim synthesis of A12/A13	Final demonstration rows with source records, matched-comparison IDs, metric-dictionary IDs, negative-result pointers, dashboard/export lineage, claim-evidence entries, and diagnostic-only flags	Missing claim-evidence or negative-result interpretation blocks final robustness or embodiment-consequence claims.

The pipeline evaluation-artifact gate therefore now covers A0–A13 with explicit local ownership: Layer 1 ablation-readiness, Layer 3 matched operating points, Layer 4–7 evaluation-outcome files, Layer 8 A12 stress evaluation files, Layer 9 A13 embodiment evaluation files, matched-comparison reports, metric dictionary entries or snapshots, negative-result notes, diagnostic-only flags, dashboard/card/export lineage, and ClaimEvidenceMatrix linkage. Evaluation ar-

tifacts are diagnostic-only or non-thesis-ready until their source-record, metric, comparison, dashboard/export, negative-result, and claim-evidence lineage is repaired.

23.16 Final pipeline definition of done

the Implementation Pipeline section is complete only when the following conditions hold.

1. The pipeline is explicitly described as the implementation governance layer for Stretch C.
2. The Phase 0–15 sequence is preserved and every phase has a purpose, preconditions, inputs, tasks, outputs, validation gate, exit condition, risks, fallback, and thesis-evidence role.
3. Every phase names its primary records and downstream consumers.
4. Capability-tier stopping points C0–C4 are defined and thesis-valid.
5. Schema, split, leakage, threshold, evaluation-artifact, policy, policy-update trace, stress, A12 StressForge evaluation-lineage, embodiment, A13 embodiment evaluation-lineage, dashboard, and thesis-traceability gates are defined.
6. Core, supporting, Stretch C, optional, and future-work branches are distinguishable.
7. Advanced policies are not allowed before calibration, IHARQ-lite, RegimeRisk, and simulation prerequisites are satisfied.
8. Pseudo-label, simulation, and embodiment outputs carry correct claim boundaries.
9. Dashboard and cards are generated from saved records and never from hidden recomputation.
10. Optional contextual-bandit/RL policy updates emit `PolicyUpdateTrace`; frozen-policy evaluation contains no mutation; disabled-update audits and separated training/adaptation versus frozen-evaluation exports are present where relevant.
11. Phase outputs connect to ablations, metric dictionary entries, matched-comparison reports, negative-result notes, claim-evidence matrix entries, figures, tables, cards, limitation statements, and final thesis claims.
12. The repository/config/schema guidance is specific enough to implement but not reduced to low-level code documentation.
13. The thesis remains stoppable at lower tiers if full Stretch C cannot be completed.

The final implementation pipeline should therefore make the maximum architecture buildable, auditable, and defensible. Its strongest contribution is not the number of phases; it is that every phase has a record contract, validation gate, fallback path, and thesis-evidence role.

24 Recommended Repository Architecture

This section defines the repository architecture for IHARQ BenchGuard under Stretch C. It is not only a suggested folder layout. It is the implementation contract that turns the functional graph, canonical records, evaluation rules, dashboard requirements, failure-mode defenses, and reproducibility obligations into a maintainable research codebase.

The main bottleneck addressed by this section is **implementation traceability**. Without a repository-level contract, the system could degrade into a single classifier notebook, a set of disconnected scripts, or a dashboard that silently recomputes results without provenance. In that case the thesis could not defend its central claim that low-calibration neural-intent reliability can be studied as a modular evidence, readiness, and simulation architecture. The Recommended Repository Architecture section therefore protects the thesis against criticism that the architecture is too broad to implement, that records are cosmetic, that Stretch C modules are unbounded, or that figures and cards cannot be traced to saved experimental evidence.

The repository is positioned after the implementation pipeline because the Implementation Pipeline section defines *when* each phase is allowed to run, while this section defines *where* the code, configurations, records, manifests, tests, figures, cards, logs, and dashboard exports must live. It is positioned before failure modes because repository organization is one of the major defenses against architectural failure: hidden leakage, unversioned outputs, undocumented simulation, policy overclaiming, and untraceable thesis tables.

If this section were removed, the architecture would still describe layers and modules, but it would not enforce modular separation in the actual implementation. The thesis would lose a practical mechanism for proving that decoders, IHARQ, RegimeRisk, Adaptive-IHARQ, learning-to-defer, contextual bandit, reinforcement-learning policy, StressForge-Lite, MyoSuite/OpenSim embodiment, dashboard, cards, ablations, and reproducibility are implemented as auditable components rather than as a tangled script.

24.1 Purpose and architectural role

The repository architecture has four roles.

1. **Scientific role.** It preserves the thesis evidence chain from public EEG data, through decoder predictions, calibration, uncertainty, IHARQ decisions, RegimeRisk states, readiness policies, simulation transitions, stress profiles, embodiment proxy states, dashboard views, cards, figures, and final claims.
2. **Engineering role.** It defines package boundaries, allowed dependency directions, configuration ownership, schema validation, run manifests, generated-artifact storage, testing obligations, and extension points.
3. **Reproducibility role.** It requires every result to be connected to a run identifier, configuration hash, seed, code version, source records, produced artifacts, warnings, and claim-boundary tags.
4. **Claim-safety role.** It propagates Layer 0 boundaries into implementation artifacts so that public-data benchmarking, simulation-only policy learning, and MyoSuite/OpenSim proxy embodiment are not misrepresented as clinical deployment, patient-safe stimulation, or validated rehabilitation control.

The repository must enforce the following implementation principles, including the already-integrated Layers 0–3 evaluation-slice responsibilities:

- canonical records are the primary interface between layers;
- source code is separated from generated artifacts;

- versioned configuration files control experiments;
- notebooks are exploratory and explanatory, not the primary source of thesis results;
- dashboard views read saved records and manifests rather than recomputing hidden logic;
- every table, figure, card, and dashboard export is traceable to source records;
- every Layers 0–3 evaluation-slice artifact has an ablation-step ID where applicable, matched-comparison lineage, metric-dictionary linkage, negative-result visibility, and Layer 0 claim-evidence boundary;
- every thesis-used Layer 8 A12 and Layer 9 A13 evaluation artifact has a local `EvaluationOutcomeRecord`, matched-comparison report, metric dictionary, negative-result note where applicable, dashboard/export lineage, and Layer 0 claim-evidence entry;
- Layer 10 dashboard persistence includes both `records/dashboard_metric_records/` for `DashboardMetricRecord` and `records/dashboard_view_records/` for `DashboardViewRecord`, with metric-dictionary, matched-comparison, diagnostic-only, negative-result, claim-evidence, warning, and export lineage preserved rather than stored only in rendered dashboard pages;
- simulation and embodiment outputs carry explicit simulation-only and non-clinical meta-data;
- optional Stretch C modules may be implemented incrementally, but incomplete modules must be marked in manifests and cards rather than silently appearing complete.

24.2 Input and output contract of the repository layer

The Recommended Repository Architecture section consumes architectural information from all preceding layers and produces implementation-facing artifacts for the remaining thesis workflow.

Input category	Repository interpretation	Repository consequence
Layer numbering and responsibilities	Each scientific layer has a source-code owner, configuration owner, record family, validation gate, and artifact output location.	The repository is organized by stable modules rather than by ad-hoc experiment names.
Canonical records	Records are the contracts by which layers communicate.	<code>records/</code> , <code>schemas/</code> , <code>runs/</code> , and validation reports must preserve record identity, schema version, producer, consumer, and provenance.
Cross-layer modules A–I	Modules define allowed dependency directions and prevent monolithic implementation.	<code>src/</code> packages map to modules A–I with explicit ownership and forbidden dependencies.
Detailed input-output proximity map	Each output must have a downstream consumer and each input must be typed.	Generated artifacts must include source-record IDs and consumer metadata.
Implementation pipeline	Phase order determines which folders and records must exist before later phases run.	Experiment commands must fail closed if required upstream records or validation gates are missing.

Ablation ladder	A0–A13 comparisons require repeatable configuration, matched splits, matched coverage or matched cost, saved metrics, Layer 1 ablation-readiness status, metric-dictionary entries, negative-result notes, and Layer 0 claim-evidence boundaries.	<code>configs/ablations/</code> , <code>src/evaluation/</code> , <code>records/</code> , <code>figures/</code> , <code>thesis_tables/</code> , and <code>manifests/layer1_ablation_readiness_l1_v1</code> must support ablation traceability.
Dashboard architecture	Dashboard pages are observability views over saved records.	<code>dashboard_app/</code> , <code>src/dashboard/</code> , and <code>src/reporting_cards/</code> must separate interactive rendering from experimental computation.
Failure modes and defensive design	Repository layout must actively prevent hidden leakage, overclaiming, untracked outputs, and optional-module ambiguity.	Tests, manifests, claim-safety configs, warning logs, and fail-closed validation are required.

The outputs of the Recommended Repository Architecture section are the repository-level architecture artifacts: expanded repository tree, module-to-directory map, configuration map, canonical record storage map, schema and manifest policy, run and experiment registry policy, dashboard and card export contract, testing and validation map, reproducibility package policy, optional-module boundary policy, repository-level failure-mode defenses, and a consistency checklist.

24.3 Repository design principles

The repository is built around the following design principles.

1. **Record-first implementation.** Layers communicate through saved canonical records, not through hidden in-memory state.
2. **One-way dependency direction.** Upstream scientific modules must not depend on downstream dashboards, embodiment demos, stress visualizations, or thesis-export scripts.
3. **No silent mutation.** One module must not overwrite or mutate another module's records. If a later module adds interpretation, it writes a new record family with references to the source records.
4. **Config-controlled experiments.** All experiments are launched from versioned YAML or JSON configurations. Configuration files are part of the scientific method because they define datasets, splits, preprocessing, models, thresholds, action costs, reward terms, stress conditions, simulation settings, and claim boundaries.
5. **Generated artifacts are not source code.** Records, figures, cards, logs, model checkpoints, and dashboard exports live outside human-written source packages.
6. **Traceability over convenience.** Every saved artifact should point to a run ID, config hash, seed, code version, source record IDs, and schema version.

7. **Dashboard read-only behavior.** Dashboard code may aggregate and visualize saved records, but it must not train decoders, tune thresholds, generate policies, inject stress profiles, or create thesis results invisibly.
8. **Simulation-only boundary preservation.** Policy-learning simulation, contextual-bandit experiments, reinforcement-learning experiments, StressForge-Lite stress runs, and MyoSuite/OpenSim embodiment are labeled as computational or simulated proxy evidence.
9. **Public-data discipline.** Public EEG datasets are referenced through manifests, dataset adapters, fingerprints, and protocol records rather than undocumented local folders.
10. **Fail-closed validation.** Missing identifiers, missing schema versions, missing source-record references, missing split metadata, missing claim-safety tags, missing ablation-readiness keys, missing matched-comparison IDs, missing metric-dictionary entries, or missing simulation-only warnings must cause validation failure rather than silent continuation.

24.4 Canonical top-level repository tree

The following tree preserves the original repository anchors and expands them into an implementation-grade repository structure.

```

IHARQ-BenchGuard/
  README.md
  LICENSE
  CITATION.cff
  pyproject.toml or setup.cfg
  requirements.txt or environment.yml
  reproducibility.md

  configs/
    datasets/
    decoders/
    calibration/
    iharq/
      action_tokens.yaml
      action_display_labels.yaml
    regimerisk/
    policies/
    stressforge/
    layer9_embodiment/
    dashboard/
    ablations/
    claim_safety/
    experiments/

  schemas/
    records/
    configs/
    manifests/
    validation_rules/

```

```
    iharq_action_tokens.yaml
    readiness_action_alias_map.yaml

src/
  common/
  data_protocol/
  decoders/
  calibration_uncertainty/
  iharq/
  regimerisk/
  adaptive_policy/
  simulation/
  stressforge/
  layer9_embodiment/
    adapters/
    mapping/
    safety/
    stress/
    outcomes/
    logging/
    cards/
  dashboard/
  reporting_cards/
  evaluation/
  validation/
  orchestration/
  claim_safety/

scripts/ or cli/
  make_protocol.py
  train_decoder.py
  log_predictions.py
  calibrate_uncertainty.py
  run_iharq.py
  run_regimerisk.py
  train_policy.py
  run_simulation.py
  run_stressforge.py
  run_embodiment_demo.py
  build_dashboard_records.py
  generate_cards.py
  generate_ablations.py
  export_thesis_artifacts.py
  validate_run.py

records/
  dataset_records/
  window_records/
  split_records/
  preprocessing_records/
  label_map_records/
```



```
artifact_flag_records/  
prediction_records/  
calibration_records/  
uncertainty_records/  
selective_prediction_records/  
threshold_registry_records/  
evidence_quality_records/  
iharq_records/  
regime_state_records/  
readiness_context_records/  
deferral_records/  
policy_records/  
policy_update_traces/  
transition_records/  
closed_loop_session_records/  
episode_summary_records/  
policy_rollout_records/  
reward_trace_records/  
environment_diagnostic_records/  
stress_records/  
    stress_profile_records.parquet  
    stress_schedule.parquet  
    stressed_record_pointers.parquet  
    stress_run_manifest.csv  
    stress_validity_report.json  
evaluation_outcomes/  
    layer8_stress_evaluation_outcomes.parquet  
    layer9_embodiment_evaluation_outcomes.parquet  
embodiment/  
    embodiment_state_records.parquet  
    embodiment_outcome_records.parquet  
    transition_records.parquet  
    safety_event_log.csv  
    reward_traces/  
    demo_manifests/  
safety_event_records/  
dashboard_metric_records/  
dashboard_view_records/  
card_records/  
warning_records/  
validation_reports/  
reproducibility_manifests/  
  
runs/  
    <run_id>/  
        config_snapshot/  
        manifest.json  
        logs/  
        records/  
        models/  
        figures/
```

```
cards/
dashboard_exports/
  dashboard_metric_records.jsonl
  dashboard_view_records.jsonl
  figure_table_manifest.json
results/
  layer7_closed_loop/
    configs/
      environment_config.yaml
      reward_config.yaml
      policy_update_config.yaml
    sessions/
      closed_loop_session_records.parquet
    transitions/
      transition_records.parquet
    rewards/
      reward_trace_records.parquet
    policy_updates/
      policy_update_traces.parquet
      update_disabled_audit.csv
    evaluation/
      matched_comparison_report.csv
      evaluation_outcomes.parquet
    summaries/
      episode_summary_records.parquet
    diagnostics/
      environment_diagnostic_records.parquet
  cards/
    closed_loop_environment_card.md
    policy_rollout_card.md
    policy_update_trace_card.md
    stress_conditioned_policy_card.md
    safety_event_card.md
    reward_configuration_card.md
    reproducibility_card.md
  layer9_embodiment/
    embodiment_states.parquet
    embodiment_outcomes.parquet
    transition_records.parquet
    safety_events.csv
    reward_trace.parquet
    command_timeline.parquet
    endpoint_error_curve.csv
    demo_manifest.json
    adapter_logs/
      embodiment_card.md
    dashboard_export_bundle/
validation_reports/
warnings/

registries/
```

experiment_registry.csv or .json
run_registry.csv or .json
model_registry.csv or .json
threshold_registry.csv or .json
policy_registry.csv or .json
stress_profile_registry.csv or .json
embodiment_mapping_registry.csv or .json
dashboard_export_registry.csv or .json
card_registry.csv or .json

models/
 decoder_checkpoints/
 calibration_models/
 evidence_quality_estimators/
 policy_models/
 simulation_policies/

figures/
 protocol/
 baselines/
 calibration/
 iharq/
 regimerisk/
 policies/
 simulation/
 stressforge/
 embodiment/
 ablations/
 dashboard_exports/
 failure_modes/

cards/
 dataset_cards/
 protocol_cards/
 reliability_cards/
 calibration_cards/
 iharq_cards/
 regime_cards/
 controller_cards/
 closed_loop_environment_cards/
 policy_rollout_cards/
 policy_update_trace_cards/
 safety_event_cards/
 reward_configuration_cards/
 stress_cards/
 embodiment_cards/
 failure_cards/
 reproducibility_cards/

dashboard_app/
 pages/

```
components/  
  static_exports/  
  app_config/  
  README.md  
  
notebooks/  
  exploration/  
  explanation/  
  sanity_checks/  
  
thesis_tables/  
  protocol_tables/  
  baseline_tables/  
  calibration_tables/  
  iharq_tables/  
  policy_tables/  
  stress_tables/  
  embodiment_tables/  
  ablation_tables/  
  limitation_tables/  
  
reports/  
  validation/  
  ablation_summaries/  
  matched_comparisons/  
    layer8_matched_stress_comparisons.csv  
    layer9_matched_embodiment_comparisons.csv  
  stress_summaries/  
  reproducibility/  
  thesis_export_manifest/  
  
metrics/  
  metric_dictionary.json  
  metric_dictionary_layer8_stress.yaml  
  metric_dictionary_layer9_embodiment.yaml  
  
notes/  
  negative_results/  
    layer8_stress_negative_results.md  
    layer9_embodiment_negative_results.md  
  
manifests/  
  layer1_manifest_l1_v1.json  
  layer1_ablation_readiness_l1_v1.json  
  claim_evidence_matrix.json  
  claim_evidence/  
    layer8_stress_claim_evidence.json  
    layer9_embodiment_claim_evidence.json  
  metric_dictionary.json  
  artifact_manifest.yaml  
  reproducibility_manifest.yaml
```

```

run_registry.yaml

logs/
  pipeline/
  training/
  evaluation/
  dashboard/
  policy_update_audits/
  validation/
  warnings/

tests/
  unit/
  integration/
  leakage/
  schema/
  validation/
  dashboard/
  reproducibility/
  fixtures/

docs/
  architecture_notes/
  record_specs/
  command_reference/
  claim_safety_notes/

```

The added folders have specific architectural purposes. `manifests/layer1_ablation_readiness_l1_v1.json` stores the public-data/protocol matched-comparison key completeness and diagnostic-only downgrade status required by the Layers 0–3 evaluation slice; `manifests/claim_evidence_matrix.json` and `manifests/metric_dictionary.json` store the Layer 0/Layer 10 claim-evidence and metric-definition anchors used by ablation exports. `schemas/` stores formal record, configuration, and manifest definitions, including `schemas/validation_rules/iharq_action_tokens.yaml` and `schemas/validation_rules/readiness_action_alias_map.yaml` for Layer 4 action-token validation and downstream display/alias mapping. `runs/` stores run-specific outputs grouped by run ID, config hash, seed, timestamp, experiment phase, and Layer 7 closed-loop result folders for configs, sessions, transitions, rewards, policy updates, evaluation, summaries, diagnostics, and cards. `records/policy_update_traces/` stores promoted `PolicyUpdateTrace` records when simulation-authorized bandit/RL updates are enabled or audited; `runs/<run_id>/results/layer7_closed_loop/sessions/closed_loop_session_records.parquet`, `transitions/transition_records.parquet`, `rewards/reward_trace_records.parquet`, `summaries/episode_summary_records.parquet`, `summaries/safety_event_records.parquet` or `summaries/safety_events.csv` where safety events are materialized, `diagnostics/environment_diagnostic_records.parquet`, and `cards/*.md` store the run-local closed-loop record/card families; `runs/<run_id>/results/layer7_closed_loop/policy_updates/policy_update_traces.parquet` stores run-local update traces; `runs/<run_id>/results/layer7_closed_loop/policy_updates/update_dis` stores the corresponding frozen-policy disabled-update audit; `cards/policy_update_trace_cards/` and `logs/policy_update_audits/` store the corresponding Layer 10 reporting and disabled-update evidence.

`records/dashboard_metric_records/` and `records/dashboard_view_records/` store the canonical Layer 10 dashboard records, while `runs/<run_id>/dashboard_exports/dashboard_metric_records.jsonl`, `runs/<run_id>/dashboard_exports/dashboard_view_records.jsonl`, and `runs/<run_id>/dashboard_exports/figure_table_manifest.json` store the run-local rendered export lineage. `records/embodiment/` stores the Layer 9 canonical embodiment state, outcome, embodiment-linked transition, safety-event-log, reward-trace, and demo-manifest families that feed Layer 10 embodiment pages, cards, exports, and A13 reproducibility checks. `registries/` stores indexes that allow the thesis to locate models, thresholds, policies, stress profiles, embodiment mappings, cards, dashboard exports, and policy-update trace exports. `scripts/` or `cli/` stores thin command entry points that call `src/` modules. `reports/` stores generated validation, ablation, stress, and reproducibility summaries. `docs/` stores implementation-facing notes, record specifications, command references, and claim-safety text.

The tree distinguishes human-written source files from generated artifacts. `src/`, `configs/`, `schemas/`, `scripts/`, `tests/`, and `docs/` are human-maintained or version-controlled implementation assets. `records/`, `runs/`, `models/`, `figures/`, `cards/`, `thesis_tables/`, `reports/`, and `logs/` are generated or curated outputs whose provenance must be recorded.

24.5 Configuration architecture

`configs/` stores scientific and engineering choices. A configuration file is not merely an engineering convenience; it is a reproducibility artifact and part of the experimental method.

Configuration path	Responsibility	Claim-safety and reproducibility role
<code>configs/datasets/</code>	Public EEG dataset selection, dataset IDs, access instructions, preprocessing options, label maps, channel sets, session/run choices, windowing, split strategy, and low-calibration settings.	Prevents undocumented local data use and supports dataset cards, protocol cards, and split-leakage checks.
<code>configs/decoders/</code>	CSP+LDA/SVM, Riemannian tangent-space, EEGNet, ShallowConvNet, optional EEG-TCNet or compact SSL, ensemble settings, training seeds, and model-selection rules.	Ensures that decoder comparisons are reproducible and that later reliability layers can cite exact prediction sources.
<code>configs/calibration/</code>	Temperature scaling, Platt scaling, isotonic or other calibration choices where used, uncertainty features, selective-prediction thresholds, threshold-selection splits, and baseline rejection rules.	Prevents threshold tuning on held-out test records and supports calibration cards and reliability diagrams.

<code>configs/iharq/</code>	IHARQ-lite rules, canonical action-token definitions, display-label mapping, accept/wait/request-evidence/combine-evidence/fallback/uncertain/unsafe decisions, reason codes, stability checks, evidence-request logic, unsafe states, and fallback rules.	Makes IHARQ decisions auditable rather than opaque thresholding and prevents downstream display labels or aliases from becoming stored Layer 4 action values.
<code>configs/regime_risk/</code>	Temporal windows, drift-risk proxies, trust-score definitions, volatility-like measures, stop-loss rules, reset rules, and feedback adapters.	Keeps RegimeRisk temporal trust monitoring explicit and testable.
<code>configs/policies/</code>	Evidence-quality estimator settings, supervised Adaptive-IHARQ, learning-to-defer, contextual bandit, reinforcement-learning policy, action spaces, Layer 4-to-policy action maps, legality masks, cost vectors, reward functions, pseudo-label construction, simulation-only flags, and optional policy-update configuration for declared simulation-training/adaptation runs.	Separates computational readiness labels from clinical labels, prevents policy-learning overclaiming, preserves source IHARQ action-token identity, and distinguishes update-enabled simulation from frozen-policy evaluation.
<code>configs/stressforge/</code>	Stress families, intensity levels, schedules, seeds, injection points, missingness or noise patterns, delay settings, guardrails, and stress claim boundaries.	Ensures robustness tests can be repeated and interpreted as stress-conditioned computational evidence.
<code>configs/layer9_embodiment/</code>	MyoSuite/OpenSim/static-replay adapter settings, L9.1–L9.13 branch selection, command mappings, simulator branch, readiness safety gates, mapping IDs, reward-trace settings where enabled, replay settings, endpoint-error output settings, command-timeline settings, demo-manifest template, screenshot/plot manifests, and non-clinical warning text.	Forces embodiment outputs to remain simulated proxy demonstrations and prevents an independent Layer 20/P20 runtime branch from reappearing.

<code>configs/dashboard/</code>	Page definitions, source-record requirements, export settings, card settings, figure and table definitions, read-only behavior, warning display rules, optional <code>DashboardMetricRecord.policy_update_filter</code> , policy-update trace card settings, frozen/update curve-separation settings, and static export settings.	Prevents the dashboard from becoming a hidden result generator or a hidden policy-update mixer.
<code>configs/ablations/</code>	A0–A13 comparisons, A0 raw-decoder baselines, A1 calibration comparisons, A2 threshold comparisons, A3 uncertainty/selective-prediction comparisons, A4 ordinary-combination controls, matched-coverage comparisons, matched-cost comparisons, reward ablations, stress-condition matching, metric dictionary references, negative-result note templates, and report templates.	Ensures complexity is tested against fair baselines and that the Layers 0–3 evaluation slice remains aligned with Layer 1 matchability and Layer 0 claim boundaries.
<code>configs/claim_safety</code>	Allowed-claim tags, limitation text, simulation-only labels, non-clinical warning templates, public-data scope statements, diagnostic-only labels, evaluation-mode labels, claim-evidence matrix schema, warning-inheritance rules, and invalid-claim warnings.	Propagates Layer 0 scope governance into all generated artifacts, including ablation tables and matched-comparison exports.
<code>configs/experiments/</code>	Named experiment bundles that combine dataset, decoder, calibration, IHARQ, RegimeRisk, policy, stress, embodiment, dashboard, and ablation configs.	Allows entire thesis experiments to be relaunched from one declared configuration set.

Each configuration should declare its schema version, intended layer, owner module, required upstream records, produced downstream records, random seed policy, and claim-safety tags. A run should store a frozen copy of the exact configuration files used under `runs/<run_id>/config_snapshot/`.

24.6 Source-code module architecture and allowed dependencies

`src/` implements the cross-layer module architecture. The repository must not collapse these packages into one monolithic training script.

Source package	Layer/module owner	Primary inputs	Primary outputs
----------------	--------------------	----------------	-----------------

<code>src/common/</code>	Shared utilities only	Run IDs, logging helpers, hashing, path handling, seed helpers, typed identifiers.	Common helper functions with no scientific decisions.
<code>src/data_protocol/</code>	Layer 1, Module A	Public EEG manifests, dataset configs, preprocessing configs, split/window configs, validation rules, and Layer 0 claim constraints.	DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, DatasetCard, ProtocolCard, ValidationReport, Layer1Manifest, and <code>layer1_ablation_readiness_l1_v1.json</code> .
<code>src/decoders/</code>	Layer 2, Module B	DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, decoder configs, label-budget configs, seed policy, and optional compact SSL checkpoints.	PredictionRecord, decoder checkpoints, ModelRegistryRecord, BaselineMetricRecord, LowCalibrationCurveRecord, SubjectProfileRecord, EnsembleControlRecord, FailureCaseIndex, ModelCard, BenchmarkCard, and Layer2ReadinessReport.
<code>src/calibration_uncertainty/</code>	Layer 3, Module C	PredictionRecord, calibration configs, calibration splits.	CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord.
<code>src/iharq/</code>	Layer 4, Module D	PredictionRecord, CalibrationRecord, UncertaintyRecord, ThresholdRegistryRecord, ArtifactFlagRecord, optional RegimeStateRecord, local/precomputed stability/agreement views, and IHARQ configs.	EvidenceQualityRecord, IHARQDecisionRecord with canonical action tokens and display-label mapping, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAbationRecord, IHARQTraceRecord, Layer 4 EvaluationOutcomeRecord, reason-code summaries, matched-comparison outputs, negative-result notes, trace exports, and IHARQ reliability-card inputs.
<code>src/regime_risk/</code>	Layer 5, Module E	Time-ordered PredictionRecord, CalibrationRecord, UncertaintyRecord, IHARQDecisionRecord, temporal configs, optional stress/transition/embodiment context where declared.	RegimeStateRecord, conditional TrustStateRecord rows, trust-score tables, regime timelines, stop-loss warnings, risk-error alignment tables, and feedback advisories.
<code>src/adaptive_policy/</code>	Layer 6, Module F	ReadinessContextRecord, EvidenceQualityRecord, DeferralRecord, costs, rewards, legality masks, pseudo-labels.	PolicyActionRecord, learned policy checkpoints, policy evaluation records, controller cards.
<code>src/simulation/</code>	Layer 7, part of Module G	PolicyActionRecord, IHARQDecisionRecord, RegimeStateRecord, simulation configs, reward configs, optional stress schedules, optional embodiment links, and optional policy-update config.	TransitionRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, RewardTraceRecord, EnvironmentDiagnosticRecord, SafetyEventRecord, optional PolicyUpdateTrace files, closed-loop cards, and disabled-update audits.
<code>src/stressforge/</code>	Layer 8, part of Module G	Source records, stress profiles, stress schedules, seeds, injection rules, matched-comparison groups, and clean-baseline manifests.	StressProfileRecord, StressScheduleRecord, stressed-record pointers, stress-run manifests, stress-validity reports, stress-conditioned metrics, Layer 8 evaluation outcomes, matched-stress comparison reports, stress negative-result notes, stress claim-evidence entries, stress cards, and dashboard-ready stress exports.

<code>src/layer9_embodiment/</code>	Layer 9, Module H	PolicyActionRecord, ReadinessContextRecord, TransitionRecord, StressProfileRecord where used, embodiment configs, command mappings, safety-gate configs, simulator/static-replay branch settings.	EmbodimentStateRecord, EmbodimentOutcomeRecord, embodiment-linked TransitionRecord pointers, SafetyEventRecord or safety-event logs, Layer9RewardTrace where enabled, command timelines, endpoint-error curves, Layer9EmbodimentDemoManifest , dashboard-compatible export bundle, simulated proxy trajectories, screenshots/plots, and embodiment-card inputs.
<code>src/dashboard/</code>	Layer 10, Module I backend	Saved records, Layer 7 TransitionRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, RewardTraceRecord, EnvironmentDiagnosticRecord, SafetyEventRecord where materialized, optional PolicyUpdateTrace files, disabled-update audits, Layer 8 StressProfileRecord and StressScheduleRecord files, stress metrics, matched-stress reports, stress cards, Layer 9 embodiment bundles, safety-event logs, Layer9RewardTrace files where enabled, Layer9EmbodimentDemoManifest files, export manifests, dashboard configs.	DashboardMetricRecord including policy_update_filter, stress_filter, embodiment_filter, and reward_trace_filter where applicable; DashboardViewRecord; static dashboard data exports; closed-loop session timeline, policy-rollout, reward-decomposition, environment-diagnostic, and StressForge pages; Layer 9 embodiment evidence pages; separated training/adaptation versus frozen-policy evaluation exports.
<code>src/reporting_cards/</code>	Layer 10, Module I reports	Saved records, Layer 7 closed-loop records, optional PolicyUpdateTrace files, disabled-update audits, warning logs, claim-safety templates, manifests.	Dataset cards, protocol cards, reliability cards, controller cards, closed-loop environment cards, policy rollout cards, policy-update trace cards, reward configuration cards, stress cards, embodiment cards, failure cards, reproducibility cards.
<code>src/evaluation/</code>	Ablation and Evaluation Architecture spine	Prediction, calibration, IHARQ, regime, policy, stress, simulation, and embodiment records.	Metrics, ablation records, statistical comparison tables, thesis-ready exports.
<code>src/validation/</code>	Cross-layer validation	Schemas, manifests, records, config snapshots.	Validation reports, failure logs, leak checks, reproducibility checks.
<code>src/orchestration/</code>	the Implementation Pipeline section pipeline controller	Experiment configs, phase definitions, dependency graph.	Phase run plans, run manifests, execution logs, missingness reports.
<code>src/claim_safety/</code>	Layer 0 propagation	Claim-safety configs, artifact metadata, limitation templates.	Allowed-claim tags, warning summaries, invalid-claim flags, simulation-only labels.

The dependency direction is deliberately constrained.

- `src/decoders/` may consume dataset, window, preprocessing, split, label-map, artifact-flag, label-budget, seed, decoder-configuration, and optional compact-SSL checkpoint inputs. It must not consume IHARQ, RegimeRisk, policy, stress, embodiment, dashboard, or thesis-table outputs.
- `src/calibration_uncertainty/` may consume decoder predictions and calibration split metadata. It must not tune thresholds on held-out test records.

- **src/iharq/** may consume calibrated probabilities, uncertainty features, stability summaries, and optional RegimeRisk context. It must not inspect future labels when forming decisions.
- **src/regime_risk/** may consume temporal histories and downstream transition summaries only when the configuration explicitly declares a feedback-analysis mode. It must not mutate original dataset, split, prediction, or calibration records.
- **src/adaptive_policy/** may consume readiness contexts, costs, rewards, pseudo-labels, deferral labels, and simulation rewards under declared split controls. It must not use evaluation-only labels for action selection.
- **src/simulation/** and **src/stressforge/** may create derived records, but must tag them as simulated or stressed variants rather than overwriting clean public-data records.
- **src/layer9_embodiment/** may consume readiness actions and simulated transitions. It must not be represented as proof of real rehabilitation control.
- **src/dashboard/** and **dashboard_app/** may read saved records and manifests. They must not train models, tune thresholds, construct pseudo-labels, or generate hidden experimental results.

24.7 Canonical record and artifact storage architecture

records/ stores canonical reusable record families. **runs/** stores run-specific copies, logs, models, figures, cards, validation reports, and manifests. **figures/**, **cards/**, **thesis_tables/**, and **reports/** store curated exports derived from records. The same record may therefore appear in a run-specific folder and, if promoted, in a canonical **records/** family. Promotion should preserve provenance rather than rewrite identity.

Record family	Owner	Storage location	Format	Consumers
DatasetRecord	Layer 1 / Module A	records/dataset_records/	JSON or Parquet summary	Decoders, dashboard, cards, thesis protocol tables.
WindowRecord	Layer 1 / Module A	records/window_records/	Parquet	Decoders, calibration, stress tests.
SplitRecord	Layer 1 / Module A	records/split_records/	JSON/CSV	All downstream training and evaluation modules.
PreprocessingRecord	Layer 1 / Module A	records/preprocessing_records/	JSON	Decoders, reproducibility cards.
LabelMapRecord	Layer 1 / Module A	records/label_map_records/	JSONs	Decoders, metrics, dashboard.

ArtifactFlagRecord	Layer 1 / Module A	records/artifact_flag_records/	CSV/Parquet	Decoders, stress analysis, limitation tables.
DatasetCard	Layer 1 / Module A	cards/dataset_cards/	Markdown/JSON/HTML	Layer 10, thesis protocol appendix, claim-safety review.
ProtocolCard	Layer 1 / Module A	cards/protocol_cards/	Markdown/JSON/HTML	Layer 10, ablation-readiness panels, thesis protocol appendix.
ValidationReport	Layer 1 / Module A	reports/layer1_validation/, reports/leakage_audits/, and run-local validation reports	JSON/HTML/Markdown	All downstream modules, Layer 10 validation panels, reproducibility package.
Layer1Manifest	Layer 1 / Module A	manifests/layer1_manifest_11_v1.json records/reproducibility_manifests/	JSON	All downstream modules.
Layer1AblationReadiness	Layer 1 / Module A	manifests/layer1_ablation_readiness/	JSON	Evaluation scripts, Layer 10 ablation-readiness panels, A0–A4 matchability reports, and reproducibility manifests.
PredictionRecord	Layer 2 / Module B	records/prediction_records/	Parquet	Calibration, IHARQ, RegimeRisk, policy-context traceability, simulated replay, stress replay, bounded embodiment provenance, evaluation, dashboard.

ModelRegistryRecord	Layer 2 / Module B	records/model_registry	CSV/Parquet/YAML	Calibration score-schema checks, evidence/policy provenance, simulation replay provenance, dashboard model cards, reproducibility manifests.
BaselineMetricRecord	Layer 2 / Module B	records/baseline_metrics	CSV/Parquet	A0 decoder-baseline reporting, ablation/evaluation scripts, dashboard baseline panels, thesis tables.
LowCalibrationCurveRecord	Layer 2 / Module B	records/low_calibration_curves	CSV/Parquet/JSON	Low-calibration figures, budget ablations, Layer 10 dashboard curves, thesis low-calibration reporting.
SubjectProfileRecord	Layer 2 / Module B	records/subject_profiles	CSV/Parquet	RegimeRisk diagnostics, policy-context analysis where allowed, replay/stress analysis where explicitly allowed, dashboard subject-profile views.
EnsembleControlRecord	Layer 2 / Module B	records/ensemble_records	CSV/Parquet/JSON	Calibration/disagreement checks, IHARQ A4 ordinary-combination controls, dashboard ensemble-control tables, ablation scripts.

FailureCaseIndex	Layer 2 / Module B	records/failure_case_index/ and reports/layer2_failure_cases/	CSV/Parquet/Markdown and IHARQ and RegimeRisk offline failure analysis, high-risk decoder examples, dashboard failure views, defensive-design sections.
Layer2ReadinessReport	Layer 2 / Module B	reports/layer2_readiness/ and run-local validation folders	JSON/HTML/Markdown Layers 3–10 consumer-readiness gates, Layer 10 readiness panels, evaluation scripts, reproducibility manifests, final thesis reporting.
ModelCard	Layer 2 records / Layer 10 rendering	cards/model_cards/	Markdown/JSON/HTML/HAR model views, thesis baseline reporting, reproducibility packages.
BenchmarkCard	Layer 2 records / Layer 10 rendering	cards/layer2_benchmark_cards/	Markdown/JSON/HTML/HAR benchmark evidence, low-calibration and ensemble-control reporting, dashboard benchmark views, thesis ablation reporting.
CalibrationRecord	Layer 3 / Module C	records/calibration_records/	JSON/Parquet IHARQ, evaluation, reliability cards.
UncertaintyRecord	Layer 3 / Module C	records/uncertainty_records/	Parquet IHARQ, RegimeRisk, policy learning.
SelectivePredictionRecord	Layer 3 / Module C	records/selective_prediction_records/	Parquet Baseline comparisons and ablations.

ThresholdRegistryRecord	Layer 3 / Module C	records/threshold_registry/records/and registries/	JSON/CSV records/IHARQ, evaluation, dashboard.
EvidenceQualityRecord	Layer 4 / Module D	records/evidence_quality/records/	IHARQ, Adaptive-IHARQ, dashboard, cards.
IHARQDecisionRecord	Layer 4 / Module D	records/iharq_records/	RegimeRisk, policy learning, simulation, dashboard; stored action field must use Layer 4 canonical tokens and optional display labels must be separate fields.
RegimeStateRecord	Layer 5 / Module E	records/regime_state_records/and run-local results/regime_risk/regime_state_records.	IHARQ context, policy, simulated-session state, simulation-only embodiment gating, dashboard views, evaluation scripts, and Regime-Card lineage.
TrustStateRecord	Layer 5 / Module E	records/trust_state_records/and run-local results/regime_risk/trust_state_records, where exported	Independent trust-timeline policy requests, Layer 10 trust-state plots, reproducibility manifests, and optional IHARQ/simulation/embodiment references through linked RegimeStateRecord fields.
ReadinessContextRecord	Layers 4–6 / Modules D–F	records/readiness_context/records/	Adaptive policies, deferral baselines, simulation.

DeferralRecord	Layer 6 / Module F	records/deferral_records/	Learning-to-defer evaluation, ablations.
PolicyActionRecord	Layer 6 / Module F	records/policy_records/	Simulation, embodiment, dashboard, controller cards; should preserve source_iharq_action and action-map references when derived from Layer 4.
PolicyUpdateTrace	Layer 6–7 / Modules F–G	records/policy_updates/ and audit runs/<run_id>/results/layer7_closed_loop/policies_updates/plus update_disabled_audit.csv when frozen-update auditing is materialized	Policy-update audit, Layer dashboard frozen/update separation, reproducibility manifests.
TransitionRecord	Layer 7 / Module G	records/transition_records/	Policy evaluation, RegimeRisk feedback analysis, dashboard, optional linkage to PolicyUpdateTrace through policy_update_trace_id; separates source IHARQ token, policy action, validated readiness action, and simulator command.
ClosedLoopSessionRecord	Layer 7 / Module G	records/closed_loop_sessions/ and runs/<run_id>/results/layer7_closed_loop/sessions/closed_	Closed-loop session time-metadata, run manifests, simulation-only warnings, and reproducibility package.

EpisodeSummaryRecord	Layer 7 / Module G	records/episode_summary/ and runs/<run_id>/results/layer7_closed_loop/summaries/episode	Parquet/CSV/ A10/A11/A12 episode sum- maries/episod comparisons, matched- comparison reports, and thesis simula- tion tables.
PolicyRolloutRecord	Layer 7 / Module G	records/policy_rollout/ and runs/<run_id>/results/layer7_closed_loop/evaluation/polic	Parquet/JSON Policy rollout cards, action- drill-downs, Layer 6 policy analysis, ab- lation scripts, and Layer 10 views.
RewardTraceRecord	Layer 7 / Module G	records/reward_trace/ and runs/<run_id>/results/layer7_closed_loop/rewards/reward_t	Parquet/CSV/JSON Reward de- composition, figures, policy- update au- dits, reward- sensitivity reports, and reproducibility manifests.
EnvironmentDiagnosticRecord	Layer 7 / Module G	records/environment_diagnostic/ and runs/<run_id>/results/layer7_closed_loop/diagnostics/envi	Parquet/JSON/Records/ action, failed- thrust/diagnostics/envi leakage, unmatched- comparison, missing-link, reward- anomaly, and disabled- update diag- nostic warn- ings for Layer 10 and claim- safety review.
StressProfileRecord	Layer 8 / Module G	records/stress_records/	JSON/Parquet Stress- conditioned evaluation, dashboard, stress cards.

StressScheduleRecord	Layer 8 / Module G	records/stress_records/ and run-local stress- record folders	records/stress_records/ summary replay, Layer 9 embodiment-side stress injection where active, Layer 10 schedule inspection, A12/A13 matched comparisons, stress cards, and reproducibility manifests.
EmbodimentStateRecord	Layer 9 / Module H	records/embodiment/ and runs/<run_id>/results/layer9_embodiments/	EmbodimentStateRecords embodiment embodiment_states evaluation, limitation tables.
EmbodimentOutcomeRecord	Layer 9 / Module H	records/embodiment/ and runs/<run_id>/results/layer9_embodiment/	EmbodimentOutcomeRecords error curves, embodiment_outcome success summaries, stress- conditioned embodiment metrics, em- bodiment cards, A13 evaluation, and limitation tables.
Layer9-linked TransitionRecord	Layer 9 / Module H with Layer 7 lineage	records/embodiment/ and runs/<run_id>/results/layer9_embodiment/	TransitionRecords only transition_record to-policy analysis where enabled, Layer 10 embodiment views, A13 lineage, and non-clinical limitation tables.

SafetyEventRecord or safety-event log	Layer 7 / Mod- ule G and Layer 9 / Mod- ule H where simulation or embodiment emits safety events	records/safety_event_CSV/Params/ runs/<run_id>/results/layer7_closed_loop_summaries/safety or summaries/ safety_events.csv, embodiment records/embodiment/safety_event_logs/ and taxon- runs/<run_id>/results/layer9_embodiment/safety_events.csv	Closed-loop summaries/ embodiment validation, failure taxon- layer events.csv 10 closed-loop and embodi- ment pages, claim-safety cards.
Layer9RewardTrace	Layer 9 / Mod- ule H where reward-enabled embodiment is configured	records/embodiment/RewardTraces/ and runs/<run_id>/results/layer9_embodiment/layer9_reward_trace.parq	A13 reward decomposi- tion/layer9_reward_trace.parq reward-trace filters, con- troller/embodiment cards, reward- overclaim warnings.
Layer9EmbodimentDemoManifest	Layer 9 / Mod- ule H	records/embodiment/demo_manifests/ and runs/<run_id>/results/layer9_embodiment/demo_manifest.js	Layer 10 em- bodiment manifest.js producibility manifest, A13 table, final demo checklist.
DashboardMetricRecords	Layer 10 / Mod- ule I	records/dashboard_metric_records/ and runs/<run_id>/dashboard_exports/dashboard_and_view_records.	Dashboard pages, cards, the- sis figures, matched- comparison/metric- dictionary/negative- result/claim- evidence lin- eage, and final demonstration evidence.

DashboardViewRecord	Layer 10 / Module I	records/dashboard_views/records/ and runs/<run_id>/dashboard_exports/dashboard_views/records.js	Static exports, cards/export
CardRecord	Layer 10 / Module I	records/card_records/JSON	Card registry, reproducibility manifest.
Validation report	Cross-layer	records/validation_reports/JSON/HTML/Manifests/ and runs/<run_id>/validation_reports/	Defensive-design section and reproducibility package.
Warning log	Layer 0 / Cross-layer	records/warning_records/JSON/CSV and logs/warnings/	Dashboard warning page, failure cards, limitation tables.
ReproducibilityManifest	Cross-layer	records/reproducibility_manifests/JSON/ and runs/<run_id>/manifest.json	Thesis tables, figures, cards, rerun instructions.

Each record family should include required identifiers: `record_id`, `schema_version`, `run_id`, `config_hash`, `seed`, `dataset_id` where applicable, `subject_id` or anonymized subject key where applicable, `session_id`, `split_id`, `producer_module`, `source_record_ids`, `timestamp`, `code_version`, `claim_safety_tags`, and `validation_status`. Simulation and embodiment records must additionally include `simulation_only=true` or an equivalent claim-boundary tag. Layer 7 closed-loop records must additionally preserve `closed_loop_session_id`, `episode_id`, `policy_rollout_id`, `reward_trace_id`, `environment_diagnostic_id`, `policy_update_mode`, and simulation-only warning fields. Policy-update records must additionally include `policy_update_mode`, `update_enabled`, `frozen_evaluation_flag`, `exploration_mode` where applicable, `transition_record_id`, and `policy_update_trace_id` links when attached to transitions.

24.8 Schema, manifest, and registry architecture

`schemas/` stores formal definitions for canonical records, configurations, manifests, and validation rules. Schema versioning is required because a thesis result may become invalid if the record definition changes after figures and tables are generated. Each schema should declare required fields, allowed missingness, allowed claim-safety tags, data types, identifier rules, producer/consumer expectations, and enumerated vocabularies. In particular, the `IHARQDecisionRecord` schema must validate `action` against the Layer 4 canonical tokens `accept`, `wait`, `request_evidence`, `combine_evidence`, `fallback`, `uncertain`, and `unsafe`;

display labels and downstream aliases must be validated through separate mapping fields or registry entries.

Each experiment run should create a run manifest under `runs/<run_id>/manifest.json`. The manifest should include:

- run ID, timestamp, machine or environment summary, and code version;
- dataset IDs, split IDs, preprocessing IDs, and label-map IDs;
- model IDs, threshold IDs, IHARQ action-token registry IDs, policy IDs, policy action-map IDs, stress profile IDs, and embodiment mapping IDs where applicable;
- all configuration file paths and configuration hashes;
- random seeds and deterministic or nondeterministic environment notes;
- produced records, models, figures, cards, dashboard exports, logs, reports, Layer 7 transition, closed-loop session, episode-summary, policy-rollout, reward-trace, environment-diagnostic, safety-event, closed-loop card paths, optional `PolicyUpdateTrace` paths, disabled-update audit paths, and policy-update trace-card paths;
- validation status, failed outputs, missing optional outputs, warnings, policy-update completeness status, frozen/update separation status, and failure-taxonomy labels;
- claim-boundary tags, especially public-data, offline, pseudo-online, simulation-only, and non-clinical tags.

`registries/` indexes reusable artifacts across runs.

- The experiment registry records named experiment bundles.
- The run registry records run status, config hashes, outputs, validation state, and thesis relevance.
- The model registry records decoder, calibration, evidence-quality, and policy checkpoints.
- The threshold registry records threshold origins, split controls, and selection criteria.
- The policy registry records supervised, learning-to-defer, contextual-bandit, and RL policy versions.
- The stress profile registry records StressForge-Lite families, intensity levels, schedules, seeds, and injection points.
- The embodiment mapping registry records action-to-simulator mappings, simulator branch, command legality, safety-gate IDs, demo-manifest IDs, and non-clinical warning metadata.
- The `Layer9EmbodimentDemoManifest` registry, when implemented separately from the mapping registry, records demo-manifest paths, safety-event-log paths, reward-trace paths where enabled, command-timeline paths, endpoint-error-curve paths, simulator-version metadata, and dashboard-compatible export bundles.
- The simulation registry records Layer 7 session, episode, policy-rollout, reward-trace, environment-diagnostic, safety-event, reward-configuration, and policy-update paths for closed-loop runs.

- The dashboard export registry records static dashboard exports, `DashboardMetricRecord.policy_update_filter`, `DashboardMetricRecord.embodiment_filter`, and `DashboardMetricRecord.reward_trace_filter` values where applicable, and their source records.
- The card registry records generated cards, source records, warnings, and thesis locations.
- The policy-update trace registry, when implemented, records `PolicyUpdateTrace` file locations, update modes, disabled-update audits, and source transition IDs.

Manifests and registries are the mechanism that lets the thesis trace every table, plot, dashboard panel, card, and claim back to the source records.

24.9 Experiment orchestration and command structure

Experiment commands should be thin wrappers around `src/` packages. They should read versioned configurations, validate dependencies, write records, write logs, update manifests, and fail closed if required upstream artifacts are missing.

The repository should support reproducible commands for the following phases:

- data and protocol generation;
- baseline decoder training and prediction logging;
- calibration, uncertainty extraction, and selective-prediction baselines;
- IHARQ-lite decision generation, canonical action-token validation, display-label export, and evidence-quality recording;
- RegimeRisk timeline generation and stop-loss warning production;
- Adaptive-IHARQ, evidence-quality estimator, and learning-to-defer training and evaluation;
- closed-loop simulation, contextual-bandit, and reinforcement-learning policy experiments;
- StressForge-Lite stress-profile generation and stress-conditioned evaluation;
- Layer 9 MyoSuite/OpenSim/static-replay embodiment replay or demo generation, including record intake validation, readiness-context assembly, command mapping, safety-event logging, reward-trace export where enabled, demo-manifest generation, and dashboard-compatible export packaging;
- dashboard data preparation and static dashboard export;
- card generation;
- ablation generation and thesis-table export;
- run validation and reproducibility replay.

The command layer should not contain scientific logic. Scientific logic belongs in `src/`; commands only connect validated inputs to declared outputs. Notebooks under `notebooks/` may support exploration, sanity checks, and explanatory figures, but any notebook result used in the thesis must be regenerated into records, figures, cards, tables, and manifests through the command structure.

24.10 Dashboard, cards, and thesis-export contract

The repository separates dashboard application code from dashboard data logic. `dashboard_app/` contains the interactive application, pages, components, static assets, and app-specific configuration. `src/dashboard/` contains reusable backend loading, aggregation, and view-model logic. `src/reporting_cards/` contains card generation, warning summaries, source-record listings, and export utilities.

Dashboard outputs must include source-record IDs, run IDs, config hashes, schema versions, warning flags, and claim-safety tags. The dashboard is an observability and reproducibility layer, not an independent result generator.

Dashboard page or export	Source records	Repository outputs
Project overview and claim-safety	ReproducibilityManifest , warning records, claim-safety configs.	Claim-safety page, warning summary, limitation export.
Dataset and split page	DatasetRecord , WindowRecord , SplitRecord , PreprocessingRecord , LabelMapRecord .	Protocol figures, dataset cards, protocol cards.
Decoder baseline page	PredictionRecord and baseline metric summaries.	Baseline figures and baseline thesis tables.
Calibration and uncertainty page	CalibrationRecord , UncertaintyRecord , SelectivePredictionRecord , ThresholdRegistryRecord .	Reliability diagrams, ECE/Brier summaries, calibration cards.
IHARQ decision-flow page	EvidenceQualityRecord , IHARQDecisionRecord , action-token/display-label mapping, reason-code summaries.	IHARQ flow figures, reliability cards, canonical action-token warning tables, and decision tables.
RegimeRisk timeline page	RegimeStateRecord , conditional TrustStateRecord rows, temporal trust summaries, trusted-set evolution, stop-loss warnings.	Timeline figures, trust-state plots, regime/trust cards, warning reports.
Adaptive policy page	ReadinessContextRecord , DeferalRecord , PolicyActionRecord .	Controller cards, policy comparison tables.
Learning-to-defer page	DeferralRecord , selective-prediction records, policy records.	Matched-coverage and matched-cost comparisons.
Contextual bandit/RL page	PolicyActionRecord , TransitionRecord , optional PolicyUpdateTrace , reward summaries, disabled-update audits, simulation-only tags.	Simulation-only controller cards, policy-update trace cards where applicable, reward-sensitivity figures, and separated training/adaptation versus frozen-policy evaluation curves.
StressForge page	StressProfileRecord and stress-conditioned metric records.	Stress cards, stress robustness plots, ablation tables.

Layer 9 embodiment page	EmbodimentStateRecord, EmbodimentOutcomeRecord, embodiment-linked TransitionRecord, SafetyEventRecord or safety-event log, Layer9RewardTrace where enabled, command timeline, endpoint-error curve, Layer9EmbodimentDemoManifest , mapping registry, simulator-version metadata, screenshots/plots, non-clinical warnings, DashboardMetricRecord lineage, and DashboardViewRecord export metadata.	Embodiment cards, simulated proxy figures, reward-decomposition plots where enabled, demo-manifest index, limitation exports, dashboard-compatible export bundle, and serialized Layer 10 view records.
Ablation and comparison page	Evaluation records, A0–A13 ablation summaries, matched-condition records.	Ablation tables, comparison figures, failure interpretation summaries.
Failure taxonomy page	Warning records, validation reports, failure-mode tags.	Failure cards and defensive-design evidence.
Cards and export page	CardRecords , card registry, thesis-export manifest, optional policy-update trace card registry entries, DashboardMetricRecord index, DashboardViewRecord index, metric-dictionary exports, matched-comparison reports, negative-result note index, and ClaimEvidenceMatrix views.	Static cards, policy-update trace cards where applicable, thesis tables, figure manifests, dashboard metric/view record exports, reproducibility package, and claim-safe export inventory.

cards/ should contain generated dataset cards, protocol cards, reliability cards, calibration cards, IHARQ cards, regime cards, controller cards, policy-update trace cards where applicable, stress cards, embodiment cards, failure cards, and reproducibility cards where applicable. **figures/** and **thesis_tables/** contain curated thesis-ready exports, but they must not become manual, untraceable outputs. Each curated export should include or reference a source manifest listing the records, run IDs, configuration hashes, and warnings from which it was generated.

24.11 Testing, validation, and defensive-design architecture

Testing is part of claim defense. **tests/** must verify not only that code runs, but also that scientific boundaries, split discipline, schema integrity, and provenance are preserved.

Test family	Checks	Failure interpretation
Schema validation tests	Required fields, schema versions, source-record IDs, claim-safety tags, data types, and allowed missingness.	Records are not thesis-grade until schema validation passes.

Dataset adapter smoke tests	Public dataset access, channel naming, label mapping, session/run parsing, and manifest creation.	Dataset results cannot be claimed if adapters are undocumented or unstable.
Split-integrity and leakage tests	Subject/session/run splits, calibration split separation, threshold-selection split separation, and no future-label use.	Leakage invalidates downstream claims.
Preprocessing reproducibility tests	Deterministic preprocessing where expected, saved preprocessing metadata, and compatibility with decoder inputs.	Prediction comparisons are not reproducible.
Decoder output tests	Output shapes, probability vector validity, class ordering, model ID registration, and prediction-record completeness.	Calibration and IHARQ cannot consume the outputs safely.
Calibration tests	Calibration split controls, threshold origin checks, reliability metric consistency, and held-out test separation.	Selective prediction and IHARQ baselines may be contaminated.
Uncertainty sanity tests	Entropy, margin, confidence, disagreement, and stability feature ranges.	Evidence-quality features may be meaningless or inverted.
IHARQ tests	Reason-code coverage, canonical action-token validation, display-label mapping checks, alias rejection, action legality, fallback behavior, unsafe-state behavior, and no future-label inspection.	IHARQ may reduce to opaque thresholding, schema-inconsistent aliasing, or unsafe decisions.
RegimeRisk tests	Timeline ordering, window lengths, reset logic, stop-loss triggers, and feedback-advisory generation.	Temporal trust claims are unsupported.
Policy tests	Legality masks, Layer 4-to-policy action-map validation, pseudo-label construction, reward/cost configurations, split controls, policy-update configuration legality, and evaluation-only label isolation.	Adaptive-IHARQ or RL claims may be invalid or action-token provenance may be lost.
Simulation transition tests	State-action-next-state consistency, reward logging, terminal state handling, simulation-only tags, optional <code>PolicyUpdateTrace</code> completeness, <code>TransitionRecord.policy_update_trace_id</code> linkage, and disabled-update audit consistency.	Closed-loop or adaptive-policy evidence is not reliable.

StressForge tests	Stress profile schema, seed reproducibility, injection point validity, and clean-versus-stressed lineage.	Robustness claims cannot be reproduced.
Embodiment adapter and export tests	Command mapping validity, record-intake validation, readiness-context assembly, simulator-branch isolation, safety gates, safety-event log completeness, reward-trace completeness where enabled, command-timeline consistency, endpoint-error-curve generation, demo-manifest completeness, dashboard-compatible export lineage, non-clinical warnings, and screenshot/plot manifests.	Embodiment output may be overclaimed or untraceable; Layer 10 must hide, warn, or mark incomplete bundles.
Dashboard read-only tests	Dashboard loaders cannot call training, threshold tuning, policy learning, stress generation, simulation commands, or policy-update hooks; policy-update filters may only select already saved records.	Dashboard becomes a hidden experiment engine or hidden update controller.
Card traceability tests	Cards list source records, run IDs, config hashes, warnings, and limitations.	Cards are not acceptable thesis evidence.
Manifest completeness tests	Produced outputs, missing optional outputs, warnings, code version, seeds, configs, and claim tags are recorded.	Results cannot be audited or reproduced.
Reproducibility replay tests	Major experiment phases can be rerun from configs and manifests to regenerate key records or summaries.	Final thesis artifacts are not reproducible.

The repository must fail closed when required identifiers, configs, records, schema versions, split metadata, or claim-boundary metadata are missing. Warnings may be allowed for optional Stretch C outputs, but core protocol, prediction, calibration, IHARQ-lite, RegimeRisk, dashboard, card, ablation, and reproducibility outputs require hard validation.

24.12 Reproducibility package architecture

The repository supports a staged reproducibility package. A minimal reproducibility package should include environment specification, dependency list, dataset access instructions, public-data license notes, configuration files, config hashes, fixed random-seed policy, run manifests, schema definitions, validation reports, generated artifact manifest, card and figure source manifest, and command references for reproducing the major experiments.

The reproducibility package must distinguish required core outputs from optional Stretch C outputs.

- **Core outputs:** public-data protocol records, split records, baseline predictions, calibra-

tion and uncertainty records, IHARQ-lite decisions, RegimeRisk states, evaluation summaries, dashboard-readable records, cards, ablation tables, and manifests.

- **Near-core outputs:** evidence-quality estimator, supervised Adaptive-IHARQ, learning-to-defer comparisons, and policy records where sufficient data and validation exist.
- **Stretch policy-update outputs:** optional `PolicyUpdateTrace` files, disabled-update audit tables, policy-update trace cards, and separated training/adaptation versus frozen-policy evaluation exports for contextual-bandit/RL simulation runs.
- **Stretch C optional outputs:** contextual-bandit experiments, reinforcement-learning policy experiments, StressForge-Lite stress runs, MyoSuite/OpenSim simulated embodiment demo, and advanced dashboard pages.

If maximum Stretch C is only partially completed, the repository must still preserve defensibility by marking optional modules as missing, mocked, partial, or future work in manifests, cards, dashboard warnings, and thesis limitation tables. Missing optional modules must not invalidate lower-level claims if the core record chain is complete.

24.13 Data, large-file, and model-artifact policy

Raw public datasets should generally not be committed directly unless licensing, size, and redistribution rules explicitly permit it. Dataset download instructions, access conditions, fingerprints, versions, local path templates, and preprocessing hashes should be saved in dataset manifests and protocol cards.

Large generated records should use efficient formats such as Parquet where appropriate. Small summaries, registries, and thesis tables may use CSV. Configuration and manifest files may use YAML or JSON. Figures should be exported in thesis-ready PNG or PDF formats, with source manifests. Cards may be Markdown, HTML, or PDF, but their source metadata should also be machine-readable. Logs should be preserved under `logs/` or run-specific log folders and kept separate from curated thesis outputs.

Trained models belong under `models/` or run-specific `runs/<run_id>/models/`. Model artifacts must be registered with model ID, source run, training records, split IDs, configuration hash, seed, code version, metric summary, and claim-safety tags. Policy models, especially contextual-bandit or RL policies, must include simulation-only tags and reward-configuration references.

24.14 Optional and Stretch C extension boundaries

The repository must make clear which components are core and which are optional or advanced. Optional folders may contain placeholders, mocks, minimal adapters, or incomplete implementations, but their status must be explicit in manifests, validation reports, dashboard warnings, cards, and thesis limitation tables.

Capability class	Components	Repository rule
------------------	------------	-----------------

Core or near-core	Public-data protocol, baseline decoders, prediction records, calibration and uncertainty, IHARQ-lite, RegimeRisk, dashboard/cards/reproducibility, ablation reporting.	These components require complete records, schemas, validation reports, and thesis-ready exports.
Adaptive extension	Evidence-quality estimator, supervised Adaptive-IHARQ, learning-to-defer, readiness context construction, deferral records.	May support stronger claims only if pseudo-label construction, cost vectors, split controls, and ablations are documented.
Stretch C advanced policy	Contextual bandit and reinforcement-learning policy.	Must be represented as simulation-only policy learning with declared rewards, baselines, reward ablations, and controller cards.
Stretch C robustness	StressForge-Lite stress runs.	Must preserve clean-versus-stressed lineage and stress-profile reproducibility.
Stretch C embodiment	MyoSuite/OpenSim embodiment demo.	Must remain simulated proxy embodiment with non-clinical warnings in records, cards, dashboard pages, and thesis text.
Future work	Any component without validated records.	May appear as a folder or placeholder only if marked as incomplete and excluded from core claims.

Optional components must not contaminate core claims. For example, failure to complete RL or MyoSuite/OpenSim does not invalidate public-data reliability, IHARQ, RegimeRisk, calibration, dashboard, or ablation claims if those records are complete. Conversely, a screenshot, mock adapter, or placeholder policy cannot be used as evidence of a completed Stretch C module.

24.15 Repository-to-layer and repository-to-module mapping

The following table maps repository paths to the architecture. It is a traceability table, not decorative documentation.

Repository path	Layer	Module	Primary outputs	Tests and dashboard/card linkage
configs/datasets/	1	A	Dataset and protocol configuration.	Split-integrity tests; dataset and protocol cards.

src/data_protocol/ 1	A	DatasetRecord, WindowRecord, SplitRecord, Pre-processingRecord, LabelMapRecord, ArtifactFlagRecord, DatasetCard, ProtocolCard, ValidationReport, Layer1Manifest, and layer1_ablation_readiness_l1_v1.json.	Dataset adapter tests; dataset/split dashboard page; validation/readiness panels; dataset and protocol cards.
records/dataset_records/	A	Public-data protocol	Protocol cards and thesis protocol tables.
src/decoders/ 2	B	PredictionRecord	Decoder output tests; and model checkpoints.
records/prediction_records/	B	Trial/window predictions and probabilities.	Calibration consumers; decoder cards and baseline figures.
src/calibration_uncertainty/	C	CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord.	Calibration split tests; reliability diagrams and calibration cards.
records/calibration_records/	C	Calibrated probability and reliability summaries.	IHARQ consumers; calibration dashboard page.
src/iharq/ 4	D	EvidenceQualityRecord, IHARQDecisionRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, and Layer 4 EvaluationOutcomeRecord.	Reason-code tests; action-token/display-label mapping tests; request/combine, fallback, unsafe-event, ablation, trace-completeness, matched-comparison, IHARQ decision-flow page, and IHARQ reliability-card tests.
records/iharq_records/	D	Canonical accept/wait/request decisions plus evidence-quality, fallback, unsafe-event, ablation, trace, and evaluation-output records.	RegimeRisk, policy, evidence/combine/evidence/failures, dashboard, evaluation, and reproducibility consumers.

src/regime_risk/	5	E	RegimeStateRecord, conditional TrustStateRecord, trust-score tables, regime timelines, stop-loss warnings, and feedback advisories.	Timeline ordering tests; trust-state lineage tests; stop-loss tests; RegimeRisk dashboard page; trust-state plots; regime cards; and trust-state cards where exported.
records/regime_state_records/	5	E	Canonical regime-state records and temporal trust evidence.	IHARQ, policy, simulation, embodiment, dashboard, evaluation, timeline figures, RegimeCards, and warning reports.
records/trust_state_records/	5	E	Conditional per-target trust-state records when independent trust timelines are exported.	Policy context where enabled, Layer 10 trust-state plots, trust-state cards, reproducibility manifests, and optional IHARQ/simulation/embodiment references through linked regime context.
src/adaptive_policy/	6	F	DeferralRecord, PolicyActionRecord, learned policy checkpoints.	Legality-mask and pseudo-label tests; adaptive policy page and controller cards.
records/policy_records/	6	F	Readiness actions and policy comparisons.	Simulation and embodiment consumers; policy ablation tables.
src/simulation/	7	G	TransitionRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, RewardTraceRecord, EnvironmentDiagnosticRecord, SafetyEventRecord, optional PolicyUpdateTrace files, disabled-update audits, and closed-loop cards.	Transition consistency tests; reward-trace tests; environment-diagnostic tests; policy-update trace completeness tests; closed-loop session timeline, policy rollout, reward-decomposition, and simulation-only dashboard pages.

runs/<run_id>/results/layer7_closed_loop/	Run-local configs, sessions, transitions, rewards, policy updates, evaluation, summaries, diagnostics, and cards.	Closed-loop completeness tests; reward-trace tests; environment-diagnostic tests; frozen-policy immutability checks; dashboard session timelines, policy rollout cards, reward-decomposition panels, policy-update trace cards, and separated training/adaptation versus frozen-evaluation exports.
records/transition_records/ G	Sequential consequence evidence, including optional policy_update_trace_id linkage.	Policy evaluation, RL ablations, and policy-update trace cards.
records/closed_loop_session_records/	Session-level closed-loop environment metadata.	Closed-loop session timelines, reproducibility manifests, and closed-loop environment cards.
records/episode_summary_records/ G	Episode-level reward, safety, stress, and policy-family summaries.	A10/A11/A12 evaluation tables and thesis simulation summaries.
records/policy_rollout_records/ G	Policy rollout behavior and action/transition/reward trace pointers.	Policy rollout cards, Layer 6 policy analysis, ablation scripts, and dashboard rollout views.
records/reward_trace_records/ G	Reward and cost decomposition over time.	Reward-decomposition panels, reward-regret figures, policy-update audits, and reward-configuration cards.
records/environment_diagnostic_records/	Invalid-action, failed-transition, missing-link, leakage, reward-anomaly, unmatched-comparison, and disabled-update diagnostic warnings.	Environment-diagnostic warning panels, failure taxonomy, validation reports, and claim-safety review.

records/policy_update_traces/	7	F/G/I	Optional simulation-authorized policy-update evidence.	Policy-update trace cards, disabled-update audits, frozen/update separation exports.
src/stressforge/	8	G	StressProfileRecord, StressScheduleRecord, stressed-record pointers, stress-run manifests, stress-validity reports, stress metrics, Layer 8 evaluation outcomes, matched-stress reports, negative-result notes, claim-evidence entries, and dashboard-ready stress exports.	Stress registry tests; schedule replay tests; matched-stress validation; StressForge page; stress cards; A12 stress ablation tables; and reproducibility manifests.
records/stress_records/	8	G	Canonical robustness-stress evidence, including stress profiles, schedules, stressed-record pointers, run manifests, validity reports, and stress lineage.	Stress ablation tables, stress sensitivity figures, StressCards, matched-stress reports, schedule inspection, and reproducibility manifests.
src/layer9_embodiment/	9	H	EmbodimentStateRecord, EmbodimentOutcomeRecord, and proxy simulator outputs.	Mock adapter tests; embodiment dashboard page and embodiment cards.
records/embodiment/	9	H	Simulated proxy embodiment evidence, including state, outcome, embodiment-linked transition, safety-event-log, reward-trace where enabled, and demo-manifest artifacts.	Non-clinical warnings, embodiment cards, A13 lineage, dashboard embodiment page, and limitation tables.
src/dashboard/	10	I	DashboardMetricRecord and DashboardViewRecord, including optional policy_update_filter, stress_filter, embodiment_filter, and reward_trace_filter support.	Read-only tests; all dashboard pages, including StressForge and embodiment pages; frozen/update separation checks; matched-stress visibility; and export-lineage checks.

<code>src/reporting_cards/</code>	10	I	Generated cards, source-record summaries, stress cards, embodiment cards, and optional policy-update trace cards.	Card traceability tests; stress-card lineage checks; policy-update trace-card checks; card registry and export page.
<code>cards/</code>	10	I	Thesis-ready cards.	Card-source validation and limitation checks.
<code>figures/</code>	10/22	I/evaluation	Thesis-ready plots.	Figure source manifests and dashboard exports.
<code>thesis_tables/</code>	22	Evaluation	Curated comparison, ablation, and limitation tables.	Table-source manifests and reproducibility checks.
<code>logs/</code>	Cross-layer	Cross-layer	Execution, validation, warning, and dashboard logs.	Manifest completeness and failure-taxonomy pages.
<code>tests/</code>	Cross-layer	Cross-layer	Validation evidence.	Defensive-design support and reproducibility replay.

24.16 Module-level repository contracts

Each major module has a repository-level contract.

Module	Bottleneck addressed	Inputs and processing	Outputs and consumers	Failure defenses
Module A: Data protocol	Undocumented public-data use and split ambiguity.	Reads dataset configs, public dataset manifests, preprocessing settings, and split rules; creates typed protocol records.	Dataset, window, split, preprocessing, label-map, artifact-flag, and Layer1 manifest records consumed by decoders and dashboard.	Adapter smoke tests, split-leakage tests, dataset fingerprints, protocol cards.

Module Decoder Baseline Measurement Engine	B: Treating reliability as a single untraceable classifier result.	Trains and evaluates CSP+LDA/SVM, Riemannian, EEGNet, ShallowConvNet, optional compact SSL/foundation-style, and ensemble-control baselines from split-safe records.	PredictionRecord, Output-ModelRegistryRecord, BaselineMetricRecord, LowCalibrationCurveRecord, SubjectProfileRecord, EnsembleControlRecord, FailureCaseIndex, model check-points, ModelCard, BenchmarkCard, and Layer2ReadinessReport.	Output-shape tests, reliability-checks, score-cardna/class-order checks, model registry, baseline tables, ensemble-control checks, and stream-readiness validation.
Module C: Calibration and uncertainty	Accuracy-only evaluation and unsafe confidence interpretation.	Computes calibration, uncertainty features, and selective-prediction baselines from prediction records using declared calibration splits.	Calibration, uncertainty, threshold, and selective-prediction records consumed by IHARQ, RegimeRisk, policy, and evaluation.	Calibration split checks, threshold registry, reliability diagrams, calibration cards.
Module IHARQ	D: Reducing readiness to naive thresholding.	Applies evidence-quality logic, stability checks, reason codes, fallback, and unsafe-state rules.	EvidenceQualityRecord and IHARQDecisionRecord consumed by RegimeRisk, policy, simulation, dashboard, and cards.	Reason-code coverage, matched threshold baselines, no future-label checks.
Module RegimeRisk	E: Ignoring temporal trust degradation.	Builds temporal trust state from ordered predictions, uncertainty, IHARQ decisions, and declared windows.	RegimeStateRecord, Timeline or-conditional Trust-StateRecord, trust timelines, stop-loss warnings, feedback advisories.	Ordering tests, trust-state lineage tests, reset tests, stop-loss tests, warning logs.

Module Adaptive policy	F:	Hand-coded readiness policy may be suboptimal or arbitrary.	Learns or compares supervised, deferral, contextual-bandit, and RL policies from readiness contexts, costs, rewards, and simulation records under split control.	PolicyActionRecord, Legality masks, policy models, pseudo-label controller cards, reports, reward ablations, policy ablations, simulation-only tags.
Module Simulation and StressForge	G:	No way to study sequential consequences or robustness stress under public EEG constraints.	Simulates transitions, emits closed-loop session, episode-summary, policy-rollout, reward-trace, environment-diagnostic, safety-event, and optional PolicyUpdate-Trace files for declared policy-update runs, and injects declared stress profiles without overwriting clean records.	TransitionRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, RewardTraceRecord, EnvironmentDiagnosticRecord, optional PolicyUpdate-Trace, StressProfileRecord, stress-conditioned metrics, simulation summaries, and closed-loop cards.
Module Layer 9 embodiment adapters	H:	No visible action-consequence bridge for readiness decisions.	Runs the Layer 9-owned L9.1–L9.13 subsystem: validates records, assembles readiness context, maps readiness actions to MyoSuite/OpenSim/safety replay proxy commands through safety gates and declared mappings, logs outcomes, and packages dashboard-compatible exports.	EmbodimentStateRecord, Adapter/export tests, mapping registry, safety-event log checks, demo-manifest checks, non-clinical warnings, optional E0/E1 fallback. Layer9RewardTrace where enabled, command time-lines, endpoint-error curves, Layer9EmbodimentDemoManifest files, screen-shots/plots, proxy trajectories, embodiment-card inputs.

Module Dashboard and reporting	I: Scientific outputs may become hard to audit or repro- duce.	Loads saved records, op- tional PolicyUp- dateTrace files, disabled-update audits, Layer 9 em- bodiment bundles, validates manifests, aggregates metrics with optional pol- icy_update_filter, embodi- ment_filter, and re- ward_trace_filter, renders pages, and generates cards.	Dashboard views, static exports, card cards, policy- update trace cards, Layer 9 embodiment evidence cards, separated train- ing/adaptation versus frozen- policy evaluation exports, thesis tables, figures, reproducibility package.	Read-only tests, card trace- ability tests, policy-update trace com- pleteness tests, frozen/update separation checks, Layer 9 bundle com- pleteness checks, source mani- fests, warning summaries.
--------------------------------------	---	--	--	--

For each module, evaluation is performed by checking whether the module produces valid records, whether its downstream consumers can load those records without hidden assumptions, whether its outputs improve or clarify thesis evidence in ablations, and whether its failure modes are visible in validation reports or cards. The main limitation is that repository correctness does not prove scientific success by itself; it proves that scientific success or failure can be audited.

24.17 Feedforward and loopback behavior in repository terms

The repository implements feedforward behavior through immutable source-record references. A downstream record should point to upstream `source_record_ids`, not copy ambiguous state. The default feedforward sequence is:

Dataset/Window/Split → Prediction → Calibration/Uncertainty → IHARQ → RegimeRisk/Policy → Simulation

Loopbacks are allowed only when declared, logged, and scientifically justified.

Source	Target	Signal	Trigger	Safety constraint and evaluation
RegimeRisk	IHARQ	Temporal trust advisory, trust-state advisory where exported, or stop-loss warning.	Drift-like, trust-degradation, or volatility-like degradation crosses configured thresholds.	IHARQ may become more conservative but must log the RegimeStateRecord ID and any linked TrustStateRecord ID used for the advisory; evaluate by stress-conditioned risk-coverage and accepted-trial accuracy.

Simulation	Policy learning	TransitionRecord, RewardTraceRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, EnvironmentDiagnosticRecord, SafetyEventRecord update mode, where materialized, optional PolicyUpdateTrace files, and disabled-update audits.	Policy-training or simulation-coordination experiment explicitly declares simulation-condition update mode, or frozen evaluation explicitly declares updates disabled.	Rewards cannot relabel held-out public test data; policy mutation must be traceable; environment diagnostics must be surfaced; evaluate by held-out sessions, reward ablations, simulation-only controller cards, policy rollout cards, reward-configuration cards, and frozen/update separation exports.
StressForge	Evaluation and policy robustness	Stress profile IDs and stressed-derived records.	Robustness experiment declares stress family, intensity, seed, and injection point.	Clean records remain immutable; evaluate clean-vs-stressed degradation and matched-condition ablations.
Layer 9 embodiment demo	Layer 10 dashboard/cards	Proxy state, outcome record, embodiment-linked transition pointer, safety-event log, Layer9RewardTrace where enabled, command timeline, endpoint-error curve, Layer9EmbodimentDemoManifest, screenshots/plots, command-mapping outcome, non-clinical warning.	Embodiment replay, static replay, MyoSuite run, or optional OpenSim replay completes.	No feedback into decoder training or calibration; evaluate as simulated proxy observability only, and Layer 10 must remain read-only over saved Layer 9 outputs.
Validation	All modules	Missingness, schema, leakage, warning, and claim-boundary failures.	Any command attempts to consume invalid artifacts.	Fail closed for core artifacts; optional modules may be marked incomplete but cannot support claims.

No arbitrary loopback is allowed. In particular, dashboard outputs must not feed back into model training; embodiment screenshots must not justify readiness thresholds; test labels must not guide calibration or policy action selection; and downstream stress results must not mutate upstream clean prediction records.

24.18 Evaluation design for the repository layer

This section is evaluated by whether the repository can support the thesis evidence chain without hidden work. The repository layer is successful if:

- every core layer has a source package, configuration family, record family, validation test, dashboard/card linkage, and thesis export location;
- every major record can be traced from source configs to produced artifacts through a manifest;
- ablations can be regenerated from saved records rather than manually copied tables;
- dashboard pages can be built from saved records without recomputing experimental logic;
- Layer 7 transition, closed-loop session, episode-summary, policy-rollout, reward-trace, environment-diagnostic, safety-event, optional `PolicyUpdateTrace` records, disabled-update audits, and `DashboardMetricRecord.policy_update_filter` values can be traced through dashboard pages, cards, manifests, and separated training/adaptation versus frozen-policy evaluation exports;
- simulation-only and non-clinical warnings appear in policy, stress, embodiment, dashboard, card, and thesis exports;
- optional Stretch C modules can be missing, partial, or mocked without contaminating core claims;
- reproducibility replay can regenerate the main protocol, prediction, calibration, IHARQ, RegimeRisk, ablation, card, and dashboard artifacts.

Repository-layer baselines include a single-notebook implementation, a flat script folder, dashboard-generated figures without manifests, and model outputs saved without schema or source-record IDs. The expected improvement is not higher classification accuracy; it is stronger traceability, modularity, leakage defense, reproducibility, and claim-safety. Failure of this layer means that thesis evidence may be scientifically interesting but not auditable.

24.19 Repository-level failure modes and safeguards

Failure mode	Repository-level safeguard
The project becomes a single notebook or single classifier script.	Enforce <code>src/</code> module ownership, command wrappers, records, manifests, and dashboard read-only tests.
Records are overwritten without versioning.	Use run IDs, config hashes, schema versions, immutable source-record IDs, and run-specific output folders.
Generated figures cannot be traced to source records.	Require figure source manifests and thesis-export manifests.
Dashboard recomputes results silently.	Add dashboard read-only tests and restrict dashboard code to loading saved records and manifests; policy-update filters may select but never mutate saved traces.

Policy learning reads evaluation-only labels.	Require split-control validation, pseudo-label construction reports, policy legality tests, policy-update configuration checks, and frozen-evaluation lockout tests.
Calibration or threshold selection uses held-out test data.	Store threshold origins in <code>ThresholdRegistryRecord</code> and fail validation on test-set threshold tuning.
StressForge profiles are undocumented or changed after results.	Use stress profile registry with seed, intensity, schedule, injection point, and config hash.
Layer 9	Require embodiment mapping registry, safety-event log, command timeline, endpoint-error curve, <code>Layer9RewardTrace</code> where enabled, <code>Layer9EmbodimentDemoManifest</code> , dashboard-compatible export, simulation-only tags, non-clinical warning templates, embodiment cards, and dashboard warning summaries.
MyoSuite/OpenSim/static-replay outputs are presented without complete bundle or warnings.	
Optional modules create the appearance of completion.	Mark optional outputs as complete, partial, mocked, missing, or future work in manifests and cards.
Manual thesis tables diverge from saved records.	Generate thesis tables from <code>src/evaluation/</code> and store source manifests under <code>thesis_tables/</code> .
Logs and manifests are incomplete.	Run manifest completeness tests and fail closed for core artifacts.
Large artifacts are committed without policy.	Keep raw datasets out of version control unless allowed; store fingerprints and access instructions; register large models and records.
Claim boundaries disappear during export.	Use claim-safety configs, warning records, card limitation sections, dashboard warning panels, policy-update trace cards, disabled-update audits, and frozen/update curve-separation manifests.

These safeguards connect directly to the defensive-design logic of the Failure Modes and Defensive Design section. The repository is itself a defensive layer: it makes architectural failure visible.

24.20 Academic and claim-safety boundaries

The repository organization must not imply real clinical deployment, real-time medical-device control, patient-safe stimulation, real rehabilitation validation, real implanted BCI control, real-world FES validation, unsupervised clinical safety, validated fatigue diagnosis, or clinical biomechanics proof from MyoSuite/OpenSim.

The following boundary rules are mandatory:

- Public EEG experiments are offline or pseudo-online computational benchmarks.
- Adaptive-IHARQ labels are computational readiness labels derived from correctness, uncertainty, future-window evidence where allowed for analysis, fallback availability, declared costs, deferral comparisons, and simulation rewards. They are not clinical action labels.
- Contextual-bandit and reinforcement-learning policies are simulation-only policy-learning branches unless future real-world validation is separately performed, and their repository evidence must preserve closed-loop sessions, policy rollouts, reward traces, environment diagnostics, optional update traces, and frozen-policy separation.

- StressForge-Lite creates stress-conditioned robustness evidence, not proof of clinical robustness.
- MyoSuite/OpenSim is a simulated proxy embodiment bridge, not proof of rehabilitation efficacy or safe biomechanical control.
- The dashboard is observability and reproducibility infrastructure, not an independent source of scientific results.

These boundaries must be encoded in `configs/claim_safety/`, record metadata, warning logs, dashboard warning summaries, card limitation sections, reproducibility manifests, and thesis limitation tables.

24.21 Layers 4–7 evaluation-lineage repository addendum

The repository contract for the MERGE_RECORD_22 Layers 4–7 propagation is narrow. It does not create a new runtime layer and it does not replace the Ablation and Evaluation Architecture as the cross-cutting evaluation register. It requires the repository to preserve local evaluation ownership for IHARQ, RegimeRisk, adaptive policy, and closed-loop simulation while keeping their artifacts reusable by the global A0–A13 evaluation scripts and by Layer 10.

Recommended canonical homes are:

- `records/evaluation_outcome_records/layer4_iharq/`;
- `records/evaluation_outcome_records/layer5_regimerisk/`;
- `records/evaluation_outcome_records/layer6_policy/`;
- `records/evaluation_outcome_records/layer7_simulation/`;
- `reports/ablation_summaries/layers4_7_evaluation_lineage/`;
- `figures/ablations/layer4_iharq/`, `figures/ablations/layer5_regimerisk/`, `figures/ablations/layer6_policy/`, and `figures/ablations/layer7_simulation/`;
- `thesis_tables/ablation_tables/layers4_7_evaluation_lineage/`;
- `manifests/layers4_7_evaluation_lineage_manifest.yaml`.

The manifest indexes local/global ablation IDs, source-record IDs, matched-comparison report paths, metric-dictionary IDs or snapshots, negative-result-note paths, dashboard/card/export IDs, claim-evidence entries, and diagnostic-only flags. No second folder tree should be created for the same concept. Local Layer 4–7 evaluation artifacts may be consumed by `src/evaluation/`, `src/dashboard/`, `src/reporting_cards/`, and final thesis-export scripts, but the local owning layer remains the producer of the nearest evaluation record. If a matched comparison fails, the repository must preserve the artifact with a diagnostic-only status rather than deleting it or moving it into an untracked notes folder.

24.22 Layer 7 policy-update repository addendum

The repository contract for Layer 7 policy-update propagation is narrow. Simulation runs that enable contextual-bandit/RL updates must write run-local update traces under `runs/<run_id>/results/layer7_closed_loop/policy_updates/policy_update_traces.parquet`. Frozen-policy evaluation runs that explicitly audit disabled updates should write `runs/<run_id>/results/layer7_closed_loop/policy_updates/update_disabled_audit.csv`. Promoted or cross-run update traces may also be copied into `records/policy_update_traces/`, but promotion must preserve run ID, simulator ID, seed, configuration hash, frozen-policy flag, and source `TransitionRecord.policy_update_trace_id` linkage.

No second repository tree should be created for the same concept. `records/policy_update_traces/`, `logs/policy_update_audits/`, `cards/policy_update_trace_cards/`, and the run-local `layer7_closed_loop/policy_updates/` folder are the canonical homes for this propagation.

24.23 Layers 8–10 evaluation-lineage repository addendum

The repository contract for the MERGE_RECORD_22 Layers 8–10 propagation is narrow. It does not create a new runtime layer and it does not replace the Ablation and Evaluation Architecture as the cross-cutting evaluation register. It requires the repository to preserve local evaluation ownership for StressForge robustness and Layer 9 embodiment-proxy consequences while keeping their artifacts reusable by the global A0–A13 evaluation scripts, Layer 10, the failure-mode register, and the final demonstration matrix.

The canonical homes for the propagated artifacts are:

- `records/evaluation_outcomes/layer8_stress_evaluation_outcomes.parquet`;
- `reports/matched_comparisons/layer8_matched_stress_comparisons.csv`;
- `metrics/metric_dictionary_layer8_stress.yaml`;
- `notes/negative_results/layer8_stress_negative_results.md`;
- `manifests/claim_evidence/layer8_stress_claim_evidence.json`;
- `records/evaluation_outcomes/layer9_embodiment_evaluation_outcomes.parquet`;
- `reports/matched_comparisons/layer9_matched_embodiment_comparisons.csv`;
- `metrics/metric_dictionary_layer9_embodiment.yaml`;
- `notes/negative_results/layer9_embodiment_negative_results.md`;
- `manifests/claim_evidence/layer9_embodiment_claim_evidence.json`;
- `records/dashboard_metric_records/` and `runs/<run_id>/dashboard_exports/dashboard_metric_r`
- `records/dashboard_view_records/` and `runs/<run_id>/dashboard_exports/dashboard_view_recon`
- `runs/<run_id>/dashboard_exports/figure_table_manifest.json`.

No second folder tree should be created for the same concept. Layer 8 and Layer 9 evaluation artifacts may be consumed by `src/evaluation/`, `src/dashboard/`, `src/reporting_cards/`,

repository validators, final thesis-export scripts, and reproducibility packaging, but the local owning layer remains the producer of the nearest evaluation record. If a matched comparison fails, the repository must preserve the artifact with diagnostic-only status and a negative-result note rather than deleting it or moving it into an untracked notes folder.

The Layer 10 dashboard/export side may create derived `DashboardMetricRecords`, `DashboardViewRecords`, figures, cards, warning summaries, export manifests, and reproducibility indexes from these files, but those derived artifacts must point back to the canonical Layer 8/Layer 9 paths above and to the metric-dictionary, matched-comparison, negative-result, and claim-evidence artifacts used for the exported view. A repository implementation that stores A12/A13 evidence only in rendered dashboard pages, manually edited thesis tables, or unnamed ad-hoc CSVs is inconsistent with this architecture.

24.24 Consistency checklist

A repository implementation is consistent with the architecture only if the following conditions hold.

- The project name remains IHARQ BenchGuard and Stretch C remains the maximum scope.
- Layer numbering remains consistent with the thesis document.
- Modules A–I retain their existing meanings.
- Decoders, calibration, IHARQ, RegimeRisk, Adaptive-IHARQ, learning-to-defer, contextual bandit, RL policy, StressForge-Lite, MyoSuite/OpenSim, dashboard, cards, ablations, and reproducibility remain separate modules.
- Layer 8 A12 stress evaluation files and Layer 9 A13 embodiment evaluation files live in the canonical evaluation-outcome, matched-comparison, metric, negative-result, and claim-evidence locations rather than being hidden in dashboard exports or manually edited thesis tables.
- Public-data and low-calibration EEG remain the experimental foundation.
- Canonical records are stored with schema versions, run IDs, config hashes, seeds, source-record IDs, and claim-safety tags.
- Dashboard pages and cards read saved records and manifests.
- Where optional policy-update traces exist, `records/policy_update_traces/`, `cards/policy_update_trace_cards/`, `logs/policy_update_audits/`, and `runs/<run_id>/results/layer7_closed_loop/policy_updates/` are cross-referenced rather than treated as unrelated outputs.
- Ablation outputs are traceable to saved records and matched configurations.
- Optional Stretch C modules are marked honestly as complete, partial, mocked, missing, or future work.
- Simulation-only and non-clinical boundaries are visible in records, cards, dashboard pages, reports, and thesis tables.
- No downstream module mutates upstream public-data, split, prediction, or calibration records.

- No thesis table, figure, card, or dashboard export is accepted without a source manifest.

Therefore, the recommended repository architecture is the practical enforcement mechanism for the full IHARQ BenchGuard architecture. It operationalizes the public-data protocol, decoder spine, calibration and uncertainty layer, IHARQ evidence verification, RegimeRisk temporal trust, Adaptive-IHARQ and deferral policies, contextual-bandit and RL simulation, StressForge-Lite robustness testing, MyoSuite/OpenSim simulated embodiment, dashboard, cards, ablation, failure taxonomy, and reproducibility package while preserving the claim boundaries required for a defensible computer-engineering thesis.

25 Failure Modes and Defensive Design

25.1 Purpose and architectural role

This section is the system-level failure-mode and defensive-design register for IHARQ BenchGuard under Stretch C. It is not a cosmetic list of risks. It is the final architectural audit layer that explains how the full record-first functional graph can fail, how each failure is detected, which defensive controls prevent or mitigate it, which records prove that the defense was applied, which dashboard or card warning exposes the issue, and which thesis claim becomes weakened, restricted, blocked, or invalidated when the failure is not controlled.

The bottleneck addressed by this section is not classification accuracy. The bottleneck is defensibility under complexity. Stretch C combines public EEG benchmarking, calibration, selective prediction, IHARQ evidence verification, RegimeRisk temporal trust, Adaptive-IHARQ, learning-to-defer, contextual bandit and reinforcement-learning branches, StressForge-Lite, optional MyoSuite/OpenSim simulated embodiment, dashboard pages, cards, ablations, and reproducibility manifests. Without a defensive-design section, the architecture could become impressive but scientifically fragile: thresholds could leak, dashboard figures could lose lineage, simulated policies could be overclaimed, pseudo-labels could be mistaken for clinical labels, or optional embodiment failures could appear to invalidate the core EEG reliability contribution.

After the Layer 7 and Layer 10 propagation updates, the defensive register also treats policy-update traceability as a global safety condition. Unlogged policy mutation, missing disabled-update audits, missing `TransitionRecord.policy_update_trace_id` linkage, hidden or incomplete policy-update trace cards, and mixed simulation-training/adaptation versus frozen-policy evaluation curves are global failure modes because they can invalidate both adaptive-learning claims and frozen-policy comparison claims.

After the Layer 20-to-Layer 9 merge and the subsequent Layer 10, cross-layer, and proximity-map propagations, the defensive register also treats Layer 9 embodiment-bundle completeness as a global safety condition for any C4/A13 embodiment claim. Unsupported command mappings, bypassed readiness gates, missing safety-event logs, missing reward traces when reward-enabled embodiment is claimed, missing command timelines, missing endpoint-error curves, missing `Layer9EmbodimentDemoManifest` files, stale simulator metadata, dashboard recomputation of embodiment evidence, and reward-trace overclaiming are failure modes because they can turn a bounded proxy demonstration into an unsupported clinical or biomechanical claim.

After the `MERGE_RECORD_22` Layers 0–3 evaluation-slice merge and its first propagation pass, the defensive register also treats evaluation-artifact traceability as a global safety condition for any A0–A4 claim. Missing claim-evidence matrix entries, missing metric-dictionary definitions, incomplete Layer 1 ablation-readiness reports, unmatched comparison keys, missing Layer 3 matched operating-point tables, hidden threshold-sufficiency or calibration-failure notes,

and dashboard/card/export lineage gaps are failure modes because they can turn a diagnostic public-data result into an unsupported thesis improvement claim.

The scientific role of this section is to protect the thesis claim boundary. The architecture may support claims about computational reliability evaluation, evidence verification, calibrated uncertainty visibility, temporal trust monitoring, adaptive readiness decisions, stress-conditioned robustness analysis, simulation-only policy learning, simulated proxy embodiment, and reproducible dashboard inspection. It must not claim clinical validation, patient safety, real rehabilitation efficacy, real FES safety, real-time medical-device deployment, implanted BCI control, or real-world therapeutic benefit. When any artifact invites one of those unsupported interpretations, the defensive design must block the claim, downgrade the result, or force explicit limitation language.

The engineering role of this section is to define fail-closed quality gates, record validation rules, warning propagation, fallback statuses, repair actions, and test requirements. If a required record is missing, invalid, leaked, untraceable, or outside scope, the system must not silently continue as though the evidence were thesis-ready. It must either stop the run, mark the component as non-thesis-ready, downgrade the supported claim, or route the result into an explicit diagnostic, optional-only, or future-work status.

If this section were removed, the architecture would lose its final audit mechanism. The implementation pipeline would still describe how to build modules, and the repository architecture would still describe where to place them, but there would be no systematic mapping from failure to detection, defense, artifact, dashboard warning, ablation impact, claim consequence, and repair. That absence would expose the thesis to criticism that IHARQ BenchGuard is overcomplex, that IHARQ is merely thresholding, that Adaptive-IHARQ depends on artificial labels, that RL overfits a simulator, that MyoSuite/OpenSim is overclaimed, and that the dashboard is decorative rather than scientific.

25.2 Layer boundaries and internal responsibility

The defensive-design layer receives artifacts from the preceding architecture and produces audit artifacts for the final demonstration, final summary, dashboard, cards, repository tests, thesis methods, thesis results, and thesis limitations. It does not train decoders, tune thresholds, estimate RegimeRisk, learn policies, inject stress, run MyoSuite/OpenSim, or render new experimental results. Its responsibility is to validate whether those modules produced traceable, claim-safe, reproducible evidence.

Boundary item	In scope	Out of scope
Technical responsibility	Failure taxonomy, defensive-control matrix, quality gates, claim-impact analysis, warning requirements, fallback statuses, audit checklists, repair actions, and negative-result interpretation.	Replacing the decoder, changing the scientific protocol after results, manually correcting figures, or using dashboard outputs as new experimental evidence.

Core submodules	Failure taxonomy register, quality-gate engine, claim-impact checker, dashboard/card warning map, reproducibility audit checklist, negative-result guide, and implementation-phase risk map.	A separate deployment safety case, clinical hazard analysis, patient-facing control policy, or medical-device risk management file.
Supporting submodules	Repository-test linkage, repair-action table, optional-component fallback map, limitation-language map, and ablation-impact map.	Creating unsupported clinical, therapeutic, or real-world deployment claims.
Optional or stretch submodules	Automated warning aggregation, dashboard-level failure taxonomy page, card completeness scoring, and cross-run failure trend summaries.	Treating automated warnings as a substitute for thesis interpretation.
Expected artifacts	SystemFailureModeRegister, DefensiveDesignMatrix, ClaimImpactMatrix, QualityGateChecklist, RepairActionTable, ReproducibilityAuditChecklist, and failure/cards/dashboard warning records.	Hidden spreadsheet-only notes, undocumented manual edits, or unversioned reviewer-facing claims.

25.3 Relationship to the full architecture

This section consumes the full architecture but does not duplicate it. It cross-connects the layer stack, the cross-layer module view, the detailed input-output proximity map, the A0–A13 ablation ladder, the implementation pipeline, the recommended repository architecture, the dashboard/card layer, and the final demonstration expectations.

Architecture source	What this section consumes	Defensive-design use
Layer 0: Claim-Safety and Scope Governance	Claim ledger, allowed/prohibited claim classes, public-data and simulation-only boundary rules.	Blocks clinical, therapeutic, patient-safety, medical-device, implanted-control, or real-world deployment claims.
Layer 1: Public-Data and Protocol Anchor	DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, DatasetCard, ProtocolCard, ValidationReport, Layer1Manifest, manifests/layer1_ablation_readiness_l1_v1.json, dataset cards, and protocol cards.	Detects protocol leakage, split drift, hidden normalization, label-map inconsistency, missing validation/readiness status, incomplete matched-comparison keys, and public-data overclaiming.

Layer 2: Decoder and Baseline Measurement Spine	PredictionRecord, model registry entries, baseline outputs, and subject/session metrics.	Detects weak or untraceable decoders, missing baselines, inconsistent score fields, and unsupported downstream claims.
Layer 3: Calibration, Uncertainty, and Selective Prediction	CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, reliability diagrams, ECE, Brier, NLL, risk-coverage curves, and high-confidence-wrong audits.	Detects calibration leakage, uncalibrated confidence, test-set threshold tuning, unmatched selective-prediction baselines, and hidden high-confidence wrong cases.
Layer 4: IHARQ Evidence Verification	EvidenceQualityRecord, IHARQDecisionRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, Layer 4 EvaluationOutcomeRecord, reason-code logs, fallback logs, unsafe-event logs, and IHARQ traces where implemented.	Detects collapse into simple thresholding, missing reason codes, absent evidence requests or combine traces, fallback-route gaps, unsafe-state inconsistency, trace incompleteness, and missing matched-cost comparisons.
Layer 5: RegimeRisk Temporal Trust Monitoring	RegimeStateRecord, conditional TrustStateRecord, trust summaries, temporal windows, stop-loss logs, regime cards, trust-state cards where exported, and regime timelines.	Detects future-information leakage, non-causal temporal features, proxy-label overclaiming, dangling trust-state links, and unused temporal trust signals.
Layer 6: Adaptive Readiness Policy Layer	ReadinessContextRecord, DeferralRecord, PolicyActionRecord, policy registry, cost configs, reward configs, and target-construction cards.	Detects artificial-label overclaiming, evaluation-only field leakage, cost opacity, legality-mask errors, missing baselines, and policy reason-code gaps.
Layer 7: Simulated Closed-Loop Readiness Environment	TransitionRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, RewardTraceRecord, EnvironmentDiagnosticRecord, SafetyEventRecord where materialized, optional PolicyUpdateTrace, disabled-update audit entries, and simulation manifests.	Detects offline/closed-loop confusion, incomplete transition logs, inconsistent reward/done logic, invalid actions, failed transitions, reward anomalies, missing safety-event or reward-trace links, unlogged policy mutation, mixed training/adaptation versus frozen-evaluation evidence, unsafe action execution, and missing simulation baselines.

Layer 8: StressForge-Lite Stress Generator	StressProfileRecord , StressScheduleRecord where present, stress configs, stress cards, and clean-vs-stressed record links.	Detects undocumented stress profiles, unpaired stress records, silent intensity changes, incompatible injection points, and clinical overinterpretation of synthetic stress.
Layer 9: MyoSuite/OpenSim Embodiment Demo	EmbodimentStateRecord , EmbodimentOutcomeRecord for executed embodiment modes, or accepted-missingness status where absent, embodiment-linked TransitionRecord , SafetyEventRecord or safety-event logs, Layer9RewardTrace where enabled, command timelines, endpoint-error curves, Layer9EmbodimentDemoManifest , missing simulation-only warn-adapter logs, safety-gate logs, command-mapping configs, and embodiment cards.	Detects overclaiming, arbitrary command mapping, bypassed readiness gates, unsupported command execution, missing or stale simulator metadata, missing safety-event logs, missing Layer9EmbodimentDemoManifest files, reward-trace overclaiming, adapter crashes, and missing simulation-only warnings.
Layer 10: Dashboard, Cards, and Reproducibility Layer	DashboardMetricRecord , DashboardMetricRecord.policy_update_history , DashboardMetricRecord.embodiments_filter , DashboardMetricRecord.reward_trace_filter , DashboardViewRecord , MetricDictionaryEntry references, MatchedComparisonReport references, NegativeResultNote references, ClaimEvidenceMatrix linkage, CardRecord , policy-update trace cards, disabled-update audit tables, Layer 9 embodiment evidence cards, export manifests, warning logs, and ReproducibilityManifest .	Detects cosmetic dashboard-update, hidden re-computation, missing lineage, missing primary or matched-comparison linkage, hidden diagnostic-only downgrades, missing negative-result visibility, missing claim-evidence linkage, mixed policy-update/evaluation curves, incomplete Layer 9 embodiment bundles, card-record conflict, incomplete manifests, and untraceable thesis figures.
A0–A13 ablation ladder	Matched baselines, ablation outputs, negative-result notes, and evaluation summaries.	Links failures to specific evidence gaps and prevents unsupported comparative claims.
Implementation pipeline and repository architecture	Phase outputs, folder ownership, configs, tests, records, cards, logs, figures, tables, and manifests.	Defines the minimum defensive artifact required before each phase can advance.

Downstream, this section requires or produces the following artifacts:

SystemFailureModeRegister ,	DefensiveDesignMatrix ,
FailureTaxonomyCard ,	BoundaryViolationChecklist ,
QualityGateChecklist ,	NegativeResultInterpretationGuide ,
DashboardWarningRequirements ,	CardWarningRequirements ,

`ReproducibilityAuditChecklist`, `ImplementationPhaseRiskMap`, `OptionalComponentFallbackMap`, and `ThesisLimitationLanguageMap`. Their consumers are the thesis methods chapter, results chapter, discussion and limitations chapter, dashboard pages, card exports, repository tests, implementation pipeline, final demonstration, and reproducibility package.

25.4 Stakeholders and inspection questions

The defensive-design layer serves six primary readers.

- The thesis author uses it to decide what must be fixed before a result can support a claim.
- The supervisor uses it to distinguish core, target, stretch, optional, diagnostic, and future-work components.
- The examiner or reviewer uses it to verify that negative cases, leakage risks, simulation limits, and overclaim risks are visible rather than hidden.
- The implementer uses it to determine which assertions, schema checks, tests, warning records, manifests, fallback branches, and command-line gates must be built.
- The dashboard user uses it to inspect whether a figure, card, or table is backed by saved records and appropriate limitations.
- The future reproducer uses it to find the records, configs, seeds, hashes, schema versions, code versions, and warning logs needed to reproduce or audit the result.

The minimum inspection questions are as follows:

- Was threshold selection isolated from the held-out test set?
- Was IHARQ compared against confidence thresholding, entropy rejection, margin rejection, selective prediction, longer windows, multi-window voting, probability averaging, and ensembles at matched coverage or matched cost?
- Were evaluation-only fields excluded from decision-time policy contexts?
- Was `RegimeRisk` temporally causal and based only on current or past evidence?
- Were Adaptive-IHARQ labels declared as computational readiness pseudo-labels rather than clinical action labels?
- Were contextual-bandit and RL rewards frozen, versioned, and tested on held-out simulated sessions?
- Were stress profiles versioned, seeded, hashed, and paired with clean source records?
- Did the MyoSuite/OpenSim adapter enforce readiness gates and block unsupported commands?
- Did the dashboard read only saved records and manifests?
- Did every thesis figure, table, dashboard view, and card preserve source-record lineage?
- Was every clinical, deployment, or rehabilitation overclaim blocked or visibly warned?

25.5 Input contract

Part 25 consumes multiple input classes. Each input must carry run identifiers, split identifiers where relevant, schema versions, config hashes, source-record identifiers, and claim-safety tags when applicable. Missingness is not interpreted as neutral; it is interpreted according to the gate status of the dependent claim.

Input class	Semantic role	Validity constraint	con-	Missingness behavior
Claim-safety puts	in-Layer 0 claim ledger, allowed/prohibited claim classes, limitation templates, <code>ClaimEvidenceMatrix</code> , and warning-inheritance rules.	Must be versioned and attached to cards, dashboard views, ablation reports, metric exports, negative-result notes, and thesis exports.		Missing claim ledger or claim-evidence matrix blocks final claim export and marks the result non-thesis-ready.
Evaluation-artifact inputs	Metric dictionary entries, matched-comparison reports, A0–A4 ablation IDs, dashboard/card/export lineage, diagnostic-only flags, and <code>NegativeResultNote</code> entries.	Must reference source records, metric definitions, comparison keys, and Layer 0 claim boundaries.		Missing evaluation-artifact lineage downgrades the result to diagnostic-only or non-thesis-ready.
Protocol inputs	<code>DatasetRecord</code> , <code>WindowRecord</code> , <code>SplitRecord</code> , <code>PreprocessingRecord</code> , <code>LabelMapRecord</code> , <code>ArtifactFlagRecord</code> , <code>Layer1Manifest</code> , and <code>manifests/layer1_ablation_readiness_l1_v1.json</code>	Splits, windows, preprocessing, label maps, artifact flags, and matched-comparison keys must be traceable and non-leaking.		Missing protocol inputs block all benchmark, calibration, IHARQ, policy, and stress claims; missing ablation-readiness status blocks thesis-ready A0–A4 comparison claims.
Decoder inputs	<code>PredictionRecord</code> , model registry, baseline outputs.	Predictions must reference model, dataset, subject/session split, window, and score fields.		Missing decoder inputs block all downstream readiness evidence.

Calibration and selective-prediction inputs	CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, Layer3MatchedOperatingPoint, high-confidence-wrong audit, and negative-result notes.	Calibration and threshold origins must exclude test-set tuning unless explicitly labeled diagnostic; matched operating points must preserve coverage/cost comparison keys.	Missing or leaked calibration blocks calibrated-reliability claims and narrows IHARQ claims; missing matched operating points blocks A1–A3 thesis-ready improvement claims.
IHARQ inputs	EvidenceQualityRecord, IHARQDecisionRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, Layer 4 EvaluationOutcomeRecord, reason-code logs, fallback logs, unsafe-event logs, and trace records where present.	Decisions must include canonical action token, display-label mapping where used, reason code, evidence source, threshold or rule version, fallback/unsafe/uncertain status, trace pointer, and source prediction IDs.	Missing reason codes, trace IDs, action-token validation, or required fallback/unsafe/ablation lineage blocks the claim that IHARQ is auditable evidence verification.
RegimeRisk inputs	RegimeStateRecord, conditional TrustStateRecord, trust-state summaries, temporal windows, stop-loss logs, regime cards.	Temporal and trust-state features must be causal and must not use future labels or future predictions at decision time.	Missing RegimeRisk records remove temporal-trust claims; missing independent trust-state rows remove per-target trust-timeline claims but need not invalidate core IHARQ-lite claims.
Policy inputs	ReadinessContextRecord, DeferralRecord, PolicyActionRecord, policy registry, cost configs, reward configs, target-construction cards, optional policy-update configs, and optional PolicyUpdateTrace where simulation updates are enabled.	Contexts must exclude evaluation-only fields; legality masks, action sets, cost vectors, pseudo-label construction rules, update mode, and frozen-evaluation status must be frozen and logged.	Missing policy inputs block Adaptive-IHARQ, learning-to-defer, bandit, and RL claims but not core decoder/calibration evidence; missing update traces invalidate update-enabled bandit/RL claims.

Simulation inputs	TransitionRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, RewardTraceRecord, EnvironmentDiagnosticRecord, SafetyEventRecord where material-ized, optional PolicyUpdateTrace, disabled-update audit entries, and simulation manifests where implemented.	Transition state, action, reward, done, safety events, reset logic, closed-loop session metadata, rollout lineage, environment diagnostics, policy-update mode, and policy-update trace linkage must be deterministic, explicitly seeded, or explicitly disabled.	Missing simulation inputs block closed-loop, contextual-bandit, RL, and embodiment-sequence claims; missing environment diagnostics or update/audit linkage blocks adaptive-learning, frozen-evaluation, and closed-loop diagnostic claims.
Stress inputs	StressProfileRecord, StressScheduleRecord, stress configs, StressCard.	Stress profiles must be versioned, seeded, intensity-bounded, paired with clean records, and non-mutating.	Missing stress inputs remove stress-conditioned robustness claims; clean baseline claims remain evaluable.
Embodiment inputs	EmbodimentStateRecord, EmbodimentOutcomeRecord, embodiment-linked TransitionRecord, SafetyEventRecord or safety-event log, Layer9RewardTrace where enabled, command timeline, endpoint-error curve, Layer9EmbodimentDemoManifest, and simulation-adapter logs, command mapping configs, safety-gate logs, EmbodimentCard.	Command mappings, readiness gates, simulator branch/version, safety events, command chronology, endpoint-error evidence, reward terms where enabled, demo-manifest lineage, and simulation-only warnings must be logged.	Missing embodiment inputs remove or downgrade simulated embodiment claims and force Layer 10 to hide or warn the affected embodiment view.

Dashboard and reproducibility inputs	DashboardMetricRecord, DashboardMetricRecord.policy_update_filter, DashboardMetricRecord.embedding_filter, DashboardMetricRecord.repair_trace_filter, DashboardViewRecord, MetricDictionaryEntry references, MatchedComparisonReport references, NegativeResultNote references, ClaimEvidenceMatrix linkage, CardRecord, optional policy-update trace cards, disabled-update audit tables, export manifest, ReproducibilityManifest, warning logs, failure taxonomy reports.	Dashboard and lineage, blocks dashboard/reproducibility claims and thesis figure export; missing policy-update filtering blocks adaptive-learning dashboard claims; missing metric/comparison/negative-result/claim-evidence lineage blocks thesis-ready export.
Ablation inputs	A0–A13 outputs, matched baselines, negative-result notes.	Ablations must use matched splits, matched configs, and declared coverage/cost/reward settings.

25.6 Output contract

The outputs of Part 25 are audit artifacts. Each output has a downstream consumer and should be stored with schema version, run ID, source-artifact references, creation timestamp, and limitation tags.

Output artifact	Content	Downstream consumer
SystemFailureModeRegister	Canonical list of failure modes, categories, affected layers, severity, status, detection method, defense, and claim consequence.	Methods chapter, repository tests, dashboard failure taxonomy page, reproducibility package.
DefensiveDesignMatrix	Failure-to-control mapping with required record, warning, validation gate, ablation impact, and repair action.	Implementation pipeline, repository validation, final thesis audit.
FailureTaxonomyCard	Human-readable card summarizing core-blocking, stretch-blocking, optional, diagnostic, and claim-safety failures.	Dashboard cards/export page and thesis appendix.
ClaimImpactMatrix	Mapping from missing or invalid evidence to weakened, narrowed, invalid, or prohibited claims.	Results chapter, discussion chapter, limitation section, claim-safety dashboard.

BoundaryViolationChecklist	Checklist for detecting clinical, deployment, rehabilitation, real-FES, patient-safety, implanted-control, or real-world benefit language.	Thesis writing, card generation, dashboard warning panel.
QualityGateChecklist	Fail-closed gate definitions from dataset-to-window through dashboard-to-thesis export.	Repository tests, CI checks, command-line validation.
RepairActionTable	Recommended repair, fallback, downgrade, or rerun action for each failure.	Implementation workflow and final integration.
NegativeResultInterpretationGuide	Preserving negative results as evidence instead of hiding them.	Results chapter, ablation analysis, discussion section.
EvaluationArtifactTraceabilityChecklist	Checklist for dictionary entries, matched-comparison reports, layer1_ablation_readiness_l1_v1, Layer3MatchedOperatingPointTable, negative-result notes, and ClaimEvidenceMatrix linkage.	Repository tests, Layer 10 export validation, the Final demonstration section
DashboardWarningRequirements	Dashboard warnings by dashboard page.	Final demonstration audit.
CardWarningRequirements	Required warning text and limitation fields by card type.	Dashboard implementation and review.
ReproducibilityAuditChecklist	Required seeds, versions, hashes, manifests, schemas, and figure source mappings.	Reporting card generator and thesis appendix.
ImplementationPhaseRiskMap	Minimum defensive artifact required before advancing each implementation phase.	Reproducibility package, repository release, examiner audit.
OptionalComponentFallbackMap	Component-specific fallback statuses for incomplete or failed Stretch C branches.	Implementation pipeline and project management.
ThesisLimitationLanguageMap	Required limitation language attached to public EEG, StressForge-Lite, bandit/RL, and MyoSuite/OpenSim claims.	Final demonstration, final architecture summary, discussion of limitations.
		Thesis writing and card export.

25.7 Failure-mode taxonomy

The failure taxonomy is organized by architectural category. Each category defines the bottleneck addressed, scientific role, engineering responsibility, inputs, outputs, dependencies, consumers, evaluation logic, reproducibility requirement, thesis evidence, and limitations.

25.7.1 A. Governance and claim-safety failures

Purpose: prevent unsupported claims from being introduced by public EEG benchmarks, simulated policy learning, StressForge-Lite, MyoSuite/OpenSim, dashboard visualizations, or thesis wording. The bottleneck is claim drift: a computational result can be accidentally described as clinical validation or deployed safety evidence.

Core failures include clinical overclaiming from public EEG, clinical overclaiming from MyoSuite/OpenSim, policy-learning overclaiming from simulated rewards, RegimeRisk described as real fatigue diagnosis, StressForge-Lite described as real physiological stress validation, dashboard plots treated as new experiments, missing limitation text in cards or thesis figures, missing claim-evidence matrix entries for evaluation artifacts, and metric improvements described without allowed/prohibited claim classes.

Inputs are Layer 0 claim rules, limitation templates, card records, dashboard view records, thesis export manifests, and all claim-bearing summaries. Outputs are claim-safety warnings, boundary violation entries, limitation-language mappings, and claim downgrades. Dependencies include the claim ledger, warning logs, card generation, dashboard export, and thesis table/figure manifests.

Detection is performed by checking claim tags, warning fields, card limitation sections, dashboard warning panels, and final text labels. The preventive design is a fail-closed claim-to-evidence gate: a claim cannot be exported unless it is mapped to source records and allowed by Layer 0. The evaluation method is manual and automated review of claim-bearing artifacts. The thesis evidence produced is not a performance gain; it is a defensible boundary showing that the system does not overstate its evidence. A limitation is that claim-safety validation cannot judge all natural-language nuance; the final thesis still needs author review.

25.7.2 B. Public-data and protocol failures

Purpose: protect the public EEG benchmark foundation. The bottleneck is leakage or protocol inconsistency that would invalidate low-calibration and cross-subject evidence before any advanced module runs.

Core failures include protocol leakage, split drift across modules, inconsistent subject/session splits, hidden global normalization, label-map inconsistency, window overlap leakage, artifact filtering that uses outcome information, missing dataset or protocol cards, missing matched-comparison keys, absent or stale `manifests/layer1_ablation_readiness_l1_v1.json`, and public healthy EEG overstated as clinical evidence.

Inputs are `DatasetRecord`, `WindowRecord`, `SplitRecord`, `PreprocessingRecord`, `LabelMapRecord`, `ArtifactFlagRecord`, `Layer1Manifest`, `DatasetCard`, and `ProtocolCard`. Outputs are protocol validity status, leakage warnings, dataset/protocol card warnings, and split integrity reports. Hard dependencies are public dataset access, deterministic split configuration, window configuration, preprocessing configuration, label-map registry, and record schema validation. Consumers include decoders, calibration, IHARQ, RegimeRisk, policies, stress experiments, dashboard, ablations, and thesis tables.

Detection uses split-overlap checks, subject/session separation checks, normalization-scope checks, label-map consistency checks, window-origin hashes, and artifact-flag provenance. The defensive action is to block benchmark and downstream claims when leakage is detected. The evaluation method is A0 baseline integrity plus repository tests under `tests/data_protocol`. Reproducibility requires config hashes, dataset fingerprints, split seeds, schema versions, and source-record IDs. The thesis evidence produced is a clean public-data anchor. The limitation is that public EEG remains offline or pseudo-online and cannot prove clinical readiness.

25.7.3 C. Decoder, calibration, and selective-prediction failures

Purpose: ensure that the measurement spine is strong enough and honest enough to support downstream reliability reasoning. The bottleneck is treating raw accuracy or uncalibrated

confidence as a sufficient reliability signal.

Core failures include weak decoders that invalidate downstream claims, missing A0 raw-decoder evidence, missing A4 ordinary-combination controls, uncalibrated confidence treated as calibrated confidence, calibration split too small or leaked, threshold selection on the test set, selective-prediction baselines not matched by coverage or cost, hidden high-confidence wrong cases, inconsistent probability fields across models, missing model registry entries, missing `Layer3MatchedOperatingPointTable`, missing metric-dictionary definitions, and hidden negative-result notes for calibration failure or threshold sufficiency.

Inputs are `PredictionRecord`, model registry artifacts, `CalibrationRecord`, `UncertaintyRecord`, `SelectivePredictionRecord`, `ThresholdRegistryRecord`, reliability diagrams, ECE, Brier score, NLL, risk-coverage curves, and high-confidence-wrong audits. Outputs are calibrated reliability reports, threshold-origin warnings, selective-prediction comparisons, and decoder adequacy flags. Hard dependencies are Layer 1 protocol records and model registry entries; soft dependencies include probability calibration method and optional correction such as temperature scaling or Platt scaling.

Detection uses calibration-origin checks, threshold-source checks, probability-field schema checks, risk-coverage matching, and high-confidence-wrong audit counts. Preventive design requires thresholds and calibration parameters to be learned only on permitted splits and recorded in `ThresholdRegistryRecord`. Evaluation includes A1–A3 style decoder/calibration/selective-prediction comparisons, ECE, Brier, NLL, accepted-trial accuracy, risk-coverage curves, and high-confidence wrong rate. If calibration does not improve metrics, the result remains valuable but the claim is narrowed to visibility/audit rather than improvement. Reproducibility requires seeds, calibration configs, split IDs, model IDs, and figure source manifests.

25.7.4 D. IHARQ evidence-verification failures

Purpose: defend IHARQ as auditable evidence verification rather than simple confidence rejection. This preserves and expands the original failure mode that reviewers may dismiss IHARQ as just thresholding.

Core failures include IHARQ collapsing into confidence rejection, missing reason codes, un-auditable evidence-request logic, unimplemented wait/request/combine/fallback actions, absent multi-window or channel-stability checks, undefined fallback decisions, inconsistent unsafe states, and IHARQ adding burden without improving matched-risk or matched-cost metrics.

Inputs are `EvidenceQualityRecord`, `IHARQDecisionRecord`, `CombinedEvidenceRecord` if present, `FallbackDecisionRecord` if present, `UnsafeEventRecord` if present, `IHARQAblationRecord` if present, and `IHARQTraceRecord` if present. Outputs are IHARQ action logs, reason-code distributions, evidence request traces, combine-evidence traces, fallback decisions, unsafe-state audits, trace-completeness reports, and IHARQ-versus-baseline views. Dependencies include calibrated predictions, uncertainty records, selective-prediction baselines, threshold registry, multi-window features, channel-stability checks, and optionally `RegimeRisk` advisories.

Detection compares IHARQ against confidence thresholding, entropy rejection, margin rejection, selective prediction, longer windows, multi-window voting, probability averaging, and ensembles at matched coverage or matched cost. Preventive design requires every IHARQ decision to store its evidence source, rule version, canonical action token, display-label mapping where used, reason code, fallback status, unsafe status, trace pointer, and source prediction ID. Evaluation uses A2–A5 ablations plus Layer 4-local ablation records, matched coverage/cost comparisons, accepted-trial accuracy, unsafe-event rate, and burden metrics. Feedback paths are allowed only

when scientifically justified: RegimeRisk may make IHARQ more conservative through a logged temporal trust advisory, but it must not rewrite past predictions or thresholds. If IHARQ is not better than matched baselines, the claim is narrowed to auditable evidence-verification design rather than empirical superiority.

25.7.5 E. RegimeRisk temporal-trust failures

Purpose: ensure that temporal trust monitoring is causal, proxy-labeled, and decision-relevant. The bottleneck is mistaking temporal summaries for validated neurological state diagnosis.

Core failures include RegimeRisk using future information, non-causal regime features, proxy regime labels presented as real fatigue or drift labels, trust scores that do not influence decisions, untested stop-loss logic, false confidence from regime detection, inconsistent temporal windows across subjects or sessions, and no improvement under clean or stressed conditions.

Inputs are `RegimeStateRecord`, conditional `TrustStateRecord`, trust summaries, stop-loss logs, temporal feature configs, regime cards, trust-state cards where exported, and regime timelines. Outputs are regime warnings, temporal trust advisories, per-target trust-state timelines when independently consumed, stop-loss events, and regime-conditioned evaluation tables. Dependencies include prior prediction records, uncertainty records, IHARQ decisions, temporal window configuration, and stress records for robustness testing. Consumers include IHARQ, Adaptive-IHARQ, policy learning, dashboard regime timeline, stress evaluation, and ablation reports.

Detection verifies that every temporal feature uses only current or past evidence at the decision time. The defensive design requires temporal-causality tests under `tests/regimerisk`, explicit proxy-label language, and stop-loss logs. Evaluation uses A7-style ablations, regime-conditioned risk-coverage, accepted-trial accuracy, false-warning analysis, and clean-vs-stressed comparisons. If RegimeRisk does not help, the claim is narrowed to diagnostic temporal observability. RegimeRisk must never be described as diagnosing fatigue, cognitive state, neurological decline, or patient risk.

25.7.6 F. Adaptive policy and learning-to-defer failures

Purpose: make policy learning honest when public EEG lacks real readiness-action labels. This preserves and expands the original failure mode that Adaptive-IHARQ labels are artificial.

Core failures include pseudo-labels treated as clinical readiness labels, evaluation-only correctness fields leaking into policy context, arbitrary or unreported cost vectors, class imbalance distorting readiness actions, supervised Adaptive-IHARQ overfitting subject/session splits, missing or unmatched learning-to-defer baselines, undefined deferral targets, inconsistent fallback availability, missing legality masks, and absent policy reason codes.

Inputs are `ReadinessContextRecord`, `EvidenceQualityRecord`, `DeferralRecord`, `PolicyActionRecord`, target-construction cards, controller cards, cost configs, reward configs, and policy registry entries. Outputs are policy action logs, pseudo-label construction reports, deferral comparisons, cost/reward sensitivity analyses, legality-mask validation, and controller cards. Dependencies include calibrated predictions, IHARQ decisions, RegimeRisk context where used, fallback definitions, and split integrity.

Detection uses protected-field leakage tests, evaluation-only field exclusion checks, cost-vector versioning, action legality tests, label distribution audits, and held-out subject/session evaluation. Preventive design requires pseudo-labels to be named computational readiness labels and to be derived only from declared correctness, cost, deferral, fallback, and simulation

rules. Evaluation uses A6–A9 ablations, learning-to-defer baselines, matched-cost and matched-coverage comparisons, action confusion matrices, and reward/cost sensitivity. If learning-to-defer matches or beats Adaptive-IHARQ, that is a valid result and narrows the claim toward deferral as a strong baseline rather than failure of the thesis.

25.7.7 G. Contextual bandit and reinforcement-learning failures

Purpose: protect simulation-only policy learning from reward hacking and simulator overfit. This preserves and expands the original failure mode that RL can overfit the simulator.

Additional policy-update traceability failures include unlogged policy mutation, missing `PolicyUpdateTrace` for simulation-authorized updates, missing disabled-update audit entries during frozen-policy evaluation, missing `TransitionRecord.policy_update_trace_id` linkage, and dashboard plots that merge training/adaptation curves with frozen-policy evaluation curves. These are not cosmetic reporting errors; they can invalidate the policy-learning claim because the evaluator can no longer tell whether improvement came from a frozen policy, an adapting policy, or an untraceable mutation.

Core failures include reward hacking, reward weights chosen after seeing held-out outcomes, policy improvement under only one reward setting, bandit/RL overfit to simulated sessions, future rewards leaked into decision-time context, inconsistent action spaces across policies, unsafe actions not masked, missing held-out simulated sessions, simulator dynamics dominating the conclusion, and hidden RL instability.

Inputs are `PolicyActionRecord`, `TransitionRecord`, `RewardTraceRecord` where present, `ClosedLoopSessionRecord` where present, `PolicyRolloutRecord` where present, `EpisodeSummaryRecord` where present, `EnvironmentDiagnosticRecord` where present, optional `PolicyUpdateTrace`, controller cards, reward configs, and simulation manifests. Outputs are policy rollouts, reward-sensitivity reports, held-out session summaries, environment-diagnostic reports, safety-event logs, controller cards, policy-rollout cards, reward-configuration cards, and policy-update trace cards where applicable. Dependencies include simulation environment, frozen reward definitions, action legality masks, clean and stressed sessions, and baseline policies such as accept-all, reject-all, threshold, IHARQ-lite, and learning-to-defer.

Detection uses held-out simulated sessions, reward ablations, seed sensitivity, policy-stability summaries, action-legality checks, `EnvironmentDiagnosticRecord` warnings, reward-trace completeness checks, rollout/session lineage checks, and baseline comparisons. Preventive design freezes reward definitions before held-out evaluation and declares all bandit/RL claims simulation-only. The propagated defense additionally requires `PolicyUpdateTrace` completeness checks, disabled-update audits, update-mode consistency checks, and strict separation of training/adaptation from frozen-policy evaluation. Evaluation uses A10–A11 ablations, reward sensitivity, stress-conditioned rollouts, episode return, unsafe-event rate, action burden, and accepted-decision quality. If RL improves only under one reward design, the claim becomes reward-specific and fragile. RL policy results must never be described as deployed control, patient-safe control, or real rehabilitation control.

25.7.8 H. StressForge-Lite failures

Purpose: ensure that synthetic stress testing is reproducible and interpreted as robustness analysis, not as physiological validation. The bottleneck is silent perturbation of records or overstated stress conclusions.

Core failures include stress profiles not logged, stress profiles not paired with clean source records,

silent intensity changes, injection at incompatible points, stress changes hidden upstream decisions, combined-stress schedules not reproducible, stress labels interpreted clinically, missing held-out stress generalization, and large degradation hidden rather than used as vulnerability evidence.

Inputs are `StressProfileRecord`, `StressScheduleRecord` where present, stress configs, stress cards, clean source-record IDs, and stress sensitivity curves. Outputs are stressed-derived records, clean-vs-stressed comparison tables, stress warning logs, `StressCards`, and robustness-boundary claims. Dependencies include immutable clean records, stress profile registry, random seeds, injection point configuration, and downstream evaluation modules.

Detection validates stress profile hashes, paired clean-record IDs, intensity bounds, seed/version logs, and injection compatibility. Preventive design keeps clean records immutable and writes stressed records as derived artifacts. Evaluation uses A12 ablations, stress sensitivity curves, clean-vs-stressed degradation, and combined-stress robustness tests. If `StressForge-Lite` causes large degradation, the result should be exposed as robustness-boundary evidence rather than hidden. `StressForge-Lite` cannot validate real physiological stress, fatigue, rehabilitation difficulty, or patient safety.

25.7.9 I. Simulated closed-loop environment failures

Purpose: distinguish offline prediction quality from sequential decision consequences. The bottleneck is confusing retrospective metrics with simulated closed-loop control evidence.

The simulator also owns the first observable location where policy-update traceability can fail. If a `TransitionRecord` is produced while a policy update occurs, the transition must either link to a valid `PolicyUpdateTrace` or the run must be rejected as untraceable. Frozen-policy evaluation must record no mutation and must expose disabled-update audit status when audit mode is enabled.

Core failures include offline metrics interpreted as sequential consequences, incomplete transition records, missing closed-loop session records, missing policy rollout records, missing reward traces, missing environment diagnostics, inconsistent reward or done-state logic, simulation state containing future information, missing accept-all or no-action baselines, unlogged safety events, missing safety-event or reward-trace links, nondeterministic reset logic, unmeasured delay and action-cost burden, and simulation results overclaimed as deployed control.

Inputs are `TransitionRecord`, `ClosedLoopSessionRecord` if present, `EpisodeSummaryRecord` if present, `PolicyRolloutRecord` if present, `EnvironmentDiagnosticRecord` if present, `SafetyEventRecord` if present, `RewardTraceRecord` if present, optional `PolicyUpdateTrace`, and simulation manifests. Outputs are transition logs, closed-loop session timelines, episode summaries, policy-rollout summaries, reward-trace decompositions, environment-diagnostic reports, safety-event summaries, baseline comparisons, and simulation-only controller reports. Dependencies include policy records, reward configs, environment seed, state definition, action space, and termination rules.

Detection checks transition completeness, closed-loop session lineage, policy-rollout lineage, reward consistency, reward-trace linkage, environment-diagnostic warnings, state causality, action legality, reset determinism, safety-event linkage, and baseline presence. Preventive design requires every simulated run to declare state, action, reward, done, seed, and source-record references. It now also checks update mode, frozen-policy flag, `TransitionRecord.policy_update_trace_id` linkage where present, and disabled-update audit status for frozen-policy evaluation. Evaluation uses A10–A12 comparisons, episode return, reward decomposition, policy-rollout behavior, unsafe-event rate, delay cost, action burden,

environment-diagnostic counts, and robustness under stress. Simulation evidence remains valid only for declared simulated environments.

25.7.10 J. MyoSuite/OpenSim embodiment failures

Purpose: preserve the optional embodiment branch as a simulated proxy demonstration rather than a clinical or biomechanical validation claim. This preserves and expands the original failure mode that MyoSuite/OpenSim may be overclaimed.

Core failures include simulated embodiment interpreted as rehabilitation validation, arbitrary or undocumented proxy command mapping, readiness gates bypassed, unsupported commands executed instead of blocked, adapter crashes silently removing results, endpoint metrics lacking traceability, missing safety-event logs, missing command timelines, missing endpoint-error curves, missing reward traces when reward-enabled embodiment is claimed, missing `Layer9EmbodimentDemoManifest` files, stale simulator-version metadata, MyoSuite and OpenSim roles being confused, Layer 10 dashboard recomputation of embodiment evidence, simulation-only warnings missing from embodiment cards, and technical failure damaging core EEG reliability claims.

Inputs are `EmbodimentStateRecord`, `EmbodimentOutcomeRecord` for executed embodiment modes, or accepted-missingness status where absent, command-mapping configs, simulator adapter logs, safety-gate logs, `EmbodimentCard`, and E0–E5 or equivalent maturity fallback levels when defined by the implementation. Outputs are embodiment state/outcome records, blocked-command logs, adapter error reports, endpoint metric summaries, screenshots or replay references, and simulation-only embodiment cards. Dependencies include policy actions, readiness gates, simulation session records, command mapping, simulator version, and adapter contracts.

Detection checks that every executed command passed through a readiness gate and command-mapping contract and that every claimed embodiment bundle has safety-event logs, command timelines, endpoint-error curves, `Layer9EmbodimentDemoManifest` files, simulator-version metadata, and `Layer9RewardTrace` files where enabled. Preventive design blocks unsupported commands, logs blocked actions, marks incomplete bundles, and labels all embodiment outputs as simulated proxy evidence only. Evaluation uses A13 ablations, endpoint traceability, safety-gate compliance, command-mapping audits, reward-trace sanity checks, demo-manifest completeness checks, Layer 10 read-only export checks, and optional MyoSuite/OpenSim comparison where implemented. If MyoSuite/OpenSim fails technically, the core EEG reliability, calibration, IHARQ, RegimeRisk, and policy evidence remain evaluable; embodiment claims are downgraded to optional-only, partial, or future work. MyoSuite/OpenSim must never be described as proof of rehabilitation efficacy or safe real-world biomechanical control.

25.7.11 K. Dashboard, cards, and reproducibility failures

Purpose: ensure that visual inspection supports reproducibility rather than replacing evidence. This preserves and expands the original failure mode that the dashboard may become cosmetic.

Policy-update reporting introduces a dashboard-specific failure: a page may correctly read saved records but still mislead by plotting update-enabled training/adaptation curves beside frozen-policy evaluation curves without labels, filters, or warning status. Layer 10 must therefore expose `DashboardMetricRecord.policy_update_filter`, policy-update trace cards, disabled-update audit tables, and curve-separation checks whenever `PolicyUpdateTrace` artifacts exist.

Core failures include plots without inspection questions, hidden dashboard recomputation, mu-

tation of source records, missing source-record lineage, missing `DashboardMetricRecord` or `DashboardViewRecord` persistence, missing metric-dictionary or matched-comparison linkage, hidden diagnostic-only downgrades, missing `NegativeResultNote` or `ClaimEvidenceMatrix` linkage where required, card text conflicting with records, hidden warnings, omission of negative cases from the failure taxonomy, missing export manifests, incomplete reproducibility manifests, missing seeds/config hashes/code versions/schema versions, and dashboard aesthetics mistaken for scientific validity.

Inputs are `DashboardMetricRecord`, `DashboardViewRecord`, `Layer 7 TransitionRecord`, `ClosedLoopSessionRecord`, `EpisodeSummaryRecord`, `PolicyRolloutRecord`, `RewardTraceRecord`, `EnvironmentDiagnosticRecord`, `SafetyEventRecord` where materialized, optional `PolicyUpdateTrace`, `CardRecord`, `ReproducibilityManifest`, export manifest, warning logs, failure taxonomy page data, and cards/export page data. Outputs are read-only dashboard views, cards, figure manifests, warning panels, reproducibility cards, and thesis-ready exports. Dependencies include all saved records, card templates, dashboard loaders, export scripts, and repository tests.

Detection verifies that dashboard pages read saved records only, do not recompute experimental logic, do not mutate source records, and do not export figures without lineage. It also verifies that closed-loop timelines read `ClosedLoopSessionRecord` and `TransitionRecord`, policy-rollout views read `PolicyRolloutRecord`, reward panels read `RewardTraceRecord`, safety-event panels read `SafetyEventRecord` where materialized, and environment warning panels read `EnvironmentDiagnosticRecord` without hiding missingness. It now also verifies policy-update trace-card completeness, disabled-update audit visibility, and no mixing of simulation-training/adaptation curves with frozen-policy evaluation curves. Preventive design requires dashboard read-only tests under `tests/dashboard`, card-record consistency tests under `tests/reporting_cards`, and export-manifest completeness tests. Evaluation is based on record lineage completeness, warning visibility, reproduction of tables/figures from saved records, and consistency between card text and records. The dashboard is observability and reproducibility infrastructure; it is not an independent source of scientific results.

25.7.12 L. Complexity, scope, and implementation-feasibility failures

Purpose: prevent Stretch C from becoming too large, tangled, or all-or-nothing. This preserves and expands the original failure mode that the system becomes too complex.

Core failures include implementing modules out of order, tangling modules into one classifier script, failing to implement records before advanced models, allowing optional components to block core claims, letting MyoSuite/OpenSim or RL failure invalidate the thesis, mismatch between repository structure and module boundaries, missing tests, and unclear core/target/stretch maturity levels.

Inputs are implementation phase outputs, repository folders, configs, tests, records, figures, cards, logs, manifests, and optional-component status records. Outputs are implementation risk maps, maturity statuses, stop-at-lower-stage guidance, optional-component fallback maps, and non-thesis-ready flags. Dependencies include the implementation pipeline, repository architecture, record schemas, CI tests, and final demonstration plan.

Detection uses phase-completion checklists, manifest completeness, test coverage, module import boundaries, and optional-component status fields. Preventive design implements phases in order and makes each phase produce independent records. The thesis can stop at a lower stretch stage if needed without destroying the architecture. Evaluation is not higher accuracy; it is the ability to preserve valid core claims even when advanced components are partial, negative, or

incomplete.

25.8 Central defensive-design matrix

The following matrix is the compact implementation guide for the most important failure modes. Detailed versions should be stored in **DefensiveDesignMatrix**.

Failure mode	Affected layer	Detection method	Preventive design / mitigation	Artifact and dashboard/card warning	Claim consequence and status
System becomes too complex.	Implementation pipeline, repository, all optional branches.	Phase checklist, missing record families, tangled scripts, missing tests.	Implement phases in order; records before advanced models; optional branches marked complete, partial, mocked, missing, or future work.	ImplementationPhaseRiskMap, OptionalComponentFallbackMap, failure taxonomy page.	ClaimerRiskMap, PhaseFallbackMap, ready; incomplete Stretch C branches become optional-only or diagnostic.
IHARQ is just thresholding.	Layer 4, Layer 3, ablations.	Matched coverage/cost comparisons against thresholding, entropy, margin, selective prediction, longer windows, voting, ensembles, with trace and ablation-lineage checks.	Require evidence requests, stability checks, combine-evidence eligibility, fallback, unsafe states, canonical action-token/display-label mapping, reason codes, trace records, and RegimeRisk integration where used.	IHARQDecisionRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, IHARQ-versus-threshold dashboard, and AblationCard.	IHARQ super-d, claim supported, baselines matched-ablation/trace lineage are missing; status stretch-blocking for IHARQ claim.
A0–A4 evaluation artifacts lack traceability.	Layers 0–3, Layer 10, the Ablation and Evaluation Architecture, and the Final Demonstration section.	Check ClaimEvidenceMatrix , metric dictionary, matched-comparison report, manifests/layer10, A0/A4 baseline summaries, and Layer3MatchedOperatingPointTable.	Fail closed, downgrade to diagnostic-only until source-record, comparison-key, manifest, readiness point, negative-result, and claim- is complete .	Evaluation-artifact warning, claim-evidence dashboard panel, matched-comparison result, violation result note.	A0–A4 improvement or sufficiency claim is non-thesis-ready until repaired.
Adaptive-IHARQ labels are artificial.	Layer policy-learning sections.	Target-construction audit, protected-field leakage test, policy-context schema check.	Declare labels computational readiness pseudo-labels; log cost vectors, fallback availability, deferral comparisons, and reward sensitivity.	Target-construction card, ControllerCard , warning on policy comparison page.	Clinical readiness claim prohibited; computational policy claim allowed only with pseudo-label boundary.

RL overfits simulator.	Layer bandit/RL branch.	7,	Held-out simulated sessions, seed sensitivity, reward ablations, baseline comparisons.	Freeze reward weights; mask unsafe actions; declare simulation-only; report instability.	Controller card, rollout manifest, reward-sensitivity report.	RL claim restricted to simulation; missing held-out tests make it non-thesis-ready or diagnostic.
Unlogged policy mutation.	Layer 6, Layer 7, Layer 10.	6, 7,	PolicyUpdateTrace completeness check, disabled-update audit check, and TransitionRecord linkage validation.	Allow policy mutation only in declared simulation-update mode; policy_update_hooks during frozen evaluation; fail runs with missing traces.	Policy-update trace warning, policy-update trace card, disabled-update trace audit table.	Adaptive-learning or frozen-evaluation claim invalid until rerun or relabeled diagnostic.
Training/adaptation curves mixed with frozen-policy evaluation.	Layer 7, Layer 10, thesis exports.	7,	DashboardMetricRecords export-manifest review, and curve-separation validation.	RecordPolicyUpdateCard filter, separation warning on policy comparison and reproducibility pages.		Comparative policy claim invalid or narrowed because evaluation mode is ambiguous.
Layer 9 MyoSuite/OpenSim replay demo is overclaimed or incomplete.	Layer 9, Layer 10, Layer 13, A13 evidence page, evaluation.	9, 10	Embodiment card text, command-gate logs, safety-event logs, reward-trace checks where enabled, command-timeline checks, endpoint-error-curve checks, demo-manifest checks, adapter warnings, claim-safety checks.	Simulated proxy labels, declared command mappings, readiness safety gates, blocked unsupported commands, complete Layer9EmbodimentDemoManifest , Layer 10 read-only export validation.	EmbodimentCard , safety-gate logs, safety-event log, Layer9EmbodimentDemoManifest , embodiment dashboard warning.	Rehabilitation or patient-safety claim DemandManifest , bodiment remains optional-only or hidden/warned if incomplete.
Dashboard becomes cosmetic.	Layer 10, repository, thesis export.	10,	Dashboard read-only tests, figure lineage checks, card-record consistency, DashboardMetricRecords persistence checks, and DashboardViewRecord persistence checks.	Dashboard reads saved records only; no hidden recomputation; metric/view Records and export manifests required.	DashboardMetricRecords , DashboardViewRecord , CardRecord , ReproducibilityManifest , source-record warning, and export manifest.	Dashboard/reproducibility claim invalid if metric/view manifest is missing; figures hidden or warned.

Split or window leakage.	Layer 1, all downstream modules.	Split overlap, subject/session, normalization-scope, window-origin checks.	Fail closed; re-run protocol; block benchmark exports.	Layer1Manifest , ProtocolCard , leakage warning.	Benchmark and low-calibration claims invalid; status claim-safety violation.
Calibration or threshold leakage.	Layer 3, IHARQ, policies.	Threshold source split in ThresholdRegistry , calibration split audit.	Block test-set threshold tuning; RecordCard , calibration on allowed split.	Calibration card, reliability dashboard warning.	Uncertainty and selective-prediction claims invalid or narrowed.
RegimeRisk uses future information.	Layer 5, policies.	Temporal-causality test, feature time-tamp audit.	Use only current/past evidence; write stop-loss logs.	Regime time-line warning, RegimeCard .	Temporal-trust claim invalid; downstream policy context must remove leaked field.
Stress profile unlogged or unpaired.	Layer 8, evaluation.	Stress hash, seed, intensity, injection point, clean-source ID check.	Treat stress as derived record; keep clean records immutable.	StressCard , stress robustness warning.	Stress robustness claim unusable; clean claims remain evaluable.
Policy reads evaluation-only fields.	Layer 6, Layer 7.	Protected-field and evaluation-only context tests.	Strip fields from ReadinessContextRecord ; rerun policy.	Policy leak- Record ; warning, ControllerCard .	Policy claim invalid due to leakage.
Figure lacks lineage.	Layer 10, thesis export.	Source-record manifest check.	Hide or warn figure; regenerate from saved records.	Export manifest, reproducibility page warning.	Figure cannot support thesis claim.

25.9 Quality gates and failure responses

Quality gates fail closed when required evidence is missing or invalid. A failed gate writes an artifact, triggers a dashboard/card warning, and assigns a claim consequence.

Gate	Required validation	Failure response and artifact	Claim consequence
Dataset-to-window gate	Dataset fingerprints, subject/session IDs, window origins, label maps, preprocessing config, artifact flags.	Block run; write protocol failure in Layer1Manifest and QualityGateChecklist .	Benchmark, low-calibration, and all downstream claims invalid until repaired.
Window-to-prediction gate	Every prediction references dataset, split, window, model, seed, and score schema.	Mark prediction family invalid; rerun decoder or baseline.	Decoder and downstream reliability claims blocked.
Prediction-to-calibration gate	Calibration split allowed; threshold source not test; uncertainty fields complete.	Mark calibration non-thesis-ready; warn dashboard.	Calibration and selective-prediction claims invalid or narrowed.

Layers 0–3 evaluation-artifact gate	Layer 1 readiness, source records, A1–A3 matched operating points, metric-dictionary entries, matched-comparison report, negative- result notes, and ClaimEvidenceMatrix linkage complete.	Mark artifact diagnostic-only or non-thesis-ready; write EvaluationArtifactTraceabilityChecklist entry and dashboard warning.	A0–A4 compar- ison, sufficiency, or improvement claimability checklist repaired.
Calibration-to- IHARQ gate	IHARQ consumes cali- brated or explicitly un- calibrated scores with reason codes and thresh- old registry.	Block IHARQ claim; allow diagnostic-only trace if labeled.	IHARQ evidence- verification claim unsupported.
IHARQ-to- RegimeRisk gate	Regime features use causal win- dows and source IHARQ/prediction records.	Remove leaked tem- poral features; write regime warning.	Temporal trust claim invalid until rerun.
RegimeRisk-to- policy gate	Policy context excludes future labels/evaluation-only fields; legality masks present.	Block policy train- ing/evaluation; write leakage warning.	Adaptive policy claims invalid.
Policy-to- transition gate	Action space, reward, done logic, reset rule, safety events, seed, up- date mode, and frozen- policy flag declared.	Mark closed-loop run non-thesis-ready; rerun simulation.	Bandit/RL/closed- loop claims blocked.
Policy-update trace gate	Every simulation- authorized policy mutation has a PolicyUpdateTrace ; frozen-policy evaluation has no mutation and records disabled-update audit status when audited.	Mark update-enabled or frozen-evaluation re- sult invalid; rerun with correct update-mode configuration and trace emission.	Adaptive-learning, bandit/RL, and frozen-policy eval- uation claims blocked or rela- beled diagnostic.
Stress-to-stressed- record gate	Stress profile version, seed, intensity, injection point, and clean-source link complete.	Reject stressed record; keep clean record un- changed.	Stress robustness claims unusable.
Policy-to- embodiment gate	Readiness gate, com- mand mapping, sim- ulator version, and blocked-command behavior logged.	Block unsupported com- mand; mark embodi- ment optional-only if adapter incomplete.	Embodiment demonstration claim narrowed or invalid.

All-records-to-dashboard gate	Dashboard reads saved records, warning logs, cards, and manifests only.	Hide or warn invalid dashboard panels; do not recompute hidden results.	Dashboard observability claim invalid for affected panel.
Dashboard-to-thesis-figure gate	Figure has source-record lineage, config hash, schema version, and warning status.	Do not export figure or export with visible warning.	Figure cannot support claim unless repaired.
Claim-to-evidence gate	Claim is allowed by Layer 0 and mapped to valid records, ablations, and limitations.	Rewrite, downgrade, or remove claim; write boundary violation.	Unsupported or clinical claim invalid.

25.10 Claim-impact matrix

The claim-impact matrix translates failures into thesis consequences. It prevents the thesis from treating all failures as equal.

Failure condition	Affected claim	Required response
Split leakage, window overlap leakage, hidden global normalization, or label-map inconsistency.	Public EEG benchmark, low-calibration, subject/session generalization, and all downstream claims.	Mark invalid due to leakage; repair protocol and rerun.
Calibration missing, leaked, or threshold selected on test data.	Calibrated uncertainty, reliability visibility, selective prediction, IHARQ threshold-dependent claims.	Invalidate or narrow claim; rerun calibration with allowed split.
Missing Layer 1 ablation-readiness, matched comparison keys, metric dictionary entries, Layer3MatchedOperatingPointTables, negative-result notes, or ClaimEvidenceMatrix.	A0–A4 public-data, decoder-baseline, calibration, threshold, selective-prediction, and ordinary-combination comparisons.	Downgrade to diagnostic-only or non-thesis-ready; repair source-record matching, metric definitions, operating-point exports, and claim-evidence linkage before thesis use.
High-confidence wrong cases are hidden.	Claim that uncertainty exposes reliability failures.	Add high-confidence-wrong audit and dashboard/card warning.
IHARQ not compared with matched thresholding, selective prediction, longer windows, voting, averaging, and ensembles.	Claim that IHARQ is more than thresholding.	Restrict to architectural proposal or diagnostic analysis; complete A2–A5 comparisons before superiority claim.
IHARQ decisions lack reason codes or trace records.	Claim that IHARQ is auditable evidence verification.	Mark non-thesis-ready until reason-code and trace logging are repaired.
RegimeRisk uses future evidence or is called real fatigue diagnosis.	Temporal trust and regime-monitoring claims.	Remove leaked features or rewrite claim as proxy temporal reliability only.

Pseudo-label construction undocumented or clinical language used for Adaptive-IHARQ.	Adaptive readiness policy claim.	Add target-construction card and computational pseudo-label limitation; remove clinical language.
Learning-to-defer baseline missing.	Adaptive-IHARQ comparative claim.	Run learning-to-defer baseline or narrow claim to internal policy analysis.
Reward sensitivity missing for bandit/RL.	Policy-learning robustness claim.	Restrict to one frozen reward setting; label fragile/simulation-specific.
<code>PolicyUpdateTrace</code> missing for simulation-authorized bandit/RL update, or disabled-update audit missing for frozen evaluation.	Adaptive-learning, online-adaptation, or frozen-policy evaluation claim.	Invalidate or relabel the run as diagnostic; re-run with declared update mode, trace emission, and disabled-update audit.
Training/adaptation curves mixed with frozen-policy evaluation curves.	Comparative policy evaluation and dashboard/reproducibility claim.	Regenerate separated exports using <code>DashboardMetricRecord.policy_update_1</code> add warning until repaired.
Held-out simulated sessions missing.	Contextual bandit and RL generalization claim.	Mark diagnostic-only until held-out simulation is run.
Stress profiles unlogged, unpaired, or mutated.	Stress-conditioned robustness claim.	Reject stress result; regenerate with versioned stress records.
Embodiment readiness gate bypassed, safety-event log missing, reward trace missing when claimed, command timeline or endpoint-error curve absent, or <code>Layer9EmbodimentDemoManifest</code> incomplete.	Layer 9 MyoSuite/OpenSim/static-replay embodiment demonstration and Layer 10 embodiment evidence page.	Invalidate or hide/warn embodiment claim; repair Layer 9 gate/logging/manifest/export contract and rerun.
Dashboard figures lack lineage or recompute hidden results.	Dashboard and reproducibility claim.	Hide or warn figures; regenerate from saved records and export manifests.
Clinical, therapeutic, patient-safety, real-FES, deployed-control, or rehabilitation-efficacy language appears.	Any corresponding claim.	Rewrite or remove claim; mark boundary violation until repaired.

25.11 Negative-result and degradation handling

A negative result is not a system failure if it is traceable, honestly interpreted, and claim-safe. IHARQ BenchGuard should expose negative results because they identify reliability bottlenecks that accuracy-only evaluation can hide.

Negative or de-graded result	Interpretation	Diagnostic action and artifact	Claim adjustment
Calibration does not improve ECE, Brier, or NLL.	Calibration method may not help this dataset/model/split, but calibration audit remains informative.	Save calibration report and high-confidence-wrong audit.	Claim visibility rather than improvement.
IHARQ does not outperform selective prediction.	IHARQ may add interpretability or action structure but not empirical superiority under matched conditions.	Save IHARQ-versus-threshold and selective-prediction ablation results.	Narrow IHARQ claim.
Longer windows, voting, probability averaging, or ensembles beat IHARQ.	Simpler evidence aggregation may be stronger under current protocol.	Add ablation note and compare burden/latency/coverage.	Treat IHARQ as conditional benefit or architectural contribution.
Evidence-quality estimator fails.	Learned reliability estimation is not robust enough for the current setting.	Fall back to IHARQ-lite rules; save estimator error analysis.	Adaptive-IHARQ claim narrowed.
RegimeRisk does not help decisions.	Temporal trust features may be diagnostic but not decision-improving.	Save regime-conditioned metrics and false-warning analysis.	Claim observability only.
Learning-to-defer matches or beats Adaptive-IHARQ.	Deferral may be a stronger or simpler policy baseline.	Preserve learning-to-defer as a main result.	Adaptive-IHARQ superiority claim removed.
Bandit/RL improves under one reward only.	Policy benefit is reward-specific and fragile.	Add reward-sensitivity analysis and controller card warning.	Simulation-specific claim only.
Bandit/RL improves only while updating and lacks frozen-policy evaluation.	The result may show online adaptation behavior, not held-out frozen-policy performance.	Save <code>PolicyUpdateTrace</code> , disabled-update audit, separated training/adaptation and frozen-evaluation curves, and controller-card warning.	Online-adaptation claim only; frozen-policy superiority claim removed until rerun.
StressForge-Lite causes large degradation.	The system has a robustness boundary worth reporting.	Save clean-vs-stressed curves and stress vulnerability note.	Robustness claim becomes boundary mapping.

MyoSuite/OpenSim fails technically.	Embodiment branch is optional and should not damage core EEG reliability claims.	Save adapter error log and optional-component status.	Embodiment claim optional-only or future work.
Dashboard outputs are incomplete.	Scientific results may exist but are not fully inspectable.	Add dashboard completeness warning and reproducibility checklist.	Dashboard/reproducibility claim narrowed.

25.12 Dashboard and card warning map

All failure modes must be visible through dashboard and card artifacts. Visibility does not create new scientific evidence. It exposes saved records, warnings, lineage, missingness, and reproducibility status.

Required dashboard exposure includes the failure taxonomy page, claim-safety warning views, ablation page, IHARQ-versus-threshold page, RegimeRisk timeline page, policy comparison page, stress robustness page, embodiment page, cards/export page, and reproducibility page. Required card warnings include `DatasetCard`, `ProtocolCard`, `ReliabilityCard`, `RegimeCard`, `ControllerCard`, `StressCard`, `EmbodimentCard`, `AblationCard`, `FailureTaxonomyCard`, and `ReproducibilityCard`.

Dashboard/card location	Required warning or inspection item	Source artifact
Claim-safety warning view	Unsupported clinical, deployment, rehabilitation, fatigue-diagnosis, or patient-safety language.	<code>BoundaryViolationChecklist</code> , Layer 0 claim ledger.
Ablation page	Missing matched baselines, unmatched coverage/cost, negative results, or incomplete A0–A13 steps.	Ablation outputs, <code>AblationCard</code> .
IHARQ-versus-threshold page	Whether IHARQ beats or fails against thresholding, selective prediction, longer windows, voting, averaging, and ensembles, with request/combine, fallback, unsafe-event, trace, and diagnostic-only lineage exposed.	<code>IHARQDecisionRecord</code> , <code>EvidenceQualityRecord</code> , <code>CombinedEvidenceRecord</code> , <code>FallbackDecisionRecord</code> , <code>UnsafeEventRecord</code> , <code>IHARQAblationRecord</code> , <code>IHARQTraceRecord</code> , Layer 4 <code>EvaluationOutcomeRecord</code> , and <code>SelectivePredictionRecord</code> .
RegimeRisk timeline page	Temporal-causality status, proxy-label warning, trust-state lineage status where exported, stop-loss events, false-warning analysis.	<code>RegimeStateRecord</code> , conditional <code>TrustStateRecord</code> , stop-loss logs.

Policy comparison page	Pseudo-label boundary, leakage test status, cost/reward version, legality mask, deferral baseline, update-mode status, and separated training/adaptation versus frozen-evaluation curves.	<code>ReadinessContextRecord</code> , <code>PolicyActionRecord</code> , optional <code>PolicyUpdateTrace</code> , disabled-update audit entries, target-construction card.
Stress robustness page	Stress profile version, seed, intensity, clean-source pair, degradation visibility.	<code>StressProfileRecord</code> , <code>StressCard</code> .
Embodiment page	Simulation-only warning, command mapping, readiness gate status, blocked commands, <code>EmbodimentOutcomeRecord</code> completeness, safety-event log, <code>Layer9RewardTrace</code> where enabled, command timeline, endpoint-error curve, <code>Layer9EmbodimentDemoManifest</code> completeness, simulator-version metadata, adapter errors, <code>DashboardMetricRecord</code> lineage, and <code>DashboardViewRecord</code> export metadata.	<code>EmbodimentStateRecord</code> , <code>EmbodimentOutcomeRecord</code> , embodiment-linked <code>TransitionRecord</code> , <code>SafetyEventRecord</code> or safety-event log, <code>Layer9RewardTrace</code> where enabled, <code>Layer9EmbodimentDemoManifest</code> , adapter logs, <code>EmbodimentCard</code> , <code>DashboardMetricRecord</code> , and <code>DashboardViewRecord</code> .
Cards/export page	Card completeness, limitation fields, warning status, source-record lineage.	<code>CardRecord</code> , export manifest.
Reproducibility page	Seeds, config hashes, schema versions, code versions, source-record lineage, run manifests.	<code>ReproducibilityManifest</code> .

25.13 Repository and test linkage

The defensive design must be executable as repository tests and validation commands. The repository folders are not merely organizational; they enforce the scientific boundary.

Required test families include `tests/data_protocol`, `tests/decoders`, `tests/calibration_uncertainty`, `tests/iharq`, `tests/regimerisk`, `tests/adaptive_policy`, `tests/simulation`, `tests/stressforge`, `tests/embodiment_adapters`, `tests/dashboard`, `tests/reporting_cards`, and `tests/evaluation`.

The minimum validation set is:

- schema completeness for every canonical record;
- split integrity and window-origin integrity;
- threshold leakage and calibration-origin validation;
- Layer 1 ablation-readiness, matched-comparison key completeness, metric dictionary completeness, `Layer3MatchedOperatingPointTable` presence, `ClaimEvidenceMatrix` linkage, and negative-result note visibility;

- protected-field and evaluation-only field leakage checks for policy contexts;
- reason-code completeness for IHARQ and policy actions;
- temporal causality for RegimeRisk;
- action legality masks for Adaptive-IHARQ, learning-to-defer, bandit, and RL policies;
- **PolicyUpdateTrace** completeness for simulation-authorized contextual-bandit/RL updates;
- disabled-update audit and no-mutation checks for frozen-policy evaluation;
- reward-version freezing before held-out simulation;
- stress-profile hashing, clean-source pairing, and injection-point validation;
- embodiment readiness-gate and command-mapping validation;
- dashboard read-only behavior;
- card-record consistency and limitation-field completeness;
- export manifest and reproducibility manifest completeness.

25.14 Implementation-phase risk map

Each phase in the implementation pipeline must produce a minimum defensive artifact before the project advances.

Phase	Primary risk	Minimum defensive artifact before advancing
Phase 0	Repository, configuration, and record schema risks.	Schema registry, config hashing, run ID convention, and manifest template.
Phase 1	Public data, split, preprocessing, and protocol risks.	Layer1Manifest , ProtocolCard , split integrity report, and manifests/layer1_ablation_readiness_l1_v1.json .
Phase 2	Decoder and PredictionRecord risks.	Model registry, prediction schema validation, baseline report, A0 raw-decoder table, and A4 ordinary-combination control summary.
Phase 3	Calibration and uncertainty risks.	CalibrationRecord , ThresholdRegistryRecord , high-confidence-wrong audit, Layer3MatchedOperatingPointTable , and negative-result notes.

Phase 4	IHARQ-lite risks.	IHARQDecisionRecord, EvidenceQualityRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, Layer 4 EvaluationOutcomeRecord, reason-code completeness, action-token/display-label mapping, trace completeness, and threshold/IHARQ baseline comparison.
Phase 5	RegimeRisk risks.	Temporal-causality report, RegimeStateRecord, conditional TrustStateRecord, regime timeline warning.
Phase 6	Evidence-quality and Adaptive-IHARQ risks.	Target-construction card, policy-context leakage test, cost-vector registry.
Phase 7	Learning-to-defer risks.	Deferral baseline report, deferral target definition, matched-cost comparison.
Phase 8	Simulated closed-loop risks.	TransitionRecord, RewardTraceRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, EnvironmentDiagnosticRecord, SafetyEventRecord where material- ized, optional PolicyUpdateTrace, reward/done/state schema, safety-event logging, frozen-policy no-mutation check, and disabled-update audit fixture.
Phase 9	StressForge-Lite risks.	StressProfileRecord, stress hash, clean-source pairing report.
Phase 10	Contextual bandit risks.	Frozen reward config, held-out simu- lated session plan, legality-mask test, PolicyUpdateTrace completeness check, and separated training/adaptation ver- sus frozen-evaluation export.
Phase 11	RL risks.	Reward ablation plan, seed sensitiv- ity plan, simulation-only controller card, PolicyUpdateTrace completeness check, and disabled-update audit for held-out frozen evaluation.
Phase 12	Layer 9 MyoSuite/static-replay embodiment risks.	Command mapping config, readiness gate log, EmbodimentOutcomeRecord completeness check, safety-event log, command timeline, endpoint-error curve, Layer9RewardTrace where en- abled, Layer9EmbodimentDemoManifest, dashboard-compatible export, and simulation-only EmbodimentCard.

Phase 13	OpenSim replay or optional comparison risks.	Adapter contract, traceability map, optional-component status.
Phase 14	Dashboard and card risks.	Dashboard read-only test, <code>DashboardMetricRecord</code> and <code>DashboardViewRecord</code> persistence checks, card-record consistency report, policy-update trace-card check, frozen/update curve-separation check, metric dictionary export, matched-comparison report, <code>ClaimEvidenceMatrix</code> view, negative-result note index, diagnostic-only status check, export manifest, and reproducibility package validation.
Phase 15	Final thesis integration risks.	<code>ClaimImpactMatrix</code> , <code>ClaimEvidenceMatrix</code> , <code>ReproducibilityAuditChecklist</code> , negative-result register, and limitation-language map.

25.15 Optional-component fallback discipline

Stretch C is designed so that incomplete optional components do not destroy the thesis. Core EEG reliability claims should remain evaluable if MyoSuite/OpenSim fails. Rule-based IHARQ-lite should remain evaluable if Adaptive-IHARQ fails. Adaptive-IHARQ should remain evaluable if contextual bandit or RL is incomplete. Learning-to-defer remains a strong baseline even if Adaptive-IHARQ underperforms. StressForge-Lite can expose robustness limits even if performance drops. Dashboard and cards can still support traceability if advanced modules are marked incomplete. Embodiment remains optional and simulation-only. RL remains simulation-only and optional relative to core reliability claims.

Fallback statuses are defined as follows:

Status	Meaning	Claim use
core-ready	Core public-data, decoder, calibration, IHARQ-lite, record lineage, and claim-safety artifacts are valid.	Core thesis claims allowed.
target-ready	Main adaptive readiness, RegimeRisk, deferral, and ablation artifacts are valid.	Target claims allowed.
stretch-ready	Stress, simulation policy, dashboard, cards, and optional embodiment artifacts are valid.	Stretch claims allowed with limitations.

diagnostic-only	Artifact exists but lacks required comparison, held-out evaluation, or completeness for thesis claim.	May appear as diagnostic evidence only.
optional-only	Component is outside the required core path or partially implemented.	May be described as optional/future work.
non-thesis-ready	Required records, tests, or manifests are missing or invalid.	Cannot support thesis claim.
invalid due to leakage	Leakage or prohibited field use contaminates the result.	Must be repaired and rerun.
invalid due to claim violation	Artifact uses unsupported clinical, deployment, therapeutic, or patient-safety language.	Claim must be rewritten or removed.

25.16 Repair-action table

Detected problem	Repair action	Fallback if repair is not completed
Missing or invalid protocol records.	Recreate splits, windows, preprocessing records, label maps, and manifests; rerun downstream modules.	No benchmark claim.
Missing Layer 1 ablation-readiness or matched-comparison keys.	Regenerate <code>manifests/layer1_ablation_readiness.json</code> ; repair or document missing dataset/subject/session/window/split/preprocessing/label-map/budget/seed keys.	A0–A4 comparisons diagnostic-only;
Threshold or calibration leakage.	Refit thresholds/calibration on allowed split and regenerate uncertainty/IHARQ outputs.	Diagnostic-only uncalibrated analysis.
Missing metric dictionary, matched operating-point table, negative-result note, or claim-evidence matrix entry.	Regenerate evaluation exports from saved records; attach metric definitions, operating-point IDs, negative-result interpretation, and Layer 0 claim-evidence mapping.	Evaluation result cannot support thesis-ready improvement or sufficiency claim.
IHARQ reason codes incomplete.	Add reason-code schema and regenerate IHARQ decisions.	No auditable IHARQ claim.

Policy context leakage.	Remove evaluation-only fields, regenerate contexts, and retrain/re-evaluate policy.	Policy branch invalid.
RegimeRisk non-causal feature.	Replace with causal feature and re-run regime/policy evaluations.	RegimeRisk diagnostic-only.
Stress profile missing hash or clean pair.	Recreate stress profile registry and regenerate stressed records.	No stress robustness claim.
RL unstable or reward-specific.	Add reward ablations, seed sensitivity, and held-out simulation.	Simulation diagnostic-only.
Embodiment adapter incomplete.	Mark commands blocked or optional; preserve adapter logs.	Embodiment optional-only or future work.
Dashboard recomputes hidden results.	Refactor dashboard to load saved records and manifests only.	Dashboard/reproducibility claim invalid.
Clinical overclaim detected.	Rewrite text using Layer 0 limitation language and attach warning.	Claim invalid due to violation.

25.17 Layers 4–7 evaluation-lineage failure-mode addendum

The MERGE_RECORD_22 Layers 4–7 propagation adds a system-level defensive control: local evaluation evidence for IHARQ, RegimeRisk, adaptive policy, and simulated closed-loop policy must not become thesis evidence unless it remains matchable, metric-defined, negative-result-visible, dashboard/export-traceable, and claim-bounded.

Failure mode	Detection	Defensive control	Claim consequence
Missing Layer 4–7 EvaluationOutcomeRecord	Final demo or dashboard references an A5–A11 result without local evaluation record	Regenerate local evaluation record or remove claim linkage	Non-thesis-ready.
Unmatched IHARQ, RegimeRisk, policy, or simulation comparison	Missing matched-comparison key or failed match audit	Mark diagnostic-only and expose reason in dashboard/card/export	No improvement claim.
Undefined or inconsistent metric dictionary	Metric lacks dictionary ID, snapshot, or definition	Attach metric dictionary or rerun evaluation with declared metric	Metric cannot support claim.
Hidden negative result	Expected negative-result note missing from reports, dashboard, or card	Export negative-result note and limitation text	Claim must be weakened or reframed.
Missing RegimeRisk negative control	A7 result lacks shuffled/random risk control when used as evidence	Run negative control or downgrade to diagnostic temporal visualization	No RegimeRisk contribution claim.
Mixed training/adaptation and frozen-policy evidence	A10/A11 figure combines update-enabled curves with frozen-policy evaluation	Split views and exports; attach update-mode lineage	Adaptive policy claim invalid until repaired.

Reward-sensitive policy ranking without sensitivity note	Policy ordering changes under reward/cost settings but no note appears	Add cost/reward sensitivity report and controller-card warning	Claim must be bounded to reward assumptions.
Dashboard/export lineage missing	Figure/table/card lacks source-record, metric, comparison, and claim-evidence IDs	Block thesis export or mark incomplete	Export cannot support thesis claim.

The repair rule is conservative. Missing metric dictionaries, missing Layer 4–7 evaluation-outcome records, missing matched-comparison reports, missing negative-result notes, missing diagnostic-only flags, or missing claim-evidence entries must be repaired by regenerating evaluation exports from saved records or by downgrading the claim. They must not be patched by manual thesis-table editing.

25.18 Layers 8–10 evaluation-lineage failure-mode addendum

The MERGE_RECORD_22 Layers 8–10 propagation adds a system-level defensive control: local evaluation evidence for StressForge robustness and Layer 9 embodiment-proxy consequences must not become thesis evidence unless it remains matchable, metric-defined, negative-result-visible, dashboard/export-traceable, and claim-bounded.

Failure mode	Detection	Defensive control	Claim consequence
Missing Layer 8 A12 EvaluationOutcomeRecord	Final demo, dashboard, stress card, or thesis table references A12 without <code>records/evaluation_outcomes/layer8_stress_evaluation_outcomes.parquet</code>	Regenerate local evaluation record or remove claim linkage	Non-thesis-ready.
Unmatched stress comparison	<code>reports/matched_comparisons/layer8_matched_stress_comparisons.csv</code> missing, failed, or not tied to clean/stressed source records	Block <code>layer8_matched_stress_comparisons.csv</code> and expose reason in dashboard/card/export	Non-thesis-ready.
Undefined stress metric dictionary	<code>metrics/metric_dictionary/layer8_stress.yaml</code> missing or inconsistent with exported metrics	At <code>layer8</code> is dictionary or rerun A12 evaluation with declared metric definitions	Metric cannot support claim.
Hidden stress negative result	Expected <code>notes/negative_results/layer8_hidden_stress_negative_results.csv</code> missing from reports, dashboard, or card	Export negative-result <code>layer8_hidden_stress_negative_results.csv</code>	Claim must be bounded or reframed as boundary mapping.
Missing Layer 9 A13 EvaluationOutcomeRecord	Final demo, dashboard, embodiment card, or thesis table references A13 without <code>records/evaluation_outcomes/layer9_embodiment_evaluation_outcomes.parquet</code>	Regenerate local evaluation record or remove claim linkage	Non-thesis-ready.

Unmatched embodiment comparison	<code>reports/matched_comparisons/layer9/matched_embodiment_comparisons.csv</code> missing, failed, or not tied to readiness/policy/stress/command source records	Mark Layer 9 embodied and expose reason in dashboard/card/export	Embodiment- consequence comparison claim.
Undefined embodiment metric dictionary	<code>metrics/metric_dictionary/layer9/embodiment.yam</code> missing or inconsistent with endpoint, command, safety, or reward metrics	At Layer 9, rerun A13 dictionary or rerun A13 evaluation with declared metric definitions	Metric cannot support claim.
Hidden embodiment negative result	<code>notes/negative_results/layer9/embodiment_negative_results.md</code> Expected missing from reports, dashboard, or card	Export negative-result limitation text	Claim must be marked optional- only, or marked diagnostic- only.
Incomplete A12/A13 claim-evidence lineage	<code>manifests/claim_evidence/layer8/stress_claim_evidence.json</code> or <code>manifests/claim_evidence/layer9/embodiment_claim_evidence.json</code> missing or not referenced by dashboard/export/final demo	Block the stress claim mark incomplete until are regenerated from saved records	<code>Evidence.json</code> robustness or claim evidence claim unsupported.
Layer 10 hidden repair of missing A12/A13 artifacts	Dashboard output contains A12/A13 table or figure without source record, matched-comparison, metric dictionary, negative-result, and claim-evidence pointers	Remove hidden repair, reload canonical artifacts, and emit warning	Dashboard/reproducibility claim invalid until repaired.

The repair rule is conservative. Missing Layer 8/Layer 9 evaluation-outcome records, missing matched-comparison reports, missing metric dictionaries, missing negative-result notes, missing diagnostic-only flags, missing dashboard/export lineage, or missing claim-evidence entries must be repaired by regenerating evaluation exports from saved records or by downgrading the claim. They must not be patched by manual thesis-table editing or dashboard-side recomputation.

25.19 Final consistency checklist for the Failure Modes and Defensive Design section

The defensive-design register is consistent with the full architecture only if the following conditions hold.

- The project remains IHARQ BenchGuard and the maximum scope remains Stretch C.
- The six original failure modes are preserved and strengthened using their original meanings: the system becomes too complex; IHARQ is just thresholding; Adaptive-IHARQ labels

are artificial; RL overfits the simulator; MyoSuite/OpenSim demo is overclaimed; and dashboard becomes cosmetic.

- Layers 0–10, the cross-layer functional module view, the detailed input-output proximity map, the A0–A13 ablation ladder, the implementation pipeline, and the repository architecture remain consistent.
- Layer 8 A12 and Layer 9 A13 evaluation artifacts are defended against missing evaluation records, unmatched comparisons, undefined metric dictionaries, hidden negative results, incomplete claim-evidence lineage, and Layer 10 hidden repair.
- Public EEG evidence remains separate from simulation-only policy, stress, and embodiment evidence.
- MyoSuite/OpenSim remains a simulated proxy embodiment branch only.
- StressForge-Lite remains controlled synthetic stress testing only.
- Contextual bandit and RL remain simulation-only policy-learning branches, and every simulation-authorized policy mutation is traceable through `PolicyUpdateTrace`.
- Adaptive-IHARQ labels remain computational readiness pseudo-labels.
- Dashboard outputs remain read-only projections from saved records and manifests, with `DashboardMetricRecord.policy_update_filter` preserving separation between simulation-training/adaptation and frozen-policy evaluation.
- Negative results are exposed through records, ablations, cards, and dashboard warnings rather than hidden.
- The Layers 0–3 evaluation slice exposes `ClaimEvidenceMatrix`, `manifests/layer1_ablation_readiness_l1_v1.json`, A0/A4 baseline-control evidence, `Layer3MatchedOperatingPointTable`, metric dictionary entries, matched-comparison reports, and negative-result notes before any A0–A4 claim is treated as thesis-ready.
- Missing records, invalid records, leakage, and untraceable figures fail closed or are explicitly downgraded.
- No clinical, therapeutic, patient-safety, medical-device, implanted-control, real-FES, deployed-control, or real-world rehabilitation claim is introduced.
- Every added failure mode has a detection method, defensive control, artifact, dashboard or card exposure, claim consequence, and repair or fallback path.
- Unlogged policy mutation, mixed training/adaptation versus frozen-evaluation curves, missing disabled-update audits, and missing `PolicyUpdateTrace` linkage are treated as global failure modes rather than Layer 7-only local warnings.

Therefore, the Failure Modes and Defensive Design section functions as the final defensive-design layer of IHARQ BenchGuard under Stretch C. It converts the architecture from a collection of advanced modules into a thesis-defensible system: every risk is mapped to evidence, every evidence gap is mapped to a claim consequence, every optional branch has a fallback status, and every dashboard or card output must remain traceable to saved records rather than visual polish.

26 What the Final Stretch C System Should Demonstrate

26.1 Purpose and architectural role

The Final Demonstration section is not an additional computational layer and it is not a marketing summary. It is the final **demonstration contract** for IHARQ BenchGuard under Stretch C. Its job is to state, with thesis-grade precision, what the completed system must demonstrate, which upstream layers produce the evidence, which records and artifacts must exist, which ablations validate each claim, which dashboard or card exposes the result, and which claims remain forbidden even if the result looks strong.

The bottleneck addressed by this section is **claim-evidence closure**. Earlier layers define datasets, decoders, uncertainty, IHARQ decisions, temporal trust, adaptive policies, stress tests, embodiment, dashboards, repository paths, and defensive controls. Without a final demonstration contract, the architecture could still be criticized as a collection of impressive modules without a disciplined answer to the question: *what exactly must the final Stretch C system prove, and what does it not prove?*

The Final Demonstration section therefore supports the thesis claim that the project is not merely an EEG-classification benchmark, but a reproducible reliability architecture for low-calibration neural-intent readiness. It protects against the following criticisms:

- that the system only improves accuracy without addressing readiness, uncertainty, wrong-accept risk, or reliability under changing conditions;
- that IHARQ, Adaptive-IHARQ, RegimeRisk, learning-to-defer, bandit/RL, StressForge-Lite, and embodiment are disconnected add-ons;
- that advanced modules are overclaimed without matched baselines, ablations, records, or failure interpretation;
- that dashboard cards are cosmetic rather than traceable scientific artifacts;
- that public EEG, proxy readiness labels, simulated stress, or simulated embodiment are being misrepresented as clinical validation.

If this section were removed, the architecture would still contain modules, but the thesis would lose its final acceptance logic: which evidence is sufficient, which claim is allowed, which claim must be weakened, which missing artifact invalidates a demonstration, and how negative results should be interpreted. The Final Demonstration section is therefore the bridge between the full system design, the implementation pipeline, the ablation architecture, the dashboard/reproducibility layer, the failure-mode defenses, and the final thesis contribution statement.

26.2 Engineering responsibility, boundary, and internal structure

The engineering responsibility of the Final Demonstration section is to consume validated records and manifests from upstream layers and convert them into thesis-facing synthesis artifacts. It does not train a decoder, tune a threshold, learn a policy, create new stress profiles, run MyoSuite/OpenSim, or recompute hidden dashboard statistics. It verifies that each final claim is backed by saved records, configuration hashes, ablation comparisons, limitation labels, and reproducible exports.

In scope. The Final Demonstration section specifies final demonstrations, claim-to-evidence mapping, capability-tier readiness, minimum/strong/full success criteria, accepted failure interpretations, dashboard/card linkage, repository artifact expectations, and final thesis-readiness checks.

Out of scope. The Final Demonstration section must not introduce new algorithms, new training procedures, new clinical claims, new patient-safety claims, new real-time deployment claims, or hidden recomputation. Any algorithmic or experimental result must originate from the appropriate upstream layer.

The layer is organized as a set of final demonstration-contract modules:

Contract module	Role	Core status	Implementation artifact
26.1 Demonstration Scope Gate	Determines whether a proposed final demonstration is core, target, stretch, diagnostic-only, optional-only, invalid, or future work.	Core	reports/final_demo/demonstration_scope_and_dashboard_warning_flags.
26.2 Capability-Tier Synthesizer	Maps final evidence to C0–C4 and prevents unsupported claims about incomplete tiers.	Core	reports/final_demo/capability_tier_synthesizer.
26.3 Demonstration-Evidence Matrix Builder	Connects each final demonstration to source layers, records, ablations, metrics, cards, claims, limitations, and failure interpretation.	Core	reports/final_demo/demonstration_evidence_matrix_builder.
26.4 Claim-to-Artifact Ledger	Links thesis claims to exact figures, tables, cards, manifests, configs, and records.	Core	reports/final_demo/claim_to_artifact_ledger.
26.5 Negative-Result Interpreter	Defines how weak, null, or baseline-matched results are reported without hiding them.	Core	reports/final_demo/negative_result_interpreter.

26.6 Reproducibility Acceptance Checker	Confirms that every major demonstration has records, configs, seeds, manifests, dashboard exports, and thesis-location lineage.	Core	reports/final_demo/reproducibility_r
26.7 Stretch Completion Reporter	Summarizes optional bandit/RL, stress, embodiment, OpenSim, and dashboard completion without making them required for minimum thesis validity.	Supporting/stretch	reports/final_demo/stretch_completion

For each contract module, the implementation rule is the same: it should read saved upstream records, manifests, card registries, dashboard metric records, and validation logs; it should write explicit final reports and machine-readable matrices; and it should fail closed when a required record, split, threshold, configuration hash, or claim-safety warning is missing.

26.3 Final Demonstration section input and output contract

The Final Demonstration section receives no raw EEG and performs no hidden model inference. Its inputs are validated records, metrics, manifests, and cards from the rest of the architecture.

Input class	Records and artifacts	Purpose inside the Final Demonstration section
-------------	-----------------------	--

Primary inputs	DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, DatasetCard, ProtocolCard, ValidationReport, Layer1Manifest, manifests/layer1_ablation_readiness_evaluation, PredictionRecord, A0/A4 baseline-control summaries, CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, Layer3MatchedOperatingPointTable, EvidenceQualityRecord, IHARQDecisionRecord, RegimeStateRecord, conditional TrustStateRecord, ReadinessContextRecord, DeferralRecord, PolicyActionRecord, TransitionRecord, StressProfileRecord, EmbodimentStateRecord, DashboardMetricRecord, metric dictionary entries, matched-comparison reports, ClaimEvidenceMatrix, and negative-result notes.	Establish the full evidence chain from public-data protocol to decoder behavior, uncertainty, matched A0–A4 evaluation, readiness decisions, temporal trust and independent trust timelines where emitted, policy learning, stress, embodiment, and readiness evaluations.
Auxiliary inputs	Dataset/protocol cards, decoder cards, calibration cards, IHARQ decision cards, regime cards, controller cards, deferral cards, stress cards, embodiment cards, failure taxonomy cards, claim-safety warning cards, and reproducibility manifests.	Confirm that evidence is inspectable, card-linked, and understandable to a thesis examiner or future reproducer.
Metadata inputs	Config files, config hashes, seed registries, code version, environment files, package-lock files, dataset version notes, run IDs, split IDs, threshold IDs, policy IDs, stress IDs, mapping IDs, simulator versions, and export manifests.	Support traceability, determinism, artifact lineage, and reproducibility claims.

Evaluation-only inputs	Ground-truth labels, correctness targets, held-out outcomes, pseudo-readiness labels, reward/cost values, expert/supervisor proxies for learning-to-defer, and embodiment outcome measures.	Used only for evaluation and final evidence synthesis; these fields must not leak into policy inputs unless explicitly allowed by the corresponding upstream layer.
Proxy inputs	Cost-derived readiness labels, fatigue-like stress variables, drift schedules, dropout schedules, weakness/noisy-intent profiles, simulated reward functions, and mapped assistive commands.	Support adaptive-policy, stress, and embodiment demonstrations while preserving the proxy/simulation-only boundary.
Stretch inputs	Contextual-bandit logs, RL controller artifacts, MyoSuite/OpenSim logs, OpenSim replay outputs, screenshots, simulator traces, and optional comparison reports.	Enable C3–C4 demonstration if implemented; otherwise these inputs may remain optional-only or future work without invalidating the minimum thesis.

The Final Demonstration section produces synthesis artifacts rather than raw computational outputs:

Output artifact	Content	Downstream consumer
demonstration_evidence_index	Final demonstration mapped to source layers, records, metrics, ablations, cards, supported claim, unsupported claim, and failure interpretation.	Thesis results/discussion chapters, dashboard cards/export page, examiner-facing appendix.
claim_to_artifact_map	Major thesis claim linked to records, figures, tables, configs, manifests, and warnings.	Layer 0 claim governance, final thesis contribution map, reproducibility appendix.
capability_tier_summary	C3–C4 completion level, evidence status, missing artifacts, and allowed claim strength.	Final architecture summary, dashboard architecture status view, implementation sign-off.
minimum_strong_full_success_checklist	Success checklist, final thesis, and full Stretch C acceptance criteria.	Project planning, final defense preparation, implementation pipeline tracking.
negative_result_interpretation	Reported language for weak, null, baseline-matched, failed, or leakage-contaminated demonstrations.	Thesis discussion chapter and failure-mode appendix.

<code>final_card_export_inventory.csv</code>	Repository cards, static figures, tables, dashboard pages, export files, and lineage manifests.	Dashboard/card generator, repository release package, final thesis export.
<code>limitation_map.md</code>	Allowed and forbidden claim boundaries for public EEG, proxy labels, simulated stress, policy learning, MyoSuite/OpenSim, dashboard, and clinical non-claims.	Layer 0, thesis limitation section, safety warning cards.
<code>reproducibility_readme.md</code>	Lineage, checklist, embodiment, versioning, deterministic outputs, record lineage, and figure/table generation status.	Reproducibility appendix and repository release checklist.

26.4 Dependency logic

The Final Demonstration section has hard dependencies on Layers 0–4 and Layer 10 for any minimum thesis-ready demonstration. It has soft dependencies on Layer 5, Layer 6, and learning-to-defer for a strong thesis demonstration. It has optional or stretch dependencies on contextual bandit, RL, StressForge-Lite, MyoSuite/OpenSim, and OpenSim replay for the full C3–C4 demonstration.

Dependency type	Required items	Development or fallback rule
Hard dependencies	Layer 0 claim-safety rules; Layer 1 dataset/split/window/protocol records; Layer 2 decoder predictions; Layer 3 calibration, uncertainty, selective prediction, and threshold registry; Layer 4 IHARQ-lite decisions; Layer 10 saved dashboard/card/export manifests.	Without these, the thesis may not claim a complete reliability architecture. Mock records may be used during Phase 0 only, but final claims require real validated records.
Soft dependencies	Layer 5 RegimeRisk records; Layer 6 evidence-quality estimator, Adaptive-IHARQ, and learning-to-defer; Ablation and Evaluation Architecture ablation summaries; Failure Modes and Defensive Design status and repair logs.	If incomplete, claims narrow to C0–C1; the thesis can still be valid if the limitation is explicit and the core baseline/IHARQ evidence is complete.
Optional/stretch dependencies	Contextual-bandit logs, RL controller artifacts, StressForge-Lite stress profiles, MyoSuite/OpenSim embodiment traces, optional OpenSim replay, full dashboard app.	These can be marked optional-only, diagnostic-only, or future work unless the thesis explicitly claims C3–C4 completion.

Conceptual dependencies	Functional graph architecture, system-wide interface contracts, detailed input-output proximity map, C0–C4 tiers, A0–A13 ablation ladder, Phase 0–15 implementation pipeline, repository architecture, and defensive-design rules.	These cannot be mocked conceptually; final claims must remain consistent with them even when implementation is partial.
-------------------------	--	---

The upstream outputs that must exist before the Final Demonstration section can run are: split-safe protocol records, prediction logs, calibration and uncertainty records, selective-prediction baselines, IHARQ decision records, at least the implemented subset of ablation summaries, dashboard metric records generated from saved data, warning flags, and a reproducibility manifest. Advanced demonstrations additionally require their corresponding policy, stress, transition, embodiment, or simulator records.

26.5 Feedforward and loopback behavior

The Final Demonstration section mainly feeds forward into the final thesis, dashboard export package, repository release, and final architecture summary. It also creates controlled loopback signals when a demonstration is not thesis-ready. These loopbacks are not arbitrary feedback loops; each exists to prevent unsupported claims or invalid evidence from entering the thesis.

Source	Target	Signal and trigger	Action	Safety constraint	Evaluation logic
Final demonstration matrix checker	Layer 0	Unsupported claim or missing limitation label.	Weaken, block, or rewrite claim language.	No clinical, deployment, patient-safety, real-FES, or real rehabilitation claim.	Claim ledger must mark the final claim as allowed before export.
Final demonstration evidence checker	Layers 1–3	Missing split, threshold, calibration, or uncertainty lineage.	Regenerate records or downgrade demonstration.	No threshold leakage, split leakage, or hidden calibration reuse.	Validation logs must pass before the claim is thesis-ready.
Final demonstration IHARQ comparison checker	Layers 3–4	IHARQ result lacks matched threshold, longer-window, voting, averaging, or ensemble comparison.	Add comparison or narrow IHARQ claim to traceability/reason-code behavior.	No performance-superiority claim without matched evidence.	A0–A7 comparison rows must be present.

Final demonstration policy checker	Layer 6 and Layer 7	Adaptive, deferral, bandit, or RL result lacks held-out evaluation, cost specification, or leakage guard.	Repair policy inputs, rerun, or mark diagnostic-only.	Public EEG has no true readiness-action labels; labels are proxy/cost-derived.	Policy claim requires pseudo-label manifest, held-out results, and leakage audit.
Final demonstration stress checker	Layer 8	Stress result lacks clean paired run, stress hash, or held-out stress profile.	Rebuild stress profile or report diagnostic-only robustness boundary.	StressForge-Lite is bounded perturbation testing, not validated physiology.	Stress claim requires clean/single/combined out profile comparison.
Final demonstration embodiment checker	Layer 9	Embodiment result lacks mapping manifest, blocked-command logs, or simulation warning.	Add mapping/safety logs or mark embodiment optional-only.	MyoSuite/OpenSimA13 and is simulated proxy embodiment, not clinical validation.	embodiment-card evidence must pass before C4 claim.
Final demonstration dashboard checker	Layer 10	Figure, card, or dashboard metric has no source-record lineage.	Block export or regenerate from saved records.	Dashboard must not silently recompute hidden scientific evidence.	Export manifest must connect every figure/table/card to source records and configs.

26.6 Demonstration tiers and thesis-valid stopping points

The final system does not need to complete all C0–C4 tiers to remain thesis-valid. However, any claim about a tier must be supported by the corresponding evidence, records, ablations, dashboard/card artifacts, and limitation statements.

Tier	Name	What the final system must demonstrate	Claim status
------	------	--	--------------

C0	Public-data reliability spine	Split-safe public EEG records; decoder baseline comparisons; low-calibration curves; subject/window/session variability; calibration visibility; uncertainty auditing; high-confidence-wrong analysis.	Minimum thesis foundation.
C1	IHARQ and RegimeRisk reliability control	Reason-coded evidence verification; wrong-accept reduction or explicit trade-off analysis; fallback and unsafe-state handling; temporal trust monitoring and stop-loss warnings where implemented.	Core/target reliability claim.
C2	Adaptive policy layer	Evidence-quality estimation; supervised Adaptive-IHARQ; learning-to-defer; cost-based pseudo-labels; matched-cost and matched-coverage policy comparisons under public-data constraints.	Strong thesis extension.
C3	Bandit/RL simulation layer	Sequential readiness-policy evaluation in a declared simulated closed-loop environment with frozen reward functions, held-out sessions, stress-conditioned policies, and controller cards.	Simulation-only stretch claim.
C4	Layer 9 embodiment and Layer 10 dashboard layer	Simulated assistive-action consequences through Layer 9 MyoSuite/OpenSim/static-replay proxy mappings plus Layer 10 read-only dashboard pages, cards, demo-manifest indexes, warnings, and reproducibility exports.	Simulated embodiment and observability claim only.

Minimum thesis-valid demonstration. A minimum defensible thesis demonstration includes public EEG protocol and split-safe dataset records; decoder baseline comparisons; low-calibration performance curves; calibration and uncertainty analysis; selective-prediction baselines; IHARQ-lite compared against simple thresholds, longer windows, voting, averaging, and ensembles where available; clear claim-safety boundaries; and reproducible records, figures, tables, cards, and manifests. This corresponds mainly to C0–C1.

Strong thesis demonstration. A strong thesis demonstration adds RegimeRisk temporal trust monitoring, evidence-quality estimation, supervised Adaptive-IHARQ, learning-to-defer, matched-cost and matched-coverage comparisons, stress-conditioned evaluation, dashboard/card traceability, and negative-result analysis. This corresponds mainly to C0–C2 plus selected C3 evidence.

Full Stretch C demonstration. A full Stretch C demonstration adds contextual-bandit policy evaluation, reinforcement-learning readiness-policy evaluation, simulated closed-loop transition and reward records, StressForge-Lite clean/single/combined/held-out stress profiles, Layer 9-owned MyoSuite/OpenSim/static-replay simulated embodiment, embodiment safety gate, proxy command mapping, safety-event logs, `Layer9RewardTrace` files where enabled, command time-

lines, endpoint-error curves, **Layer9EmbodimentDemoManifest** files, final dashboard app, static exports, cards, reproducibility package, and claim-to-artifact matrix. This corresponds to C0–C4 and is the most ambitious, most claim-restricted version.

26.7 Final demonstration-evidence matrix

The final Stretch C system should demonstrate the following ten points in increasing ambition. The original ten demonstration meanings are preserved, but each is now expressed as a traceable demonstration contract. Each row identifies the required layers, records, evaluation mode, ablations, metrics, dashboard/reporting artifacts, supported claim, unsupported claim, and failure interpretation.

Demonstration statement	Primary layers and required records	Evaluation mode, ablations, and metrics	Dashboard/reporting artifact, supported claim, unsupported claim, and failure interpretation
1. Public EEG decoders are not equally reliable across subjects, windows, and low-calibration settings.	Layers 0–2 and 10. Required records: DatasetRecord , WindowRecord , SplitRecord , PreprocessingRecord , LabelMapRecord , ArtifactFlagRecord , Layer1Manifest , manifests/layer1_ and PredictionRecord .	Offline public EEG benchmark. Ablations: A0 raw decoder, A4 longer-window, multi-window windowing/averaging, and ensemble baselines where available; A3 is ablated for fairness. Metrics: accuracy, balanced accuracy, subject-wise variance, low-calibration curves, session/run variability, window-length sensitivity, and matched-comparison key completeness.	Artifacts: dataset/protocol cards, decoder cards, Layer 1 ablation-readiness report, low-calibration figures, subject difficulty table, A0/A4 baseline-control tables, and results/layer2/. Supported claim: decoder reliability varies materially across public-data conditions under traceable protocol keys. Unsupported claim: real patient readiness or clinical performance. Failure interpretation: if variability is weak or matchability is incomplete, the thesis should narrow this point to measured benchmark characterization or diagnostic-only evidence rather than a universal decoder-reliability claim.

2. Calibration and uncertainty reveal high-confidence wrong predictions that accuracy alone hides.	Layers 2–3 and 10. Required records: PredictionRecord , CalibrationRecord , UncertaintyRecord , SelectivePredictionRecord , ThresholdRegistryRecord , Layer3MatchedOperatingPointTable , metric dictionary entries, and negative-result notes.	Calibration and selective-prediction evaluation. Ablations: A0 raw decoder, A1 calibration, A2 on-boarding, and A3. MatchedOperatingPointTable , selective prediction. Metrics: ECE, Brier score, NLL, reliability diagrams, entropy, margin, confidence, high-confidence wrong rate, risk-coverage curves, accepted-trial accuracy, coverage, matched coverage, matched cost, and wrong-accept rate.	Artifacts: calibration card, uncertainty/high-confidence-wrong audit, reliability diagram, selective-prediction table, matched operating-point table, threshold audit, and negative-result note. Supported claim: accuracy alone hides readiness-relevant uncertainty and wrong-accept risk when metrics and operating points are traceable. Unsupported claim: calibrated probabilities alone guarantee safe decisions. Failure interpretation: if calibration does not improve ECE/Brier/NLL or if thresholding is sufficient under matched cost/coverage, report calibration limitations or threshold sufficiency rather than hiding the negative result.
--	--	--	--

3. IHARQ evidence verification can reduce unsafe or wrong accepted decisions compared with simple thresholds and ordinary ensembles at matched coverage/cost.	Layers 3–4, the Ablation and Evaluation Architecture, the Failure Modes and Defensive Design section, and Layer 10. Required records: CalibrationRecord , UncertaintyRecord , SelectivePredictionRecord , ThresholdRegistryRecord , EvidenceQualityRecord , and IHARQDecisionRecord .	Evidence-readiness evaluation. Ablations: A2 threshold, A3 uncertainty/selective prediction, A4 longer-window/voting/averaging controls, A5 IHARQ-lite, A6 IHARQ plus on-record quality Record, and A7 IHARQ plus RegimeRisk when available. Metrics: wrong-accept rate, accepted-trial accuracy, coverage, evidence-request burden, fallback burden, unsafe-state rate, delay/cost, and matched-coverage or matched-cost comparison.	Artifacts: IHARQ decision card, reason-code distribution, wrong-accept table, matched-cost comparison, failure taxonomy card, and results/layer4_iharq/ . Supported claim: reason-coded evidence verification can improve or clarify readiness decisions under explicit cost/coverage constraints. Unsupported claim: IHARQ is clinically safe or always superior. Failure interpretation: if thresholding, longer windows, voting, averaging, or ensembles match IHARQ, narrow the claim to traceability, reason codes, controllable evidence requests, or explicit reliability-delay trade-off.
---	---	---	--

4. RegimeRisk can detect or summarize temporal changes in reliability and improve readiness decisions when used as context.	Layers 3–5, Layer 6 where used, Layer 8 under stress, and Layer 10. Required records: UncertaintyRecord , EvidenceQualityRecord , IHARQDecisionRecord , RegimeStateRecord , conditional TrustStateRecord when independent trust timelines are evaluated, ReadinessContextRecord , and optionally StressProfileRecord .	Temporal-trust evaluation. Ablations: A5 IHARQ-lite, A7 IHARQ plus RegimeRisk, and stress-conditioned A421 when used. Metrics: RegimeRisk score, regime-transition summaries, trust-score trend, trust-state trend/band where reported, stop-loss events, downstream recommendation accuracy, wrong-accept change, fallback burden, and robustness under drift/fatigue-like stress.	Artifacts: RegimeRisk timeline card, trust-state plot/card where exported, stop-loss warning log, temporal trust figure, and regime transition table. Supported claim: temporal trust signals can characterize and sometimes improve readiness decisions under changing conditions. Unsupported claim: real physiological fatigue detection or medically validated drift diagnosis. Failure interpretation: if RegimeRisk does not improve decisions, present it as a temporal trust characterization and warning tool rather than a performance-improving controller.
5. Adaptive-IHARQ can learn readiness actions from principled pseudo-labels without pretending that public EEG has real readiness labels.	Layers 3–7, the Ablation and Evaluation Architecture, the Failure Modes and Defensive Design section, and Layer 10. Required records: ReadinessContextRecord , EvidenceQualityRecord , IHARQDecisionRecord , RegimeStateRecord , conditional TrustStateRecord when independent trust-timeline policy features are used, PolicyActionRecord , and pseudo-label/cost manifests.	Adaptive-policy evaluation. Ablations: A5 IHARQ-lite, A6 evidence-quality estimator, A8 learning-to-defer, A9 supervised Adaptive-IHARQ. Metrics: policy accuracy against proxy target, cost, wrong-accept rate, coverage, action distribution, evidence burden, fallback burden, class imbalance, calibration of evidence-quality score, and matched-cost comparisons.	Artifacts: controller card, pseudo-label manifest, action-distribution figure, leakage audit, and Adaptive-IHARQ comparison table. Supported claim: supervised readiness policies can be evaluated using explicit computational pseudo-labels and cost functions. Unsupported claim: public EEG provides real readiness-action labels or real online safety labels. Failure interpretation: if Adaptive-IHARQ fails, analyze pseudo-label quality, cost sensitivity, class imbalance, feature leakage guards, and whether simpler baselines are sufficient.

6. Learning-to-defer provides a strong baseline and a meaningful routing mechanism for uncertain cases.	Layers 3–7 and 10. Required records: UncertaintyRecord , SelectivePredictionReadinessContextRecord , DeferralRecord , PolicyActionRecord , and fallback-source logs.	Deferral evaluation. Ablations: A3 selective prediction, A5 IHARQ-lite, A8 learning-to-defer, A9 Adaptive-IHARQ. Metrics: deferral date, direct-versus-defer expected cost, accepted accuracy, fallback availability, wrong-accept reduction, burden, matched-coverage performance, and cases routed to defer.	Artifacts: deferral card, deferral curve, fallback-source table, and policy comparison report. Supported claim: deferral is a strong comparator and provides an interpretable route for uncertain cases. Unsupported claim: an actual human expert or clinical fallback exists unless explicitly implemented and validated. Failure interpretation: if learning-to-defer matches Adaptive-IHARQ, treat deferral as a strong baseline and narrow adaptive-policy novelty claims.
7. Contextual bandit and RL policies can be evaluated in a declared simulated environment, especially under stress profiles.	Layers 6–8, the Ablation and Evaluation Architecture, the Implementation Pipeline section, the Failure Modes and Defensive Design section, and Layer 10. Required records: ReadinessContextRecord , PolicyActionRecord , TransitionRecord , StressProfileRecord , reward/cost manifests, controller configs, and simulation-only flags.	Simulated closed-loop and policy-learning evaluation. Ablations: A10 contextual bandit, A11 reinforcement-learning readiness policy, A12 stress-conditioned full system. Metrics: ReadinessContextRecord , reward/cost, regret or cumulative reward where appropriate, action distribution, unsafe-state rate, reward sensitivity, seed sensitivity, held-out session performance, and stress-conditioned degradation.	Artifacts: bandit card, RL controller card, reward/cost curves, simulation manifest, and controller warning card. Supported claim: sequential readiness policies can be compared under declared simulation assumptions. Unsupported claim: deployed online learning controller, real-time medical device, or real patient-safe policy. Failure interpretation: if bandit/RL overfit or underperform, preserve them as simulation-only exploratory Stretch C components and rely on C0–C2 for thesis validity.

8. StressForge-Lite exposes where each method fails under delay, dropout, weakness, noisy intent, drift, fatigue-like degradation, and combined stress.	Layers 5–8, Layer 9 where embodiment is stressed, the Ablation and Evaluation Architecture, the Failure Modes and Defensive Design section, and Layer 10. Required records: StressProfileRecord , ChangeRecord , PredictionRecord or replay references, RegimeStateRecord , PolicyActionRecord , TransitionRecord , and optional EmbodimentStateRecords	Stress-conditioned evaluation. Ablations: A0–A12 depending on implemented methods, especially A12 stress-conditioned full system. Metrics: robustness drop, stress sensitivity, wrong-accept change, fallback burden, delay sensitivity, dropout sensitivity, drift/fatigue-like degradation response, stop-loss events, and held-out stress profile performance.	Artifacts: stress card, stress-run manifest, clean-versus-stressed comparison, robustness table, and failure taxonomy. Supported claim: controlled perturbation testing reveals robustness boundaries and failure modes. Unsupported claim: StressForge-Lite is a validated physiological model of all real-world stress. Failure interpretation: if all methods fail under severe stress, report a robustness boundary rather than treating it as architectural collapse.
---	--	---	--

9. Layer 9 MyoSuite/OpenSim replay embodiment can show how readiness gating affects simulated assistive-action outcomes under proxy mappings.	Layers 7–10, Abstraction and Evaluation Architecture, Implementation Pipeline, Failure Modes and Defensive Design, and Repository Architecture. Required records/artifacts: PolicyActionRecord , ReadinessContextRecord , TransitionRecord , StressProfileRecord , EmbodimentStateRecord , EmbodimentOutcomeRecord , embodiment- linked transition pointer, command mapping manifest, simulator config/version, safety-event log or SafetyEventRecord , Layer9RewardTrace where enabled, command timeline, endpoint-error curve, Layer9EmbodimentDemoManifest , dashboard- compatible export, and optional OpenSim output manifest.	Embodiment-proxy evaluation. Ablations: A13.0–A13.8 as implemented, including no embodiment, static replay, accept-all, confidence-only, IHARQ-gated, RegimeRisk-aware, adaptive-policy, stress-conditioned, and optional OpenSim replay variants. Metrics: endpoint error, command success, command block rate, unsafe-event rate, blocked unsupported command rate, over/under-assist flags, reward terms where enabled, fallback/no-action burden, command-timeline consistency, demo-manifest completeness, and stress-conditioned embodiment outcomes.	Artifacts: embodiment card, command mapping manifest, safety-event log, Layer9RewardTrace where enabled, command timeline, endpoint-error curve, Layer9EmbodimentDemoManifest , MyoSuite/OpenSim plots or screenshots, Layer 10 read-only embodiment page/export, A13 table, and simulation-only warning. Supported claim: readiness gates affect simulated assistive-action outcomes under declared proxy mappings. Unsupported claim: clinical motor rehabilitation, real muscle stimulation safety, real assistive-device performance, real biomechanics validation, or patient efficacy. Failure interpretation: weak, incomplete, or unstable embodiment remains useful as a boundary demonstration and future-work bridge, not as clinical evidence.
--	--	--	--

10. Dashboard and cards make the architecture inspectable, reproducible, and thesis-ready.	Layer 10 plus all layers that produce validated records. Required records: DashboardMetricRecord or CardRecord or card registry entries where implemented, DashboardViewRecord or view specification where implemented, ReproducibilityManifest export manifests, figure/table lineage manifests, and warning flags.	Dashboard and reproducibility evaluation. Ablations: dashboard/card completeness checks plus A0–A13 visibility. Metrics: dashboard traceability completeness, card completeness, export determinism, source-record coverage, manifest-flag coverage, figure/table lineage completeness, and reproducibility package pass/fail.	Artifacts: dataset/protocol/decoder/calibration/IHARQ safety cards, static figures, thesis tables, export manifest, and reproducibility guide. Supported claim: the final system is inspectable, auditable, and reproducible from saved records. Unsupported claim: dashboard visuals are independent scientific evidence or hidden recomputation is acceptable. Failure interpretation: if the dashboard is incomplete or untraceable, mark affected claims as non-thesis-ready until lineage is repaired.
--	--	--	---

26.8 Layers 4–7 evaluation-lineage final-demonstration addendum

A final Stretch C demonstration may use Layers 4–7 evaluation results only when their local ownership and global ablation role are both visible. The demonstration evidence matrix must therefore include Layer 4 A5 IHARQ-lite and A6/A7 participation evidence; Layer 5 A7 temporal-trust evidence and negative controls; Layer 6 A6/A8/A9 policy-readiness evidence and A10–A13 participation evidence; and Layer 7 A10/A11 frozen-policy held-out simulation evidence plus A12/A13 participation evidence where applicable.

For each such demonstration, the final matrix must record the source layer, global ablation ID, local layer ablation ID, source-record IDs, matched-comparison key, metric-dictionary ID or snapshot, dashboard/card/export lineage, negative-result note pointer where applicable, diagnostic-only flag where matching fails, claim-boundary label, and the exact claim that is allowed. If any of these fields is missing, the final demonstration must be downgraded to diagnostic-only, optional-only, non-thesis-ready, invalid due to leakage, or invalid due to claim violation according to the failure-mode register.

26.9 Layers 8–10 evaluation-lineage final-demonstration addendum

A final Stretch C demonstration may use Layer 8 and Layer 9 evaluation results only when their local ownership and global ablation role are both visible. The demonstration evidence matrix must therefore include Layer 8 A12 StressForge robustness evidence and Layer 9 A13 embodiment-proxy evidence as locally produced records, not merely as dashboard plots or global the Ablation and Evaluation Architecture labels. Layer 10 may render and export the evidence, but it must remain a read-only consumer.

For Layer 8 A12, the final demonstration must record the

source layer, global ablation ID, local layer ablation ID, records/evaluation_outcomes/layer8_stress_evaluation_outcomes.parquet, reports/matched_comparisons/layer8_matched_stress_comparisons.csv, metrics/metric_dictionary_layer8_stress.yaml, notes/negative_results/layer8_stress_negative_results where applicable, dashboard/card/export lineage, manifests/claim_evidence/layer8_stress_claim_evidence where applicable, diagnostic-only flag, claim-boundary label, and the exact robustness or boundary-mapping claim that is allowed.

For Layer 9 A13, the final demonstration must record the source layer, global ablation ID, local layer ablation ID, records/evaluation_outcomes/layer9_embodiment_evaluation_outcomes.parquet, reports/matched_comparisons/layer9_matched_embodiment_comparisons.csv, metrics/metric_dictionary_layer9_embodiment.yaml, notes/negative_results/layer9_embodiment_negative_results where applicable, dashboard/card/export lineage, manifests/claim_evidence/layer9_embodiment_claim_evidence where applicable, diagnostic-only flag, non-clinical claim-boundary label, and the exact simulated proxy action-consequence claim that is allowed.

If any of these fields is missing, the final demonstration must downgrade the corresponding A12/A13 result to diagnostic-only, optional-only, non-thesis-ready, invalid due to leakage, or invalid due to claim violation according to the failure-mode register. The final demonstration must not use manually edited stress or embodiment tables to fill missing lineage, and it must not imply that Layer 10 dashboard rendering creates scientific evidence.

26.10 Required claim-evidence discipline

The final Stretch C system may claim the following, if and only if the required records, ablations, metric definitions, matched-comparison reports, claim-evidence entries, negative-result notes, and artifacts exist:

- it evaluates a reliability architecture for low-calibration neural-intent readiness using public EEG datasets;
- it shows that decoder accuracy alone is insufficient for readiness decisions because calibration, uncertainty, high-confidence wrong predictions, coverage, matched cost, operating-point choice, and wrong-accept risk matter;
- it compares calibrated uncertainty, selective prediction, IHARQ, RegimeRisk, Adaptive-IHARQ, learning-to-defer, contextual bandit/RL policies, stress tests, and simulated embodiment under declared protocols, including Layer 8 A12 and Layer 9 A13 local evaluation records, matched comparisons, metric dictionaries, negative-result notes where applicable, and claim-evidence entries;
- it demonstrates simulated readiness consequences and reproducibility artifacts, including cards, dashboard views, A12/A13 warning panels, matched-comparison exports, negative-result views, claim-evidence views, and export manifests;
- it identifies failure modes, negative results, robustness limits, and fallback conditions without hiding them.

The final Stretch C system must not claim:

- clinical validation, patient safety, medical-device readiness, or real BCI-FES deployment;

- implanted BCI control, real rehabilitation efficacy, real muscle stimulation safety, or assistive-device certification;
- real physiological fatigue detection unless real fatigue labels support that claim;
- real online learning safety or real deployed controller safety;
- that public EEG contains true readiness-action labels;
- that MyoSuite/OpenSim proves real clinical motor-rehabilitation performance;
- that StressForge-Lite is a validated model of all physiological or real-world stress;
- that dashboard visuals are evidence by themselves without saved records, metric definitions, matched-comparison reports, claim-evidence entries, negative-result notes, and lineage.

For Adaptive-IHARQ, learning-to-defer, contextual bandit, and RL, readiness labels are computational, proxy-derived, and cost-based. For StressForge-Lite, stress profiles are bounded evaluation perturbations. For MyoSuite/OpenSim, embodiment is a simulated proxy bridge showing how readiness gates affect simulated action outcomes under declared mappings. For the dashboard, all views and cards must be read-only projections from saved records and manifests.

26.11 Negative-result and fallback interpretation

The Final Demonstration section explicitly allows the thesis to remain scientifically valuable when advanced components do not outperform simpler baselines. The following interpretation rules prevent overclaiming and make negative results academically useful.

Weak or negative result	Required interpretation	Claim consequence
Calibration does not improve ECE, Brier score, or NLL.	Report calibration limitations, inspect probability quality, compare calibration methods, and avoid claiming calibration improvement.	Narrow C0.
Selective prediction matches IHARQ.	State that thresholding may be sufficient under the protocol; IHARQ claims shift to reason codes, evidence-control behavior, and traceability.	Narrow C1.
Longer windows dominate IHARQ.	Report the reliability-delay trade-off and quantify the cost of additional evidence duration.	Trade-off claim only.
Voting, averaging, or ensembles match IHARQ.	Acknowledge that simpler aggregation may be sufficient under that dataset/model split; avoid performance-superiority claims.	Narrow C1.
RegimeRisk does not improve downstream decisions.	Present it as temporal trust characterization, warning, and audit evidence rather than a controller that improves performance.	Diagnostic/target claim.
Adaptive-IHARQ underperforms.	Analyze pseudo-label quality, cost sensitivity, class imbalance, feature leakage, and policy-input design.	C2 weakened.

Learning-to-defer matches Adaptive-IHARQ.	Treat deferral as a strong baseline and routing mechanism; narrow adaptive-policy novelty claims.	C2 narrowed.
Bandit or RL overfits, is unstable, or underperforms.	Keep it as simulation-only exploratory evidence; do not let it affect C0–C2 thesis validity.	C3 diagnostic-only.
StressForge-Lite causes all methods to fail.	Report a robustness boundary and stress sensitivity; do not hide the failure.	Robustness-boundary evidence.
MyoSuite/OpenSim outcome is weak, unstable, or visually unconvincing.	Preserve it as a simulated embodiment boundary and future-work bridge, not as clinical evidence.	C4 optional/diagnostic.
Dashboard lacks DashboardMetricRecord/DashboardViewRecords, lineage, metric-dictionary or matched-comparison linkage, diagnostic-only status, negative-result visibility, claim-evidence linkage, or complete cards.	Mark unsupported claims non-thesis-ready until DashboardViewRecords, lineage artifacts, and exports are repaired.	Export blocked.
Leakage is detected.	Rerun affected experiments after repair; contaminated results cannot support claims.	Invalid until repaired.

This negative-result discipline is essential to the thesis. It means the architecture can produce evidence even when a sophisticated module fails, because the failure itself becomes a bounded result about when reliability control, temporal trust, adaptive policy, stress testing, or embodiment does not add value.

26.12 Connection to the implementation pipeline

A final demonstration cannot be claimed as complete unless the corresponding implementation phases from the Implementation Pipeline section have produced the required records, validation gates, and traceability artifacts.

Phase	Required phase outputs for the Final Demonstration section	Supported demonstration tier
Phase 0	Schemas, configs, repository skeleton, validation scripts, and mock records.	Development only; no final claim.
Phase 1	DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, DatasetCard, ProtocolCard, ValidationReport, Layer1Manifest, and manifests/layer1_ablation_readiness_l1_v1.json.	C0 foundation and A0–A4 matchability.

Phase 2	PredictionRecord, checkpoints, decoder cards, baseline metrics, A0 raw-decoder tables, and A4 ordinary-combination control summaries.	C0 decoder demonstration and A0/A4 evidence.
Phase 3	CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, Layer3MatchedOperatingPointTable, and negative-result notes.	C0 uncertainty/selective prediction and A1–A3 evidence.
Phase 4	IHARQDecisionRecord, evidence-quality logs, and reason-code taxonomy.	C1 IHARQ.
Phase 5	RegimeStateRecord, conditional TrustStateRecord, RegimeRisk timelines, trust-state timelines where exported, and stop-loss warning logs.	C1 temporal trust.
Phase 6	ReadinessContextRecord, EvidenceQualityRecord, PolicyActionRecord, and pseudo-label files.	C2 Adaptive-IHARQ.
Phase 7	DeferralRecord, deferral curves, and fallback-source logs.	C2 learning-to-defer.
Phase 8	TransitionRecord, ClosedLoopSessionRecord, EpisodeSummaryRecord, PolicyRolloutRecord, RewardTraceRecord, EnvironmentDiagnosticRecord, SafetyEventRecord where materialized, optional PolicyUpdateTrace, simulated session logs, and reward/cost curves.	C3 simulation environment.
Phase 9	StressProfileRecord, stress schedules, stress-run manifests, and stress cards.	C3 stress evaluation.
Phase 10	Bandit PolicyActionRecords, reward/regret curves, and bandit card.	C3 contextual bandit.
Phase 11	RL policy artifacts, RL PolicyActionRecords, and controller cards.	C3 RL policy.
Phase 12	EmbodimentStateRecord, EmbodimentOutcomeRecord, embodiment-linked TransitionRecord, command mapping manifest, safety-event logs, Layer9RewardTrace where enabled, command timelines, endpoint-error curves, Layer9EmbodimentDemoManifest, dashboard-compatible exports, and embodiment cards.	C4 Layer 9 MyoSuite/static-replay embodiment.
Phase 13	OpenSim output manifest, screenshots or plots, optional comparison report, and the corresponding Layer 9 demo-manifest linkage where exported.	C4 optional OpenSim replay.

Phase 14	DashboardMetricRecord, DashboardViewRecord, dashboard app, static figures, cards, metric dictionary exports, matched-comparison reports, ClaimEvidenceMatrix views, negative-result index, diagnostic-only status exports, and export manifest.	C0–C4 observability and evaluation-artifact lineage.
Phase 15	Claim-to-artifact traceability matrix, ClaimEvidenceMatrix, reproduction guide, final limitation map, negative-result register, and final thesis export inventory.	Final thesis readiness.

If a required phase artifact is missing, The Final Demonstration section must mark the related demonstration as diagnostic-only, optional-only, future work, or non-thesis-ready. This makes implementation status part of the scientific claim boundary rather than a hidden project-management issue.

26.13 Dashboard, cards, repository, and reproducibility linkage

Every major demonstration must have a corresponding dashboard/card/export artifact. Dashboard artifacts are not decorative; they are part of the architecture’s observability, auditability, reproducibility, and claim-safety mechanism.

The required final card and export inventory should include dataset/protocol cards, decoder cards, calibration cards, uncertainty or high-confidence-wrong audits, Layer 1 ablation-readiness reports, A0/A4 decoder-control tables, A1–A3 matched operating-point tables, metric dictionary exports, matched-comparison reports, negative-result notes, IHARQ decision cards, reason-code distributions, RegimeRisk timeline cards, controller cards, deferral cards, closed-loop environment cards, policy-rollout cards, policy-update trace cards where applicable, reward-configuration cards, safety-event cards where materialized, stress cards, embodiment cards, DashboardMetricRecord exports, DashboardViewRecord exports, failure taxonomy cards, claim-safety warning cards, claim-evidence matrix exports, reproducibility manifests, figure/table lineage manifests, and final thesis export manifests.

All dashboard figures and cards must be generated from saved records rather than hidden recomputation. Recommended repository locations include:

- reports/final_demo/demonstration_evidence_matrix.csv;
- reports/final_demo/claim_to_artifact_matrix.csv;
- reports/final_demo/claim_evidence_matrix.csv;
- reports/final_demo/metric_dictionary.csv;
- reports/final_demo/matched_comparison_report.csv;
- reports/final_demo/negative_result_register.md;
- reports/final_demo/dashboard_metric_record_index.json;
- reports/final_demo/dashboard_view_record_index.json;
- reports/final_demo/capability_tier_summary.csv;

- `reports/final_demo/negative_result_interpretation.md`;
- `reports/final_demo/limitation_map.md`;
- `reports/final_demo/reproducibility_readiness_checklist.yaml`;
- `cards/final_demo/` for synthesis cards and examiner-facing summaries;
- `figures/final_demo/` for thesis-ready final figures;
- `exports/thesis_package/` for static figures, tables, cards, manifests, and reproduction guide.

For reproducibility, The Final Demonstration section requires each final figure and table to have a source-record list, config hash, seed, code version, environment reference, generation script, export timestamp, and warning flag status. The final thesis should use these artifacts in the methodology chapter, evaluation chapter, discussion/limitation chapter, dashboard appendix, and reproducibility appendix.

26.14 Final system acceptance criteria

The final Stretch C system is thesis-ready only when the implemented tier has passed the following acceptance criteria:

1. Every claimed demonstration has a row in `demonstration_evidence_matrix.csv`.
2. Every major thesis claim has a row in `claim_to_artifact_matrix.csv`.
3. Every required record for the claimed tier exists, validates, and has a configuration hash.
4. Every comparison claim has the corresponding A0–A13 ablation evidence, or the claim is narrowed.
5. Every final A12 stress claim and A13 embodiment claim points to its local evaluation outcome file, matched-comparison report, metric dictionary, negative-result note where applicable, dashboard/export lineage, claim-evidence entry, and diagnostic-only status when matching fails.
6. Every A0–A4 comparison claim has Layer 1 ablation-readiness status, A0/A4 decoder-control evidence, A1–A3 matched operating-point evidence, metric dictionary definitions, matched-comparison reports, negative-result notes where applicable, and `ClaimEvidenceMatrix` linkage.
7. Every threshold, calibration model, pseudo-label rule, policy, stress profile, and embodiment mapping has a manifest and leakage guard.
8. Every dashboard figure, card, and table points back to saved records rather than hidden recomputation.
9. Every optional or incomplete branch is labeled core-ready, target-ready, stretch-ready, diagnostic-only, optional-only, non-thesis-ready, invalid due to leakage, or invalid due to claim violation.
10. Every negative or weak result has an interpretation in `negative_result_interpretation.md`.

11. Every clinical, deployment, real-time patient-safety, real-FES, implanted-control, therapeutic, or real rehabilitation claim is blocked by Layer 0.
12. The final repository package contains the records, configs, cards, static figures, tables, manifests, and reproduction guide required for the claimed tier.

26.15 Final consistency checklist for the Final Demonstration section

The Final Demonstration section is globally consistent with the full architecture only if the following conditions hold.

- The project name remains IHARQ BenchGuard and the maximum scope remains Stretch C.
- The architecture remains a functional graph, not a rigid one-pass stack.
- Layers 0–10 and detailed Layers 19–25 remain the source of upstream records and evidence.
- The existing ten final demonstration points are all preserved and expanded, not collapsed into vague claims.
- The C0–C4 capability tiers remain consistent with the Stretch C definition.
- The A0–A13 ablation ladder remains the main mechanism for testing component value.
- The Phase 0–15 implementation pipeline remains the readiness gate for final demonstrations.
- Public-data, offline/pseudo-online, proxy-label, simulation-only, stress-only, embodiment-only, and dashboard read-only boundaries are preserved.
- MyoSuite/OpenSim remains a simulated embodiment demo and not clinical evidence.
- Contextual bandit and RL remain simulation-bounded policy-learning components.
- Adaptive-IHARQ readiness labels remain computational, proxy-derived, and cost-based.
- StressForge-Lite remains bounded stress-generation evaluation and not a validated physiological model.
- Dashboard and cards remain read-only observability, auditability, reproducibility, and claim-safety artifacts.
- No downstream claim is allowed without record, metric, metric-dictionary, matched-comparison, ablation, artifact, negative-result, limitation, claim-evidence, and warning support.

Therefore, the final Stretch C system should demonstrate a complete and claim-safe path from public EEG evidence to ablation-ready public-data/protocol matchability, A0 raw decoder and A4 ordinary-control evidence, A1–A3 calibrated uncertainty and selective-prediction evidence, IHARQ evidence verification, temporal trust, adaptive or deferred readiness decisions, stress-conditioned evaluation, optional simulated embodiment, and dashboard-based reproducibility. The strongest version of the system is C0–C4, but the thesis remains defensible at a minimum C0–C1 level if the evidence is complete, the baselines are fair, the records are reproducible, and the limitations are explicit.

27 Final Architecture Summary

IHARQ BenchGuard under Stretch C is a record-first, claim-safe, evaluation-traceable, functional-graph reliability architecture for low-calibration neural-intent readiness. Its central architectural question is not only whether a decoder can infer a likely motor-intent label from public EEG, but whether the surrounding system can determine when that evidence is calibrated, stable, timely, regime-appropriate, policy-actionable, stress-robust, and sufficiently bounded to support simulated assistive-action decisions. The public EEG decoder is therefore the measurement source, not the whole contribution. The contribution is the architecture that surrounds decoder outputs with explicit records, uncertainty estimates, evidence verification, temporal trust monitoring, adaptive readiness policies, simulation-only consequence models, bounded stress tests, proxy embodiment, dashboard inspection, and claim-safety governance.

The Final Architecture Summary is the final synthesis section of the document. It consumes the governance rules, layer definitions, module contracts, input-output proximity map, policy-learning architecture, embodiment architecture, dashboard architecture, the cross-cutting ablation plan with its Layers 0–3 slice now integrated into the core layers, implementation pipeline, repository architecture, failure-mode defenses, and final demonstration contracts defined in the preceding sections. It does not produce a runtime record and it does not introduce a new controller. Its outputs are thesis-level synthesis artifacts: the final architecture narrative, the final claim-safe contribution statement, the final evidence-chain summary, the final implementation-readiness summary, the final evaluation and reproducibility posture, the final limitation boundary, and the examiner-facing explanation of why the architecture is coherent, implementable, evaluable, reproducible, and claim-safe.

27.1 Final system identity

The final system identity is a multi-plane reliability architecture for low-calibration neural-intent readiness. It is not a single decoder benchmark, not a policy-only system, not a clinical safety system, not an embodied rehabilitation controller, and not a restored Layer 20/P20 runtime branch. It is an architecture for transforming noisy public EEG evidence into traceable readiness evidence under declared public-data, proxy-label, pseudo-online replay, simulation-only, stress-only, and Layer 9-owned simulated-embodiment boundaries, with Layer 10 providing read-only observability over saved evidence.

The bottleneck addressed by this final summary section is architectural integration. Earlier layers define specific mechanisms: public-data ingestion, windowing, split control, decoder training, calibration, uncertainty, selective prediction, IHARQ evidence verification, RegimeRisk temporal trust, Layer 6 policy-learning ownership through P0/6.0 IHARQ-lite, evidence-quality estimation, supervised Adaptive-IHARQ, learning-to-defer, contextual bandit and reinforcement-learning policies, simulated sessions, StressForge-Lite stress profiles, MyoSuite/OpenSim proxy embodiment, and dashboard exports. The former standalone Layer 19 policy-learning dossier is retired as an independent section; its authoritative content is now owned by Layer 6. Without the final summary section, the reader could see these as disconnected modules or as an unnecessarily complicated wrapper around a classifier. The Final Architecture Summary closes that gap by showing that each mechanism has a bounded role in one evidence chain: from measured intent evidence to readiness decisions and reproducible thesis artifacts.

The seven architectural planes define the system identity:

- **Governance plane:** Layer 0 enforces claim-safety, provenance, limitation language, forbidden-claim checks, thesis boundary discipline, and the evaluation claim-evidence gate

for ablation artifacts.

- **Measurement plane:** Layers 1–2 create the public EEG protocol foundation, matched-comparison key discipline, decoder measurement spine, A0 raw-decoder evidence, and A4 ordinary-combination controls.
- **Evidence plane:** Layers 3–4 convert raw decoder scores into calibrated uncertainty, threshold/selective-prediction baselines, matched operating-point evidence, evidence-quality assessment, and reason-coded IHARQ decisions.
- **Regime plane:** Layer 5 monitors temporal reliability, changing trust, and regime-conditioned risk without claiming medical fatigue diagnosis.
- **Policy plane:** Layer 6 owns the merged policy-learning plane and studies P0/6.0 IHARQ-lite, evidence-quality estimation, supervised Adaptive-IHARQ, learning-to-defer, contextual adaptation, and sequential policy behavior under explicit cost, proxy-label, legality-mask, update-trace, and simulation boundaries.
- **Simulation and embodiment plane:** Layers 7–9 provide simulated closed-loop transitions, bounded stress profiles, and MyoSuite/OpenSim proxy embodiment as action-level demonstrations only.
- **Reporting and dashboard plane:** Layer 10 turns saved records into cards, figures, tables, manifests, warnings, and claim-to-artifact traceability files.

This identity preserves the original purpose of IHARQ BenchGuard under Stretch C while making the thesis role explicit. Calibration and uncertainty expose whether decoder confidence is trustworthy. IHARQ evidence verification determines whether evidence should be accepted, delayed, combined, rejected, escalated, or marked unsafe. RegimeRisk summarizes temporal trust and reliability change. Adaptive-IHARQ, learning-to-defer, contextual bandit, and reinforcement-learning modules study readiness decision policies under declared cost, reward, proxy-label, and simulation limits. StressForge-Lite exposes robustness limits under specified stress profiles. MyoSuite/OpenSim demonstrates only simulated assistive-action consequences through proxy mappings. Dashboard, card, manifest, and reproducibility artifacts make the system inspectable rather than opaque.

27.2 Whole-architecture flow as a functional graph

The architecture is a functional graph rather than a rigid sequential stack. Modules consume immutable or versioned records, produce new records or artifacts, and expose reason codes, configuration hashes, seeds, split references, and limitation flags. The primary feedforward chain can be summarized as:

Public EEG datasets → fixed protocol, low-calibration splits, and Layer 1 ablation-readiness manifests → baseline decoders, **PredictionRecords**, A0 raw-decoder evidence, and A4 ordinary-combination controls → calibration, **UncertaintyRecords**, and A1 calibration comparisons → A2 threshold baselines, A3 selective-prediction/uncertainty baselines, matched operating-point tables, and negative-result notes → **EvidenceQualityRecords** and **IHARQDecisionRecords** → **RegimeStateRecords** and conditional **TrustStateRecords** → **ReadinessContextRecords** → expanded **PolicyActionRecords** and **DeferralRecords** → **TransitionRecords**,

RewardTraceRecords, ClosedLoopSessionRecords, EpisodeSummaryRecords, PolicyRolloutRecords, EnvironmentDiagnosticRecords, SafetyEventRecords where materialized, and optional PolicyUpdateTrace files for simulation-authorized bandit/RL updates → StressProfileRecords under StressForge-Lite → EmbodimentStateRecords in MyoSuite/OpenSim proxy embodiment → DashboardMetricRecords, cards, figures, tables, manifests, claim-evidence matrices, metric dictionaries, matched-comparison reports, and claim-to-artifact traceability files.

This flow should be read as a graph of reusable evidence products. For example, Layer 3 outputs are consumed by IHARQ, selective-prediction baselines, learning-to-defer, Adaptive-IHARQ, dashboard audits, and ablation studies. Layer 5 RegimeRisk outputs can feed IHARQ context, adaptive policy context, stress-conditioned evaluation, and dashboard timeline cards. Layer 8 stress profiles can perturb simulated sessions, embodiment demonstrations, and robustness reports without becoming a hidden source of unmarked data manipulation.

The controlled loopback paths are also bounded:

Loopback source	Loopback target	Signal and trigger	Action, safety constraint, and evaluation logic
RegimeRisk temporal trust monitor	IHARQ and adaptive policy layer	RegimeStateRecord, conditional TrustStateRecord, trust score, regime state, stop-loss warning, or temporal instability flag triggered by reliability change.	IHARQ may adjust reason-coded readiness decisions or request more evidence; policies may include regime context or declared per-target trust-state context as a feature. The signal is not a medical fatigue diagnosis. Evaluation compares matched settings with and without RegimeRisk context.
Simulated closed-loop environment	Contextual bandit and RL policies	Reward, cost, transition, unsafe-state, delay, burden, or consequence signal stored in TransitionRecord, with optional PolicyUpdateTrace when simulation-authorized updates are enabled or audited.	Policy updates are allowed only inside declared simulation and must be separated from frozen-policy evaluation. Offline, held-out, seed-sensitive, update-trace-complete, and reward-sensitivity tests prevent claims of deployable online control.

StressForge-Lite	Simulation, policy evaluation, RegimeRisk evaluation, and embodiment replay	Declared StressProfileRecord containing delay, dropout, noisy intent, weakness, drift, fatigue-like degradation, or combined stress. Missingness flags, stale-artifact warnings, validation failures, limitation cards, and claim-safety warnings.	Stress profiles define robustness conditions, not physiological truth. Evaluation reports degradation, failure thresholds, and negative results rather than hiding stress failures.
Dashboard and cards	Human inspection, quality assurance, and thesis reporting		Dashboard feedback is human-inspection and quality-assurance feedback only. It is read-only over saved records and must not silently recompute, tune, or control the pipeline.

No loopback is arbitrary. Each loop has a scientific role and an engineering constraint: RegimeRisk tests whether temporal trust information changes readiness behavior; simulation tests policy consequences under declared reward models; StressForge-Lite tests robustness under named perturbations; dashboard feedback audits completeness and claim safety without becoming an automatic controller.

27.3 Layer integration summary

The final architecture remains coherent because each layer has a distinct responsibility and an explicit boundary.

1. **Layer 0 – claim-safety governance:** protects the thesis from clinical, deployment, patient-safety, real-time stimulation, therapeutic, and real rehabilitation overclaiming. It defines the allowed claim space, limitation language, provenance tags, dashboard warnings, and controller/embodiment safety cards that bound every later layer.
2. **Layer 1 – public-data and protocol anchor:** creates the public-data foundation through dataset records, window records, label maps, preprocessing records, artifact flags, split records, and a Layer 1 manifest. Its purpose is to prevent the architecture from relying on hidden data preparation or unstable protocol choices.
3. **Layer 2 – decoder and baseline measurement spine:** trains and evaluates public EEG decoders under fixed low-calibration splits and baseline conditions. Its output is a measurement spine of **PredictionRecords**, not a complete readiness controller.
4. **Layer 3 – calibration, uncertainty, and selective prediction:** converts raw decoder scores into calibrated probabilities, uncertainty summaries, selective-prediction decisions, and threshold registries. It exposes high-confidence wrong cases and establishes baselines that IHARQ and policy layers must beat fairly.
5. **Layer 4 – IHARQ evidence verification:** performs reason-coded evidence verification. It consumes calibrated uncertainty, evidence-quality signals, thresholds, and optional context to produce **IHARQDecisionRecords** that accept, reject, defer, request more evidence, escalate, or mark evidence unsafe under explicit rules.

6. **Layer 5 – RegimeRisk temporal trust monitoring:** adds temporal trust context by summarizing reliability change, instability, stop-loss events, and regime state over sequences. It is a reliability-risk monitor under declared proxy conditions, not a medical fatigue detector.
7. **Layer 6 – adaptive readiness policy layer:** owns the merged policy-learning specification formerly detailed in Layer 19. It stages P0/6.0 rule-based IHARQ-lite, P1/6.1 evidence-quality estimation, P2/6.2 supervised Adaptive-IHARQ, P3/6.3 learning-to-defer, P4/6.4 contextual bandit, and P5/6.5 RL readiness policy under proxy-label, cost-model, legality-mask, `PolicyUpdateTrace`, and simulation-only boundaries. It separates selective prediction, deferral, supervised policy learning, bandit adaptation, and RL control rather than collapsing them into one concept.
8. **Layer 7 – simulated closed-loop readiness environment:** provides replay-based and simulation-only transition dynamics for testing how readiness actions affect reward, delay, burden, unsafe-state rate, update/frozen-mode separation, and downstream simulated consequences. It produces `TransitionRecords`, `ClosedLoopSessionRecords`, `EpisodeSummaryRecords`, `PolicyRolloutRecords`, `RewardTraceRecords`, `EnvironmentDiagnosticRecords`, `SafetyEventRecords` where materialized, and optional `PolicyUpdateTrace` files, and never claims real patient control.
9. **Layer 8 – StressForge-Lite:** defines bounded stress profiles for delay, dropout, noisy intent, weakness, drift, fatigue-like degradation, and combined stress. It exposes when methods fail and supplies named conditions for robustness tests rather than serving as a validated physiological stress model.
10. **Layer 9 – MyoSuite/OpenSim embodiment demo:** connects readiness-gated intent to simulated assistive-action consequences using L9.1–L9.13 proxy adapters, command mappings, safety gates, embodiment-state and embodiment-outcome records, safety-event logs, `Layer9RewardTrace` files where enabled, command timelines, endpoint-error curves, and `Layer9EmbodimentDemoManifest` files. It demonstrates action-level consequence visualization only and does not validate real BCI-FES deployment or rehabilitation biomechanics.
11. **Layer 10 – dashboard, cards, and reproducibility layer:** converts saved records, including Layer 9 dashboard-compatible embodiment bundles where implemented, into dashboard views, metric cards, figures, tables, run manifests, artifact manifests, warnings, filters, and thesis exports. It is read-only over records and functions as an observability, auditability, reproducibility, and claim-safety layer rather than as a hidden result generator or simulator controller.

The Final Architecture Summary is the final place where these layers are shown as one system. The layer view explains responsibility, the functional-graph view explains data and decision flow, and the repository/module view explains how implementation artifacts remain testable and replaceable.

27.4 Evidence and record-contract summary

The architecture is scientifically defensible because it is built around explicit records rather than hidden notebook variables. The Final Architecture Summary consumes the full upstream record chain and summarizes it as a set of evidence contracts. The primary inputs are the

records and artifacts produced by Layers 0–10 and the detailed architecture sections on module structure, input-output proximity, policy learning, embodiment, dashboard, ablation, implementation, repository design, failure modes, and final demonstrations. Auxiliary inputs include configuration files, run manifests, dataset metadata, seed manifests, environment references, validation logs, dashboard exports, and artifact manifests. Evaluation-only inputs include ablation definitions, matched-comparison settings, stress schedules, reward/cost definitions, held-out split references, and negative-result interpretation files. Proxy and stretch inputs include pseudo-label manifests, simulated reward functions, StressForge-Lite profiles, and MyoSuite/OpenSim proxy mappings.

The layer does not create a runtime schema. Its expected output artifacts are thesis artifacts: final architecture narrative, final contribution statement, final evidence-chain summary, final implementation-readiness summary, final reproducibility posture, final limitation map, final claim-to-artifact traceability summary, and examiner-facing synthesis text.

The major record families are:

- **Public-data and protocol records:** `DatasetRecord`, `WindowRecord`, `SplitRecord`, `PreprocessingRecord`, `LabelMapRecord`, `ArtifactFlagRecord`, `DatasetCard`, `ProtocolCard`, `ValidationReport`, `Layer1Manifest`, and `layer1_ablation_readiness_l1_v1.json`. These preserve dataset provenance, protocol choices, window definitions, split boundaries, preprocessing decisions, label mappings, artifact handling, leakage guards, card-level limitations, validation status, and matched-comparison readiness.
- **Decoder and calibration records:** `PredictionRecord`, `CalibrationRecord`, `UncertaintyRecord`, `SelectivePredictionRecord`, and `ThresholdRegistryRecord`. These expose decoder behavior, calibration quality, uncertainty signals, retained-versus-rejected predictions, threshold selection rules, and high-confidence wrong cases.
- **Evidence and temporal-trust records:** `EvidenceQualityRecord`, `IHARQDecisionRecord`, `RegimeStateRecord`, and conditional `TrustStateRecord`. These make readiness decisions reason-coded, temporally contextualized, trust-state traceable where exported, and auditable.
- **Policy and simulation records:** `ReadinessContextRecord`, `DeferralRecord`, expanded `PolicyActionRecord`, `TransitionRecord`, and optional `PolicyUpdateTrace`. These support P0/6.0–P5/6.5 Layer 6 policy comparisons under explicit proxy-label, legality-mask, reward, cost, update-trace, and simulation boundaries.
- **Stress and embodiment records:** `StressProfileRecord` and `EmbodimentStateRecord`. These make stress conditions and simulated embodiment consequences reproducible, inspectable, and bounded.
- **Reporting and reproducibility records:** `DashboardMetricRecord`, cards, figures, tables, Layer 6 policy-stage/context-ablation/action-legality/cost-reward/dependency/claim-boundary matrices, run manifests, artifact manifests, reproducibility manifests, figure/table lineage files, validation logs, warning files, and claim-to-artifact traceability files. These connect thesis statements to saved evidence and expose missingness, stale artifacts, or claim-safety violations.

These record contracts matter because they preserve provenance, prevent leakage and hidden tuning, make ablations possible, make dashboard exports traceable, allow runs to be reproduced, and support claim-safety inspection. They also make development mockable: downstream IHARQ, RegimeRisk, policy, simulation, stress, embodiment, and dashboard modules

can be tested against validated mock records before full upstream implementations are complete. Mocking is acceptable only when the mock record carries a mock/proxy flag and cannot be used to support final performance claims.

27.5 Capability tiers and stoppable thesis value

Stretch C is intentionally designed to be stoppable. The strongest version is C0–C4, but the thesis does not collapse if later stretch components are incomplete. Each tier has a coherent evidence value and a bounded claim scope.

Tier	Implemented capability	Thesis value and boundary
C0	Public EEG low-calibration reliability benchmark with calibration, uncertainty, selective prediction, and reproducible records.	Supports claims about decoder reliability, calibration quality, high-confidence wrong cases, uncertainty, risk-coverage, and split-safe benchmarking. It does not support readiness-policy or embodiment claims.
C1	Adds IHARQ evidence verification and RegimeRisk temporal trust monitoring.	Supports claims about reason-coded evidence verification, temporal trust characterization, and reliability-aware decision gating under offline or pseudo-online replay. It does not claim medical fatigue detection or real-time safety.
C2	Adds supervised Adaptive-IHARQ and learning-to-defer under proxy-label and cost-model discipline.	Supports claims about learned readiness decisions, deferral, matched-cost comparisons, and pseudo-label sensitivity. It does not claim that public EEG contains real clinical readiness labels.
C3	Adds simulated closed-loop policy learning, StressForge-Lite robustness tests, contextual bandit, and reinforcement-learning evaluation.	Supports simulation-bounded claims about sequential policy behavior, reward/cost trade-offs, robustness under declared stress profiles, and held-out stress behavior. It does not claim deployed online learning control.
C4	Adds MyoSuite/OpenSim proxy embodiment and dashboard-driven final integration.	Supports claims about simulated action-level consequences, embodiment-state traceability, dashboard observability, cards, manifests, and claim-to-artifact export. It does not prove real rehabilitation biomechanics, therapeutic efficacy, or clinical BCI-FES readiness.

This tiered structure protects thesis validity. If C4 is incomplete, C0–C3 can still produce a valid reliability, uncertainty, evidence-verification, temporal-trust, and simulation-policy thesis. If C3 is incomplete, C0–C2 can still produce a strong public EEG reliability and readiness-policy benchmark. If C2 is incomplete, C0–C1 remain defensible as a calibrated, reason-coded,

temporally aware low-calibration reliability architecture. The final claim must always match the highest completed, validated, and reproducible tier.

27.6 Evaluation and ablation synthesis

The architecture is evaluated by evidence quality, decision behavior, temporal stability, policy consequences, stress robustness, embodiment traceability, dashboard completeness, and reproducibility. It is not judged by a single decoder-accuracy number. The ablation ladder protects the thesis from the criticism that the system is merely a complicated wrapper around a classifier.

The evaluation program connects each architectural claim to baselines, ablations, metrics, and failure interpretation:

- Public-data and protocol evidence must first be checked for matched-comparison readiness using Layer 1 dataset, subject/session, trial/window, split, preprocessing, label-map, budget, seed, and protocol-version identifiers.
- Decoder performance must be measured across datasets, subjects, sessions, windows, and low-calibration budgets using accuracy, balanced accuracy, subject-wise variance, low-calibration curves, split-safe protocol summaries, and A0 raw-decoder evidence tables.
- Calibration and uncertainty must be evaluated using A1 calibration deltas, ECE, Brier score, NLL, reliability diagrams, entropy, margin, confidence, calibration curves, high-confidence-wrong audits, metric-dictionary entries, and negative-result notes.
- Selective prediction and threshold baselines must be evaluated using A2 threshold comparisons, A3 uncertainty/selective-prediction comparisons, matched coverage, matched cost, risk-coverage, retained accuracy, retained error, coverage, rejection rate, accepted-trial error, and high-confidence wrong rate.
- IHARQ must be compared against confidence thresholding, entropy rejection, margin rejection, longer windows, voting, averaging, ensembles, IHARQ-lite, and evidence-quality variants under matched coverage or matched cost; ordinary longer-window/voting/averaging/ensemble controls are the A4 baseline family rather than evidence-verification logic.
- RegimeRisk must be evaluated under temporal instability, drift-like behavior, fatigue-like degradation, dropout, changing trust, and stress-conditioned sequences. If it does not improve downstream decisions, it remains valuable as a temporal trust characterization and warning mechanism rather than a performance-improving controller.
- Adaptive-IHARQ and learning-to-defer must be evaluated under matched cost, matched coverage, pseudo-label sensitivity, class imbalance checks, leakage guards, and explicit fallback-source assumptions.
- Contextual bandit and RL policies must be evaluated only inside declared simulated environments using reward/cost curves, unsafe-state rate, action distribution, seed sensitivity, reward sensitivity, held-out sessions, and held-out stress profiles.
- StressForge-Lite must test delay, dropout, weakness, noisy intent, drift, fatigue-like degradation, and combined stress. Negative stress results should become failure-mode evidence, not hidden implementation failures.

- MyoSuite/OpenSim/static-replay embodiment must be evaluated as simulated action-level consequence and traceability only, using Layer 9 command mapping manifests, embodiment-state and embodiment-outcome records, embodiment-linked transition pointers, safety-event logs, `Layer9RewardTrace` files where enabled, command timelines, endpoint-error curves, `Layer9EmbodimentDemoManifest` files, plots, screenshots, dashboard-compatible exports, and embodiment cards.
- Dashboard quality must be evaluated through traceability, completeness, reproducibility, stale-artifact detection, missingness visibility, warning propagation, card generation, static export quality, and claim-safety inspection.

Success means that each supported claim has record-backed evidence, a fair baseline, a matched comparison where appropriate, a metric-dictionary definition, a negative-result or diagnostic-only interpretation where applicable, a reproducible configuration, a Layer 0 claim-evidence boundary, and a bounded interpretation. Failure does not invalidate the architecture automatically. It narrows the claim, exposes a limitation, or identifies a future-work boundary. This is a core thesis feature: negative results, weak improvements, or stress failures are preserved as evidence about reliability limits.

27.7 Implementation and repository closure

The architecture is implementable because the implementation pipeline is staged, split-safe, validation-gated, and tied to repository artifacts. Phase 0 establishes repository layout, schemas, configs, mock records, validators, seed discipline, and continuous validation. Phases 1–3 establish the public-data, decoder, calibration, uncertainty, and selective-prediction spine. Phases 4–7 establish IHARQ, RegimeRisk, Adaptive-IHARQ, and learning-to-defer. Phases 8–11 establish simulated closed-loop evaluation, StressForge-Lite, contextual bandit, and reinforcement-learning policies. Phases 12–13 establish Layer 9 MyoSuite/OpenSim/static-replay embodiment outputs and optional OpenSim replay. Phase 14 establishes Layer 10 dashboard, cards, static exports, and reporting artifacts as read-only consumers of saved records. Phase 15 establishes final thesis integration, claim-to-artifact traceability, reproduction guides, and final limitation maps.

The repository architecture keeps the system from becoming a tangled classifier script. The separation between `configs/`, `src/` modules, `records/`, `models/`, `figures/`, `cards/`, `dashboard_app/`, `notebooks/`, `thesis_tables/`, `logs/`, `reports/`, `manifests/`, `exports/`, and `tests/` makes each module independently testable, replaceable, mockable, and reproducible; the added `manifests/layer1_ablation_readiness_l1_v1.json` path records whether Layers 0–3 ablation comparisons are matchable or diagnostic-only. Configuration files define datasets, splits, windows, calibration methods, thresholds, pseudo-label rules, costs, rewards, stress profiles, embodiment mappings, dashboard exports, and claim-safety settings. Validators check record schemas, split leakage, threshold provenance, calibration split separation, missing artifacts, stale exports, and forbidden claims.

The final implementation-readiness criterion is traceability from every thesis figure, table, card, and claim back to source records, configuration hashes, seeds, code version, environment reference, validation logs, and limitation flags. If an artifact cannot be traced, it must not support a final claim. If an implementation phase is incomplete, the associated demonstration must be labeled diagnostic-only, optional-only, future work, non-thesis-ready, invalid due to leakage, or invalid due to claim violation.

27.8 Quality attributes and failure-mode defenses

The final architecture’s quality attributes are enforced through mechanisms, not slogans.

Quality attribute	Architectural mechanism
Claim safety	Enforced through Layer 0 governance, provenance tags, limitation statements, forbidden-claim checks, dashboard warnings, controller cards, embodiment cards, and final claim-to-artifact traceability.
Traceability	Enforced through canonical records, run IDs, subject/session/window references, source-record links, configuration hashes, artifact manifests, figure/table lineage files, and dashboard metric records.
Reproducibility	Enforced through repository structure, versioned configs, seeds, environment assumptions, validation scripts, record manifests, saved exports, reproduction guides, and static dashboard outputs.
Modularity	Enforced through record-first module boundaries, independent validators, mockable records, replaceable decoders, replaceable policy modules, optional embodiment adapters, and separate dashboard generation.
Leakage control	Enforced through SplitRecords , calibration splits, selection splits, threshold registry provenance, held-out sessions, held-out stress profiles, validation gates, pseudo-label manifests, and explicit invalidation rules.
Decision interpretability	Enforced through IHARQ reason codes, evidence-quality records, deferral reasons, policy-action metadata, RegimeRisk timeline cards, stress-condition labels, and dashboard failure taxonomy cards.
Evaluation fairness	Enforced through Layer 1 matched-comparison keys, <code>layer1_ablation_readiness_l1_v1.json</code> , matched coverage, matched cost, A0 raw-decoder baselines, A1 calibration deltas, A2 threshold baselines, A3 selective-prediction/uncertainty baselines, A4 longer-window/voting/averaging/ensemble controls, reward sensitivity, seed sensitivity, metric dictionaries, claim-evidence gates, and negative-result interpretation files.
Simulation boundary discipline	Enforced through simulation-only flags, reward/cost manifests, transition records, stress-profile declarations, controller warning cards, embodiment proxy-mapping manifests, and non-claim boundaries.
Dashboard observability	Enforced through read-only access to saved records, completeness checks, stale-artifact detection, missingness visibility, cards, figures, tables, export manifests, and claim-safety warnings.
Stoppability	Enforced through C0–C4 capability tiers, Phase 0–15 exit gates, required artifact inventories, tier-specific claim scopes, and explicit fallback contribution statements.

Maintainability and extensibility	Enforced through repository separation, schema versioning, module interfaces, tests, configuration-driven experiments, optional/stretching branches, and mockable upstream artifacts.
-----------------------------------	---

The major failure-mode defenses from the architecture are therefore integrated into the final system: public-data limitations are handled by provenance and non-claim boundaries; split leakage is handled by split records and validators; overfitted thresholds are handled by threshold registries and held-out selection; calibration failure is handled by reporting ECE, Brier score, NLL, reliability diagrams, and high-confidence-wrong audits; IHARQ overcomplexity is handled by fair ablations and matched comparisons; RegimeRisk overinterpretation is blocked by temporal-trust wording; policy-learning overclaiming is blocked by proxy-label and simulation-only flags; stress profiles are declared rather than hidden; embodiment remains proxy-only; dashboard artifacts are read-only over saved records; and incomplete components are labeled rather than silently treated as complete.

27.9 Claim-safety and non-claim boundaries

The final architecture can support claims about low-calibration public EEG reliability analysis, calibration and uncertainty as reliability evidence, selective prediction and threshold baselines, IHARQ-style evidence verification under offline or pseudo-online replay constraints, RegimeRisk temporal trust characterization, learned readiness decisions under proxy-label and cost-model assumptions, learning-to-defer and selective-prediction comparisons, contextual bandit and RL policy behavior in declared simulation, robustness behavior under StressForge-Lite stress profiles, simulated embodiment consequences under proxy mappings, and dashboard-based traceability, reproducibility, and claim-safety inspection.

The final architecture must not claim clinical validation, medical-device readiness, real patient safety, real BCI-FES deployment, therapeutic efficacy, real-time stimulation control, real implanted BCI controller validation, or real rehabilitation efficacy. It must not claim that public EEG contains true clinical readiness labels. It must not claim that proxy mappings in MyoSuite/OpenSim prove real rehabilitation biomechanics. It must not claim that contextual bandit or RL policies are clinically deployable controllers. It must not claim that RegimeRisk is a medical fatigue detector or that StressForge-Lite is a validated physiological stress model.

This boundary discipline is not a weakness of the architecture. It is the mechanism that makes the thesis defensible. The system can make strong engineering and scientific claims because it states exactly what evidence exists, how it was produced, which assumptions were used, which artifacts support it, which comparisons were run, which failures occurred, and which clinical or deployment interpretations remain outside scope.

27.10 Layers 4–7 evaluation-lineage closure

The MERGE_RECORD_22 Layers 4–7 propagation closes an important consistency gap in the final architecture summary. The final evidence chain now makes explicit that A5–A11 and Layers 4–7-supported A12–A13 claims are not supported by aggregate ablation labels alone. They are supported only when the producing layers preserve local evaluation records, matched-comparison keys, metric definitions, negative-result notes, dashboard/card/export lineage, and Layer 0 claim boundaries.

In the final architecture, Layer 4 contributes A5 IHARQ-lite evidence and participates in A6/A7

through evidence-quality and RegimeRisk-aware IHARQ comparisons. Layer 5 contributes A7 temporal-trust evidence through matched regime-aware versus regime-unaware comparisons and shuffled/random risk negative controls. Layer 6 contributes A6 evidence-quality, A8 learning-to-defer, A9 supervised Adaptive-IHARQ, A10 contextual-bandit, A11 RL, and A12/A13 participation evidence through matched cost, coverage, reward, and stress comparisons plus controller-card lineage. Layer 7 contributes A10/A11 claim-supporting evidence through frozen-policy held-out simulation reports and participates in A12/A13 only through explicitly matched stress or embodiment-ready records.

The architecture's final fairness and reproducibility posture therefore includes Layer 1 matched-comparison keys, A0–A4 baseline and calibration/selective-prediction evidence, A5–A11 local evaluation-outcome records, A12 Layer 8 stress evaluation records, A13 Layer 9 embodiment evaluation records, A12–A13 participation lineage, metric dictionaries or snapshots, negative-result notes, diagnostic-only labels, and claim-evidence matrix entries. This reinforces the central identity of IHARQ BenchGuard: a reliability and evidence-governance framework around low-calibration public EEG decoders, not an overclaimed clinical controller or a dashboard-only reporting exercise.

27.11 Layers 8–10 evaluation-lineage closure

The MERGE_RECORD_22 Layers 8–10 propagation closes the final consistency gap for the stress, embodiment, dashboard, repository, pipeline, failure-mode, and final-demonstration parts of the architecture. The final evidence chain now makes explicit that A12 and A13 are not supported by aggregate ablation labels, dashboard plots, or Ablation and Evaluation Architecture summary text alone. They are supported only when the producing layers preserve local evaluation records, matched-comparison keys, metric definitions, negative-result notes, dashboard/card/export lineage, and Layer 0 claim boundaries.

In the final architecture, Layer 8 contributes A12 StressForge robustness evidence through StressProfileRecord, clean/stressed source-record links, Layer 8 EvaluationOutcomeRecords, records/evaluation_outcomes/layer8_stress_evaluation_outcomes.parquet, reports/matched_comparisons/layer8_matched_stress_comparisons.csv, metrics/metric_dictionary_layer8_stress.yaml, notes/negative_results/layer8_stress_negative_ where applicable, dashboard/export lineage, and manifests/claim_evidence/layer8_stress_claim_eviden Layer 8 may also participate in A13 when stress-conditioned embodiment outcomes are evaluated, but the embodiment consequence claim remains Layer 9-owned.

Layer 9 contributes A13 embodiment-proxy evidence through EmbodimentStateRecord, EmbodimentOutcomeRecord, embodiment-linked transition pointers, safety-event logs, command timelines, endpoint-error curves, Layer9RewardTrace files where enabled, Layer9EmbodimentDemoManifest files, Layer 9 EvaluationOutcomeRecords, records/evaluation_outcomes/layer9_embodiment_evaluation_outcomes.parquet, reports/matched_comparisons/layer9_matched_embodiment_comparisons.csv, metrics/metric_dictionary_layer9_embodiment.yaml, notes/negative_results/layer9_embodiment_ where applicable, dashboard/export lineage, and manifests/claim_evidence/layer9_embodiment_claim_ev Layer 9 evidence remains simulation-only and non-clinical even when the visual demonstration is successful.

Layer 10 closes the loop by consuming A12/A13 artifacts read-only through StressForge, embodiment, ablation/comparison, card/export, warning, and reproducibility views. It may expose missing lineage, diagnostic-only status, negative results, and claim-evidence gaps, but it must not repair missing stress or embodiment artifacts by hidden recomputation. The final summary

therefore preserves the central identity of IHARQ BenchGuard: a reliability and evidence-governance framework around low-calibration public EEG decoders, strengthened by bounded stress and proxy embodiment evaluation, not an overclaimed clinical controller or dashboard-only reporting exercise.

27.12 Final thesis contribution statement

The final contribution of IHARQ BenchGuard under Stretch C is not a single decoder, not a single threshold, not a single adaptive policy, and not a clinical system. Its contribution is a disciplined architecture for transforming noisy low-calibration neural-intent evidence into calibrated, verified, regime-aware, policy-mediated, stress-tested, simulation-bounded, embodiment-demonstrated, and reproducibly reported readiness evidence.

In thesis terms, IHARQ BenchGuard contributes a claim-safe functional graph for studying whether neural-intent evidence is not only likely, but also trustworthy enough, stable enough, timely enough, and sufficiently bounded to justify a simulated readiness action. Policy learning is not a detached late-stage add-on: it is owned by Layer 6, while the former Layer 19 dossier is retained only as retired/merged traceability. The system's defensibility comes from its explicit records, split-safe protocols, calibrated uncertainty, reason-coded evidence verification, temporal trust monitoring, matched ablations, proxy-label discipline, simulation-only policy evaluation, bounded stress testing with Layer 8 A12 evaluation lineage, proxy embodiment with Layer 9 A13 evaluation lineage, dashboard observability, and reproducible claim-to-artifact traceability. The reader should remember the final architecture as a reliability and evidence-governance framework around low-calibration public EEG decoders, not as an overclaimed clinical controller.

28 Selected Technical References and Tool Anchors

This final section is the technical-anchor and citation-governance register for IHARQ BenchGuard under Stretch C. It is not a generic bibliography, not a related-work chapter, and not an attempt to prove the architecture by citation alone. Its architectural purpose is to map the external papers, tool documents, benchmark frameworks, simulator references, implementation libraries, methodology anchors, and reproducibility practices that support the preceding layers of the system.

The bottleneck addressed by this section is **traceability between architectural claims and external foundations**. Without this section, the thesis could contain strong module descriptions but weak evidence about why a dataset, benchmark, decoder family, uncertainty metric, deferral method, policy-learning branch, simulator adapter, dashboard artifact, or repository practice was selected. This section protects the thesis against three common criticisms: first, that references were added as decoration rather than as implementation anchors; second, that tools and papers were cited without version, license, or reproducibility obligations; and third, that references were used to imply clinical, real-time, patient-safety, rehabilitation, or deployment claims that the implemented architecture does not actually test.

The thesis role of this section is therefore to support the central contribution of IHARQ BenchGuard: a record-first, claim-safe, functional-graph architecture for studying low-calibration neural-intent readiness using public EEG evidence, calibrated uncertainty, IHARQ evidence verification, RegimeRisk temporal trust monitoring, adaptive readiness policy learning, simulated closed-loop behavior, StressForge-Lite stress testing, optional MyoSuite/OpenSim proxy embodiment, and dashboard-based reproducibility. The section connects those architectural components to declared technical anchors while preserving the non-negotiable evidence bound-

ary: references can justify methods, tools, metrics, interfaces, and evaluation protocols, but they do not strengthen claims beyond the artifacts produced by this system.

The final thesis may expand this section into a formal bibliography, related-work chapter, implementation-environment appendix, or reproducibility appendix. In this architecture document, however, the section is a **reference-to-layer traceability artifact**. It specifies what must be cited, what must be versioned, what must be treated as optional or simulation-only, and what must be kept outside the thesis claim boundary.

28.1 Architectural responsibility and boundaries of this section

Technical responsibility. The Selected Technical References and Tool Anchors section records the external anchors that support the architecture. It defines which papers, tools, documentation pages, benchmark frameworks, simulators, and reproducibility practices are associated with each architectural layer, record family, evaluation metric, implementation branch, and thesis claim boundary. It also defines the minimum metadata that must be captured before final submission: exact paper metadata, official documentation URLs, software versions, installation sources, licenses or access notes, configuration files, repository folders, tests, run manifests, and dashboard export references.

In scope. This layer covers governance and claim-safety anchors; public EEG dataset and benchmark anchors; preprocessing and protocol anchors; decoder and baseline anchors; calibration, uncertainty, selective prediction, and risk-coverage anchors; IHARQ and evidence-verification conceptual anchors; RegimeRisk and temporal trust anchors; Adaptive-IHARQ, learning-to-defer, contextual-bandit, and reinforcement-learning anchors; simulated closed-loop and reward-design anchors; StressForge-Lite robustness anchors; MyoSuite/OpenSim/MuJoCo simulator anchors; dashboard and card anchors; repository, schema, and reproducibility anchors; and failure-mode and negative-result reporting anchors.

Out of scope. This layer does not introduce new runtime algorithms, change the layer numbering, modify canonical record schemas, replace the evaluation design, add new clinical claims, define a medical-device pathway, or convert simulated embodiment into real rehabilitation validation. It also does not require every candidate tool listed here to be implemented. Candidate and optional anchors remain candidate or optional until the implementation pipeline and repository manifests prove that they were actually used.

Core, supporting, optional, stretch, and future-work parts. The core part of this section is the anchor taxonomy, layer-to-anchor matrix, claim-boundary discipline, and citation-cleanup requirements. Supporting parts include tool-card and method-card schemas, record traceability, repository linkage, dashboard-card linkage, and failure-mode linkage. Optional parts include candidate implementation tools such as MNE-Python, pyRiemann, Braindecode, Streamlit, Dash, MLflow, Weights & Biases, DVC, and other tools that are useful only if adopted by the actual implementation. Stretch-only or simulation-only parts include MyoSuite, OpenSim, MuJoCo, Gymnasium-style simulator APIs, contextual-bandit adaptation, and reinforcement-learning policy evaluation. Future-work parts include clinical, real-time, rehabilitation, FES, implanted-control, and patient-facing anchors, which must not be cited as evidence for this thesis unless a future project actually performs the required validation.

28.2 Layer input and output contracts

The Selected Technical References and Tool Anchors section is documentation-facing, but it still follows the record-first style of the architecture. Its inputs and outputs are explicit so that

the reference section remains tied to implementation rather than becoming an unstructured bibliography.

Input class	Received artifacts	Role in the Selected Technical References and Tool Anchors section
Primary architecture inputs	Layer 0–10 definitions, Cross-Layer Functional Module Architecture, Detailed Input-Output Proximity Map, Layer 6 policy-learning specification and retired Layer 19 trace note, ablation architecture, implementation pipeline, repository architecture, failure-mode section, final demonstration section, and final architecture summary.	Define which architectural components need external anchors and which claims must be bounded.
Auxiliary implementation inputs	Repository folder plan, configuration families, run-manifest design, schema validators, test plans, dashboard-card definitions, export manifests, and planned environment files.	Determine which tools require version, license, installation, dependency, and reproducibility metadata.
Metadata inputs	Candidate citation metadata, tool documentation URLs, software versions, dataset access notes, release dates, license notes, access dates, seeds, configuration hashes, and environment assumptions.	Convert references into reproducibility-ready anchor cards rather than informal citations.
Evaluation-only inputs	Metric definitions, ablation ladder A0–A13, low-calibration curves, cross-subject and within-subject protocols, risk-coverage curves, stress sensitivity outputs, policy simulation outputs, embodiment proxy metrics, dashboard QA outputs, and negative-result files.	Identify which references support evaluation design and which outputs become thesis evidence.

Proxy and stretch inputs	MyoSuite tasks, OpenSim replay or comparison notes, MuJoCo dependency notes, Gymnasium-style simulator interface assumptions, contextual-bandit reward models, RL reward models, and StressForge-Lite stress profiles.	Keep simulation-only, proxy-only, and stretch-only anchors clearly separated from public-data evidence and thesis-core claims.
--------------------------	--	--

Reference-section output	Contents	Downstream consumer
ReferenceAnchorMatrix	Documentation artifact mapping layers, modules, records, metrics, evaluation branches, repository folders, and claim boundaries to paper and tool anchors. It is not a runtime record unless implemented later.	Thesis architecture chapter, implementation appendix, ablation chapter, examiner review, repository documentation.
ToolAnchorCard	Tool or documentation card containing tool name, category, architectural role, affected records, required or optional status, versioning obligation, license/access note, reproducibility role, fallback, and claim boundary.	Repository docs/, environment manifests, dashboard reproducibility card, final thesis appendix.
MethodAnchorCard	Method card containing method family, reference family, layer, problem solved, IHARQ inputs and outputs, metrics affected, ablation relevance, limitations, claim boundary, and implementation implication.	Evaluation chapter, ablation design, claim-evidence matrix, method section.
CitationCleanupChecklist	Pre-submission checklist for replacing placeholders with exact bibliographic metadata, DOIs, arXiv IDs, official URLs, versions, licenses, access dates, and final BibTeX.	Final thesis writing, repository release, reproducibility package.
ClaimBoundaryWarnings	Warnings that public EEG, simulator, dashboard, stress, bandit, RL, and deferral references do not imply clinical validation or deployment readiness.	Layer 0 governance, dashboard warning fields, final contribution statement, examiner-facing limitations.

AnchorFailureRegister	Failure-mode notes for missing, stale, misused, unversioned, unsupported, or overclaimed anchors.	Failure Modes and Defensive Design section, repository tests, final limitation discussion.
------------------------------	---	--

28.3 Internal module structure of the Selected Technical References and Tool Anchors section

The Selected Technical References and Tool Anchors section is implemented as a set of documentation and governance modules. These modules are not part of the online decision path, but they are part of the thesis evidence path.

Module	Purpose	Inputs and processing logic	Outputs, consumers, and evaluation
Anchor Taxonomy Register	Defines anchor types and prevents references from becoming a flat bibliography.	Consumes the full layer list, module families, record families, and evaluation branches. It classifies anchors as governance, dataset, benchmark, preprocessing, decoder, calibration, IHARQ, RegimeRisk, policy-learning, simulation, stress, embodiment, dashboard, reproducibility, or failure-mode anchors.	Produces the anchor taxonomy and supports the layer-to-anchor matrix. Evaluated by coverage: every major architecture component must have at least one declared anchor family or an explicit “needs citation cleanup” note.
Layer-to-Anchor Mapper	Maps each layer and major detailed section to paper, tool, benchmark, simulator, method, and claim-safety anchors.	Consumes Layer 0–10, the Layer 6 policy-learning specification and retired Layer 19 trace note, ablation, implementation, repository, failure-mode, and final-demonstration descriptions. It assigns anchor categories, candidate anchors, supported records, evaluation roles, status labels, and claim boundaries.	Produces the ReferenceAnchorMatrix . Consumers are the thesis architecture chapter, implementation plan, ablation chapter, and examiner-facing traceability appendix. Failure mode: a layer without anchors becomes under-justified.

Tool-Anchor Card Builder	Converts tool references into reproducibility-aware tool cards.	Consumes candidate tool names, official documentation sources, version information, license/access notes, installation routes, repository locations, tests, and fallback plans.	Produces ToolAnchorCards for MOABB, MNE-Python if used, pyRiemann if used, scikit-learn, PyTorch/Braindecode if used, MyoSuite, OpenSim, MuJoCo if used, dashboard tools, schema tools, and testing tools. Evaluated by version completeness and reproducibility readiness.
Method-Anchor Card Builder	Converts method references into module-aware method cards.	Consumes decoder, calibration, uncertainty, selective-prediction, IHARQ, RegimeRisk, deferral, bandit, RL, stress, embodiment, and dashboard method families. It records the problem solved, IHARQ inputs and outputs, metrics affected, ablation relevance, limitations, and claim boundary.	Produces MethodAnchorCards . Consumers are the method chapter, ablation architecture, evaluation chapter, and claim-evidence matrix. Failure mode: method is implemented but not defensibly anchored.

Record and Artifact Traceability Mapper	Links anchors to the canonical record families.	Consumes record schemas and planned output directories. It checks whether each anchor supports DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, DatasetCard, ProtocolCard, ValidationReport, Layer1Manifest, layer1_ablation_readiness_l1_v1.json, PredictionRecord, CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, EvidenceQualityRecord, IHARQDecisionRecord, RegimeStateRecord, TrustStateRecord where independent trust timelines are exported, ReadinessContextRecord, DeferralRecord, PolicyActionRecord, TransitionRecord, StressProfileRecord, EmbodimentStateRecord, and DashboardMetricRecord where applicable.	Produces record-to-anchor traceability tables. Evaluated by schema coverage and by whether dashboard cards can expose the relevant anchor metadata.
Claim-Safety Citation Gate	Prevents references from expanding the evidence boundary.	Consumes Layer 0 claim-safety rules, final contribution statement, simulation-only flags, optional-only flags, diagnostic-only flags, and failure-mode warnings. It checks every reference against what the implemented system actually evaluates.	Produces ClaimBoundaryWarnings and final citation-cleanup requirements. Consumers are the thesis limitation section, dashboard cards, and final defense narrative. Failure mode: claim violation due to overclaiming.

28.4 Reference and tool-anchor selection rules

The following rules govern inclusion of anchors in the final thesis and repository documentation.

1. Prefer primary sources, official tool documentation, peer-reviewed method papers, benchmark framework papers, stable project documentation, and institutionally acceptable software citations.
2. Use arXiv or preprint references only when clearly marked as preprint or when no peer-reviewed or official documentation source is more appropriate.
3. Use official documentation for tools such as MOABB, MyoSuite, OpenSim, MuJoCo, Gymnasium-style APIs, dashboard frameworks, and software packages.
4. Do not cite a tool unless the architecture actually uses it, plans to use it, or explicitly marks it as optional, candidate, stretch-only, simulation-only, or documentation-only.
5. Do not allow a reference to expand the scientific claim boundary beyond the evidence produced by the system.
6. Do not cite clinical, FES, rehabilitation, implanted-control, patient-safety, or medical-device references in a way that implies clinical validation unless the architecture actually performs that validation, which Stretch C does not.
7. Every anchor must have a declared architectural use: method, metric, interface, dataset, benchmark, simulator, evaluation protocol, record family, repository practice, dashboard artifact, or claim boundary.
8. Every implemented tool anchor must include exact version, installation source, documentation URL or citation, license or access note, environment dependency, related repository folder, related configuration file, related run-manifest entry, related record family, related tests, and known limitations.
9. Every method anchor must specify which module, metric, ablation, figure, table, or card it supports.
10. Every simulator anchor must be labeled simulation-only, proxy-only, optional-only, or stretch-only where relevant.
11. Every reference must be removed, demoted, or relabeled if the corresponding tool, method, metric, or branch is not implemented.

28.5 Anchor taxonomy

The reference register uses the following taxonomy. A single paper or tool may appear in more than one family, but every appearance must have an architectural reason.

Anchor family	Architectural function	Typical status and boundary
---------------	------------------------	-----------------------------

Governance and claim-safety anchors	Support scope control, non-claim boundaries, model-card style discipline, dataset-card style discipline, and limitation reporting.	Core, documentation-facing, claim-safety only. They do not create experimental evidence.
Public EEG dataset and benchmark anchors	Support dataset access, dataset provenance, public BCI benchmarking, within-subject and cross-subject protocol discipline, and low-calibration benchmarking.	Core for C0. They support public-data reliability claims, not clinical readiness claims.
EEG preprocessing and protocol anchors	Support filtering, epoching, artifact flags, channel handling, label maps, split construction, leakage checks, and preprocessing manifests.	Core if implemented. They must be versioned and tied to <code>DatasetRecord</code> , <code>WindowRecord</code> , <code>SplitRecord</code> , and <code>PreprocessingRecord</code> .
Classical decoder and baseline anchors	Support CSP plus LDA/SVM, logistic baselines, calibration splits, threshold baselines, and reproducible baseline comparisons.	Core or target. They support baseline measurement, not readiness safety by themselves.
Riemannian and tangent-space anchors	Support covariance-based features, tangent-space baselines, low-calibration comparisons, and ablation alternatives to deep decoders.	Target or optional depending on implementation. They must be fairly matched to decoder splits and calibration protocols.
Deep EEG decoder anchors	Support EEGNet, ShallowConvNet, compact CNN baselines, deep-model record generation, and model-card documentation.	Target or optional. EEGNet is preserved as an existing compact deep decoder anchor.
Optional compact SSL or foundation-model anchors	Support optional representation-learning branches only if explicitly implemented.	Optional or future work. They must not be cited as core evidence if not run.
Calibration, uncertainty, selective prediction, and risk-coverage anchors	Support ECE, Brier score, NLL, reliability diagrams, entropy, margin, high-confidence wrong audits, selective prediction, threshold registries, and risk-coverage curves.	Core for Layer 3. They support reliability evidence, not clinical safety.

IHARQ and evidence-verification anchors	Support evidence aggregation, verification logic, reason-coded decisions, request-evidence, combine, fallback, unsafe, and wait behaviors.	Core for Layer 4. IHARQ is an architectural evidence-verification analogy, not a literal wireless-protocol or clinical safety guarantee.
RegimeRisk and temporal trust anchors	Support drift, non-stationarity, temporal instability, regime-state monitoring, rolling reliability, and trust-state timelines.	Target or core for C1. They support temporal reliability characterization, not medical fatigue diagnosis.
Adaptive-IHARQ and policy-learning anchors	Support evidence-quality estimation, readiness pseudo-labels, learning-to-defer, limited expert prediction settings, contextual bandits, and reinforcement-learning policy comparison.	Target, stretch, or simulation-only. They support simulated or proxy policy experiments only.
Simulated closed-loop and reward-design anchors	Support transition records, cost models, reward sensitivity, delay costs, fallback costs, and policy behavior under declared simulation.	Stretch and simulation-only. They do not imply deployed controller validity.
StressForge-Lite anchors	Support declared synthetic stress profiles: delay, dropout, weakness, noisy intent, drift, fatigue-like degradation, and combined stress.	Target or stretch. Stress profiles are controlled perturbations, not physiological diagnosis.
MyoSuite, OpenSim, MuJoCo, and simulator-adapter anchors	Support optional embodiment, proxy command mapping, simulator adapters, musculoskeletal environment interaction, replay, and visualization.	Stretch-only, optional-only, simulation-only, proxy-only. They do not support real rehabilitation efficacy claims.
Dashboard, cards, reporting, and visualization anchors	Support read-only inspection, DatasetCard, ModelCard, ReliabilityCard, RegimeCard, ControllerCard, StressCard, EmbodimentCard, export manifests, and warning displays.	Core for Layer 10 if implemented. They support traceability and reporting, not new experimental evidence.

Reproducibility, configuration, manifest, schema, and repository-tooling anchors	Support environment capture, seeds, hashes, data manifests, schema validation, testing, export manifests, and repeatable runs.	Core. They support rerun and audit claims only if versioned and tested.
Failure-mode, defensive-design, negative-result, and limitation-reporting anchors	Support leakage invalidation, missingness reporting, negative-result handling, overclaim prevention, and fallback contribution statements.	Core claim-safety family. They prevent unsupported conclusions.

28.6 Layer-to-anchor matrix

The following matrix maps the major architectural layers and detailed sections to their required technical-anchor categories. Candidate anchors are deliberately described as anchor families unless exact citations must be preserved. Final thesis submission must replace placeholder families with exact references and versions.

Layer or section	Required anchor categories	Supported records, modules, or artifacts	Evaluation or implementation role and claim boundary	Status
Layer 0: Claim-Safety and Scope Governance	Claim-safety, dataset-card/model-card discipline, limitation reporting, negative-result handling.	Claim registry, non-claim boundary text, warning fields, claim-evidence matrix.	Ensures references do not imply clinical validation, patient safety, real FES safety, implanted control, therapeutic efficacy, or real-world rehabilitation.	Core
Layer 1: Public-Data and Protocol Anchor	Public EEG dataset documentation, MOABB benchmark anchor, preprocessing documentation, split-protocol, validation-report, card-generation, ablation-readiness, and leakage-control anchors.	DatasetRecord, WindowRecord, SplitRecord, PreprocessingRecord, LabelMapRecord, ArtifactFlagRecord, DatasetCard, ProtocolCard, ValidationReport, Layer1Manifest, and layer1_ablation_readiness_anchor.	Supports public-data reproducibility, within-subject and cross-subject evaluation, low-calibration protocol discipline, validation/reporting readiness, and matched A0-A4 readiness; does not create true clinical readiness labels.	Core

Layer 2: Decoder and Baseline Measurement Spine	CSP/LDA/SVM anchors, Riemannian/tangent- space anchors if implemented, EEGNet and ShallowConvNet anchors if implemented, compact SSL/foundation- style anchors where feasible, MOABB baseline discipline.	PredictionRecord, ModelRegistryRecord, BaselineMetricRecord, LowCalibrationCurveRecord, SubjectProfileRecord, EnsembleControlRecord, model configuration files, decoder manifests, ModelCard, and BenchmarkCard.	Supports baseline decoder evaluation, low-calibration reporting, subject-profile analysis, A0/A4 controls, and ablations; does not establish readiness safety without calibration and governance layers.	Core/target
Layer 3: Calibration, Uncertainty, and Selective Prediction	Calibration metrics, uncertainty metrics, ECE, Brier score, NLL, reliability diagrams, entropy, margin, risk-coverage, selective prediction.	CalibrationRecord, UncertaintyRecord, SelectivePredictionRecord, ThresholdRegistryRecord, ReliabilityCard.	Supports low-calibration records, high-confidence wrong audits, and threshold baselines; does not imply clinical safety.	Core
Layer 4: IHARQ Evidence Verification	Evidence verification, canonical action-token decision logic, evidence-quality estimation, request/combine behavior, fallback and unsafe-state auditing, selective- prediction comparisons, en- semble/threshold comparison anchors, and traceable matched ablations.	EvidenceQualityRecord, IHARQDecisionRecord, CombinedEvidenceRecord, FallbackDecisionRecord, UnsafeEventRecord, IHARQAblationRecord, IHARQTraceRecord, Layer 4 EvaluationOutcomeRecord, reason-code tables, matched-comparison reports, negative-result notes, and IHARQ reliability-card exports.	Supports IHARQ vs threshold vs ensemble comparisons. IHARQ remains an architectural analogy for evidence verification, not a literal communication- system claim or safety certificate; IHARQ cards must not be collapsed into Layer 3 calibration-only ReliabilityCard semantics.	Core

Layer 5: RegimeRisk Temporal Trust Monitoring	Drift, non-stationarity, rolling reliability, temporal trust, regime detection, monitoring, trust-state tracking, and warning-card anchors.	RegimeStateRecord , conditional TrustStateRecord , RegimeCard , TrustStateCard where exported, temporal trust timelines.	Supports regime timelines and temporal reliability characterization; does not diagnose fatigue or physiological state.	Target/core
Layer 6: Adaptive Readiness Policy Layer	Adaptive-IHARQ, evidence-quality estimation, readiness pseudo-label construction, learning-to-defer, limited expert prediction, selective prediction, contextual decision anchors.	ReadinessContextRecord , DeferralRecord , PolicyActionRecord , ControllerCard .	Supports supervised policy and deferral comparisons under pseudo-label and cost-model assumptions; does not route real clinical experts unless real expert labels exist.	Target/stretch
Layer 7: Simulated Closed-Loop Readiness Environment	Simulator interface, transition-model, reward-design, contextual-bandit, RL policy-evaluation, policy-update trace, frozen-policy evaluation, and cost-sensitivity anchors.	TransitionRecord , optional PolicyUpdateTrace , reward manifests, simulation configs, ControllerCard , policy-update trace card.	Supports simulation-only policy behavior, seed sensitivity, reward sensitivity, closed-loop proxy evaluation, and update/frozen separation; not deployed control.	Stretch/simulation- only
Layer 8: StressForge-Lite Stress Generator	Robustness, perturbation, delay, dropout, drift, weakness, noisy-intent, and fatigue-like degradation anchors.	StressProfileRecord , stress-pairing manifests, StressCard .	Supports declared synthetic stress sensitivity tests and matched stress/no-stress comparisons; not physiological diagnosis.	Target/stretch

Layer 9: MyoSuite/OpenSim Embodiment Demo	MyoSuite documentation, OpenSim project documentation, MuJoCo if required, Gym-style API anchors, simulator-adapter references.	EmbodimentStateRecord , EmbodimentOutcomeRecord , embodiment-linked TransitionRecord , SafetyEventRecord or safety-event log, Layer9RewardTrace where enabled, command timeline, endpoint-error curve, Layer9EmbodimentDemoManifest , adapter manifests, embodiment proxy-mapping files, EmbodimentCard .	Supports optional simulated proxy embodiment and re- play/visualization under Layer 9 ownership; not real biomechanics validation or rehabilitation evidence.	Stretch- only
Layer 10: Dashboard, Cards, and Reproducibility Layer	Dashboard framework documentation, visualization anchors, card/reporting discipline, schema validation, manifests, testing, reproducibility tools.	DashboardMetricRecord , including policy-update, embodiment, and reward-trace filters where applicable; DatasetCard , ModelCard , ReliabilityCard , RegimeCard , ControllerCard , StressCard , EmbodimentCard , ReproducibilityCard , export manifest, and Layer 9 embodiment evidence export.	Supports read-only inspection, traceability, stale-artifact detection, Layer 9 embodiment- bundle validation, and reproducibility reporting; dashboard cards do not create new evidence.	Core/target
Cross-Layer Functional Module Architecture	Functional graph, interface contracts, schema validation, module-boundary documentation, dependency- injection/testing anchors.	Module families, canonical record contracts, mockable artifacts, integration tests.	Supports modular implementation and feedfor- ward/loopback inspection; does not change scientific scope.	Core
Detailed Input-Output Proximity Map	Data-lineage, provenance, schema, and record-linkage anchors.	All canonical records and proximity maps.	Supports traceability from input artifacts to output artifacts and helps detect leakage, missingness, and stale records.	Core

Retired Layer 19 policy-learning dossier / authoritative Layer 6 policy-learning specification	Learning-to-defer, limited expert prediction, contextual bandit, RL, reward design, simulation evaluation, pseudo-label limitations, policy-stage/action-legality/cost-sensitivity matrices, and update-trace discipline.	ReadinessContextRecord , DeferralRecord , expanded PolicyActionRecord , optional PolicyUpdateTrace , TransitionRecord , ControllerCard , policy-matrix card.	Supports Layer 6 policy-learning ablations and simulation-only comparisons; the retired Layer 19 note is traceability only and does not define a separate runtime layer.	Stretch/simulation-only
Ablation and Evaluation Architecture	Baseline evaluation, matched ablations, A0–A13 ladder, risk-coverage, calibration, temporal trust, policy, stress, embodiment, dashboard QA anchors.	Ablation tables, metric exports, figure manifests, claim-evidence matrix.	Ensures references justify comparisons and metrics rather than serving as decorations.	Core
Implementation Pipeline for Stretch C	Environment, versioning, execution order, staged artifacts, configuration, manifest, and test anchors.	Phase gates, configs, run manifests, logs, seeds, hashes.	Supports repeatable implementation and stoppability. If a branch is incomplete, it must be labeled non-thesis-ready or optional-only.	Core
Recommended Repository Architecture	Repository layout, schema validation, testing, documentation, dashboard export, and citation-table anchors.	configs/ , src/ , records/ , reports/ , tests/ , docs/ .	Ensures the reference section maps to implementation locations and tests.	Core

Failure Modes and Defensive Design	Leakage, miscalibration, overfitting, missing anchor, simulator overclaim, policy leakage, stale artifact, and negative-result reporting anchors.	AnchorFailureRegister warnings, invalidation flags, limitation files.	Converts reference misuse into an explicit failure mode: invalid due to leakage, invalid due to claim violation, diagnostic-only, stretch-only, or simulation-only.	Core
Final Demonstration and Final Architecture Summary	Thesis contribution, evidence boundary, dashboard exports, cards, figures, final claim matrix, and limitation anchors.	Final contribution statement, demonstration checklist, final cards and export package.	Ensures final claims remain tied to actual artifacts and not to references alone.	Core

28.7 Preserved and strengthened existing anchors

The original section contained seven selected anchors. They are preserved here and strengthened into architecture-facing entries. Exact bibliographic metadata, version numbers, official URLs, DOIs, access dates, and final thesis citation formatting must still be verified during citation cleanup.

Anchor	Architectural role	Relevant layers, records, and artifacts	Claim boundary and implementation note
MyoSuite documentation [1]	Simulator/tool anchor for optional musculoskeletal environments exposed through a gym-style interaction pattern.	Layer 9, Layer 7 if simulator interface is reused, EmbodimentStateRecord proxy-mapping manifest, EmbodimentCard , simulation configs.	Stretch-only, optional-only, simulation-only, proxy-only . It supports simulated embodiment and command-gating demonstrations, not clinical rehabilitation validation or real biomechanics claims. Implementation must record MyoSuite version, dependency stack, task name, license/access note, and fallback if unavailable.

OpenSim project documentation [2]	Simulator/documentation anchor for musculoskeletal modeling, dynamic simulation, replay, comparison, or explanatory embodiment context.	Layer 9 MyoSuite/OpenSim Embodiment Demo, optional OpenSim replay adapter, EmbodimentStateRecord , EmbodimentOutcomeRecord where generated, replay manifests, Layer9EmbodimentDemoManifest , EmbodimentCard , and Layer 10 read-only evidence exports.	Optional/stretch-only. It supports proxy embodiment comparison or replay documentation, not proof of patient outcomes. Implementation must state whether OpenSim is actively used, used only for documentation, or omitted.
Mozannar and Sontag learning-to-defer anchor [3]	Method anchor for learning-to-defer formulation, deferral losses, and comparison between model action and expert or fallback action.	Layer 6, retired Layer 19 trace note, DeferralRecord , PolicyActionRecord , ControllerCard , ablation ladder.	Supports deferral-method design, not clinical expert routing unless real expert labels exist. If no real expert is present, the expert channel must be described as proxy, oracle, simulated, or cost-model based.
Hemmer et al. limited-expert-prediction anchor [4]	Method anchor for constrained deferral when expert predictions are limited, incomplete, or costly.	Layer 6, retired Layer 19 trace note, DeferralRecord , pseudo-label manifests, ControllerCard , evaluation under limited feedback.	Supports limited deferral comparisons under declared assumptions. It does not justify undisclosed human-in-the-loop clinical claims. Implementation must declare which labels are available and which are simulated or proxy.
Contextual bandit anchor [5]	Method anchor for adaptive action or threshold selection under context and changing environment.	Layer 6, Layer 7, retired Layer 19 trace note, ReadinessContextRecord , expanded PolicyActionRecord , optional PolicyUpdateTrace , TransitionRecord , reward manifests, ControllerCard , policy-update trace card.	Stretch and simulation-only unless the implementation is explicitly limited to offline logged evaluation. It supports adaptive policy experiments, not deployed patient-facing control; update-enabled runs require traceability and frozen-policy evaluation separation.

MOABB anchor [6]	Benchmark/tool anchor for public BCI algorithm benchmarking, dataset handling, and reproducible evaluation discipline.	Layer 1, Layer 2, <code>DatasetRecord</code> , <code>SplitRecord</code> , <code>Layer1Manifest</code> , <code>PredictionRecord</code> , <code>DatasetCard</code> , <code>ModelCard</code> .	Core benchmark anchor for public EEG. It supports BCI benchmarking and public-data reproducibility, not proof of real assistive-control safety. Implementation must record MOABB version, dataset names, split settings, and access constraints.
EEGNet anchor [7]	Compact deep EEG decoder anchor and target deep baseline reference.	Layer 2, optional Layer 3 calibration outputs, <code>PredictionRecord</code> , <code>ModelRegistryRecord</code> , <code>ModelCard</code> , decoder configs.	Supports compact EEG decoding baseline. EEGNet alone does not provide readiness safety, temporal trust, or clinical deployability. Implementation must record architecture variant, training configuration, seeds, calibration split, and comparison protocol.

28.8 Tool-anchor card schema

Every implemented or thesis-visible tool must have a tool-anchor card. Candidate tools may remain in a planned or optional state, but they must not be described as required unless the repository and run manifests prove that they were used.

Field	Required content
Tool or documentation name	Official tool name, documentation title, or project name.
Category	Dataset/benchmark, preprocessing, decoder, calibration, policy, simulation, embodiment, dashboard, schema, repository, testing, or documentation-only.
Primary architectural role	The exact role in IHARQ BenchGuard and the layer or module it supports.
Relevant layer or module	Layer number, detailed architecture section, and module family.
Records or artifacts affected	Canonical records, cards, manifests, figures, tables, configs, or repository folders influenced by the tool.
Required or optional status	Core, target, optional, candidate, stretch-only, simulation-only, proxy-only, documentation-only, or replacement candidate.

Expected implementation use	How the tool is called, configured, or inspected in the system.
Versioning requirement	Exact package version, release, commit hash where relevant, installation source, and environment dependency.
License/access note	License, data-use constraint, documentation license, access date, dataset permissions, or institutional access note.
Reproducibility role	Which manifests, seeds, hashes, exported records, tests, or cards must include the tool metadata.
Failure or fallback if unavailable	Mock, omit, replace, label optional-only, label non-thesis-ready, or disable the branch.
Claim-safety boundary	What claims the tool supports and what claims it must not be used to imply.

Candidate tool cards include MOABB for public EEG benchmarking and dataset access; MNE-Python if used for EEG preprocessing and data handling; pyRiemann if used for covariance and tangent-space baselines; scikit-learn if used for LDA, SVM, logistic models, calibration helpers, or classical baselines; PyTorch and/or Braindecode if used for EEGNet, ShallowConvNet, or deep EEG baselines; MyoSuite for optional simulated embodiment; OpenSim for optional replay, comparison, or documentation; MuJoCo if required by MyoSuite; Gymnasium or an OpenAI Gym-style interface if used for simulation wrapper structure; pandas, NumPy, SciPy, Parquet/Arrow, JSON/YAML, and schema-validation tools if used for records and manifests; Streamlit, Dash, Plotly, or equivalent if used for dashboard implementation; DVC, MLflow, Weights & Biases, or lightweight custom manifests only if actually used or clearly marked optional; and pytest or equivalent testing frameworks for repository validation.

28.9 Method-anchor card schema

Every implemented method family must also have a method-anchor card. Method cards differ from tool cards because they justify an algorithmic or evaluation choice rather than a software dependency.

Field	Required content
Method or concept	Name of the method family, metric, evaluation protocol, or conceptual anchor.
Primary reference or reference family	Exact paper, benchmark paper, official documentation, or placeholder citation family to be resolved.
Architectural layer	Layer or detailed section where the method is used.
Problem solved	The bottleneck addressed in IHARQ BenchGuard terms.
Inputs and outputs	Canonical input records and output records affected by the method.
Internal processing implication	How the method changes processing, decision logic, thresholds, policies, records, or cards.
Metrics or evaluation artifacts influenced	Figures, tables, dashboards, ablations, reliability diagrams, risk-coverage curves, regime timelines, stress curves, or embodiment metrics.

Baseline or ablation relevance	Whether the method is a baseline, target method, ablation component, stress test, stretch branch, or comparison method.
Known limitations	Assumptions, data limitations, proxy-label limitations, calibration limits, simulator limits, or external-validity threats.
Claim boundary	What the method supports and what it does not support.
Implementation implication	Required configs, seeds, manifests, tests, versioning, and failure/fallback handling.

Method-anchor candidates include CSP plus LDA/SVM baselines; Riemannian and tangent-space baselines; EEGNet and ShallowConvNet; calibration and uncertainty metrics such as ECE, Brier score, NLL, reliability diagrams, entropy, margin, and high-confidence wrong rate; selective prediction and risk-coverage evaluation; IHARQ-style evidence verification, with explicit wording that communication-theory inspiration is an analogy and not a literal wireless-protocol claim; evidence-quality estimation; temporal drift, non-stationarity, and RegimeRisk monitoring; learning-to-defer; limited expert prediction; contextual-bandit adaptation; reinforcement-learning policy evaluation in a declared simulated environment; StressForge-Lite robustness testing; simulated embodiment and proxy command mapping; and dataset, model, reliability, regime, policy, stress, embodiment, and reproducibility cards.

28.10 Record and artifact traceability map

The reference system must remain compatible with the document’s canonical records. The Selected Technical References and Tool Anchors section may define documentation artifacts such as `ReferenceAnchorMatrix`, `ToolAnchorCard`, and `MethodAnchorCard`, but these are documentation artifacts, not runtime records, unless the implementation later chooses to serialize them.

Record family	Anchor families that support it	Required traceability note
<code>DatasetRecord</code>	Public EEG dataset documentation, MOABB, dataset access notes.	Dataset source, version/access note, subject/session/task labels, license or data-use boundary, and <code>DatasetCard</code> references.
<code>WindowRecord</code>	Preprocessing, epoching, windowing, EEG protocol anchors.	Window length, overlap, channel selection, filter settings, event alignment, and leakage constraints.
<code>SplitRecord</code>	Benchmark protocol, leakage-control, within-subject/cross-subject evaluation anchors.	Split seed, subject/session partition, calibration split, selection split, held-out split, and invalidation rules.
<code>PreprocessingRecord</code>	MNE-Python or equivalent preprocessing anchors if used, signal-processing documentation.	Filter, resampling, channel, artifact, and preprocessing config provenance.

LabelMapRecord	Dataset documentation, motor-imagery task definitions, pseudo-label limitations.	Label mapping must not be interpreted as true clinical readiness.
ArtifactFlagRecord	Artifact handling, missingness, signal-quality, and preprocessing anchors.	Artifact flags must be available to RegimeRisk, IHARQ, dashboard, and failure analysis.
Layer1Manifest	MOABB/dataset/preprocessing/anchors and repository manifests.	Complete data-protocol provenance for reproducibility and leakage audit.
PredictionRecord	Decoder anchors: CSP/LDA/SVM, Riemannian, EEGNet, ShallowConvNet, or implemented alternatives.	Model identity, seed, logits/probabilities, trial/window ID, split ID, and model-card traceability.
CalibrationRecord	Calibration metrics, ECE, Brier, NLL, reliability-diagram anchors.	Calibration split, method, metric definitions, curve export, and reliability-card link.
UncertaintyRecord	Entropy, margin, confidence, disagreement, uncertainty-estimation anchors.	Uncertainty features must be aligned to prediction IDs and available to IHARQ and policies.
SelectivePredictionRecord	Selective prediction, reject option, risk-coverage anchors.	Coverage, risk, threshold, reject/wait/fallback interpretation, and matched comparison protocol.
ThresholdRegistryRecord	Threshold selection, validation split, registry and tuning anchors.	Threshold provenance, tuning metric, split isolation, and held-out evaluation boundary.
EvidenceQualityRecord	IHARQ evidence verification, calibration, uncertainty, consistency, and signal-quality anchors.	Evidence features, reason codes, quality score, and limitations.
IHARQDecisionRecord	IHARQ, selective prediction, evidence aggregation, fallback, unsafe, and request-evidence anchors.	Action, reason code, consumed evidence, threshold registry reference, and comparison baseline.
RegimeStateRecord	Drift, non-stationarity, temporal trust, rolling metrics, RegimeRisk anchors.	Regime state, trigger, time index, warning label, optional trust-state links, and non-diagnostic boundary.
TrustStateRecord	Trust allocation, per-target reliability, trusted-set evolution, temporal trust anchors.	Trust target type/ID, trust score, trend, band, update source, source regime link, and non-clinical trust-proxy boundary.

ReadinessContextRecord	Adaptive-IHARQ, policy-learning, context construction, deferral, bandit, and RL anchors.	Context features must exclude future labels and record proxy-label assumptions.
DeferralRecord	Learning-to-defer and limited expert prediction anchors.	Defer target, proxy expert/oracle/fallback definition, cost model, and available labels.
PolicyActionRecord	Adaptive policy, contextual bandit, RL, reward-design anchors.	Policy action, context ID, source evidence IDs, action-space version, legality mask, cost-model ID, reward-model ID, claim boundary, and simulation-only flag where applicable.
PolicyUpdateTrace	Contextual-bandit/RL simulation-update, reward-design, disabled-update audit, and frozen-policy evaluation anchors.	Policy ID, context ID, action, reward, next-state ID, update step, exploration mode, frozen-policy flag, simulator ID, seed, config hash, and simulation-only boundary.
TransitionRecord	Simulated closed-loop, RL environment, reward, delay, and simulator-interface anchors.	State, action, next state, reward, done flag, seed, and environment configuration.
StressProfileRecord	StressForge-Lite stress and robustness anchors.	Stress type, intensity, pairing, random seed, synthetic-stress warning, and matched-control link.
EmbodimentStateRecord	MyoSuite, OpenSim, MuJoCo, simulator adapter, proxy mapping anchors, safety-event logging, reward-trace, command-timeline, endpoint-error, and demo-manifest anchors.	Proxy command, simulated state, simulator version, task ID, gate result, safety-event linkage, reward-trace linkage where enabled, command-timeline ID, endpoint-error-curve ID, demo-manifest ID, and embodiment boundary warning.
DashboardMetricRecord	Dashboard, card, visualization, export manifest, schema, and reproducibility anchors.	Metric source record, card reference, stale-artifact status, version metadata, and read-only guarantee.

28.11 Dependency logic

Hard dependencies. The reference register cannot be finalized until the implemented layers, canonical records, evaluation metrics, repository layout, and thesis claim boundaries are known. At minimum, The Selected Technical References and Tool Anchors section depends on Layer 0 claim-safety rules, Layer 1 public-data protocol, Layer 2 decoder spine, Layer 3 calibration

outputs, Layer 4 IHARQ decision records, Layer 5 RegimeRisk states, Layer 6 policy records, Layer 7 simulation records if implemented, Layer 8 stress records if implemented, Layer 9 embodiment records if implemented, and Layer 10 dashboard/export records.

Soft dependencies. Tool cards for MNE-Python, pyRiemann, Braindecode, Streamlit, Dash, Plotly, MLflow, Weights & Biases, DVC, Parquet/Arrow, schema validators, or pytest depend on the actual implementation choices. If a tool is not used, it must be removed or marked optional/candidate.

Optional and stretch dependencies. MyoSuite, OpenSim, MuJoCo, Gymnasium-style APIs, contextual bandits, and reinforcement-learning environments are optional, stretch-only, simulation-only, or proxy-only unless actually implemented and explicitly bounded.

Conceptual dependencies. Calibration anchors depend conceptually on valid probabilistic predictions and split-safe validation. IHARQ depends conceptually on calibrated uncertainty, evidence-quality features, and reason-coded decisions. RegimeRisk depends conceptually on temporal ordering and causal availability of past evidence. Policy learning depends conceptually on declared cost models, pseudo-label construction, and context leakage prevention. Embodiment depends conceptually on proxy command mappings and simulator gates.

Mockable dependencies. During development, optional simulator outputs can be mocked with synthetic `TransitionRecords` and `EmbodimentStateRecords`; dashboard records can be mocked with small schema-valid samples; policy-learning records can be mocked with explicitly labeled proxy labels; and tool-card metadata can be staged as candidate entries. Mocked artifacts must be marked as mocked, diagnostic-only, or non-thesis-ready until replaced by real implementation outputs.

28.12 Feedforward and loopback behavior

The Selected Technical References and Tool Anchors section receives information feedforward from the entire architecture. It does not alter runtime decisions, but it feeds documentation, implementation, evaluation, and dashboard artifacts.

Loopback source	Loopback target	Signal and trigger	Action and safety constraint	Evaluation
Reference cleanup review	Implementation pipeline and repository architecture	Missing version, missing license, missing official URL, missing citation metadata, or unused tool reference.	Add metadata to environment/run manifests, demote tool to candidate, or remove it. Do not keep decorative references.	Completeness of tool cards and manifest consistency.

Claim-safety citation gate	Layer 0 and final contribution statement	A reference appears to imply clinical validation, patient safety, real FES safety, real-time deployment, implanted control, or rehabilitation efficacy.	Add warning, revise wording, remove reference, or label it future work. References cannot expand evidence boundaries.	Claim-evidence matrix and final defense check-list.
Anchor-to-record audit	Canonical records and dashboard cards	Anchor is cited but no record, metric, card, or artifact consumes it.	Create a traceability note, connect to a real artifact, or remove/demote the anchor.	Record coverage and dashboard card completeness.
Ablation and evaluation review	Layer-to-anchor matrix and method cards	A method, metric, or comparison is implemented without a method anchor, or an anchor exists without an evaluation role.	Add a method card, add an ablation reference, or mark the method optional/non-thesis-ready.	A0–A13 ablation coverage and evaluation reproducibility.
Simulator boundary review	Layer 7, Layer 8, Layer 9, and dashboard warnings	Simulator, stress, bandit, or RL anchors are described without simulation-only, proxy-only, or stretch-only labeling.	Add explicit labels and dashboard warnings; prevent clinical or deployment wording.	Failure-mode audit and Embodiment-Card/ControllerCard warning fields.

Each loopback has an engineering purpose: it prevents broken traceability, stale or decorative citations, missing tool metadata, leakage through improperly described protocols, and unsupported claim expansion. No loopback changes a runtime prediction or readiness action directly.

28.13 Evaluation and ablation linkage

The Selected Technical References and Tool Anchors section is evaluated by whether it makes the technical foundations of the architecture inspectable and defensible. Its success is not measured by the number of citations. Its success is measured by traceability coverage, version completeness, claim-boundary compliance, and alignment between references, records, metrics, and thesis claims.

Evaluation area	Anchor linkage required	Success criterion and failure interpretation
Baseline decoder evaluation	Dataset, MOABB, preprocessing, CSP/LDA/SVM, Riemannian, EEGNet, ShallowConvNet, and model-card anchors.	Every baseline table and low-calibration curve has dataset, split, decoder, and metric provenance. Failure indicates weak benchmark defensibility.
Cross-subject and within-subject evaluation	Dataset protocol, split protocol, leakage-control, and benchmark anchors.	Every split and result identifies subject/session boundaries and seeds. Failure may invalidate results due to leakage.
Calibration and uncertainty evaluation	ECE, Brier, NLL, reliability diagram, entropy, margin, high-confidence wrong, and selective-prediction anchors.	Calibration metrics and reliability diagrams have definitions and split provenance. Failure weakens uncertainty claims.
Risk-coverage and threshold evaluation	Selective-prediction, reject option, threshold registry, and risk-coverage anchors.	Thresholds are selected on valid splits and evaluated on held-out records. Failure indicates overfitted threshold claims.
IHARQ comparison	IHARQ evidence-verification anchors, threshold baselines, ensemble comparisons, reason-code completeness.	IHARQ is compared fairly to threshold and baseline alternatives. Failure indicates architectural overcomplexity without evidence.
RegimeRisk temporal trust evaluation	Drift, non-stationarity, rolling reliability, temporal-ordering anchors.	Regime timelines use only past or current information and include non-diagnostic warnings. Failure indicates temporal leakage or overclaiming.
Adaptive-IHARQ and pseudo-label evaluation	Evidence-quality, cost-model, pseudo-label, deferral, limited expert prediction anchors.	Pseudo-label construction is declared and separated from real readiness labels. Failure indicates invalid label interpretation.

Learning-to-defer comparisons	Mozannar/Sontag and limited-expert-prediction anchors, deferral records, cost assumptions.	Deferral baselines identify oracle, fallback, or proxy expert source. Failure indicates unsupported expert-routing claims.
Contextual bandit and RL simulation evaluation	Contextual-bandit, RL, reward-design, simulation-environment, and transition-record anchors.	Results are labeled simulation-only, with reward sensitivity and seed sensitivity. Failure indicates overclaiming of deployed policy behavior.
StressForge-Lite sensitivity	Stress, perturbation, delay, dropout, drift, fatigue-like degradation, and matched-pair anchors.	Stress results include stress profile, intensity, seed, and matched control. Failure indicates non-reproducible robustness claims.
MyoSuite/OpenSim embodiment metrics	Simulator documentation, proxy mapping, gate validation, MuJoCo dependency, OpenSim replay/documentation anchors.	Embodiment results include simulator version, task, proxy mapping, gate decision, and warning that results are simulation-only. Failure indicates simulator overclaim.
Dashboard quality assurance	Dashboard framework, card, schema, export manifest, read-only inspection anchors.	Cards expose source records, stale status, version metadata, and warnings. Failure weakens traceability and reproducibility claims.
A0–A13 ablation ladder	Method anchors for each ablated family and claim-evidence matrix anchors.	Each ablation row has method, metric, record, and claim boundary traceability. Failure makes ablation claims hard to defend.
Negative-result handling	Failure-mode, limitation-reporting, and defensive-design anchors.	Failed branches are labeled diagnostic-only, optional-only, non-thesis-ready, invalid due to leakage, invalid due to claim violation, stretch-only, or simulation-only. Failure encourages hidden negative results.

28.14 Versioning, reproducibility, and implementation requirements

Before final thesis submission, every implemented tool anchor must be synchronized with the repository and reproducibility package. The required fields are: tool name, exact version, installation source, documentation URL or official citation, license or access note, environment dependency, related repository folder, related configuration file, related run-manifest entry, related record family, related tests, known limitations, and access date where required.

These requirements apply to all implemented branches, including `configs/` for dataset, decoder, calibration, IHARQ, RegimeRisk, policy, stress, simulation, embodiment, and dashboard settings; `src/` modules for data protocol, decoders, calibration, IHARQ, RegimeRisk, adaptive

policy, simulation, StressForge-Lite, embodiment adapters, dashboard, cards, and evaluation; **records/** folders for canonical record outputs; **reports/** and dashboard exports; **tests/** for schema validation, leakage checks, reason-code completeness, temporal causality, policy-context leakage, stress pairing, embodiment gate validation, dashboard read-only validation, and reproducibility manifest validation; and **docs/** or thesis appendices for citation and tool-version tables.

Required reproducibility artifacts include environment files, seed declarations, configuration hashes, run manifests, export manifests, record schema versions, tool-version manifests, dataset access notes, dashboard export manifests, and final citation cleanup notes. If a tool cannot be versioned or its license/access terms are unclear, the corresponding branch must be labeled non-thesis-ready until resolved.

28.15 Dashboard and card linkage

The Selected Technical References and Tool Anchors section must appear in dashboard and reporting artifacts as metadata, source references, warnings, and reproducibility fields. The dashboard remains read-only over saved records and must not infer new evidence from references.

Card or report	Reference and tool-anchor content	Warning fields
DatasetCard	Dataset documentation, MOABB anchor, access notes, split protocol, preprocessing references.	Public-data-only; labels are task labels, not true clinical readiness.
ModelCard	Decoder anchors such as CSP/LDA/SVM, Riemannian baselines, EEGNet, ShallowConvNet, implementation framework, seed, and training config.	Decoder performance does not imply readiness safety.
ReliabilityCard	Calibration, uncertainty, selective prediction, risk-coverage, threshold registry, and high-confidence wrong audit anchors.	Reliability metrics are benchmark evidence, not patient-safety validation.
IHARQ reliability card	IHARQ evidence-verification anchors, canonical action-token/display-label mapping, reason-code tables, request/combine traces, fallback and unsafe-event records, trace completeness, and matched-ablation lineage.	IHARQ card evidence is proxy evidence-verification and readiness-control evidence, not a patient-safety certificate or literal communication-system validation.
RegimeCard	Drift, non-stationarity, rolling reliability, temporal trust, and RegimeRisk anchors.	RegimeRisk is not a medical fatigue or health-state diagnosis.

ControllerCard	Adaptive-IHARQ, learning-to-defer, limited expert prediction, contextual-bandit, RL, reward-design, and pseudo-label anchors.	Pseudo-labels are computational readiness labels; bandit/RL is simulation-only unless otherwise proven.
StressCard	StressForge-Lite stress definitions, perturbation anchors, delay/dropout/drift/fatigue-like degradation configs.	Synthetic stress profiles are not physiological diagnosis.
EmbodimentCard	MyoSuite, OpenSim, MuJoCo, simulator adapter, proxy mapping, gate validation, safety-event log, Layer9RewardTrace where enabled, command timeline, endpoint-error curve, Layer9EmbodimentDemoManifest , task documentation anchors, and Layer 10 read-only export anchors.	Embodiment is optional, simulated, proxy-only, Layer 9-owned, Layer 10-observable, and not rehabilitation validation.
ReproducibilityCard or export manifest	Tool versions, installation sources, licenses/access notes, configs, seeds, hashes, schema versions, run manifests, and citation cleanup status.	Missing or stale metadata invalidates reproducibility claims for affected branches.

28.16 Failure-mode linkage

Reference failures are architecture failures when they make implementation, evaluation, or claims non-traceable. The following failure modes must be tracked in the failure-mode register and dashboard warnings.

Anchor failure	Architectural consequence	Fallback label or action
Missing dataset or benchmark anchor	Public-data reproducibility is weakened and dataset/protocol claims become hard to defend.	Add anchor, mark dataset protocol incomplete, or block benchmark claim.
Missing split or preprocessing anchor	Leakage and preprocessing ambiguity increase.	Invalid due to leakage until split and preprocessing provenance are documented.
Missing decoder anchor	Baseline comparison lacks methodological support.	Mark baseline documentation incomplete or remove unsupported baseline.

Missing calibration or uncertainty anchor	Reliability and low-calibration claims are weakened.	Diagnostic-only until metrics are defined and records are generated.
Missing selective-prediction or threshold anchor	Risk-coverage and threshold comparisons are under-justified.	Non-thesis-ready for selective-prediction claims.
Missing IHARQ anchor or reason-code traceability	Evidence-verification claims become opaque.	Add method card and reason-code validation or demote IHARQ claims.
Missing RegimeRisk temporal anchor	Temporal trust claims become unsupported or overinterpreted.	Diagnostic-only; add non-diagnostic warning.
Missing learning-to-defer anchor	Deferral comparisons lack method grounding.	Remove deferral claim or label limited/proxy deferral.
Missing contextual-bandit or RL anchor	Policy-learning framing becomes weak.	Stretch-only or simulation-only; remove deployed-policy wording.
Missing StressForge-Lite stress anchor	Robustness/stress claims become non-reproducible.	Add stress profile manifest or mark stress branch non-thesis-ready.
Missing MyoSuite/OpenSim/MuJoC documentation	Embodiment cannot be reproduced or bounded.	Optional-only, simulation-only, or remove embodiment claim.
Missing dashboard or reproducibility anchor	Inspection and rerun claims weaken.	Add export manifest and version card or mark dashboard diagnostic-only.
Citing clinical references without matching evidence	Claim violation risk.	Remove, demote to future work, or label as outside scope.
Treating simulator references as validation	Misleading embodiment or rehabilitation claim.	Add proxy-only and simulation-only warnings; block clinical wording.
Unused tool remains cited as implemented	Decorative or misleading citation.	Remove or mark candidate/optional.

28.17 Claim-safety and boundary discipline

References do not automatically authorize stronger claims than the project evidence supports. Public EEG references support benchmark and reliability evaluation claims, not clinical deployment claims. MOABB supports BCI benchmarking discipline, not proof of real assistive-control safety. EEGNet supports compact EEG decoding, not readiness safety by itself. Calibration and uncertainty references support reliability measurement, not patient-safety certification. IHARQ anchors support evidence-verification design and reason-coded decision logic, not a medical-device safety case. RegimeRisk anchors support temporal trust characterization, not diagnosis of fatigue or health state. Learning-to-defer references support deferral-method design, not clin-

ical expert routing unless real expert labels are available. Contextual-bandit and RL references support simulated policy-learning experiments, not deployed patient-facing control. MyoSuite, OpenSim, and MuJoCo references support simulated embodiment and proxy command demonstration, not clinical rehabilitation validation. Dashboard and card references support interpretability, traceability, and reporting, not new experimental evidence. StressForge-Lite anchors support declared synthetic stress testing, not real physiological stress or fatigue diagnosis.

The non-negotiable boundaries remain unchanged: no clinical validation claim; no patient-safety claim; no therapeutic efficacy claim; no real FES safety claim; no implanted-control claim; no medical-device deployment claim; no real-world rehabilitation claim; no claim that simulated embodiment proves real biomechanics or patient outcomes; and no claim that public EEG labels are true readiness labels.

28.18 Final thesis citation cleanup requirements

Before final thesis submission, the selected anchors in this architecture document must be upgraded into exact final citations and implementation-specific tool records. The writer or implementer must:

1. Replace placeholder references with exact bibliographic metadata.
2. Confirm paper titles, author names, venues, years, DOIs, arXiv IDs, and official documentation URLs.
3. Verify tool versions used in the actual implementation.
4. Record installation sources, environment dependencies, access dates where required, and license or access notes.
5. Remove tools not actually used or mark them optional, candidate, future work, documentation-only, stretch-only, simulation-only, or proxy-only.
6. Ensure that every cited tool or paper has an architectural reason to appear.
7. Ensure that every major architectural claim has at least one method, benchmark, tool, evaluation, record, or dashboard anchor.
8. Ensure that citations are not used as decorations.
9. Ensure that references do not imply unsupported clinical, deployment, therapeutic, patient-safety, rehabilitation, FES, implanted-control, or medical-device claims.
10. Ensure that final BibTeX or bibliography formatting is consistent with thesis requirements.
11. Ensure that web documentation references include access dates if required by the institution.
12. Synchronize all citation and version tables with repository manifests, exported records, dashboard cards, tests, and final thesis figures.

28.19 Selected anchor bibliography for architecture-stage use

The following bibliography is intentionally selected and architecture-stage. It preserves the existing anchors and adds placeholder families for final citation cleanup. The final thesis should expand, verify, replace, or remove entries according to the implementation actually completed.

References

- [1] MyoSuite documentation. *MyoSuite: musculoskeletal environments and tasks simulated with MuJoCo and exposed through an OpenAI gym-style API*. Available at: <https://myosuite.readthedocs.io/>
- [2] OpenSim Project. *Open-source musculoskeletal modeling and dynamic simulation software*. Available at: <https://simtk.org/projects/opensim> and <https://opensim.stanford.edu/>
- [3] H. Mozannar and D. Sontag. *Consistent Estimators for Learning to Defer to an Expert*. arXiv:2006.01862, 2020.
- [4] P. Hemmer, L. Thede, M. Voessing, J. Jakubik, and N. Kuehl. *Learning to Defer with Limited Expert Predictions*. AAAI, 2023.
- [5] D. Rao et al. *Contextual Bandits for adapting to changing environments*. arXiv:2009.10073, 2020.
- [6] A. Jayaram and A. Barachant. *MOABB: trustworthy algorithm benchmarking for BCIs*. Journal of Neural Engineering, 2018.
- [7] V. J. Lawhern et al. *EEGNet: a compact convolutional neural network for EEG-based brain-computer interfaces*. Journal of Neural Engineering, 2018.
- [8] MNE-Python documentation and/or primary software citation. Architecture-stage candidate anchor for EEG preprocessing, data handling, channel metadata, filtering, epoching, and reproducible signal-processing pipelines. Include only if used in implementation; otherwise mark optional or remove.
- [9] pyRiemann documentation and/or primary software citation. Architecture-stage candidate anchor for covariance-based Riemannian features and tangent-space baselines. Include only if implemented.
- [10] scikit-learn documentation and/or primary software citation. Architecture-stage candidate anchor for LDA, SVM, logistic regression, calibration helpers, and classical baseline utilities. Include only for actual implemented components.
- [11] PyTorch, Braindecode, or equivalent deep-learning framework documentation and citations. Architecture-stage candidate anchors for EEGNet, ShallowConvNet, compact CNN baselines, and training manifests. Include only if used.
- [12] Calibration, uncertainty, selective-prediction, and risk-coverage reference family. Final thesis must cite exact sources for ECE, Brier score, NLL, reliability diagrams, selective prediction, reject option, risk-coverage curves, entropy/margin uncertainty, and high-confidence wrong audits used in the evaluation.
- [13] Drift, non-stationarity, temporal reliability, and monitoring reference family. Final thesis must cite exact sources used to justify RegimeRisk temporal trust monitoring and rolling reliability analysis.
- [14] Contextual-bandit and reinforcement-learning simulation reference family. Final thesis must cite exact sources used for policy evaluation, reward design, off-policy cautions, seed sensitivity, and simulation-only boundaries.

-
- [15] Dashboard, model-card, dataset-card, reporting, schema-validation, and reproducibility reference family. Final thesis must cite exact sources or documentation used for dashboard frameworks, card structure, manifests, schema validation, environment capture, and reproducibility packaging.

29 Layer 1 M8 synchronization addendum (T12)

Status. This addendum synchronizes the architecture with the completed Layer 1 Detailed Design / Nuts-and-Bolts Specification M8 and the applied Registry R18 profile update. It is specialization-only: the Layers 0–10 graph, official module/submodule names, phase ownership, A0–A13 identities, and downstream responsibilities are unchanged.

29.1 Architecture realization lock

Layer 1 remains the Public-Data and Protocol Anchor. Its eleven official implementation-ready design units remain exactly:

1. Dataset registry manager;
2. Dataset loader;
3. Metadata normalizer;
4. Task and label ontology mapper;
5. Preprocessing registry;
6. Artifact and channel-quality annotator;
7. Split and protocol manager;
8. Trial/window generator;
9. Data validation and leakage auditor;
10. Dataset-card and protocol-card generator;
11. Provenance and manifest builder.

No architecture module is added, removed, renamed, or reassigned. Lower-level adapters, validators, transformation functions, candidate assemblies, and serialization helpers remain internal implementation responsibilities rather than new architecture nodes.

29.2 Canonical output and ownership clarification

The Layer 1 canonical family remains eleven objects. A module that supplies source fields, validates an object, renders it, or packages it does not thereby become the canonical producer. The controlling producer/contributor interpretation is:

Object	Canonical producer/materializer	Authorized contributors / boundary
DatasetRecord	Dataset registry manager / Metadata normalizer	Dataset loader supplies source/access content; Category 2 supplies referenced preprocessing_id/split_id through binding
WindowRecord	Trial/window generator	Metadata normalizer; label mapper; preprocessing registry; split manager; quality annotator

Object	Canonical producer/materializer	pro-ducer	Authorized contributors / boundary
SplitRecord	Split and protocol manager		Metadata/labels; Validation auditor verifies legality
PreprocessingRecord	Preprocessing registry		Source metadata and Protocol configuration; Validation auditor verifies fit scope
LabelMapRecord	Task and label ontology mapper		Dataset loader/normalizer supply original context and evidence
ArtifactFlagRecord	Artifact and channel-quality annotator		Source quality evidence; validation checks provenance/completeness
ValidationReport	Data validation and leakage auditor		All Category 1/2 records and supporting reports are inputs; Category 3 package checks remain supporting
DatasetCard	Dataset-card and protocol-card generator		Validated records, source evidence, ValidationReport
ProtocolCard	Dataset-card and protocol-card generator		Validated protocol records and Validation-Report
Layer1Manifest	Provenance and manifest builder		All validated records/cards/reports/config identities
manifests/layer1_abstraction_readiness_truth_data	Provenance and manifest builder (canonical byte materializer/publisher)		Validation and leakage auditor is readiness-truth/evidence authority; Registry lists both producer modules

Standalone TrialRecord, BudgetRecord, LeakageReportRecord, NegativeControlRecord, and StagingDatasetRecord are not introduced. Trial/source-event lineage remains represented through WindowRecord, source metadata, and supporting evidence.

29.3 Source-truth and representation-view rule

Architecture-level source truth is field-family specific. The original dataset/source, source files, wrapper-generated representation, converted container, and computational view may each be authoritative for different facts. The implementation must preserve those views separately and use five evidence states: VERIFIED_TRUE, VERIFIED_FALSE, UNKNOWN, CONFLICTED, and BLOCKED. Wrapper metadata never silently overwrites verified source/file facts.

29.4 Layer 1 completion and boundary status

The Layer 1 internal technical design is DESIGN-COMPLETE and ready for Implementation Build Book intake. This does not mean that live source loading, final Protocol v1.0 numbers, A0–A13 execution, canonical freeze, clinical validation, rehabilitation efficacy, medical-device readiness, real stimulation control, or deployment readiness has occurred. Layer 0 remains claim authority and Layer 10 remains a read-only evidence renderer.

29.5 Synchronization references

The controlling Layer 1 design source is `IHARQ_BenchGuard_Stretch_C_Layer1_Detailed_Design_Nuts_and_`. The controlling Registry source is the R18 Layer 1 M8 controlled-profile application. All corrections remain versioned and dependency-aware; validated predecessor evidence is never mutated in place.

30 Layer 1 bidirectional synchronization and change-propagation lock (T13)

Status. T13 preserves T12 and adds the reverse synchronization contract needed after the Layer 1 M8/M9 completion. Bidirectional means that accepted architecture requirements are traceable into the Layer 1 design and that implementation-proven architectural implications are routed back to the architecture owner. It does not permit Layer 1 to alter topology or ownership locally.

30.1 Architecture-to-Layer-1 binding

The eleven official design units, the Public-Data and Protocol Anchor mission, Layer 0 claim boundary, Layer 10 read-only boundary, Layers 2–10 consumer graph, and Phase 0–15 responsibility boundaries remain binding inputs to M10. Every Layer 1 internal component remains subordinate to one of the eleven design units or a non-canonical cross-cutting responsibility.

30.2 Layer-1-to-architecture feedback

M10 reports implementation feasibility, source/representation conflicts, canonical output realization, and downstream handoff needs. These findings may clarify architecture text but do not create architecture nodes. A proposed new official design unit, moved responsibility, changed Layer 1 scope, or new cross-layer feedback path requires an architecture successor before Registry, Plan, Playbook, Method Selection, Framework, or Layer 1 design updates can treat it as authoritative.

30.3 Dependency invalidation

An accepted architecture change triggers an impact review over canonical producers, all 52 Layer 1 output families, Phase 1 evidence, A0–A13 readiness, Layer 0/10 boundaries, Build Book interfaces, and every downstream layer consuming the affected object. Frozen evidence is never mutated; affected artifacts are regenerated under new version identities or explicitly invalidated.

30.4 Current closure

No topology, module, phase, ablation, or responsibility change is required by M8–M10. The architecture is bidirectionally synchronized at design level. Runtime verification and final canonical freeze remain external gates.

30.5 Controlling references

The current detailed design status is M10. The current Registry synchronization is R19. The exact document-pair, module, canonical-object, output-family, requirement, phase, ablation, and change-propagation matrices are packaged with the R19 synchronization release.

31 Layer 1 bidirectional integrity and compatibility lock (T14)

Compatibility set. L1-BISYNC-R20-20260714. T14 preserves T13 and adds a release-integrity lock. Bidirectional synchronization remains asymmetric in authority: Layer 1 may report implementation evidence and propose changes, but only the owning document may accept a normative change.

31.1 Current compatible revisions

The authoritative companion revisions are Registry R20, Execution Plan R20, Experiment Protocol R20, Playbook R20, Method Selection V10, Layer 1 framework R3, and Layer 1 M11. The two generic guides remain unchanged. Exact hashes are carried in the external R20 stack lock so this document does not contain circular self-hashes.

31.2 Topology and ownership protection

No new layer, official design unit, canonical object, phase, ablation, metric authority, or claim authority is introduced. Any proposed topology or responsibility change must be accepted in an architecture successor before downstream documents may treat it as normative.

31.3 Reverse change propagation

An accepted architecture change triggers an impact review over Registry profiles, all eleven Layer 1 design units, all eleven canonical objects, all 52 output families, Phase 0–15, A0–A13 readiness, Layer 0/10 boundaries, Build Book interfaces and downstream evidence. Frozen predecessor artifacts remain immutable.

31.4 Current closure

The R20 orphan/collision/drift audit reports no missing authority requirement, unauthorised output, duplicate canonical materializer, forbidden canonical-object promotion or stale current compatibility reference. Runtime and freeze gates remain external.

32 Layer 1 green-light revalidation and source-assertion firewall (T15)

Compatibility set. L1-GREENLIGHT-R21-20260715. T15 preserves T14 content and records the final green-light revalidation. No topology, module, canonical object, phase, ablation, metric authority, or claim authority changes.

32.1 Newly explicit source contradiction

The MOABB v1.5 metadata for BNCI2014_001 is internally inconsistent with its own dataset description: participant count, class count/labels, filter history and EOG presence conflict. Layer 1 therefore treats wrapper-generated metadata as a field-scoped assertion, marks contradictions CONFLICTED, and uses original source/file evidence and context-sensitive mappings without silently guessing.

32.2 Green-light scope

The Layer 1 internal technical design is complete for Implementation Build Book intake and downstream design/planning. Runtime source support, final Protocol numerics, canonical freeze, A0–A13 results, and final claims remain external gates.

32.3 Preservation and synchronization

All eleven official design units and canonical objects remain unchanged. Any accepted future authority change creates immutable successors and triggers dependency regeneration or invalidation.

33 Layer 1 stage-appropriate green-light and lifecycle clarification (T16)

Compatibility set. L1-STAGE-GREENLIGHT-R22-20260715. T16 preserves T15 and clarifies the meaning of green light across the project lifecycle. A current-stage green light confirms that the Layer 1 method-selection, Nuts-and-Bolts design, upstream synchronization and Implementation Build Book handoff are complete. It does not require future execution artifacts to exist before their producing phases occur.

33.1 Lifecycle sequence

The controlling sequence is: Layer 1 Method Selection Register; Layer 1 Nuts-and-Bolts Specification; Implementation Build Book; Experiment/Ablation/Evaluation Protocol v1.0 finalization; phase execution and A0–A13 evidence production; Paper/Thesis Evidence Map and final claim review.

33.2 Stage-appropriate status

The Layer 1 internal technical design and synchronized document stack are complete. Implementation Build Book authoring and Phase 1 realization are green-lit to begin. Protocol v1.0 finalization is green-lit as its scheduled future document stage once executable pins and feasibility evidence are available. A0–A13 readiness contracts are complete, while empirical A0–A13 results are intentionally produced by later layers/phases. The concrete Layer 1 package freeze is performed after runtime validation; the freeze procedure itself is complete.

33.3 Authority and non-overclaim

This clarification does not create new modules, canonical objects, phases, ablations, metrics or claims. Later-stage non-execution is not classified as a current-stage design failure. A genuine block exists only when a requirement due at the current stage is missing or contradictory.

33.4 Mutable technical baseline correction

As of 15 July 2026, BIDS Validator 3.0.0 is the latest release and documents breaking public-library API changes. Earlier T15/R21 current-baseline wording that treated 2.4.1 as latest is historical. The Implementation Build Book must pin and requalify the selected validator invocation.

34 T17 Layer 2 Nuts-and-Bolts Post-Finalization Synchronization

Status. This T17 addendum incorporates only architecture-level clarifications proven by the finalized Layer 2 Detailed Design / Nuts-and-Bolts M7 specification. The preceding T16 architecture is otherwise unchanged. No layer, plane, module, submodule, connection, phase, ablation, record family, or capability tier is added, removed, renamed, split, merged, or reordered.

34.1 Exact Layer 2 module inventory remains unchanged

Layer 2 remains the *Decoder and Baseline Measurement Spine* with exactly eight official modules:

1. Baseline trainer;
2. Prediction logger;
3. Low-calibration curve builder;
4. Subject difficulty profiler;
5. Model-family registry;
6. Ensemble comparison builder;
7. Compact SSL adapter;
8. Downstream readiness validator.

Metric-source computation, matching, terminal reconciliation, failure indexing, card-source preparation, snapshotting, and handoff assembly remain cross-cutting functions under these authorized owners and are not additional architecture modules.

34.2 Clarified producer, consumer, and lifecycle boundaries

Surface	Controlling architecture clarification	Noninterference rule
PredictionRecord	The Prediction logger is the sole canonical writer. C1 and the Compact SSL adapter provide qualified models and native-output contracts; they do not write canonical predictions.	Layer 3 consumes eligible raw scores and predictions but does not repair Layer 2 records.
ModelRegistryRecord	C1 writes an immutable initial registry version before C2 predictions exist. Where a reverse prediction reference is required, a later immutable successor version is written after C2 output validation; predecessor bytes are never overwritten.	C2 may forward-reference the exact model ID/version/hash but never mutates or becomes writer of the registry record.
Compact adapter	SSL Produces a qualified immutable pretrained-model bundle, adapter/input contract, provenance, resource and failure evidence.	Any architecture shorthand suggesting direct canonical prediction production is interpreted through the Prediction logger ownership rule.

Surface	Controlling architecture clarification	Noninterference rule
SubjectProfileRecord	Descriptive subject/session heterogeneity profile only.	It is not a physiological diagnosis, readiness decision, policy-routing label, or clinical statement.
EnsembleControlRecord/A4	C1/C2 longer-window, multi-window vote/average, and fixed-ensemble controls.	Registered thresholds, rejection, calibration, and selective prediction remain Layer 3/A2–A3; A4 does not absorb them.
Failure and readiness	C1/C2 originate failures where detected. C3 reconciles terminal accounting, validates referential closure, builds the failure index, and writes the final consumer-specific readiness report.	C3 cannot manufacture missing upstream evidence or convert a failed/diagnostic branch into claim-bearing evidence.
Layer 10 and Layer 0	Layer 10 renders immutable source snapshots; Layer 0 approves claim scope.	No hidden metric recomputation, evidence repair, denominator change, or claim approval occurs in Layer 2 or Layer 10.

34.3 Phase and ablation boundary confirmation

Layer 2 directly owns A0 and A4 evidence production. It supplies preparation, lineage, compatibility, and readiness inputs for A1–A3 and A5–A13 without implementing their downstream methods. Phase 2 retains the established fourteen-step sequence. This addendum changes no Phase 0–15 ordering or owner.

34.4 Architecture handoff status

The Layer 2 M7 specification is architecture-conformant and handoff-ready for Registry synchronization, Protocol v1.0 completion, the Implementation Build Book, and live Phase 2 execution. This statement is a design-stage green light only and is not an empirical-performance or thesis-claim approval.

35 T18 Layer 2 Bidirectional Synchronization and Compatibility Lock

Compatibility set. L2-NB-M8-BIDIRECTIONAL-SYNC-R2-20260718. T18 preserves the T17 architecture and adds only the reverse synchronization and release-integrity contract. The eight Layer 2 modules, C1/C2/C3 workstreams, layer topology, phase order, A0/A4 ownership, and Layer 0/10 boundaries are unchanged.

35.1 Compatible companion revisions

The compatible design-stage set is Registry R25, Execution Plan R24, Experiment Protocol R24, Playbook R24, Integrated Method Registers R3, Layer 1 Nuts-and-Bolts M15, Layer 2 Framework V9, and Layer 2 Nuts-and-Bolts M8. Historical companion names in preserved text are audit history only.

35.2 Bidirectional but asymmetric authority

Architecture requirements flow into M8. M8 may provide implementation and feasibility evidence back to the architecture owner, but it may not add, remove, rename, split, merge, or move a module, layer, phase, ablation, connection, producer, consumer, or claim authority locally. A change becomes normative only in an architecture successor.

35.3 Reverse change propagation

An accepted architecture change triggers impact review over Registry profiles, all 49 Layer 2 decisions, C1/C2/C3 algorithms, Phase 2 Steps 1–14, A0–A13 relationships, Layer 1 intake, Layer 3 consumers, Layer 10 source views, Layer 0 limitations, Build Book interfaces, and all dependent records/artifacts. Frozen evidence is never edited; affected products are invalidated or superseded and regenerated under new identities.

35.4 No-drift rule

Documents from another compatibility set may not be combined with T18 without an explicit impact review and a new synchronized successor set. Cross-cutting metric, matching, failure, snapshot, packaging, and referential functions remain internal under the eight official modules and do not become architecture modules.

35.5 Current closure

No architecture change is required by M8. The design stack is bidirectionally synchronized and ready for Protocol v1.0, Build Book and live Phase 2 work. Empirical results and final claims remain future evidence gates.

36 T19 Layer 3 Detailed-Design Realization and Bidirectional Synchronization Lock

Compatibility set. L3-NB-R16-UPSTREAM-SYNC-R1-20260721. This addendum synchronizes the final Layer 3 R16 detailed design back into architecture-owned topology, responsibility, connection and handoff language. It does not create, remove, rename, split, merge or transfer any architecture layer, module, phase, ablation or claim authority.

36.1 Architectural conclusion

The R16 realization confirms that the existing Layer 3 architecture is implementable as one trust-conversion layer organized into three internal workstreams (C1, C2 and C3) and ten official module families. No architecture correction is required. R16 specialization is accepted as a detailed realization beneath the existing architecture, subject to Registry, Protocol, Build Book and empirical evidence gates.

36.2 Ten-module realization lock

Module	WP	Inputs	Outputs	Ablation
Probability calibrator	C1	PredictionRecord; ModelRegistryRecord; legal split/config	CalibrationRecord; calibration parameter/config lineage	A1
Reliability audit	C1	CalibrationRecord; PredictionRecord; legal evaluation labels; Metric Dictionary	ReliabilityAuditReport; reliability-bin/diagram sources; CalibrationDashboard- MetricRecord candidate	A1A3
Uncertainty extractor	C2	PredictionRecord; lawful CalibrationRecord/raw source; optional validated A4 members	UncertaintyRecord; feature summaries	A3
Threshold and selective-prediction baselines	C2	CalibrationRecord; UncertaintyRecord; legal selection/evaluation roles	SelectivePredictionRecord; RiskCoverage- CurveSource	A2/A3
Threshold registry	C2	Selective results; uncertainty/calibration records; applicability config	ThresholdRegistryRecord; Build Book serialization view	A2/A3
Split/leakage guard	C1	Layer 12 records; split/visibility/config identities	SplitIntegrityReport; LeakageWarningRecord; CalibrationEligibilityTable candidate	A1A3

Module	WP	Inputs	Outputs	Ablation
High-confidence error miner	C1	Prediction/calibration/understanding plus evaluation-only labels	HighConfidenceWrongTable, NegativeResultNote, links	A1A3 diagnostic
Dashboard/card exporter	C3	Saved C1/C2 records, reports, metrics, warnings and limitations	CalibrationDashboardMetrics, candidate; ReliabilityCard source bundle; Phase 3 handoff/freeze manifest	AdA3 reporting
Group-wise audit / calibration-budget analysis	C1 shared with C2	Calibration/reliability/understanding evidence; declared groups/budgets	GroupwiseAudit, tables; sparse-support warnings	A1A3
Conformal proxy / temporal summaries	C2	Legal conformal calibration evidence; ordered past-only histories	Optional UncertaintyRecord fields; conformal/temporal diagnostic summaries	A3 conditional

36.3 Architecture-owned handoff and noninterference clarifications

- Layer 1 and Layer 2 records are immutable source evidence. Layer 3 may validate eligibility, join by declared identities and write derived records; it may not mutate upstream scores, class order, split roles, preprocessing lineage, labels or model identity.
- Layer 3 writes calibration, uncertainty, selective-prediction and threshold-governance evidence and supporting reports. It does not train decoders, make Layer 4 IHARQ decisions, create Layer 5 RegimeRisk states, learn Layer 6–7 policies, own Layer 8–9 stress/embodiment, recompute primary evidence in Layer 10 or approve claims in Layer 0.
- A1 is the calibration/reliability comparison; A2 is the simple registered confidence-threshold baseline; A3 is uncertainty/selective prediction. A0 and A4 remain Layer 2-owned. A5–A13 remain downstream-owned.
- The fourteen-step Phase 3 sequence is unchanged. C1 owns Steps 1–6, C2 owns Steps 7–10 and C3 owns Steps 11–14.
- Reverse feedback from the Layer 3 implementation may identify an architecture defect, but any topology or ownership change requires a future architecture successor. Layer 3 cannot self-authorize it.

36.4 Artifact-class boundary

The four canonical Layer 3 records remain `CalibrationRecord`, `UncertaintyRecord`, `SelectivePredictionRecord` and `ThresholdRegistryRecord`. Reports, warnings, matched-operating-point evidence, negative-result artifacts, source bundles and manifests retain the Registry statuses assigned in the synchronized Registry successor. Candidate objects are not promoted by architecture language.

36.5 Reverse invalidation contract

Any accepted change to architecture, Registry semantics, Layer 1–2 source contracts, Protocol estimands/matching, Playbook sequencing or Method Selection decisions invalidates only the transitive dependent closure when that closure is complete; otherwise the conservative scope is the whole Layer 3 evidence package. Old evidence remains immutable and visible as superseded, blocked or diagnostic-only.

36.6 Architecture readiness statement

The Layer 3 architecture is design-stage conformant to R16 and may advance to Protocol v1.0 completion and the Implementation Build Book. This is not empirical performance, deployment, safety or final-claim approval.

37 T20 Layer 3 Full Bidirectional Synchronization and Reciprocal Authority Lock

Synchronization master. 00_IHARQ_Layer3_Post_Finalization_Bidirectional_Synchronization_Master_R2.md. This section strengthens the prior T19 synchronization into a complete two-way contract. It changes no layer, module, phase, ablation, canonical status, accepted method or owner boundary.

37.1 Effective companion set

Surface	Controlling file	Authority domain	Contract
Architecture	01_IHARQ_Master_Architecture_T20_LAYER3_FULL_BIDIRECTIONAL_SYNC_R2.tex	layer/module topology, ownership, phase and ablation placement, cross-layer connections and prohibited authority transfer	SYNC-L3-ARCH-R2
Canonical Registry	02_IHARQ_Canonical_Artifact_Record_Interface_Registry_R30_LAYER3_FULL_BIDIRECTIONAL_SYNC.md	canonical identity, logical profiles, vocabulary, status, producer-consumer authorization, validation, compatibility, migration and lifecycle	SYNC-L3-REG-R2
Execution and Evidence Plan	03_IHARQ_Execution_and_Evidence_Plan_R29_LAYER3_FULL_BIDIRECTIONAL_SYNC.md	evidence obligations, evidence status, phase products, entry/exit and definition-of-done gates, reproducibility and handoff evidence	SYNC-L3-PLAN-R2
Experiment/Ablation/Evaluation Protocol	04_IHARQ_Experiment_Ablation_and_Evaluation_Protocol_R29_LAYER3_FULL_BIDIRECTIONAL_SYNC.md	A0-A13 identity, estimands, matching, metrics, statistical units, confirmatory/diagnostic legality, numerical constants and outcome interpretation	SYNC-L3-PROTO-R2
Phase Execution Playbook	05_IHARQ_Phase_Execution_Playbook_R29_LAYER3_FULL_BIDIRECTIONAL_SYNC.md	phase and step order, prerequisites, execution transitions, rollback, repair, reopen, terminal states and handoff procedure	SYNC-L3-PLAY-R2
Layer 1-5 Method Registers	06_IHARQ_Layer1_to_Layer5_Method_Selection_and_Design_Rationale_Registers_FINAL_INTEGRATED_R3_LAYER3_FULL_BIDIRECTIONAL_SYNC.md	accepted method identity, role, rationale, alternatives, conditionality, fallback, rejection and controlled reopen criteria	SYNC-L3-METHOD-R2
Layer 1-2 Nuts-and-Bolts	07_IHARQ_Layer1_and_Layer2_Detailed_Design_Nuts_and_Bolts_FINAL_INTEGRATED_R3_LAYER3_FULL_BIDIRECTIONAL_SYNC.md	Layer 1-2 detailed design plus immutable Layer 3 intake, readiness projection, score/probability handshake and reverse correction	SYNC-L3-L12-R2
Layer 3 Framework	08_IHARQ_Layer3_Detailed_Design_Nuts_and_Bolts_Framework_R10_FULL_BIDIRECTIONAL_SYNC.md	Layer 3 requirements, section population contract, category work-package prompts, completeness criteria and controlled-reopen use	SYNC-L3-FW-R2
Layer 3 Detailed Design	09_IHARQ_Layer3_Detailed_Design_Nuts_and_Bolts_R18_FULL_BIDIRECTIONAL_SYNC.md	Layer 3 internal technical realization, integration, failure-aware behavior and reciprocal authority handoffs	SYNC-L3-CONSOLIDATED-R2

37.2 Architecture to Layer 3 binding obligations

ID	Obligation	Layer 3 enforcement	Failure/correction
ARCH-IN-01	Preserve the ten official Layer 3 module families and C1/C2/C3 workstream partition.	R18 maps every module to exact inputs, outputs, consumers and failures.	Architecture successor required before any create/remove/merge/split/rename/transfer.
ARCH-IN-02	Preserve Layer 1-2 producer ownership and Layer 3 derived-record-only behavior.	Intake and transformation rules prohibit mutation of upstream records.	Block or route defect upstream; never patch source bytes locally.
ARCH-IN-03	Preserve A0/A4 upstream ownership, A1-A3 Layer 3 ownership and A5-A13 downstream ownership.	Ablation metadata and interfaces remain owner-scoped.	Protocol/architecture impact if ownership or identity conflicts.
ARCH-IN-04	Preserve the fourteen-step Phase 3 identity and Phase 0-15 layer boundaries.	Playbook and design retain Steps 1-14 with C1/C2/C3 ownership.	Playbook/architecture successor for step or phase identity changes.
ARCH-IN-05	Preserve Layer 10 read-only rendering and Layer 0 claim authority.	C3 exports source-linked evidence and explicit limitations only.	Block downstream use that recomputes or overclaims.
ARCH-IN-06	Preserve no hidden back-edge from downstream policies into frozen upstream evidence.	All downstream feedback is a correction request or successor trigger.	Invalidate dependent closure; do not mutate frozen evidence.

37.3 Layer 3 to Architecture reverse-impact obligations

ID	Trigger	Required impact evidence	Owner action/effect
ARCH-OUT-01	A ten-module realization is infeasible without a topology or ownership change.	Architecture impact packet with failing dependency, attempted alternatives, affected modules and no-local-workaround proof.	Architecture owner issues successor or rejects impact; Layer 3 remains blocked meanwhile.
ARCH-OUT-02	A hidden cross-layer dependency or unauthorized consumer is discovered.	Producer-consumer edge, visibility, timing, status and claim consequence.	Architecture/Registry owners classify and update only their domains.
ARCH-OUT-03	A downstream requirement cannot be represented by the existing handoff.	Exact missing capability, consumer, lifecycle and noninterference analysis.	Architecture successor only if topology/ownership changes; otherwise Registry/Plan/Build route.

ID	Trigger	Required impact evidence	Owner action/effect
ARCH-OUT-04	A source correction changes the transitive dependency graph.	Dependency closure and conservative fallback scope.	Architecture records edge semantics only if architectural; evidence is regenerated by execution owners.

37.4 Ten-module reciprocal architecture matrix

The following field-by-field matrix is deliberately vertical so that every module obligation remains readable and no long identifier is visually collapsed.

Field	Value
Module	Probability calibrator
Primary	C1
workstream	
Required inputs	PredictionRecord; ModelRegistryRecord; legal split/config
Expected	CalibrationRecord; calibration parameter/config lineage
products	
Authorized	Reliability audit; Uncertainty extractor; Layer 4; Layer 6; Layer 10;
consumers	evaluation
Ablation	A1
relationship	
Module	Reliability audit
Primary	C1
workstream	
Required inputs	CalibrationRecord; PredictionRecord; legal evaluation labels; Metric
Expected	Dictionary
products	ReliabilityAuditReport; reliability-bin/diagram sources;
Authorized	CalibrationDashboardMetricRecord candidate
consumers	C2 diagnostics; C3; Layer 10; Layer 0; evaluation
Ablation	A1A3
relationship	
Module	Uncertainty extractor
Primary	C2
workstream	
Required inputs	PredictionRecord; lawful CalibrationRecord/raw source; optional
Expected	validated A4 members
products	UncertaintyRecord; feature summaries
Authorized	A2/A3; Layer 4; Layer 6; Layer 10; optional Layer 5
consumers	
Ablation	A3
relationship	
Module	Threshold and selective-prediction baselines
Primary	C2
workstream	
Required inputs	CalibrationRecord; UncertaintyRecord; legal selection/evaluation roles
Expected	SelectivePredictionRecord; RiskCoverageCurveSource
products	

Field	Value
Authorized consumers	Threshold registry; matched operating points; C3; Layer 4/6/10
Ablation relationship	A2/A3
Module	Threshold registry
Primary	C2
workstream	
Required inputs	Selective results; uncertainty/calibration records; applicability config
Expected	ThresholdRegistryRecord; Build Book serialization view
products	
Authorized consumers	Layer 4; Layer 6; Layer 10; experiment controller
Ablation relationship	A2/A3
Module	Split/leakage guard
Primary	C1
workstream	
Required inputs	Layer 12 records; split/visibility/config identities
Expected	SplitIntegrityReport; LeakageWarningRecord;
products	CalibrationEligibilityTable candidate
Authorized consumers	All C1/C2/C3 modules; readiness and claims
Ablation relationship	A1A3
Module	High-confidence error miner
Primary	C1
workstream	
Required inputs	Prediction/calibration/uncertainty plus evaluation-only labels
Expected	HighConfidenceWrongTable; NegativeResultNote links
products	
Authorized consumers	Reliability audit; C3; Layer 10; Layer 0
Ablation relationship	A1A3 diagnostic
Module	Dashboard/card exporter
Primary	C3
workstream	
Required inputs	Saved C1/C2 records, reports, metrics, warnings and limitations
Expected	CalibrationDashboardMetricRecord candidate; ReliabilityCard source
products	bundle; Phase 3 handoff/freeze manifest
Authorized consumers	Layer 10; Layer 0; downstream readiness consumers
Ablation relationship	A1A3 reporting
Module	Group-wise audit / calibration-budget analysis
Primary	C1 shared with C2
workstream	
Required inputs	Calibration/reliability/uncertainty/selective evidence; declared groups/budgets
Expected	Group/budget audit tables; sparse-support warnings
products	
Authorized consumers	C2; C3; Layer 10; Layer 0

Field	Value
Ablation relationship	A1A3
Module Primary workstream	Conformal proxy / temporal summaries C2
Required inputs	Legal conformal calibration evidence; ordered past-only histories
Expected products	Optional UncertaintyRecord fields; conformal/temporal diagnostic summaries
Authorized consumers	C3; Layer 4/6/10; optional Layer 5
Ablation relationship	A3 conditional

37.5 Contradiction, successor and invalidation rule

A conflict is localized to one proposition and classified by authority domain. Architecture controls topology and ownership; it does not control canonical field spellings, estimands, evidence status, procedure, code or claims. Layer 3 emits an impact packet and remains blocked or downgraded until the lawful owner accepts a successor. Released evidence remains immutable; affected transitive dependents are marked stale or superseded and regenerated. If dependency closure is uncertain, the whole Layer 3 evidence package is rerun.

37.6 T20 acceptance statement

The architecture and Layer 3 detailed design are reciprocally synchronized for design-stage continuation. This statement is not empirical performance, implementation qualification, deployment readiness, clinical validation or Layer 0 claim approval.

38 T21 Layer 4 Nuts-and-Bolts Post-Finalization Synchronization

38.1 Purpose, authority and preservation

This T21 successor synchronizes the finalized Layer 4 Detailed Design / Nuts-and-Bolts R15 release with the Architecture after the complete three-work-package merge and final seven-document audit. It changes no layer topology, official module, support submodule, owner, phase, gate, global ablation or cross-layer responsibility. The complete T20 predecessor remains preserved by the release package and all unaffected T20 content remains controlling.

The controlling Layer 4 technical-design source is IHARQ_Layer4_Detailed_Design_Nuts_and_Bolts_FULLY_M SHA-256 947000cc014dab26e6c73b8e0ff796dd5e92598ea83421a7739ef0d10085c1fa, compatibility set L4-NB-R15-POST-FINALIZATION-UPSTREAM-SYNC-R1-20260723. This appendix records architecture-level closure only; Registry physical profiles, Protocol v1.0 numerical freeze, Build Book implementation, live consumer verification, empirical Phase 4 evidence and Layer 0 claims remain separately governed.

38.2 Layer 4 fixed architecture closure

Layer 4 remains exactly three work packages: WP-L4-NB-A, WP-L4-NB-B and WP-L4-NB-C. The thirteen official modules and their architecture-level expected products are:

ID	Official module	Work package	Expected products	Authorized consumers
L4-MOD-01	Input validator and degraded-mode handler	WP-L4-NB-A	validation status; degraded-evidence flags; integrity reasons; blocked/unavailable paths	A/B/C; manifests; Layer 10 warnings
L4-MOD-02	Evidence feature builder	WP-L4-NB-A	typed evidence components; rule-derived EvidenceQualityRecord	B/C; Layer 6 readiness context; Layer 10
L4-MOD-03	Window stability checker	WP-L4-NB-A	window stability score/timeline/reasons	B/C; Layer 10
L4-MOD-04	Channel-subset checker	WP-L4-NB-A	channel stability score; corrupted-channel flag; disagreement reason	B/C; Layer 10
L4-MOD-05	Model-family agreement checker	WP-L4-NB-A	agreement score; trusted-family set; disagreement reason	B/C; Layer 10
L4-MOD-06	Reason-code registry	WP-L4-NB-B	versioned reason-code configuration, metadata, explanations and display labels	B/C; Layer 10; Layer 0
L4-MOD-07	IHARQ rule engine	WP-L4-NB-B	IHARQDecisionRecord	C; Layers 510; Layer 0
L4-MOD-08	Evidence combiner	WP-L4-NB-B	CombinedEvidenceRecord and successor decision event where reevaluated	B rule engine; C; Layers 6/7/10
L4-MOD-09	Fallback router	WP-L4-NB-B	FallbackDecisionRecord	C; Layers 6/7/9/10; Layer 0
L4-MOD-10	Unsafe-state monitor	WP-L4-NB-B	UnsafeEventRecord; unsafe override/trigger fields and proxy boundary	C; Layers 7/9/10; Layer 0
L4-MOD-11	Decision logger	WP-L4-NB-B	IHARQTraceRecord; immutable decision logs; manifest source rows	C; Layer 10; audit
L4-MOD-12	Ablation and matched-comparison exporter	WP-L4-NB-C	IHARQAblationRecord; Layer 4 EvaluationOutcomeRecord; MatchedComparisonReport; metric references/candidates	Layer 10; Layer 0; Phase 15
L4-MOD-13	Dashboard and claim-evidence exporter	WP-L4-NB-C	dashboard-ready source rows; claim-evidence links; negative/diagnostic links; export manifests; trace warnings	Layer 10; Layer 0; archives

38.3 Principal product-family closure

The finalized Layer 4 design retains the following governed product and interface families. This list does not promote candidate physical fields or metrics beyond Registry authority.

Product/interface	Status	Producer	Cardinality	Consumers
IHARQDecisionRecord	canonical_entry	B	exactly one per decision event/reevaluation	WP-C; Layers 510; Layer 0
IHARQTraceRecord	canonical_entry/trace_record	B	exactly one per IHARQDecisionRecord	WP-C; Layer 10; audit
EvidenceQualityRecord	canonical record with Registry R31 producer_variant profile applied	A when producer_variant=RULE_DERIVED_LAYER4; Layer 6 when producer_variant=LEARNED_LAYER6	zero/one per eligible producer_variant and Layer 6 identity; variants distinct	WP-B; WP-C; Layer 6
CombinedEvidenceRecord	canonical_entry conditional	B	zero/one per decision; only combine_evidence	WP-C; Layer 10
FallbackDecisionRecord	canonical_entry conditional	B	zero/one per decision; only fallback	WP-C; downstream adapters; Layer 10
UnsafeEventRecord	canonical_entry conditional computational/proxy	B	zero/one per decision; only unsafe	WP-C; Layer 10; Layer 0
IHARQAblationRecord	canonical/evaluation artifact	C	one per eligible parent/variant/run identity	Protocol; Layer 10; thesis evidence map
Layer 4 EvaluationOutcomeRecord	evaluation_artifact	C	one per lawful evaluation outcome identity	Layer 10; Layer 0; Phase 15
MatchedComparisonReport	evaluation_artifact	C	one per registered comparison/common-support identity	Protocol; Layer 10; Layer 0
MetricDictionaryEntry	canonical/evaluation reference or candidate	external evaluation authority; C references/prepares candidates	reference per reported metric; candidate requires approval	Protocol; Registry; Layer 10; Layer 0
NegativeResultNote	evaluation/claim-safety artifact	C	one per governed negative/null/adverse/baseline-sufficient identity	Layer 10; Layer 0
DiagnosticOnlyFlag	governed diagnostic status/artifact	A/B may trigger; C persists/propagates	one per governed diagnostic reason/row/result identity	all consumers
Phase 4 manifests and handoff bundles	governed supporting artifact/manifest	C	versioned bundle per run/facet/consumer	Layers 510; Layer 0; archive

38.4 Action, reason and ablation invariants

The canonical Layer 4 stored actions remain **accept**, **wait**, **request_evidence**, **combine_evidence**, **fallback**, **uncertain** and **unsafe**. The reason-code parents remain Acceptance, Waiting, Request evidence, Combine evidence, Fallback, Uncertain, Unsafe and System integrity. Phase 4 remains Steps 1–16 with Gates 4.1–4.11. Global ablations remain A0–A13. No A14 or additional official Layer 4 category/module is introduced.

38.5 A5, A6 and A7 boundary

A5 remains the deterministic, non-learned, finite, interpretable, replayable and regime-unaware Layer 4 policy. A6 remains a separately versioned Layer 6-owned learned evidence-quality/adaptive-policy result and cannot overwrite the original Layer 4 action, reasons, trace, source generation, semantic configuration, lineage, limitations or original A5 identity. A7 remains a separately versioned strict-predecessor Layer 5-informed rerun and cannot create a same-event causal cycle or overwrite A5.

38.6 T21 acceptance statement

The Architecture and finalized Layer 4 R15 Nuts-and-Bolts design are synchronized for progression to the next document stage. This is design-stage closure, not Registry physical implementation, Protocol freeze, executable implementation qualification, empirical Phase 4 confirmation, deployment readiness, clinical validation or Layer 0 claim approval.

39 T22 Layer 4 Post-Finalization Bidirectional Synchronization

39.1 Purpose and precedence

This T22 successor completes the two-way synchronization between the finalized Layer 4 R15 Nuts-and-Bolts specification and Architecture. It preserves T21 exactly in the release package and changes no topology, official module, ownership assignment, phase, gate or ablation. Compatibility set: L4-NB-R15-POST-FINALIZATION-BIDIRECTIONAL-SYNC-R2-20260723. Finalized Layer 4 SHA-256: 947000cc014dab26e6c73b8e0ff796dd5e92598ea83421a7739ef0d10085c1fa.

39.2 Architecture to Layer 4 contract [AUTHORITY_TO_LAYER4]

Architecture fixes the three Layer 4 work packages, thirteen official modules/support submodules, principal ownership, cross-layer relationships, Phase 4 identity, A0–A13, and the Layer 0/5/6/10 authority boundaries. Layer 4 must implement these responsibilities without creating a fourth category, new module, new owner, new action, new reason parent, new phase/gate or A14.

39.3 Layer 4 to Architecture return contract [LAYER4_TO_AUTHORITY]

The finalized Layer 4 design returns module/product closure, producer-consumer feasibility, cardinality and lifecycle evidence, A5/A6/A7 boundary verification, downstream source-action preservation, and evidence-backed architecture-impact packets. A return packet cannot change topology or ownership; only the Architecture owner may accept such a successor.

39.4 Bidirectional module and product closure

For every official Layer 4 module, Architecture supplies identity, responsibility and authorized connections; Layer 4 returns the detailed inputs, outputs, algorithms/state semantics, validation, failure behavior, cardinality, lifecycle and consumers. The governing module/product matrix is `Layer4_13_Module_Bidirectional_Closure_Matrix_R2.csv` and the 110-output closure is `Layer4_Expected_Artifact_Bidirectional_Closure_Matrix_R2.csv`.

39.5 Reverse-impact and correction protocol

When detailed design or bounded execution identifies an architecture-level inconsistency, Layer 4 emits a versioned impact packet containing the affected proposition, evidence, modules, products, consumers, phases/gates/ablations, migration impact and closure criterion. Layer 4 remains blocked or bounded until Architecture disposition. Accepted successors preserve prede-

cessors and trigger dependency-based invalidation and regeneration in Registry, Plan, Protocol, Playbook, Method Selection, upstream technical contracts and Layer 4 artifacts as applicable.

39.6 Cross-layer noninterference

Layer 4 does not redesign Layer 1–3 internals, Layer 5 RegimeRisk, Layer 6 learned evidence quality/adaptive policy, Layers 7–9 internals, Layer 10 dashboards or Layer 0 claim authority. Original A5, Layer 6 A6 and strict-predecessor A7 remain separate identities; downstream outputs preserve the original Layer 4 action, reasons, trace, semantic configuration, source generation, lineage, limitations and A5 identity.

39.7 T22 acceptance statement

Architecture and Layer 4 are synchronized in both directions for design-stage progression. This statement is not Registry physical implementation, Protocol freeze, executable implementation qualification, consumer runtime verification, empirical Phase 4 confirmation, deployment readiness, clinical validation or Layer 0 claim approval.

40 Layer 4 Complete-Chain Bidirectional Synchronization

Compatibility set: L4-NB-R15-POST-FINALIZATION-BIDIRECTIONAL-SYNC-R3-COMPLETE-CHAIN-20260723.

Architecture continues to fix the three Layer 4 work packages, thirteen official modules/support submodules, ownership, cross-layer boundaries, Phase 4 identity and A0–A13. Finalized Layer 4 may return module/product feasibility evidence or an architecture-impact packet, but only the Architecture owner may change topology or ownership.

The general Nuts-and-Bolts Guide governs technical-document purpose, depth, verification and Build Book handoff. Layer 4 Framework R8 governs exact category execution, research admission, fixed inventories, dossier completeness, merge and definition of done. Neither may override superior Architecture, Registry, Plan, Protocol, Playbook or Method Selection authority.

An accepted successor preserves its predecessor and triggers explicit dependency invalidation and regeneration from the earliest changed authority input. No architecture topology, module, owner, connection, phase, gate, action, reason, ablation, method or claim boundary changes in T23.

41 T24 Layer 5 R7 Post-Finalization Bidirectional Synchronization

41.1 Identity, scope, and precedence

Synchronization date: 25 July 2026.

Compatibility set: L5-R7-POST-FINALIZATION-UPSTREAM-BIDIRECTIONAL-SYNC-R1-20260725.

Finalized Layer 5 SHA-256: f1a82a1eb2e3f81525c373dda0568da036818b8d87c20e562065a1414740da72.

This T24 successor does not change the accepted scientific method, architecture topology, ownership, official module inventory, Phase 5 identity, gates, global ablations, or claim authority. It makes the finalized Layer 5 realization and its reverse-impact contract explicit. Architecture remains superior to detailed design.

41.2 Exact Layer 5 topology and responsibility closure

The Layer 5 architecture remains exactly three workstreams with 47 accepted decisions allocated 13/17/17, eight official modules, one conditional non-numbered sequence-estimator adjunct, and one whole-layer A7/evaluation/governance responsibility that is not a ninth module. A5 remains immutable and regime-unaware; A7 remains separately versioned and strict-predecessor; A6 remains Layer 6-owned; A0–A13 remain the complete global ablation set and no A14 is created.

Responsibility	Selected realization	Owned outputs	Consumers
5.1 Temporal feature builder	Causal short/medium/long/session feature views; masks/age/staleness/effective support	Temporal feature table/helper views; feature dictionary; provenance	5.25.8; evaluation; Layer 10
5.2 Volatility-like monitor	Robust MAD/IQR/EWMA portfolio under frozen configs and simple controls	Volatility component fields/references; trend/duration/reason evidence	5.4; 5.5; 5.7; evaluation
5.3 Drawdown-like monitor	Past-only reference, bounded drawdown, duration/recovery	Drawdown/reference/recovery fields and reason lineage	5.4; 5.5; 5.7; evaluation
5.4 Regime classifier	Transparent deterministic FSM as authority; grouped monotone risk; conditional HSMM adjunct only	RegimeStateRecord; regime_timelines.csv; transition/contribution/config lineage	Layers 4,6,7,9,10; 5.55.8
5.5 Failure-risk predictor	Runtime proxy plus preregistered complete-outcome classifier or censor-aware evaluation branch	Runtime failure proxy fields; Layer 5 EvaluationOutcomeRecord; target/censoring/calibration manifest	5.4; 5.6; 5.7; Part III evaluation
5.6 Trust allocator	Bounded target-specific trust; evidence-gated asymmetry; explicit trusted set/empty-set fallback	Conditional TrustStateRecord; trust_scores.csv; trust summary/trusted set	Layers 4,6,7,9,10; 5.7
5.7 Stop-loss module	Warning-before-active except governed critical bypass; bounded budget; cooldown/recovery/re-entry	stop_loss_warnings.csv; status/recommendation/recovery fields; warning trace	Layers 4,6,7,9,10; failure taxonomy
5.8 Feedback adapter	Read-only, causally lagged, ownership-preserving handoffs; immutable A5 vs separately versioned A7	L4 feedback; L6 context; L7 simulation state; L9 gate; L10/Layer0/export/promotion bundles	Layers 4,6,7,9,10,0; repository validators
Conditional non-numbered sequence adjunct	Explicit-duration HSMM only if Build Book/Protocol/validation gates pass; otherwise local/conditional comparator	Adjunct state/duration diagnostics linked to RegimeStateRecord; no new official module	5.4 and evaluation only

Responsibility	Selected realization	Owned outputs	Consumers
Whole-layer A7/evaluation/governance responsibility	Separately versioned artifact-predecessor A7; local variants/negative controls; granular negative-result preservation	MatchedComparisonReport; negative-control report; NegativeResultNote; DiagnosticOnlyFlag; RegimeCard lineage; manifests	Layer 10; Layer 0; Build Book/Protocol/Registry owners

41.3 Canonical product-family interpretation

Architecture recognizes the existing `RegimeStateRecord`, conditional `TrustStateRecord`, governed timeline/trust/warning exports, Layer 5 `EvaluationOutcomeRecord`, canonical `MatchedComparisonReport` with A7 identity, negative-control evidence, `NegativeResultNote`, `DiagnosticOnlyFlag`, Metric Dictionary references, the existing governed `RegimeCard` source-lineage/manifest bundle, Phase 5 validation reports, reproducibility manifests, and conditional BPCRs. The phrase “A7 `MatchedComparisonReport`” is descriptive only and does not create a family. Layer 5 prepares `RegimeCard` source lineage; Layer 10 renders and exports read-only.

41.4 Complete governed boundary set

Interface	Direction, ownership, consumer, and payload contract
Layer 4 IHARQ Decision Interface	Direction: Layer 45. Producer: Layer 4. Consumer: Layer 5 Parts I/II/III. Payload: <code>IHARQDecisionRecord</code> plus trace/evidence/fallback/unsafe links.
Layer 5-to-L4 RegimeRisk Feedback Interface	Direction: Layer 54. Producer: Layer 5 / Part III 5.8. Consumer: Layer 4 separately versioned A7 rerun. Payload: <code>RegimeStateRecord</code> ; conditional <code>TrustStateRecord</code> ; stop-loss warnings.
Layer5ToLayer6PolicyContext Interface	Direction: Layer 56. Producer: Layer 5 / Part III 5.8. Consumer: Layer 6 readiness-context/policy modules. Payload: <code>RegimeStateRecord</code> ; optional <code>TrustStateRecord</code> ; stop-loss context.
Layer5ToLayer7SimulationState Interface	Direction: Layer 57. Producer: Layer 5 / Part III 5.8. Consumer: Layer 7 state assembler / transition logic. Payload: regime/risk/trust/recovery/stop-loss context.
Layer8ToLayer40r5Stress ConditioningInterface	Direction: Layer 8Layer 5 optional. Producer: Layer 8. Consumer: Layer 5 optional stress-conditioned analysis. Payload: <code>StressProfileRecord</code> ; <code>StressScheduleRecord</code> ; clean/stressed pointers.
Layer40r5ToLayer9 EmbodimentGateInterface	Direction: Layer 5/49. Producer: Layer 4/5 through Part III 5.8. Consumer: Layer 9 embodiment adapter. Payload: fallback/no-action/unsafe-compatible gate and simulated readiness context.
Layer40r5ToLayer10 DashboardReadInterface	Direction: Layer 5/410. Producer: Layer 4/5 record/export modules. Consumer: Layer 10 dashboard/card/repro modules. Payload: validated records, evaluation artifacts, metric references and lineage.
Layer40r5ToLayer0ClaimSafety Interface	Direction: Layer 5/40. Producer: Layer 4/5 evidence/export modules. Consumer: Layer 0 claim-safety governance. Payload: claim-evidence rows, limitations, <code>DiagnosticOnlyFlag</code> , <code>NegativeResultNote</code> .
Layer 4/5 Evaluation Export Interface	Direction: Layer 4/5evaluation scripts/Layer 10/claim-evidence generation. Producer: Layer 45 local evaluation exporters. Consumer: Evaluation scripts; Layer 10; claim-evidence generation. Payload: <code>EvaluationOutcomeRecord</code> ; <code>MatchedComparisonReport</code> ; <code>MetricDictionaryEntry</code> ; <code>NegativeResultNote</code> ; <code>ClaimEvidenceMatrix</code> row.
Repository Promotion Interface	Direction: producing Layer 5 modulerepository validators/canonical storage. Producer: Producing Layer 5 module plus repository validators. Consumer: Promoted record/artifact family and reproducibility manifests. Payload: validated record/artifact family with schema version, run/source IDs, config hash, validation/lifecycle/branch status and optional checksums.

41.5 Bidirectional authority contract

Architecture supplies topology, module identity, ownership, cross-layer direction, product families, phase identity, gates, ablation boundaries, and noninterference. Layer 5 returns detailed feasibility, exact product/cardinality/lifecycle evidence, interface payload and failure semantics, and owner-scoped impact packets. Layer 5 cannot change Architecture by implementation convention. An accepted architecture successor preserves predecessors and triggers explicit Registry, Plan, Protocol, Playbook, Method Selection, Nuts-and-Bolts, Build Book, and descendant-evidence invalidation as applicable.

41.6 Maturity qualification

This synchronization is design-stage closure only. Registry physical-schema approval, Protocol v1.0 numerical/statistical freeze, Build Book implementation, project-instance gate passage, consumer acceptance, empirical A7 benefit/no-harm/equivalence, deployment readiness, clinical validity, patient safety, and Layer 0 claim authorization remain separate and unclaimed.

42 T25 Layer 5 R8 Full Bidirectional Synchronization and Successor Rebinding

42.1 Identity and scope

Synchronization date: 25 July 2026. **Compatibility set:** L5-R8-POST-FINALIZATION-FULL-BIDIRECTIONAL-SYNC-R2-20260725. **Layer 5 R7 predecessor hash:** f1a82a1eb2e3f81525c373dda0568da036818b8d87c20e562065a1414740da72. T25 preserves T24 and all earlier Architecture content. It adds the missing downstream successor acknowledgement and a complete authority/realization handshake. No method, topology, official module, phase, gate, record family, global ablation or claim authority is changed.

42.2 Normative successor set

The synchronized forward set is Architecture T25, Registry R36, Plan R35, Protocol R35, Playbook R35, Method Selection R6, Layers 1–4 Nuts-and-Bolts R3 and Layer 5 Nuts-and-Bolts R8. Earlier version references remain historical provenance, not the current forward interpretation.

42.3 Authority-to-realization and realization-to-authority handshake

Document	Controls	Permitted return
Architecture T25	Layer identity, topology, modules/responsibilities, ownership, directionality and claim boundary	Detailed feasibility, products, interfaces and impact evidence; never topology mutation
Registry R36	Canonical family/profile, artifact class, interface semantics, lifecycle and promotion	Logical identity requirements and representability evidence; no physical schema invention
Plan R35	Steps, products, owners, gates and evidence obligations	Exact product realization, failure/absence and regeneration evidence; no gate self-approval

Document	Controls	Permitted return
Protocol R35	Ablations, comparands, metrics, denominators, matching, statistical/negative-result rules	Algorithmic feasibility and candidate freeze slots; no numerical post-outcome freeze
Playbook R35	Execution order, stops, rollback, re-entry, promotion and handoff procedure	Operational dependencies and failure triggers; no unlogged bypass
Method Selection R6	Accepted 47-decision portfolio and controlled reopen	Realization verification and new evidence; no preference-based reselection
Layers 1-4 N&B R3	Source-layer ownership, record semantics, immutable A5 and read-only intake	Consumer validation and defect packets; no upstream repair
Layer 5 R8	Implementation-neutral detailed design within all above constraints	Artifacts, tests, handoffs, reverse-impact evidence and Build Book intake

42.4 Exact retained Layer 5 closure

The accepted Layer 5 realization remains three workstreams, 47 decisions allocated 13/17/17, eight official modules, one conditional non-numbered sequence adjunct, one whole-layer A7/evaluation/governance responsibility, exact Steps 1–15, exact Gates 5.1–5.11, ten semantic interfaces, 47 expected outputs/artifacts, immutable regime-unaware A5, Layer 6-owned A6, separately versioned strict-predecessor A7 and A0–A13 only.

42.5 Bidirectional synchronization packet

A bidirectional synchronization packet contains at minimum: packet ID and version; initiating document/version; affected proposition or decision IDs; exact evidence and source hashes; direction (‘authority_to_realization’ or ‘realization_to_authority’); impacted record/artifact/interface/step/gate/ablation/module/consumer sets; earliest invalidation root; compatibility and migration effects; owner and decision state; required regeneration order; closure tests; predecessor/successor pointers; and residual owner-gated matters. A packet cannot mark a facet complete unless the responsible owner and required evidence exist.

42.6 Invalidation roots

Change class	Owner	Earliest root	Required reassessment	Forbidden silent action
Architecture topology/module/ownership change	Architecture owner	Architecture proposition	Registry, Plan, Protocol, Playbook, Method Selection, all N&B, Build Book, descendant evidence	local reinterpretation or ownership transfer
Canonical family/interface/lifecycle change	Registry owner	Registry profile/version	Plan/Protocol/Playbook/N&B/Build adapters/records/manifests/consumers	local alias, field repair or schema invention
Phase step/gate/evidence change	Plan owner	affected step/gate definition	Playbook, Protocol where evaluation affected, N&B products, Build workflow, evidence	skipping/reordering or declaring passage
Metric/statistical/ablation/protocol change	Protocol owner	Protocol version/run-matrix slot	Plan/Playbook evaluation paths, A7 reports, controls, negative evidence, claims	post-outcome threshold or denominator change
Execution/rollback/promotion procedure change	Playbook owner	playbook procedure version	Build orchestration, validation, promotion, consumer handoff	unlogged bypass or hidden promotion
Accepted method becomes infeasible/dominated/authority-incompatible	Method-selection owner	specific decision ID	affected module decisions, formulas, records, steps/gates, interfaces, Build and evidence	implementation-preference reselection

Change class	Owner	Earliest root	Required reassessment	Forbidden silent action
Layer 1-4 source contract correction	source-layer owner	corrected source generation/version	dependent Layer 5 features/states/trust/warnings/A7/reports/manifests and downstream handoffs	repairing source in Layer 5
Layer 5 implementation defect with authorities intact	Layer 5/Build owner	defective derived artifact/config/build version	dependent Layer 5 descendants and consumer bundles only	upstream authority edit without evidence packet

42.7 Maturity boundary

This is documentary and implementation-neutral design synchronization. It does not approve physical schemas, Protocol v1.0 constants or statistics, Build Book code, real project inputs, gate passage, consumer acceptance, empirical A7 benefit/no-harm/equivalence, deployment, clinical validity, patient safety or Layer 0 claims.

43 T26 Layer 5 R9 Full Bidirectional Semantic Closure

43.1 Identity, preservation and scope

Synchronization date: 25 July 2026.

Compatibility set: L5-R9-POST-FINALIZATION-FULL-BIDIRECTIONAL-SEMANTIC-CLOSURE-R3-20260725.

Immediate predecessor SHA-256: 372b9b79f4623bbd26a3bf5c33e63520ae54892a98127b9d4a969a51d1eab105.

Prior R2 package SHA-256: 04642dc9a4431cb43febadbd42db129bab2555156d838ac1c59dcb2e5a83132b.

T26 preserves the complete T25 predecessor before this section. This is a documentary semantic-closure successor only: no method reselection, topology or official-module change, canonical-family approval, phase/gate change, or global-ablation change is made.

43.2 Domain authority and conflict precedence

Domain	Authority	Detailed-design role	Conflict rule
topology, layer ownership, official modules, claim boundary	Architecture T26	conform and report feasibility/defects	Architecture controls until accepted successor
canonical records/artifacts/interfaces/lifecycle/promotion steps, products, gates, evidence obligations	Registry R37	realize logical profiles; do not invent physical schema	Registry controls identity/classification
ablations, comparands, metrics, statistics, negative evidence	Plan R36	implement exact products and gate evidence	Plan controls execution obligations
procedure, stop/re-entry, rollback, promotion, handoff	Protocol R36	supply lawful realization and candidate slots	Protocol controls confirmatory evaluation
accepted methods and reopen triggers	Playbook R36	expose operational prerequisites/failures	Playbook controls procedure
Layers 1-4 source semantics and A5 ownership	Method Selection R7	realize; provide material evidence only	accepted decision controls until formal reopen
	Layers 1-4 N&B R4	read-only consume; validate; report defects	source owner corrects source successor

Domain	Authority	Detailed-design role	Conflict rule
Layer 5 implementation-neutral detailed design	Layer 5 N&B R9	own Layer 5 realization only	must conform to all superior domain authorities

43.3 Two-way document handshake

Document	Inbound obligation	Outbound obligation
Architecture T26	receives Layer 5 feasibility/ownership/interface impact packets	publishes topology, official-module, directionality, maturity constraints
Registry R37	receives logical identity, representability, canonical-name and interface findings	publishes canonical classes, interface identities, lifecycle and promotion rules
Plan R36	receives realized product/failure/regeneration evidence	publishes steps, products, owners, gates and evidence obligations
Protocol R36	receives candidate method feasibility and negative-control evidence	publishes A0-A13, comparands, metric/statistical/freeze rules
Playbook R36	receives operational dependencies and failure triggers	publishes sequence, stops, rollback, re-entry, promotion and handoff procedure
Method Selection R7	receives material controlled-reopen evidence only	publishes the accepted 47-decision portfolio
Layers 1-4 N&B R4	receives source-defect packets and A7 feedback contract evidence	publishes immutable source records, chronology, A5 and read-only intake contracts
Layer 5 N&B R9	receives all seven exact authority successors	publishes detailed realization, artifacts, tests, handoffs and reverse-impact evidence

43.4 Machine-verifiable semantic assertions

Assertion	Normalized value	Role
SYNC-A001	three workstreams	owns
SYNC-A002	47 accepted Layer 5 decisions	acknowledges
SYNC-A003	Part I=13; Part II=17; Part III=17	acknowledges
SYNC-A004	8 official Layer 5 modules	owns
SYNC-A005	1 conditional non-numbered sequence adjunct	owns
SYNC-A006	1 whole-layer non-module A7/evaluation/governance responsibility	owns
SYNC-A007	15 exact Phase 5 steps	acknowledges
SYNC-A008	11 exact gates 5.1-5.11	acknowledges
SYNC-A009	10 governed semantic interfaces	acknowledges
SYNC-A010	47 exact expected outputs/artifacts	acknowledges
SYNC-A011	A5 is immutable, regime-unaware, and never overwritten	acknowledges
SYNC-A012	A6 remains Layer 6-owned	acknowledges
SYNC-A013	A7 is a separately versioned Layer 4 rerun using strict-predecessor Layer 5 context	acknowledges
SYNC-A014	A0-A13 only; no A14	acknowledges
SYNC-A016	Layer 5 prepares RegimeCard source lineage; Layer 10 renders/exports read-only	acknowledges
SYNC-A017	Layer 10 is a read-only renderer/exporter for governed Layer 5 evidence	owns
SYNC-A018	Layer 0 alone authorizes claim scope; Layer 5 supplies evidence and limitations	owns
SYNC-A019	governed correction is append-only successor plus transitive invalidation/regeneration	acknowledges
SYNC-A021	Layer 5 reads Layers 1-4 without hidden repair or mutation	acknowledges
SYNC-A023	physical Registry schemas remain owner-gated where marked	acknowledges
SYNC-A024	Protocol v1.0 numerical/statistical freeze remains pending	acknowledges
SYNC-A025	Implementation Build Book/code/package parity remains pending	acknowledges
SYNC-A026	project-instance gates, consumer acceptance, empirical A7 benefit/no-harm/equivalence, deployment, clinical, safety, and claim approval are not established	owns
SYNC-A027	no method reselection	acknowledges
SYNC-A028	no architecture topology or official-module change	owns
SYNC-A029	no new canonical record family is approved by this synchronization pass	acknowledges
SYNC-A030	every accepted cross-document correction requires a versioned two-way change packet, successor binding, invalidation root, regeneration order, and closure test	owns

43.5 Versioned change-packet rule

Every accepted cross-document correction requires a stable packet identifier and version, exact initiating document/hash, authorized target owner, proposition and governed-surface identifiers, evidence, earliest invalidation root, owner decision, successor binding, topological regeneration order, closure tests, consumer acknowledgments, and residual owner-gated facets. Same-version circular repair is forbidden; feedback creates a new immutable successor.

43.6 Maturity boundary

The synchronized set permits Implementation Build Book intake only. Physical Registry schemas, Protocol v1.0 numerical/statistical freeze, code/package parity, project-instance gate passage, consumer acceptance, empirical A7 benefit/no-harm/equivalence, deployment readiness, clinical validity, patient safety, and Layer 0 claim authorization remain unestablished unless separately evidenced by their owners.

A T27 Layer 6 R11 Post-Finalization Architecture Closure

A.1 Architecture disposition

The finalized Layer 6 Detailed Design / Nuts-and-Bolts specification realizes the existing Layer 6 architecture without changing topology. The architecture retains exactly three workstreams, six numbered submodules 6.0–6.5, the conditional non-numbered extension 6.E, and the existing cross-cutting evaluation, reporting, governance, merge and freeze responsibility. No fourth workstream, submodule 6.6, A14, new action token, new phase, new layer or ownership transfer is authorized.

A.2 Exact canonical Layer 6 taxonomy

Surface	Exact canonical name	Primary owner
6.0	Rule-based IHARQ-lite reference policy	WP-L6-NB-A
6.1	Learned evidence-quality estimator	WP-L6-NB-B
6.2	Supervised Adaptive-IHARQ policy	WP-L6-NB-B
6.3	Learning-to-defer policy	WP-L6-NB-B
6.4	Contextual-bandit readiness policy	WP-L6-NB-C
6.5	RL readiness policy	WP-L6-NB-C
6.E	Stress-conditioned and embodiment-conditioned policy context/evaluation extension; conditional and non-numbered	WP-L6-NB-C with Layers 8/9 ownership preserved

A.3 Workstream and decision allocation

- Part I / WP-L6-NB-A: governed policy foundations, common substrate and 6.0; 20 decisions.
- Part II / WP-L6-NB-B: 6.1, 6.2, 6.3 and A6/A9/A8; 25 decisions.
- Part III / WP-L6-NB-C: 6.4, 6.5, 6.E, interfaces, evaluation, reporting, synchronization, merge and freeze; 31 decisions.

A.4 Principal architecture products

Surface	Governed product families
Shared substrate	ReadinessContextRecord, context validation/diagnostic evidence, semantic profile identities, action profiles, legality, budgets, fallback, cost/reward and reason configurations.
6.0	PolicyActionRecord rule-reference producer variant and ControllerCard source bundle.
6.1 / A6	Target/feature/model/calibration/threshold artifacts, learned EvidenceQualityRecord producer variant, explanation/generalization evidence and matched reports.
6.2 / A9	Per-action cost targets, pseudo-labels, supervised model artifacts, PolicyActionRecord supervised producer variant, collapse diagnostics and matched reports.

6.3 / A8	Direct/defer target, route qualification, router artifacts, DeferralRecord, causal/support/partial-identification evidence and burden/capacity reports.
6.4 / A10	Actual-propensity log, support/ESS, OPE, pessimistic authorization, PolicyUpdateTrace or no-update trace and matched evidence.
6.5 / A11	RL necessity, trajectory/state/action/reward/constraint profiles, policy/checkpoint artifacts, simulator/sensitivity evidence, PolicyUpdateTrace and frozen rollouts.
6.E	Stress/embodiment context profiles, timing/ownership links and conditional A12/A13 evidence.
Whole Layer 6	EvaluationOutcomeRecord profile, MatchedComparisonReport, negative/diagnostic evidence, manifests, Layer 10 source bundles, Layer 0 limitation handoff, semantic delta, acknowledgment, atomic publish and rollback evidence.

A.5 Cross-layer ownership invariants

Layer 6 consumes Layers 1–5 records read-only; preserves Layer 4 source actions, reasons, traces and immutable A5; consumes Layer 5 RegimeRisk/TrustState/stop-loss without reconstructing them; selects only registered legal policy actions; does not own Layer 7 transitions or realized rewards; does not own Layer 8 stress generation or Layer 9 embodiment commands; supplies immutable evidence to read-only Layer 10; and cannot authorize Layer 0 claims.

A.6 Maturity boundary

Architecture and detailed design are complete for Implementation Build Book intake. Registry physical approval, Protocol v1.0 numerical freeze, Build Book implementation, project-instance data, consumer readiness, route outcomes, support, propensities, trajectories, simulator validity, empirical Phase 6/7/10/11 results and Layer 0 claim authorization remain separate gates.

A T28 Layer 6 R11 Complete Bidirectional Authority Synchronization

A.1 Purpose and noninterference

This addendum strengthens the post-finalization synchronization at the architecture boundary. It confirms exact taxonomy, product topology, interface directionality, reverse-correction routing, maturity and consumer acknowledgment. It does not create a new layer, workstream, numbered submodule, phase, gate, ablation, action token, canonical record, method selection or claim authority.

A.2 Exact Layer 6 surfaces and ownership

Surface	Canonical architecture name	Primary steward
6.0	Rule-based IHARQ-lite reference policy	WP-L6-NB-A
6.1	Learned evidence-quality estimator	WP-L6-NB-B
6.2	Supervised Adaptive-IHARQ policy	WP-L6-NB-B
6.3	Learning-to-defer policy	WP-L6-NB-B
6.4	Contextual-bandit readiness policy	WP-L6-NB-C
6.5	RL readiness policy	WP-L6-NB-C
6.E	Stress-conditioned and embodiment-conditioned policy context/evaluation extension; conditional and non-numbered	WP-L6-NB-C; Layers 8/9 ownership preserved

A.3 Workstream and decision checksum

The architecture retains exactly three workstreams: WP-L6-NB-A with 20 decisions, WP-L6-NB-B with 25 decisions and WP-L6-NB-C with 31 decisions. Shared evaluation, reporting, Registry synchronization, merge, rollback, freeze and handoff are cross-cutting responsibilities, not a fourth workstream or seventh numbered submodule.

A.4 Bidirectional interface closure

- Architecture to Layer 6: topology, ownership, six numbered submodules, conditional 6.E and high-level producer/consumer directions remain controlling.
- Layer 6 to Architecture: finalized feasibility, product closure and interface evidence confirm realization but do not self-amend topology.
- Layers 1–5 to Layer 6: source records are immutable, versioned and consumed read-only.
- Layer 6 to upstream owners: only versioned defect/impact packets, invalidation roots and consumer requirements are returned; no silent source repair.
- Layer 6 to Layers 7–9: policy requests and identities are emitted; environment transitions, realized reward, stress generation and embodiment commands remain downstream-owned.
- Layer 6 to Layer 10 and Layer 0: immutable evidence/limitations are supplied; rendering cannot recompute and Layer 0 alone authorizes claims.

A.5 Product topology closure

The 64 finalized product rows are allocated across the shared WP-A substrate, submodules 6.0–6.5, conditional 6.E and whole-layer evaluation/governance. Registry controls canonical/candidate/helper status; the Plan controls evidence and maturity; the Protocol controls estimands, matching, metrics and numerical freeze; the Playbook controls production, rollback and handoff. Architecture acknowledges these families without promoting a new canonical object.

A.6 Maturity boundary

The Layer 6 Nuts-and-Bolts design and this authority synchronization permit Implementation Build Book intake. Registry physical approval, Protocol v1.0 and MetricDictionary freeze, runtime project evidence, consumer readiness, support/propensity/route/trajectory/simulator evidence, empirical phase execution and Layer 0 claim authorization remain separate gates.

A T29 Layer 6 R12 Closed-Loop Bidirectional Authority Synchronization

A.1 Purpose

T28 synchronized the architecture to finalized Layer 6 R11. T29 closes the round trip by recognizing the Layer 6 R12 synchronization receipt and the exact seven-authority successor set. It adds no topology, workstream, numbered submodule, phase, gate, ablation, action token, canonical record, method or claim authority.

A.2 Closed-loop successor set

Authority	Closed-loop successor
Architecture	T29, this addendum
Canonical Registry	R40 Layer 6 R12 closed-loop synchronization
Execution and Evidence Plan	R39 Layer 6 R12 closed-loop synchronization
Experiment/Ablation/Evaluation Protocol	R39 Layer 6 R12 closed-loop synchronization
Phase Execution Playbook	R39 Layer 6 R12 closed-loop synchronization
Integrated Layers 1–6 Method Selection	R10 realization synchronization; no reselection
Integrated Layers 1–5 Nuts-and-Bolts	R4 read-only consumer and reverse-correction synchronization
Layer 6 Nuts-and-Bolts	R12 upstream-acknowledged exact-R11 successor

A.3 Round-trip authority rule

Architecture controls topology and ownership. The six canonical Layer 6 submodules remain 6.0 Rule-based IHARQ-lite reference policy, 6.1 Learned evidence-quality estimator, 6.2 Supervised Adaptive-IHARQ policy, 6.3 Learning-to-defer policy, 6.4 Contextual-bandit readiness policy and 6.5 RL readiness policy. 6.E remains conditional and non-numbered. Layer 6 R12 acknowledges these identities and cannot self-amend them.

A.4 Forward and reverse correction cycle

An authority successor flows forward through Registry/Plan/Protocol/Playbook/Method Selection/technical-contract dependencies into a new Layer 6 generation. A Layer 6 defect or impact finding flows backward only as a versioned packet. The owner accepts, rejects or defers it. Acceptance identifies the earliest invalidation root, publishes an immutable successor, regenerates dependent products topologically, records semantic deltas and obtains consumer acknowledgment. Rejection leaves the current contract unchanged.

A.5 Completeness and maturity

The synchronized set covers all 76 decisions, all 64 finalized Layer 6 products, Phase 0–15 and A0–A13. It permits Implementation Build Book intake. Registry physical approval, Protocol

v1.0/MetricDictionary freeze, runtime evidence, project-instance verification, consumer readiness, support/propensity/route/trajectory/simulator evidence, empirical execution and Layer 0 claim authorization remain separately gated.